Tcl/Tk 8.6 C API

Reference manual pages

2026-10-25

Contents

Tcl C API **7**
Access 7
AddErrInfo 8
Alloc 12
AllowExc 14
AppInit 14
AssocData 15
Async 17
BackgdErr 19
Backslash 20
BoolObj 21
ByteArrObj 22
CallDel 23
Cancel 24
ChnlStack 25
Class 27
CmdCmplt 31
Concat 32
CrtAlias 32
CrtChannel 36
CrtChnlHdlr 50
CrtCloseHdlr 52
CrtCommand 52
CrtFileHdlr 54
CrtInterp 56
CrtMathFnc 58
CrtObjCmd 60
CrtTimerHdlr 65
CrtTrace 66
DString 68
DetachPids 71
DictObj 72
DoOneEvent 75
DoWhenIdle 77
DoubleObj 78
DumpActiveMemory 79
Encoding 80
Ensemble 89
Environment 93
Eval 93
Exit 96
ExprLong 98
ExprLongObj 99
FileSystem 101

FindExec 126
GetCwd 127
GetHostName 127
GetIndex 128
GetInt 129
GetOpnFl 131
GetStdChan 131
GetTime 133
GetVersion 134
Hash 135
Init 140
InitStubs 140
IntObj 142
Interp 144
Limit 146
LinkVar 149
ListObj 152
Load 155
Method 156
NRE 161
Namespace 165
Notifier 167
OOInitStubs 177
Object 178
ObjectType 182
OpenFileChnl 186
OpenTcp 196
Panic 199
ParseArgs 200
ParseCmd 203
PkgRequire 209
Preserve 211
PrintDbl 212
RecEvalObj 213
RecordEval 214
RegConfig 214
RegExp 216
SaveResult 221
SetChanErr 223
SetErrno 225
SetRecLmt 226
SetResult 227
SetVar 230
Signal 233
Sleep 234
SourceRCFile 235

SplitList 235
SplitPath 238
StaticPkg 239
StdChannels 240
StrMatch 242
StringObj 242
SubstObj 248
TCL_MEM_DEBUG 249
TclZlib 251
Tcl_Main 255
Thread 258
ToUpper 262
TraceCmd 263
TraceVar 265
Translate 271
UniCharIsAlpha 272
UpVar 273
Utf 274
WrongNumArgs 278
Tk C API 279
3DBorder 280
AddOption 284
BindTable 285
CanvPsY 287
CanvTkwin 289
CanvTxtInfo 292
Clipboard 293
ClrSelect 294
ConfigWidg 295
ConfigWind 303
CoordToWin 306
CrtCmHdlr 306
CrtConsoleChan 307
CrtErrHdlr 308
CrtGenHdlr 310
CrtImgType 311
CrtItemType 316
CrtPhImgFmt 326
CrtSelHdlr 330
CrtWindow 332
DeleteImg 334
DrawFocHlt 334
EventHndlr 335
FindPhoto 336
FontId 340
FreeXId 342

GeomReq 342
GetAnchor 344
GetBitmap 345
GetCapStyl 349
GetClrmap 350
GetColor 351
GetCursor 354
GetDash 357
GetFont 358
GetGC 360
GetHINSTANCE 361
GetHWND 361
GetImage 362
GetJoinStl 364
GetJustify 365
GetOption 366
GetPixels 367
GetPixmap 368
GetRelief 369
GetRootCrd 370
GetScroll 371
GetSelect 372
GetUid 373
GetVRoot 374
GetVisual 375
Grab 376
HWNDToWindow 377
HandleEvent 377
IdToWindow 378
ImgChanged 379
Inactive 380
InternAtom 380
MainLoop 381
MainWin 381
MaintGeom 382
ManageGeom 383
MapWindow 385
MeasureChar 385
MoveToplev 387
Name 388
NameOfImg 389
OwnSelect 390
ParseArgv 391
QWinEvent 396
Restack 397
RestrictEv 398

SetAppName 399
SetCaret 400
SetClass 400
SetClassProcs 401
SetGrid 403
SetOptions 404
SetVisual 414
StrictMotif 415
TextLayout 415
TkInitStubs 420
Tk_Init 421
Tk_Main 422
WindowId 423
ttk_Geometry 426
ttk_Theme 430
License 430
Tcl C API license terms 430
Tk C API license terms 431

Tcl C API

Access

Name

Tcl_Access, Tcl_Stat - check file permissions and other attributes

Synopsis

#include <tcl.h>

int
Tcl_Access(*path*, *mode*)

int
Tcl_Stat(*path*, *statPtr*)

Arguments

const char *_path_ (in)
Native name of the file to check the attributes of.

int _mode_ (in)
Mask consisting of one or more of **R_OK**, **W_OK**, **X_OK** and **F_OK**. **R_OK**, **W_OK** and **X_OK** request checking whether the file exists and has read, write and execute permissions, respectively. **F_OK** just requests a check for the existence of the file.

struct stat *_statPtr_ (out)
The structure that contains the result.

Description

As of Tcl 8.4, the object-based APIs **Tcl_FSAccess** and **Tcl_FSStat** should be used in preference to **Tcl_Access** and **Tcl_Stat**, wherever possible. Those functions also support Tcl's virtual filesystem layer, which these do not.

Obsolete functions

There are two reasons for calling **Tcl_Access** and **Tcl_Stat** rather than calling system level functions **access** and **stat** directly. First, the Windows implementation of both functions fixes some bugs in the system level calls. Second, both **Tcl_Access** and **Tcl_Stat** (as well as **Tcl_OpenFileChannelProc**) hook into a linked list of functions. This allows the possibility to reroute file access to alternative media or access methods.

Tcl_Access checks whether the process would be allowed to read, write or test for existence of the file (or other file system object) whose name is *path*. If *path* is a symbolic link on Unix, then permissions of the file referred by this symbolic link are tested.

On success (all requested permissions granted), zero is returned. On error (at least one bit in mode asked for a permission that is denied, or some other error occurred), -1 is returned.

Tcl_Stat fills the stat structure *statPtr* with information about the specified file. You do not need any access rights to the file to get this information but you need search rights to all directories named in the path leading to the file. The stat structure includes info regarding device, inode (always 0 on Windows), privilege mode, nlink (always 1 on Windows), user id (always 0 on Windows), group id (always 0 on Windows), rdev (same as device on Windows), size, last access

time, last modification time, and creation time.

If *path* exists, **Tcl_Stat** returns 0 and the stat structure is filled with data. Otherwise, -1 is returned, and no stat info is given.

Keywords: Access, stat

Copyright (c) 1998-1999 Scriptics Corporation

AddErrInfo

Name

Tcl_GetReturnOptions, Tcl_SetReturnOptions, Tcl_AddErrorInfo, Tcl_AppendObjToErrorInfo, Tcl_AddObjErrorInfo, Tcl_SetObjErrorCode, Tcl_SetErrorCode, Tcl_SetErrorCodeVA, Tcl_SetErrorLine, Tcl_GetErrorLine, Tcl_PosixError, Tcl_LogCommandInfo - retrieve or record information about errors and other return options

Synopsis

#include <tcl.h>

Tcl_Obj *
Tcl_GetReturnOptions(*interp*, *code*)

int
Tcl_SetReturnOptions(*interp*, *options*)

Tcl_AddErrorInfo(*interp*, *message*)

Tcl_AppendObjToErrorInfo(*interp*, *objPtr*)

Tcl_AddObjErrorInfo(*interp*, *message*, *length*)

Tcl_SetObjErrorCode(*interp*, *errorObjPtr*)

Tcl_SetErrorCode(*interp*, *element*, *element*, ... (**char** *)**NULL**)

Tcl_SetErrorCodeVA(*interp*, *argList*)

Tcl_GetErrorLine(*interp*)

Tcl_SetErrorLine(*interp*, *lineNum*)

const char *
Tcl_PosixError(*interp*)

void
Tcl_LogCommandInfo(*interp*, *script*, *command*, *commandLength*)

Arguments

Tcl_Interp ****interp* **(in)**
Interpreter in which to record information.

int *code*
The code returned from script evaluation.

Tcl_Obj ****options*
A dictionary of return options.

const char ****message* **(in)**
For **Tcl_AddErrorInfo**, this is a conventional C string to append to the **-errorinfo** return

option. For **Tcl_AddObjErrorInfo**, this points to the first byte of an array of *length* bytes containing a string to append to the **-errorinfo** return option. This byte array may contain embedded null bytes unless *length* is negative.

Tcl_Obj *_objPtr_ (in)
A message to be appended to the **-errorinfo** return option in the form of a Tcl_Obj value.

int _length_ (in)
The number of bytes to copy from *message* when appending to the **-errorinfo** return option. If negative, all bytes up to the first null byte are used.

Tcl_Obj *_errorObjPtr_ (in)
The **-errorcode** return option will be set to this value.

const char *_element_ (in)
String to record as one element of the **-errorcode** return option. Last *element* argument must be (char *)NULL.

va_list _argList_ (in)
An argument list which must have been initialized using **va_start**, and cleared using **va_end**.

int _lineNum_
The line number of a script where an error occurred.

const char *_script_ (in)
Pointer to first character in script containing command (must be <= command)

const char *_command_ (in)
Pointer to first character in command that generated the error

int _commandLength_ (in)
Number of bytes in command; -1 means use all bytes up to first null byte

Description

The **Tcl_SetReturnOptions** and **Tcl_GetReturnOptions** routines expose the same capabilities as the **return** and **catch** commands, respectively, in the form of a C interface.

Tcl_GetReturnOptions retrieves the dictionary of return options from an interpreter following a script evaluation. Routines such as **Tcl_Eval** are called to evaluate a script in an interpreter. These routines return an integer completion code. These routines also leave in the interpreter both a result and a dictionary of return options generated by script evaluation. Just as **Tcl_GetObjResult** retrieves the result, **Tcl_GetReturnOptions** retrieves the dictionary of return options. The integer completion code should be passed as the *code* argument to **Tcl_GetReturnOptions** so that all required options will be present in the dictionary. Specifically, a *code* value of **TCL_ERROR** will ensure that entries for the keys **-errorinfo**, **-errorcode**, and **-errorline** will appear in the dictionary. Also, the entries for the keys **-code** and **-level** will be adjusted if necessary to agree with the value of *code*. The **(Tcl_Obj *)** returned by **Tcl_GetReturnOptions** points to an unshared **Tcl_Obj** with reference count of zero. The dictionary may be written to, either adding, removing, or overwriting any entries in it, without the need to check for a shared value. As with any **Tcl_Obj** with reference count of zero, it is up to the caller to arrange for its disposal with **Tcl_DecrRefCount** or to a reference to it via **Tcl_IncrRefCount** (or one of the many functions that call that, notably including **Tcl_SetObjResult** and **Tcl_SetVar2Ex**).

A typical usage for **Tcl_GetReturnOptions** is to retrieve the stack trace when script evaluation

returns **TCL_ERROR**, like so:

```
int code = Tcl_Eval(interp, script);
if (code == TCL_ERROR) {
    Tcl_Obj *options = Tcl_GetReturnOptions(interp, code);
    Tcl_Obj *key = Tcl_NewStringObj("-errorinfo", -1);
    Tcl_Obj *stackTrace;
    Tcl_IncrRefCount(key);
    Tcl_DictObjGet(NULL, options, key, &stackTrace);
    Tcl_DecrRefCount(key);
    /* Do something with stackTrace */
    Tcl_DecrRefCount(options);
}
```

Tcl_SetReturnOptions sets the return options of *interp* to be *options*. If *options* contains any invalid value for any key, TCL_ERROR will be returned, and the interp result will be set to an appropriate error message. Otherwise, a completion code in agreement with the **-code** and **-level** keys in *options* will be returned.

As an example, Tcl's **return** command itself could be implemented in terms of **Tcl_SetReturnOptions** like so:

```
if ((objc % 2) == 0) { /* explicit result argument */
    objc--;
    Tcl_SetObjResult(interp, objv[objc]);
}
return Tcl_SetReturnOptions(interp, Tcl_NewListObj(objc-1, objv+1));
```

(It is not really implemented that way. Internal access privileges allow for a more efficient alternative that meshes better with the bytecode compiler.)

Note that a newly created **Tcl_Obj** may be passed in as the *options* argument without the need to tend to any reference counting. This is analogous to **Tcl_SetObjResult**.

While **Tcl_SetReturnOptions** provides a general interface to set any collection of return options, there are a handful of return options that are very frequently used. Most notably the **-errorinfo** and **-errorcode** return options should be set properly when the command procedure of a command returns **TCL_ERROR**. The **-errorline** return option is also read by commands that evaluate scripts and wish to supply detailed error location information in the stack trace text they append to the **-errorinfo** option. Tcl provides several simpler interfaces to more directly set these return options.

The **-errorinfo** option holds a stack trace of the operations that were in progress when an error occurred, and is intended to be human-readable. The **-errorcode** option holds a Tcl list of items that are intended to be machine-readable. The first item in the **-errorcode** value identifies the class of error that occurred (e.g., POSIX means an error occurred in a POSIX system call) and additional elements hold additional pieces of information that depend on the class. See the manual entry on the **errorCode** variable for details on the various formats for the **-errorcode** option used by Tcl's built-in commands.

The **-errorinfo** option value is gradually built up as an error unwinds through the nested operations. Each time an error code is returned to **Tcl_Eval**, or any of the routines that performs

script evaluation, the procedure **Tcl_AddErrorInfo** is called to add additional text to the **-errorinfo** value describing the command that was being executed when the error occurred. By the time the error has been passed all the way back to the application, it will contain a complete trace of the activity in progress when the error occurred.

It is sometimes useful to add additional information to the **-errorinfo** value beyond what can be supplied automatically by the script evaluation routines. **Tcl_AddErrorInfo** may be used for this purpose: its *message* argument is an additional string to be appended to the **-errorinfo** option. For example, when an error arises during the **source** command, the procedure **Tcl_AddErrorInfo** is called to record the name of the file being processed and the line number on which the error occurred. Likewise, when an error arises during evaluation of a Tcl procedures, the procedure name and line number within the procedure are recorded, and so on. The best time to call **Tcl_AddErrorInfo** is just after a script evaluation routine has returned **TCL_ERROR**. The value of the **-errorline** return option (retrieved via a call to **Tcl_GetReturnOptions**) often makes up a useful part of the *message* passed to **Tcl_AddErrorInfo**.

Tcl_AppendObjToErrorInfo is an alternative interface to the same functionality as **Tcl_AddErrorInfo**. **Tcl_AppendObjToErrorInfo** is called when the string value to be appended to the **-errorinfo** option is available as a **Tcl_Obj** instead of as a **char** array.

Tcl_AddObjErrorInfo is nearly identical to **Tcl_AddErrorInfo**, except that it has an additional *length* argument. This allows the *message* string to contain embedded null bytes. This is essentially never a good idea. If the *message* needs to contain the null character **U+0000**, Tcl's usual internal encoding rules should be used to avoid the need for a null byte. If the **Tcl_AddObjErrorInfo** interface is used at all, it should be with a negative *length* value.

The procedure **Tcl_SetObjErrorCode** is used to set the **-errorcode** return option to the list value *errorObjPtr* built up by the caller. **Tcl_SetObjErrorCode** is typically invoked just before returning an error. If an error is returned without calling **Tcl_SetObjErrorCode** or **Tcl_SetErrorCode** the Tcl interpreter automatically sets the **-errorcode** return option to **NONE**.

The procedure **Tcl_SetErrorCode** is also used to set the **-errorcode** return option. However, it takes one or more strings to record instead of a value. Otherwise, it is similar to **Tcl_SetObjErrorCode** in behavior.

Tcl_SetErrorCodeVA is the same as **Tcl_SetErrorCode** except that instead of taking a variable number of arguments it takes an argument list.

The procedure **Tcl_GetErrorLine** is used to read the integer value of the **-errorline** return option without the overhead of a full call to **Tcl_GetReturnOptions**. Likewise, **Tcl_SetErrorLine** sets the **-errorline** return option value.

Tcl_PosixError sets the **-errorcode** variable after an error in a POSIX kernel call. It reads the value of the **errno** C variable and calls **Tcl_SetErrorCode** to set the **-errorcode** return option in the **POSIX** format. The caller must previously have called **Tcl_SetErrno** to set **errno**; this is necessary on some platforms (e.g. Windows) where Tcl is linked into an application as a shared library, or when the error occurs in a dynamically loaded extension. See the manual entry for **Tcl_SetErrno** for more information.

Tcl_PosixError returns a human-readable diagnostic message for the error (this is the same value that will appear as the third element in the **-errorcode** value). It may be convenient to include this string as part of the error message returned to the application in the interpreter's result.

Tcl_LogCommandInfo is invoked after an error occurs in an interpreter. It adds information about the command that was being executed when the error occurred to the **-errorinfo** value, and the line number stored internally in the interpreter is set.

In older releases of Tcl, there was no **Tcl_GetReturnOptions** routine. In its place, the global Tcl variables **errorInfo** and **errorCode** were the only place to retrieve the error information. Much existing code written for older Tcl releases still access this information via those global variables.

It is important to realize that while reading from those global variables remains a supported way to access these return option values, it is important not to assume that writing to those global variables will properly set the corresponding return options. It has long been emphasized in this manual page that it is important to call the procedures described here rather than setting **errorInfo** or **errorCode** directly with **Tcl_ObjSetVar2**.

If the procedure **Tcl_ResetResult** is called, it clears all of the state of the interpreter associated with script evaluation, including the entire return options dictionary. In particular, the **-errorinfo** and **-errorcode** options are reset. If an error had occurred, the **Tcl_ResetResult** call will clear the error state to make it appear as if no error had occurred after all. The global variables **errorInfo** and **errorCode** are not modified by **Tcl_ResetResult** so they continue to hold a record of information about the most recent error seen in an interpreter.

Keywords: AddErrInfo, error, value, value result, stack, trace, variable

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

Alloc

Name

Tcl_Alloc, Tcl_Free, Tcl_Realloc, Tcl_AttemptAlloc, Tcl_AttemptRealloc, Tcl_GetMemoryInfo, ckalloc, ckfree, ckrealloc, attemptckalloc, attemptckrealloc - allocate or free heap memory

Synopsis

#include <tcl.h>

char *
Tcl_Alloc(*size*)

void
Tcl_Free(*ptr*)

char *
Tcl_Realloc(*ptr*, *size*)

char *
Tcl_AttemptAlloc(*size*)

char *
Tcl_AttemptRealloc(*ptr*, *size*)

void
Tcl_GetMemoryInfo(*dsPtr*)

char *
ckalloc(*size*)

void
ckfree(*ptr*)

char *
ckrealloc(*ptr*, *size*)

char *
attemptckalloc(*size*)

char *
attemptckrealloc(*ptr*, *size*)

Arguments

unsigned int *size* (in)
Size in bytes of the memory block to allocate.

char **ptr* (in)
Pointer to memory block to free or realloc.

Tcl_DString **dsPtr* (in)
Initialized DString pointer.

Description

These procedures provide a platform and compiler independent interface for memory allocation. Programs that need to transfer ownership of memory blocks between Tcl and other modules should use these routines rather than the native **malloc()** and **free()** routines provided by the C run-time library.

Tcl_Alloc returns a pointer to a block of at least *size* bytes suitably aligned for any use.

Tcl_Free makes the space referred to by *ptr* available for further allocation.

Tcl_Realloc changes the size of the block pointed to by *ptr* to *size* bytes and returns a pointer to the new block. The contents will be unchanged up to the lesser of the new and old sizes. The returned location may be different from *ptr*. If *ptr* is NULL, this is equivalent to calling **Tcl_Alloc** with just the *size* argument.

Tcl_AttemptAlloc and **Tcl_AttemptRealloc** are identical in function to **Tcl_Alloc** and **Tcl_Realloc**, except that **Tcl_AttemptAlloc** and **Tcl_AttemptRealloc** will not cause the Tcl interpreter to **panic** if the memory allocation fails. If the allocation fails, these functions will return NULL. Note that on some platforms, but not all, attempting to allocate a zero-sized block of memory will also cause these functions to return NULL.

The procedures **ckalloc**, **ckfree**, **ckrealloc**, **attemptckalloc**, and **attemptckrealloc** are implemented as macros. Normally, they are synonyms for the corresponding procedures documented on this page. When Tcl and all modules calling Tcl are compiled with **TCL_MEM_DEBUG** defined, however, these macros are redefined to be special debugging versions of these procedures. To support Tcl's memory debugging within a module, use the macros rather than direct calls to **Tcl_Alloc**, etc.

Tcl_GetMemoryInfo appends a list-of-lists of memory stats to the provided DString. This function cannot be used in stub-enabled extensions, and it is only available if Tcl is compiled with the threaded memory allocator.

Keywords: Alloc, allocation, free, malloc, memory, realloc, TCL_MEM_DEBUG

Copyright (c) 1995-1996 Sun Microsystems, Inc.

AllowExc

Name

Tcl_AllowExceptions - allow all exceptions in next script evaluation

Synopsis

#include <tcl.h>

Tcl_AllowExceptions(*interp*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter in which script will be evaluated.

Description

If a script is evaluated at top-level (i.e. no other scripts are pending evaluation when the script is invoked), and if the script terminates with a completion code other than **TCL_OK**, **TCL_ERROR** or **TCL_RETURN**, then Tcl normally converts this into a **TCL_ERROR** return with an appropriate message. The particular script evaluation procedures of Tcl that act in the manner are **Tcl_EvalObjEx**, **Tcl_EvalObjv**, **Tcl_Eval**, **Tcl_EvalEx**, **Tcl_GlobalEval**, **Tcl_GlobalEvalObj**, **Tcl_VarEval** and **Tcl_VarEvalVA**.

However, if **Tcl_AllowExceptions** is invoked immediately before calling one of those a procedures, then arbitrary completion codes are permitted from the script, and they are returned without modification. This is useful in cases where the caller can deal with exceptions such as **TCL_BREAK** or **TCL_CONTINUE** in a meaningful way.

Keywords: AllowExc, continue, break, exception, interpreter

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

AppInit

Name

Tcl_AppInit - perform application-specific initialization

Synopsis

#include <tcl.h>

int
Tcl_AppInit(*interp*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter for the application.

Description

Tcl_AppInit is a "hook" procedure that is invoked by the main programs for Tcl applications such

as **tclsh** and **wish**. Its purpose is to allow new Tcl applications to be created without modifying the main programs provided as part of Tcl and Tk. To create a new application you write a new version of **Tcl_AppInit** to replace the default version provided by Tcl, then link your new **Tcl_AppInit** with the Tcl library.

Tcl_AppInit is invoked by **Tcl_Main** and **Tk_Main** after their own initialization and before entering the main loop to process commands. Here are some examples of things that **Tcl_AppInit** might do:

[1]
Call initialization procedures for various packages used by the application. Each initialization procedure adds new commands to *interp* for its package and performs other package-specific initialization.

[2]
Process command-line arguments, which can be accessed from the Tcl variables **argv** and **argv0** in *interp*.

[3]
Invoke a startup script to initialize the application.

[4]
Use the routines **Tcl_SetStartupScript** and **Tcl_GetStartupScript** to set or query the file and encoding that the active **Tcl_Main** or **Tk_Main** routine will use as a startup script. **Tcl_AppInit** returns **TCL_OK** or **TCL_ERROR**. If it returns **TCL_ERROR** then it must leave an error message in for the interpreter's result; otherwise the result is ignored. In addition to **Tcl_AppInit**, your application should also contain a procedure **main** that calls **Tcl_Main** as follows:

```
Tcl_Main(argc, argv, Tcl_AppInit);
```

The third argument to **Tcl_Main** gives the address of the application-specific initialization procedure to invoke. This means that you do not have to use the name **Tcl_AppInit** for the procedure, but in practice the name is nearly always **Tcl_AppInit** (in versions before Tcl 7.4 the name **Tcl_AppInit** was implicit; there was no way to specify the procedure explicitly). The best way to get started is to make a copy of the file **tclAppInit.c** from the Tcl library or source directory. It already contains a **main** procedure and a template for **Tcl_AppInit** that you can modify for your application.

Keywords: AppInit, application, argument, command, initialization, interpreter

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

AssocData

Name

Tcl_GetAssocData, Tcl_SetAssocData, Tcl_DeleteAssocData - manage associations of string keys and user specified data with Tcl interpreters

Synopsis

#include <tcl.h>

ClientData

Tcl_GetAssocData(*interp*, *key*, *delProcPtr*)

Tcl_SetAssocData(*interp*, *key*, *delProc*, *clientData*)

Tcl_DeleteAssocData(*interp*, *key*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter in which to execute the specified command.

const char *_key_ (in)
Key for association with which to store data or from which to delete or retrieve data. Typically the module prefix for a package.

Tcl_InterpDeleteProc *_delProc_ (in)
Procedure to call when *interp* is deleted.

Tcl_InterpDeleteProc **_delProcPtr_ (in)
Pointer to location in which to store address of current deletion procedure for association. Ignored if NULL.

ClientData _clientData_ (in)
Arbitrary one-word value associated with the given key in this interpreter. This data is owned by the caller.

Description

These procedures allow extensions to associate their own data with a Tcl interpreter. An association consists of a string key, typically the name of the extension, and a one-word value, which is typically a pointer to a data structure holding data specific to the extension. Tcl makes no interpretation of either the key or the value for an association.

Storage management is facilitated by storing with each association a procedure to call when the interpreter is deleted. This procedure can dispose of the storage occupied by the client's data in any way it sees fit.

Tcl_SetAssocData creates an association between a string key and a user specified datum in the given interpreter. If there is already an association with the given *key*, **Tcl_SetAssocData** overwrites it with the new information. It is up to callers to organize their use of names to avoid conflicts, for example, by using package names as the keys. If the *deleteProc* argument is non-NULL it specifies the address of a procedure to invoke if the interpreter is deleted before the association is deleted. *DeleteProc* should have arguments and result that match the type **Tcl_InterpDeleteProc**:

```
typedef void Tcl_InterpDeleteProc(
        ClientData clientData,
        Tcl_Interp *interp);
```

When *deleteProc* is invoked the *clientData* and *interp* arguments will be the same as the corresponding arguments passed to **Tcl_SetAssocData**. The deletion procedure will *not* be invoked if the association is deleted before the interpreter is deleted.

Tcl_GetAssocData returns the datum stored in the association with the specified key in the given interpreter, and if the *delProcPtr* field is non-**NULL**, the address indicated by it gets the address of the delete procedure stored with this association. If no association with the specified

key exists in the given interpreter **Tcl_GetAssocData** returns **NULL**.

Tcl_DeleteAssocData deletes an association with a specified key in the given interpreter. Then it calls the deletion procedure.

Keywords: AssocData, association, data, deletion procedure, interpreter, key

Copyright (c) 1995-1996 Sun Microsystems, Inc.

Async

Name

Tcl_AsyncCreate, Tcl_AsyncMark, Tcl_AsyncInvoke, Tcl_AsyncDelete, Tcl_AsyncReady - handle asynchronous events

Synopsis

#include <tcl.h>

Tcl_AsyncHandler
Tcl_AsyncCreate(*proc*, *clientData*)

void
Tcl_AsyncMark(*async*)

int
Tcl_AsyncInvoke(*interp*, *code*)

void
Tcl_AsyncDelete(*async*)

int
Tcl_AsyncReady()

Arguments

Tcl_AsyncProc *_proc_ (in)
Procedure to invoke to handle an asynchronous event.

ClientData _clientData_ (in)
One-word value to pass to *proc*.

Tcl_AsyncHandler _async_ (in)
Token for asynchronous event handler.

Tcl_Interp *_interp_ (in)
Tcl interpreter in which command was being evaluated when handler was invoked, or NULL if handler was invoked when there was no interpreter active.

int _code_ (in)
Completion code from command that just completed in *interp*, or 0 if *interp* is NULL.

Description

These procedures provide a safe mechanism for dealing with asynchronous events such as signals. If an event such as a signal occurs while a Tcl script is being evaluated then it is not safe to take any substantive action to process the event. For example, it is not safe to evaluate a Tcl script since the interpreter may already be in the middle of evaluating a script; it may not even be safe to allocate memory, since a memory allocation could have been in progress when the

event occurred. The only safe approach is to set a flag indicating that the event occurred, then handle the event later when the world has returned to a clean state, such as after the current Tcl command completes.

Tcl_AsyncCreate, **Tcl_AsyncDelete**, and **Tcl_AsyncReady** are thread sensitive. They access and/or set a thread-specific data structure in the event of a core built with *--enable-threads*. The token created by **Tcl_AsyncCreate** contains the needed thread information it was called from so that calling **Tcl_AsyncMark**(*token*) will only yield the origin thread into the asynchronous handler.

Tcl_AsyncCreate creates an asynchronous handler and returns a token for it. The asynchronous handler must be created before any occurrences of the asynchronous event that it is intended to handle (it is not safe to create a handler at the time of an event). When an asynchronous event occurs the code that detects the event (such as a signal handler) should call **Tcl_AsyncMark** with the token for the handler. **Tcl_AsyncMark** will mark the handler as ready to execute, but it will not invoke the handler immediately. Tcl will call the *proc* associated with the handler later, when the world is in a safe state, and *proc* can then carry out the actions associated with the asynchronous event. *Proc* should have arguments and result that match the type **Tcl_AsyncProc**:

```
typedef int Tcl_AsyncProc(
        ClientData clientData,
        Tcl_Interp *interp,
        int code);
```

The *clientData* will be the same as the *clientData* argument passed to **Tcl_AsyncCreate** when the handler was created. If *proc* is invoked just after a command has completed execution in an interpreter, then *interp* will identify the interpreter in which the command was evaluated and *code* will be the completion code returned by that command. The command's result will be present in the interpreter's result. When *proc* returns, whatever it leaves in the interpreter's result will be returned as the result of the command and the integer value returned by *proc* will be used as the new completion code for the command.

It is also possible for *proc* to be invoked when no interpreter is active. This can happen, for example, if an asynchronous event occurs while the application is waiting for interactive input or an X event. In this case *interp* will be NULL and *code* will be 0, and the return value from *proc* will be ignored.

The procedure **Tcl_AsyncInvoke** is called to invoke all of the handlers that are ready. The procedure **Tcl_AsyncReady** will return non-zero whenever any asynchronous handlers are ready; it can be checked to avoid calls to **Tcl_AsyncInvoke** when there are no ready handlers. Tcl calls **Tcl_AsyncReady** after each command is evaluated and calls **Tcl_AsyncInvoke** if needed. Applications may also call **Tcl_AsyncInvoke** at interesting times for that application. For example, Tcl's event handler calls **Tcl_AsyncReady** after each event and calls **Tcl_AsyncInvoke** if needed. The *interp* and *code* arguments to **Tcl_AsyncInvoke** have the same meaning as for *proc*: they identify the active interpreter, if any, and the completion code from the command that just completed.

Tcl_AsyncDelete removes an asynchronous handler so that its *proc* will never be invoked again. A handler can be deleted even when ready, and it will still not be invoked.

If multiple handlers become active at the same time, the handlers are invoked in the order they

were created (oldest handler first). The *code* and the interpreter's result for later handlers reflect the values returned by earlier handlers, so that the most recently created handler has last say about the interpreter's result and completion code. If new handlers become ready while handlers are executing, **Tcl_AsyncInvoke** will invoke them all; at each point it invokes the highest-priority (oldest) ready handler, repeating this over and over until there are no longer any ready handlers.

Warning

It is almost always a bad idea for an asynchronous event handler to modify the interpreter's result or return a code different from its *code* argument. This sort of behavior can disrupt the execution of scripts in subtle ways and result in bugs that are extremely difficult to track down. If an asynchronous event handler needs to evaluate Tcl scripts then it should first save the interpreter's state by calling **Tcl_SaveInterpState**, passing in the *code* argument. When the asynchronous handler is finished it should restore the interpreter's state by calling **Tcl_RestoreInterpState**, and then returning the *code* argument.

Keywords: Async, asynchronous event, handler, signal, Tcl_SaveInterpState, thread

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

BackgdErr

Name

Tcl_BackgroundException, Tcl_BackgroundError - report Tcl exception that occurred in background processing

Synopsis

#include <tcl.h>

Tcl_BackgroundException(*interp*, *code*)

Tcl_BackgroundError(*interp*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter in which the exception occurred.

int _code_ (in)
The exceptional return code to be reported.

Description

This procedure is typically invoked when a Tcl exception (any return code other than TCL_OK) occurs during "background processing" such as executing an event handler. When such an exception occurs, the condition is reported to Tcl or to a widget or some other C code, and there is not usually any obvious way for that code to report the exception to the user. In these cases the code calls **Tcl_BackgroundException** with an *interp* argument identifying the interpreter in which the exception occurred, and a *code* argument holding the return code value of the exception. The state of the interpreter, including any error message in the interpreter result, and the values of any entries in the return options dictionary, is captured and saved.
Tcl_BackgroundException then arranges for the event loop to invoke at some later time the command registered in that interpreter to handle background errors by the **interp bgerror**

command, passing the captured values as arguments. The registered handler command is meant to report the exception in an application-specific fashion. The handler command receives two arguments, the result of the interp, and the return options of the interp at the time the error occurred. If the application registers no handler command, the default handler command will attempt to call **bgerror** to report the error. If an error condition arises while invoking the handler command, then **Tcl_BackgroundException** reports the error itself by printing a message on the standard error file.

Tcl_BackgroundException does not invoke the handler command immediately because this could potentially interfere with scripts that are in process at the time the error occurred. Instead, it invokes the handler command later as an idle callback.

It is possible for many background exceptions to accumulate before the handler command is invoked. When this happens, each of the exceptions is processed in order. However, if the handler command returns a break exception, then all remaining error reports for the interpreter are skipped.

The **Tcl_BackgroundError** routine is an older and simpler interface useful when the exception code reported is **TCL_ERROR**. It is equivalent to:

```
Tcl_BackgroundException(interp, TCL_ERROR);
```

Keywords: BackgdErr, background, bgerror, error, interp

Copyright (c) 1992-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

Backslash

Name

Tcl_Backslash - parse a backslash sequence

Synopsis

#include <tcl.h>

char
Tcl_Backslash(*src*, *countPtr*)

Arguments

const char *_src_ (in)
Pointer to a string starting with a backslash.

int *_countPtr_ (out)
If *countPtr* is not NULL, **countPtr* gets filled in with number of characters in the backslash sequence, including the backslash character.

Description

The use of **Tcl_Backslash** is deprecated in favor of **Tcl_UtfBackslash**.

This is a utility procedure provided for backwards compatibility with non-internationalized Tcl extensions. It parses a backslash sequence and returns the low byte of the Unicode character corresponding to the sequence. **Tcl_Backslash** modifies **countPtr* to contain the number of characters in the backslash sequence.

See the Tcl manual entry for information on the valid backslash sequences. All of the sequences described in the Tcl manual entry are supported by **Tcl_Backslash**.

Keywords: Backslash, parse

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

BoolObj

Name

Tcl_NewBooleanObj, Tcl_SetBooleanObj, Tcl_GetBooleanFromObj - store/retrieve boolean value in a Tcl_Obj

Synopsis

#include <tcl.h>

Tcl_Obj *
Tcl_NewBooleanObj(*intValue*)

Tcl_SetBooleanObj(*objPtr*, *intValue*)

int
Tcl_GetBooleanFromObj(*interp*, *objPtr*, *intPtr*)

Arguments

int *intValue* (in)
Integer value to be stored as a boolean value in a Tcl_Obj.

Tcl_Obj **objPtr* (in/out)
Points to the Tcl_Obj in which to store, or from which to retrieve a boolean value.

Tcl_Interp **interp* (in/out)
If a boolean value cannot be retrieved, an error message is left in the interpreter's result value unless *interp* is NULL.

int **intPtr* (out)
Points to place where **Tcl_GetBooleanFromObj** stores the boolean value (0 or 1) obtained from *objPtr*.

Description

These procedures are used to pass boolean values to and from Tcl as Tcl_Obj's. When storing a boolean value into a Tcl_Obj, any non-zero integer value in *intValue* is taken to be the boolean value **1**, and the integer value **0** is taken to be the boolean value **0**.

Tcl_NewBooleanObj creates a new Tcl_Obj, stores the boolean value *intValue* in it, and returns a pointer to the new Tcl_Obj. The new Tcl_Obj has reference count of zero.

Tcl_SetBooleanObj accepts *objPtr*, a pointer to an existing Tcl_Obj, and stores in the Tcl_Obj **objPtr* the boolean value *intValue*. This is a write operation on **objPtr*, so *objPtr* must be unshared. Attempts to write to a shared Tcl_Obj will panic. A successful write of *intValue* into **objPtr* implies the freeing of any former value stored in **objPtr*.

Tcl_GetBooleanFromObj attempts to retrieve a boolean value from the value stored in **objPtr*. If *objPtr* holds a string value recognized by **Tcl_GetBoolean**, then the recognized boolean value

is written at the address given by *intPtr*. If *objPtr* holds any value recognized as a number by Tcl, then if that value is zero a 0 is written at the address given by *intPtr* and if that value is non-zero a 1 is written at the address given by *intPtr*. In all cases where a value is written at the address given by *intPtr*, **Tcl_GetBooleanFromObj** returns **TCL_OK**. If the value of *objPtr* does not meet any of the conditions above, then **TCL_ERROR** is returned and an error message is left in the interpreter's result unless *interp* is NULL. **Tcl_GetBooleanFromObj** may also make changes to the internal fields of **objPtr* so that future calls to **Tcl_GetBooleanFromObj** on the same *objPtr* can be performed more efficiently.

Note that the routines **Tcl_GetBooleanFromObj** and **Tcl_GetBoolean** are not functional equivalents. The set of values for which **Tcl_GetBooleanFromObj** will return **TCL_OK** is strictly larger than the set of values for which **Tcl_GetBoolean** will do the same. For example, the value "5" passed to **Tcl_GetBooleanFromObj** will lead to a **TCL_OK** return (and the boolean value 1), while the same value passed to **Tcl_GetBoolean** will lead to a **TCL_ERROR** return.

Keywords: BoolObj, boolean, value

Copyright (c) 1996-1997 Sun Microsystems, Inc.
Contributions from Don Porter, NIST, 2005. (not subject to US copyright)

ByteArrObj

Name

Tcl_NewByteArrayObj, Tcl_SetByteArrayObj, Tcl_GetByteArrayFromObj, Tcl_SetByteArrayLength - manipulate Tcl values as a arrays of bytes

Synopsis

#include <tcl.h>

Tcl_Obj *
Tcl_NewByteArrayObj(*bytes*, *length*)

void
Tcl_SetByteArrayObj(*objPtr*, *bytes*, *length*)

unsigned char *
Tcl_GetByteArrayFromObj(*objPtr*, *lengthPtr*)

unsigned char *
Tcl_SetByteArrayLength(*objPtr*, *length*)

Arguments

const unsigned char *_bytes_ (in)
The array of bytes used to initialize or set a byte-array value. May be NULL even if *length* is non-zero.

int _length_ (in)
The length of the array of bytes. It must be >= 0.

Tcl_Obj *_objPtr_ (in/out)
For **Tcl_SetByteArrayObj**, this points to the value to be converted to byte-array type. For **Tcl_GetByteArrayFromObj** and **Tcl_SetByteArrayLength**, this points to the value from which to get the byte-array value; if *objPtr* does not already point to a byte-array value, it will be converted to one.

int *_lengthPtr_ (out)
If non-NULL, filled with the length of the array of bytes in the value.

Description

These procedures are used to create, modify, and read Tcl byte-array values from C code. Byte-array values are typically used to hold the results of binary IO operations or data structures created with the **binary** command. In Tcl, an array of bytes is not equivalent to a string. Conceptually, a string is an array of Unicode characters, while a byte-array is an array of 8-bit quantities with no implicit meaning. Accessor functions are provided to get the string representation of a byte-array or to convert an arbitrary value to a byte-array. Obtaining the string representation of a byte-array value (by calling **Tcl_GetStringFromObj**) produces a properly formed UTF-8 sequence with a one-to-one mapping between the bytes in the internal representation and the UTF-8 characters in the string representation.

Tcl_NewByteArrayObj and **Tcl_SetByteArrayObj** will create a new value of byte-array type or modify an existing value to have a byte-array type. Both of these procedures set the value's type to be byte-array and set the value's internal representation to a copy of the array of bytes given by *bytes*. **Tcl_NewByteArrayObj** returns a pointer to a newly allocated value with a reference count of zero. **Tcl_SetByteArrayObj** invalidates any old string representation and, if the value is not already a byte-array value, frees any old internal representation. If *bytes* is NULL then the new byte array contains arbitrary values.

Tcl_GetByteArrayFromObj converts a Tcl value to byte-array type and returns a pointer to the value's new internal representation as an array of bytes. The length of this array is stored in *lengthPtr* if *lengthPtr* is non-NULL. The storage for the array of bytes is owned by the value and should not be freed. The contents of the array may be modified by the caller only if the value is not shared and the caller invalidates the string representation.

Tcl_SetByteArrayLength converts the Tcl value to byte-array type and changes the length of the value's internal representation as an array of bytes. If *length* is greater than the space currently allocated for the array, the array is reallocated to the new length; the newly allocated bytes at the end of the array have arbitrary values. If *length* is less than the space currently allocated for the array, the length of array is reduced to the new length. The return value is a pointer to the value's new array of bytes.

Keywords: ByteArrObj, value, binary data, byte array, utf, unicode, internationalization

Copyright (c) 1997 Sun Microsystems, Inc.

CallDel

Name

Tcl_CallWhenDeleted, Tcl_DontCallWhenDeleted - Arrange for callback when interpreter is deleted

Synopsis

#include <tcl.h>

Tcl_CallWhenDeleted(*interp*, *proc*, *clientData*)

Tcl_DontCallWhenDeleted(*interp*, *proc*, *clientData*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter with which to associated callback.

Tcl_InterpDeleteProc *_proc_ (in)
Procedure to call when *interp* is deleted.

ClientData _clientData_ (in)
Arbitrary one-word value to pass to *proc*.

Description

Tcl_CallWhenDeleted arranges for *proc* to be called by **Tcl_DeleteInterp** if/when *interp* is deleted at some future time. *Proc* will be invoked just before the interpreter is deleted, but the interpreter will still be valid at the time of the call. *Proc* should have arguments and result that match the type **Tcl_InterpDeleteProc**:

```
typedef void Tcl_InterpDeleteProc(
        ClientData clientData,
        Tcl_Interp *interp);
```

The *clientData* and *interp* parameters are copies of the *clientData* and *interp* arguments given to **Tcl_CallWhenDeleted**. Typically, *clientData* points to an application-specific data structure that *proc* uses to perform cleanup when an interpreter is about to go away. *Proc* does not return a value.

Tcl_DontCallWhenDeleted cancels a previous call to **Tcl_CallWhenDeleted** with the same arguments, so that *proc* will not be called after all when *interp* is deleted. If there is no deletion callback that matches *interp*, *proc*, and *clientData* then the call to **Tcl_DontCallWhenDeleted** has no effect.

Note that if the callback is being used to delete a resource that *must* be released on exit, **Tcl_CreateExitHandler** should be used to ensure that a callback is received even if the application terminates without deleting the interpreter.

Keywords: CallDel, callback, cleanup, delete, interpreter

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

Cancel

Name

Tcl_CancelEval, Tcl_Canceled - cancel Tcl scripts

Synopsis

#include <tcl.h>
int
Tcl_CancelEval(*interp*, *resultObjPtr*, *clientData*, *flags*)

int
Tcl_Canceled(*interp*, *flags*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter in which to cancel the script.

Tcl_Obj *_resultObjPtr_ (in)
Error message to use in the cancellation, or NULL to use a default message. If not NULL, this object will have its reference count decremented before **Tcl_CancelEval** returns.

int _flags_ (in)
OR'ed combination of flag bits that specify additional options. For **Tcl_CancelEval**, only **TCL_CANCEL_UNWIND** is currently supported. For **Tcl_Canceled**, only **TCL_LEAVE_ERR_MSG** and **TCL_CANCEL_UNWIND** are currently supported.

ClientData _clientData_ (in)
Currently reserved for future use. It should be set to NULL.

Description

Tcl_CancelEval cancels or unwinds the script in progress soon after the next invocation of asynchronous handlers, causing **TCL_ERROR** to be the return code for that script. This function is thread-safe and may be called from any thread in the process.

Tcl_Canceled checks if the script in progress has been canceled and returns **TCL_ERROR** if it has. Otherwise, **TCL_OK** is returned. Extensions can use this function to check to see if they should abort a long running command. This function is thread sensitive and may only be called from the thread the interpreter was created in.

Flag bits

Any OR'ed combination of the following values may be used for the *flags* argument to procedures such as **Tcl_CancelEval**:

TCL_CANCEL_UNWIND
This flag is used by **Tcl_CancelEval** and **Tcl_Canceled**. For **Tcl_CancelEval**, if this flag is set, the script in progress is canceled and the evaluation stack for the interpreter is unwound. For **Tcl_Canceled**, if this flag is set, the script in progress is considered to be canceled only if the evaluation stack for the interpreter is being unwound.

TCL_LEAVE_ERR_MSG
This flag is only used by **Tcl_Canceled**; it is ignored by other procedures. If an error is returned and this bit is set in *flags*, then an error message will be left in the interpreter's result, where it can be retrieved with **Tcl_GetObjResult** or **Tcl_GetStringResult**. If this flag bit is not set then no error message is left and the interpreter's result will not be modified.

Keywords: Cancel, unwind

Copyright (c) 2006-2008 Joe Mistachkin.

ChnlStack

Name

Tcl_StackChannel, Tcl_UnstackChannel, Tcl_GetStackedChannel, Tcl_GetTopChannel - manipulate stacked I/O channels

Synopsis

.nf

#include <tcl.h>

Tcl_Channel
Tcl_StackChannel(*interp*, *typePtr*, *clientData*, *mask*, *channel*)

int
Tcl_UnstackChannel(*interp*, *channel*)

Tcl_Channel
Tcl_GetStackedChannel(*channel*)

Tcl_Channel
Tcl_GetTopChannel(*channel*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter for error reporting.

const Tcl_ChannelType *_typePtr_ (in)
The new channel I/O procedures to use for *channel*.

ClientData _clientData_ (in)
Arbitrary one-word value to pass to channel I/O procedures.

int _mask_ (in)
Conditions under which *channel* will be used: OR-ed combination of **TCL_READABLE**, **TCL_WRITABLE** and **TCL_EXCEPTION**. This can be a subset of the operations currently allowed on *channel*.

Tcl_Channel _channel_ (in)
An existing Tcl channel such as returned by **Tcl_CreateChannel**.

Description

These functions are for use by extensions that add processing layers to Tcl I/O channels. Examples include compression and encryption modules. These functions transparently stack and unstack a new channel on top of an existing one. Any number of channels can be stacked together.

The implementation of the Tcl channel code was rewritten in 8.3.2 to correct some problems with the previous implementation with regard to stacked channels. Anyone using stacked channels or creating stacked channel drivers should update to the new **TCL_CHANNEL_VERSION_2** **Tcl_ChannelType** structure. See **Tcl_CreateChannel** for details.

Tcl_StackChannel stacks a new *channel* on an existing channel with the same name that was registered for *channel* by **Tcl_RegisterChannel**.

Tcl_StackChannel works by creating a new channel structure and placing itself on top of the channel stack. EOL translation, encoding and buffering options are shared between all channels in the stack. The hidden channel does no buffering, newline translations, or character set encoding. Instead, the buffering, newline translations, and encoding functions all remain at the top of the channel stack. A pointer to the new top channel structure is returned. If an error occurs when stacking the channel, NULL is returned instead.

The *mask* parameter specifies the operations that are allowed on the new channel. These can be a subset of the operations allowed on the original channel. For example, a read-write channel may become read-only after the **Tcl_StackChannel** call.

Closing a channel closes the channels stacked below it. The close of stacked channels is executed in a way that allows buffered data to be properly flushed.

Tcl_UnstackChannel reverses the process. The old channel is associated with the channel name, and the processing module added by **Tcl_StackChannel** is destroyed. If there is no old channel, then **Tcl_UnstackChannel** is equivalent to **Tcl_Close**. If an error occurs unstacking the channel, **TCL_ERROR** is returned, otherwise **TCL_OK** is returned.

Tcl_GetTopChannel returns the top channel in the stack of channels the supplied channel is part of.

Tcl_GetStackedChannel returns the channel in the stack of channels which is just below the supplied channel.

Keywords: ChnlStack, channel, compression

Copyright (c) 1999-2000 Ajuba Solutions.

Class

Name

Tcl_ClassGetMetadata, Tcl_ClassSetMetadata, Tcl_CopyObjectInstance, Tcl_GetClassAsObject, Tcl_GetObjectAsClass, Tcl_GetObjectCommand, Tcl_GetObjectFromObj, Tcl_GetObjectName, Tcl_GetObjectNamespace, Tcl_NewObjectInstance, Tcl_ObjectDeleted, Tcl_ObjectGetMetadata, Tcl_ObjectGetMethodNameMapper, Tcl_ObjectSetMetadata, Tcl_ObjectSetMethodNameMapper - manipulate objects and classes

Synopsis

#include <tclOO.h>

Tcl_Object
Tcl_GetObjectFromObj(*interp*, *objPtr*)

Tcl_Object
Tcl_GetClassAsObject(*class*)

Tcl_Class
Tcl_GetObjectAsClass(*object*)

Tcl_Obj *
Tcl_GetObjectName(*interp*, *object*)

Tcl_Command
Tcl_GetObjectCommand(*object*)

Tcl_Namespace *
Tcl_GetObjectNamespace(*object*)

Tcl_Object
Tcl_NewObjectInstance(*interp*, *class*, *name*, *nsName*, *objc*, *objv*, *skip*)

Tcl_Object
Tcl_CopyObjectInstance(*interp*, *object*, *name*, *nsName*)

int
Tcl_ObjectDeleted(*object*)

ClientData

Tcl_ObjectGetMetadata(*object*, *metaTypePtr*)

Tcl_ObjectSetMetadata(*object*, *metaTypePtr*, *metadata*)

ClientData
Tcl_ClassGetMetadata(*class*, *metaTypePtr*)

Tcl_ClassSetMetadata(*class*, *metaTypePtr*, *metadata*)

Tcl_ObjectMapMethodNameProc
Tcl_ObjectGetMethodNameMapper(*object*)

Tcl_ObjectSetMethodNameMapper(*object*, *methodNameMapper*)

Arguments

Tcl_Interp *_interp_ (in/out)
Interpreter providing the context for looking up or creating an object, and into whose result error messages will be written on failure.

Tcl_Obj *_objPtr_ (in)
The name of the object to look up.

Tcl_Object _object_ (in)
Reference to the object to operate upon.

Tcl_Class _class_ (in)
Reference to the class to operate upon.

const char *_name_ (in)
The name of the object to create, or NULL if a new unused name is to be automatically selected.

const char *_nsName_ (in)
The name of the namespace to create for the object's private use, or NULL if a new unused name is to be automatically selected. The namespace must not already exist.

int _objc_ (in)
The number of elements in the *objv* array.

Tcl_Obj *const *_objv_ (in)
The arguments to the command to create the instance of the class.

int _skip_ (in)
The number of arguments at the start of the argument array, *objv*, that are not arguments to any constructors. This allows the generation of correct error messages even when complicated calling patterns are used (e.g., via the **next** command).

Tcl_ObjectMetadataType *_metaTypePtr_ (in)
The type of *metadata* being set with **Tcl_ClassSetMetadata** or retrieved with **Tcl_ClassGetMetadata**.

ClientData _metadata_ (in)
An item of metadata to attach to the class, or NULL to remove the metadata associated with a particular *metaTypePtr*.

Tcl_ObjectMapMethodNameProc _methodNameMapper_ (in)
A pointer to a function to call to adjust the mapping of objects and method names to implementations, or NULL when no such mapping is required.

Description

Objects are typed entities that have a set of operations ("methods") associated with them. Classes are objects that can manufacture objects. Each class can be viewed as an object itself; the object view can be retrieved using **Tcl_GetClassAsObject** which always returns the object when applied to a non-destroyed class, and an object can be viewed as a class with the aid of the **Tcl_GetObjectAsClass** (which either returns the class, or NULL if the object is not a class). An object may be looked up using the **Tcl_GetObjectFromObj** function, which either returns an object or NULL (with an error message in the interpreter result) if the object cannot be found. The correct way to look up a class by name is to look up the object with that name, and then to use **Tcl_GetObjectAsClass**.

Every object has its own command and namespace associated with it. The command may be retrieved using the **Tcl_GetObjectCommand** function, the name of the object (and hence the name of the command) with **Tcl_GetObjectName**, and the namespace may be retrieved using the **Tcl_GetObjectNamespace** function. Note that the Tcl_Obj reference returned by **Tcl_GetObjectName** is a shared reference. You can also get whether the object has been marked for deletion with **Tcl_ObjectDeleted** (it returns true if deletion of the object has begun); this can be useful during the processing of methods.

Instances of classes are created using **Tcl_NewObjectInstance**, which creates an object from any class (and which is internally called by both the **create** and **new** methods of the **oo::class** class). It takes parameters that optionally give the name of the object and namespace to create, and which describe the arguments to pass to the class's constructor (if any). The result of the function will be either a reference to the newly created object, or NULL if the creation failed (when an error message will be left in the interpreter result). In addition, objects may be copied by using **Tcl_CopyObjectInstance** which creates a copy of an object without running any constructors.

Note that the lifetime management of objects is handled internally within TclOO, and does not use **Tcl_Preserve**. *It is not safe to put a Tcl_Object handle in a C structure with a lifespan different to the object;* you should use the object's command name (as retrieved with **Tcl_GetObjectName**) instead. It is safe to use a Tcl_Object handle for the lifespan of a call of a method on that object; handles do not become invalid while there is an outstanding call on their object (even if the only operation guaranteed to be safe on them is **Tcl_ObjectDeleted**; the other operations are only guaranteed to work on non-deleted objects).

Object and class metadata

Every object and every class may have arbitrary amounts of metadata attached to it, which the object or class attaches no meaning to beyond what is described in a Tcl_ObjectMetadataType structure instance. Metadata to be attached is described by the type of the metadata (given in the *metaTypePtr* argument) and an arbitrary pointer (the *metadata* argument) that are given to **Tcl_ObjectSetMetadata** and **Tcl_ClassSetMetadata**, and a particular piece of metadata can be retrieved given its type using **Tcl_ObjectGetMetadata** and **Tcl_ClassGetMetadata**. If the *metadata* parameter to either **Tcl_ObjectSetMetadata** or **Tcl_ClassSetMetadata** is NULL, the metadata is removed if it was attached, and the results of **Tcl_ObjectGetMetadata** and **Tcl_ClassGetMetadata** are NULL if the given type of metadata was not attached. It is not an error to request or remove a piece of metadata that was not attached.

Tcl_objectmetadatatype structure

The contents of the Tcl_ObjectMetadataType structure are as follows:

```
typedef const struct {
    int version;
    const char *name;
    Tcl_ObjectMetadataDeleteProc *deleteProc;
    Tcl_CloneProc *cloneProc;
} Tcl_ObjectMetadataType;
```

The *version* field allows for future expansion of the structure, and should always be declared equal to TCL_OO_METADATA_VERSION_CURRENT. The *name* field provides a human-readable name for the type, and is reserved for debugging.

The *deleteProc* field gives a function of type Tcl_ObjectMetadataDeleteProc that is used to delete a particular piece of metadata, and is called when the attached metadata is replaced or removed; the field must not be NULL.

The *cloneProc* field gives a function that is used to copy a piece of metadata (used when a copy of an object is created using **Tcl_CopyObjectInstance**); if NULL, the metadata will be just directly copied.

Tcl_objectmetadatadeleteproc function signature

Functions matching this signature are used to delete metadata associated with a class or object.

```
typedef void Tcl_ObjectMetadataDeleteProc(
        ClientData metadata);
```

The *metadata* argument gives the address of the metadata to be deleted.

Tcl_cloneproc function signature

Functions matching this signature are used to create copies of metadata associated with a class or object.

```
typedef int Tcl_CloneProc(
        Tcl_Interp *interp,
        ClientData srcMetadata,
        ClientData *dstMetadataPtr);
```

The *interp* argument gives a place to write an error message when the attempt to clone the object is to fail, in which case the clone procedure must also return TCL_ERROR; it should return TCL_OK otherwise. The *srcMetadata* argument gives the address of the metadata to be cloned, and the cloned metadata should be written into the variable pointed to by *dstMetadataPtr*; a NULL should be written if the metadata is to not be cloned but the overall object copy operation is still to succeed.

Object method name mapping

It is possible to control, on a per-object basis, what methods are invoked when a particular method is invoked. Normally this is done by looking up the method name in the object and then in the class hierarchy, but fine control of exactly what the value used to perform the look up is afforded through the ability to set a method name mapper callback via

Tcl_ObjectSetMethodNameMapper (and its introspection counterpart, **Tcl_ObjectGetMethodNameMapper**, which returns the current mapper). The current mapper (if any) is invoked immediately before looking up what chain of method implementations is to be used.

Tcl_objectmapmethodnameproc function signature

The *Tcl_ObjectMapMethodNameProc* callback is defined as follows:

```
typedef int Tcl_ObjectMapMethodNameProc(
        Tcl_Interp *interp,
        Tcl_Object object,
        Tcl_Class *startClsPtr,
        Tcl_Obj *methodNameObj);
```

If the result is TCL_OK, the remapping is assumed to have been done. If the result is TCL_ERROR, an error message will have been left in *interp* and the method call will fail. If the result is TCL_BREAK, the standard method name lookup rules will be used; the behavior of other result codes is currently undefined. The *object* parameter says which object is being processed. The *startClsPtr* parameter points to a variable that contains the first class to provide a definition in the method chain to process, or NULL if the whole chain is to be processed (the argument itself is never NULL); this variable may be updated by the callback. The *methodNameObj* parameter gives an unshared object containing the name of the method being invoked, as provided by the user; this object may be updated by the callback.

Keywords: Class, constructor, object

Copyright (c) 2007 Donal K. Fellows

CmdCmplt

Name

Tcl_CommandComplete - Check for unmatched braces in a Tcl command

Synopsis

#include <tcl.h>

int
Tcl_CommandComplete(*cmd*)

Arguments

const char *_cmd_ (in)
Command string to test for completeness.

Description

Tcl_CommandComplete takes a Tcl command string as argument and determines whether it contains one or more complete commands (i.e. there are no unclosed quotes, braces, brackets, or variable references). If the command string is complete then it returns 1; otherwise it returns 0.

Keywords: CmdCmplt, complete command, partial command

Copyright (c) 1989-1993 The Regents of the University of California.

Copyright (c) 1994-1996 Sun Microsystems, Inc.

Concat

Name

Tcl_Concat - concatenate a collection of strings

Synopsis

#include <tcl.h>

const char *
Tcl_Concat(*argc*, *argv*)

Arguments

int *argc* (in)
Number of strings.

const char *const *argv[]* (in)
Array of strings to concatenate. Must have *argc* entries.

Description

Tcl_Concat is a utility procedure used by several of the Tcl commands. Given a collection of strings, it concatenates them together into a single string, with the original strings separated by spaces. This procedure behaves differently than **Tcl_Merge**, in that the arguments are simply concatenated: no effort is made to ensure proper list structure. However, in most common usage the arguments will all be proper lists themselves; if this is true, then the result will also have proper list structure.

Tcl_Concat eliminates leading and trailing white space as it copies strings from **argv** to the result. If an element of **argv** consists of nothing but white space, then that string is ignored entirely. This white-space removal was added to make the output of the **concat** command cleaner-looking.

The result string is dynamically allocated using **Tcl_Alloc**; the caller must eventually release the space by calling **Tcl_Free**.

Keywords: Concat, concatenate, strings

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

CrtAlias

Name

Tcl_IsSafe, Tcl_MakeSafe, Tcl_CreateChild, Tcl_CreateSlave, Tcl_GetChild, Tcl_GetSlave, Tcl_GetParent, Tcl_GetMaster, Tcl_GetInterpPath, Tcl_CreateAlias, Tcl_CreateAliasObj, Tcl_GetAlias, Tcl_GetAliasObj, Tcl_ExposeCommand, Tcl_HideCommand - manage multiple Tcl interpreters, aliases and hidden commands

Synopsis

#include <tcl.h>

int

int
Tcl_IsSafe(*interp*)

int
Tcl_MakeSafe(*interp*)

Tcl_Interp *
Tcl_CreateChild(*interp*, *name*, *isSafe*)

Tcl_Interp *
Tcl_CreateSlave(*interp*, *name*, *isSafe*)

Tcl_Interp *
Tcl_GetChild(*interp*, *name*)

Tcl_Interp *
Tcl_GetSlave(*interp*, *name*)

Tcl_Interp *
Tcl_GetParent(*interp*)

Tcl_Interp *
Tcl_GetMaster(*interp*)

int
Tcl_GetInterpPath(*interp*, *childInterp*)

int
Tcl_CreateAlias(*childInterp*, *childCmd*, *targetInterp*, *targetCmd*, *argc*, *argv*)

int
Tcl_CreateAliasObj(*childInterp*, *childCmd*, *targetInterp*, *targetCmd*, *objc*, *objv*)

int
Tcl_GetAlias(*interp*, *childCmd*, *targetInterpPtr*, *targetCmdPtr*, *argcPtr*, *argvPtr*)

int
Tcl_GetAliasObj(*interp*, *childCmd*, *targetInterpPtr*, *targetCmdPtr*, *objcPtr*, *objvPtr*)

int
Tcl_ExposeCommand(*interp*, *hiddenCmdName*, *cmdName*)

int
Tcl_HideCommand(*interp*, *cmdName*, *hiddenCmdName*)

Arguments

Tcl_Interp ****interp* **(in)**
Interpreter in which to execute the specified command.

const char ****name* **(in)**
Name of child interpreter to create or manipulate.

int *isSafe* (in)
If non-zero, a "safe" child that is suitable for running untrusted code is created, otherwise a trusted child is created.

Tcl_Interp *_childInterp_ (in)
Interpreter to use for creating the source command for an alias (see below).

const char *_childCmd_ (in)
Name of source command for alias.

Tcl_Interp *_targetInterp_ (in)
Interpreter that contains the target command for an alias.

const char *_targetCmd_ (in)
Name of target command for alias in *targetInterp*.

int _argc_ (in)
Count of additional arguments to pass to the alias command.

const char *const *_argv_ (in)
Vector of strings, the additional arguments to pass to the alias command. This storage is owned by the caller.

int _objc_ (in)
Count of additional value arguments to pass to the aliased command.

Tcl_Obj **_objv_ (in)
Vector of Tcl_Obj structures, the additional value arguments to pass to the aliased command. This storage is owned by the caller.

Tcl_Interp **_targetInterpPtr_ (in)
Pointer to location to store the address of the interpreter where a target command is defined for an alias.

const char **_targetCmdPtr_ (out)
Pointer to location to store the address of the name of the target command for an alias.

int *_argcPtr_ (out)
Pointer to location to store count of additional arguments to be passed to the alias. The location is in storage owned by the caller.

const char ***_argvPtr_ (out)
Pointer to location to store a vector of strings, the additional arguments to pass to an alias. The location is in storage owned by the caller, the vector of strings is owned by the called function.

int *_objcPtr_ (out)
Pointer to location to store count of additional value arguments to be passed to the alias. The location is in storage owned by the caller.

Tcl_Obj ***_objvPtr_ (out)
Pointer to location to store a vector of Tcl_Obj structures, the additional arguments to pass to an alias command. The location is in storage owned by the caller, the vector of Tcl_Obj structures is owned by the called function.

const char *_cmdName_ (in)
Name of an exposed command to hide or create.

const char *_hiddenCmdName_ (in)
Name under which a hidden command is stored and with which it can be exposed or invoked.

Description

These procedures are intended for access to the multiple interpreter facility from inside C

programs. They enable managing multiple interpreters in a hierarchical relationship, and the management of aliases, commands that when invoked in one interpreter execute a command in another interpreter. The return value for those procedures that return an **int** is either **TCL_OK** or **TCL_ERROR**. If **TCL_ERROR** is returned then the interpreter's result contains an error message.

Tcl_CreateSlave creates a new interpreter as a child of *interp*. It also creates a child command named *childName* in *interp* which allows *interp* to manipulate the new child. If *isSafe* is zero, the command creates a trusted child in which Tcl code has access to all the Tcl commands. If it is **1**, the command creates a "safe" child in which Tcl code has access only to set of Tcl commands defined as "Safe Tcl"; see the manual entry for the Tcl **interp** command for details. If the creation of the new child interpreter failed, **NULL** is returned.

Tcl_CreateChild is a synonym for **Tcl_CreateSlave**.

Tcl_IsSafe returns **1** if *interp* is "safe" (was created with the **TCL_SAFE_INTERPRETER** flag specified), **0** otherwise.

Tcl_MakeSafe marks *interp* as "safe", so that future calls to **Tcl_IsSafe** will return 1. It also removes all known potentially-unsafe core functionality (both commands and variables) from *interp*. However, it cannot know what parts of an extension or application are safe and does not make any attempt to remove those parts, so safety is not guaranteed after calling **Tcl_MakeSafe**. Callers will want to take care with their use of **Tcl_MakeSafe** to avoid false claims of safety. For many situations, **Tcl_CreateSlave** may be a better choice, since it creates interpreters in a known-safe state.

Tcl_GetSlave returns a pointer to a child interpreter of *interp*. The child interpreter is identified by *childName*. If no such child interpreter exists, **NULL** is returned.

Tcl_GetChild is a synonym for **Tcl_GetSlave**.

Tcl_GetMaster returns a pointer to the master interpreter of *interp*. If *interp* has no master (it is a top-level interpreter) then **NULL** is returned.

Tcl_GetParent is a synonym for **Tcl_GetMaster**.

Tcl_GetInterpPath stores in the result of *interp* the relative path between *interp* and *childInterp* ; *childInterp* must be a child of *interp*. If the computation of the relative path succeeds, **TCL_OK** is returned, else **TCL_ERROR** is returned and an error message is stored as the result of *interp*.

Tcl_CreateAlias creates a command named *childCmd* in *childInterp* that when invoked, will cause the command *targetCmd* to be invoked in *targetInterp*. The arguments specified by the strings contained in *argv* are always prepended to any arguments supplied in the invocation of *childCmd* and passed to *targetCmd*. This operation returns **TCL_OK** if it succeeds, or **TCL_ERROR** if it fails; in that case, an error message is left in the value result of *childInterp*. Note that there are no restrictions on the ancestry relationship (as created by **Tcl_CreateSlave**) between *childInterp* and *targetInterp*. Any two interpreters can be used, without any restrictions on how they are related.

Tcl_CreateAliasObj is similar to **Tcl_CreateAlias** except that it takes a vector of values to pass as additional arguments instead of a vector of strings.

Tcl_GetAliasObj returns information in the form of a pointer to a vector of Tcl_Obj structures about an alias *aliasName* in *interp*. Any of the result fields can be **NULL**, in which case the corresponding datum is not returned. If a result field is non-**NULL**, the address indicated is set to the corresponding datum. For example, if *targetCmdPtr* is non-**NULL** it is set to a pointer to the

string containing the name of the target command.

Tcl_GetAlias is similar to **Tcl_GetAliasObj** except that it returns a pointer to a vector of string instead of a vector of Tcl_Obj structures. **Tcl_GetAlias** is deprecated.

Tcl_ExposeCommand moves the command named *hiddenCmdName* from the set of hidden commands to the set of exposed commands, putting it under the name *cmdName*. *HiddenCmdName* must be the name of an existing hidden command, or the operation will return **TCL_ERROR** and leave an error message as the result of *interp*. If an exposed command named *cmdName* already exists, the operation returns **TCL_ERROR** and leaves an error message as the result of *interp*. If the operation succeeds, it returns **TCL_OK**. After executing this command, attempts to use *cmdName* in any script evaluation mechanism will again succeed.

Tcl_HideCommand moves the command named *cmdName* from the set of exposed commands to the set of hidden commands, under the name *hiddenCmdName*. *CmdName* must be the name of an existing exposed command, or the operation will return **TCL_ERROR** and leave an error message as the result of *interp*. Currently both *cmdName* and *hiddenCmdName* must not contain namespace qualifiers, or the operation will return **TCL_ERROR** and leave an error message as the result of *interp*. The *CmdName* will be looked up in the global namespace, and not relative to the current namespace, even if the current namespace is not the global one. If a hidden command whose name is *hiddenCmdName* already exists, the operation also returns **TCL_ERROR** and an error message is left as the result of *interp*. If the operation succeeds, it returns **TCL_OK**. After executing this command, attempts to use *cmdName* in any script evaluation mechanism will fail.

For a description of the Tcl interface to multiple interpreters, see *interp(n)*.

Keywords: CrtAlias, alias, command, exposed commands, hidden commands, interpreter, invoke, parent, child

Copyright (c) 1995-1996 Sun Microsystems, Inc.

CrtChannel

Name

Tcl_CreateChannel, Tcl_GetChannelInstanceData, Tcl_GetChannelType, Tcl_GetChannelName, Tcl_GetChannelHandle, Tcl_GetChannelMode, Tcl_GetChannelBufferSize, Tcl_SetChannelBufferSize, Tcl_NotifyChannel, Tcl_BadChannelOption, Tcl_ChannelName, Tcl_ChannelVersion, Tcl_ChannelBlockModeProc, Tcl_ChannelCloseProc, Tcl_ChannelClose2Proc, Tcl_ChannelInputProc, Tcl_ChannelOutputProc, Tcl_ChannelSeekProc, Tcl_ChannelWideSeekProc, Tcl_ChannelTruncateProc, Tcl_ChannelSetOptionProc, Tcl_ChannelGetOptionProc, Tcl_ChannelWatchProc, Tcl_ChannelGetHandleProc, Tcl_ChannelFlushProc, Tcl_ChannelHandlerProc, Tcl_ChannelThreadActionProc, Tcl_IsChannelShared, Tcl_IsChannelRegistered, Tcl_CutChannel, Tcl_SpliceChannel, Tcl_IsChannelExisting, Tcl_ClearChannelHandlers, Tcl_GetChannelThread, Tcl_ChannelBuffered - procedures for creating and manipulating channels

Synopsis

#include <tcl.h>

Tcl_Channel
Tcl_CreateChannel(*typePtr*, *channelName*, *instanceData*, *mask*)

ClientData
Tcl_GetChannelInstanceData(*channel*)

const Tcl_ChannelType *
Tcl_GetChannelType(*channel*)

const char *
Tcl_GetChannelName(*channel*)

int
Tcl_GetChannelHandle(*channel*, *direction*, *handlePtr*)

Tcl_ThreadId
Tcl_GetChannelThread(*channel*)

int
Tcl_GetChannelMode(*channel*)

int
Tcl_GetChannelBufferSize(*channel*)

Tcl_SetChannelBufferSize(*channel*, *size*)

Tcl_NotifyChannel(*channel*, *mask*)

int
Tcl_BadChannelOption(*interp*, *optionName*, *optionList*)

int
Tcl_IsChannelShared(*channel*)

int
Tcl_IsChannelRegistered(*interp*, *channel*)

int
Tcl_IsChannelExisting(*channelName*)

void
Tcl_CutChannel(*channel*)

void
Tcl_SpliceChannel(*channel*)

void
Tcl_ClearChannelHandlers(*channel*)

int
Tcl_ChannelBuffered(*channel*)

const char *
Tcl_ChannelName(*typePtr*)

Tcl_ChannelTypeVersion
Tcl_ChannelVersion(*typePtr*)

Tcl_DriverBlockModeProc *
Tcl_ChannelBlockModeProc(*typePtr*)

Tcl_DriverCloseProc *
Tcl_ChannelCloseProc(*typePtr*)

Tcl_DriverClose2Proc *

Tcl_ChannelClose2Proc(*typePtr*)

Tcl_DriverInputProc *
Tcl_ChannelInputProc(*typePtr*)

Tcl_DriverOutputProc *
Tcl_ChannelOutputProc(*typePtr*)

Tcl_DriverSeekProc *
Tcl_ChannelSeekProc(*typePtr*)

Tcl_DriverWideSeekProc *
Tcl_ChannelWideSeekProc(*typePtr*)

Tcl_DriverThreadActionProc *
Tcl_ChannelThreadActionProc(*typePtr*)

Tcl_DriverTruncateProc *
Tcl_ChannelTruncateProc(*typePtr*)

Tcl_DriverSetOptionProc *
Tcl_ChannelSetOptionProc(*typePtr*)

Tcl_DriverGetOptionProc *
Tcl_ChannelGetOptionProc(*typePtr*)

Tcl_DriverWatchProc *
Tcl_ChannelWatchProc(*typePtr*)

Tcl_DriverGetHandleProc *
Tcl_ChannelGetHandleProc(*typePtr*)

Tcl_DriverFlushProc *
Tcl_ChannelFlushProc(*typePtr*)

Tcl_DriverHandlerProc *
Tcl_ChannelHandlerProc(*typePtr*)

Arguments

const Tcl_ChannelType *_typePtr_ (in)
Points to a structure containing the addresses of procedures that can be called to perform I/O and other functions on the channel.

const char *_channelName_ (in)
The name of this channel, such as **file3**; must not be in use by any other channel. Can be NULL, in which case the channel is created without a name. If the created channel is assigned to one of the standard channels (**stdin**, **stdout** or **stderr**), the assigned channel name will be the name of the standard channel.

ClientData _instanceData_ (in)
Arbitrary one-word value to be associated with this channel. This value is passed to procedures in *typePtr* when they are invoked.

int _mask_ (in)
OR-ed combination of **TCL_READABLE** and **TCL_WRITABLE** to indicate whether a channel is readable and writable.

Tcl_Channel _channel_ (in)
The channel to operate on.

int *direction* (in)
TCL_READABLE means the input handle is wanted; **TCL_WRITABLE** means the output handle is wanted.

ClientData **handlePtr* (out)
Points to the location where the desired OS-specific handle should be stored.

int *size* (in)
The size, in bytes, of buffers to allocate in this channel.

int *mask* (in)
An OR-ed combination of **TCL_READABLE**, **TCL_WRITABLE** and **TCL_EXCEPTION** that indicates events that have occurred on this channel.

Tcl_Interp **interp* (in)
Current interpreter. (can be NULL)

const char **optionName* (in)
Name of the invalid option.

const char **optionList* (in)
Specific options list (space separated words, without "-") to append to the standard generic options list. Can be NULL for generic options error message only.

Description

Tcl uses a two-layered channel architecture. It provides a generic upper layer to enable C and Tcl programs to perform input and output using the same APIs for a variety of files, devices, sockets etc. The generic C APIs are described in the manual entry for **Tcl_OpenFileChannel**.

The lower layer provides type-specific channel drivers for each type of device supported on each platform. This manual entry describes the C APIs used to communicate between the generic layer and the type-specific channel drivers. It also explains how new types of channels can be added by providing new channel drivers.

Channel drivers consist of a number of components: First, each channel driver provides a **Tcl_ChannelType** structure containing pointers to functions implementing the various operations used by the generic layer to communicate with the channel driver. The **Tcl_ChannelType** structure and the functions referenced by it are described in the section **TCL_CHANNELTYPE**, below.

Second, channel drivers usually provide a Tcl command to create instances of that type of channel. For example, the Tcl **open** command creates channels that use the file and command channel drivers, and the Tcl **socket** command creates channels that use TCP sockets for network communication.

Third, a channel driver optionally provides a C function to open channel instances of that type. For example, **Tcl_OpenFileChannel** opens a channel that uses the file channel driver, and **Tcl_OpenTcpClient** opens a channel that uses the TCP network protocol. These creation functions typically use **Tcl_CreateChannel** internally to open the channel.

To add a new type of channel you must implement a C API or a Tcl command that opens a channel by invoking **Tcl_CreateChannel**. When your driver calls **Tcl_CreateChannel** it passes in a **Tcl_ChannelType** structure describing the driver's I/O procedures. The generic layer will then invoke the functions referenced in that structure to perform operations on the channel.

Tcl_CreateChannel opens a new channel and associates the supplied *typePtr* and *instanceData*

with it. The channel is opened in the mode indicated by *mask*. For a discussion of channel drivers, their operations and the **Tcl_ChannelType** structure, see the section **TCL_CHANNELTYPE**, below.

Tcl_CreateChannel interacts with the code managing the standard channels. Once a standard channel was initialized either through a call to **Tcl_GetStdChannel** or a call to **Tcl_SetStdChannel** closing this standard channel will cause the next call to **Tcl_CreateChannel** to make the new channel the new standard channel too. See **Tcl_StandardChannels** for a general treatise about standard channels and the behavior of the Tcl library with regard to them.

Tcl_GetChannelInstanceData returns the instance data associated with the channel in *channel* . This is the same as the *instanceData* argument in the call to **Tcl_CreateChannel** that created this channel.

Tcl_GetChannelType returns a pointer to the **Tcl_ChannelType** structure used by the channel in the *channel* argument. This is the same as the *typePtr* argument in the call to **Tcl_CreateChannel** that created this channel.

Tcl_GetChannelName returns a string containing the name associated with the channel, or NULL if the *channelName* argument to **Tcl_CreateChannel** was NULL.

Tcl_GetChannelHandle places the OS-specific device handle associated with *channel* for the given *direction* in the location specified by *handlePtr* and returns **TCL_OK**. If the channel does not have a device handle for the specified direction, then **TCL_ERROR** is returned instead. Different channel drivers will return different types of handle. Refer to the manual entries for each driver to determine what type of handle is returned.

Tcl_GetChannelThread returns the id of the thread currently managing the specified *channel*. This allows channel drivers to send their file events to the correct event queue even for a multi-threaded core.

Tcl_GetChannelMode returns an OR-ed combination of **TCL_READABLE** and **TCL_WRITABLE**, indicating whether the channel is open for input and output.

Tcl_GetChannelBufferSize returns the size, in bytes, of buffers allocated to store input or output in *channel*. If the value was not set by a previous call to **Tcl_SetChannelBufferSize**, described below, then the default value of 4096 is returned.

Tcl_SetChannelBufferSize sets the size, in bytes, of buffers that will be allocated in subsequent operations on the channel to store input or output. The *size* argument should be between one and one million, allowing buffers of one byte to one million bytes. If *size* is outside this range, **Tcl_SetChannelBufferSize** sets the buffer size to 4096.

Tcl_NotifyChannel is called by a channel driver to indicate to the generic layer that the events specified by *mask* have occurred on the channel. Channel drivers are responsible for invoking this function whenever the channel handlers need to be called for the channel (or other pending tasks like a write flush should be performed). See **WATCHPROC** below for more details.

Tcl_BadChannelOption is called from driver specific *setOptionProc* or *getOptionProc* to generate a complete error message.

Tcl_ChannelBuffered returns the number of bytes of input currently buffered in the internal buffer (push back area) of the channel itself. It does not report about the data in the overall buffers for the stack of channels the supplied channel is part of.

Tcl_IsChannelShared checks the refcount of the specified *channel* and returns whether the *channel* was shared among multiple interpreters (result == 1) or not (result == 0).

Tcl_IsChannelRegistered checks whether the specified *channel* is registered in the given *interp*reter (result == 1) or not (result == 0).

Tcl_IsChannelExisting checks whether a channel with the specified name is registered in the (thread)-global list of all channels (result == 1) or not (result == 0).

Tcl_CutChannel removes the specified *channel* from the (thread)global list of all channels (of the current thread). Application to a channel still registered in some interpreter is not allowed. Also notifies the driver if the **Tcl_ChannelType** version is **TCL_CHANNEL_VERSION_4** (or higher), and **Tcl_DriverThreadActionProc** is defined for it.

Tcl_SpliceChannel adds the specified *channel* to the (thread)global list of all channels (of the current thread). Application to a channel registered in some interpreter is not allowed. Also notifies the driver if the **Tcl_ChannelType** version is **TCL_CHANNEL_VERSION_4** (or higher), and **Tcl_DriverThreadActionProc** is defined for it.

Tcl_ClearChannelHandlers removes all channel handlers and event scripts associated with the specified *channel*, thus shutting down all event processing for this channel.

Tcl_channeltype

A channel driver provides a **Tcl_ChannelType** structure that contains pointers to functions that implement the various operations on a channel; these operations are invoked as needed by the generic layer. The structure was versioned starting in Tcl 8.3.2/8.4 to correct a problem with stacked channel drivers. See the **OLD CHANNEL TYPES** section below for details about the old structure.

The **Tcl_ChannelType** structure contains the following fields:

```
typedef struct Tcl_ChannelType {
        const char *typeName;
        Tcl_ChannelTypeVersion version;
        Tcl_DriverCloseProc *closeProc;
        Tcl_DriverInputProc *inputProc;
        Tcl_DriverOutputProc *outputProc;
        Tcl_DriverSeekProc *seekProc;
        Tcl_DriverSetOptionProc *setOptionProc;
        Tcl_DriverGetOptionProc *getOptionProc;
        Tcl_DriverWatchProc *watchProc;
        Tcl_DriverGetHandleProc *getHandleProc;
        Tcl_DriverClose2Proc *close2Proc;
        Tcl_DriverBlockModeProc *blockModeProc;
        Tcl_DriverFlushProc *flushProc;
        Tcl_DriverHandlerProc *handlerProc;
        Tcl_DriverWideSeekProc *wideSeekProc;
        Tcl_DriverThreadActionProc *threadActionProc;
        Tcl_DriverTruncateProc *truncateProc;
} Tcl_ChannelType;
```

It is not necessary to provide implementations for all channel operations. Those which are not

necessary may be set to NULL in the struct: *blockModeProc*, *seekProc*, *setOptionProc*, *getOptionProc*, *getHandleProc*, and *close2Proc*, in addition to *flushProc*, *handlerProc*, *threadActionProc*, and *truncateProc*. Other functions that cannot be implemented in a meaningful way should return **EINVAL** when called, to indicate that the operations they represent are not available. Also note that *wideSeekProc* can be NULL if *seekProc* is.

The user should only use the above structure for **Tcl_ChannelType** instantiation. When referencing fields in a **Tcl_ChannelType** structure, the following functions should be used to obtain the values: **Tcl_ChannelName**, **Tcl_ChannelVersion**, **Tcl_ChannelBlockModeProc**, **Tcl_ChannelCloseProc**, **Tcl_ChannelClose2Proc**, **Tcl_ChannelInputProc**, **Tcl_ChannelOutputProc**, **Tcl_ChannelSeekProc**, **Tcl_ChannelWideSeekProc**, **Tcl_ChannelThreadActionProc**, **Tcl_ChannelTruncateProc**, **Tcl_ChannelSetOptionProc**, **Tcl_ChannelGetOptionProc**, **Tcl_ChannelWatchProc**, **Tcl_ChannelGetHandleProc**, **Tcl_ChannelFlushProc**, or **Tcl_ChannelHandlerProc**.

The change to the structures was made in such a way that standard channel types are binary compatible. However, channel types that use stacked channels (i.e. TLS, Trf) have new versions to correspond to the above change since the previous code for stacked channels had problems.

Typename

The *typeName* field contains a null-terminated string that identifies the type of the device implemented by this driver, e.g. **file** or **socket**.

This value can be retrieved with **Tcl_ChannelName**, which returns a pointer to the string.

Version

The *version* field should be set to the version of the structure that you require. **TCL_CHANNEL_VERSION_2** is the minimum recommended. **TCL_CHANNEL_VERSION_3** must be set to specify the *wideSeekProc* member. **TCL_CHANNEL_VERSION_4** must be set to specify the *threadActionProc* member (includes *wideSeekProc*). **TCL_CHANNEL_VERSION_5** must be set to specify the *truncateProc* members (includes *wideSeekProc* and *threadActionProc*). If it is not set to any of these, then this **Tcl_ChannelType** is assumed to have the original structure. See **OLD CHANNEL TYPES** for more details. While Tcl will recognize and function with either structures, stacked channels must be of at least **TCL_CHANNEL_VERSION_2** to function correctly.

This value can be retrieved with **Tcl_ChannelVersion**, which returns one of **TCL_CHANNEL_VERSION_5**, **TCL_CHANNEL_VERSION_4**, **TCL_CHANNEL_VERSION_3**, **TCL_CHANNEL_VERSION_2** or **TCL_CHANNEL_VERSION_1**.

Blockmodeproc

The *blockModeProc* field contains the address of a function called by the generic layer to set blocking and nonblocking mode on the device. *BlockModeProc* should match the following prototype:

```
typedef int Tcl_DriverBlockModeProc(
        ClientData instanceData,
        int mode);
```

The *instanceData* is the same as the value passed to **Tcl_CreateChannel** when this channel was created. The *mode* argument is either **TCL_MODE_BLOCKING** or

TCL_MODE_NONBLOCKING to set the device into blocking or nonblocking mode. The function should return zero if the operation was successful, or a nonzero POSIX error code if the operation failed.

If the operation is successful, the function can modify the supplied *instanceData* to record that the channel entered blocking or nonblocking mode and to implement the blocking or nonblocking behavior. For some device types, the blocking and nonblocking behavior can be implemented by the underlying operating system; for other device types, the behavior must be emulated in the channel driver.

This value can be retrieved with **Tcl_ChannelBlockModeProc**, which returns a pointer to the function.

A channel driver **not** supplying a *blockModeProc* has to be very, very careful. It has to tell the generic layer exactly which blocking mode is acceptable to it, and should this also document for the user so that the blocking mode of the channel is not changed to an unacceptable value. Any confusion here may lead the interpreter into a (spurious and difficult to find) deadlock.

Closeproc and close2proc

The *closeProc* field contains the address of a function called by the generic layer to clean up driver-related information when the channel is closed. *CloseProc* must match the following prototype:

```
typedef int Tcl_DriverCloseProc(
        ClientData instanceData,
        Tcl_Interp *interp);
```

The *instanceData* argument is the same as the value provided to **Tcl_CreateChannel** when the channel was created. The function should release any storage maintained by the channel driver for this channel, and close the input and output devices encapsulated by this channel. All queued output will have been flushed to the device before this function is called, and no further driver operations will be invoked on this instance after calling the *closeProc*. If the close operation is successful, the procedure should return zero; otherwise it should return a nonzero POSIX error code. In addition, if an error occurs and *interp* is not NULL, the procedure should store an error message in the interpreter's result.

Alternatively, channels that support closing the read and write sides independently may set *closeProc* to **TCL_CLOSE2PROC** and set *close2Proc* to the address of a function that matches the following prototype:

```
typedef int Tcl_DriverClose2Proc(
        ClientData instanceData,
        Tcl_Interp *interp,
        int flags);
```

The *close2Proc* will be called with *flags* set to an OR'ed combination of **TCL_CLOSE_READ** or **TCL_CLOSE_WRITE** to indicate that the driver should close the read and/or write side of the channel. The channel driver may be invoked to perform additional operations on the channel after *close2Proc* is called to close one or both sides of the channel. If *flags* is **0** (zero), the driver should close the channel in the manner described above for *closeProc*. No further operations will be invoked on this instance after *close2Proc* is called with all flags cleared. In all cases, the

close2Proc function should return zero if the close operation was successful; otherwise it should return a nonzero POSIX error code. In addition, if an error occurs and *interp* is not NULL, the procedure should store an error message in the interpreter's result.

The *closeProc* and *close2Proc* values can be retrieved with **Tcl_ChannelCloseProc** or **Tcl_ChannelClose2Proc**, which return a pointer to the respective function.

Inputproc

The *inputProc* field contains the address of a function called by the generic layer to read data from the file or device and store it in an internal buffer. *InputProc* must match the following prototype:

```
typedef int Tcl_DriverInputProc(
        ClientData instanceData,
        char *buf,
        int bufSize,
        int *errorCodePtr);
```

InstanceData is the same as the value passed to **Tcl_CreateChannel** when the channel was created. The *buf* argument points to an array of bytes in which to store input from the device, and the *bufSize* argument indicates how many bytes are available at *buf*.

The *errorCodePtr* argument points to an integer variable provided by the generic layer. If an error occurs, the function should set the variable to a POSIX error code that identifies the error that occurred.

The function should read data from the input device encapsulated by the channel and store it at *buf*. On success, the function should return a nonnegative integer indicating how many bytes were read from the input device and stored at *buf*. On error, the function should return -1. If an error occurs after some data has been read from the device, that data is lost.

If *inputProc* can determine that the input device has some data available but less than requested by the *bufSize* argument, the function should only attempt to read as much data as is available and return without blocking. If the input device has no data available whatsoever and the channel is in nonblocking mode, the function should return an **EAGAIN** error. If the input device has no data available whatsoever and the channel is in blocking mode, the function should block for the shortest possible time until at least one byte of data can be read from the device; then, it should return as much data as it can read without blocking.

This value can be retrieved with **Tcl_ChannelInputProc**, which returns a pointer to the function.

Outputproc

The *outputProc* field contains the address of a function called by the generic layer to transfer data from an internal buffer to the output device. *OutputProc* must match the following prototype:

```
typedef int Tcl_DriverOutputProc(
        ClientData instanceData,
        const char *buf,
        int toWrite,
        int *errorCodePtr);
```

InstanceData is the same as the value passed to **Tcl_CreateChannel** when the channel was created. The *buf* argument contains an array of bytes to be written to the device, and the *toWrite* argument indicates how many bytes are to be written from the *buf* argument.

The *errorCodePtr* argument points to an integer variable provided by the generic layer. If an error occurs, the function should set this variable to a POSIX error code that identifies the error.

The function should write the data at *buf* to the output device encapsulated by the channel. On success, the function should return a nonnegative integer indicating how many bytes were written to the output device. The return value is normally the same as *toWrite*, but may be less in some cases such as if the output operation is interrupted by a signal. If an error occurs the function should return -1. In case of error, some data may have been written to the device.

If the channel is nonblocking and the output device is unable to absorb any data whatsoever, the function should return -1 with an **EAGAIN** error without writing any data.

This value can be retrieved with **Tcl_ChannelOutputProc**, which returns a pointer to the function.

Seekproc and wideseekproc

The *seekProc* field contains the address of a function called by the generic layer to move the access point at which subsequent input or output operations will be applied. *SeekProc* must match the following prototype:

```
typedef int Tcl_DriverSeekProc(
        ClientData instanceData,
        long offset,
        int seekMode,
        int *errorCodePtr);
```

The *instanceData* argument is the same as the value given to **Tcl_CreateChannel** when this channel was created. *Offset* and *seekMode* have the same meaning as for the **Tcl_Seek** procedure (described in the manual entry for **Tcl_OpenFileChannel**).

The *errorCodePtr* argument points to an integer variable provided by the generic layer for returning **errno** values from the function. The function should set this variable to a POSIX error code if an error occurs. The function should store an **EINVAL** error code if the channel type does not implement seeking.

The return value is the new access point or -1 in case of error. If an error occurred, the function should not move the access point.

If there is a non-NULL *seekProc* field, the *wideSeekProc* field may contain the address of an alternative function to use which handles wide (i.e. larger than 32-bit) offsets, so allowing seeks within files larger than 2GB. The *wideSeekProc* will be called in preference to the *seekProc*, but both must be defined if the *wideSeekProc* is defined. *WideSeekProc* must match the following prototype:

```
typedef Tcl_WideInt Tcl_DriverWideSeekProc(
        ClientData instanceData,
        Tcl_WideInt offset,
        int seekMode,
        int *errorCodePtr);
```

The arguments and return values mean the same thing as with *seekProc* above, except that the type of offsets and the return type are different.

The *seekProc* value can be retrieved with **Tcl_ChannelSeekProc**, which returns a pointer to the function, and similarly the *wideSeekProc* can be retrieved with **Tcl_ChannelWideSeekProc**.

Setoptionproc

The *setOptionProc* field contains the address of a function called by the generic layer to set a channel type specific option on a channel. *setOptionProc* must match the following prototype:

```
typedef int Tcl_DriverSetOptionProc(
        ClientData instanceData,
        Tcl_Interp *interp,
        const char *optionName,
        const char *newValue);
```

optionName is the name of an option to set, and *newValue* is the new value for that option, as a string. The *instanceData* is the same as the value given to **Tcl_CreateChannel** when this channel was created. The function should do whatever channel type specific action is required to implement the new value of the option.

Some options are handled by the generic code and this function is never called to set them, e.g. **-blockmode**. Other options are specific to each channel type and the *setOptionProc* procedure of the channel driver will get called to implement them. The *setOptionProc* field can be NULL, which indicates that this channel type supports no type specific options.

If the option value is successfully modified to the new value, the function returns **TCL_OK**. It should call **Tcl_BadChannelOption** which itself returns **TCL_ERROR** if the *optionName* is unrecognized. If *newValue* specifies a value for the option that is not supported or if a system call error occurs, the function should leave an error message in the result of *interp* if *interp* is not NULL. The function should also call **Tcl_SetErrno** to store an appropriate POSIX error code.

This value can be retrieved with **Tcl_ChannelSetOptionProc**, which returns a pointer to the function.

Getoptionproc

The *getOptionProc* field contains the address of a function called by the generic layer to get the value of a channel type specific option on a channel. *getOptionProc* must match the following prototype:

```
typedef int Tcl_DriverGetOptionProc(
        ClientData instanceData,
        Tcl_Interp *interp,
        const char *optionName,
        Tcl_DString *optionValue);
```

OptionName is the name of an option supported by this type of channel. If the option name is not NULL, the function stores its current value, as a string, in the Tcl dynamic string *optionValue*. If *optionName* is NULL, the function stores in *optionValue* an alternating list of all supported options and their current values. On success, the function returns **TCL_OK**. It should call **Tcl_BadChannelOption** which itself returns **TCL_ERROR** if the *optionName* is unrecognized. If a system call error occurs, the function should leave an error message in the result of *interp* if

interp is not NULL. The function should also call **Tcl_SetErrno** to store an appropriate POSIX error code.

Some options are handled by the generic code and this function is never called to retrieve their value, e.g. **-blockmode**. Other options are specific to each channel type and the *getOptionProc* procedure of the channel driver will get called to implement them. The *getOptionProc* field can be NULL, which indicates that this channel type supports no type specific options.

This value can be retrieved with **Tcl_ChannelGetOptionProc**, which returns a pointer to the function.

Watchproc

The *watchProc* field contains the address of a function called by the generic layer to initialize the event notification mechanism to notice events of interest on this channel. *WatchProc* should match the following prototype:

```
typedef void Tcl_DriverWatchProc(
        ClientData instanceData,
        int mask);
```

The *instanceData* is the same as the value passed to **Tcl_CreateChannel** when this channel was created. The *mask* argument is an OR-ed combination of **TCL_READABLE**, **TCL_WRITABLE** and **TCL_EXCEPTION**; it indicates events the caller is interested in noticing on this channel.

The function should initialize device type specific mechanisms to notice when an event of interest is present on the channel. When one or more of the designated events occurs on the channel, the channel driver is responsible for calling **Tcl_NotifyChannel** to inform the generic channel module. The driver should take care not to starve other channel drivers or sources of callbacks by invoking Tcl_NotifyChannel too frequently. Fairness can be insured by using the Tcl event queue to allow the channel event to be scheduled in sequence with other events. See the description of **Tcl_QueueEvent** for details on how to queue an event.

This value can be retrieved with **Tcl_ChannelWatchProc**, which returns a pointer to the function.

Gethandleproc

The *getHandleProc* field contains the address of a function called by the generic layer to retrieve a device-specific handle from the channel. *GetHandleProc* should match the following prototype:

```
typedef int Tcl_DriverGetHandleProc(
        ClientData instanceData,
        int direction,
        ClientData *handlePtr);
```

InstanceData is the same as the value passed to **Tcl_CreateChannel** when this channel was created. The *direction* argument is either **TCL_READABLE** to retrieve the handle used for input, or **TCL_WRITABLE** to retrieve the handle used for output.

If the channel implementation has device-specific handles, the function should retrieve the appropriate handle associated with the channel, according the *direction* argument. The handle should be stored in the location referred to by *handlePtr*, and **TCL_OK** should be returned. If the channel is not open for the specified direction, or if the channel implementation does not use

device handles, the function should return **TCL_ERROR**.

This value can be retrieved with **Tcl_ChannelGetHandleProc**, which returns a pointer to the function.

Flushproc

The *flushProc* field is currently reserved for future use. It should be set to NULL. *FlushProc* should match the following prototype:

```
typedef int Tcl_DriverFlushProc(
        ClientData instanceData);
```

This value can be retrieved with **Tcl_ChannelFlushProc**, which returns a pointer to the function.

Handlerproc

The *handlerProc* field contains the address of a function called by the generic layer to notify the channel that an event occurred. It should be defined for stacked channel drivers that wish to be notified of events that occur on the underlying (stacked) channel. *HandlerProc* should match the following prototype:

```
typedef int Tcl_DriverHandlerProc(
        ClientData instanceData,
        int interestMask);
```

InstanceData is the same as the value passed to **Tcl_CreateChannel** when this channel was created. The *interestMask* is an OR-ed combination of **TCL_READABLE** or **TCL_WRITABLE**; it indicates what type of event occurred on this channel.

This value can be retrieved with **Tcl_ChannelHandlerProc**, which returns a pointer to the function.

Threadactionproc

The *threadActionProc* field contains the address of the function called by the generic layer when a channel is created, closed, or going to move to a different thread, i.e. whenever thread-specific driver state might have to initialized or updated. It can be NULL. The action *TCL_CHANNEL_THREAD_REMOVE* is used to notify the driver that it should update or remove any thread-specific data it might be maintaining for the channel.

The action *TCL_CHANNEL_THREAD_INSERT* is used to notify the driver that it should update or initialize any thread-specific data it might be maintaining using the calling thread as the associate. See **Tcl_CutChannel** and **Tcl_SpliceChannel** for more detail.

```
typedef void Tcl_DriverThreadActionProc(
        ClientData instanceData,
        int action);
```

InstanceData is the same as the value passed to **Tcl_CreateChannel** when this channel was created.

These values can be retrieved with **Tcl_ChannelThreadActionProc**, which returns a pointer to the function.

Truncateproc

The *truncateProc* field contains the address of the function called by the generic layer when a channel is truncated to some length. It can be NULL.

```
typedef int Tcl_DriverTruncateProc(
        ClientData instanceData,
        Tcl_WideInt length);
```

InstanceData is the same as the value passed to **Tcl_CreateChannel** when this channel was created, and *length* is the new length of the underlying file, which should not be negative. The result should be 0 on success or an errno code (suitable for use with **Tcl_SetErrno**) on failure.

These values can be retrieved with **Tcl_ChannelTruncateProc**, which returns a pointer to the function.

Tcl_badchanneloption

This procedure generates a "bad option" error message in an (optional) interpreter. It is used by channel drivers when an invalid Set/Get option is requested. Its purpose is to concatenate the generic options list to the specific ones and factorize the generic options error message string.

It always returns **TCL_ERROR**

An error message is generated in *interp*'s result value to indicate that a command was invoked with a bad option. The message has the form

```
    bad option "blah": should be one of
    <...generic options...>+<...specific options...>
```

so you get for instance:

```
    bad option "-blah": should be one of -blocking,
    -buffering, -buffersize, -eofchar, -translation,
    -peername, or -sockname
```

when called with *optionList* equal to "peername sockname"

"blah" is the *optionName* argument and "<specific options>" is a space separated list of specific option words. The function takes good care of inserting minus signs before each option, commas after, and an "or" before the last option.

Old channel types

The original (8.3.1 and below) **Tcl_ChannelType** structure contains the following fields:

```
typedef struct Tcl_ChannelType {
    const char *typeName;
    Tcl_DriverBlockModeProc *blockModeProc;
    Tcl_DriverCloseProc *closeProc;
    Tcl_DriverInputProc *inputProc;
    Tcl_DriverOutputProc *outputProc;
    Tcl_DriverSeekProc *seekProc;
    Tcl_DriverSetOptionProc *setOptionProc;
    Tcl_DriverGetOptionProc *getOptionProc;
    Tcl_DriverWatchProc *watchProc;
```

```
    Tcl_DriverGetHandleProc *getHandleProc;
    Tcl_DriverClose2Proc *close2Proc;
} Tcl_ChannelType;
```

It is still possible to create channel with the above structure. The internal channel code will determine the version. It is imperative to use the new **Tcl_ChannelType** structure if you are creating a stacked channel driver, due to problems with the earlier stacked channel implementation (in 8.2.0 to 8.3.1).

Prior to 8.4.0 (i.e. during the later releases of 8.3 and early part of the 8.4 development cycle) the **Tcl_ChannelType** structure contained the following fields:

```
typedef struct Tcl_ChannelType {
    const char *typeName;
    Tcl_ChannelTypeVersion version;
    Tcl_DriverCloseProc *closeProc;
    Tcl_DriverInputProc *inputProc;
    Tcl_DriverOutputProc *outputProc;
    Tcl_DriverSeekProc *seekProc;
    Tcl_DriverSetOptionProc *setOptionProc;
    Tcl_DriverGetOptionProc *getOptionProc;
    Tcl_DriverWatchProc *watchProc;
    Tcl_DriverGetHandleProc *getHandleProc;
    Tcl_DriverClose2Proc *close2Proc;
    Tcl_DriverBlockModeProc *blockModeProc;
    Tcl_DriverFlushProc *flushProc;
    Tcl_DriverHandlerProc *handlerProc;
    Tcl_DriverTruncateProc *truncateProc;
} Tcl_ChannelType;
```

When the above structure is registered as a channel type, the *version* field should always be **TCL_CHANNEL_VERSION_2**.

Keywords: CrtChannel, blocking, channel driver, channel registration, channel type, nonblocking

Copyright (c) 1996-1997 Sun Microsystems, Inc.
Copyright (c) 1997-2000 Ajuba Solutions.

CrtChnlHdlr

Name

Tcl_CreateChannelHandler, Tcl_DeleteChannelHandler - call a procedure when a channel becomes readable or writable

Synopsis

.nf
#include <tcl.h>

void
Tcl_CreateChannelHandler(*channel*, *mask*, *proc*, *clientData*)

void
Tcl_DeleteChannelHandler(*channel*, *proc*, *clientData*)

Arguments

Tcl_Channel *channel* (in)
Tcl channel such as returned by **Tcl_CreateChannel**.

int *mask* (in)
Conditions under which *proc* should be called: OR-ed combination of **TCL_READABLE**, **TCL_WRITABLE** and **TCL_EXCEPTION**. Specify a zero value to temporarily disable an existing handler.

Tcl_FileProc **proc* (in)
Procedure to invoke whenever the channel indicated by *channel* meets the conditions specified by *mask*.

ClientData *clientData* (in)
Arbitrary one-word value to pass to *proc*.

Description

Tcl_CreateChannelHandler arranges for *proc* to be called in the future whenever input or output becomes possible on the channel identified by *channel*, or whenever an exceptional condition exists for *channel*. The conditions of interest under which *proc* will be invoked are specified by the *mask* argument. See the manual entry for **fileevent** for a precise description of what it means for a channel to be readable or writable. *Proc* must conform to the following prototype:

```
typedef void Tcl_ChannelProc(
        ClientData clientData,
        int mask);
```

The *clientData* argument is the same as the value passed to **Tcl_CreateChannelHandler** when the handler was created. Typically, *clientData* points to a data structure containing application-specific information about the channel. *Mask* is an integer mask indicating which of the requested conditions actually exists for the channel; it will contain a subset of the bits from the *mask* argument to **Tcl_CreateChannelHandler** when the handler was created.

Each channel handler is identified by a unique combination of *channel*, *proc* and *clientData*. There may be many handlers for a given channel as long as they do not have the same *channel*, *proc*, and *clientData*. If **Tcl_CreateChannelHandler** is invoked when there is already a handler for *channel*, *proc*, and *clientData*, then no new handler is created; instead, the *mask* is changed for the existing handler.

Tcl_DeleteChannelHandler deletes a channel handler identified by *channel*, *proc* and *clientData*; if no such handler exists, the call has no effect.

Channel handlers are invoked via the Tcl event mechanism, so they are only useful in applications that are event-driven. Note also that the conditions specified in the *mask* argument to *proc* may no longer exist when *proc* is invoked: for example, if there are two handlers for **TCL_READABLE** on the same channel, the first handler could consume all of the available input so that the channel is no longer readable when the second handler is invoked. For this reason it may be useful to use nonblocking I/O on channels for which there are event handlers.

Keywords: CrtChnlHdlr, blocking, callback, channel, events, handler, nonblocking

Copyright (c) 1996 Sun Microsystems, Inc.

CrtCloseHdlr

Name

Tcl_CreateCloseHandler, Tcl_DeleteCloseHandler - arrange for callbacks when channels are closed

Synopsis

#include <tcl.h>

void
Tcl_CreateCloseHandler(*channel*, *proc*, *clientData*)

void
Tcl_DeleteCloseHandler(*channel*, *proc*, *clientData*)

Arguments

Tcl_Channel *channel* (in)
The channel for which to create or delete a close callback.

Tcl_CloseProc **proc* (in)
The procedure to call as the callback.

ClientData *clientData* (in)
Arbitrary one-word value to pass to *proc*.

Description

Tcl_CreateCloseHandler arranges for *proc* to be called when *channel* is closed with **Tcl_Close** or **Tcl_UnregisterChannel**, or using the Tcl **close** command. *Proc* should match the following prototype:

```
typedef void Tcl_CloseProc(
        ClientData clientData);
```

The *clientData* is the same as the value provided in the call to **Tcl_CreateCloseHandler**.

Tcl_DeleteCloseHandler removes a close callback for *channel*. The *proc* and *clientData* identify which close callback to remove; **Tcl_DeleteCloseHandler** does nothing if its *proc* and *clientData* arguments do not match the *proc* and *clientData* for a close handler for *channel*.

Keywords: CrtCloseHdlr, callback, channel closing

Copyright (c) 1994-1996 Sun Microsystems, Inc.

CrtCommand

Name

Tcl_CreateCommand - implement new commands in C

Synopsis

#include <tcl.h>

Tcl_Command
Tcl_CreateCommand(*interp*, *cmdName*, *proc*, *clientData*, *deleteProc*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter in which to create new command.

const char *_cmdName_ (in)
Name of command.

Tcl_CmdProc *_proc_ (in)
Implementation of new command: *proc* will be called whenever *cmdName* is invoked as a command.

ClientData _clientData_ (in)
Arbitrary one-word value to pass to *proc* and *deleteProc*.

Tcl_CmdDeleteProc *_deleteProc_ (in)
Procedure to call before *cmdName* is deleted from the interpreter; allows for command-specific cleanup. If NULL, then no procedure is called before the command is deleted.

Description

Tcl_CreateCommand defines a new command in *interp* and associates it with procedure *proc* such that whenever *cmdName* is invoked as a Tcl command (via a call to **Tcl_Eval**) the Tcl interpreter will call *proc* to process the command. It differs from **Tcl_CreateObjCommand** in that a new string-based command is defined; that is, a command procedure is defined that takes an array of argument strings instead of values. The value-based command procedures registered by **Tcl_CreateObjCommand** can execute significantly faster than the string-based command procedures defined by **Tcl_CreateCommand**. This is because they take Tcl values as arguments and those values can retain an internal representation that can be manipulated more efficiently. Also, Tcl's interpreter now uses values internally. In order to invoke a string-based command procedure registered by **Tcl_CreateCommand**, it must generate and fetch a string representation from each argument value before the call. New commands should be defined using **Tcl_CreateObjCommand**. We support **Tcl_CreateCommand** for backwards compatibility.

The procedures **Tcl_DeleteCommand**, **Tcl_GetCommandInfo**, and **Tcl_SetCommandInfo** are used in conjunction with **Tcl_CreateCommand**.

Tcl_CreateCommand will delete an existing command *cmdName*, if one is already associated with the interpreter. It returns a token that may be used to refer to the command in subsequent calls to **Tcl_GetCommandName**. If *cmdName* contains any **::** namespace qualifiers, then the command is added to the specified namespace; otherwise the command is added to the global namespace. If **Tcl_CreateCommand** is called for an interpreter that is in the process of being deleted, then it does not create a new command and it returns NULL. *Proc* should have arguments and result that match the type **Tcl_CmdProc**:

```
typedef int Tcl_CmdProc(
        ClientData clientData,
        Tcl_Interp *interp,
        int argc,
        const char *argv[]);
```

When *proc* is invoked the *clientData* and *interp* parameters will be copies of the *clientData* and *interp* arguments given to **Tcl_CreateCommand**. Typically, *clientData* points to an application-specific data structure that describes what to do when the command procedure is invoked. *Argc* and *argv* describe the arguments to the command, *argc* giving the number of arguments (including the command name) and *argv* giving the values of the arguments as strings. The *argv* array will contain *argc*+1 values; the first *argc* values point to the argument strings, and the last value is NULL. Note that the argument strings should not be modified as they may point to constant strings or may be shared with other parts of the interpreter.

Note that the argument strings are encoded in normalized UTF-8 since version 8.1 of Tcl.

Proc must return an integer code that is expected to be one of **TCL_OK**, **TCL_ERROR**, **TCL_RETURN**, **TCL_BREAK**, or **TCL_CONTINUE**. See the Tcl overview man page for details on what these codes mean. Most normal commands will only return **TCL_OK** or **TCL_ERROR**. In addition, *proc* must set the interpreter result; in the case of a **TCL_OK** return code this gives the result of the command, and in the case of **TCL_ERROR** it gives an error message. The **Tcl_SetResult** procedure provides an easy interface for setting the return value; for complete details on how the interpreter result field is managed, see the **Tcl_Interp** man page. Before invoking a command procedure, **Tcl_Eval** sets the interpreter result to point to an empty string, so simple commands can return an empty result by doing nothing at all.

The contents of the *argv* array belong to Tcl and are not guaranteed to persist once *proc* returns: *proc* should not modify them, nor should it set the interpreter result to point anywhere within the *argv* values. Call **Tcl_SetResult** with status **TCL_VOLATILE** if you want to return something from the *argv* array.

DeleteProc will be invoked when (if) *cmdName* is deleted. This can occur through a call to **Tcl_DeleteCommand** or **Tcl_DeleteInterp**, or by replacing *cmdName* in another call to **Tcl_CreateCommand**. *DeleteProc* is invoked before the command is deleted, and gives the application an opportunity to release any structures associated with the command. *DeleteProc* should have arguments and result that match the type **Tcl_CmdDeleteProc**:

```
typedef void Tcl_CmdDeleteProc(
        ClientData clientData);
```

The *clientData* argument will be the same as the *clientData* argument passed to **Tcl_CreateCommand**.

Keywords: CrtCommand, bind, command, create, delete, interpreter, namespace

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

CrtFileHdlr

Name

Tcl_CreateFileHandler, Tcl_DeleteFileHandler - associate procedure callbacks with files or devices (Unix only)

Synopsis

#include <tcl.h>

Tcl_CreateFileHandler(*fd*, *mask*, *proc*, *clientData*)

Tcl_DeleteFileHandler(*fd*)

Arguments

int *fd* (in)
Unix file descriptor for an open file or device.

int *mask* (in)
Conditions under which *proc* should be called: OR-ed combination of **TCL_READABLE**, **TCL_WRITABLE**, and **TCL_EXCEPTION**. May be set to 0 to temporarily disable a handler.

Tcl_FileProc **proc* (in)
Procedure to invoke whenever the file or device indicated by *file* meets the conditions specified by *mask*.

ClientData *clientData* (in)
Arbitrary one-word value to pass to *proc*.

Description

Tcl_CreateFileHandler arranges for *proc* to be invoked in the future whenever I/O becomes possible on a file or an exceptional condition exists for the file. The file is indicated by *fd*, and the conditions of interest are indicated by *mask*. For example, if *mask* is **TCL_READABLE**, *proc* will be called when the file is readable. The callback to *proc* is made by **Tcl_DoOneEvent**, so **Tcl_CreateFileHandler** is only useful in programs that dispatch events through **Tcl_DoOneEvent** or through Tcl commands such as **vwait**.

Proc should have arguments and result that match the type **Tcl_FileProc**:

```
typedef void Tcl_FileProc(
        ClientData clientData,
        int mask);
```

The *clientData* parameter to *proc* is a copy of the *clientData* argument given to **Tcl_CreateFileHandler** when the callback was created. Typically, *clientData* points to a data structure containing application-specific information about the file. *Mask* is an integer mask indicating which of the requested conditions actually exists for the file; it will contain a subset of the bits in the *mask* argument to **Tcl_CreateFileHandler**.

There may exist only one handler for a given file at a given time. If **Tcl_CreateFileHandler** is called when a handler already exists for *fd*, then the new callback replaces the information that was previously recorded.

Tcl_DeleteFileHandler may be called to delete the file handler for *fd*; if no handler exists for the file given by *fd* then the procedure has no effect.

The purpose of file handlers is to enable an application to respond to events while waiting for files to become ready for I/O. For this to work correctly, the application may need to use

non-blocking I/O operations on the files for which handlers are declared. Otherwise the application may block if it reads or writes too much data; while waiting for the I/O to complete the application will not be able to service other events. Use **Tcl_SetChannelOption** with **-blocking** to set the channel into blocking or nonblocking mode as required.

Note that these interfaces are only supported by the Unix implementation of the Tcl notifier.

Keywords: CrtFileHdlr, callback, file, handler

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

CrtInterp

Name

Tcl_CreateInterp, Tcl_DeleteInterp, Tcl_InterpActive, Tcl_InterpDeleted - create and delete Tcl command interpreters

Synopsis

#include <tcl.h>

Tcl_Interp *
Tcl_CreateInterp()

Tcl_DeleteInterp(*interp*)

int
Tcl_InterpDeleted(*interp*)

int
Tcl_InterpActive(*interp*)

Arguments

Tcl_Interp ****interp* **(in)
Token for interpreter to be destroyed or queried.

Description

Tcl_CreateInterp creates a new interpreter structure and returns a token for it. The token is required in calls to most other Tcl procedures, such as **Tcl_CreateCommand**, **Tcl_Eval**, and **Tcl_DeleteInterp**. The token returned by **Tcl_CreateInterp** may only be passed to Tcl routines called from the same thread as the original **Tcl_CreateInterp** call. It is not safe for multiple threads to pass the same token to Tcl's routines. The new interpreter is initialized with the built-in Tcl commands and with standard variables like **tcl_platform** and **env**. To bind in additional commands, call **Tcl_CreateCommand**, and to create additional variables, call **Tcl_SetVar**.

Tcl_DeleteInterp marks an interpreter as deleted; the interpreter will eventually be deleted when all calls to **Tcl_Preserve** for it have been matched by calls to **Tcl_Release**. At that time, all of the resources associated with it, including variables, procedures, and application-specific command bindings, will be deleted. After **Tcl_DeleteInterp** returns any attempt to use **Tcl_Eval** on the interpreter will fail and return **TCL_ERROR**. After the call to **Tcl_DeleteInterp** it is safe to examine the interpreter's result, query or set the values of variables, define, undefine or retrieve procedures, and examine the runtime evaluation stack. See below, in the section

INTERPRETERS AND MEMORY MANAGEMENT for details.

Tcl_InterpDeleted returns nonzero if **Tcl_DeleteInterp** was called with *interp* as its argument; this indicates that the interpreter will eventually be deleted, when the last call to **Tcl_Preserve** for it is matched by a call to **Tcl_Release**. If nonzero is returned, further calls to **Tcl_Eval** in this interpreter will return **TCL_ERROR**.

Tcl_InterpDeleted is useful in deletion callbacks to distinguish between when only the memory the callback is responsible for is being deleted and when the whole interpreter is being deleted. In the former case the callback may recreate the data being deleted, but this would lead to an infinite loop if the interpreter were being deleted.

Tcl_InterpActive is useful for determining whether there is any execution of scripts ongoing in an interpreter, which is a useful piece of information when Tcl is embedded in a garbage-collected environment and it becomes necessary to determine whether the interpreter is a candidate for deletion. The function returns a true value if the interpreter has at least one active execution running inside it, and a false value otherwise.

Interpreters and memory management

Tcl_DeleteInterp can be called at any time on an interpreter that may be used by nested evaluations and C code in various extensions. Tcl implements a simple mechanism that allows callers to use interpreters without worrying about the interpreter being deleted in a nested call, and without requiring special code to protect the interpreter, in most cases. This mechanism ensures that nested uses of an interpreter can safely continue using it even after **Tcl_DeleteInterp** is called.

The mechanism relies on matching up calls to **Tcl_Preserve** with calls to **Tcl_Release**. If **Tcl_DeleteInterp** has been called, only when the last call to **Tcl_Preserve** is matched by a call to **Tcl_Release**, will the interpreter be freed. See the manual entry for **Tcl_Preserve** for a description of these functions.

The rules for when the user of an interpreter must call **Tcl_Preserve** and **Tcl_Release** are simple:

Interpreters Passed As Arguments
: Functions that are passed an interpreter as an argument can safely use the interpreter without any special protection. Thus, when you write an extension consisting of new Tcl commands, no special code is needed to protect interpreters received as arguments. This covers the majority of all uses.

Interpreter Creation And Deletion
: When a new interpreter is created and used in a call to **Tcl_Eval**, **Tcl_VarEval**, **Tcl_GlobalEval**, **Tcl_SetVar**, or **Tcl_GetVar**, a pair of calls to **Tcl_Preserve** and **Tcl_Release** should be wrapped around all uses of the interpreter. Remember that it is unsafe to use the interpreter once **Tcl_Release** has been called. To ensure that the interpreter is properly deleted when it is no longer needed, call **Tcl_InterpDeleted** to test if some other code already called **Tcl_DeleteInterp**; if not, call **Tcl_DeleteInterp** before calling **Tcl_Release** in your own code.

Retrieving An Interpreter From A Data Structure
: When an interpreter is retrieved from a data structure (e.g. the client data of a callback) for use in one of the evaluation functions (**Tcl_Eval**, **Tcl_VarEval**, **Tcl_GlobalEval**, **Tcl_EvalObjv**,

etc.) or variable access functions (**Tcl_SetVar**, **Tcl_GetVar**, **Tcl_SetVar2Ex**, etc.), a pair of calls to **Tcl_Preserve** and **Tcl_Release** should be wrapped around all uses of the interpreter; it is unsafe to reuse the interpreter once **Tcl_Release** has been called. If an interpreter is stored inside a callback data structure, an appropriate deletion cleanup mechanism should be set up by the code that creates the data structure so that the interpreter is removed from the data structure (e.g. by setting the field to NULL) when the interpreter is deleted. Otherwise, you may be using an interpreter that has been freed and whose memory may already have been reused. All uses of interpreters in Tcl and Tk have already been protected. Extension writers should ensure that their code also properly protects any additional interpreters used, as described above. Note that the protection mechanisms do not work well with conventional garbage collection systems. When in such a managed environment, **Tcl_InterpActive** should be used to determine when an interpreter is a candidate for deletion due to inactivity.

Keywords: CrtInterp, command, create, delete, interpreter

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

CrtMathFnc

Name

Tcl_CreateMathFunc, Tcl_GetMathFuncInfo, Tcl_ListMathFuncs - Define, query and enumerate math functions for expressions

Notice of eventual deprecation

The **Tcl_CreateMathFunc** and **Tcl_GetMathFuncInfo** functions are rendered somewhat obsolete by the ability to create functions for expressions by placing commands in the **tcl::mathfunc** namespace, as described in the **mathfunc** manual page; the API described on this page is not expected to be maintained indefinitely.

Synopsis

#include <tcl.h>

void
Tcl_CreateMathFunc(*interp*, *name*, *numArgs*, *argTypes*, *proc*, *clientData*)

int
Tcl_GetMathFuncInfo(*interp*, *name*, *numArgsPtr*, *argTypesPtr*, *procPtr*, *clientDataPtr*)

Tcl_Obj *
Tcl_ListMathFuncs(*interp*, *pattern*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter in which new function will be defined.

const char *_name_ (in)
Name for new function.

int _numArgs_ (in)
Number of arguments to new function; also gives size of *argTypes* array.

Tcl_ValueType *_argTypes_ (in)
Points to an array giving the permissible types for each argument to function.

Tcl_MathProc *_proc_ (in)
Procedure that implements the function.

ClientData _clientData_ (in)
Arbitrary one-word value to pass to *proc* when it is invoked.

int *_numArgsPtr_ (out)
Points to a variable that will be set to contain the number of arguments to the function.

Tcl_ValueType **_argTypesPtr_ (out)
Points to a variable that will be set to contain a pointer to an array giving the permissible types for each argument to the function which will need to be freed up using *Tcl_Free*.

Tcl_MathProc **_procPtr_ (out)
Points to a variable that will be set to contain a pointer to the implementation code for the function (or NULL if the function is implemented directly in bytecode).

ClientData *_clientDataPtr_ (out)
Points to a variable that will be set to contain the clientData argument passed to *Tcl_CreateMathFunc* when the function was created if the function is not implemented directly in bytecode.

const char *_pattern_ (in)
Pattern to match against function names so as to filter them (by passing to *Tcl_StringMatch*), or NULL to not apply any filter.

Description

Tcl allows a number of mathematical functions to be used in expressions, such as **sin**, **cos**, and **hypot**. These functions are represented by commands in the namespace, **tcl::mathfunc**. The **Tcl_CreateMathFunc** function is an obsolete way for applications to add additional functions to those already provided by Tcl or to replace existing functions. It should not be used by new applications, which should create math functions using **Tcl_CreateObjCommand** to create a command in the **tcl::mathfunc** namespace.

In the **Tcl_CreateMathFunc** interface, *Name* is the name of the function as it will appear in expressions. If *name* does not already exist in the **::tcl::mathfunc** namespace, then a new command is created in that namespace. If *name* does exist, then the existing function is replaced. *NumArgs* and *argTypes* describe the arguments to the function. Each entry in the *argTypes* array must be one of **TCL_INT**, **TCL_DOUBLE**, **TCL_WIDE_INT**, or **TCL_EITHER** to indicate whether the corresponding argument must be an integer, a double-precision floating value, a wide (64-bit) integer, or any, respectively.

Whenever the function is invoked in an expression Tcl will invoke *proc*. *Proc* should have arguments and result that match the type **Tcl_MathProc**:

```
typedef int Tcl_MathProc(
        ClientData clientData,
        Tcl_Interp *interp,
        Tcl_Value *args,
        Tcl_Value *resultPtr);
```

When *proc* is invoked the *clientData* and *interp* arguments will be the same as those passed to

Tcl_CreateMathFunc. *Args* will point to an array of *numArgs* Tcl_Value structures, which describe the actual arguments to the function:

```
typedef struct Tcl_Value {
    Tcl_ValueType type;
    long intValue;
    double doubleValue;
    Tcl_WideInt wideValue;
} Tcl_Value;
```

The *type* field indicates the type of the argument and is one of **TCL_INT**, **TCL_DOUBLE** or **TCL_WIDE_INT**. It will match the *argTypes* value specified for the function unless the *argTypes* value was **TCL_EITHER**. Tcl converts the argument supplied in the expression to the type requested in *argTypes*, if that is necessary. Depending on the value of the *type* field, the *intValue* , *doubleValue* or *wideValue* field will contain the actual value of the argument.

Proc should compute its result and store it either as an integer in *resultPtr->intValue* or as a floating value in *resultPtr->doubleValue*. It should set also *resultPtr->type* to one of **TCL_INT**, **TCL_DOUBLE** or **TCL_WIDE_INT** to indicate which value was set. Under normal circumstances *proc* should return **TCL_OK**. If an error occurs while executing the function, *proc* should return **TCL_ERROR** and leave an error message in the interpreter's result.

Tcl_GetMathFuncInfo retrieves the values associated with function *name* that were passed to a preceding **Tcl_CreateMathFunc** call. Normally, the return code is **TCL_OK** but if the named function does not exist, **TCL_ERROR** is returned and an error message is placed in the interpreter's result.

If an error did not occur, the array reference placed in the variable pointed to by *argTypesPtr* is newly allocated, and should be released by passing it to **Tcl_Free**. Some functions (the standard set implemented in the core, and those defined by placing commands in the **tcl::mathfunc** namespace) do not have argument type information; attempting to retrieve values for them causes a NULL to be stored in the variable pointed to by *procPtr* and the variable pointed to by *clientDataPtr* will not be modified. The variable pointed to by *numArgsPointer* will contain -1, and no argument types will be stored in the variable pointed to by *argTypesPointer*.

Tcl_ListMathFuncs returns a Tcl value containing a list of all the math functions defined in the interpreter whose name matches *pattern*. The returned value has a reference count of zero.

Keywords: CrtMathFnc, expression, mathematical function

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

CrtObjCmd

Name

Tcl_CreateObjCommand, Tcl_DeleteCommand, Tcl_DeleteCommandFromToken, Tcl_GetCommandInfo, Tcl_GetCommandInfoFromToken, Tcl_SetCommandInfo, Tcl_SetCommandInfoFromToken, Tcl_GetCommandName, Tcl_GetCommandFullName, Tcl_GetCommandFromObj - implement new commands in C

Synopsis

#include <tcl.h>

Tcl_Command
Tcl_CreateObjCommand(*interp*, *cmdName*, *proc*, *clientData*, *deleteProc*)

int
Tcl_DeleteCommand(*interp*, *cmdName*)

int
Tcl_DeleteCommandFromToken(*interp*, *token*)

int
Tcl_GetCommandInfo(*interp*, *cmdName*, *infoPtr*)

int
Tcl_SetCommandInfo(*interp*, *cmdName*, *infoPtr*)

int
Tcl_GetCommandInfoFromToken(*token*, *infoPtr*)

int
Tcl_SetCommandInfoFromToken(*token*, *infoPtr*)

const char *
Tcl_GetCommandName(*interp*, *token*)

void
Tcl_GetCommandFullName(*interp*, *token*, *objPtr*)

Tcl_Command
Tcl_GetCommandFromObj(*interp*, *objPtr*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter in which to create a new command or that contains a command.

const char *_cmdName_ (in)
Name of command.

Tcl_ObjCmdProc *_proc_ (in)
Implementation of the new command: *proc* will be called whenever *cmdName* is invoked as a command.

ClientData _clientData_ (in)
Arbitrary one-word value to pass to *proc* and *deleteProc*.

Tcl_CmdDeleteProc *_deleteProc_ (in)
Procedure to call before *cmdName* is deleted from the interpreter; allows for command-specific cleanup. If NULL, then no procedure is called before the command is deleted.

Tcl_Command _token_ (in)
Token for command, returned by previous call to **Tcl_CreateObjCommand**. The command must not have been deleted.

Tcl_CmdInfo *_infoPtr_ (in/out)
Pointer to structure containing various information about a Tcl command.

Tcl_Obj *_objPtr_ (in)
Value containing the name of a Tcl command.

Description

Tcl_CreateObjCommand defines a new command in *interp* and associates it with procedure *proc* such that whenever *name* is invoked as a Tcl command (e.g., via a call to **Tcl_EvalObjEx**) the Tcl interpreter will call *proc* to process the command.

Tcl_CreateObjCommand deletes any existing command *name* already associated with the interpreter (however see below for an exception where the existing command is not deleted). It returns a token that may be used to refer to the command in subsequent calls to **Tcl_GetCommandName**. If *name* contains any **::** namespace qualifiers, then the command is added to the specified namespace; otherwise the command is added to the global namespace. If **Tcl_CreateObjCommand** is called for an interpreter that is in the process of being deleted, then it does not create a new command and it returns NULL. *proc* should have arguments and result that match the type **Tcl_ObjCmdProc**:

```
typedef int Tcl_ObjCmdProc(
        ClientData clientData,
        Tcl_Interp *interp,
        int objc,
        Tcl_Obj *const objv[]);
```

When *proc* is invoked, the *clientData* and *interp* parameters will be copies of the *clientData* and *interp* arguments given to **Tcl_CreateObjCommand**. Typically, *clientData* points to an application-specific data structure that describes what to do when the command procedure is invoked. *Objc* and *objv* describe the arguments to the command, *objc* giving the number of argument values (including the command name) and *objv* giving the values of the arguments. The *objv* array will contain *objc* values, pointing to the argument values. Unlike *argv*[*argv*] used in a string-based command procedure, *objv*[*objc*] will not contain NULL.

Additionally, when *proc* is invoked, it must not modify the contents of the *objv* array by assigning new pointer values to any element of the array (for example, *objv*[**2**] = **NULL**) because this will cause memory to be lost and the runtime stack to be corrupted. The **const** in the declaration of *objv* will cause ANSI-compliant compilers to report any such attempted assignment as an error. However, it is acceptable to modify the internal representation of any individual value argument. For instance, the user may call **Tcl_GetIntFromObj** on *objv*[**2**] to obtain the integer representation of that value; that call may change the type of the value that *objv*[**2**] points at, but will not change where *objv*[**2**] points.

proc must return an integer code that is either **TCL_OK**, **TCL_ERROR**, **TCL_RETURN**, **TCL_BREAK**, or **TCL_CONTINUE**. See the Tcl overview man page for details on what these codes mean. Most normal commands will only return **TCL_OK** or **TCL_ERROR**. In addition, if *proc* needs to return a non-empty result, it can call **Tcl_SetObjResult** to set the interpreter's result. In the case of a **TCL_OK** return code this gives the result of the command, and in the case of **TCL_ERROR** this gives an error message. Before invoking a command procedure, **Tcl_EvalObjEx** sets interpreter's result to point to a value representing an empty string, so simple commands can return an empty result by doing nothing at all.

The contents of the *objv* array belong to Tcl and are not guaranteed to persist once *proc* returns: *proc* should not modify them. Call **Tcl_SetObjResult** if you want to return something from the *objv* array.

Ordinarily, **Tcl_CreateObjCommand** deletes any existing command *name* already associated

with the interpreter. However, if the existing command was created by a previous call to **Tcl_CreateCommand**, **Tcl_CreateObjCommand** does not delete the command but instead arranges for the Tcl interpreter to call the **Tcl_ObjCmdProc** *proc* in the future. The old string-based **Tcl_CmdProc** associated with the command is retained and its address can be obtained by subsequent **Tcl_GetCommandInfo** calls. This is done for backwards compatibility.

DeleteProc will be invoked when (if) *name* is deleted. This can occur through a call to **Tcl_DeleteCommand**, **Tcl_DeleteCommandFromToken**, or **Tcl_DeleteInterp**, or by replacing *name* in another call to **Tcl_CreateObjCommand**. *DeleteProc* is invoked before the command is deleted, and gives the application an opportunity to release any structures associated with the command. *DeleteProc* should have arguments and result that match the type **Tcl_CmdDeleteProc**:

```
typedef void Tcl_CmdDeleteProc(
        ClientData clientData);
```

The *clientData* argument will be the same as the *clientData* argument passed to **Tcl_CreateObjCommand**.

Tcl_DeleteCommand deletes a command from a command interpreter. Once the call completes, attempts to invoke *cmdName* in *interp* will result in errors. If *cmdName* is not bound as a command in *interp* then **Tcl_DeleteCommand** does nothing and returns -1; otherwise it returns 0. There are no restrictions on *cmdName*: it may refer to a built-in command, an application-specific command, or a Tcl procedure. If *name* contains any **::** namespace qualifiers, the command is deleted from the specified namespace.

Given a token returned by **Tcl_CreateObjCommand**, **Tcl_DeleteCommandFromToken** deletes the command from a command interpreter. It will delete a command even if that command has been renamed. Once the call completes, attempts to invoke the command in *interp* will result in errors. If the command corresponding to *token* has already been deleted from *interp* then **Tcl_DeleteCommand** does nothing and returns -1; otherwise it returns 0.

Tcl_GetCommandInfo checks to see whether its *cmdName* argument exists as a command in *interp*. *cmdName* may include **::** namespace qualifiers to identify a command in a particular namespace. If the command is not found, then it returns 0. Otherwise it places information about the command in the **Tcl_CmdInfo** structure pointed to by *infoPtr* and returns 1. A **Tcl_CmdInfo** structure has the following fields:

```
typedef struct Tcl_CmdInfo {
    int isNativeObjectProc;
    Tcl_ObjCmdProc *objProc;
    ClientData objClientData;
    Tcl_CmdProc *proc;
    ClientData clientData;
    Tcl_CmdDeleteProc *deleteProc;
    ClientData deleteData;
    Tcl_Namespace *namespacePtr;
} Tcl_CmdInfo;
```

The *isNativeObjectProc* field has the value 1 if **Tcl_CreateObjCommand** was called to register the command; it is 0 if only **Tcl_CreateCommand** was called. It allows a program to determine

whether it is faster to call *objProc* or *proc*: *objProc* is normally faster if *isNativeObjectProc* has the value 1. The fields *objProc* and *objClientData* have the same meaning as the *proc* and *clientData* arguments to **Tcl_CreateObjCommand**; they hold information about the value-based command procedure that the Tcl interpreter calls to implement the command. The fields *proc* and *clientData* hold information about the string-based command procedure that implements the command. If **Tcl_CreateCommand** was called for this command, this is the procedure passed to it; otherwise, this is a compatibility procedure registered by **Tcl_CreateObjCommand** that simply calls the command's value-based procedure after converting its string arguments to Tcl values. The field *deleteData* is the ClientData value to pass to *deleteProc*; it is normally the same as *clientData* but may be set independently using the **Tcl_SetCommandInfo** procedure. The field *namespacePtr* holds a pointer to the Tcl_Namespace that contains the command.

Tcl_GetCommandInfoFromToken is identical to **Tcl_GetCommandInfo** except that it uses a command token returned from **Tcl_CreateObjCommand** in place of the command name. If the *token* parameter is NULL, it returns 0; otherwise, it returns 1 and fills in the structure designated by *infoPtr*.

Tcl_SetCommandInfo is used to modify the procedures and ClientData values associated with a command. Its *cmdName* argument is the name of a command in *interp*. *cmdName* may include **::** namespace qualifiers to identify a command in a particular namespace. If this command does not exist then **Tcl_SetCommandInfo** returns 0. Otherwise, it copies the information from **infoPtr* to Tcl's internal structure for the command and returns 1.

Tcl_SetCommandInfoFromToken is identical to **Tcl_SetCommandInfo** except that it takes a command token as returned by **Tcl_CreateObjCommand** instead of the command name. If the *token* parameter is NULL, it returns 0. Otherwise, it copies the information from **infoPtr* to Tcl's internal structure for the command and returns 1.

Note that **Tcl_SetCommandInfo** and **Tcl_SetCommandInfoFromToken** both allow the ClientData for a command's deletion procedure to be given a different value than the ClientData for its command procedure.

Note that neither **Tcl_SetCommandInfo** nor **Tcl_SetCommandInfoFromToken** will change a command's namespace. Use **Tcl_Eval** to call the **rename** command to do that.

Tcl_GetCommandName provides a mechanism for tracking commands that have been renamed. Given a token returned by **Tcl_CreateObjCommand** when the command was created, **Tcl_GetCommandName** returns the string name of the command. If the command has been renamed since it was created, then **Tcl_GetCommandName** returns the current name. This name does not include any **::** namespace qualifiers. The command corresponding to *token* must not have been deleted. The string returned by **Tcl_GetCommandName** is in dynamic memory owned by Tcl and is only guaranteed to retain its value as long as the command is not deleted or renamed; callers should copy the string if they need to keep it for a long time.

Tcl_GetCommandFullName produces the fully qualified name of a command from a command token. The name, including all namespace prefixes, is appended to the value specified by *objPtr*.

Tcl_GetCommandFromObj returns a token for the command specified by the name in a **Tcl_Obj**. The command name is resolved relative to the current namespace. Returns NULL if the command is not found.

Keywords: CrtObjCmd, bind, command, create, delete, namespace, value

Copyright (c) 1996-1997 Sun Microsystems, Inc.

CrtTimerHdlr

Name

Tcl_CreateTimerHandler, Tcl_DeleteTimerHandler - call a procedure at a given time

Synopsis

#include <tcl.h>

Tcl_TimerToken
Tcl_CreateTimerHandler(*milliseconds*, *proc*, *clientData*)

Tcl_DeleteTimerHandler(*token*)

Arguments

int *milliseconds* (in)
How many milliseconds to wait before invoking *proc*.

Tcl_TimerProc ****proc* **(in)**
Procedure to invoke after *milliseconds* have elapsed.

ClientData *clientData* (in)
Arbitrary one-word value to pass to *proc*.

Tcl_TimerToken *token* (in)
Token for previously created timer handler (the return value from some previous call to **Tcl_CreateTimerHandler**).

Description

Tcl_CreateTimerHandler arranges for *proc* to be invoked at a time *milliseconds* milliseconds in the future. The callback to *proc* will be made by **Tcl_DoOneEvent**, so **Tcl_CreateTimerHandler** is only useful in programs that dispatch events through **Tcl_DoOneEvent** or through Tcl commands such as **vwait**. The call to *proc* may not be made at the exact time given by *milliseconds*: it will be made at the next opportunity after that time. For example, if **Tcl_DoOneEvent** is not called until long after the time has elapsed, or if there are other pending events to process before the call to *proc*, then the call to *proc* will be delayed.

Proc should have arguments and return value that match the type **Tcl_TimerProc**:

```
typedef void Tcl_TimerProc(
        ClientData clientData);
```

The *clientData* parameter to *proc* is a copy of the *clientData* argument given to **Tcl_CreateTimerHandler** when the callback was created. Typically, *clientData* points to a data structure containing application-specific information about what to do in *proc*.

Tcl_DeleteTimerHandler may be called to delete a previously created timer handler. It deletes the handler indicated by *token* so that no call to *proc* will be made; if that handler no longer exists (e.g. because the time period has already elapsed and *proc* has been invoked then **Tcl_DeleteTimerHandler** does nothing. The tokens returned by **Tcl_CreateTimerHandler** never have a value of NULL, so if NULL is passed to **Tcl_DeleteTimerHandler** then the procedure does nothing.

Keywords: CrtTimerHdlr, callback, clock, handler, timer

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

CrtTrace

Name

Tcl_CreateTrace, Tcl_CreateObjTrace, Tcl_DeleteTrace - arrange for command execution to be traced

Synopsis

#include <tcl.h>

Tcl_Trace
Tcl_CreateTrace(*interp*, *level*, *proc*, *clientData*)

Tcl_Trace
Tcl_CreateObjTrace(*interp*, *level*, *flags*, *objProc*, *clientData*, *deleteProc*)

Tcl_DeleteTrace(*interp*, *trace*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter containing command to be traced or untraced.

int _level_ (in)
Only commands at or below this nesting level will be traced unless 0 is specified. 1 means top-level commands only, 2 means top-level commands or those that are invoked as immediate consequences of executing top-level commands (procedure bodies, bracketed commands, etc.) and so on. A value of 0 means that commands at any level are traced.

int _flags_ (in)
Flags governing the trace execution. See below for details.

Tcl_CmdObjTraceProc *_objProc_ (in)
Procedure to call for each command that is executed. See below for details of the calling sequence.

Tcl_CmdTraceProc *_proc_ (in)
Procedure to call for each command that is executed. See below for details on the calling sequence.

ClientData _clientData_ (in)
Arbitrary one-word value to pass to *objProc* or *proc*.

Tcl_CmdObjTraceDeleteProc *_deleteProc_ (in)
Procedure to call when the trace is deleted. See below for details of the calling sequence. A NULL pointer is permissible and results in no callback when the trace is deleted.

Tcl_Trace _trace_ (in)
Token for trace to be removed (return value from previous call to **Tcl_CreateTrace**).

Description

Tcl_CreateObjTrace arranges for command tracing. After it is called, *objProc* will be invoked before the Tcl interpreter calls any command procedure when evaluating commands in *interp*. The return value from **Tcl_CreateObjTrace** is a token for the trace, which may be passed to

Tcl_DeleteTrace to remove the trace. There may be many traces in effect simultaneously for the same interpreter.

objProc should have arguments and result that match the type, **Tcl_CmdObjTraceProc**:

```
typedef int Tcl_CmdObjTraceProc(
        ClientData clientData,
        Tcl_Interp* interp,
        int level,
        const char *command,
        Tcl_Command commandToken,
        int objc,
        Tcl_Obj *const objv[]);
```

The *clientData* and *interp* parameters are copies of the corresponding arguments given to **Tcl_CreateTrace**. *ClientData* typically points to an application-specific data structure that describes what to do when *objProc* is invoked. The *level* parameter gives the nesting level of the command (1 for top-level commands passed to **Tcl_Eval** by the application, 2 for the next-level commands passed to **Tcl_Eval** as part of parsing or interpreting level-1 commands, and so on). The *command* parameter points to a string containing the text of the command, before any argument substitution. The *commandToken* parameter is a Tcl command token that identifies the command to be invoked. The token may be passed to **Tcl_GetCommandName**, **Tcl_GetCommandInfoFromToken**, or **Tcl_SetCommandInfoFromToken** to manipulate the definition of the command. The *objc* and *objv* parameters designate the final parameter count and parameter vector that will be passed to the command, and have had all substitutions performed.

The *objProc* callback is expected to return a standard Tcl status return code. If this code is **TCL_OK** (the normal case), then the Tcl interpreter will invoke the command. Any other return code is treated as if the command returned that status, and the command is *not* invoked.

The *objProc* callback must not modify *objv* in any way.

Tracing will only occur for commands at nesting level less than or equal to the *level* parameter (i.e. the *level* parameter to *objProc* will always be less than or equal to the *level* parameter to **Tcl_CreateTrace**).

Tracing has a significant effect on runtime performance because it causes the bytecode compiler to refrain from generating in-line code for Tcl commands such as **if** and **while** in order that they may be traced. If traces for the built-in commands are not required, the *flags* parameter may be set to the constant value **TCL_ALLOW_INLINE_COMPILATION**. In this case, traces on built-in commands may or may not result in trace callbacks, depending on the state of the interpreter, but run-time performance will be improved significantly. (This functionality is desirable, for example, when using **Tcl_CreateObjTrace** to implement an execution time profiler.)

Calls to *objProc* will be made by the Tcl parser immediately before it calls the command procedure for the command (*cmdProc*). This occurs after argument parsing and substitution, so tracing for substituted commands occurs before tracing of the commands containing the substitutions. If there is a syntax error in a command, or if there is no command procedure associated with a command name, then no tracing will occur for that command. If a string passed to Tcl_Eval contains multiple commands (bracketed, or on different lines) then multiple calls to *objProc* will occur, one for each command.

Tcl_DeleteTrace removes a trace, so that no future calls will be made to the procedure associated with the trace. After **Tcl_DeleteTrace** returns, the caller should never again use the *trace* token.

When **Tcl_DeleteTrace** is called, the interpreter invokes the *deleteProc* that was passed as a parameter to **Tcl_CreateObjTrace**. The *deleteProc* must match the type, **Tcl_CmdObjTraceDeleteProc**:

```
typedef void Tcl_CmdObjTraceDeleteProc(
        ClientData clientData);
```

The *clientData* parameter will be the same as the *clientData* parameter that was originally passed to **Tcl_CreateObjTrace**.

Tcl_CreateTrace is an alternative interface for command tracing, *not recommended for new applications*. It is provided for backward compatibility with code that was developed for older versions of the Tcl interpreter. It is similar to **Tcl_CreateObjTrace**, except that its *proc* parameter should have arguments and result that match the type **Tcl_CmdTraceProc**:

```
typedef void Tcl_CmdTraceProc(
        ClientData clientData,
        Tcl_Interp *interp,
        int level,
        char *command,
        Tcl_CmdProc *cmdProc,
        ClientData cmdClientData,
        int argc,
        const char *argv[]);
```

The parameters to the *proc* callback are similar to those of the *objProc* callback above. The *commandToken* is replaced with *cmdProc*, a pointer to the (string-based) command procedure that will be invoked; and *cmdClientData*, the client data that will be passed to the procedure. The *objc* parameter is replaced with an *argv* parameter, that gives the arguments to the command as character strings. *Proc* must not modify the *command* or *argv* strings.

If a trace created with **Tcl_CreateTrace** is in effect, inline compilation of Tcl commands such as **if** and **while** is always disabled. There is no notification when a trace created with **Tcl_CreateTrace** is deleted. There is no way to be notified when the trace created by **Tcl_CreateTrace** is deleted. There is no way for the *proc* associated with a call to **Tcl_CreateTrace** to abort execution of *command*.

Keywords: CrtTrace, command, create, delete, interpreter, trace

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2002 Kevin B. Kenny kennykb@acm.org*. All rights reserved.*

DString

Name

Tcl_DStringInit, Tcl_DStringAppend, Tcl_DStringAppendElement, Tcl_DStringStartSublist, Tcl_DStringEndSublist, Tcl_DStringLength, Tcl_DStringValue, Tcl_DStringSetLength,

Tcl_DStringTrunc, Tcl_DStringFree, Tcl_DStringResult, Tcl_DStringGetResult - manipulate dynamic strings

Synopsis

#include <tcl.h>

Tcl_DStringInit(*dsPtr*)

char *
Tcl_DStringAppend(*dsPtr*, *bytes*, *length*)

char *
Tcl_DStringAppendElement(*dsPtr*, *element*)

Tcl_DStringStartSublist(*dsPtr*)

Tcl_DStringEndSublist(*dsPtr*)

int
Tcl_DStringLength(*dsPtr*)

char *
Tcl_DStringValue(*dsPtr*)

Tcl_DStringSetLength(*dsPtr*, *newLength*)

Tcl_DStringTrunc(*dsPtr*, *newLength*)

Tcl_DStringFree(*dsPtr*)

Tcl_DStringResult(*interp*, *dsPtr*)

Tcl_DStringGetResult(*interp*, *dsPtr*)

Arguments

Tcl_DString *_dsPtr_ (in/out)
Pointer to structure that is used to manage a dynamic string.

const char *_bytes_ (in)
Pointer to characters to append to dynamic string.

const char *_element_ (in)
Pointer to characters to append as list element to dynamic string.

int _length_ (in)
Number of bytes from *bytes* to add to dynamic string. If -1, add all characters up to null terminating character.

int _newLength_ (in)
New length for dynamic string, not including null terminating character.

Tcl_Interp *_interp_ (in/out)
Interpreter whose result is to be set from or moved to the dynamic string.

Description

Dynamic strings provide a mechanism for building up arbitrarily long strings by gradually appending information. If the dynamic string is short then there will be no memory allocation overhead; as the string gets larger, additional space will be allocated as needed.

Tcl_DStringInit initializes a dynamic string to zero length. The Tcl_DString structure must have

been allocated by the caller. No assumptions are made about the current state of the structure; anything already in it is discarded. If the structure has been used previously, **Tcl_DStringFree** should be called first to free up any memory allocated for the old string.

Tcl_DStringAppend adds new information to a dynamic string, allocating more memory for the string if needed. If *length* is less than zero then everything in *bytes* is appended to the dynamic string; otherwise *length* specifies the number of bytes to append. **Tcl_DStringAppend** returns a pointer to the characters of the new string. The string can also be retrieved from the *string* field of the Tcl_DString structure.

Tcl_DStringAppendElement is similar to **Tcl_DStringAppend** except that it does not take a *length* argument (it appends all of *element*) and it converts the string to a proper list element before appending. **Tcl_DStringAppendElement** adds a separator space before the new list element unless the new list element is the first in a list or sub-list (i.e. either the current string is empty, or it contains the single character "{", or the last two characters of the current string are " {"). **Tcl_DStringAppendElement** returns a pointer to the characters of the new string.

Tcl_DStringStartSublist and **Tcl_DStringEndSublist** can be used to create nested lists. To append a list element that is itself a sublist, first call **Tcl_DStringStartSublist**, then call **Tcl_DStringAppendElement** for each of the elements in the sublist, then call **Tcl_DStringEndSublist** to end the sublist. **Tcl_DStringStartSublist** appends a space character if needed, followed by an open brace; **Tcl_DStringEndSublist** appends a close brace. Lists can be nested to any depth.

Tcl_DStringLength is a macro that returns the current length of a dynamic string (not including the terminating null character). **Tcl_DStringValue** is a macro that returns a pointer to the current contents of a dynamic string.

Tcl_DStringSetLength changes the length of a dynamic string. If *newLength* is less than the string's current length, then the string is truncated. If *newLength* is greater than the string's current length, then the string will become longer and new space will be allocated for the string if needed. However, **Tcl_DStringSetLength** will not initialize the new space except to provide a terminating null character; it is up to the caller to fill in the new space. **Tcl_DStringSetLength** does not free up the string's storage space even if the string is truncated to zero length, so **Tcl_DStringFree** will still need to be called.

Tcl_DStringTrunc changes the length of a dynamic string. This procedure is now deprecated. **Tcl_DStringSetLength** should be used instead.

Tcl_DStringFree should be called when you are finished using the string. It frees up any memory that was allocated for the string and reinitializes the string's value to an empty string.

Tcl_DStringResult sets the result of *interp* to the value of the dynamic string given by *dsPtr*. It does this by moving a pointer from *dsPtr* to the interpreter's result. This saves the cost of allocating new memory and copying the string. **Tcl_DStringResult** also reinitializes the dynamic string to an empty string.

Tcl_DStringGetResult does the opposite of **Tcl_DStringResult**. It sets the value of *dsPtr* to the result of *interp* and it clears *interp*'s result. If possible it does this by moving a pointer rather than by copying the string.

Keywords: DString, append, dynamic string, free, result

Copyright (c) 1993 The Regents of the University of California.

Copyright (c) 1994-1996 Sun Microsystems, Inc.

DetachPids

Name

Tcl_DetachPids, Tcl_ReapDetachedProcs, Tcl_WaitPid - manage child processes in background

Synopsis

#include <tcl.h>

Tcl_DetachPids(*numPids*, *pidPtr*)

Tcl_ReapDetachedProcs()

Tcl_Pid
Tcl_WaitPid(*pid*, *statusPtr*, *options*)

Arguments

int *numPids* (in)
Number of process ids contained in the array pointed to by *pidPtr*.

int **pidPtr* (in)
Address of array containing *numPids* process ids.

Tcl_Pid *pid* (in)
The id of the process (pipe) to wait for.

int **statusPtr* (out)
The result of waiting on a process (pipe). Either 0 or ECHILD.

int *options* (in)
The options controlling the wait. WNOHANG specifies not to wait when checking the process.

Description

Tcl_DetachPids and **Tcl_ReapDetachedProcs** provide a mechanism for managing subprocesses that are running in background. These procedures are needed because the parent of a process must eventually invoke the **waitpid** kernel call (or one of a few other similar kernel calls) to wait for the child to exit. Until the parent waits for the child, the child's state cannot be completely reclaimed by the system. If a parent continually creates children and doesn't wait on them, the system's process table will eventually overflow, even if all the children have exited.

Tcl_DetachPids may be called to ask Tcl to take responsibility for one or more processes whose process ids are contained in the *pidPtr* array passed as argument. The caller presumably has started these processes running in background and does not want to have to deal with them again.

Tcl_ReapDetachedProcs invokes the **waitpid** kernel call on each of the background processes so that its state can be cleaned up if it has exited. If the process has not exited yet, **Tcl_ReapDetachedProcs** does not wait for it to exit; it will check again the next time it is invoked. Tcl automatically calls **Tcl_ReapDetachedProcs** each time the **exec** command is executed, so in most cases it is not necessary for any code outside of Tcl to invoke **Tcl_ReapDetachedProcs**. However, if you call **Tcl_DetachPids** in situations where the **exec** command may never get executed, you may wish to call **Tcl_ReapDetachedProcs** from time to time so that background processes can be cleaned up.

Tcl_WaitPid is a thin wrapper around the facilities provided by the operating system to wait on the end of a spawned process and to check a whether spawned process is still running. It is used by **Tcl_ReapDetachedProcs** and the channel system to portably access the operating system.

Keywords: DetachPids, background, child, detach, process, wait

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

DictObj

Name

Tcl_NewDictObj, Tcl_DictObjPut, Tcl_DictObjGet, Tcl_DictObjRemove, Tcl_DictObjSize, Tcl_DictObjFirst, Tcl_DictObjNext, Tcl_DictObjDone, Tcl_DictObjPutKeyList, Tcl_DictObjRemoveKeyList - manipulate Tcl values as dictionaries

Synopsis

#include <tcl.h>

Tcl_Obj *
Tcl_NewDictObj()

int
Tcl_DictObjGet(*interp*, *dictPtr*, *keyPtr*, *valuePtrPtr*)

int
Tcl_DictObjPut(*interp*, *dictPtr*, *keyPtr*, *valuePtr*)

int
Tcl_DictObjRemove(*interp*, *dictPtr*, *keyPtr*)

int
Tcl_DictObjSize(*interp*, *dictPtr*, *sizePtr*)

int
Tcl_DictObjFirst(*interp*, *dictPtr*, *searchPtr*,
keyPtrPtr, *valuePtrPtr*, *donePtr*)

void
Tcl_DictObjNext(*searchPtr*, *keyPtrPtr*, *valuePtrPtr*, *donePtr*)

void
Tcl_DictObjDone(*searchPtr*)

int
Tcl_DictObjPutKeyList(*interp*, *dictPtr*, *keyc*, *keyv*, *valuePtr*)

int
Tcl_DictObjRemoveKeyList(*interp*, *dictPtr*, *keyc*, *keyv*)

Arguments

Tcl_Interp *_interp_ (in)
If an error occurs while converting a value to be a dictionary value, an error message is left in the interpreter's result value unless *interp* is NULL.

Tcl_Obj *_dictPtr_ (in/out)

Points to the dictionary value to be manipulated. If *dictPtr* does not already point to a dictionary value, an attempt will be made to convert it to one.

Tcl_Obj *_keyPtr_ (in)
Points to the key for the key/value pair being manipulated within the dictionary value.

Tcl_Obj **_keyPtrPtr_ (out)
Points to a variable that will have the key from a key/value pair placed within it. May be NULL to indicate that the caller is not interested in the key.

Tcl_Obj *_valuePtr_ (in)
Points to the value for the key/value pair being manipulated within the dictionary value (or sub-value, in the case of **Tcl_DictObjPutKeyList**.)

Tcl_Obj **_valuePtrPtr_ (out)
Points to a variable that will have the value from a key/value pair placed within it. For **Tcl_DictObjFirst** and **Tcl_DictObjNext**, this may be NULL to indicate that the caller is not interested in the value.

int *_sizePtr_ (out)
Points to a variable that will have the number of key/value pairs contained within the dictionary placed within it.

Tcl_DictSearch *_searchPtr_ (in/out)
Pointer to record to use to keep track of progress in enumerating all key/value pairs in a dictionary. The contents of the record will be initialized by the call to **Tcl_DictObjFirst**. If the enumerating is to be terminated before all values in the dictionary have been returned, the search record *must* be passed to **Tcl_DictObjDone** to enable the internal locks to be released.

int *_donePtr_ (out)
Points to a variable that will have a non-zero value written into it when the enumeration of the key/value pairs in a dictionary has completed, and a zero otherwise.

int _keyc_ (in)
Indicates the number of keys that will be supplied in the *keyv* array.

Tcl_Obj *const *_keyv_ (in)
Array of *keyc* pointers to values that **Tcl_DictObjPutKeyList** and **Tcl_DictObjRemoveKeyList** will use to locate the key/value pair to manipulate within the sub-dictionaries of the main dictionary value passed to them.

Description

Tcl dictionary values have an internal representation that supports efficient mapping from keys to values and which guarantees that the particular ordering of keys within the dictionary remains the same modulo any keys being deleted (which removes them from the order) or added (which adds them to the end of the order). If reinterpreted as a list, the values at the even-valued indices in the list will be the keys of the dictionary, and each will be followed (in the odd-valued index) by the value associated with that key.

The procedures described in this man page are used to create, modify, index, and iterate over dictionary values from C code.

Tcl_NewDictObj creates a new, empty dictionary value. The string representation of the value will be invalid, and the reference count of the value will be zero.

Tcl_DictObjGet looks up the given key within the given dictionary and writes a pointer to the value associated with that key into the variable pointed to by *valuePtrPtr*, or a NULL if the key has no mapping within the dictionary. The result of this procedure is **TCL_OK**, or **TCL_ERROR** if the *dictPtr* cannot be converted to a dictionary.

Tcl_DictObjPut updates the given dictionary so that the given key maps to the given value; any key may exist at most once in any particular dictionary. The dictionary must not be shared, but the key and value may be. This procedure may increase the reference count of both key and value if it proves necessary to store them. Neither key nor value should be NULL. The result of this procedure is **TCL_OK**, or **TCL_ERROR** if the *dictPtr* cannot be converted to a dictionary.

Tcl_DictObjRemove updates the given dictionary so that the given key has no mapping to any value. The dictionary must not be shared, but the key may be. The key actually stored in the dictionary will have its reference count decremented if it was present. It is not an error if the key did not previously exist. The result of this procedure is **TCL_OK**, or **TCL_ERROR** if the *dictPtr* cannot be converted to a dictionary.

Tcl_DictObjSize updates the given variable with the number of key/value pairs currently in the given dictionary. The result of this procedure is **TCL_OK**, or **TCL_ERROR** if the *dictPtr* cannot be converted to a dictionary.

Tcl_DictObjFirst commences an iteration across all the key/value pairs in the given dictionary, placing the key and value in the variables pointed to by the *keyPtrPtr* and *valuePtrPtr* arguments (which may be NULL to indicate that the caller is uninterested in they key or variable respectively.) The next key/value pair in the dictionary may be retrieved with **Tcl_DictObjNext**. Concurrent updates of the dictionary's internal representation will not modify the iteration processing unless the dictionary is unshared, when this will trigger premature termination of the iteration instead (which Tcl scripts cannot trigger via the **dict** command.) The *searchPtr* argument points to a piece of context that is used to identify which particular iteration is being performed, and is initialized by the call to **Tcl_DictObjFirst**. The *donePtr* argument points to a variable that is updated to be zero of there are further key/value pairs to be iterated over, or non-zero if the iteration is complete. The order of iteration is implementation-defined. If the *dictPtr* argument cannot be converted to a dictionary, **Tcl_DictObjFirst** returns **TCL_ERROR** and the iteration is not commenced, and otherwise it returns **TCL_OK**.

When **Tcl_DictObjFirst** is called upon a dictionary, a lock is placed on the dictionary to enable that dictionary to be iterated over safely without regard for whether the dictionary is modified during the iteration. Because of this, once the iteration over a dictionary's keys has finished (whether because all values have been iterated over as indicated by the variable indicated by the *donePtr* argument being set to one, or because no further values are required) the **Tcl_DictObjDone** function must be called with the same *searchPtr* as was passed to **Tcl_DictObjFirst** so that the internal locks can be released. Once a particular *searchPtr* is passed to **Tcl_DictObjDone**, passing it to **Tcl_DictObjNext** (without first initializing it with **Tcl_DictObjFirst**) will result in no values being produced and the variable pointed to by *donePtr* being set to one. It is safe to call **Tcl_DictObjDone** multiple times on the same *searchPtr* for each call to **Tcl_DictObjFirst**.

The procedures **Tcl_DictObjPutKeyList** and **Tcl_DictObjRemoveKeyList** are the close analogues of **Tcl_DictObjPut** and **Tcl_DictObjRemove** respectively, except that instead of working with a single dictionary, they are designed to operate on a nested tree of dictionaries, with inner dictionaries stored as values inside outer dictionaries. The *keyc* and *keyv* arguments

specify a list of keys (with outermost keys first) that acts as a path to the key/value pair to be affected. Note that there is no corresponding operation for reading a value for a path as this is easy to construct from repeated use of **Tcl_DictObjGet**. With **Tcl_DictObjPutKeyList**, nested dictionaries are created for non-terminal keys where they do not already exist. With **Tcl_DictObjRemoveKeyList**, all non-terminal keys must exist and have dictionaries as their values.

Example

Using the dictionary iteration interface to search determine if there is a key that maps to itself:

```
Tcl_DictSearch search;
Tcl_Obj *key, *value;
int done;

/*
 * Assume interp and objPtr are parameters.  This is the
 * idiomatic way to start an iteration over the dictionary; it
 * sets a lock on the internal representation that ensures that
 * there are no concurrent modification issues when normal
 * reference count management is also used.  The lock is
 * released automatically when the loop is finished, but must
 * be released manually when an exceptional exit from the loop
 * is performed. However it is safe to try to release the lock
 * even if we've finished iterating over the loop.
 */
if (Tcl_DictObjFirst(interp, objPtr, &search,
        &key, &value, &done) != TCL_OK) {
    return TCL_ERROR;
}
for (; !done ; Tcl_DictObjNext(&search, &key, &value, &done)) {
    /*
     * Note that strcmp() is not a good way of comparing
     * values and is just used here for demonstration
     * purposes.
     */
    if (!strcmp(Tcl_GetString(key), Tcl_GetString(value))) {
        break;
    }
}
Tcl_DictObjDone(&search);
Tcl_SetObjResult(interp, Tcl_NewBooleanObj(!done));
return TCL_OK;
```

Keywords: DictObj, dict, dict value, dictionary, dictionary value, hash table, iteration, value

Copyright (c) 2003 Donal K. Fellows

DoOneEvent

Name

Tcl_DoOneEvent - wait for events and invoke event handlers

Synopsis

#include <tcl.h>

int
Tcl_DoOneEvent(*flags*)

Arguments

int *flags* (in)

This parameter is normally zero. It may be an OR-ed combination of any of the following flag bits: **TCL_WINDOW_EVENTS**, **TCL_FILE_EVENTS**, **TCL_TIMER_EVENTS**, **TCL_IDLE_EVENTS**, **TCL_ALL_EVENTS**, or **TCL_DONT_WAIT**.

Description

This procedure is the entry point to Tcl's event loop; it is responsible for waiting for events and dispatching event handlers created with procedures such as **Tk_CreateEventHandler**, **Tcl_CreateFileHandler**, **Tcl_CreateTimerHandler**, and **Tcl_DoWhenIdle**. **Tcl_DoOneEvent** checks to see if events are already present on the Tcl event queue; if so, it calls the handler(s) for the first (oldest) event, removes it from the queue, and returns. If there are no events ready to be handled, then **Tcl_DoOneEvent** checks for new events from all possible sources. If any are found, it puts all of them on Tcl's event queue, calls handlers for the first event on the queue, and returns. If no events are found, **Tcl_DoOneEvent** checks for **Tcl_DoWhenIdle** callbacks; if any are found, it invokes all of them and returns. Finally, if no events or idle callbacks have been found, then **Tcl_DoOneEvent** sleeps until an event occurs; then it adds any new events to the Tcl event queue, calls handlers for the first event, and returns. The normal return value is 1 to signify that some event was processed (see below for other alternatives).

If the *flags* argument to **Tcl_DoOneEvent** is non-zero, it restricts the kinds of events that will be processed by **Tcl_DoOneEvent**. *Flags* may be an OR-ed combination of any of the following bits:

TCL_WINDOW_EVENTS -

Process window system events.

TCL_FILE_EVENTS -

Process file events.

TCL_TIMER_EVENTS -

Process timer events.

TCL_IDLE_EVENTS -

Process idle callbacks.

TCL_ALL_EVENTS -

Process all kinds of events: equivalent to OR-ing together all of the above flags or specifying none of them.

TCL_DONT_WAIT -

Do not sleep: process only events that are ready at the time of the call. If any of the flags **TCL_WINDOW_EVENTS**, **TCL_FILE_EVENTS**, **TCL_TIMER_EVENTS**, or **TCL_IDLE_EVENTS** is set, then the only events that will be considered are those for which flags are set. Setting none of these flags is equivalent to the value **TCL_ALL_EVENTS**, which causes all event

types to be processed. If an application has defined additional event sources with **Tcl_CreateEventSource**, then additional *flag* values may also be valid, depending on those event sources. The **TCL_DONT_WAIT** flag causes **Tcl_DoOneEvent** not to put the process to sleep: it will check for events but if none are found then it returns immediately with a return value of 0 to indicate that no work was done. **Tcl_DoOneEvent** will also return 0 without doing anything if the only alternative is to block forever (this can happen, for example, if *flags* is **TCL_IDLE_EVENTS** and there are no **Tcl_DoWhenIdle** callbacks pending, or if no event handlers or timer handlers exist). **Tcl_DoOneEvent** may be invoked recursively. For example, it is possible to invoke **Tcl_DoOneEvent** recursively from a handler called by **Tcl_DoOneEvent**. This sort of operation is useful in some modal situations, such as when a notification dialog has been popped up and an application wishes to wait for the user to click a button in the dialog before doing anything else.

Keywords: DoOneEvent, callback, event, handler, idle, timer

Copyright (c) 1990-1992 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

DoWhenIdle

Name

Tcl_DoWhenIdle, Tcl_CancelIdleCall - invoke a procedure when there are no pending events

Synopsis

#include <tcl.h>

Tcl_DoWhenIdle(*proc*, *clientData*)

Tcl_CancelIdleCall(*proc*, *clientData*)

Arguments

Tcl_IdleProc *_proc_ (in)
Procedure to invoke.

ClientData _clientData_ (in)
Arbitrary one-word value to pass to *proc*.

Description

Tcl_DoWhenIdle arranges for *proc* to be invoked when the application becomes idle. The application is considered to be idle when **Tcl_DoOneEvent** has been called, could not find any events to handle, and is about to go to sleep waiting for an event to occur. At this point all pending **Tcl_DoWhenIdle** handlers are invoked. For each call to **Tcl_DoWhenIdle** there will be a single call to *proc*; after *proc* is invoked the handler is automatically removed. **Tcl_DoWhenIdle** is only usable in programs that use **Tcl_DoOneEvent** to dispatch events.

Proc should have arguments and result that match the type **Tcl_IdleProc**:

```
typedef void Tcl_IdleProc(
        ClientData clientData);
```

The *clientData* parameter to *proc* is a copy of the *clientData* argument given to **Tcl_DoWhenIdle**. Typically, *clientData* points to a data structure containing application-specific

information about what *proc* should do.

Tcl_CancelIdleCall may be used to cancel one or more previous calls to **Tcl_DoWhenIdle**: if there is a **Tcl_DoWhenIdle** handler registered for *proc* and *clientData*, then it is removed without invoking it. If there is more than one handler on the idle list that refers to *proc* and *clientData*, all of the handlers are removed. If no existing handlers match *proc* and *clientData* then nothing happens.

Tcl_DoWhenIdle is most useful in situations where (a) a piece of work will have to be done but (b) it is possible that something will happen in the near future that will change what has to be done or require something different to be done. **Tcl_DoWhenIdle** allows the actual work to be deferred until all pending events have been processed. At this point the exact work to be done will presumably be known and it can be done exactly once.

For example, **Tcl_DoWhenIdle** might be used by an editor to defer display updates until all pending commands have been processed. Without this feature, redundant redisplays might occur in some situations, such as the processing of a command file.

Bugs

At present it is not safe for an idle callback to reschedule itself continuously. This will interact badly with certain features of Tk that attempt to wait for all idle callbacks to complete. If you would like for an idle callback to reschedule itself continuously, it is better to use a timer handler with a zero timeout period.

Keywords: DoWhenIdle, callback, defer, idle callback

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

DoubleObj

Name

Tcl_NewDoubleObj, Tcl_SetDoubleObj, Tcl_GetDoubleFromObj - manipulate Tcl values as floating-point values

Synopsis

#include <tcl.h>

Tcl_Obj *
Tcl_NewDoubleObj(*doubleValue*)

Tcl_SetDoubleObj(*objPtr*, *doubleValue*)

int
Tcl_GetDoubleFromObj(*interp*, *objPtr*, *doublePtr*)

Arguments

double *doubleValue* (in)
A double-precision floating-point value used to initialize or set a Tcl value.

Tcl_Obj **objPtr* (in/out)
For **Tcl_SetDoubleObj**, this points to the value in which to store a double value. For **Tcl_GetDoubleFromObj**, this refers to the value from which to retrieve a double value.

Tcl_Interp **interp* (in/out)

When non-NULL, an error message is left here when double value retrieval fails.

double *_doublePtr_ (out)

Points to place to store the double value obtained from *objPtr*.

Description

These procedures are used to create, modify, and read Tcl values that hold double-precision floating-point values.

Tcl_NewDoubleObj creates and returns a new Tcl value initialized to the double value *doubleValue*. The returned Tcl value is unshared.

Tcl_SetDoubleObj sets the value of an existing Tcl value pointed to by *objPtr* to the double value *doubleValue*. The *objPtr* argument must point to an unshared Tcl value. Any attempt to set the value of a shared Tcl value violates Tcl's copy-on-write policy. Any existing string representation or internal representation in the unshared Tcl value will be freed as a consequence of setting the new value.

Tcl_GetDoubleFromObj attempts to retrieve a double value from the Tcl value *objPtr*. If the attempt succeeds, then **TCL_OK** is returned, and the double value is written to the storage pointed to by *doublePtr*. If the attempt fails, then **TCL_ERROR** is returned, and if *interp* is non-NULL, an error message is left in *interp*. The **Tcl_ObjType** of *objPtr* may be changed to make subsequent calls to **Tcl_GetDoubleFromObj** more efficient.

Keywords: DoubleObj, double, double value, double type, internal representation, value, value type, string representation

Copyright (c) 1996-1997 Sun Microsystems, Inc.

DumpActiveMemory

Name

Tcl_DumpActiveMemory, Tcl_InitMemory, Tcl_ValidateAllMemory - Validated memory allocation interface

Synopsis

#include <tcl.h>

int
Tcl_DumpActiveMemory(*fileName*)

void
Tcl_InitMemory(*interp*)

void
Tcl_ValidateAllMemory(*fileName*, *line*)

Arguments

Tcl_Interp *_interp_ (in)

Tcl interpreter in which to add commands.

const char *_fileName_ (in)

For **Tcl_DumpActiveMemory**, name of the file to which memory information will be written. For **Tcl_ValidateAllMemory**, name of the file from which the call is being made (normally _

FILE_).

int *line* (in)

Line number at which the call to **Tcl_ValidateAllMemory** is made (normally **_LINE_**).

Description

These functions provide access to Tcl memory debugging information. They are only functional when Tcl has been compiled with **TCL_MEM_DEBUG** defined at compile-time. When **TCL_MEM_DEBUG** is not defined, these functions are all no-ops.

Tcl_DumpActiveMemory will output a list of all currently allocated memory to the specified file. The information output for each allocated block of memory is: starting and ending addresses (excluding guard zone), size, source file where **ckalloc** was called to allocate the block and line number in that file. It is especially useful to call **Tcl_DumpActiveMemory** after the Tcl interpreter has been deleted.

Tcl_InitMemory adds the Tcl **memory** command to the interpreter given by *interp*. **Tcl_InitMemory** is called by **Tcl_Main**.

Tcl_ValidateAllMemory forces a validation of the guard zones of all currently allocated blocks of memory. Normally validation of a block occurs when its freed, unless full validation is enabled, in which case validation of all blocks occurs when **ckalloc** and **ckfree** are called. This function forces the validation to occur at any point.

Keywords: DumpActiveMemory, memory, debug

Copyright (c) 1992-1999 Karl Lehenbauer & Mark Diekhans.
Copyright (c) 2000 Scriptics Corporation.
All rights reserved.

Encoding

Name

Tcl_GetEncoding, Tcl_FreeEncoding, Tcl_GetEncodingFromObj, Tcl_ExternalToUtfDString, Tcl_ExternalToUtf, Tcl_UtfToExternalDString, Tcl_UtfToExternal, Tcl_WinTCharToUtf, Tcl_WinUtfToTChar, Tcl_GetEncodingName, Tcl_SetSystemEncoding, Tcl_GetEncodingNameFromEnvironment, Tcl_GetEncodingNames, Tcl_CreateEncoding, Tcl_GetEncodingSearchPath, Tcl_SetEncodingSearchPath, Tcl_GetDefaultEncodingDir, Tcl_SetDefaultEncodingDir - procedures for creating and using encodings

Synopsis

#include <tcl.h>

Tcl_Encoding
Tcl_GetEncoding(*interp*, *name*)

void
Tcl_FreeEncoding(*encoding*)

int
Tcl_GetEncodingFromObj(*interp*, *objPtr*, *encodingPtr*)

char *
Tcl_ExternalToUtfDString(*encoding*, *src*, *srcLen*, *dstPtr*)

char *
Tcl_UtfToExternalDString(*encoding*, *src*, *srcLen*, *dstPtr*)

int
Tcl_ExternalToUtf(*interp*, *encoding*, *src*, *srcLen*, *flags*, *statePtr*, *dst*, *dstLen*, *srcReadPtr*, *dstWrotePtr*, *dstCharsPtr*)

int
Tcl_UtfToExternal(*interp*, *encoding*, *src*, *srcLen*, *flags*, *statePtr*, *dst*, *dstLen*, *srcReadPtr*, *dstWrotePtr*, *dstCharsPtr*)

char *
Tcl_WinTCharToUtf(*tsrc*, *srcLen*, *dstPtr*)

TCHAR *
Tcl_WinUtfToTChar(*src*, *srcLen*, *dstPtr*)

const char *
Tcl_GetEncodingName(*encoding*)

int
Tcl_SetSystemEncoding(*interp*, *name*)

const char *
Tcl_GetEncodingNameFromEnvironment(*bufPtr*)

void
Tcl_GetEncodingNames(*interp*)

Tcl_Encoding
Tcl_CreateEncoding(*typePtr*)

Tcl_Obj *
Tcl_GetEncodingSearchPath()

int
Tcl_SetEncodingSearchPath(*searchPath*)

const char *
Tcl_GetDefaultEncodingDir(*void*)

void
Tcl_SetDefaultEncodingDir(*path*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter to use for error reporting, or NULL if no error reporting is desired.

const char *_name_ (in)
Name of encoding to load.

Tcl_Encoding _encoding_ (in)
The encoding to query, free, or use for converting text. If *encoding* is NULL, the current system encoding is used.

Tcl_Obj *_objPtr_ (in)
Name of encoding to get token for.

Tcl_Encoding *_encodingPtr_ (out)

Points to storage where encoding token is to be written.

const char *_src_ (in)
For the **Tcl_ExternalToUtf** functions, an array of bytes in the specified encoding that are to be converted to UTF-8. For the **Tcl_UtfToExternal** and **Tcl_WinUtfToTChar** functions, an array of UTF-8 characters to be converted to the specified encoding.

const TCHAR *_tsrc_ (in)
An array of Windows TCHAR characters to convert to UTF-8.

int _srcLen_ (in)
Length of *src* or *tsrc* in bytes. If the length is negative, the encoding-specific length of the string is used.

Tcl_DString *_dstPtr_ (out)
Pointer to an uninitialized or free **Tcl_DString** in which the converted result will be stored.

int _flags_ (in)
Various flag bits OR-ed together. **TCL_ENCODING_START** signifies that the source buffer is the first block in a (potentially multi-block) input stream, telling the conversion routine to reset to an initial state and perform any initialization that needs to occur before the first byte is converted. **TCL_ENCODING_END** signifies that the source buffer is the last block in a (potentially multi-block) input stream, telling the conversion routine to perform any finalization that needs to occur after the last byte is converted and then to reset to an initial state. **TCL_ENCODING_STOPONERROR** signifies that the conversion routine should return immediately upon reading a source character that does not exist in the target encoding; otherwise a default fallback character will automatically be substituted.

Tcl_EncodingState *_statePtr_ (in/out)
Used when converting a (generally long or indefinite length) byte stream in a piece-by-piece fashion. The conversion routine stores its current state in *_statePtr_ after *src* (the buffer containing the current piece) has been converted; that state information must be passed back when converting the next piece of the stream so the conversion routine knows what state it was in when it left off at the end of the last piece. May be NULL, in which case the value specified for *flags* is ignored and the source buffer is assumed to contain the complete string to convert.

char *_dst_ (out)
Buffer in which the converted result will be stored. No more than *dstLen* bytes will be stored in *dst*.

int _dstLen_ (in)
The maximum length of the output buffer *dst* in bytes.

int *_srcReadPtr_ (out)
Filled with the number of bytes from *src* that were actually converted. This may be less than the original source length if there was a problem converting some source characters. May be NULL.

int *_dstWrotePtr_ (out)
Filled with the number of bytes that were actually stored in the output buffer as a result of the conversion. May be NULL.

int *_dstCharsPtr_ (out)
Filled with the number of characters that correspond to the number of bytes stored in the output buffer. May be NULL.

Tcl_DString *_bufPtr_ (out)
Storage for the prescribed system encoding name.

const Tcl_EncodingType *_typePtr_ (in)
Structure that defines a new type of encoding.

Tcl_Obj *_searchPath_ (in)
List of filesystem directories in which to search for encoding data files.

const char *_path_ (in)
A path to the location of the encoding file.

Introduction

These routines convert between Tcl's internal character representation, UTF-8, and character representations used by various operating systems or file systems, such as Unicode, ASCII, or Shift-JIS. When operating on strings, such as such as obtaining the names of files or displaying characters using international fonts, the strings must be translated into one or possibly multiple formats that the various system calls can use. For instance, on a Japanese Unix workstation, a user might obtain a filename represented in the EUC-JP file encoding and then translate the characters to the jisx0208 font encoding in order to display the filename in a Tk widget. The purpose of the encoding package is to help bridge the translation gap. UTF-8 provides an intermediate staging ground for all the various encodings. In the example above, text would be translated into UTF-8 from whatever file encoding the operating system is using. Then it would be translated from UTF-8 into whatever font encoding the display routines require.

Some basic encodings are compiled into Tcl. Others can be defined by the user or dynamically loaded from encoding files in a platform-independent manner.

Description

Tcl_GetEncoding finds an encoding given its *name*. The name may refer to a built-in Tcl encoding, a user-defined encoding registered by calling **Tcl_CreateEncoding**, or a dynamically-loadable encoding file. The return value is a token that represents the encoding and can be used in subsequent calls to procedures such as **Tcl_GetEncodingName**, **Tcl_FreeEncoding**, and **Tcl_UtfToExternal**. If the name did not refer to any known or loadable encoding, NULL is returned and an error message is returned in *interp*.

The encoding package maintains a database of all encodings currently in use. The first time *name* is seen, **Tcl_GetEncoding** returns an encoding with a reference count of 1. If the same *name* is requested further times, then the reference count for that encoding is incremented without the overhead of allocating a new encoding and all its associated data structures.

When an *encoding* is no longer needed, **Tcl_FreeEncoding** should be called to release it. When an *encoding* is no longer in use anywhere (i.e., it has been freed as many times as it has been gotten) **Tcl_FreeEncoding** will release all storage the encoding was using and delete it from the database.

Tcl_GetEncodingFromObj treats the string representation of *objPtr* as an encoding name, and finds an encoding with that name, just as **Tcl_GetEncoding** does. When an encoding is found, it is cached within the **objPtr** value for future reference, the **Tcl_Encoding** token is written to the storage pointed to by *encodingPtr*, and the value **TCL_OK** is returned. If no such encoding is found, the value **TCL_ERROR** is returned, and no writing to ***encodingPtr* takes place. Just as with **Tcl_GetEncoding**, the caller should call **Tcl_FreeEncoding** on the resulting encoding token

when that token will no longer be used.

Tcl_ExternalToUtfDString converts a source buffer *src* from the specified *encoding* into UTF-8. The converted bytes are stored in *dstPtr*, which is then null-terminated. The caller should eventually call **Tcl_DStringFree** to free any information stored in *dstPtr*. When converting, if any of the characters in the source buffer cannot be represented in the target encoding, a default fallback character will be used. The return value is a pointer to the value stored in the DString.

Tcl_ExternalToUtf converts a source buffer *src* from the specified *encoding* into UTF-8. Up to *srcLen* bytes are converted from the source buffer and up to *dstLen* converted bytes are stored in *dst*. In all cases, **srcReadPtr* is filled with the number of bytes that were successfully converted from *src* and **dstWrotePtr* is filled with the corresponding number of bytes that were stored in *dst*. The return value is one of the following:

TCL_OK
: All bytes of *src* were converted.

TCL_CONVERT_NOSPACE
: The destination buffer was not large enough for all of the converted data; as many characters as could fit were converted though.

TCL_CONVERT_MULTIBYTE
: The last few bytes in the source buffer were the beginning of a multibyte sequence, but more bytes were needed to complete this sequence. A subsequent call to the conversion routine should pass a buffer containing the unconverted bytes that remained in *src* plus some further bytes from the source stream to properly convert the formerly split-up multibyte sequence.

TCL_CONVERT_SYNTAX
: The source buffer contained an invalid character sequence. This may occur if the input stream has been damaged or if the input encoding method was misidentified.

TCL_CONVERT_UNKNOWN
: The source buffer contained a character that could not be represented in the target encoding and **TCL_ENCODING_STOPONERROR** was specified.

Tcl_UtfToExternalDString converts a source buffer *src* from UTF-8 into the specified *encoding*. The converted bytes are stored in *dstPtr*, which is then terminated with the appropriate encoding-specific null. The caller should eventually call **Tcl_DStringFree** to free any information stored in *dstPtr*. When converting, if any of the characters in the source buffer cannot be represented in the target encoding, a default fallback character will be used. The return value is a pointer to the value stored in the DString.

Tcl_UtfToExternal converts a source buffer *src* from UTF-8 into the specified *encoding*. Up to *srcLen* bytes are converted from the source buffer and up to *dstLen* converted bytes are stored in *dst*. In all cases, **srcReadPtr* is filled with the number of bytes that were successfully converted from *src* and **dstWrotePtr* is filled with the corresponding number of bytes that were stored in *dst*. The return values are the same as the return values for **Tcl_ExternalToUtf**.

Tcl_WinUtfToTChar and **Tcl_WinTCharToUtf** are Windows-only convenience functions for converting between UTF-8 and Windows strings based on the TCHAR type which is by convention a Unicode character on Windows NT.

Tcl_GetEncodingName is roughly the inverse of **Tcl_GetEncoding**. Given an *encoding*, the return value is the *name* argument that was used to create the encoding. The string returned by **Tcl_GetEncodingName** is only guaranteed to persist until the *encoding* is deleted. The caller

must not modify this string.

Tcl_SetSystemEncoding sets the default encoding that should be used whenever the user passes a NULL value for the *encoding* argument to any of the other encoding functions. If *name* is NULL, the system encoding is reset to the default system encoding, **binary**. If the name did not refer to any known or loadable encoding, **TCL_ERROR** is returned and an error message is left in *interp*. Otherwise, this procedure increments the reference count of the new system encoding, decrements the reference count of the old system encoding, and returns **TCL_OK**.

Tcl_GetEncodingNameFromEnvironment provides a means for the Tcl library to report the encoding name it believes to be the correct one to use as the system encoding, based on system calls and examination of the environment suitable for the platform. It accepts *bufPtr*, a pointer to an uninitialized or freed **Tcl_DString** and writes the encoding name to it. The **Tcl_DStringValue** is returned.

Tcl_GetEncodingNames sets the *interp* result to a list consisting of the names of all the encodings that are currently defined or can be dynamically loaded, searching the encoding path specified by **Tcl_SetDefaultEncodingDir**. This procedure does not ensure that the dynamically-loadable encoding files contain valid data, but merely that they exist.

Tcl_CreateEncoding defines a new encoding and registers the C procedures that are called back to convert between the encoding and UTF-8. Encodings created by **Tcl_CreateEncoding** are thereafter visible in the database used by **Tcl_GetEncoding**. Just as with the **Tcl_GetEncoding** procedure, the return value is a token that represents the encoding and can be used in subsequent calls to other encoding functions. **Tcl_CreateEncoding** returns an encoding with a reference count of 1. If an encoding with the specified *name* already exists, then its entry in the database is replaced with the new encoding; the token for the old encoding will remain valid and continue to behave as before, but users of the new token will now call the new encoding procedures.

The *typePtr* argument to **Tcl_CreateEncoding** contains information about the name of the encoding and the procedures that will be called to convert between this encoding and UTF-8. It is defined as follows:

```
typedef struct Tcl_EncodingType {
    const char *encodingName;
    Tcl_EncodingConvertProc *toUtfProc;
    Tcl_EncodingConvertProc *fromUtfProc;
    Tcl_EncodingFreeProc *freeProc;
    ClientData clientData;
    int nullSize;
} Tcl_EncodingType;
```

The *encodingName* provides a string name for the encoding, by which it can be referred in other procedures such as **Tcl_GetEncoding**. The *toUtfProc* refers to a callback procedure to invoke to convert text from this encoding into UTF-8. The *fromUtfProc* refers to a callback procedure to invoke to convert text from UTF-8 into this encoding. The *freeProc* refers to a callback procedure to invoke when this encoding is deleted. The *freeProc* field may be NULL. The *clientData* contains an arbitrary one-word value passed to *toUtfProc*, *fromUtfProc*, and *freeProc* whenever they are called. Typically, this is a pointer to a data structure containing encoding-specific information that can be used by the callback procedures. For instance, two very similar

encodings such as **ascii** and **macRoman** may use the same callback procedure, but use different values of *clientData* to control its behavior. The *nullSize* specifies the number of zero bytes that signify end-of-string in this encoding. It must be **1** (for single-byte or multi-byte encodings like ASCII or Shift-JIS) or **2** (for double-byte encodings like Unicode). Constant-sized encodings with 3 or more bytes per character (such as CNS11643) are not accepted.

The callback procedures *toUtfProc* and *fromUtfProc* should match the type **Tcl_EncodingConvertProc**:

```
typedef int Tcl_EncodingConvertProc(
        ClientData clientData,
        const char *src,
        int srcLen,
        int flags,
        Tcl_EncodingState *statePtr,
        char *dst,
        int dstLen,
        int *srcReadPtr,
        int *dstWrotePtr,
        int *dstCharsPtr);
```

The *toUtfProc* and *fromUtfProc* procedures are called by the **Tcl_ExternalToUtf** or **Tcl_UtfToExternal** family of functions to perform the actual conversion. The *clientData* parameter to these procedures is the same as the *clientData* field specified to **Tcl_CreateEncoding** when the encoding was created. The remaining arguments to the callback procedures are the same as the arguments, documented at the top, to **Tcl_ExternalToUtf** or **Tcl_UtfToExternal**, with the following exceptions. If the *srcLen* argument to one of those high-level functions is negative, the value passed to the callback procedure will be the appropriate encoding-specific string length of *src*. If any of the *srcReadPtr*, *dstWrotePtr*, or *dstCharsPtr* arguments to one of the high-level functions is NULL, the corresponding value passed to the callback procedure will be a non-NULL location.

The callback procedure *freeProc*, if non-NULL, should match the type **Tcl_EncodingFreeProc**:

```
typedef void Tcl_EncodingFreeProc(
        ClientData clientData);
```

This *freeProc* function is called when the encoding is deleted. The *clientData* parameter is the same as the *clientData* field specified to **Tcl_CreateEncoding** when the encoding was created.

Tcl_GetEncodingSearchPath and **Tcl_SetEncodingSearchPath** are called to access and set the list of filesystem directories searched for encoding data files.

The value returned by **Tcl_GetEncodingSearchPath** is the value stored by the last successful call to **Tcl_SetEncodingSearchPath**. If no calls to **Tcl_SetEncodingSearchPath** have occurred, Tcl will compute an initial value based on the environment. There is one encoding search path for the entire process, shared by all threads in the process.

Tcl_SetEncodingSearchPath stores *searchPath* and returns **TCL_OK**, unless *searchPath* is not a valid Tcl list, which causes **TCL_ERROR** to be returned. The elements of *searchPath* are not verified as existing readable filesystem directories. When searching for encoding data files takes place, and non-existent or non-readable filesystem directories on the *searchPath* are silently

ignored.

Tcl_GetDefaultEncodingDir and **Tcl_SetDefaultEncodingDir** are obsolete interfaces best replaced with calls to **Tcl_GetEncodingSearchPath** and **Tcl_SetEncodingSearchPath**. They are called to access and set the first element of the *searchPath* list. Since Tcl searches *searchPath* for encoding data files in list order, these routines establish the "default" directory in which to find encoding data files.

Encoding files

Space would prohibit precompiling into Tcl every possible encoding algorithm, so many encodings are stored on disk as dynamically-loadable encoding files. This behavior also allows the user to create additional encoding files that can be loaded using the same mechanism. These encoding files contain information about the tables and/or escape sequences used to map between an external encoding and Unicode. The external encoding may consist of single-byte, multi-byte, or double-byte characters.

Each dynamically-loadable encoding is represented as a text file. The initial line of the file, beginning with a "#" symbol, is a comment that provides a human-readable description of the file. The next line identifies the type of encoding file. It can be one of the following letters:

[1] S
A single-byte encoding, where one character is always one byte long in the encoding. An example is **iso8859-1**, used by many European languages.

[2] D
A double-byte encoding, where one character is always two bytes long in the encoding. An example is **big5**, used for Chinese text.

[3] M
A multi-byte encoding, where one character may be either one or two bytes long. Certain bytes are lead bytes, indicating that another byte must follow and that together the two bytes represent one character. Other bytes are not lead bytes and represent themselves. An example is **shiftjis**, used by many Japanese computers.

[4] E
An escape-sequence encoding, specifying that certain sequences of bytes do not represent characters, but commands that describe how following bytes should be interpreted. The rest of the lines in the file depend on the type. Cases [1], [2], and [3] are collectively referred to as table-based encoding files. The lines in a table-based encoding file are in the same format as this example taken from the **shiftjis** encoding (this is not the complete file):

```
# Encoding file: shiftjis, multi-byte
M
003F 0 40
00
000000010002000300040005000600070008000900 0A000B000C000D000E000F
001000110012001300140015001600170018001900 1A001B001C001D001E001F
002000210022002300240025002600270028002900 2A002B002C002D002E002F
003000310032003300340035003600370038003900 3A003B003C003D003E003F
004000410042004300440045004600470048004900 4A004B004C004D004E004F
005000510052005300540055005600570058005900 5A005B005C005D005E005F
```

```
006000610062006300640065006600670068006900 6A006B006C006D006E006F
007000710072007300740075007600770078007900 7A007B007C007D203E007F
008000000000000000000000000000000000000000 0000000000000000000000
000000000000000000000000000000000000000000 0000000000000000000000
0000FF61FF62FF63FF64FF65FF66FF67FF68FF69FF 6AFF6BFF6CFF6DFF6EFF6F
FF70FF71FF72FF73FF74FF75FF76FF77FF78FF79FF 7AFF7BFF7CFF7DFF7EFF7F
FF80FF81FF82FF83FF84FF85FF86FF87FF88FF89FF 8AFF8BFF8CFF8DFF8EFF8F
FF90FF91FF92FF93FF94FF95FF96FF97FF98FF99FF 9AFF9BFF9CFF9DFF9EFF9F
000000000000000000000000000000000000000000 0000000000000000000000
000000000000000000000000000000000000000000 0000000000000000000000
81
000000000000000000000000000000000000000000 0000000000000000000000
000000000000000000000000000000000000000000 0000000000000000000000
000000000000000000000000000000000000000000 0000000000000000000000
000000000000000000000000000000000000000000 0000000000000000000000
300030013002FF0CFF0E30FBFF1AFF1BFF1FFF0130 9B309C00B4FF4000A8FF3E
FFE3FF3F30FD30FE309D309E30034EDD30053006300730FC20152010FF0F005C
301C2016FF5C2026202520182019201C201DFF08FF 0930143015FF3BFF3DFF5B
FF5D30083009300A300B300C300D300E300F30103011FF0B221200B100D70000
00F7FF1D2260FF1CFF1E22662267221E22342642264000B0203220332103FFE5
FF0400A200A3FF05FF03FF06FF0AFF2000A72606260525CB25CF25CE25C725C6
25A125A025B325B225BD25BC203B3012219221902191219330130000000000000
000000000000000000000000000002208220B2286228722822283222A2229
000000000000000000000000000002227222800AC21D221D4220022030000
000000000000000000000000000000000000222022A52312220222072261
2252226A226B221A223D221D2235222B222C0000000000000000000000000000
212B2030266F266D266A2020202100B60000000000000025EF000000000000
```

The third line of the file is three numbers. The first number is the fallback character (in base 16) to use when converting from UTF-8 to this encoding. The second number is a **1** if this file represents the encoding for a symbol font, or **0** otherwise. The last number (in base 10) is how many pages of data follow.

Subsequent lines in the example above are pages that describe how to map from the encoding into 2-byte Unicode. The first line in a page identifies the page number. Following it are 256 double-byte numbers, arranged as 16 rows of 16 numbers. Given a character in the encoding, the high byte of that character is used to select which page, and the low byte of that character is used as an index to select one of the double-byte numbers in that page - the value obtained being the corresponding Unicode character. By examination of the example above, one can see that the characters 0x7E and 0x8163 in **shiftjis** map to 203E and 2026 in Unicode, respectively.

Following the first page will be all the other pages, each in the same format as the first: one number identifying the page followed by 256 double-byte Unicode characters. If a character in the encoding maps to the Unicode character 0000, it means that the character does not actually exist. If all characters on a page would map to 0000, that page can be omitted.

Case [4] is the escape-sequence encoding file. The lines in an this type of file are in the same format as this example taken from the **iso2022-jp** encoding:

```
# Encoding file: iso2022-jp, escape-driven
E
init                    {}
final                   {}
iso8859-1       \x1b(B
jis0201                 \x1b(J
jis0208                 \x1b$@
jis0208                 \x1b$B
jis0212                 \x1b$(D
gb2312                  \x1b$A
ksc5601                 \x1b$(C
```

In the file, the first column represents an option and the second column is the associated value. **init** is a string to emit or expect before the first character is converted, while **final** is a string to emit or expect after the last character. All other options are names of table-based encodings; the associated value is the escape-sequence that marks that encoding. Tcl syntax is used for the values; in the above example, for instance, "**{}**" represents the empty string and "**\x1b**" represents character 27.

When **Tcl_GetEncoding** encounters an encoding *name* that has not been loaded, it attempts to load an encoding file called *name***.enc** from the **encoding** subdirectory of each directory that Tcl searches for its script library. If the encoding file exists, but is malformed, an error message will be left in *interp*.

Keywords: Encoding, utf, convert

Copyright (c) 1997-1998 Sun Microsystems, Inc.

Ensemble

Name

Tcl_CreateEnsemble, Tcl_FindEnsemble, Tcl_GetEnsembleFlags, Tcl_GetEnsembleMappingDict, Tcl_GetEnsembleNamespace, Tcl_GetEnsembleParameterList, Tcl_GetEnsembleUnknownHandler, Tcl_GetEnsembleSubcommandList, Tcl_IsEnsemble, Tcl_SetEnsembleFlags, Tcl_SetEnsembleMappingDict, Tcl_SetEnsembleParameterList, Tcl_SetEnsembleSubcommandList, Tcl_SetEnsembleUnknownHandler - manipulate ensemble commands

Synopsis

#include <tcl.h>

Tcl_Command
Tcl_CreateEnsemble(*interp*, *name*, *namespacePtr*, *ensFlags*)

Tcl_Command
Tcl_FindEnsemble(*interp*, *cmdNameObj*, *flags*)

int
Tcl_IsEnsemble(*token*)

int
Tcl_GetEnsembleFlags(*interp*, *token*, *ensFlagsPtr*)

int

Tcl_SetEnsembleFlags(*interp*, *token*, *ensFlags*)

int
Tcl_GetEnsembleMappingDict(*interp*, *token*, *dictObjPtr*)

int
Tcl_SetEnsembleMappingDict(*interp*, *token*, *dictObj*)

int
Tcl_GetEnsembleParameterList(*interp*, *token*, *listObjPtr*)

int
Tcl_SetEnsembleParameterList(*interp*, *token*, *listObj*)

int
Tcl_GetEnsembleSubcommandList(*interp*, *token*, *listObjPtr*)

int
Tcl_SetEnsembleSubcommandList(*interp*, *token*, *listObj*)

int
Tcl_GetEnsembleUnknownHandler(*interp*, *token*, *listObjPtr*)

int
Tcl_SetEnsembleUnknownHandler(*interp*, *token*, *listObj*)

int
Tcl_GetEnsembleNamespace(*interp*, *token*, *namespacePtrPtr*)

Arguments

Tcl_Interp *_interp_ (in/out)
The interpreter in which the ensemble is to be created or found. Also where error result messages are written. The functions whose names start with **Tcl_GetEnsemble** may have a NULL for the *interp*, but all other functions must not.

const char *_name_ (in)
The name of the ensemble command to be created.

Tcl_Namespace *_namespacePtr_ (in)
The namespace to which the ensemble command is to be bound, or NULL for the current namespace.

int _ensFlags_ (in)
An OR'ed set of flag bits describing the basic configuration of the ensemble. Currently only one bit has meaning, **TCL_ENSEMBLE_PREFIX**, which is present when the ensemble command should also match unambiguous prefixes of subcommands.

Tcl_Obj *_cmdNameObj_ (in)
A value holding the name of the ensemble command to look up.

int _flags_ (in)
An OR'ed set of flag bits controlling the behavior of **Tcl_FindEnsemble**. Currently only **TCL_LEAVE_ERR_MSG** is supported.

Tcl_Command _token_ (in)
A normal command token that refers to an ensemble command, or which you wish to use for testing as an ensemble command in **Tcl_IsEnsemble**.

int *_ensFlagsPtr_ (out)

Pointer to a variable into which to write the current ensemble flag bits; currently only the bit **TCL_ENSEMBLE_PREFIX** is defined.

Tcl_Obj *_dictObj_ (in)
A dictionary value to use for the subcommand to implementation command prefix mapping dictionary in the ensemble. May be NULL if the mapping dictionary is to be removed.

Tcl_Obj **_dictObjPtr_ (out)
Pointer to a variable into which to write the current ensemble mapping dictionary.

Tcl_Obj *_listObj_ (in)
A list value to use for the list of formal pre-subcommand parameters, the defined list of subcommands in the dictionary or the unknown subcommand handler command prefix. May be NULL if the subcommand list or unknown handler are to be removed.

Tcl_Obj **_listObjPtr_ (out)
Pointer to a variable into which to write the current list of formal pre-subcommand parameters, the defined list of subcommands or the current unknown handler prefix.

Tcl_Namespace **_namespacePtrPtr_ (out)
Pointer to a variable into which to write the handle of the namespace to which the ensemble is bound.

Description

An ensemble is a command, bound to some namespace, which consists of a collection of subcommands implemented by other Tcl commands. The first argument to the ensemble command is always interpreted as a selector that states what subcommand to execute.

Ensembles are created using **Tcl_CreateEnsemble**, which takes four arguments: the interpreter to work within, the name of the ensemble to create, the namespace within the interpreter to bind the ensemble to, and the default set of ensemble flags. The result of the function is the command token for the ensemble, which may be used to further configure the ensemble using the API described below in **ENSEMBLE PROPERTIES**.

Given the name of an ensemble command, the token for that command may be retrieved using **Tcl_FindEnsemble**. If the given command name (in *cmdNameObj*) does not refer to an ensemble command, the result of the function is NULL and (if the **TCL_LEAVE_ERR_MSG** bit is set in *flags*) an error message is left in the interpreter result.

A command token may be checked to see if it refers to an ensemble using **Tcl_IsEnsemble**. This returns 1 if the token refers to an ensemble, or 0 otherwise.

Ensemble properties

Every ensemble has four read-write properties and a read-only property. The properties are:

flags (read-write)
The set of flags for the ensemble, expressed as a bit-field. Currently, the only public flag is **TCL_ENSEMBLE_PREFIX** which is set when unambiguous prefixes of subcommands are permitted to be resolved to implementations as well as exact matches. The flags may be read and written using **Tcl_GetEnsembleFlags** and **Tcl_SetEnsembleFlags** respectively. The result of both of those functions is a Tcl result code (**TCL_OK**, or **TCL_ERROR** if the token does not refer to an ensemble).

mapping dictionary (read-write)
A dictionary containing a mapping from subcommand names to lists of words to use as a

command prefix (replacing the first two words of the command which are the ensemble command itself and the subcommand name), or NULL if every subcommand is to be mapped to the command with the same unqualified name in the ensemble's bound namespace. Defaults to NULL. May be read and written using **Tcl_GetEnsembleMappingDict** and **Tcl_SetEnsembleMappingDict** respectively. The result of both of those functions is a Tcl result code (**TCL_OK**, or **TCL_ERROR** if the token does not refer to an ensemble) and the dictionary obtained from **Tcl_GetEnsembleMappingDict** should always be treated as immutable even if it is unshared. All command names in prefixes set via **Tcl_SetEnsembleMappingDict** must be fully qualified.

formal pre-subcommand parameter list (read-write)
A list of formal parameter names (the names only being used when generating error messages) that come at invocation of the ensemble between the name of the ensemble and the subcommand argument. NULL (the default) is equivalent to the empty list. May be read and written using **Tcl_GetEnsembleParameterList** and **Tcl_SetEnsembleParameterList** respectively. The result of both of those functions is a Tcl result code (**TCL_OK**, or **TCL_ERROR** if the token does not refer to an ensemble) and the dictionary obtained from **Tcl_GetEnsembleParameterList** should always be treated as immutable even if it is unshared.

subcommand list (read-write)
A list of all the subcommand names for the ensemble, or NULL if this is to be derived from either the keys of the mapping dictionary (see above) or (if that is also NULL) from the set of commands exported by the bound namespace. May be read and written using **Tcl_GetEnsembleSubcommandList** and **Tcl_SetEnsembleSubcommandList** respectively. The result of both of those functions is a Tcl result code (**TCL_OK**, or **TCL_ERROR** if the token does not refer to an ensemble) and the list obtained from **Tcl_GetEnsembleSubcommandList** should always be treated as immutable even if it is unshared.

unknown subcommand handler command prefix (read-write)
A list of words to prepend on the front of any subcommand when the subcommand is unknown to the ensemble (according to the current prefix handling rule); see the **namespace ensemble** command for more details. If NULL, the default behavior - generate a suitable error message - will be used when an unknown subcommand is encountered. May be read and written using **Tcl_GetEnsembleUnknownHandler** and **Tcl_SetEnsembleUnknownHandler** respectively. The result of both functions is a Tcl result code (**TCL_OK**, or **TCL_ERROR** if the token does not refer to an ensemble) and the list obtained from **Tcl_GetEnsembleUnknownHandler** should always be treated as immutable even if it is unshared.

bound namespace (read-only)
The namespace to which the ensemble is bound; when the namespace is deleted, so too will the ensemble, and this namespace is also the namespace whose list of exported commands is used if both the mapping dictionary and the subcommand list properties are NULL. May be read using **Tcl_GetEnsembleNamespace** which returns a Tcl result code (**TCL_OK**, or **TCL_ERROR** if the token does not refer to an ensemble).

Keywords: Ensemble, command

Copyright (c) 2005 Donal K. Fellows

Environment

Name

Tcl_PutEnv - procedures to manipulate the environment

Synopsis

#include <tcl.h>

int
Tcl_PutEnv(*assignment*)

Arguments

const char ****assignment* **(in)**

Info about environment variable in the format "*NAME*=*value*". The *assignment* argument is in the system encoding.

Description

Tcl_PutEnv sets an environment variable. The information is passed in a single string of the form "*NAME*=*value*". This procedure is intended to be a stand-in for the UNIX **putenv** system call. All Tcl-based applications using **putenv** should redefine it to **Tcl_PutEnv** so that they will interface properly to the Tcl runtime.

Keywords: Environment, variable

Copyright (c) 1997-1998 Sun Microsystems, Inc.

Eval

Name

Tcl_EvalObjEx, Tcl_EvalFile, Tcl_EvalObjv, Tcl_Eval, Tcl_EvalEx, Tcl_GlobalEval, Tcl_GlobalEvalObj, Tcl_VarEval, Tcl_VarEvalVA - execute Tcl scripts

Synopsis

#include <tcl.h>

int
Tcl_EvalObjEx(*interp*, *objPtr*, *flags*)

int
Tcl_EvalFile(*interp*, *fileName*)

int
Tcl_EvalObjv(*interp*, *objc*, *objv*, *flags*)

int
Tcl_Eval(*interp*, *script*)

int
Tcl_EvalEx(*interp*, *script*, *numBytes*, *flags*)

int
Tcl_GlobalEval(*interp*, *script*)

int

Tcl_GlobalEvalObj(*interp*, *objPtr*)

int
Tcl_VarEval(*interp*, *part*, *part*, ... (**char** *)**NULL**)

int
Tcl_VarEvalVA(*interp*, *argList*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter in which to execute the script. The interpreter's result is modified to hold the result or error message from the script.

Tcl_Obj *_objPtr_ (in)
A Tcl value containing the script to execute.

int _flags_ (in)
OR'ed combination of flag bits that specify additional options. **TCL_EVAL_GLOBAL** and **TCL_EVAL_DIRECT** are currently supported.

const char *_fileName_ (in)
Name of a file containing a Tcl script.

int _objc_ (in)
The number of values in the array pointed to by *objv*; this is also the number of words in the command.

Tcl_Obj **_objv_ (in)
Points to an array of pointers to values; each value holds the value of a single word in the command to execute.

int _numBytes_ (in)
The number of bytes in *script*, not including any null terminating character. If -1, then all characters up to the first null byte are used.

const char *_script_ (in)
Points to first byte of script to execute (null-terminated and UTF-8).

const char *_part_ (in)
String forming part of a Tcl script.

va_list _argList_ (in)
An argument list which must have been initialized using **va_start**, and cleared using **va_end**.

Description

The procedures described here are invoked to execute Tcl scripts in various forms. **Tcl_EvalObjEx** is the core procedure and is used by many of the others. It executes the commands in the script stored in *objPtr* until either an error occurs or the end of the script is reached. If this is the first time *objPtr* has been executed, its commands are compiled into bytecode instructions which are then executed. The bytecodes are saved in *objPtr* so that the compilation step can be skipped if the value is evaluated again in the future.

The return value from **Tcl_EvalObjEx** (and all the other procedures described here) is a Tcl completion code with one of the values **TCL_OK**, **TCL_ERROR**, **TCL_RETURN**, **TCL_BREAK**, or **TCL_CONTINUE**, or possibly some other integer value originating in an extension. In addition, a result value or error message is left in *interp*'s result; it can be retrieved using

Tcl_GetObjResult.

Tcl_EvalFile reads the file given by *fileName* and evaluates its contents as a Tcl script. It returns the same information as **Tcl_EvalObjEx**. If the file could not be read then a Tcl error is returned to describe why the file could not be read. The eofchar for files is "\32" (^Z) for all platforms. If you require a "^Z" in code for string comparison, you can use "\032" or "\u001a", which will be safely substituted by the Tcl interpreter into "^Z".

Tcl_EvalObjv executes a single preparsed command instead of a script. The *objc* and *objv* arguments contain the values of the words for the Tcl command, one word in each value in *objv*. **Tcl_EvalObjv** evaluates the command and returns a completion code and result just like **Tcl_EvalObjEx**. The caller of **Tcl_EvalObjv** has to manage the reference count of the elements of *objv*, insuring that the values are valid until **Tcl_EvalObjv** returns.

Tcl_Eval is similar to **Tcl_EvalObjEx** except that the script to be executed is supplied as a string instead of a value and no compilation occurs. The string should be a proper UTF-8 string as converted by **Tcl_ExternalToUtfDString** or **Tcl_ExternalToUtf** when it is known to possibly contain upper ASCII characters whose possible combinations might be a UTF-8 special code. The string is parsed and executed directly (using **Tcl_EvalObjv**) instead of compiling it and executing the bytecodes. In situations where it is known that the script will never be executed again, **Tcl_Eval** may be faster than **Tcl_EvalObjEx**. **Tcl_Eval** returns a completion code and result just like **Tcl_EvalObjEx**. Note: for backward compatibility with versions before Tcl 8.0, **Tcl_Eval** copies the value result in *interp* to *interp->result* (use is deprecated) where it can be accessed directly. This makes **Tcl_Eval** somewhat slower than **Tcl_EvalEx**, which does not do the copy.

Tcl_EvalEx is an extended version of **Tcl_Eval** that takes additional arguments *numBytes* and *flags*. For the efficiency reason given above, **Tcl_EvalEx** is generally preferred over **Tcl_Eval**.

Tcl_GlobalEval and **Tcl_GlobalEvalObj** are older procedures that are now deprecated. They are similar to **Tcl_EvalEx** and **Tcl_EvalObjEx** except that the script is evaluated in the global namespace and its variable context consists of global variables only (it ignores any Tcl procedures that are active). These functions are equivalent to using the **TCL_EVAL_GLOBAL** flag (see below).

Tcl_VarEval takes any number of string arguments of any length, concatenates them into a single string, then calls **Tcl_Eval** to execute that string as a Tcl command. It returns the result of the command and also modifies *interp->result* in the same way as **Tcl_Eval**. The last argument to **Tcl_VarEval** must be (char *)NULL to indicate the end of arguments.

Tcl_VarEvalVA is the same as **Tcl_VarEval** except that instead of taking a variable number of arguments it takes an argument list. **Tcl_VarEvalVA** is now deprecated.

Flag bits

Any OR'ed combination of the following values may be used for the *flags* argument to procedures such as **Tcl_EvalObjEx**:

TCL_EVAL_DIRECT

This flag is only used by **Tcl_EvalObjEx**; it is ignored by other procedures. If this flag bit is set, the script is not compiled to bytecodes; instead it is executed directly as is done by **Tcl_EvalEx**. The **TCL_EVAL_DIRECT** flag is useful in situations where the contents of a value are going to change immediately, so the bytecodes will not be reused in a future execution. In this case, it is faster to execute the script directly.

TCL_EVAL_GLOBAL
If this flag is set, the script is evaluated in the global namespace instead of the current namespace and its variable context consists of global variables only (it ignores any Tcl procedures that are active).

Miscellaneous details

During the processing of a Tcl command it is legal to make nested calls to evaluate other commands (this is how procedures and some control structures are implemented). If a code other than **TCL_OK** is returned from a nested **Tcl_EvalObjEx** invocation, then the caller should normally return immediately, passing that same return code back to its caller, and so on until the top-level application is reached. A few commands, like **for**, will check for certain return codes, like **TCL_BREAK** and **TCL_CONTINUE**, and process them specially without returning.

Tcl_EvalObjEx keeps track of how many nested **Tcl_EvalObjEx** invocations are in progress for *interp*. If a code of **TCL_RETURN**, **TCL_BREAK**, or **TCL_CONTINUE** is about to be returned from the topmost **Tcl_EvalObjEx** invocation for *interp*, it converts the return code to **TCL_ERROR** and sets *interp*'s result to an error message indicating that the **return**, **break**, or **continue** command was invoked in an inappropriate place. This means that top-level applications should never see a return code from **Tcl_EvalObjEx** other than **TCL_OK** or **TCL_ERROR**.

Keywords: Eval, execute, file, global, result, script, value

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.
Copyright (c) 2000 Scriptics Corporation.

Exit

Name

Tcl_Exit, Tcl_Finalize, Tcl_CreateExitHandler, Tcl_DeleteExitHandler, Tcl_ExitThread, Tcl_FinalizeThread, Tcl_CreateThreadExitHandler, Tcl_DeleteThreadExitHandler, Tcl_SetExitProc - end the application or thread (and invoke exit handlers)

Synopsis

#include <tcl.h>

Tcl_Exit(*status*)

Tcl_Finalize()

Tcl_CreateExitHandler(*proc*, *clientData*)

Tcl_DeleteExitHandler(*proc*, *clientData*)

Tcl_ExitThread(*status*)

Tcl_FinalizeThread()

Tcl_CreateThreadExitHandler(*proc*, *clientData*)

Tcl_DeleteThreadExitHandler(*proc*, *clientData*)

Tcl_ExitProc *
Tcl_SetExitProc(*proc*)

Arguments

int *status* (in)
Provides information about why the application or thread exited. Exact meaning may be platform-specific. 0 usually means a normal exit, any nonzero value usually means that an error occurred.

Tcl_ExitProc **proc* (in)
Procedure to invoke before exiting application, or (for **Tcl_SetExitProc**) NULL to uninstall the current application exit procedure.

ClientData *clientData* (in)
Arbitrary one-word value to pass to *proc*.

Description

The procedures described here provide a graceful mechanism to end the execution of a **Tcl** application. Exit handlers are invoked to cleanup the application's state before ending the execution of **Tcl** code.

Invoke **Tcl_Exit** to end a **Tcl** application and to exit from this process. This procedure is invoked by the **exit** command, and can be invoked anyplace else to terminate the application. No-one should ever invoke the **exit** system procedure directly; always invoke **Tcl_Exit** instead, so that it can invoke exit handlers. Note that if other code invokes **exit** system procedure directly, or otherwise causes the application to terminate without calling **Tcl_Exit**, the exit handlers will not be run. **Tcl_Exit** internally invokes the **exit** system call, thus it never returns control to its caller. If an application exit handler has been installed (see **Tcl_SetExitProc**), that handler is invoked with an argument consisting of the exit status (cast to ClientData); the application exit handler should not return control to Tcl.

Tcl_Finalize is similar to **Tcl_Exit** except that it does not exit from the current process. It is useful for cleaning up when a process is finished using **Tcl** but wishes to continue executing, and when **Tcl** is used in a dynamically loaded extension that is about to be unloaded. Your code should always invoke **Tcl_Finalize** when **Tcl** is being unloaded, to ensure proper cleanup. **Tcl_Finalize** can be safely called more than once.

Tcl_ExitThread is used to terminate the current thread and invoke per-thread exit handlers. This finalization is done by **Tcl_FinalizeThread**, which you can call if you just want to clean up per-thread state and invoke the thread exit handlers. **Tcl_Finalize** calls **Tcl_FinalizeThread** for the current thread automatically.

Tcl_CreateExitHandler arranges for *proc* to be invoked by **Tcl_Finalize** and **Tcl_Exit**. **Tcl_CreateThreadExitHandler** arranges for *proc* to be invoked by **Tcl_FinalizeThread** and **Tcl_ExitThread**. This provides a hook for cleanup operations such as flushing buffers and freeing global memory. *Proc* should match the type **Tcl_ExitProc**:

```
typedef void Tcl_ExitProc(
        ClientData clientData);
```

The *clientData* parameter to *proc* is a copy of the *clientData* argument given to **Tcl_CreateExitHandler** or **Tcl_CreateThreadExitHandler** when the callback was created. Typically, *clientData* points to a data structure containing application-specific information about what to do in *proc*.

Tcl_DeleteExitHandler and **Tcl_DeleteThreadExitHandler** may be called to delete a previously-created exit handler. It removes the handler indicated by *proc* and *clientData* so that no call to *proc* will be made. If no such handler exists then **Tcl_DeleteExitHandler** or **Tcl_DeleteThreadExitHandler** does nothing.

Tcl_Finalize and **Tcl_Exit** execute all registered exit handlers, in reverse order from the order in which they were registered. This matches the natural order in which extensions are loaded and unloaded; if extension **A** loads extension **B**, it usually unloads **B** before it itself is unloaded. If extension **A** registers its exit handlers before loading extension **B**, this ensures that any exit handlers for **B** will be executed before the exit handlers for **A**.

Tcl_Finalize and **Tcl_Exit** call **Tcl_FinalizeThread** and the thread exit handlers *after* the process-wide exit handlers. This is because thread finalization shuts down the I/O channel system, so any attempt at I/O by the global exit handlers will vanish into the bitbucket.

Tcl_SetExitProc installs an application exit handler, returning the previously-installed application exit handler or NULL if no application handler was installed. If an application exit handler is installed, that exit handler takes over complete responsibility for finalization of Tcl's subsystems via **Tcl_Finalize** at an appropriate time. The argument passed to *proc* when it is invoked will be the exit status code (as passed to **Tcl_Exit**) cast to a ClientData value.

Keywords: Exit, abort, callback, cleanup, dynamic loading, end application, unloading, thread

Copyright (c) 1995-1996 Sun Microsystems, Inc.

ExprLong

Name

Tcl_ExprLong, Tcl_ExprDouble, Tcl_ExprBoolean, Tcl_ExprString - evaluate an expression

Synopsis

#include <tcl.h>

int
Tcl_ExprLong(*interp*, *expr*, *longPtr*)

int
Tcl_ExprDouble(*interp*, *expr*, *doublePtr*)

int
Tcl_ExprBoolean(*interp*, *expr*, *booleanPtr*)

int
Tcl_ExprString(*interp*, *expr*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter in whose context to evaluate *expr*.

const char *_expr_ (in)
Expression to be evaluated.

long *_longPtr_ (out)
Pointer to location in which to store the integer value of the expression.

int *_doublePtr_ (out)

Pointer to location in which to store the floating-point value of the expression.

int *_booleanPtr_ (out)

Pointer to location in which to store the 0/1 boolean value of the expression.

Description

These four procedures all evaluate the expression given by the *expr* argument and return the result in one of four different forms. The expression can have any of the forms accepted by the **expr** command. Note that these procedures have been largely replaced by the value-based procedures **Tcl_ExprLongObj**, **Tcl_ExprDoubleObj**, **Tcl_ExprBooleanObj**, and **Tcl_ExprObj**. Those value-based procedures evaluate an expression held in a Tcl value instead of a string. The value argument can retain an internal representation that is more efficient to execute.

The *interp* argument refers to an interpreter used to evaluate the expression (e.g. for variables and nested Tcl commands) and to return error information.

For all of these procedures the return value is a standard Tcl result: **TCL_OK** means the expression was successfully evaluated, and **TCL_ERROR** means that an error occurred while evaluating the expression. If **TCL_ERROR** is returned then the interpreter's result will hold a message describing the error. If an error occurs while executing a Tcl command embedded in the expression then that error will be returned.

If the expression is successfully evaluated, then its value is returned in one of four forms, depending on which procedure is invoked. **Tcl_ExprLong** stores an integer value at **longPtr*. If the expression's actual value is a floating-point number, then it is truncated to an integer. If the expression's actual value is a non-numeric string then an error is returned.

Tcl_ExprDouble stores a floating-point value at **doublePtr*. If the expression's actual value is an integer, it is converted to floating-point. If the expression's actual value is a non-numeric string then an error is returned.

Tcl_ExprBoolean stores a 0/1 integer value at **booleanPtr*. If the expression's actual value is an integer or floating-point number, then they store 0 at **booleanPtr* if the value was zero and 1 otherwise. If the expression's actual value is a non-numeric string then it must be one of the values accepted by **Tcl_GetBoolean** such as "yes" or "no", or else an error occurs.

Tcl_ExprString returns the value of the expression as a string stored in the interpreter's result.

Keywords: ExprLong, boolean, double, evaluate, expression, integer, value, string

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

ExprLongObj

Name

Tcl_ExprLongObj, Tcl_ExprDoubleObj, Tcl_ExprBooleanObj, Tcl_ExprObj - evaluate an expression

Synopsis

#include <tcl.h>

int
Tcl_ExprLongObj(*interp*, *objPtr*, *longPtr*)

int

Tcl_ExprDoubleObj(*interp*, *objPtr*, *doublePtr*)

int
Tcl_ExprBooleanObj(*interp*, *objPtr*, *booleanPtr*)

int
Tcl_ExprObj(*interp*, *objPtr*, *resultPtrPtr*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter in whose context to evaluate *objPtr*.

Tcl_Obj *_objPtr_ (in)
Pointer to a value containing the expression to evaluate.

long *_longPtr_ (out)
Pointer to location in which to store the integer value of the expression.

int *_doublePtr_ (out)
Pointer to location in which to store the floating-point value of the expression.

int *_booleanPtr_ (out)
Pointer to location in which to store the 0/1 boolean value of the expression.

Tcl_Obj **_resultPtrPtr_ (out)
Pointer to location in which to store a pointer to the value that is the result of the expression.

Description

These four procedures all evaluate an expression, returning the result in one of four different forms. The expression is given by the *objPtr* argument, and it can have any of the forms accepted by the **expr** command.

The *interp* argument refers to an interpreter used to evaluate the expression (e.g. for variables and nested Tcl commands) and to return error information.

For all of these procedures the return value is a standard Tcl result: **TCL_OK** means the expression was successfully evaluated, and **TCL_ERROR** means that an error occurred while evaluating the expression. If **TCL_ERROR** is returned, then a message describing the error can be retrieved using **Tcl_GetObjResult**. If an error occurs while executing a Tcl command embedded in the expression then that error will be returned.

If the expression is successfully evaluated, then its value is returned in one of four forms, depending on which procedure is invoked. **Tcl_ExprLongObj** stores an integer value at **longPtr*. If the expression's actual value is a floating-point number, then it is truncated to an integer. If the expression's actual value is a non-numeric string then an error is returned.

Tcl_ExprDoubleObj stores a floating-point value at **doublePtr*. If the expression's actual value is an integer, it is converted to floating-point. If the expression's actual value is a non-numeric string then an error is returned.

Tcl_ExprBooleanObj stores a 0/1 integer value at **booleanPtr*. If the expression's actual value is an integer or floating-point number, then they store 0 at **booleanPtr* if the value was zero and 1 otherwise. If the expression's actual value is a non-numeric string then it must be one of the values accepted by **Tcl_GetBoolean** such as "yes" or "no", or else an error occurs.

If **Tcl_ExprObj** successfully evaluates the expression, it stores a pointer to the Tcl value containing the expression's value at **resultPtrPtr*. In this case, the caller is responsible for calling

Tcl_DecrRefCount to decrement the value's reference count when it is finished with the value.

Keywords: ExprLongObj, boolean, double, evaluate, expression, integer, value, string

Copyright (c) 1996-1997 Sun Microsystems, Inc.

FileSystem

Name

Tcl_FSRegister, Tcl_FSUnregister, Tcl_FSData, Tcl_FSMountsChanged, Tcl_FSGetFileSystemForPath, Tcl_FSGetPathType, Tcl_FSCopyFile, Tcl_FSCopyDirectory, Tcl_FSCreateDirectory, Tcl_FSDeleteFile, Tcl_FSRemoveDirectory, Tcl_FSRenameFile, Tcl_FSListVolumes, Tcl_FSEvalFile, Tcl_FSEvalFileEx, Tcl_FSLoadFile, Tcl_FSUnloadFile, Tcl_FSMatchInDirectory, Tcl_FSLink, Tcl_FSLstat, Tcl_FSUtime, Tcl_FSFileAttrsGet, Tcl_FSFileAttrsSet, Tcl_FSFileAttrStrings, Tcl_FSStat, Tcl_FSAccess, Tcl_FSOpenFileChannel, Tcl_FSGetCwd, Tcl_FSChdir, Tcl_FSPathSeparator, Tcl_FSJoinPath, Tcl_FSSplitPath, Tcl_FSEqualPaths, Tcl_FSGetNormalizedPath, Tcl_FSJoinToPath, Tcl_FSConvertToPathType, Tcl_FSGetInternalRep, Tcl_FSGetTranslatedPath, Tcl_FSGetTranslatedStringPath, Tcl_FSNewNativePath, Tcl_FSGetNativePath, Tcl_FSFileSystemInfo, Tcl_GetAccessTimeFromStat, Tcl_GetBlockSizeFromStat, Tcl_GetBlocksFromStat, Tcl_GetChangeTimeFromStat, Tcl_GetDeviceTypeFromStat, Tcl_GetFSDeviceFromStat, Tcl_GetFSInodeFromStat, Tcl_GetGroupIdFromStat, Tcl_GetLinkCountFromStat, Tcl_GetModeFromStat, Tcl_GetModificationTimeFromStat, Tcl_GetSizeFromStat, Tcl_GetUserIdFromStat, Tcl_AllocStatBuf - procedures to interact with any filesystem

Synopsis

#include <tcl.h>

int
Tcl_FSRegister(*clientData*, *fsPtr*)

int
Tcl_FSUnregister(*fsPtr*)

void *
Tcl_FSData(*fsPtr*)

Tcl_FSMountsChanged(*fsPtr*)

const Tcl_Filesystem *
Tcl_FSGetFileSystemForPath(*pathPtr*)

Tcl_PathType
Tcl_FSGetPathType(*pathPtr*)

int
Tcl_FSCopyFile(*srcPathPtr*, *destPathPtr*)

int
Tcl_FSCopyDirectory(*srcPathPtr*, *destPathPtr*, *errorPtr*)

int
Tcl_FSCreateDirectory(*pathPtr*)

int
Tcl_FSDeleteFile(*pathPtr*)

int
Tcl_FSRemoveDirectory(*pathPtr*, *recursive*, *errorPtr*)

int
Tcl_FSRenameFile(*srcPathPtr*, *destPathPtr*)

Tcl_Obj *
Tcl_FSListVolumes(*void*)

int
Tcl_FSEvalFileEx(*interp*, *pathPtr*, *encodingName*)

int
Tcl_FSEvalFile(*interp*, *pathPtr*)

int
Tcl_FSLoadFile(*interp*, *pathPtr*, *sym1*, *sym2*, *proc1Ptr*, *proc2Ptr*, *loadHandlePtr*, *unloadProcPtr*)

int
Tcl_FSUnloadFile(*interp*, *loadHandle*)

int
Tcl_FSMatchInDirectory(*interp*, *resultPtr*, *pathPtr*, *pattern*, *types*)

Tcl_Obj *
Tcl_FSLink(*linkNamePtr*, *toPtr*, *linkAction*)

int
Tcl_FSLstat(*pathPtr*, *statPtr*)

int
Tcl_FSUtime(*pathPtr*, *tval*)

int
Tcl_FSFileAttrsGet(*interp*, *index*, *pathPtr*, *objPtrRef*)

int
Tcl_FSFileAttrsSet(*interp*, *index*, *pathPtr*, *objPtr*)

const char *const *
Tcl_FSFileAttrStrings(*pathPtr*, *objPtrRef*)

int
Tcl_FSStat(*pathPtr*, *statPtr*)

int
Tcl_FSAccess(*pathPtr*, *mode*)

Tcl_Channel
Tcl_FSOpenFileChannel(*interp*, *pathPtr*, *modeString*, *permissions*)

Tcl_Obj *
Tcl_FSGetCwd(*interp*)

int
Tcl_FSChdir(*pathPtr*)

Tcl_Obj *
Tcl_FSPathSeparator(*pathPtr*)

Tcl_Obj *
Tcl_FSJoinPath(*listObj*, *elements*)

Tcl_Obj *
Tcl_FSSplitPath(*pathPtr*, *lenPtr*)

int
Tcl_FSEqualPaths(*firstPtr*, *secondPtr*)

Tcl_Obj *
Tcl_FSGetNormalizedPath(*interp*, *pathPtr*)

Tcl_Obj *
Tcl_FSJoinToPath(*basePtr*, *objc*, *objv*)

int
Tcl_FSConvertToPathType(*interp*, *pathPtr*)

void *
Tcl_FSGetInternalRep(*pathPtr*, *fsPtr*)

Tcl_Obj *
Tcl_FSGetTranslatedPath(*interp*, *pathPtr*)

const char *
Tcl_FSGetTranslatedStringPath(*interp*, *pathPtr*)

Tcl_Obj *
Tcl_FSNewNativePath(*fsPtr*, *clientData*)

const void *
Tcl_FSGetNativePath(*pathPtr*)

Tcl_Obj *
Tcl_FSFileSystemInfo(*pathPtr*)

Tcl_StatBuf *
Tcl_AllocStatBuf()

Tcl_WideInt
Tcl_GetAccessTimeFromStat(*statPtr*)

unsigned
Tcl_GetBlockSizeFromStat(*statPtr*)

Tcl_WideUInt
Tcl_GetBlocksFromStat(*statPtr*)

Tcl_WideInt
Tcl_GetChangeTimeFromStat(*statPtr*)

int
Tcl_GetDeviceTypeFromStat(*statPtr*)

unsigned
Tcl_GetFSDeviceFromStat(*statPtr*)

unsigned
Tcl_GetFSInodeFromStat(*statPtr*)

int

Tcl_GetGroupIdFromStat(*statPtr*)

int
Tcl_GetLinkCountFromStat(*statPtr*)

unsigned
Tcl_GetModeFromStat(*statPtr*)

Tcl_WideInt
Tcl_GetModificationTimeFromStat(*statPtr*)

Tcl_WideUInt
Tcl_GetSizeFromStat(*statPtr*)

int
Tcl_GetUserIdFromStat(*statPtr*)

Arguments

const Tcl_Filesystem *_fsPtr_ (in)
Points to a structure containing the addresses of procedures that can be called to perform the various filesystem operations.

Tcl_Obj *_pathPtr_ (in)
The path represented by this value is used for the operation in question. If the value does not already have an internal **path** representation, it will be converted to have one.

Tcl_Obj *_srcPathPtr_ (in)
As for *pathPtr*, but used for the source file for a copy or rename operation.

Tcl_Obj *_destPathPtr_ (in)
As for *pathPtr*, but used for the destination filename for a copy or rename operation.

int _recursive_ (in)
Whether to remove subdirectories and their contents as well.

const char *_encodingName_ (in)
The encoding of the data stored in the file identified by *pathPtr* and to be evaluated.

const char *_pattern_ (in)
Only files or directories matching this pattern will be returned.

Tcl_GlobTypeData *_types_ (in)
Only files or directories matching the type descriptions contained in this structure will be returned. This parameter may be NULL.

Tcl_Interp *_interp_ (in)
Interpreter to use either for results, evaluation, or reporting error messages.

void *_clientData_ (in)
The native description of the path value to create.

Tcl_Obj *_firstPtr_ (in)
The first of two path values to compare. The value may be converted to **path** type.

Tcl_Obj *_secondPtr_ (in)
The second of two path values to compare. The value may be converted to **path** type.

Tcl_Obj *_listObj_ (in)
The list of path elements to operate on with a **join** operation.

int _elements_ (in)

The number of elements in the *listObj* which should be joined together. If negative, then all elements are joined.

Tcl_Obj ****errorPtr* **(out)
In the case of an error, filled with a value containing the name of the file which caused an error in the various copy/rename operations.

int *index* (in)
The index of the attribute in question.

Tcl_Obj ****objPtr* **(in)
The value to set in the operation.

Tcl_Obj ****objPtrRef* **(out)
Filled with a value containing the result of the operation.

Tcl_Obj ****resultPtr* **(out)
Preallocated value in which to store (using **Tcl_ListObjAppendElement**) the list of files or directories which are successfully matched.

int *mode* (in)
Mask consisting of one or more of R_OK, W_OK, X_OK and F_OK. R_OK, W_OK and X_OK request checking whether the file exists and has read, write and execute permissions, respectively. F_OK just requests checking for the existence of the file.

Tcl_StatBuf ****statPtr* **(out)
The structure that contains the result of a stat or lstat operation.

const char ****sym1* **(in)
Name of a procedure to look up in the file's symbol table

const char ****sym2* **(in)
Name of a procedure to look up in the file's symbol table

Tcl_PackageInitProc ****proc1Ptr* **(out)
Filled with the init function for this code.

Tcl_PackageInitProc ****proc2Ptr* **(out)
Filled with the safe-init function for this code.

void ****clientDataPtr* **(out)
Filled with the clientData value to pass to this code's unload function when it is called.

Tcl_LoadHandle ****loadHandlePtr* **(out)
Filled with an abstract token representing the loaded file.

Tcl_FSUnloadFileProc ****unloadProcPtr* **(out)
Filled with the function to use to unload this piece of code.

Tcl_LoadHandle *loadHandle* (in)
Handle to the loaded library to be unloaded.

utimbuf ****tval* **(in)
The access and modification times in this structure are read and used to set those values for a given file.

const char ****modeString* **(in)
Specifies how the file is to be accessed. May have any of the values allowed for the *mode* argument to the Tcl **open** command.

int *permissions* (in)
POSIX-style permission flags such as 0644. If a new file is created, these permissions will be

set on the created file.

int *_lenPtr_ (out)
If non-NULL, filled with the number of elements in the split path.

Tcl_Obj *_basePtr_ (in)
The base path on to which to join the given elements. May be NULL.

int _objc_ (in)
The number of elements in *objv*.

Tcl_Obj *const _objv[]_ (in)
The elements to join to the given base path.

Tcl_Obj *_linkNamePtr_ (in)
The name of the link to be created or read.

Tcl_Obj *_toPtr_ (in)
What the link called *linkNamePtr* should be linked to, or NULL if the symbolic link specified by *linkNamePtr* is to be read.

int _linkAction_ (in)
OR-ed combination of flags indicating what kind of link should be created (will be ignored if *toPtr* is NULL). Valid bits to set are **TCL_CREATE_SYMBOLIC_LINK** and **TCL_CREATE_HARD_LINK**. When both flags are set and the underlying filesystem can do either, symbolic links are preferred.

Description

There are several reasons for calling the **Tcl_FS** API functions (e.g.\ **Tcl_FSAccess** and **Tcl_FSStat**) rather than calling system level functions like **access** and **stat** directly. First, they will work cross-platform, so an extension which calls them should work unmodified on Unix and Windows. Second, the Windows implementation of some of these functions fixes some bugs in the system level calls. Third, these function calls deal with any "Utf to platform-native" path conversions which may be required (and may cache the results of such conversions for greater efficiency on subsequent calls). Fourth, and perhaps most importantly, all of these functions are "virtual filesystem aware". Any virtual filesystem (VFS for short) which has been registered (through **Tcl_FSRegister**) may reroute file access to alternative media or access methods. This means that all of these functions (and therefore the corresponding **file**, **glob**, **pwd**, **cd**, **open**, etc.\ Tcl commands) may be operate on "files" which are not native files in the native filesystem. This also means that any Tcl extension which accesses the filesystem (FS for short) through this API is automatically "virtual filesystem aware". Of course, if an extension accesses the native filesystem directly (through platform-specific APIs, for example), then Tcl cannot intercept such calls.

If appropriate VFSes have been registered, the "files" may, to give two examples, be remote (e.g.\ situated on a remote ftp server) or archived (e.g.\ lying inside a .zip archive). Such registered filesystems provide a lookup table of functions to implement all or some of the functionality listed here. Finally, the **Tcl_FSStat** and **Tcl_FSLstat** calls abstract away from what the "struct stat" buffer is actually declared to be, allowing the same code to be used both on systems with and systems without support for files larger than 2GB in size.

The **Tcl_FS** API is **Tcl_Obj**-ified and may cache internal representations and other path-related strings (e.g.\ the current working directory). One side-effect of this is that one must not pass in values with a reference count of zero to any of these functions. If such calls were handled, they

might result in memory leaks (under some circumstances, the filesystem code may wish to retain a reference to the passed in value, and so one must not assume that after any of these calls return, the value still has a reference count of zero - it may have been incremented) or in a direct segmentation fault (or other memory access error) due to the value being freed part way through the complex value manipulation required to ensure that the path is fully normalized and absolute for filesystem determination. The practical lesson to learn from this is that

```
Tcl_Obj *path = Tcl_NewStringObj(...);
Tcl_FSWhatever(path);
Tcl_DecrRefCount(path);
```

is wrong, and may cause memory errors. The *path* must have its reference count incremented before passing it in, or decrementing it. For this reason, values with a reference count of zero are considered not to be valid filesystem paths and calling any Tcl_FS API function with such a value will result in no action being taken.

Fs api functions

Tcl_FSCopyFile attempts to copy the file given by *srcPathPtr* to the path name given by *destPathPtr*. If the two paths given lie in the same filesystem (according to **Tcl_FSGetFileSystemForPath**) then that filesystem's "copy file" function is called (if it is non-NULL). Otherwise the function returns -1 and sets the **errno** global C variable to the "EXDEV" POSIX error code (which signifies a "cross-domain link").

Tcl_FSCopyDirectory attempts to copy the directory given by *srcPathPtr* to the path name given by *destPathPtr*. If the two paths given lie in the same filesystem (according to **Tcl_FSGetFileSystemForPath**) then that filesystem's "copy file" function is called (if it is non-NULL). Otherwise the function returns -1 and sets the **errno** global C variable to the "EXDEV" POSIX error code (which signifies a "cross-domain link").

Tcl_FSCreateDirectory attempts to create the directory given by *pathPtr* by calling the owning filesystem's "create directory" function.

Tcl_FSDeleteFile attempts to delete the file given by *pathPtr* by calling the owning filesystem's "delete file" function.

Tcl_FSRemoveDirectory attempts to remove the directory given by *pathPtr* by calling the owning filesystem's "remove directory" function.

Tcl_FSRenameFile attempts to rename the file or directory given by *srcPathPtr* to the path name given by *destPathPtr*. If the two paths given lie in the same filesystem (according to **Tcl_FSGetFileSystemForPath**) then that filesystem's "rename file" function is called (if it is non-NULL). Otherwise the function returns -1 and sets the **errno** global C variable to the "EXDEV" POSIX error code (which signifies a "cross-domain link").

Tcl_FSListVolumes calls each filesystem which has a non-NULL "list volumes" function and asks them to return their list of root volumes. It accumulates the return values in a list which is returned to the caller (with a reference count of 0).

Tcl_FSEvalFileEx reads the file given by *pathPtr* using the encoding identified by *encodingName* and evaluates its contents as a Tcl script. It returns the same information as **Tcl_EvalObjEx**. If *encodingName* is NULL, the system encoding is used for reading the file contents. If the file could not be read then a Tcl error is returned to describe why the file could not be read. The eofchar for files is "\x1A" (^Z) for all platforms. If you require a "^Z" in code for

string comparison, you can use "\x1A", which will be safely substituted by the Tcl interpreter into "^Z". **Tcl_FSEvalFile** is a simpler version of **Tcl_FSEvalFileEx** that always uses the system encoding when reading the file.

Tcl_FSLoadFile dynamically loads a binary code file into memory and returns the addresses of two procedures within that file, if they are defined. The appropriate function for the filesystem to which *pathPtr* belongs will be called. If that filesystem does not implement this function (most virtual filesystems will not, because of OS limitations in dynamically loading binary code), Tcl will attempt to copy the file to a temporary directory and load that temporary file.

Tcl_FSUnloadFile reverses the operation, asking for the library indicated by the *loadHandle* to be removed from the process. Note that, unlike with the **unload** command, this does not give the library any opportunity to clean up.

Both the above functions return a standard Tcl completion code. If an error occurs, an error message is left in the *interp*'s result.

The token provided via the variable indicated by *loadHandlePtr* may be used with **Tcl_FindSymbol**.

Tcl_FSMatchInDirectory is used by the globbing code to search a directory for all files which match a given pattern. The appropriate function for the filesystem to which *pathPtr* belongs will be called.

The return value is a standard Tcl result indicating whether an error occurred in globbing. Error messages are placed in interp (unless interp is NULL, which is allowed), but good results are placed in the resultPtr given.

Note that the **glob** code implements recursive patterns internally, so this function will only ever be passed simple patterns, which can be matched using the logic of **string match**. To handle recursion, Tcl will call this function frequently asking only for directories to be returned. A special case of being called with a NULL pattern indicates that the path needs to be checked only for the correct type.

Tcl_FSLink replaces the library version of **readlink**, and extends it to support the creation of links. The appropriate function for the filesystem to which *linkNamePtr* belongs will be called.

If the *toPtr* is NULL, a "read link" action is performed. The result is a Tcl_Obj specifying the contents of the symbolic link given by *linkNamePtr*, or NULL if the link could not be read. The result is owned by the caller, which should call **Tcl_DecrRefCount** when the result is no longer needed. If the *toPtr* is not NULL, Tcl should create a link of one of the types passed in in the *linkAction* flag. This flag is an OR'ed combination of **TCL_CREATE_SYMBOLIC_LINK** and **TCL_CREATE_HARD_LINK**. Where a choice exists (i.e.\ more than one flag is passed in), the Tcl convention is to prefer symbolic links. When a link is successfully created, the return value should be *toPtr* (which is therefore already owned by the caller). If unsuccessful, NULL is returned.

Tcl_FSLstat fills the *Tcl_StatBuf* structure *statPtr* with information about the specified file. You do not need any access rights to the file to get this information but you need search rights to all directories named in the path leading to the file. The *Tcl_StatBuf* structure includes info regarding device, inode (always 0 on Windows), privilege mode, nlink (always 1 on Windows), user id (always 0 on Windows), group id (always 0 on Windows), rdev (same as device on Windows), size, last access time, last modification time, and last metadata change time. See **PORTABLE STAT RESULT API** for a description of how to write portable code to allocate and

access the *Tcl_StatBuf* structure.

If *path* exists, **Tcl_FSLstat** returns 0 and the stat structure is filled with data. Otherwise, -1 is returned, and no stat info is given.

Tcl_FSUtime replaces the library version of utime.

This returns 0 on success and -1 on error (as per the **utime** documentation). If successful, the function will update the "atime" and "mtime" values of the file given.

Tcl_FSFileAttrsGet implements read access for the hookable **file attributes** subcommand. The appropriate function for the filesystem to which *pathPtr* belongs will be called.

If the result is **TCL_OK**, then a value was placed in *objPtrRef*, which will only be temporarily valid (unless **Tcl_IncrRefCount** is called).

Tcl_FSFileAttrsSet implements write access for the hookable **file attributes** subcommand. The appropriate function for the filesystem to which *pathPtr* belongs will be called.

Tcl_FSFileAttrStrings implements part of the hookable **file attributes** subcommand. The appropriate function for the filesystem to which *pathPtr* belongs will be called.

The called procedure may either return an array of strings, or may instead return NULL and place a Tcl list into the given *objPtrRef*. Tcl will take that list and first increment its reference count before using it. On completion of that use, Tcl will decrement its reference count. Hence if the list should be disposed of by Tcl when done, it should have a reference count of zero, and if the list should not be disposed of, the filesystem should ensure it retains a reference count to the value.

Tcl_FSAccess checks whether the process would be allowed to read, write or test for existence of the file (or other filesystem object) whose name is *pathname*. If *pathname* is a symbolic link on Unix, then permissions of the file referred by this symbolic link are tested.

On success (all requested permissions granted), zero is returned. On error (at least one bit in mode asked for a permission that is denied, or some other error occurred), -1 is returned.

Tcl_FSStat fills the *Tcl_StatBuf* structure *statPtr* with information about the specified file. You do not need any access rights to the file to get this information but you need search rights to all directories named in the path leading to the file. The *Tcl_StatBuf* structure includes info regarding device, inode (always 0 on Windows), privilege mode, nlink (always 1 on Windows), user id (always 0 on Windows), group id (always 0 on Windows), rdev (same as device on Windows), size, last access time, last modification time, and last metadata change time. See **PORTABLE STAT RESULT API** for a description of how to write portable code to allocate and access the *Tcl_StatBuf* structure.

If *path* exists, **Tcl_FSStat** returns 0 and the stat structure is filled with data. Otherwise, -1 is returned, and no stat info is given.

Tcl_FSOpenFileChannel opens a file specified by *pathPtr* and returns a channel handle that can be used to perform input and output on the file. This API is modeled after the **fopen** procedure of the Unix standard I/O library. The syntax and meaning of all arguments is similar to those given in the Tcl **open** command when opening a file. If an error occurs while opening the channel, **Tcl_FSOpenFileChannel** returns NULL and records a POSIX error code that can be retrieved with **Tcl_GetErrno**. In addition, if *interp* is non-NULL, **Tcl_FSOpenFileChannel** leaves an error message in *interp*'s result after any error.

The newly created channel is not registered in the supplied interpreter; to register it, use

Tcl_RegisterChannel. If one of the standard channels, **stdin**, **stdout** or **stderr** was previously closed, the act of creating the new channel also assigns it as a replacement for the standard channel.

Tcl_FSGetCwd replaces the library version of **getcwd**.

It returns the Tcl library's current working directory. This may be different to the native platform's working directory, which happens when the current working directory is not in the native filesystem.

The result is a pointer to a Tcl_Obj specifying the current directory, or NULL if the current directory could not be determined. If NULL is returned, an error message is left in the *interp*'s result.

The result already has its reference count incremented for the caller. When it is no longer needed, that reference count should be decremented. This is needed for thread-safety purposes, to allow multiple threads to access this and related functions, while ensuring the results are always valid.

Tcl_FSChdir replaces the library version of **chdir**. The path is normalized and then passed to the filesystem which claims it. If that filesystem does not implement this function, Tcl will fallback to a combination of **stat** and **access** to check whether the directory exists and has appropriate permissions.

For results, see **chdir** documentation. If successful, we keep a record of the successful path in *cwdPathPtr* for subsequent calls to **Tcl_FSGetCwd**.

Tcl_FSPathSeparator returns the separator character to be used for most specific element of the path specified by *pathPtr* (i.e.\ the last part of the path).

The separator is returned as a Tcl_Obj containing a string of length 1. If the path is invalid, NULL is returned.

Tcl_FSJoinPath takes the given Tcl_Obj, which must be a valid list (which is allowed to have a reference count of zero), and returns the path value given by considering the first *elements* elements as valid path segments (each path segment may be a complete path, a partial path or just a single possible directory or file name). If any path segment is actually an absolute path, then all prior path segments are discarded. If *elements* is less than 0, we use the entire list.

It is possible that the returned value is actually an element of the given list, so the caller should be careful to increment the reference count of the result before freeing the list.

The returned value, typically with a reference count of zero (but it could be shared under some conditions), contains the joined path. The caller must add a reference count to the value before using it. In particular, the returned value could be an element of the given list, so freeing the list might free the value prematurely if no reference count has been taken. If the number of elements is zero, then the returned value will be an empty-string Tcl_Obj.

Tcl_FSSplitPath takes the given Tcl_Obj, which should be a valid path, and returns a Tcl list value containing each segment of that path as an element. It returns a list value with a reference count of zero. If the passed in *lenPtr* is non-NULL, the variable it points to will be updated to contain the number of elements in the returned list.

Tcl_FSEqualPaths tests whether the two paths given represent the same filesystem object. It returns 1 if the paths are equal, and 0 if they are different. If either path is NULL, 0 is always returned.

Tcl_FSGetNormalizedPath attempts to extract from the given Tcl_Obj a unique normalized path representation, whose string value can be used as a unique identifier for the file.

It returns the normalized path value, owned by Tcl, or NULL if the path was invalid or could otherwise not be successfully converted. Extraction of absolute, normalized paths is very efficient (because the filesystem operates on these representations internally), although the result when the filesystem contains numerous symbolic links may not be the most user-friendly version of a path. The return value is owned by Tcl and has a lifetime equivalent to that of the *pathPtr* passed in (unless that is a relative path, in which case the normalized path value may be freed any time the cwd changes) - the caller can of course increment the reference count if it wishes to maintain a copy for longer.

Tcl_FSJoinToPath takes the given value, which should usually be a valid path or NULL, and joins onto it the array of paths segments given.

Returns a value, typically with reference count of zero (but it could be shared under some conditions), containing the joined path. The caller must add a reference count to the value before using it. If any of the values passed into this function (*pathPtr* or *path* elements) have a reference count of zero, they will be freed when this function returns.

Tcl_FSConvertToPathType tries to convert the given Tcl_Obj to a valid Tcl path type, taking account of the fact that the cwd may have changed even if this value is already supposedly of the correct type. The filename may begin with "~" (to indicate current user's home directory) or "~<user>" (to indicate any user's home directory).

If the conversion succeeds (i.e.\ the value is a valid path in one of the current filesystems), then **TCL_OK** is returned. Otherwise **TCL_ERROR** is returned, and an error message may be left in the interpreter.

Tcl_FSGetInternalRep extracts the internal representation of a given path value, in the given filesystem. If the path value belongs to a different filesystem, we return NULL. If the internal representation is currently NULL, we attempt to generate it, by calling the filesystem's **Tcl_FSCreateInternalRepProc**.

Returns NULL or a valid internal path representation. This internal representation is cached, so that repeated calls to this function will not require additional conversions.

Tcl_FSGetTranslatedPath attempts to extract the translated path from the given Tcl_Obj.

If the translation succeeds (i.e.\ the value is a valid path), then it is returned. Otherwise NULL will be returned, and an error message may be left in the interpreter. A "translated" path is one which contains no "~" or "~user" sequences (these have been expanded to their current representation in the filesystem). The value returned is owned by the caller, which must store it or call **Tcl_DecrRefCount** to ensure memory is freed. This function is of little practical use, and **Tcl_FSGetNormalizedPath** or **Tcl_FSGetNativePath** are usually better functions to use for most purposes.

Tcl_FSGetTranslatedStringPath does the same as **Tcl_FSGetTranslatedPath**, but returns a character string or NULL. The string returned is dynamically allocated and owned by the caller, which must store it or call **ckfree** to ensure it is freed. Again, **Tcl_FSGetNormalizedPath** or **Tcl_FSGetNativePath** are usually better functions to use for most purposes.

Tcl_FSNewNativePath performs something like the reverse of the usual obj->path->nativerep conversions. If some code retrieves a path in native form (from, e.g.\ **readlink** or a native

dialog), and that path is to be used at the Tcl level, then calling this function is an efficient way of creating the appropriate path value type.

The resulting value is a pure "path" value, which will only receive a UTF-8 string representation if that is required by some Tcl code.

Tcl_FSGetNativePath is for use by the Win/Unix native filesystems, so that they can easily retrieve the native (char* or TCHAR*) representation of a path. This function is a convenience wrapper around **Tcl_FSGetInternalRep**. It may be desirable in the future to have non-string-based native representations (for example, on macOS, a representation using a fileSpec of FSRef structure would probably be more efficient). On Windows a full Unicode representation would allow for paths of unlimited length. Currently the representation is simply a character string which may contain either the relative path or a complete, absolute normalized path in the native encoding (complex conditions dictate which of these will be provided, so neither can be relied upon, unless the path is known to be absolute). If you need a native path which must be absolute, then you should ask for the native version of a normalized path. If for some reason a non-absolute, non-normalized version of the path is needed, that must be constructed separately (e.g.\ using **Tcl_FSGetTranslatedPath**).

The native representation is cached so that repeated calls to this function will not require additional conversions. The return value is owned by Tcl and has a lifetime equivalent to that of the *pathPtr* passed in (unless that is a relative path, in which case the native representation may be freed any time the cwd changes).

Tcl_FSFileSystemInfo returns a list of two elements. The first element is the name of the filesystem (e.g. "native", "vfs", "zip", or "prowrap", perhaps), and the second is the particular type of the given path within that filesystem (which is filesystem dependent). The second element may be empty if the filesystem does not provide a further categorization of files.

A valid list value is returned, unless the path value is not recognized, when NULL will be returned.

Tcl_FSGetFileSystemForPath returns a pointer to the **Tcl_Filesystem** which accepts this path as valid.

If no filesystem will accept the path, NULL is returned.

Tcl_FSGetPathType determines whether the given path is relative to the current directory, relative to the current volume, or absolute.

It returns one of **TCL_PATH_ABSOLUTE**, **TCL_PATH_RELATIVE**, or **TCL_PATH_VOLUME_RELATIVE**

Portable stat result api

Tcl_AllocStatBuf allocates a *Tcl_StatBuf* on the system heap (which may be deallocated by being passed to **ckfree**). This allows extensions to invoke **Tcl_FSStat** and **Tcl_FSLstat** without being dependent on the size of the buffer. That in turn depends on the flags used to build Tcl.

The portable fields of a *Tcl_StatBuf* may be read using the following functions, each of which returns the value of the corresponding field listed in the table below. Note that on some platforms there may be other fields in the *Tcl_StatBuf* as it is an alias for a suitable system structure, but only the portable ones are made available here. See your system documentation for a full description of these fields.

```
Access Function Field
 Tcl_GetFSDeviceFromStat  st_dev
 Tcl_GetFSInodeFromStat  st_ino
 Tcl_GetModeFromStat  st_mode
 Tcl_GetLinkCountFromStat  st_nlink
 Tcl_GetUserIdFromStat  st_uid
 Tcl_GetGroupIdFromStat  st_gid
 Tcl_GetDeviceTypeFromStat  st_rdev
 Tcl_GetAccessTimeFromStat  st_atime
 Tcl_GetModificationTimeFromStat  st_mtime
 Tcl_GetChangeTimeFromStat  st_ctime
 Tcl_GetSizeFromStat  st_size
 Tcl_GetBlocksFromStat  st_blocks
 Tcl_GetBlockSizeFromStat  st_blksize
```

The virtual filesystem api

A filesystem provides a **Tcl_Filesystem** structure that contains pointers to functions that implement the various operations on a filesystem; these operations are invoked as needed by the generic layer, which generally occurs through the functions listed above.

The **Tcl_Filesystem** structures are manipulated using the following methods.

Tcl_FSRegister takes a pointer to a filesystem structure and an optional piece of data to associated with that filesystem. On calling this function, Tcl will attach the filesystem to the list of known filesystems, and it will become fully functional immediately. Tcl does not check if the same filesystem is registered multiple times (and in general that is not a good thing to do). **TCL_OK** will be returned.

Tcl_FSUnregister removes the given filesystem structure from the list of known filesystems, if it is known, and returns **TCL_OK**. If the filesystem is not currently registered, **TCL_ERROR** is returned.

Tcl_FSData will return the clientData associated with the given filesystem, if that filesystem is registered. Otherwise it will return NULL.

Tcl_FSMountsChanged is used to inform the Tcl's core that the set of mount points for the given (already registered) filesystem have changed, and that cached file representations may therefore no longer be correct.

The tcl_filesystem structure

The **Tcl_Filesystem** structure contains the following fields:

```
typedef struct Tcl_Filesystem {
    const char *typeName;
    int structureLength;
    Tcl_FSVersion version;
    Tcl_FSPathInFilesystemProc *pathInFilesystemProc;
    Tcl_FSDupInternalRepProc *dupInternalRepProc;
    Tcl_FSFreeInternalRepProc *freeInternalRepProc;
    Tcl_FSInternalToNormalizedProc *internalToNormalizedProc;
```

```
    Tcl_FSCreateInternalRepProc *createInternalRepProc;
    Tcl_FSNormalizePathProc *normalizePathProc;
    Tcl_FSFilesystemPathTypeProc *filesystemPathTypeProc;
    Tcl_FSFilesystemSeparatorProc *filesystemSeparatorProc;
    Tcl_FSStatProc *statProc;
    Tcl_FSAccessProc *accessProc;
    Tcl_FSOpenFileChannelProc *openFileChannelProc;
    Tcl_FSMatchInDirectoryProc *matchInDirectoryProc;
    Tcl_FSUtimeProc *utimeProc;
    Tcl_FSLinkProc *linkProc;
    Tcl_FSListVolumesProc *listVolumesProc;
    Tcl_FSFileAttrStringsProc *fileAttrStringsProc;
    Tcl_FSFileAttrsGetProc *fileAttrsGetProc;
    Tcl_FSFileAttrsSetProc *fileAttrsSetProc;
    Tcl_FSCreateDirectoryProc *createDirectoryProc;
    Tcl_FSRemoveDirectoryProc *removeDirectoryProc;
    Tcl_FSDeleteFileProc *deleteFileProc;
    Tcl_FSCopyFileProc *copyFileProc;
    Tcl_FSRenameFileProc *renameFileProc;
    Tcl_FSCopyDirectoryProc *copyDirectoryProc;
    Tcl_FSLstatProc *lstatProc;
    Tcl_FSLoadFileProc *loadFileProc;
    Tcl_FSGetCwdProc *getCwdProc;
    Tcl_FSChdirProc *chdirProc;
} Tcl_Filesystem;
```

Except for the first three fields in this structure which contain simple data elements, all entries contain addresses of functions called by the generic filesystem layer to perform the complete range of filesystem related actions.

The many functions in this structure are broken down into three categories: infrastructure functions (almost all of which must be implemented), operational functions (which must be implemented if a complete filesystem is provided), and efficiency functions (which need only be implemented if they can be done so efficiently, or if they have side-effects which are required by the filesystem; Tcl has less efficient emulations it can fall back on). It is important to note that, in the current version of Tcl, most of these fallbacks are only used to handle commands initiated in Tcl, not in C. What this means is, that if a **file rename** command is issued in Tcl, and the relevant filesystem(s) do not implement their *Tcl_FSRenameFileProc*, Tcl's core will instead fallback on a combination of other filesystem functions (it will use *Tcl_FSCopyFileProc* followed by *Tcl_FSDeleteFileProc*, and if *Tcl_FSCopyFileProc* is not implemented there is a further fallback). However, if a *Tcl_FSRenameFileProc* command is issued at the C level, no such fallbacks occur. This is true except for the last four entries in the filesystem table (**lstat**, **load**, **getcwd** and **chdir**) for which fallbacks do in fact occur at the C level.

Any functions which take path names in Tcl_Obj form take those names in UTF-8 form. The filesystem infrastructure API is designed to support efficient, cached conversion of these UTF-8 paths to other native representations.

Example filesystem definition

Here is the filesystem lookup table used by the "vfs" extension which allows filesystem actions to be implemented in Tcl.

```
static Tcl_Filesystem vfsFilesystem = {
    "tclvfs",
    sizeof(Tcl_Filesystem),
    TCL_FILESYSTEM_VERSION_1,
    &VfsPathInFilesystem,
    &VfsDupInternalRep,
    &VfsFreeInternalRep,
    /* No internal to normalized, since we don't create
     * any pure 'internal' Tcl_Obj path representations */
    NULL,
    /* No create native rep function, since we don't use
     * it and don't choose to support uses of
     * Tcl_FSNewNativePath */
    NULL,
    /* Normalize path isn't needed - we assume paths only
     * have one representation */
    NULL,
    &VfsFilesystemPathType,
    &VfsFilesystemSeparator,
    &VfsStat,
    &VfsAccess,
    &VfsOpenFileChannel,
    &VfsMatchInDirectory,
    &VfsUtime,
    /* We choose not to support symbolic links inside our
     * VFS's */
    NULL,
    &VfsListVolumes,
    &VfsFileAttrStrings,
    &VfsFileAttrsGet,
    &VfsFileAttrsSet,
    &VfsCreateDirectory,
    &VfsRemoveDirectory,
    &VfsDeleteFile,
    /* No copy file; use the core fallback mechanism */
    NULL,
    /* No rename file; use the core fallback mechanism */
    NULL,
    /* No copy directory; use the core fallback mechanism */
    NULL,
    /* Core will use stat for lstat */
    NULL,
```

```
    /* No load; use the core fallback mechanism */
    NULL,
    /* We don't need a getcwd or chdir; the core's own
     * internal value is suitable */
    NULL,
    NULL
};
```

Filesystem infrastructure

These fields contain basic information about the filesystem structure and addresses of functions which are used to associate a particular filesystem with a file path, and deal with the internal handling of path representations, for example copying and freeing such representations.

Typename

The *typeName* field contains a null-terminated string that identifies the type of the filesystem implemented, e.g. "native", "zip" or "vfs".

Structure length

The *structureLength* field is generally implemented as *sizeof(Tcl_Filesystem)*, and is there to allow easier binary backwards compatibility if the size of the structure changes in a future Tcl release.

Version

The *version* field should be set to **TCL_FILESYSTEM_VERSION_1**.

Pathinfilesystemproc

The *pathInFilesystemProc* field contains the address of a function which is called to determine whether a given path value belongs to this filesystem or not. Tcl will only call the rest of the filesystem functions with a path for which this function has returned **TCL_OK**. If the path does not belong, -1 should be returned (the behavior of Tcl for any other return value is not defined). If **TCL_OK** is returned, then the optional *clientDataPtr* output parameter can be used to return an internal (filesystem specific) representation of the path, which will be cached inside the path value, and may be retrieved efficiently by the other filesystem functions. Tcl will simultaneously cache the fact that this path belongs to this filesystem. Such caches are invalidated when filesystem structures are added or removed from Tcl's internal list of known filesystems.

```
typedef int Tcl_FSPathInFilesystemProc(
        Tcl_Obj *pathPtr,
        ClientData *clientDataPtr);
```

Dupinternalrepproc

This function makes a copy of a path's internal representation, and is called when Tcl needs to duplicate a path value. If NULL, Tcl will simply not copy the internal representation, which may then need to be regenerated later.

```
typedef ClientData Tcl_FSDupInternalRepProc(
        ClientData clientData);
```

Freeinternalrepproc

Free the internal representation. This must be implemented if internal representations need freeing (i.e.\ if some memory is allocated when an internal representation is generated), but may otherwise be NULL.

```
typedef void Tcl_FSFreeInternalRepProc(
        ClientData clientData);
```

Internaltonormalizedproc

Function to convert internal representation to a normalized path. Only required if the filesystem creates pure path values with no string/path representation. The return value is a Tcl value whose string representation is the normalized path.

```
typedef Tcl_Obj *Tcl_FSInternalToNormalizedProc(
        ClientData clientData);
```

Createinternalrepproc

Function to take a path value, and calculate an internal representation for it, and store that native representation in the value. May be NULL if paths have no internal representation, or if the *Tcl_FSPathInFilesystemProc* for this filesystem always immediately creates an internal representation for paths it accepts.

```
typedef ClientData Tcl_FSCreateInternalRepProc(
        Tcl_Obj *pathPtr);
```

Normalizepathproc

Function to normalize a path. Should be implemented for all filesystems which can have multiple string representations for the same path value. In Tcl, every "path" must have a single unique "normalized" string representation. Depending on the filesystem, there may be more than one unnormalized string representation which refers to that path (e.g.\ a relative path, a path with different character case if the filesystem is case insensitive, a path contain a reference to a home directory such as "~", a path containing symbolic links, etc). If the very last component in the path is a symbolic link, it should not be converted into the value it points to (but its case or other aspects should be made unique). All other path components should be converted from symbolic links. This one exception is required to agree with Tcl's semantics with **file delete**, **file rename**, **file copy** operating on symbolic links. This function may be called with *nextCheckpoint* either at the beginning of the path (i.e.\ zero), at the end of the path, or at any intermediate file separator in the path. It will never point to any other arbitrary position in the path. In the last of the three valid cases, the implementation can assume that the path up to and including the file separator is known and normalized.

```
typedef int Tcl_FSNormalizePathProc(
        Tcl_Interp *interp,
        Tcl_Obj *pathPtr,
        int nextCheckpoint);
```

Filesystem operations

The fields in this section of the structure contain addresses of functions which are called to carry

out the basic filesystem operations. A filesystem which expects to be used with the complete standard Tcl command set must implement all of these. If some of them are not implemented, then certain Tcl commands may fail when operating on paths within that filesystem. However, in some instances this may be desirable (for example, a read-only filesystem should not implement the last four functions, and a filesystem which does not support symbolic links need not implement the **readlink** function, etc. The Tcl core expects filesystems to behave in this way).

Filesystempathtypeproc

Function to determine the type of a path in this filesystem. May be NULL, in which case no type information will be available to users of the filesystem. The "type" is used only for informational purposes, and should be returned as the string representation of the Tcl_Obj which is returned. A typical return value might be "networked", "zip" or "ftp". The Tcl_Obj result is owned by the filesystem and so Tcl will increment the reference count of that value if it wishes to retain a reference to it.

```
typedef Tcl_Obj *Tcl_FSFilesystemPathTypeProc(
        Tcl_Obj *pathPtr);
```

Filesystemseparatorproc

Function to return the separator character(s) for this filesystem. This need only be implemented if the filesystem wishes to use a different separator than the standard string "/". Amongst other uses, it is returned by the **file separator** command. The return value should be a value with reference count of zero.

```
typedef Tcl_Obj *Tcl_FSFilesystemSeparatorProc(
        Tcl_Obj *pathPtr);
```

Statproc

Function to process a **Tcl_FSStat** call. Must be implemented for any reasonable filesystem, since many Tcl level commands depend crucially upon it (e.g.\ **file atime**, **file isdirectory**, **file size**, **glob**).

```
typedef int Tcl_FSStatProc(
        Tcl_Obj *pathPtr,
        Tcl_StatBuf *statPtr);
```

The **Tcl_FSStatProc** fills the stat structure *statPtr* with information about the specified file. You do not need any access rights to the file to get this information but you need search rights to all directories named in the path leading to the file. The stat structure includes info regarding device, inode (always 0 on Windows), privilege mode, nlink (always 1 on Windows), user id (always 0 on Windows), group id (always 0 on Windows), rdev (same as device on Windows), size, last access time, last modification time, and last metadata change time.

If the file represented by *pathPtr* exists, the **Tcl_FSStatProc** returns 0 and the stat structure is filled with data. Otherwise, -1 is returned, and no stat info is given.

Accessproc

Function to process a **Tcl_FSAccess** call. Must be implemented for any reasonable filesystem, since many Tcl level commands depend crucially upon it (e.g.\ **file exists**, **file readable**).

```
typedef int Tcl_FSAccessProc(
        Tcl_Obj *pathPtr,
        int mode);
```

The **Tcl_FSAccessProc** checks whether the process would be allowed to read, write or test for existence of the file (or other filesystem object) whose name is in *pathPtr*. If the pathname refers to a symbolic link, then the permissions of the file referred by this symbolic link should be tested.

On success (all requested permissions granted), zero is returned. On error (at least one bit in mode asked for a permission that is denied, or some other error occurred), -1 is returned.

Openfilechannelproc

Function to process a **Tcl_FSOpenFileChannel** call. Must be implemented for any reasonable filesystem, since any operations which require open or accessing a file's contents will use it (e.g.\ **open**, **encoding**, and many Tk commands).

```
typedef Tcl_Channel Tcl_FSOpenFileChannelProc(
        Tcl_Interp *interp,
        Tcl_Obj *pathPtr,
        int mode,
        int permissions);
```

The **Tcl_FSOpenFileChannelProc** opens a file specified by *pathPtr* and returns a channel handle that can be used to perform input and output on the file. This API is modeled after the **fopen** procedure of the Unix standard I/O library. The syntax and meaning of all arguments is similar to those given in the Tcl **open** command when opening a file, where the *mode* argument is a combination of the POSIX flags O_RDONLY, O_WRONLY, etc. If an error occurs while opening the channel, the **Tcl_FSOpenFileChannelProc** returns NULL and records a POSIX error code that can be retrieved with **Tcl_GetErrno**. In addition, if *interp* is non-NULL, the **Tcl_FSOpenFileChannelProc** leaves an error message in *interp*'s result after any error.

The newly created channel must not be registered in the supplied interpreter by a **Tcl_FSOpenFileChannelProc**; that task is up to the caller of **Tcl_FSOpenFileChannel** (if necessary). If one of the standard channels, **stdin**, **stdout** or **stderr** was previously closed, the act of creating the new channel also assigns it as a replacement for the standard channel.

Matchindirectoryproc

Function to process a **Tcl_FSMatchInDirectory** call. If not implemented, then glob and recursive copy functionality will be lacking in the filesystem (and this may impact commands like **encoding names** which use glob functionality internally).

```
typedef int Tcl_FSMatchInDirectoryProc(
        Tcl_Interp *interp,
        Tcl_Obj *resultPtr,
        Tcl_Obj *pathPtr,
        const char *pattern,
        Tcl_GlobTypeData *types);
```

The function should return all files or directories (or other filesystem objects) which match the

given pattern and accord with the *types* specification given. There are two ways in which this function may be called. If *pattern* is NULL, then *pathPtr* is a full path specification of a single file or directory which should be checked for existence and correct type. Otherwise, *pathPtr* is a directory, the contents of which the function should search for files or directories which have the correct type. In either case, *pathPtr* can be assumed to be both non-NULL and non-empty. It is not currently documented whether *pathPtr* will have a file separator at its end of not, so code should be flexible to both possibilities.

The return value is a standard Tcl result indicating whether an error occurred in the matching process. Error messages are placed in *interp*, unless *interp* in NULL in which case no error message need be generated; on a **TCL_OK** result, results should be added to the *resultPtr* value given (which can be assumed to be a valid unshared Tcl list). The matches added to *resultPtr* should include any path prefix given in *pathPtr* (this usually means they will be absolute path specifications). Note that if no matches are found, that simply leads to an empty result; errors are only signaled for actual file or filesystem problems which may occur during the matching process.

The **Tcl_GlobTypeData** structure passed in the *types* parameter contains the following fields:

```
typedef struct Tcl_GlobTypeData {
    /* Corresponds to bcdpfls as in 'find -t' */
    int type;
    /* Corresponds to file permissions */
    int perm;
    /* Acceptable mac type */
    Tcl_Obj *macType;
    /* Acceptable mac creator */
    Tcl_Obj *macCreator;
} Tcl_GlobTypeData;
```

There are two specific cases which it is important to handle correctly, both when *types* is non-NULL. The two cases are when *types->types & TCL_GLOB_TYPE_DIR* or *types->types & TCL_GLOB_TYPE_MOUNT* are true (and in particular when the other flags are false). In the first of these cases, the function must list the contained directories. Tcl uses this to implement recursive globbing, so it is critical that filesystems implement directory matching correctly. In the second of these cases, with **TCL_GLOB_TYPE_MOUNT**, the filesystem must list the mount points which lie within the given *pathPtr* (and in this case, *pathPtr* need not lie within the same filesystem - different to all other cases in which this function is called). Support for this is critical if Tcl is to have seamless transitions between from one filesystem to another.

Utimeproc

Function to process a **Tcl_FSUtime** call. Required to allow setting (not reading) of times with **file mtime**, **file atime** and the open-r/open-w/fcopy implementation of **file copy**.

```
typedef int Tcl_FSUtimeProc(
        Tcl_Obj *pathPtr,
        struct utimbuf *tval);
```

The access and modification times of the file specified by *pathPtr* should be changed to the values given in the *tval* structure.

The return value should be 0 on success and -1 on an error, as with the system **utime**.

Linkproc

Function to process a **Tcl_FSLink** call. Should be implemented only if the filesystem supports links, and may otherwise be NULL.

```
typedef Tcl_Obj *Tcl_FSLinkProc(
        Tcl_Obj *linkNamePtr,
        Tcl_Obj *toPtr,
        int linkAction);
```

If *toPtr* is NULL, the function is being asked to read the contents of a link. The result is a Tcl_Obj specifying the contents of the link given by *linkNamePtr*, or NULL if the link could not be read. The result is owned by the caller (and should therefore have its ref count incremented before being returned). Any callers should call **Tcl_DecrRefCount** on this result when it is no longer needed. If *toPtr* is not NULL, the function should attempt to create a link. The result in this case should be *toPtr* if the link was successful and NULL otherwise. In this case the result is not owned by the caller (i.e.\ no reference count manipulations on either end are needed). See the documentation for **Tcl_FSLink** for the correct interpretation of the *linkAction* flags.

Listvolumesproc

Function to list any filesystem volumes added by this filesystem. Should be implemented only if the filesystem adds volumes at the head of the filesystem, so that they can be returned by **file volumes**.

```
typedef Tcl_Obj *Tcl_FSListVolumesProc(void);
```

The result should be a list of volumes added by this filesystem, or NULL (or an empty list) if no volumes are provided. The result value is considered to be owned by the filesystem (not by Tcl's core), but should be given a reference count for Tcl. Tcl will use the contents of the list and then decrement that reference count. This allows filesystems to choose whether they actually want to retain a "global list" of volumes or not (if not, they generate the list on the fly and pass it to Tcl with a reference count of 1 and then forget about the list, if yes, then they simply increment the reference count of their global list and pass it to Tcl which will copy the contents and then decrement the count back to where it was).

Therefore, Tcl considers return values from this proc to be read-only.

Fileattrstringsproc

Function to list all attribute strings which are valid for this filesystem. If not implemented the filesystem will not support the **file attributes** command. This allows arbitrary additional information to be attached to files in the filesystem. If it is not implemented, there is no need to implement the **get** and **set** methods.

```
typedef const char *const *Tcl_FSFileAttrStringsProc(
        Tcl_Obj *pathPtr,
        Tcl_Obj **objPtrRef);
```

The called function may either return an array of strings, or may instead return NULL and place a Tcl list into the given *objPtrRef*. Tcl will take that list and first increment its reference count before using it. On completion of that use, Tcl will decrement its reference count. Hence if the list

should be disposed of by Tcl when done, it should have a reference count of zero, and if the list should not be disposed of, the filesystem should ensure it returns a value with a reference count of at least one.

Fileattrsgetproc

Function to process a **Tcl_FSFileAttrsGet** call, used by **file attributes**.

```
typedef int Tcl_FSFileAttrsGetProc(
        Tcl_Interp *interp,
        int index,
        Tcl_Obj *pathPtr,
        Tcl_Obj **objPtrRef);
```

Returns a standard Tcl return code. The attribute value retrieved, which corresponds to the *index* 'th element in the list returned by the **Tcl_FSFileAttrStringsProc**, is a Tcl_Obj placed in *objPtrRef* (if **TCL_OK** was returned) and is likely to have a reference count of zero. Either way we must either store it somewhere (e.g.\ the Tcl result), or Incr/Decr its reference count to ensure it is properly freed.

Fileattrssetproc

Function to process a **Tcl_FSFileAttrsSet** call, used by **file attributes**. If the filesystem is read-only, there is no need to implement this.

```
typedef int Tcl_FSFileAttrsSetProc(
        Tcl_Interp *interp,
        int index,
        Tcl_Obj *pathPtr,
        Tcl_Obj *objPtr);
```

The attribute value of the *index*'th element in the list returned by the Tcl_FSFileAttrStringsProc should be set to the *objPtr* given.

Createdirectoryproc

Function to process a **Tcl_FSCreateDirectory** call. Should be implemented unless the FS is read-only.

```
typedef int Tcl_FSCreateDirectoryProc(
        Tcl_Obj *pathPtr);
```

The return value is a standard Tcl result indicating whether an error occurred in the process. If successful, a new directory should have been added to the filesystem in the location specified by *pathPtr*.

Removedirectoryproc

Function to process a **Tcl_FSRemoveDirectory** call. Should be implemented unless the FS is read-only.

```
typedef int Tcl_FSRemoveDirectoryProc(
        Tcl_Obj *pathPtr,
        int recursive,
        Tcl_Obj **errorPtr);
```

The return value is a standard Tcl result indicating whether an error occurred in the process. If successful, the directory specified by *pathPtr* should have been removed from the filesystem. If the *recursive* flag is given, then a non-empty directory should be deleted without error. If this flag is not given, then and the directory is non-empty a POSIX "EEXIST" error should be signaled. If an error does occur, the name of the file or directory which caused the error should be placed in *errorPtr*.

Deletefileproc

Function to process a **Tcl_FSDeleteFile** call. Should be implemented unless the FS is read-only.

```
typedef int Tcl_FSDeleteFileProc(
        Tcl_Obj *pathPtr);
```

The return value is a standard Tcl result indicating whether an error occurred in the process. If successful, the file specified by *pathPtr* should have been removed from the filesystem. Note that, if the filesystem supports symbolic links, Tcl will always call this function and not Tcl_FSRemoveDirectoryProc when needed to delete them (even if they are symbolic links to directories).

Filesystem efficiency

These functions need not be implemented for a particular filesystem because the core has a fallback implementation available. See each individual description for the consequences of leaving the field NULL.

Lstatproc

Function to process a **Tcl_FSLstat** call. If not implemented, Tcl will attempt to use the *statProc* defined above instead. Therefore it need only be implemented if a filesystem can differentiate between **stat** and **lstat** calls.

```
typedef int Tcl_FSLstatProc(
        Tcl_Obj *pathPtr,
        Tcl_StatBuf *statPtr);
```

The behavior of this function is very similar to that of the **Tcl_FSStatProc** defined above, except that if it is applied to a symbolic link, it returns information about the link, not about the target file.

Copyfileproc

Function to process a **Tcl_FSCopyFile** call. If not implemented Tcl will fall back on **open**-r, **open** -w and **fcopy** as a copying mechanism. Therefore it need only be implemented if the filesystem can perform that action more efficiently.

```
typedef int Tcl_FSCopyFileProc(
        Tcl_Obj *srcPathPtr,
        Tcl_Obj *destPathPtr);
```

The return value is a standard Tcl result indicating whether an error occurred in the copying process. Note that, *destPathPtr* is the name of the file which should become the copy of *srcPathPtr*. It is never the name of a directory into which *srcPathPtr* could be copied (i.e.\ the function is much simpler than the Tcl level **file copy** subcommand). Note that, if the filesystem supports symbolic links, Tcl will always call this function and not *copyDirectoryProc* when needed to copy them (even if they are symbolic links to directories). Finally, if the filesystem determines it cannot support the **file copy** action, calling **Tcl_SetErrno(EXDEV)** and returning a non-**TCL_OK** result will tell Tcl to use its standard fallback mechanisms.

Renamefileproc

Function to process a **Tcl_FSRenameFile** call. If not implemented, Tcl will fall back on a copy and delete mechanism. Therefore it need only be implemented if the filesystem can perform that action more efficiently.

```
typedef int Tcl_FSRenameFileProc(
        Tcl_Obj *srcPathPtr,
        Tcl_Obj *destPathPtr);
```

The return value is a standard Tcl result indicating whether an error occurred in the renaming process. If the filesystem determines it cannot support the **file rename** action, calling **Tcl_SetErrno(EXDEV)** and returning a non-**TCL_OK** result will tell Tcl to use its standard fallback mechanisms.

Copydirectoryproc

Function to process a **Tcl_FSCopyDirectory** call. If not implemented, Tcl will fall back on a recursive **file mkdir**, **file copy** mechanism. Therefore it need only be implemented if the filesystem can perform that action more efficiently.

```
typedef int Tcl_FSCopyDirectoryProc(
        Tcl_Obj *srcPathPtr,
        Tcl_Obj *destPathPtr,
        Tcl_Obj **errorPtr);
```

The return value is a standard Tcl result indicating whether an error occurred in the copying process. If an error does occur, the name of the file or directory which caused the error should be placed in *errorPtr*. Note that, *destPathPtr* is the name of the directory-name which should become the mirror-image of *srcPathPtr*. It is not the name of a directory into which *srcPathPtr* should be copied (i.e.\ the function is much simpler than the Tcl level **file copy** subcommand). Finally, if the filesystem determines it cannot support the directory copy action, calling **Tcl_SetErrno(EXDEV)** and returning a non-**TCL_OK** result will tell Tcl to use its standard fallback mechanisms.

Loadfileproc

Function to process a **Tcl_FSLoadFile** call. If not implemented, Tcl will fall back on a copy to native-temp followed by a **Tcl_FSLoadFile** on that temporary copy. Therefore it need only be

implemented if the filesystem can load code directly, or it can be implemented simply to return **TCL_ERROR** to disable load functionality in this filesystem entirely.

```
typedef int Tcl_FSLoadFileProc(
        Tcl_Interp *interp,
        Tcl_Obj *pathPtr,
        Tcl_LoadHandle *handlePtr,
        Tcl_FSUnloadFileProc *unloadProcPtr);
```

Returns a standard Tcl completion code. If an error occurs, an error message is left in the *interp*'s result. The function dynamically loads a binary code file into memory. On a successful load, the *handlePtr* should be filled with a token for the dynamically loaded file, and the *unloadProcPtr* should be filled in with the address of a procedure. The unload procedure will be called with the given **Tcl_LoadHandle** as its only parameter when Tcl needs to unload the file. For example, for the native filesystem, the **Tcl_LoadHandle** returned is currently a token which can be used in the private **TclpFindSymbol** to access functions in the new code. Each filesystem is free to define the **Tcl_LoadHandle** as it requires. Finally, if the filesystem determines it cannot support the file load action, calling **Tcl_SetErrno(EXDEV)** and returning a non-**TCL_OK** result will tell Tcl to use its standard fallback mechanisms.

Unloadfileproc

Function to unload a previously successfully loaded file. If load was implemented, then this should also be implemented, if there is any cleanup action required.

```
typedef void Tcl_FSUnloadFileProc(
        Tcl_LoadHandle loadHandle);
```

Getcwdproc

Function to process a **Tcl_FSGetCwd** call. Most filesystems need not implement this. It will usually only be called once, if **getcwd** is called before **chdir**. May be NULL.

```
typedef Tcl_Obj *Tcl_FSGetCwdProc(
        Tcl_Interp *interp);
```

If the filesystem supports a native notion of a current working directory (which might perhaps change independent of Tcl), this function should return that cwd as the result, or NULL if the current directory could not be determined (e.g.\ the user does not have appropriate permissions on the cwd directory). If NULL is returned, an error message is left in the *interp*'s result.

Chdirproc

Function to process a **Tcl_FSChdir** call. If filesystems do not implement this, it will be emulated by a series of directory access checks. Otherwise, virtual filesystems which do implement it need only respond with a positive return result if the *pathPtr* is a valid, accessible directory in their filesystem. They need not remember the result, since that will be automatically remembered for use by **Tcl_FSGetCwd**. Real filesystems should carry out the correct action (i.e.\ call the correct system **chdir** API).

```
typedef int Tcl_FSChdirProc(
        Tcl_Obj *pathPtr);
```

The **Tcl_FSChdirProc** changes the applications current working directory to the value specified in *pathPtr*. The function returns -1 on error or 0 on success.

Keywords: FileSystem, stat, access, vfs, virtual filesystem

Copyright (c) 2001 Vincent Darley
Copyright (c) 2008-2010 Donal K. Fellows

FindExec

Name

Tcl_FindExecutable, Tcl_GetNameOfExecutable - identify or return the name of the binary file containing the application

Synopsis

#include <tcl.h>

void
Tcl_FindExecutable(*argv0*)

const char *
Tcl_GetNameOfExecutable()

Arguments

char ****argv0* **(in)
The first command-line argument to the program, which gives the application's name.

Description

The **Tcl_FindExecutable** procedure computes the full path name of the executable file from which the application was invoked and saves it for Tcl's internal use. The executable's path name is needed for several purposes in Tcl. For example, it is needed on some platforms in the implementation of the **load** command. It is also returned by the **info nameofexecutable** command.

On UNIX platforms this procedure is typically invoked as the very first thing in the application's main program; it must be passed *argv[0]* as its argument. It is important not to change the working directory before the invocation. **Tcl_FindExecutable** uses *argv0* along with the **PATH** environment variable to find the application's executable, if possible. If it fails to find the binary, then future calls to **info nameofexecutable** will return an empty string.

On Windows platforms this procedure is typically invoked as the very first thing in the application's main program as well; Its *argv[0]* argument is only used to indicate whether the executable has a stderr channel (any non-null value) or not (the value null). If **Tcl_SetPanicProc** is never called and no debugger is running, this determines whether the panic message is sent to stderr or to a standard system dialog.

Tcl_GetNameOfExecutable simply returns a pointer to the internal full path name of the executable file as computed by **Tcl_FindExecutable**. This procedure call is the C API equivalent to the **info nameofexecutable** command. NULL is returned if the internal full path name has not been computed or unknown.

Keywords: FindExec, binary, executable file

Copyright (c) 1995-1996 Sun Microsystems, Inc.

GetCwd

Name

Tcl_GetCwd, Tcl_Chdir - manipulate the current working directory

Synopsis

#include <tcl.h>

char *
Tcl_GetCwd(*interp*, *bufferPtr*)

int
Tcl_Chdir(*dirName*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter in which to report an error, if any.

Tcl_DString *_bufferPtr_ (in/out)
This dynamic string is used to store the current working directory. At the time of the call it should be uninitialized or free. The caller must eventually call **Tcl_DStringFree** to free up anything stored here.

const char *_dirName_ (in)
File path in UTF-8 format.

Description

These procedures may be used to manipulate the current working directory for the application. They provide C-level access to the same functionality as the Tcl **pwd** command.

Tcl_GetCwd returns a pointer to a string specifying the current directory, or NULL if the current directory could not be determined. If NULL is returned, an error message is left in the *interp*'s result. Storage for the result string is allocated in bufferPtr; the caller must call **Tcl_DStringFree()** when the result is no longer needed. The format of the path is UTF-8.

Tcl_Chdir changes the applications current working directory to the value specified in *dirName*. The format of the passed in string must be UTF-8. The function returns -1 on error or 0 on success.

Keywords: GetCwd, pwd

Copyright (c) 1998-1999 Scriptics Corporation

GetHostName

Name

Tcl_GetHostName - get the name of the local host

Synopsis

#include <tcl.h>

const char *

Tcl_GetHostName()

Description

Tcl_GetHostName is a utility procedure used by some of the Tcl commands. It returns a pointer to a string containing the name for the current machine, or an empty string if the name cannot be determined. The string is statically allocated, and the caller must not modify of free it.

Keywords: GetHostName, hostname

Copyright (c) 1998-2000 Scriptics Corporation.
All rights reserved.

GetIndex

Name

Tcl_GetIndexFromObj, Tcl_GetIndexFromObjStruct - lookup string in table of keywords

Synopsis

#include <tcl.h>

int
Tcl_GetIndexFromObj(*interp*, *objPtr*, *tablePtr*, *msg*, *flags*, *indexPtr*)

int
Tcl_GetIndexFromObjStruct(*interp*, *objPtr*, *structTablePtr*, *offset*, *msg*, *flags*, *indexPtr*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter to use for error reporting; if NULL, then no message is provided on errors.

Tcl_Obj *_objPtr_ (in/out)
The string value of this value is used to search through *tablePtr*. The internal representation is modified to hold the index of the matching table entry.

const char *const *_tablePtr_ (in)
An array of null-terminated strings. The end of the array is marked by a NULL string pointer. Note that references to the *tablePtr* may be retained in the internal representation of *objPtr*, so this should represent the address of a statically-allocated array.

const void *_structTablePtr_ (in)
An array of arbitrary type, typically some **struct** type. The first member of the structure must be a null-terminated string. The size of the structure is given by *offset*. Note that references to the *structTablePtr* may be retained in the internal representation of *objPtr*, so this should represent the address of a statically-allocated array of structures.

int _offset_ (in)
The offset to add to structTablePtr to get to the next entry. The end of the array is marked by a NULL string pointer.

const char *_msg_ (in)
Null-terminated string describing what is being looked up, such as **option**. This string is included in error messages.

int *flags* (in)
OR-ed combination of bits providing additional information for operation. The only bit that is currently defined is **TCL_EXACT**.

int *indexPtr (out)
The index of the string in *tablePtr* that matches the value of *objPtr* is returned here.

Description

These procedures provide an efficient way for looking up keywords, switch names, option names, and similar things where the literal value of a Tcl value must be chosen from a predefined set. **Tcl_GetIndexFromObj** compares *objPtr* against each of the strings in *tablePtr* to find a match. A match occurs if *objPtr*'s string value is identical to one of the strings in *tablePtr*, or if it is a non-empty unique abbreviation for exactly one of the strings in *tablePtr* and the **TCL_EXACT** flag was not specified; in either case the index of the matching entry is stored at *indexPtr and **TCL_OK** is returned.

If there is no matching entry, **TCL_ERROR** is returned and an error message is left in *interp*'s result if *interp* is not NULL. *Msg* is included in the error message to indicate what was being looked up. For example, if *msg* is **option** the error message will have a form like "**bad option** " **firt**": **must be first, second, or third**".

If **Tcl_GetIndexFromObj** completes successfully it modifies the internal representation of *objPtr* to hold the address of the table and the index of the matching entry. If **Tcl_GetIndexFromObj** is invoked again with the same *objPtr* and *tablePtr* arguments (e.g. during a reinvocation of a Tcl command), it returns the matching index immediately without having to redo the lookup operation. Note: **Tcl_GetIndexFromObj** assumes that the entries in *tablePtr* are static: they must not change between invocations. If the value of *objPtr* is the empty string, **Tcl_GetIndexFromObj** will treat it as a non-matching value and return **TCL_ERROR**.

Tcl_GetIndexFromObjStruct works just like **Tcl_GetIndexFromObj**, except that instead of treating *tablePtr* as an array of string pointers, it treats it as a pointer to the first string in a series of strings that have *offset* bytes between them (i.e. that there is a pointer to the first array of characters at *tablePtr*, a pointer to the second array of characters at *tablePtr*+*offset* bytes, etc.) This is particularly useful when processing things like **Tk_ConfigurationSpec**, whose string keys are in the same place in each of several array elements.

Keywords: GetIndex, index, option, value, table lookup

Copyright (c) 1997 Sun Microsystems, Inc.

GetInt

Name

Tcl_GetInt, Tcl_GetDouble, Tcl_GetBoolean - convert from string to integer, double, or boolean

Synopsis

#include <tcl.h>

int
Tcl_GetInt(*interp*, *src*, *intPtr*)

int
Tcl_GetDouble(*interp*, *src*, *doublePtr*)

int
Tcl_GetBoolean(*interp*, *src*, *intPtr*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter to use for error reporting.

const char *_src_ (in)
Textual value to be converted.

int *_intPtr_ (out)
Points to place to store integer value converted from *src*.

double *_doublePtr_ (out)
Points to place to store double-precision floating-point value converted from *src*.

Description

These procedures convert from strings to integers or double-precision floating-point values or booleans (represented as 0- or 1-valued integers). Each of the procedures takes a *src* argument, converts it to an internal form of a particular type, and stores the converted value at the location indicated by the procedure's third argument. If all goes well, each of the procedures returns **TCL_OK**. If *src* does not have the proper syntax for the desired type then **TCL_ERROR** is returned, an error message is left in the interpreter's result, and nothing is stored at *_intPtr_ or *_doublePtr_.

Tcl_GetInt expects *src* to consist of a collection of integer digits, optionally signed and optionally preceded and followed by white space. If the first two characters of *src* after the optional white space and sign are "**0x**" then *src* is expected to be in hexadecimal form; otherwise, if the first such characters are "**0o**" then *src* is expected to be in octal form; otherwise, if the first such characters are "**0b**" then *src* is expected to be in binary form; otherwise, if the first such character is "**0**" then *src* is expected to be in octal form; otherwise, *src* is expected to be in decimal form.

Tcl_GetDouble expects *src* to consist of a floating-point number, which is: white space; a sign; a sequence of digits; a decimal point "**.**"; a sequence of digits; the letter "**e**"; a signed decimal exponent; and more white space. Any of the fields may be omitted, except that the digits either before or after the decimal point must be present and if the "**e**" is present then it must be followed by the exponent number. If there are no fields apart from the sign and initial sequence of digits (i.e., no decimal point or exponent indicator), that initial sequence of digits should take one of the forms that **Tcl_GetInt** supports, described above. The use of "**,**" as a decimal point is not supported nor should any other sort of inter-digit separator be present.

Tcl_GetBoolean expects *src* to specify a boolean value. If *src* is any of **0**, **false**, **no**, or **off**, then **Tcl_GetBoolean** stores a zero value at *_intPtr_. If *src* is any of **1**, **true**, **yes**, or **on**, then 1 is stored at *_intPtr_. Any of these values may be abbreviated, and upper-case spellings are also acceptable.

Keywords: GetInt, boolean, conversion, double, floating-point, integer

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

GetOpnFl

Name

Tcl_GetOpenFile - Return a FILE* for a channel registered in the given interpreter (Unix only)

Synopsis

#include <tcl.h>

int
Tcl_GetOpenFile(*interp*, *chanID*, *write*, *checkUsage*, *filePtr*)

Arguments

Tcl_Interp *_interp_ (in)
Tcl interpreter from which file handle is to be obtained.

const char *_chanID_ (in)
String identifying channel, such as **stdin** or **file4**.

int _write_ (in)
Non-zero means the file will be used for writing, zero means it will be used for reading.

int _checkUsage_ (in)
If non-zero, then an error will be generated if the file was not opened for the access indicated by *write*.

ClientData *_filePtr_ (out)
Points to word in which to store pointer to FILE structure for the file given by *chanID*.

Description

Tcl_GetOpenFile takes as argument a file identifier of the form returned by the **open** command and returns at *_filePtr_ a pointer to the FILE structure for the file. The *write* argument indicates whether the FILE pointer will be used for reading or writing. In some cases, such as a channel that connects to a pipeline of subprocesses, different FILE pointers will be returned for reading and writing. **Tcl_GetOpenFile** normally returns **TCL_OK**. If an error occurs in **Tcl_GetOpenFile** (e.g. *chanID* did not make any sense or *checkUsage* was set and the file was not opened for the access specified by *write*) then **TCL_ERROR** is returned and the interpreter's result will contain an error message. In the current implementation *checkUsage* is ignored and consistency checks are always performed.

Note that this interface is only supported on the Unix platform.

Keywords: GetOpnFl, channel, file handle, permissions, pipeline, read, write

Copyright (c) 1996-1997 Sun Microsystems, Inc.

GetStdChan

Name

Tcl_GetStdChannel, Tcl_SetStdChannel - procedures for retrieving and replacing the standard channels

Synopsis

#include <tcl.h>

Tcl_Channel
Tcl_GetStdChannel(*type*)

Tcl_SetStdChannel(*channel*, *type*)

Arguments

int *type* (in)
The identifier for the standard channel to retrieve or modify. Must be one of **TCL_STDIN**, **TCL_STDOUT**, or **TCL_STDERR**.

Tcl_Channel *channel* (in)
The channel to use as the new value for the specified standard channel.

Description

Tcl defines three special channels that are used by various I/O related commands if no other channels are specified. The standard input channel has a channel name of **stdin** and is used by **read** and **gets**. The standard output channel is named **stdout** and is used by **puts**. The standard error channel is named **stderr** and is used for reporting errors. In addition, the standard channels are inherited by any child processes created using **exec** or **open** in the absence of any other redirections.

The standard channels are actually aliases for other normal channels. The current channel associated with a standard channel can be retrieved by calling **Tcl_GetStdChannel** with one of **TCL_STDIN**, **TCL_STDOUT**, or **TCL_STDERR** as the *type*. The return value will be a valid channel, or NULL.

A new channel can be set for the standard channel specified by *type* by calling **Tcl_SetStdChannel** with a new channel or NULL in the *channel* argument. If the specified channel is closed by a later call to **Tcl_Close**, then the corresponding standard channel will automatically be set to NULL.

If a non-NULL value for *channel* is passed to **Tcl_SetStdChannel**, then that same value should be passed to **Tcl_RegisterChannel**, like so:

```
Tcl_RegisterChannel(NULL, channel);
```

This is a workaround for a misfeature in **Tcl_SetStdChannel** that it fails to do some reference counting housekeeping. This misfeature cannot be corrected without contradicting the assumptions of some existing code that calls **Tcl_SetStdChannel**.

If **Tcl_GetStdChannel** is called before **Tcl_SetStdChannel**, Tcl will construct a new channel to wrap the appropriate platform-specific standard file handle. If **Tcl_SetStdChannel** is called before **Tcl_GetStdChannel**, then the default channel will not be created.

If one of the standard channels is set to NULL, either by calling **Tcl_SetStdChannel** with a NULL *channel* argument, or by calling **Tcl_Close** on the channel, then the next call to **Tcl_CreateChannel** will automatically set the standard channel with the newly created channel. If more than one standard channel is NULL, then the standard channels will be assigned starting with standard input, followed by standard output, with standard error being last.

See **Tcl_StandardChannels** for a general treatise about standard channels and the behavior of the Tcl library with regard to them.

Keywords: GetStdChan, standard channel, standard input, standard output, standard error

Copyright (c) 1996 Sun Microsystems, Inc.

GetTime

Name

Tcl_GetTime, Tcl_SetTimeProc, Tcl_QueryTimeProc - get date and time

Synopsis

#include <tcl.h>

Tcl_GetTime(*timePtr*)

Tcl_SetTimeProc(*getProc*, *scaleProc*, *clientData*)

Tcl_QueryTimeProc(*getProcPtr*, *scaleProcPtr*, *clientDataPtr*)

Arguments

Tcl_Time **timePtr* (out)
Points to memory in which to store the date and time information.

Tcl_GetTimeProc *getProc* (in)
Pointer to handler function replacing **Tcl_GetTime**'s access to the OS.

Tcl_ScaleTimeProc *scaleProc* (in)
Pointer to handler function for the conversion of time delays in the virtual domain to real-time.

ClientData *clientData* (in)
Value passed through to the two handler functions.

Tcl_GetTimeProc **getProcPtr* (out)
Pointer to place the currently registered get handler function into.

Tcl_ScaleTimeProc **scaleProcPtr* (out)
Pointer to place the currently registered scale handler function into.

ClientData **clientDataPtr* (out)
Pointer to place the currently registered pass-through value into.

Description

The **Tcl_GetTime** function retrieves the current time as a *Tcl_Time* structure in memory the caller provides. This structure has the following definition:

```
typedef struct Tcl_Time {
    long sec;
    long usec;
} Tcl_Time;
```

On return, the *sec* member of the structure is filled in with the number of seconds that have elapsed since the *epoch:* the epoch is the point in time of 00:00 UTC, 1 January 1970. This number does *not* count leap seconds - an interval of one day advances it by 86400 seconds regardless of whether a leap second has been inserted.

The *usec* member of the structure is filled in with the number of microseconds that have elapsed since the start of the second designated by *sec*. The Tcl library makes every effort to keep this number as precise as possible, subject to the limitations of the computer system. On

multiprocessor variants of Windows, this number may be limited to the 10- or 20-ms granularity of the system clock. (On single-processor Windows systems, the *usec* field is derived from a performance counter and is highly precise.)

Virtualized time

The **Tcl_SetTimeProc** function registers two related handler functions with the core. The first handler function is a replacement for **Tcl_GetTime**, or rather the OS access made by **Tcl_GetTime**. The other handler function is used by the Tcl notifier to convert wait/block times from the virtual domain into real time.

The **Tcl_QueryTimeProc** function returns the currently registered handler functions. If no external handlers were set then this will return the standard handlers accessing and processing the native time of the OS. The arguments to the function are allowed to be NULL; and any argument which is NULL is ignored and not set.

The signatures of the handler functions are as follows:

```
typedef void Tcl_GetTimeProc(
        Tcl_Time *timebuf,
        ClientData clientData);
typedef void Tcl_ScaleTimeProc(
        Tcl_Time *timebuf,
        ClientData clientData);
```

The *timebuf* fields contain the time to manipulate, and the *clientData* fields contain a pointer supplied at the time the handler functions were registered.

Any handler pair specified has to return data which is consistent between them. In other words, setting one handler of the pair to something assuming a 10-times slowdown, and the other handler of the pair to something assuming a two-times slowdown is wrong and not allowed.

The set handler functions are allowed to run the delivered time backwards, however this should be avoided. We have to allow it as the native time can run backwards as the user can fiddle with the system time one way or other. Note that the insertion of the hooks will not change the behavior of the Tcl core with regard to this situation, i.e. the existing behavior is retained.

Keywords: GetTime, date, time

Copyright (c) 2001 Kevin B. Kenny kennykb@acm.org*.*

GetVersion

Name

Tcl_GetVersion - get the version of the library at runtime

Synopsis

#include <tcl.h>

Tcl_GetVersion(*major*, *minor*, *patchLevel*, *type*)

Arguments

int ****major* **(out)

Major version number of the Tcl library.

int *_minor_ (out)
Minor version number of the Tcl library.

int *_patchLevel_ (out)
The patch level of the Tcl library (or alpha or beta number).

Tcl_ReleaseType *_type_ (out)
The type of release, also indicates the type of patch level. Can be one of **TCL_ALPHA_RELEASE**, **TCL_BETA_RELEASE**, or **TCL_FINAL_RELEASE**.

Description

Tcl_GetVersion should be used to query the version number of the Tcl library at runtime. This is useful when using a dynamically loaded Tcl library or when writing a stubs-aware extension. For instance, if you write an extension that is linked against the Tcl stubs library, it could be loaded into a program linked to an older version of Tcl than you expected. Use **Tcl_GetVersion** to verify that fact, and possibly to change the behavior of your extension.

Tcl_GetVersion accepts NULL for any of the arguments. For instance if you do not care about the *patchLevel* of the library, pass a NULL for the *patchLevel* argument.

Keywords: GetVersion, version, patchlevel, major, minor, alpha, beta, release

Copyright (c) 1999 Scriptics Corporation

Hash

Name

Tcl_InitHashTable, Tcl_InitCustomHashTable, Tcl_InitObjHashTable, Tcl_DeleteHashTable, Tcl_CreateHashEntry, Tcl_DeleteHashEntry, Tcl_FindHashEntry, Tcl_GetHashValue, Tcl_SetHashValue, Tcl_GetHashKey, Tcl_FirstHashEntry, Tcl_NextHashEntry, Tcl_HashStats - procedures to manage hash tables

Synopsis

#include <tcl.h>

Tcl_InitHashTable(*tablePtr*, *keyType*)

Tcl_InitCustomHashTable(*tablePtr*, *keyType*, *typePtr*)

Tcl_InitObjHashTable(*tablePtr*)

Tcl_DeleteHashTable(*tablePtr*)

Tcl_HashEntry *
Tcl_CreateHashEntry(*tablePtr*, *key*, *newPtr*)

Tcl_DeleteHashEntry(*entryPtr*)

Tcl_HashEntry *
Tcl_FindHashEntry(*tablePtr*, *key*)

ClientData
Tcl_GetHashValue(*entryPtr*)

Tcl_SetHashValue(*entryPtr*, *value*)

void *
Tcl_GetHashKey(*tablePtr*, *entryPtr*)

Tcl_HashEntry *
Tcl_FirstHashEntry(*tablePtr*, *searchPtr*)

Tcl_HashEntry *
Tcl_NextHashEntry(*searchPtr*)

char *
Tcl_HashStats(*tablePtr*)

Arguments

Tcl_HashTable ****tablePtr* **(in)
Address of hash table structure (for all procedures but **Tcl_InitHashTable**, this must have been initialized by previous call to **Tcl_InitHashTable**).

int *keyType* (in)
Kind of keys to use for new hash table. Must be either **TCL_STRING_KEYS**, **TCL_ONE_WORD_KEYS**, **TCL_CUSTOM_TYPE_KEYS**, **TCL_CUSTOM_PTR_KEYS**, or an integer value greater than 1.

Tcl_HashKeyType ****typePtr* **(in)
Address of structure which defines the behavior of the hash table.

const void ****key* **(in)
Key to use for probe into table. Exact form depends on *keyType* used to create table.

int ****newPtr* **(out)
The word at **newPtr* is set to 1 if a new entry was created and 0 if there was already an entry for *key*.

Tcl_HashEntry ****entryPtr* **(in)
Pointer to hash table entry.

ClientData *value* (in)
New value to assign to hash table entry. Need not have type ClientData, but must fit in same space as ClientData.

Tcl_HashSearch ****searchPtr* **(in)
Pointer to record to use to keep track of progress in enumerating all the entries in a hash table.

Description

A hash table consists of zero or more entries, each consisting of a key and a value. Given the key for an entry, the hashing routines can very quickly locate the entry, and hence its value. There may be at most one entry in a hash table with a particular key, but many entries may have the same value. Keys can take one of four forms: strings, one-word values, integer arrays, or custom keys defined by a Tcl_HashKeyType structure (See section **THE TCL_HASHKEYTYPE STRUCTURE** below). All of the keys in a given table have the same form, which is specified when the table is initialized.

The value of a hash table entry can be anything that fits in the same space as a "char *" pointer. Values for hash table entries are managed entirely by clients, not by the hash module itself. Typically each entry's value is a pointer to a data structure managed by client code.

Hash tables grow gracefully as the number of entries increases, so that there are always less than three entries per hash bucket, on average. This allows for fast lookups regardless of the

number of entries in a table.

The core provides three functions for the initialization of hash tables, Tcl_InitHashTable, Tcl_InitObjHashTable and Tcl_InitCustomHashTable.

Tcl_InitHashTable initializes a structure that describes a new hash table. The space for the structure is provided by the caller, not by the hash module. The value of *keyType* indicates what kinds of keys will be used for all entries in the table. All of the key types described later are allowed, with the exception of **TCL_CUSTOM_TYPE_KEYS** and **TCL_CUSTOM_PTR_KEYS**.

Tcl_InitObjHashTable is a wrapper around **Tcl_InitCustomHashTable** and initializes a hash table whose keys are Tcl_Obj *.

Tcl_InitCustomHashTable initializes a structure that describes a new hash table. The space for the structure is provided by the caller, not by the hash module. The value of *keyType* indicates what kinds of keys will be used for all entries in the table. *KeyType* must have one of the following values:

TCL_STRING_KEYS
Keys are null-terminated strings. They are passed to hashing routines using the address of the first character of the string.

TCL_ONE_WORD_KEYS
Keys are single-word values; they are passed to hashing routines and stored in hash table entries as "char *" values. The pointer value is the key; it need not (and usually does not) actually point to a string.

TCL_CUSTOM_TYPE_KEYS
Keys are of arbitrary type, and are stored in the entry. Hashing and comparison is determined by *typePtr*. The Tcl_HashKeyType structure is described in the section **THE TCL_HASHKEYTYPE STRUCTURE** below.

TCL_CUSTOM_PTR_KEYS
Keys are pointers to an arbitrary type, and are stored in the entry. Hashing and comparison is determined by *typePtr*. The Tcl_HashKeyType structure is described in the section **THE TCL_HASHKEYTYPE STRUCTURE** below.

other
If *keyType* is not one of the above, then it must be an integer value greater than 1. In this case the keys will be arrays of "int" values, where *keyType* gives the number of ints in each key. This allows structures to be used as keys. All keys must have the same size. Array keys are passed into hashing functions using the address of the first int in the array.
Tcl_DeleteHashTable deletes all of the entries in a hash table and frees up the memory associated with the table's bucket array and entries. It does not free the actual table structure (pointed to by *tablePtr*), since that memory is assumed to be managed by the client.
Tcl_DeleteHashTable also does not free or otherwise manipulate the values of the hash table entries. If the entry values point to dynamically-allocated memory, then it is the client's responsibility to free these structures before deleting the table. **Tcl_CreateHashEntry** locates the entry corresponding to a particular key, creating a new entry in the table if there was not already one with the given key. If an entry already existed with the given key then **newPtr* is set to zero. If a new entry was created, then **newPtr* is set to a non-zero value and the value of the new entry will be set to zero. The return value from **Tcl_CreateHashEntry** is a pointer to the entry, which may be used to retrieve and modify the entry's value or to

delete the entry from the table. **Tcl_DeleteHashEntry** will remove an existing entry from a table. The memory associated with the entry itself will be freed, but the client is responsible for any cleanup associated with the entry's value, such as freeing a structure that it points to. **Tcl_FindHashEntry** is similar to **Tcl_CreateHashEntry** except that it does not create a new entry if the key doesn't exist; instead, it returns NULL as result. **Tcl_GetHashValue** and **Tcl_SetHashValue** are used to read and write an entry's value, respectively. Values are stored and retrieved as type "ClientData", which is large enough to hold a pointer value. On almost all machines this is large enough to hold an integer value too. **Tcl_GetHashKey** returns the key for a given hash table entry, either as a pointer to a string, a one-word ("char *") key, or as a pointer to the first word of an array of integers, depending on the *keyType* used to create a hash table. In all cases **Tcl_GetHashKey** returns a result with type "char *". When the key is a string or array, the result of **Tcl_GetHashKey** points to information in the table entry; this information will remain valid until the entry is deleted or its table is deleted. **Tcl_FirstHashEntry** and **Tcl_NextHashEntry** may be used to scan all of the entries in a hash table. A structure of type "Tcl_HashSearch", provided by the client, is used to keep track of progress through the table. **Tcl_FirstHashEntry** initializes the search record and returns the first entry in the table (or NULL if the table is empty). Each subsequent call to **Tcl_NextHashEntry** returns the next entry in the table or NULL if the end of the table has been reached. A call to **Tcl_FirstHashEntry** followed by calls to **Tcl_NextHashEntry** will return each of the entries in the table exactly once, in an arbitrary order. It is inadvisable to modify the structure of the table, e.g. by creating or deleting entries, while the search is in progress, with the exception of deleting the entry returned by **Tcl_FirstHashEntry** or **Tcl_NextHashEntry**. **Tcl_HashStats** returns a dynamically-allocated string with overall information about a hash table, such as the number of entries it contains, the number of buckets in its hash array, and the utilization of the buckets. It is the caller's responsibility to free the result string by passing it to **ckfree**. The header file **tcl.h** defines the actual data structures used to implement hash tables. This is necessary so that clients can allocate Tcl_HashTable structures and so that macros can be used to read and write the values of entries. However, users of the hashing routines should never refer directly to any of the fields of any of the hash-related data structures; use the procedures and macros defined here.

The tcl_hashkeytype structure

Extension writers can define new hash key types by defining four procedures, initializing a **Tcl_HashKeyType** structure to describe the type, and calling **Tcl_InitCustomHashTable**. The **Tcl_HashKeyType** structure is defined as follows:

```
typedef struct Tcl_HashKeyType {
    int version;
    int flags;
    Tcl_HashKeyProc *hashKeyProc;
    Tcl_CompareHashKeysProc *compareKeysProc;
    Tcl_AllocHashEntryProc *allocEntryProc;
    Tcl_FreeHashEntryProc *freeEntryProc;
} Tcl_HashKeyType;
```

The *version* member is the version of the table. If this structure is extended in future then the version can be used to distinguish between different structures. It should be set to

TCL_HASH_KEY_TYPE_VERSION.

The *flags* member is 0 or one or more of the following values OR'ed together:

TCL_HASH_KEY_RANDOMIZE_HASH
: There are some things, pointers for example which do not hash well because they do not use the lower bits. If this flag is set then the hash table will attempt to rectify this by randomizing the bits and then using the upper N bits as the index into the table.

TCL_HASH_KEY_SYSTEM_HASH
: This flag forces Tcl to use the memory allocation procedures provided by the operating system when allocating and freeing memory used to store the hash table data structures, and not any of Tcl's own customized memory allocation routines. This is important if the hash table is to be used in the implementation of a custom set of allocation routines, or something that a custom set of allocation routines might depend on, in order to avoid any circular dependency. The *hashKeyProc* member contains the address of a function called to calculate a hash value for the key.

```
typedef unsigned int Tcl_HashKeyProc(
        Tcl_HashTable *tablePtr,
        void *keyPtr);
```

If this is NULL then *keyPtr* is used and **TCL_HASH_KEY_RANDOMIZE_HASH** is assumed.

The *compareKeysProc* member contains the address of a function called to compare two keys.

```
typedef int Tcl_CompareHashKeysProc(
        void *keyPtr,
        Tcl_HashEntry *hPtr);
```

If this is NULL then the *keyPtr* pointers are compared. If the keys do not match then the function returns 0, otherwise it returns 1.

The *allocEntryProc* member contains the address of a function called to allocate space for an entry and initialize the key and clientData.

```
typedef Tcl_HashEntry *Tcl_AllocHashEntryProc(
        Tcl_HashTable *tablePtr,
        void *keyPtr);
```

If this is NULL then **Tcl_Alloc** is used to allocate enough space for a Tcl_HashEntry, the key pointer is assigned to key.oneWordValue and the clientData is set to NULL. String keys and array keys use this function to allocate enough space for the entry and the key in one block, rather than doing it in two blocks. This saves space for a pointer to the key from the entry and another memory allocation. Tcl_Obj* keys use this function to allocate enough space for an entry and increment the reference count on the value.

The *freeEntryProc* member contains the address of a function called to free space for an entry.

```
typedef void Tcl_FreeHashEntryProc(
        Tcl_HashEntry *hPtr);
```

If this is NULL then **Tcl_Free** is used to free the space for the entry. Tcl_Obj* keys use this function to decrement the reference count on the value.

Keywords: Hash, hash table, key, lookup, search, value

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

Init

Name
Tcl_Init - find and source initialization script

Synopsis
#include <tcl.h>

int
Tcl_Init(*interp*)

Arguments
Tcl_Interp *_interp_ (in)
Interpreter to initialize.

Description
Tcl_Init is a helper procedure that finds and **source**s the **init.tcl** script, which should exist somewhere on the Tcl library path.

Tcl_Init is typically called from **Tcl_AppInit** procedures.

Keywords: Init, application, initialization, interpreter

Copyright (c) 1998-2000 Scriptics Corporation.
All rights reserved.

InitStubs

Name
Tcl_InitStubs - initialize the Tcl stubs mechanism

Synopsis
#include <tcl.h>

const char *
Tcl_InitStubs(*interp*, *version*, *exact*)

Arguments
Tcl_Interp *_interp_ (in)
Tcl interpreter handle.

const char *_version_ (in)
A version string, indicating which minimal version of Tcl is accepted. Normally just **"8.6"**. Or **"8.6-"** if both 8.6 and 9.0 are accepted.

int _exact_ (in)
1 means that only the particular version specified by *version* is accepted. 0 means that versions newer than *version* are also accepted. If the*version* ends with **-**, higher major versions are accepted as well, otherwise the major version must be the same as in *version*.

Other bits have no effect.

Introduction

The Tcl stubs mechanism defines a way to dynamically bind extensions to a particular Tcl implementation at run time. This provides two significant benefits to Tcl users:

1)
Extensions that use the stubs mechanism can be loaded into multiple versions of Tcl without being recompiled or relinked, as long as the major Tcl version is the same.

2)
Extensions that use the stubs mechanism can be dynamically loaded into statically-linked Tcl applications. The stubs mechanism accomplishes this by exporting function tables that define an interface to the Tcl API. The extension then accesses the Tcl API through offsets into the function table, so there are no direct references to any of the Tcl library's symbols. This redirection is transparent to the extension, so an extension writer can continue to use all public Tcl functions as documented. The stubs mechanism requires no changes to applications incorporating Tcl interpreters. Only developers creating C-based Tcl extensions need to take steps to use the stubs mechanism with their extensions. Enabling the stubs mechanism for an extension requires the following steps:

1)
Call **Tcl_InitStubs** in the extension before calling any other Tcl functions.

2)
Define the **USE_TCL_STUBS** symbol. Typically, you would include the **-DUSE_TCL_STUBS** flag when compiling the extension.

3)
Link the extension with the Tcl stubs library instead of the standard Tcl library. For example, to use the Tcl 8.6 ABI on Unix platforms, the library name is *libtclstub8.6.a*; on Windows platforms, the library name is *tclstub86.lib*. If the extension also requires the Tk API, it must also call **Tk_InitStubs** to initialize the Tk stubs interface and link with the Tk stubs libraries. See the **Tk_InitStubs** page for more information.

Description

Tcl_InitStubs attempts to initialize the stub table pointers and ensure that the correct version of Tcl is loaded. In addition to an interpreter handle, it accepts as arguments a version number and a Boolean flag indicating whether the extension requires an exact version match or not. If *exact* is 0, then versions newer than *version* are also accepted. If the*version* ends with **-**, higher major versions are accepted as well, otherwise the major version must be the same as in *version*. 1 means that only the specified *version* is accepted. *version* can be any construct as described for **package require** (**PACKAGE** manual page in the section **REQUIREMENT**). Multiple requirement strings like with **package require** are not supported.

Tcl_InitStubs returns a string containing the actual version of Tcl satisfying the request, or NULL if the Tcl version is not accepted, does not support stubs, or any other error condition occurred.

Keywords: InitStubs, stubs

Copyright (c) 1998-1999 Scriptics Corporation

IntObj

Name

Tcl_NewIntObj, Tcl_NewLongObj, Tcl_NewWideIntObj, Tcl_SetIntObj, Tcl_SetLongObj, Tcl_SetWideIntObj, Tcl_GetIntFromObj, Tcl_GetLongFromObj, Tcl_GetWideIntFromObj, Tcl_NewBignumObj, Tcl_SetBignumObj, Tcl_GetBignumFromObj, Tcl_TakeBignumFromObj - manipulate Tcl values as integers

Synopsis

#include <tcl.h>

Tcl_Obj *
Tcl_NewIntObj(*intValue*)

Tcl_Obj *
Tcl_NewLongObj(*longValue*)

Tcl_Obj *
Tcl_NewWideIntObj(*wideValue*)

Tcl_SetIntObj(*objPtr*, *intValue*)

Tcl_SetLongObj(*objPtr*, *longValue*)

Tcl_SetWideIntObj(*objPtr*, *wideValue*)

int
Tcl_GetIntFromObj(*interp*, *objPtr*, *intPtr*)

int
Tcl_GetLongFromObj(*interp*, *objPtr*, *longPtr*)

int
Tcl_GetWideIntFromObj(*interp*, *objPtr*, *widePtr*)

#include <tclTomMath.h>

Tcl_Obj *
Tcl_NewBignumObj(*bigValue*)

Tcl_SetBignumObj(*objPtr*, *bigValue*)

int
Tcl_GetBignumFromObj(*interp*, *objPtr*, *bigValue*)

int
Tcl_TakeBignumFromObj(*interp*, *objPtr*, *bigValue*)

int
Tcl_InitBignumFromDouble(*interp*, *doubleValue*, *bigValue*)

Arguments

int *intValue* (in)
Integer value used to initialize or set a Tcl value.

long *longValue* (in)
Long integer value used to initialize or set a Tcl value.

Tcl_WideInt *wideValue* (in)

Wide integer value used to initialize or set a Tcl value.

Tcl_Obj *_objPtr_ (in/out)
For **Tcl_SetIntObj**, **Tcl_SetLongObj**, **Tcl_SetWideIntObj**, and **Tcl_SetBignumObj**, this points to the value in which to store an integral value. For **Tcl_GetIntFromObj**, **Tcl_GetLongFromObj**, **Tcl_GetWideIntFromObj**, **Tcl_GetBignumFromObj**, and **Tcl_TakeBignumFromObj**, this refers to the value from which to retrieve an integral value.

Tcl_Interp *_interp_ (in/out)
When non-NULL, an error message is left here when integral value retrieval fails.

int *_intPtr_ (out)
Points to place to store the integer value retrieved from *objPtr*.

long *_longPtr_ (out)
Points to place to store the long integer value retrieved from *objPtr*.

Tcl_WideInt *_widePtr_ (out)
Points to place to store the wide integer value retrieved from *objPtr*.

mp_int *_bigValue_ (in/out)
Points to a multi-precision integer structure declared by the LibTomMath library.

double _doubleValue_ (in)
Double value from which the integer part is determined and used to initialize a multi-precision integer value.

Description

These procedures are used to create, modify, and read Tcl values that hold integral values.

The different routines exist to accommodate different integral types in C with which values might be exchanged. The C integral types for which Tcl provides value exchange routines are **int** , **long int**, **Tcl_WideInt**, and **mp_int**. The **int** and **long int** types are provided by the C language standard. The **Tcl_WideInt** type is a typedef defined to be whatever signed integral type covers at least the 64-bit integer range (-9223372036854775808 to 9223372036854775807). Depending on the platform and the C compiler, the actual type might be **long int**, **long long int**, **__int64**, or something else. The **mp_int** type is a multiple-precision integer type defined by the LibTomMath multiple-precision integer library.

The **Tcl_NewIntObj**, **Tcl_NewLongObj**, **Tcl_NewWideIntObj**, and **Tcl_NewBignumObj** routines each create and return a new Tcl value initialized to the integral value of the argument. The returned Tcl value is unshared.

The **Tcl_SetIntObj**, **Tcl_SetLongObj**, **Tcl_SetWideIntObj**, and **Tcl_SetBignumObj** routines each set the value of an existing Tcl value pointed to by *objPtr* to the integral value provided by the other argument. The *objPtr* argument must point to an unshared Tcl value. Any attempt to set the value of a shared Tcl value violates Tcl's copy-on-write policy. Any existing string representation or internal representation in the unshared Tcl value will be freed as a consequence of setting the new value.

The **Tcl_GetIntFromObj**, **Tcl_GetLongFromObj**, **Tcl_GetWideIntFromObj**, **Tcl_GetBignumFromObj**, and **Tcl_TakeBignumFromObj** routines attempt to retrieve an integral value of the appropriate type from the Tcl value *objPtr*. If the attempt succeeds, then **TCL_OK** is returned, and the value is written to the storage provided by the caller. The attempt might fail if *objPtr* does not hold an integral value, or if the value exceeds the range of the target

type. If the attempt fails, then **TCL_ERROR** is returned, and if *interp* is non-NULL, an error message is left in *interp*. The **Tcl_ObjType** of *objPtr* may be changed to make subsequent calls to the same routine more efficient. Unlike the other functions, **Tcl_TakeBignumFromObj** may set the content of the Tcl value *objPtr* to an empty string in the process of retrieving the multiple-precision integer value.

The choice between **Tcl_GetBignumFromObj** and **Tcl_TakeBignumFromObj** is governed by how the caller will continue to use *objPtr*. If after the **mp_int** value is retrieved from *objPtr*, the caller will make no more use of *objPtr*, then using **Tcl_TakeBignumFromObj** permits Tcl to detect when an unshared *objPtr* permits the value to be moved instead of copied, which should be more efficient. If anything later in the caller requires *objPtr* to continue to hold the same value, then **Tcl_GetBignumFromObj** must be chosen.

The **Tcl_InitBignumFromDouble** routine is a utility procedure that extracts the integer part of *doubleValue* and stores that integer value in the **mp_int** value *bigValue*.

Keywords: IntObj, integer, integer value, integer type, internal representation, value, value type, string representation

Copyright (c) 1996-1997 Sun Microsystems, Inc.

Interp

Name

Tcl_Interp - client-visible fields of interpreter structures

Synopsis

#include <tcl.h>

typedef struct {
char **result*;
Tcl_FreeProc **freeProc*;
int *errorLine*;
} Tcl_Interp;

typedef void **Tcl_FreeProc**(
char **blockPtr*);

Description

The **Tcl_CreateInterp** procedure returns a pointer to a Tcl_Interp structure. Callers of **Tcl_CreateInterp** should use this pointer as an opaque token, suitable for nothing other than passing back to other routines in the Tcl interface. Accessing fields directly through the pointer as described below is no longer supported. The supported public routines **Tcl_SetResult**, **Tcl_GetResult**, **Tcl_SetErrorLine**, **Tcl_GetErrorLine** must be used instead.

For legacy programs and extensions no longer being maintained, compiles against the Tcl 8.6 header files are only possible with the compiler directives

```
#define USE_INTERP_RESULT
```

and/or

```
#define USE_INTERP_ERRORLINE
```

depending on which fields of the **Tcl_Interp** struct are accessed. These directives may be embedded in code or supplied via compiler options.

The *result* and *freeProc* fields are used to return results or error messages from commands. This information is returned by command procedures back to **Tcl_Eval**, and by **Tcl_Eval** back to its callers. The *result* field points to the string that represents the result or error message, and the *freeProc* field tells how to dispose of the storage for the string when it is not needed anymore. The easiest way for command procedures to manipulate these fields is to call procedures like **Tcl_SetResult** or **Tcl_AppendResult**; they will hide all the details of managing the fields. The description below is for those procedures that manipulate the fields directly.

Whenever a command procedure returns, it must ensure that the *result* field of its interpreter points to the string being returned by the command. The *result* field must always point to a valid string. If a command wishes to return no result then *interp->result* should point to an empty string. Normally, results are assumed to be statically allocated, which means that the contents will not change before the next time **Tcl_Eval** is called or some other command procedure is invoked. In this case, the *freeProc* field must be zero. Alternatively, a command procedure may dynamically allocate its return value (e.g. using **Tcl_Alloc**) and store a pointer to it in *interp->result*. In this case, the command procedure must also set *interp->freeProc* to the address of a procedure that can free the value, or **TCL_DYNAMIC** if the storage was allocated directly by Tcl or by a call to **Tcl_Alloc**. If *interp->freeProc* is non-zero, then Tcl will call *freeProc* to free the space pointed to by *interp->result* before it invokes the next command. If a client procedure overwrites *interp->result* when *interp->freeProc* is non-zero, then it is responsible for calling *freeProc* to free the old *interp->result* (the **Tcl_FreeResult** macro should be used for this purpose).

FreeProc should have arguments and result that match the **Tcl_FreeProc** declaration above: it receives a single argument which is a pointer to the result value to free. In most applications **TCL_DYNAMIC** is the only non-zero value ever used for *freeProc*. However, an application may store a different procedure address in *freeProc* in order to use an alternate memory allocator or in order to do other cleanup when the result memory is freed.

As part of processing each command, **Tcl_Eval** initializes *interp->result* and *interp->freeProc* just before calling the command procedure for the command. The *freeProc* field will be initialized to zero, and *interp->result* will point to an empty string. Commands that do not return any value can simply leave the fields alone. Furthermore, the empty string pointed to by *result* is actually part of an array of **TCL_RESULT_SIZE** characters (approximately 200). If a command wishes to return a short string, it can simply copy it to the area pointed to by *interp->result*. Or, it can use the sprintf procedure to generate a short result string at the location pointed to by *interp->result* .

It is a general convention in Tcl-based applications that the result of an interpreter is normally in the initialized state described in the previous paragraph. Procedures that manipulate an interpreter's result (e.g. by returning an error) will generally assume that the result has been initialized when the procedure is called. If such a procedure is to be called after the result has been changed, then **Tcl_ResetResult** should be called first to reset the result to its initialized state. The direct use of *interp->result* is strongly deprecated (see **Tcl_SetResult**).

The *errorLine* field is valid only after **Tcl_Eval** returns a **TCL_ERROR** return code. In this situation the *errorLine* field identifies the line number of the command being executed when the error occurred. The line numbers are relative to the command being executed: 1 means the first

line of the command passed to **Tcl_Eval**, 2 means the second line, and so on. The *errorLine* field is typically used in conjunction with **Tcl_AddErrorInfo** to report information about where an error occurred. *ErrorLine* should not normally be modified except by **Tcl_Eval**.

Keywords: Interp, free, initialized, interpreter, malloc, result

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

Limit

Name

Tcl_LimitAddHandler, Tcl_LimitCheck, Tcl_LimitExceeded, Tcl_LimitGetCommands, Tcl_LimitGetGranularity, Tcl_LimitGetTime, Tcl_LimitReady, Tcl_LimitRemoveHandler, Tcl_LimitSetCommands, Tcl_LimitSetGranularity, Tcl_LimitSetTime, Tcl_LimitTypeEnabled, Tcl_LimitTypeExceeded, Tcl_LimitTypeReset, Tcl_LimitTypeSet - manage and check resource limits on interpreters

Synopsis

#include <tcl.h>

int
Tcl_LimitCheck(*interp*)

int
Tcl_LimitReady(*interp*)

int
Tcl_LimitExceeded(*interp*)

int
Tcl_LimitTypeExceeded(*interp*, *type*)

int
Tcl_LimitTypeEnabled(*interp*, *type*)

void
Tcl_LimitTypeSet(*interp*, *type*)

void
Tcl_LimitTypeReset(*interp*, *type*)

int
Tcl_LimitGetCommands(*interp*)

void
Tcl_LimitSetCommands(*interp*, *commandLimit*)

void
Tcl_LimitGetTime(*interp*, *timeLimitPtr*)

void
Tcl_LimitSetTime(*interp*, *timeLimitPtr*)

int
Tcl_LimitGetGranularity(*interp*, *type*)

void

Tcl_LimitSetGranularity(*interp*, *type*, *granularity*)

void
Tcl_LimitAddHandler(*interp*, *type*, *handlerProc*, *clientData*, *deleteProc*)

void
Tcl_LimitRemoveHandler(*interp*, *type*, *handlerProc*, *clientData*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter that the limit being managed applies to or that will have its limits checked.

int _type_ (in)
The type of limit that the operation refers to. This must be either **TCL_LIMIT_COMMANDS** or **TCL_LIMIT_TIME**.

int _commandLimit_ (in)
The maximum number of commands (as reported by **info cmdcount**) that may be executed in the interpreter.

Tcl_Time *_timeLimitPtr_ (in/out)
A pointer to a structure that will either have the new time limit read from (**Tcl_LimitSetTime**) or the current time limit written to (**Tcl_LimitGetTime**).

int _granularity_ (in)
Divisor that indicates how often a particular limit should really be checked. Must be at least 1.

Tcl_LimitHandlerProc *_handlerProc_ (in)
Function to call when a particular limit is exceeded. If the *handlerProc* removes or raises the limit during its processing, the limited interpreter will be permitted to continue to process after the handler returns. Many handlers may be attached to the same interpreter limit; their order of execution is not defined, and they must be identified by *handlerProc* and *clientData* when they are deleted.

ClientData _clientData_ (in)
Arbitrary pointer-sized word used to pass some context to the *handlerProc* function.

Tcl_LimitHandlerDeleteProc *_deleteProc_ (in)
Function to call whenever a handler is deleted. May be NULL if the *clientData* requires no deletion.

Description

Tcl's interpreter resource limit subsystem allows for close control over how much computation time a script may use, and is useful for cases where a program is divided into multiple pieces where some parts are more trusted than others (e.g. web application servers).

Every interpreter may have a limit on the wall-time for execution, and a limit on the number of commands that the interpreter may execute. Since checking of these limits is potentially expensive (especially the time limit), each limit also has a checking granularity, which is a divisor for an internal count of the number of points in the core where a check may be performed (which is immediately before executing a command and at an unspecified frequency between running commands, which can happen in empty-bodied **while** loops).

The final component of the limit engine is a callback scheme which allows for notifications of when a limit has been exceeded. These callbacks can just provide logging, or may allocate more

resources to the interpreter to permit it to continue processing longer.

When a limit is exceeded (and the callbacks have run; the order of execution of the callbacks is unspecified) execution in the limited interpreter is stopped by raising an error and setting a flag that prevents the **catch** command in that interpreter from trapping that error. It is up to the context that started execution in that interpreter (typically the main interpreter) to handle the error.

Limit checking api

To check the resource limits for an interpreter, call **Tcl_LimitCheck**, which returns **TCL_OK** if the limit was not exceeded (after processing callbacks) and **TCL_ERROR** if the limit was exceeded (in which case an error message is also placed in the interpreter result). That function should only be called when **Tcl_LimitReady** returns non-zero so that granularity policy is enforced. This API is designed to be similar in usage to **Tcl_AsyncReady** and **Tcl_AsyncInvoke**.

When writing code that may behave like **catch** in respect of errors, you should only trap an error if **Tcl_LimitExceeded** returns zero. If it returns non-zero, the interpreter is in a limit-exceeded state and errors should be allowed to propagate to the calling context. You can also check whether a particular type of limit has been exceeded using **Tcl_LimitTypeExceeded**.

Limit configuration

To check whether a limit has been set (but not whether it has actually been exceeded) on an interpreter, call **Tcl_LimitTypeEnabled** with the type of limit you want to check. To enable a particular limit call **Tcl_LimitTypeSet**, and to disable a limit call **Tcl_LimitTypeReset**.

The level of a command limit may be set using **Tcl_LimitSetCommands**, and retrieved using **Tcl_LimitGetCommands**. Similarly for a time limit with **Tcl_LimitSetTime** and **Tcl_LimitGetTime** respectively, but with that API the time limit is copied from and to the Tcl_Time structure that the *timeLimitPtr* argument points to.

The checking granularity for a particular limit may be set using **Tcl_LimitSetGranularity** and retrieved using **Tcl_LimitGetGranularity**. Note that granularities must always be positive.

Limit callbacks

To add a handler callback to be invoked when a limit is exceeded, call **Tcl_LimitAddHandler**. The *handlerProc* argument describes the function that will actually be called; it should have the following prototype:

```
typedef void Tcl_LimitHandlerProc(
        ClientData clientData,
        Tcl_Interp *interp);
```

The *clientData* argument to the handler will be whatever is passed to the *clientData* argument to **Tcl_LimitAddHandler**, and the *interp* is the interpreter that had its limit exceeded.

The *deleteProc* argument to **Tcl_LimitAddHandler** is a function to call to delete the *clientData* value. It may be **TCL_STATIC** or NULL if no deletion action is necessary, or **TCL_DYNAMIC** if all that is necessary is to free the structure with **Tcl_Free**. Otherwise, it should refer to a function with the following prototype:

```
typedef void Tcl_LimitHandlerDeleteProc(
        ClientData clientData);
```

A limit handler may be deleted using **Tcl_LimitRemoveHandler**; the handler removed will be the first one found (out of the handlers added with **Tcl_LimitAddHandler**) with exactly matching *type*, *handlerProc* and *clientData* arguments. This function always invokes the *deleteProc* on the *clientData* (unless the *deleteProc* was NULL or **TCL_STATIC**).

Keywords: Limit, interpreter, resource, commands, time, callback

Copyright (c) 2004 Donal K. Fellows

LinkVar

Name

Tcl_LinkVar, Tcl_UnlinkVar, Tcl_UpdateLinkedVar - link Tcl variable to C variable

Synopsis

#include <tcl.h>

int
Tcl_LinkVar(*interp*, *varName*, *addr*, *type*)

Tcl_UnlinkVar(*interp*, *varName*)

Tcl_UpdateLinkedVar(*interp*, *varName*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter that contains *varName*. Also used by **Tcl_LinkVar** to return error messages.

const char *_varName_ (in)
Name of global variable.

char *_addr_ (in)
Address of C variable that is to be linked to *varName*.

int _type_ (in)
Type of C variable. Must be one of **TCL_LINK_INT**, **TCL_LINK_UINT**, **TCL_LINK_CHAR**, **TCL_LINK_UCHAR**, **TCL_LINK_SHORT**, **TCL_LINK_USHORT**, **TCL_LINK_LONG**, **TCL_LINK_ULONG**, **TCL_LINK_WIDE_INT**, **TCL_LINK_WIDE_UINT**, **TCL_LINK_FLOAT**, **TCL_LINK_DOUBLE**, **TCL_LINK_BOOLEAN**, or **TCL_LINK_STRING**, optionally OR'ed with **TCL_LINK_READ_ONLY** to make Tcl variable read-only.

Description

Tcl_LinkVar uses variable traces to keep the Tcl variable named by *varName* in sync with the C variable at the address given by *addr*. Whenever the Tcl variable is read the value of the C variable will be returned, and whenever the Tcl variable is written the C variable will be updated to have the same value. **Tcl_LinkVar** normally returns **TCL_OK**; if an error occurs while setting up the link (e.g. because *varName* is the name of array) then **TCL_ERROR** is returned and the interpreter's result contains an error message.

The *type* argument specifies the type of the C variable, and must have one of the following values, optionally OR'ed with **TCL_LINK_READ_ONLY**:

TCL_LINK_INT
The C variable is of type **int**. Any value written into the Tcl variable must have a proper integer form acceptable to **Tcl_GetIntFromObj**; attempts to write non-integer values into

varName will be rejected with Tcl errors. Incomplete integer representations (like the empty string, '+', '-' or the hex/octal/binary prefix) are accepted as if they are valid too.

TCL_LINK_UINT
The C variable is of type **unsigned int**. Any value written into the Tcl variable must have a proper unsigned integer form acceptable to **Tcl_GetWideIntFromObj** and in the platform's defined range for the **unsigned int** type; attempts to write non-integer values (or values outside the range) into *varName* will be rejected with Tcl errors. Incomplete integer representations (like the empty string, '+', '-' or the hex/octal/binary prefix) are accepted as if they are valid too.

TCL_LINK_CHAR
The C variable is of type **char**. Any value written into the Tcl variable must have a proper integer form acceptable to **Tcl_GetIntFromObj** and be in the range of the **char** datatype; attempts to write non-integer or out-of-range values into *varName* will be rejected with Tcl errors. Incomplete integer representations (like the empty string, '+', '-' or the hex/octal/binary prefix) are accepted as if they are valid too.

TCL_LINK_UCHAR
The C variable is of type **unsigned char**. Any value written into the Tcl variable must have a proper unsigned integer form acceptable to **Tcl_GetIntFromObj** and in the platform's defined range for the **unsigned char** type; attempts to write non-integer values (or values outside the range) into *varName* will be rejected with Tcl errors. Incomplete integer representations (like the empty string, '+', '-' or the hex/octal/binary prefix) are accepted as if they are valid too.

TCL_LINK_SHORT
The C variable is of type **short**. Any value written into the Tcl variable must have a proper integer form acceptable to **Tcl_GetIntFromObj** and be in the range of the **short** datatype; attempts to write non-integer or out-of-range values into *varName* will be rejected with Tcl errors. Incomplete integer representations (like the empty string, '+', '-' or the hex/octal/binary prefix) are accepted as if they are valid too.

TCL_LINK_USHORT
The C variable is of type **unsigned short**. Any value written into the Tcl variable must have a proper unsigned integer form acceptable to **Tcl_GetIntFromObj** and in the platform's defined range for the **unsigned short** type; attempts to write non-integer values (or values outside the range) into *varName* will be rejected with Tcl errors. Incomplete integer representations (like the empty string, '+', '-' or the hex/octal/binary prefix) are accepted as if they are valid too.

TCL_LINK_LONG
The C variable is of type **long**. Any value written into the Tcl variable must have a proper integer form acceptable to **Tcl_GetLongFromObj**; attempts to write non-integer or out-of-range values into *varName* will be rejected with Tcl errors. Incomplete integer representations (like the empty string, '+', '-' or the hex/octal/binary prefix) are accepted as if they are valid too.

TCL_LINK_ULONG
The C variable is of type **unsigned long**. Any value written into the Tcl variable must have a proper unsigned integer form acceptable to **Tcl_GetWideIntFromObj** and in the platform's defined range for the **unsigned long** type; attempts to write non-integer values (or values

outside the range) into *varName* will be rejected with Tcl errors. Incomplete integer representations (like the empty string, '+', '-' or the hex/octal/binary prefix) are accepted as if they are valid too.

TCL_LINK_DOUBLE
The C variable is of type **double**. Any value written into the Tcl variable must have a proper real form acceptable to **Tcl_GetDoubleFromObj**; attempts to write non-real values into *varName* will be rejected with Tcl errors. Incomplete integer or real representations (like the empty string, '.', '+', '-' or the hex/octal/binary prefix) are accepted as if they are valid too.

TCL_LINK_FLOAT
The C variable is of type **float**. Any value written into the Tcl variable must have a proper real form acceptable to **Tcl_GetDoubleFromObj** and must be within the range acceptable for a **float**; attempts to write non-real values (or values outside the range) into *varName* will be rejected with Tcl errors. Incomplete integer or real representations (like the empty string, '.', '+', '-' or the hex/octal/binary prefix) are accepted as if they are valid too.

TCL_LINK_WIDE_INT
The C variable is of type **Tcl_WideInt** (which is an integer type at least 64-bits wide on all platforms that can support it.) Any value written into the Tcl variable must have a proper integer form acceptable to **Tcl_GetWideIntFromObj**; attempts to write non-integer values into *varName* will be rejected with Tcl errors. Incomplete integer representations (like the empty string, '+', '-' or the hex/octal/binary prefix) are accepted as if they are valid too.

TCL_LINK_WIDE_UINT
The C variable is of type **Tcl_WideUInt** (which is an unsigned integer type at least 64-bits wide on all platforms that can support it.) Any value written into the Tcl variable must have a proper unsigned integer form acceptable to **Tcl_GetWideIntFromObj** (it will be cast to unsigned); attempts to write non-integer values into *varName* will be rejected with Tcl errors. Incomplete integer representations (like the empty string, '+', '-' or the hex/octal/binary prefix) are accepted as if they are valid too.

TCL_LINK_BOOLEAN
The C variable is of type **int**. If its value is zero then it will read from Tcl as "0"; otherwise it will read from Tcl as "1". Whenever *varName* is modified, the C variable will be set to a 0 or 1 value. Any value written into the Tcl variable must have a proper boolean form acceptable to **Tcl_GetBooleanFromObj**; attempts to write non-boolean values into *varName* will be rejected with Tcl errors.

TCL_LINK_STRING
The C variable is of type **char ***. If its value is not NULL then it must be a pointer to a string allocated with **Tcl_Alloc** or **ckalloc**. Whenever the Tcl variable is modified the current C string will be freed and new memory will be allocated to hold a copy of the variable's new value. If the C variable contains a NULL pointer then the Tcl variable will read as "NULL". If the **TCL_LINK_READ_ONLY** flag is present in *type* then the variable will be read-only from Tcl, so that its value can only be changed by modifying the C variable. Attempts to write the variable from Tcl will be rejected with errors. **Tcl_UnlinkVar** removes the link previously set up for the variable given by *varName*. If there does not exist a link for *varName* then the procedure has no effect. **Tcl_UpdateLinkedVar** may be invoked after the C variable has changed to force the Tcl variable to be updated immediately. In many cases this procedure is not needed, since any attempt to read the Tcl variable will return the latest value of the C variable. However, if a

trace has been set on the Tcl variable (such as a Tk widget that wishes to display the value of the variable), the trace will not trigger when the C variable has changed. **Tcl_UpdateLinkedVar** ensures that any traces on the Tcl variable are invoked. Note that, as with any call to a Tcl interpreter, **Tcl_UpdateLinkedVar** must be called from the same thread that created the interpreter. The safest mechanism is to ensure that the C variable is only ever updated from the same thread that created the interpreter (possibly in response to an event posted with **Tcl_ThreadQueueEvent**), but when it is necessary to update the variable in a separate thread, it is advised that **Tcl_AsyncMark** be used to indicate to the thread hosting the interpreter that it is ready to run **Tcl_UpdateLinkedVar**.

Keywords: LinkVar, boolean, integer, link, read-only, real, string, trace, variable

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

ListObj

Name

Tcl_ListObjAppendList, Tcl_ListObjAppendElement, Tcl_NewListObj, Tcl_SetListObj, Tcl_ListObjGetElements, Tcl_ListObjLength, Tcl_ListObjIndex, Tcl_ListObjReplace - manipulate Tcl values as lists

Synopsis

#include <tcl.h>

int
Tcl_ListObjAppendList(*interp*, *listPtr*, *elemListPtr*)

int
Tcl_ListObjAppendElement(*interp*, *listPtr*, *objPtr*)

Tcl_Obj *
Tcl_NewListObj(*objc*, *objv*)

Tcl_SetListObj(*objPtr*, *objc*, *objv*)

int
Tcl_ListObjGetElements(*interp*, *listPtr*, *objcPtr*, *objvPtr*)

int
Tcl_ListObjLength(*interp*, *listPtr*, *lengthPtr*)

int
Tcl_ListObjIndex(*interp*, *listPtr*, *index*, *objPtrPtr*)

int
Tcl_ListObjReplace(*interp*, *listPtr*, *first*, *count*, *objc*, *objv*)

Arguments

Tcl_Interp ****interp* **(in)
If an error occurs while converting a value to be a list value, an error message is left in the interpreter's result value unless *interp* is NULL.

Tcl_Obj ****listPtr* **(in/out)
Points to the list value to be manipulated. If *listPtr* does not already point to a list value, an

attempt will be made to convert it to one.

Tcl_Obj *_elemListPtr_ (in/out)
For **Tcl_ListObjAppendList**, this points to a list value containing elements to be appended onto *listPtr*. Each element of *elemListPtr will become a new element of *listPtr*. If *elemListPtr is not NULL and does not already point to a list value, an attempt will be made to convert it to one.

Tcl_Obj *_objPtr_ (in)
For **Tcl_ListObjAppendElement**, points to the Tcl value that will be appended to *listPtr*. For **Tcl_SetListObj**, this points to the Tcl value that will be converted to a list value containing the *objc* elements of the array referenced by *objv*.

int *_objcPtr_ (in)
Points to location where **Tcl_ListObjGetElements** stores the number of element values in *listPtr*.

Tcl_Obj ***_objvPtr_ (out)
A location where **Tcl_ListObjGetElements** stores a pointer to an array of pointers to the element values of *listPtr*.

int _objc_ (in)
The number of Tcl values that **Tcl_NewListObj** will insert into a new list value, and **Tcl_ListObjReplace** will insert into *listPtr*. For **Tcl_SetListObj**, the number of Tcl values to insert into *objPtr*.

Tcl_Obj *const _objv[]_ (in)
An array of pointers to values. **Tcl_NewListObj** will insert these values into a new list value and **Tcl_ListObjReplace** will insert them into an existing *listPtr*. Each value will become a separate list element.

int *_lengthPtr_ (out)
Points to location where **Tcl_ListObjLength** stores the length of the list.

int _index_ (in)
Index of the list element that **Tcl_ListObjIndex** is to return. The first element has index 0.

Tcl_Obj **_objPtrPtr_ (out)
Points to place where **Tcl_ListObjIndex** is to store a pointer to the resulting list element value.

int _first_ (in)
Index of the starting list element that **Tcl_ListObjReplace** is to replace. The list's first element has index 0.

int _count_ (in)
The number of elements that **Tcl_ListObjReplace** is to replace.

Description

Tcl list values have an internal representation that supports the efficient indexing and appending. The procedures described in this man page are used to create, modify, index, and append to Tcl list values from C code.

Tcl_ListObjAppendList and **Tcl_ListObjAppendElement** both add one or more values to the end of the list value referenced by *listPtr*. **Tcl_ListObjAppendList** appends each element of the list value referenced by *elemListPtr* while **Tcl_ListObjAppendElement** appends the single value

referenced by *objPtr*. Both procedures will convert the value referenced by *listPtr* to a list value if necessary. If an error occurs during conversion, both procedures return **TCL_ERROR** and leave an error message in the interpreter's result value if *interp* is not NULL. Similarly, if *elemListPtr* does not already refer to a list value, **Tcl_ListObjAppendList** will attempt to convert it to one and if an error occurs during conversion, will return **TCL_ERROR** and leave an error message in the interpreter's result value if interp is not NULL. Both procedures invalidate any old string representation of *listPtr* and, if it was converted to a list value, free any old internal representation. Similarly, **Tcl_ListObjAppendList** frees any old internal representation of *elemListPtr* if it converts it to a list value. After appending each element in *elemListPtr*, **Tcl_ListObjAppendList** increments the element's reference count since *listPtr* now also refers to it. For the same reason, **Tcl_ListObjAppendElement** increments *objPtr*'s reference count. If no error occurs, the two procedures return **TCL_OK** after appending the values.

Tcl_NewListObj and **Tcl_SetListObj** create a new value or modify an existing value to hold the *objc* elements of the array referenced by *objv* where each element is a pointer to a Tcl value. If *objc* is less than or equal to zero, they return an empty value. If *objv* is NULL, the resulting list contains 0 elements, with reserved space in an internal representation for *objc* more elements (to avoid its reallocation later). The new value's string representation is left invalid. The two procedures increment the reference counts of the elements in *objc* since the list value now refers to them. The new list value returned by **Tcl_NewListObj** has reference count zero.

Tcl_ListObjGetElements returns a count and a pointer to an array of the elements in a list value. It returns the count by storing it in the address *objcPtr*. Similarly, it returns the array pointer by storing it in the address *objvPtr*. The memory pointed to is managed by Tcl and should not be freed or written to by the caller. If the list is empty, 0 is stored at *objcPtr* and NULL at *objvPtr*. If *listPtr* is not already a list value, **Tcl_ListObjGetElements** will attempt to convert it to one; if the conversion fails, it returns **TCL_ERROR** and leaves an error message in the interpreter's result value if *interp* is not NULL. Otherwise it returns **TCL_OK** after storing the count and array pointer.

Tcl_ListObjLength returns the number of elements in the list value referenced by *listPtr*. It returns this count by storing an integer in the address *lengthPtr*. If the value is not already a list value, **Tcl_ListObjLength** will attempt to convert it to one; if the conversion fails, it returns **TCL_ERROR** and leaves an error message in the interpreter's result value if *interp* is not NULL. Otherwise it returns **TCL_OK** after storing the list's length.

The procedure **Tcl_ListObjIndex** returns a pointer to the value at element *index* in the list referenced by *listPtr*. It returns this value by storing a pointer to it in the address *objPtrPtr*. If *listPtr* does not already refer to a list value, **Tcl_ListObjIndex** will attempt to convert it to one; if the conversion fails, it returns **TCL_ERROR** and leaves an error message in the interpreter's result value if *interp* is not NULL. If the index is out of range, that is, *index* is negative or greater than or equal to the number of elements in the list, **Tcl_ListObjIndex** stores a NULL in *objPtrPtr* and returns **TCL_OK**. Otherwise it returns **TCL_OK** after storing the element's value pointer. The reference count for the list element is not incremented; the caller must do that if it needs to retain a pointer to the element.

Tcl_ListObjReplace replaces zero or more elements of the list referenced by *listPtr* with the *objc* values in the array referenced by *objv*. If *listPtr* does not point to a list value, **Tcl_ListObjReplace** will attempt to convert it to one; if the conversion fails, it returns **TCL_ERROR** and leaves an error message in the interpreter's result value if *interp* is not NULL.

Otherwise, it returns **TCL_OK** after replacing the values. If *objv* is NULL, no new elements are added. If the argument *first* is zero or negative, it refers to the first element. If *first* is greater than or equal to the number of elements in the list, then no elements are deleted; the new elements are appended to the list. *count* gives the number of elements to replace. If *count* is zero or negative then no elements are deleted; the new elements are simply inserted before the one designated by *first*. **Tcl_ListObjReplace** invalidates *listPtr*'s old string representation. The reference counts of any elements inserted from *objv* are incremented since the resulting list now refers to them. Similarly, the reference counts for any replaced values are decremented.

Because **Tcl_ListObjReplace** combines both element insertion and deletion, it can be used to implement a number of list operations. For example, the following code inserts the *objc* values referenced by the array of value pointers *objv* just before the element *index* of the list referenced by *listPtr*:

```
result = Tcl_ListObjReplace(interp, listPtr, index, 0,
        objc, objv);
```

Similarly, the following code appends the *objc* values referenced by the array *objv* to the end of the list *listPtr*:

```
result = Tcl_ListObjLength(interp, listPtr, &length);
if (result == TCL_OK) {
    result = Tcl_ListObjReplace(interp, listPtr, length, 0,
            objc, objv);
}
```

The *count* list elements starting at *first* can be deleted by simply calling **Tcl_ListObjReplace** with a NULL *objvPtr*:

```
result = Tcl_ListObjReplace(interp, listPtr, first, count,
        0, NULL);
```

Keywords: ListObj, append, index, insert, internal representation, length, list, list value, list type, value, value type, replace, string representation

Copyright (c) 1996-1997 Sun Microsystems, Inc.

Load

Name

Tcl_LoadFile, Tcl_FindSymbol - platform-independent dynamic library loading

Synopsis

#include <tcl.h>

int
Tcl_LoadFile(*interp*, *pathPtr*, *symbols*, *flags*, *procPtrs*, *loadHandlePtr*)

void *
Tcl_FindSymbol(*interp*, *loadHandle*, *symbol*)

Arguments

Tcl_Interp ****interp* **(in)

Interpreter to use for reporting error messages.

Tcl_Obj *_pathPtr_ (in)
The name of the file to load. If it is a single name, the library search path of the current environment will be used to resolve it.

const char *const _symbols[]_ (in)
Array of names of symbols to be resolved during the load of the library, or NULL if no symbols are to be resolved. If an array is given, the last entry in the array must be NULL.

int _flags_ (in)
The value should normally be 0, but *TCL_LOAD_GLOBAL* or *TCL_LOAD_LAZY* or a combination of those two is allowed as well.

void *_procPtrs_ (out)
Points to an array that will hold the addresses of the functions described in the *symbols* argument. Should be NULL if no symbols are to be resolved.

Tcl_LoadHandle *_loadHandlePtr_ (out)
Points to a variable that will hold the handle to the abstract token describing the library that has been loaded.

Tcl_LoadHandle _loadHandle_ (in)
Abstract token describing the library to look up a symbol in.

const char *_symbol_ (in)
The name of the symbol to look up.

Description

Tcl_LoadFile loads a file from the filesystem (including potentially any virtual filesystem that has been installed) and provides a handle to it that may be used in further operations. The *symbols* array, if non-NULL, supplies a set of names of symbols (typically functions) that must be resolved from the library and which will be stored in the array indicated by *procPtrs*. If any of the symbols is not resolved, the loading of the file will fail with an error message left in the interpreter (if that is non-NULL). The result of **Tcl_LoadFile** is a standard Tcl error code. The library may be unloaded with **Tcl_FSUnloadFile**.

Tcl_FindSymbol locates a symbol in a loaded library and returns it. If the symbol cannot be found, it returns NULL and sets an error message in the given *interp* (if that is non-NULL). Note that it is unsafe to use this operation on a handle that has been passed to **Tcl_FSUnloadFile**.

Keywords: Load, binary code, loading, shared library

Copyright (c) 2009-2010 Kevin B. Kenny
Copyright (c) 2010 Donal K. Fellows

Method

Name

Tcl_ClassSetConstructor, Tcl_ClassSetDestructor, Tcl_MethodDeclarerClass, Tcl_MethodDeclarerObject, Tcl_MethodIsPublic, Tcl_MethodIsType, Tcl_MethodName, Tcl_NewInstanceMethod, Tcl_NewMethod, Tcl_ObjectContextInvokeNext, Tcl_ObjectContextIsFiltering, Tcl_ObjectContextMethod, Tcl_ObjectContextObject, Tcl_ObjectContextSkippedArgs - manipulate methods and method-call contexts

Synopsis

#include <tclOO.h>

Tcl_Method
Tcl_NewMethod(*interp*, *class*, *nameObj*, *isPublic*, *methodTypePtr*, *clientData*)

Tcl_Method
Tcl_NewInstanceMethod(*interp*, *object*, *nameObj*, *isPublic*, *methodTypePtr*, *clientData*)

Tcl_ClassSetConstructor(*interp*, *class*, *method*)

Tcl_ClassSetDestructor(*interp*, *class*, *method*)

Tcl_Class
Tcl_MethodDeclarerClass(*method*)

Tcl_Object
Tcl_MethodDeclarerObject(*method*)

Tcl_Obj *
Tcl_MethodName(*method*)

int
Tcl_MethodIsPublic(*method*)

int
Tcl_MethodIsType(*method*, *methodTypePtr*, *clientDataPtr*)

int
Tcl_ObjectContextInvokeNext(*interp*, *context*, *objc*, *objv*, *skip*)

int
Tcl_ObjectContextIsFiltering(*context*)

Tcl_Method
Tcl_ObjectContextMethod(*context*)

Tcl_Object
Tcl_ObjectContextObject(*context*)

int
Tcl_ObjectContextSkippedArgs(*context*)

Arguments

Tcl_Interp *_interp_ (in/out)
The interpreter holding the object or class to create or update a method in.

Tcl_Object _object_ (in)
The object to create the method in.

Tcl_Class _class_ (in)
The class to create the method in.

Tcl_Obj *_nameObj_ (in)
The name of the method to create. Should not be NULL unless creating constructors or destructors.

int _isPublic_ (in)

A flag saying what the visibility of the method is. The only supported public values of this flag are 0 for a non-exported method, and 1 for an exported method.

Tcl_MethodType ****methodTypePtr* **(in)
A description of the type of the method to create, or the type of method to compare against.

ClientData *clientData* (in)
A piece of data that is passed to the implementation of the method without interpretation.

ClientData ****clientDataPtr* **(out)
A pointer to a variable in which to write the *clientData* value supplied when the method was created. If NULL, the *clientData* value will not be retrieved.

Tcl_Method *method* (in)
A reference to a method to query.

Tcl_ObjectContext *context* (in)
A reference to a method-call context. Note that client code *must not* retain a reference to a context.

int *objc* (in)
The number of arguments to pass to the method implementation.

Tcl_Obj *const ****objv* **(in)
An array of arguments to pass to the method implementation.

int *skip* (in)
The number of arguments passed to the method implementation that do not represent "real" arguments.

Description

A method is an operation carried out on an object that is associated with the object. Every method must be attached to either an object or a class; methods attached to a class are associated with all instances (direct and indirect) of that class.

Given a method, the entity that declared it can be found using **Tcl_MethodDeclarerClass** which returns the class that the method is attached to (or NULL if the method is not attached to any class) and **Tcl_MethodDeclarerObject** which returns the object that the method is attached to (or NULL if the method is not attached to an object). The name of the method can be retrieved with **Tcl_MethodName** and whether the method is exported is retrieved with **Tcl_MethodIsPublic**. The type of the method can also be introspected upon to a limited degree; the function **Tcl_MethodIsType** returns whether a method is of a particular type, assigning the per-method *clientData* to the variable pointed to by *clientDataPtr* if (that is non-NULL) if the type is matched.

Method creation

Methods are created by **Tcl_NewMethod** and **Tcl_NewInstanceMethod**, which create a method attached to a class or an object respectively. In both cases, the *nameObj* argument gives the name of the method to create, the *isPublic* argument states whether the method should be exported initially, the *methodTypePtr* argument describes the implementation of the method (see the **METHOD TYPES** section below) and the *clientData* argument gives some implementation-specific data that is passed on to the implementation of the method when it is called.

When the *nameObj* argument to **Tcl_NewMethod** is NULL, an unnamed method is created,

which is used for constructors and destructors. Constructors should be installed into their class using the **Tcl_ClassSetConstructor** function, and destructors (which must not require any arguments) should be installed into their class using the **Tcl_ClassSetDestructor** function. Unnamed methods should not be used for any other purpose, and named methods should not be used as either constructors or destructors. Also note that a NULL *methodTypePtr* is used to provide internal signaling, and should not be used in client code.

Method call contexts

When a method is called, a method-call context reference is passed in as one of the arguments to the implementation function. This context can be inspected to provide information about the caller, but should not be retained beyond the moment when the method call terminates.

The method that is being called can be retrieved from the context by using **Tcl_ObjectContextMethod**, and the object that caused the method to be invoked can be retrieved with **Tcl_ObjectContextObject**. The number of arguments that are to be skipped (e.g. the object name and method name in a normal method call) is read with **Tcl_ObjectContextSkippedArgs**, and the context can also report whether it is working as a filter for another method through **Tcl_ObjectContextIsFiltering**.

During the execution of a method, the method implementation may choose to invoke the stages of the method call chain that come after the current method implementation. This (the core of the **next** command) is done using **Tcl_ObjectContextInvokeNext**. Note that this function does not manipulate the call-frame stack, unlike the **next** command; if the method implementation has pushed one or more extra frames on the stack as part of its implementation, it is also responsible for temporarily popping those frames from the stack while the **Tcl_ObjectContextInvokeNext** function is executing. Note also that the method-call context is *never* deleted during the execution of this function.

Method types

The types of methods are described by a pointer to a Tcl_MethodType structure, which is defined as:

```
typedef struct {
    int version;
    const char *name;
    Tcl_MethodCallProc *callProc;
    Tcl_MethodDeleteProc *deleteProc;
    Tcl_CloneProc *cloneProc;
} Tcl_MethodType;
```

The *version* field allows for future expansion of the structure, and should always be declared equal to TCL_OO_METHOD_VERSION_CURRENT. The *name* field provides a human-readable name for the type, and is the value that is exposed via the **info class methodtype** and **info object methodtype** Tcl commands.

The *callProc* field gives a function that is called when the method is invoked; it must never be NULL.

The *deleteProc* field gives a function that is used to delete a particular method, and is called when the method is replaced or removed; if the field is NULL, it is assumed that the method's

clientData needs no special action to delete.

The *cloneProc* field is either a function that is used to copy a method's *clientData* (as part of **Tcl_CopyObjectInstance**) or NULL to indicate that the *clientData* can just be copied directly.

Tcl_methodcallproc function signature

Functions matching this signature are called when the method is invoked.

```
typedef int Tcl_MethodCallProc(
        ClientData clientData,
        Tcl_Interp *interp,
        Tcl_ObjectContext objectContext,
        int objc,
        Tcl_Obj *const *objv);
```

The *clientData* argument to a Tcl_MethodCallProc is the value that was given when the method was created, the *interp* is a place in which to execute scripts and access variables as well as being where to put the result of the method, and the *objc* and *objv* fields give the parameter objects to the method. The calling context of the method can be discovered through the *objectContext* argument, and the return value from a Tcl_MethodCallProc is any Tcl return code (e.g. TCL_OK, TCL_ERROR).

Tcl_methoddeleteproc function signature

Functions matching this signature are used when a method is deleted, whether through a new method being created or because the object or class is deleted.

```
typedef void Tcl_MethodDeleteProc(
        ClientData clientData);
```

The *clientData* argument to a Tcl_MethodDeleteProc will be the same as the value passed to the *clientData* argument to **Tcl_NewMethod** or **Tcl_NewInstanceMethod** when the method was created.

Tcl_cloneproc function signature

Functions matching this signature are used to copy a method when the object or class is copied using **Tcl_CopyObjectInstance** (or **oo::copy**).

```
typedef int Tcl_CloneProc(
        Tcl_Interp *interp,
        ClientData oldClientData,
        ClientData *newClientDataPtr);
```

The *interp* argument gives a place to write an error message when the attempt to clone the object is to fail, in which case the clone procedure must also return TCL_ERROR; it should return TCL_OK otherwise. The *oldClientData* field to a Tcl_CloneProc gives the value from the method being copied from, and the *newClientDataPtr* field will point to a variable in which to write the value for the method being copied to.

Keywords: Method, constructor, object

Copyright (c) 2007 Donal K. Fellows

NRE

Name

Tcl_NRCreateCommand, Tcl_NRCallObjProc, Tcl_NREvalObj, Tcl_NREvalObjv, Tcl_NRCmdSwap, Tcl_NRExprObj, Tcl_NRAddCallback - Non-Recursive (stackless) evaluation of Tcl scripts.

Synopsis

#include <tcl.h>

Tcl_Command
Tcl_NRCreateCommand(*interp*, *cmdName*, *proc*, *nreProc*, *clientData*, *deleteProc*)

int
Tcl_NRCallObjProc(*interp*, *nreProc*, *clientData*, *objc*, *objv*)

int
Tcl_NREvalObj(*interp*, *objPtr*, *flags*)

int
Tcl_NREvalObjv(*interp*, *objc*, *objv*, *flags*)

int
Tcl_NRCmdSwap(*interp*, *cmd*, *objc*, *objv*, *flags*)

int
Tcl_NRExprObj(*interp*, *objPtr*, *resultPtr*)

void
Tcl_NRAddCallback(*interp*, *postProcPtr*, *data0*, *data1*, *data2*, *data3*)

Arguments

Tcl_Interp *_interp_ (in)
The relevant Interpreter.

const char *_cmdName_ (in)
Name of the command to create.

Tcl_ObjCmdProc *_proc_ (in)
Called in order to evaluate a command. Is often just a small wrapper that uses **Tcl_NRCallObjProc** to call *nreProc* using a new trampoline. Behaves in the same way as the *proc* argument to **Tcl_CreateObjCommand**(3) (*q.v.*).

Tcl_ObjCmdProc *_nreProc_ (in)
Called instead of *proc* when a trampoline is already in use.

ClientData _clientData_ (in)
Arbitrary one-word value passed to *proc*, *nreProc*, *deleteProc* and *objProc*.

Tcl_CmdDeleteProc *_deleteProc_ (in/out)
Called before *cmdName* is deleted from the interpreter, allowing for command-specific cleanup. May be NULL.

int _objc_ (in)
Number of items in *objv*.

Tcl_Obj **_objv_ (in)
Words in the command.

Tcl_Obj *_objPtr_ (in)
A script or expression to evaluate.

int _flags_ (in)
As described for *Tcl_EvalObjv*.

Tcl_Command _cmd_ (in)
Token to use instead of one derived from the first word of *objv* in order to evaluate a command.

Tcl_Obj *_resultPtr_ (out)
Pointer to an unshared Tcl_Obj where the result of the evaluation is stored if the return code is TCL_OK.

Tcl_NRPostProc *_postProcPtr_ (in)
A function to push.

ClientData _data0_ (in)
...see next...

ClientData _data1_ (in)
...see next...

ClientData _data2_ (in)
...see next...

ClientData _data3_ (in)
data0 through *data3* are four one-word values that will be passed to the function designated by *postProcPtr* when it is invoked.

Description

These functions provide an interface to the function stack that an interpreter iterates through to evaluate commands. The routine behind a command is implemented by an initial function and any additional functions that the routine pushes onto the stack as it progresses. The interpreter itself pushes functions onto the stack to react to the end of a routine and to exercise other forms of control such as switching between in-progress stacks and the evaluation of other scripts at additional levels without adding frames to the C stack. To execute a routine, the initial function for the routine is called and then a small bit of code called a *trampoline* iteratively takes functions off the stack and calls them, using the value of the last call as the value of the routine.

Tcl_NRCallObjProc calls *nreProc* using a new trampoline.

Tcl_NRCreateCommand, an alternative to **Tcl_CreateObjCommand**, resolves *cmdName*, which may contain namespace qualifiers, relative to the current namespace, creates a command by that name, and returns a token for the command which may be used in subsequent calls to **Tcl_GetCommandName**. Except for a few cases noted below any existing command by the same name is first deleted. If *interp* is in the process of being deleted **Tcl_NRCreateCommand** does not create any command, does not delete any command, and returns NULL.

Tcl_NREvalObj pushes a function that is like **Tcl_EvalObjEx** but consumes no space on the C stack.

Tcl_NREvalObjv pushes a function that is like **Tcl_EvalObjv** but consumes no space on the C stack.

Tcl_NRCmdSwap is like **Tcl_NREvalObjv**, but uses *cmd*, a token previously returned by **Tcl_CreateObjCommand** or **Tcl_GetCommandFromObj**, instead of resolving the first word of

objv..

Tcl_NRExprObj pushes a function that evaluates *objPtr* as an expression in the same manner as **Tcl_ExprObj** but without consuming space on the C stack.

All of the functions return **TCL_OK** if the evaluation of the script, command, or expression has been scheduled successfully. Otherwise (for example if the command name cannot be resolved), they return **TCL_ERROR** and store a message as the interpreter's result.

Tcl_NRAddCallback pushes *postProcPtr*. The signature for **Tcl_NRPostProc** is:

```
typedef int
Tcl_NRPostProc(
        ClientData data[],
        Tcl_Interp *interp,
        int result);
```

data is a pointer to an array containing *data0* through *data3*. *result* is the value returned by the previous function implementing part the routine.

Example

The following command uses **Tcl_EvalObjEx**, which consumes space on the C stack, to evalute a script:

```
int
TheCmdOldObjProc(
    ClientData clientData,
    Tcl_Interp *interp,
    int objc,
    Tcl_Obj *const objv[])
{
    int result;
    Tcl_Obj *objPtr;

    ... preparation ...

    result = Tcl_EvalObjEx(interp, objPtr, 0);

    ... postprocessing ...

    return result;
}
Tcl_CreateObjCommand(interp, "theCommand",
        TheCmdOldObjProc, clientData, TheCmdDeleteProc);
```

To avoid consuming space on the C stack, *TheCmdOldObjProc* is renamed to *TheCmdNRObjProc* and the postprocessing step is split into a separate function, *TheCmdPostProc*, which is pushed onto the function stack. *Tcl_EvalObjEx* is replaced with *Tcl_NREvalObj*, which uses a trampoline instead of consuming space on the C stack. A new version of *TheCmdOldObjProc* is just a a wrapper that uses **Tcl_NRCallObjProc** to call *TheCmdNRObjProc*:

```
int
TheCmdOldObjProc(
    ClientData clientData,
    Tcl_Interp *interp,
    int objc,
    Tcl_Obj *const objv[])
{
    return Tcl_NRCallObjProc(interp, TheCmdNRObjProc,
            clientData, objc, objv);
}
```

```
int
TheCmdNRObjProc
    ClientData clientData,
    Tcl_Interp *interp,
    int objc,
    Tcl_Obj *const objv[])
{
    Tcl_Obj *objPtr;

    ... preparation ...

    Tcl_NRAddCallback(interp, TheCmdPostProc,
            data0, data1, data2, data3);
    /* data0 .. data3 are up to four one-word items to
     * pass to the postprocessing procedure */

    return Tcl_NREvalObj(interp, objPtr, 0);
}
```

```
int
TheCmdNRPostProc(
    ClientData data[],
    Tcl_Interp *interp,
    int result)
{
    /* data[0] .. data[3] are the four words of data
     * passed to Tcl_NRAddCallback */

    ... postprocessing ...

    return result;
}
```

Any function comprising a routine can push other functions, making it possible implement looping and sequencing constructs using the function stack.

Copyright

Copyright © 2008 Kevin B. Kenny. Copyright © 2018 Nathan Coulter.

Keywords: NRE, stackless, nonrecursive, execute, command, global, value, result, script

Copyright (c) 2008 Kevin B. Kenny.
Copyright (c) 2018 Nathan Coulter.

Namespace

Name

Tcl_AppendExportList, Tcl_CreateNamespace, Tcl_DeleteNamespace, Tcl_Export, Tcl_FindCommand, Tcl_FindNamespace, Tcl_ForgetImport, Tcl_GetCurrentNamespace, Tcl_GetGlobalNamespace, Tcl_GetNamespaceUnknownHandler, Tcl_Import, Tcl_SetNamespaceUnknownHandler - manipulate namespaces

Synopsis

#include <tcl.h>

Tcl_Namespace *
Tcl_CreateNamespace(*interp*, *name*, *clientData*, *deleteProc*)

Tcl_DeleteNamespace(*nsPtr*)

int
Tcl_AppendExportList(*interp*, *nsPtr*, *objPtr*)

int
Tcl_Export(*interp*, *nsPtr*, *pattern*, *resetListFirst*)

int
Tcl_Import(*interp*, *nsPtr*, *pattern*, *allowOverwrite*)

int
Tcl_ForgetImport(*interp*, *nsPtr*, *pattern*)

Tcl_Namespace *
Tcl_GetCurrentNamespace(*interp*)

Tcl_Namespace *
Tcl_GetGlobalNamespace(*interp*)

Tcl_Namespace *
Tcl_FindNamespace(*interp*, *name*, *contextNsPtr*, *flags*)

Tcl_Command
Tcl_FindCommand(*interp*, *name*, *contextNsPtr*, *flags*)

Tcl_Obj *
Tcl_GetNamespaceUnknownHandler(*interp*, *nsPtr*)

int
Tcl_SetNamespaceUnknownHandler(*interp*, *nsPtr*, *handlerPtr*)

Arguments

Tcl_Interp ****interp* **(in/out)
The interpreter in which the namespace exists and where name lookups are performed. Also

where error result messages are written.

const char *_name_ (in)
The name of the namespace or command to be created or accessed.

ClientData _clientData_ (in)
A context pointer by the creator of the namespace. Not interpreted by Tcl at all.

Tcl_NamespaceDeleteProc *_deleteProc_ (in)
A pointer to function to call when the namespace is deleted, or NULL if no such callback is to be performed.

Tcl_Namespace *_nsPtr_ (in)
The namespace to be manipulated, or NULL (for other than **Tcl_DeleteNamespace**) to manipulate the current namespace.

Tcl_Obj *_objPtr_ (out)
A reference to an unshared value to which the function output will be written.

const char *_pattern_ (in)
The glob-style pattern (see **Tcl_StringMatch**) that describes the commands to be imported or exported.

int _resetListFirst_ (in)
Whether the list of export patterns should be reset before adding the current pattern to it.

int _allowOverwrite_ (in)
Whether new commands created by this import action can overwrite existing commands.

Tcl_Namespace *_contextNsPtr_ (in)
The location in the namespace hierarchy where the search for a namespace or command should be conducted relative to when the search term is not rooted at the global namespace. NULL indicates the current namespace.

int _flags_ (in)
OR-ed combination of bits controlling how the search is to be performed. The following flags are supported: **TCL_GLOBAL_ONLY** (indicates that the search is always to be conducted relative to the global namespace), **TCL_NAMESPACE_ONLY** (just for **Tcl_FindCommand**; indicates that the search is always to be conducted relative to the context namespace), and **TCL_LEAVE_ERR_MSG** (indicates that an error message should be left in the interpreter if the search fails.)

Tcl_Obj *_handlerPtr_ (in)
A script fragment to be installed as the unknown command handler for the namespace, or NULL to reset the handler to its default.

Description

Namespaces are hierarchic naming contexts that can contain commands and variables. They also maintain a list of patterns that describes what commands are exported, and can import commands that have been exported by other namespaces. Namespaces can also be manipulated through the Tcl command **namespace**.

The *Tcl_Namespace* structure encapsulates a namespace, and is guaranteed to have the following fields in it: *name* (the local name of the namespace, with no namespace separator characters in it, with empty denoting the global namespace), *fullName* (the fully specified name of the namespace), *clientData*, *deleteProc* (the values specified in the call to

Tcl_CreateNamespace), and *parentPtr* (a pointer to the containing namespace, or NULL for the global namespace.)

Tcl_CreateNamespace creates a new namespace. The *deleteProc* will have the following type signature:

```
typedef void Tcl_NamespaceDeleteProc(
        ClientData clientData);
```

Tcl_DeleteNamespace deletes a namespace, calling the *deleteProc* defined for the namespace (if any).

Tcl_AppendExportList retrieves the export patterns for a namespace given namespace and appends them (as list items) to *objPtr*.

Tcl_Export sets and appends to the export patterns for a namespace. Patterns are appended unless the *resetListFirst* flag is true.

Tcl_Import imports commands matching a pattern into a namespace. Note that the pattern must include the name of the namespace to import from. This function returns TCL_ERROR if an attempt to import a command over an existing command is made, unless the *allowOverwrite* flag has been set.

Tcl_ForgetImport removes imports matching a pattern.

Tcl_GetCurrentNamespace returns the current namespace for an interpreter.

Tcl_GetGlobalNamespace returns the global namespace for an interpreter.

Tcl_FindNamespace searches for a namespace named *name* within the context of the namespace *contextNsPtr*. If the namespace cannot be found, NULL is returned.

Tcl_FindCommand searches for a command named *name* within the context of the namespace *contextNsPtr*. If the command cannot be found, NULL is returned.

Tcl_GetNamespaceUnknownHandler returns the unknown command handler for the namespace, or NULL if none is set.

Tcl_SetNamespaceUnknownHandler sets the unknown command handler for the namespace. If *handlerPtr* is NULL, then the handler is reset to its default.

Keywords: Namespace, command

Copyright (c) 2003 Donal K. Fellows

Notifier

Name

Tcl_CreateEventSource, Tcl_DeleteEventSource, Tcl_SetMaxBlockTime, Tcl_QueueEvent, Tcl_ThreadQueueEvent, Tcl_ThreadAlert, Tcl_GetCurrentThread, Tcl_DeleteEvents, Tcl_InitNotifier, Tcl_FinalizeNotifier, Tcl_WaitForEvent, Tcl_AlertNotifier, Tcl_SetTimer, Tcl_ServiceAll, Tcl_ServiceEvent, Tcl_GetServiceMode, Tcl_SetServiceMode, Tcl_ServiceModeHook, Tcl_SetNotifier - the event queue and notifier interfaces

Synopsis

#include <tcl.h>

void

Tcl_CreateEventSource(*setupProc*, *checkProc*, *clientData*)

void
Tcl_DeleteEventSource(*setupProc*, *checkProc*, *clientData*)

void
Tcl_SetMaxBlockTime(*timePtr*)

void
Tcl_QueueEvent(*evPtr*, *position*)

void
Tcl_ThreadQueueEvent(*threadId*, *evPtr*, *position*)

void
Tcl_ThreadAlert(*threadId*)

Tcl_ThreadId
Tcl_GetCurrentThread()

void
Tcl_DeleteEvents(*deleteProc*, *clientData*)

ClientData
Tcl_InitNotifier()

void
Tcl_FinalizeNotifier(*clientData*)

int
Tcl_WaitForEvent(*timePtr*)

void
Tcl_AlertNotifier(*clientData*)

void
Tcl_SetTimer(*timePtr*)

int
Tcl_ServiceAll()

int
Tcl_ServiceEvent(*flags*)

int
Tcl_GetServiceMode()

int
Tcl_SetServiceMode(*mode*)

void
Tcl_ServiceModeHook(*mode*)

void
Tcl_SetNotifier(*notifierProcPtr*)

Arguments

Tcl_EventSetupProc ****setupProc* **(in)
Procedure to invoke to prepare for event wait in **Tcl_DoOneEvent**.

Tcl_EventCheckProc *_checkProc_ (in)
Procedure for **Tcl_DoOneEvent** to invoke after waiting for events. Checks to see if any events have occurred and, if so, queues them.

ClientData _clientData_ (in)
Arbitrary one-word value to pass to *setupProc*, *checkProc*, or *deleteProc*.

const Tcl_Time *_timePtr_ (in)
Indicates the maximum amount of time to wait for an event. This is specified as an interval (how long to wait), not an absolute time (when to wakeup). If the pointer passed to **Tcl_WaitForEvent** is NULL, it means there is no maximum wait time: wait forever if necessary.

Tcl_Event *_evPtr_ (in)
An event to add to the event queue. The storage for the event must have been allocated by the caller using **Tcl_Alloc** or **ckalloc**.

Tcl_QueuePosition _position_ (in)
Where to add the new event in the queue: **TCL_QUEUE_TAIL**, **TCL_QUEUE_HEAD**, or **TCL_QUEUE_MARK**.

Tcl_ThreadId _threadId_ (in)
A unique identifier for a thread.

Tcl_EventDeleteProc *_deleteProc_ (in)
Procedure to invoke for each queued event in **Tcl_DeleteEvents**.

int _flags_ (in)
What types of events to service. These flags are the same as those passed to **Tcl_DoOneEvent**.

int _mode_ (in)
Indicates whether events should be serviced by **Tcl_ServiceAll**. Must be one of **TCL_SERVICE_NONE** or **TCL_SERVICE_ALL**.

Tcl_NotifierProcs* _notifierProcPtr_ (in)
Structure of function pointers describing notifier procedures that are to replace the ones installed in the executable. See **REPLACING THE NOTIFIER** for details.

Introduction

The interfaces described here are used to customize the Tcl event loop. The two most common customizations are to add new sources of events and to merge Tcl's event loop with some other event loop, such as one provided by an application in which Tcl is embedded. Each of these tasks is described in a separate section below.

The procedures in this manual entry are the building blocks out of which the Tcl event notifier is constructed. The event notifier is the lowest layer in the Tcl event mechanism. It consists of three things:

[1]
Event sources: these represent the ways in which events can be generated. For example, there is a timer event source that implements the **Tcl_CreateTimerHandler** procedure and the **after** command, and there is a file event source that implements the **Tcl_CreateFileHandler** procedure on Unix systems. An event source must work with the notifier to detect events at the right times, record them on the event queue, and eventually

notify higher-level software that they have occurred. The procedures **Tcl_CreateEventSource**, **Tcl_DeleteEventSource**, and **Tcl_SetMaxBlockTime**, **Tcl_QueueEvent**, and **Tcl_DeleteEvents** are used primarily by event sources.

[2]

The event queue: for non-threaded applications, there is a single queue for the whole application, containing events that have been detected but not yet serviced. Event sources place events onto the queue so that they may be processed in order at appropriate times during the event loop. The event queue guarantees a fair discipline of event handling, so that no event source can starve the others. It also allows events to be saved for servicing at a future time. Threaded applications work in a similar manner, except that there is a separate event queue for each thread containing a Tcl interpreter. **Tcl_QueueEvent** is used (primarily by event sources) to add events to the event queue and **Tcl_DeleteEvents** is used to remove events from the queue without processing them. In a threaded application, **Tcl_QueueEvent** adds an event to the current thread's queue, and **Tcl_ThreadQueueEvent** adds an event to a queue in a specific thread.

[3]

The event loop: in order to detect and process events, the application enters a loop that waits for events to occur, places them on the event queue, and then processes them. Most applications will do this by calling the procedure **Tcl_DoOneEvent**, which is described in a separate manual entry. Most Tcl applications need not worry about any of the internals of the Tcl notifier. However, the notifier now has enough flexibility to be retargeted either for a new platform or to use an external event loop (such as the Motif event loop, when Tcl is embedded in a Motif application). The procedures **Tcl_WaitForEvent** and **Tcl_SetTimer** are normally implemented by Tcl, but may be replaced with new versions to retarget the notifier (the **Tcl_InitNotifier**, **Tcl_AlertNotifier**, **Tcl_FinalizeNotifier**, **Tcl_Sleep**, **Tcl_CreateFileHandler**, and **Tcl_DeleteFileHandler** must also be replaced; see CREATING A NEW NOTIFIER below for details). The procedures **Tcl_ServiceAll**, **Tcl_ServiceEvent**, **Tcl_GetServiceMode**, and **Tcl_SetServiceMode** are provided to help connect Tcl's event loop to an external event loop such as Motif's.

Notifier basics

The easiest way to understand how the notifier works is to consider what happens when **Tcl_DoOneEvent** is called. **Tcl_DoOneEvent** is passed a *flags* argument that indicates what sort of events it is OK to process and also whether or not to block if no events are ready. **Tcl_DoOneEvent** does the following things:

[1]

Check the event queue to see if it contains any events that can be serviced. If so, service the first possible event, remove it from the queue, and return. It does this by calling **Tcl_ServiceEvent** and passing in the *flags* argument.

[2]

Prepare to block for an event. To do this, **Tcl_DoOneEvent** invokes a *setup procedure* in each event source. The event source will perform event-source specific initialization and possibly call **Tcl_SetMaxBlockTime** to limit how long **Tcl_WaitForEvent** will block if no new events occur.

[3]

Call **Tcl_WaitForEvent**. This procedure is implemented differently on different platforms; it waits for an event to occur, based on the information provided by the event sources. It may cause the application to block if *timePtr* specifies an interval other than 0. **Tcl_WaitForEvent** returns when something has happened, such as a file becoming readable or the interval given by *timePtr* expiring. If there are no events for **Tcl_WaitForEvent** to wait for, so that it would block forever, then it returns immediately and **Tcl_DoOneEvent** returns 0.

[4]
Call a *check procedure* in each event source. The check procedure determines whether any events of interest to this source occurred. If so, the events are added to the event queue.

[5]
Check the event queue to see if it contains any events that can be serviced. If so, service the first possible event, remove it from the queue, and return.

[6]
See if there are idle callbacks pending. If so, invoke all of them and return.

[7]
Either return 0 to indicate that no events were ready, or go back to step [2] if blocking was requested by the caller.

Creating a new event source

An event source consists of three procedures invoked by the notifier, plus additional C procedures that are invoked by higher-level code to arrange for event-driven callbacks. The three procedures called by the notifier consist of the setup and check procedures described above, plus an additional procedure that is invoked when an event is removed from the event queue for servicing.

The procedure **Tcl_CreateEventSource** creates a new event source. Its arguments specify the setup procedure and check procedure for the event source. *SetupProc* should match the following prototype:

```
typedef void Tcl_EventSetupProc(
        ClientData clientData,
        int flags);
```

The *clientData* argument will be the same as the *clientData* argument to **Tcl_CreateEventSource**; it is typically used to point to private information managed by the event source. The *flags* argument will be the same as the *flags* argument passed to **Tcl_DoOneEvent** except that it will never be 0 (**Tcl_DoOneEvent** replaces 0 with **TCL_ALL_EVENTS**). *Flags* indicates what kinds of events should be considered; if the bit corresponding to this event source is not set, the event source should return immediately without doing anything. For example, the file event source checks for the **TCL_FILE_EVENTS** bit.

SetupProc's job is to make sure that the application wakes up when events of the desired type occur. This is typically done in a platform-dependent fashion. For example, under Unix an event source might call **Tcl_CreateFileHandler**; under Windows it might request notification with a Windows event. For timer-driven event sources such as timer events or any polled event, the event source can call **Tcl_SetMaxBlockTime** to force the application to wake up after a specified time even if no events have occurred. If no event source calls **Tcl_SetMaxBlockTime**

then **Tcl_WaitForEvent** will wait as long as necessary for an event to occur; otherwise, it will only wait as long as the shortest interval passed to **Tcl_SetMaxBlockTime** by one of the event sources. If an event source knows that it already has events ready to report, it can request a zero maximum block time. For example, the setup procedure for the X event source looks to see if there are events already queued. If there are, it calls **Tcl_SetMaxBlockTime** with a 0 block time so that **Tcl_WaitForEvent** does not block if there is no new data on the X connection. The *timePtr* argument to **Tcl_WaitForEvent** points to a structure that describes a time interval in seconds and microseconds:

```
typedef struct Tcl_Time {
    long sec;
    long usec;
} Tcl_Time;
```

The *usec* field should be less than 1000000.

Information provided to **Tcl_SetMaxBlockTime** is only used for the next call to **Tcl_WaitForEvent**; it is discarded after **Tcl_WaitForEvent** returns. The next time an event wait is done each of the event sources' setup procedures will be called again, and they can specify new information for that event wait.

If the application uses an external event loop rather than **Tcl_DoOneEvent**, the event sources may need to call **Tcl_SetMaxBlockTime** at other times. For example, if a new event handler is registered that needs to poll for events, the event source may call **Tcl_SetMaxBlockTime** to set the block time to zero to force the external event loop to call Tcl. In this case, **Tcl_SetMaxBlockTime** invokes **Tcl_SetTimer** with the shortest interval seen since the last call to **Tcl_DoOneEvent** or **Tcl_ServiceAll**.

In addition to the generic procedure **Tcl_SetMaxBlockTime**, other platform-specific procedures may also be available for *setupProc*, if there is additional information needed by **Tcl_WaitForEvent** on that platform. For example, on Unix systems the **Tcl_CreateFileHandler** interface can be used to wait for file events.

The second procedure provided by each event source is its check procedure, indicated by the *checkProc* argument to **Tcl_CreateEventSource**. *CheckProc* must match the following prototype:

```
typedef void Tcl_EventCheckProc(
        ClientData clientData,
        int flags);
```

The arguments to this procedure are the same as those for *setupProc*. **CheckProc** is invoked by **Tcl_DoOneEvent** after it has waited for events. Presumably at least one event source is now prepared to queue an event. **Tcl_DoOneEvent** calls each of the event sources in turn, so they all have a chance to queue any events that are ready. The check procedure does two things. First, it must see if any events have triggered. Different event sources do this in different ways.

If an event source's check procedure detects an interesting event, it must add the event to Tcl's event queue. To do this, the event source calls **Tcl_QueueEvent**. The *evPtr* argument is a pointer to a dynamically allocated structure containing the event (see below for more information on memory management issues). Each event source can define its own event structure with whatever information is relevant to that event source. However, the first element

of the structure must be a structure of type **Tcl_Event**, and the address of this structure is used when communicating between the event source and the rest of the notifier. A **Tcl_Event** has the following definition:

```
typedef struct {
    Tcl_EventProc *proc;
    struct Tcl_Event *nextPtr;
} Tcl_Event;
```

The event source must fill in the *proc* field of the event before calling **Tcl_QueueEvent**. The *nextPtr* is used to link together the events in the queue and should not be modified by the event source.

An event may be added to the queue at any of three positions, depending on the *position* argument to **Tcl_QueueEvent**:

TCL_QUEUE_TAIL
Add the event at the back of the queue, so that all other pending events will be serviced first. This is almost always the right place for new events.

TCL_QUEUE_HEAD
Add the event at the front of the queue, so that it will be serviced before all other queued events.

TCL_QUEUE_MARK
Add the event at the front of the queue, unless there are other events at the front whose position is **TCL_QUEUE_MARK**; if so, add the new event just after all other **TCL_QUEUE_MARK** events. This value of *position* is used to insert an ordered sequence of events at the front of the queue, such as a series of Enter and Leave events synthesized during a grab or ungrab operation in Tk. When it is time to handle an event from the queue (steps 1 and 4 above) **Tcl_ServiceEvent** will invoke the *proc* specified in the first queued **Tcl_Event** structure. *Proc* must match the following prototype:

```
typedef int Tcl_EventProc(
        Tcl_Event *evPtr,
        int flags);
```

The first argument to *proc* is a pointer to the event, which will be the same as the first argument to the **Tcl_QueueEvent** call that added the event to the queue. The second argument to *proc* is the *flags* argument for the current call to **Tcl_ServiceEvent**; this is used by the event source to return immediately if its events are not relevant.

It is up to *proc* to handle the event, typically by invoking one or more Tcl commands or C-level callbacks. Once the event source has finished handling the event it returns 1 to indicate that the event can be removed from the queue. If for some reason the event source decides that the event cannot be handled at this time, it may return 0 to indicate that the event should be deferred for processing later; in this case **Tcl_ServiceEvent** will go on to the next event in the queue and attempt to service it. There are several reasons why an event source might defer an event. One possibility is that events of this type are excluded by the *flags* argument. For example, the file event source will always return 0 if the **TCL_FILE_EVENTS** bit is not set in *flags*. Another example of deferring events happens in Tk if **Tk_RestrictEvents** has been invoked to defer certain kinds of window events.

When *proc* returns 1, **Tcl_ServiceEvent** will remove the event from the event queue and free its storage. Note that the storage for an event must be allocated by the event source (using **Tcl_Alloc** or the Tcl macro **ckalloc**) before calling **Tcl_QueueEvent**, but it will be freed by **Tcl_ServiceEvent**, not by the event source.

Threaded applications work in a similar manner, except that there is a separate event queue for each thread containing a Tcl interpreter. Calling **Tcl_QueueEvent** in a multithreaded application adds an event to the current thread's queue. To add an event to another thread's queue, use **Tcl_ThreadQueueEvent**. **Tcl_ThreadQueueEvent** accepts as an argument a Tcl_ThreadId argument, which uniquely identifies a thread in a Tcl application. To obtain the Tcl_ThreadId for the current thread, use the **Tcl_GetCurrentThread** procedure. (A thread would then need to pass this identifier to other threads for those threads to be able to add events to its queue.) After adding an event to another thread's queue, you then typically need to call **Tcl_ThreadAlert** to "wake up" that thread's notifier to alert it to the new event.

Tcl_DeleteEvents can be used to explicitly remove one or more events from the event queue. **Tcl_DeleteEvents** calls *proc* for each event in the queue, deleting those for with the procedure returns 1. Events for which the procedure returns 0 are left in the queue. *Proc* should match the following prototype:

```
typedef int Tcl_EventDeleteProc(
        Tcl_Event *evPtr,
        ClientData clientData);
```

The *clientData* argument will be the same as the *clientData* argument to **Tcl_DeleteEvents**; it is typically used to point to private information managed by the event source. The *evPtr* will point to the next event in the queue.

Tcl_DeleteEventSource deletes an event source. The *setupProc*, *checkProc*, and *clientData* arguments must exactly match those provided to the **Tcl_CreateEventSource** for the event source to be deleted. If no such source exists, **Tcl_DeleteEventSource** has no effect.

Creating a new notifier

The notifier consists of all the procedures described in this manual entry, plus **Tcl_DoOneEvent** and **Tcl_Sleep**, which are available on all platforms, and **Tcl_CreateFileHandler** and **Tcl_DeleteFileHandler**, which are Unix-specific. Most of these procedures are generic, in that they are the same for all notifiers. However, none of the procedures are notifier-dependent: **Tcl_InitNotifier**, **Tcl_AlertNotifier**, **Tcl_FinalizeNotifier**, **Tcl_SetTimer**, **Tcl_Sleep**, **Tcl_WaitForEvent**, **Tcl_CreateFileHandler**, **Tcl_DeleteFileHandler** and **Tcl_ServiceModeHook**. To support a new platform or to integrate Tcl with an application-specific event loop, you must write new versions of these procedures.

Tcl_InitNotifier initializes the notifier state and returns a handle to the notifier state. Tcl calls this procedure when initializing a Tcl interpreter. Similarly, **Tcl_FinalizeNotifier** shuts down the notifier, and is called by **Tcl_Finalize** when shutting down a Tcl interpreter.

Tcl_WaitForEvent is the lowest-level procedure in the notifier; it is responsible for waiting for an "interesting" event to occur or for a given time to elapse. Before **Tcl_WaitForEvent** is invoked, each of the event sources' setup procedure will have been invoked. The *timePtr* argument to **Tcl_WaitForEvent** gives the maximum time to block for an event, based on calls to **Tcl_SetMaxBlockTime** made by setup procedures and on other information (such as the

TCL_DONT_WAIT bit in *flags*).

Ideally, **Tcl_WaitForEvent** should only wait for an event to occur; it should not actually process the event in any way. Later on, the event sources will process the raw events and create Tcl_Events on the event queue in their *checkProc* procedures. However, on some platforms (such as Windows) this is not possible; events may be processed in **Tcl_WaitForEvent**, including queuing Tcl_Events and more (for example, callbacks for native widgets may be invoked). The return value from **Tcl_WaitForEvent** must be either 0, 1, or -1. On platforms such as Windows where events get processed in **Tcl_WaitForEvent**, a return value of 1 means that there may be more events still pending that have not been processed. This is a sign to the caller that it must call **Tcl_WaitForEvent** again if it wants all pending events to be processed. A 0 return value means that calling **Tcl_WaitForEvent** again will not have any effect: either this is a platform where **Tcl_WaitForEvent** only waits without doing any event processing, or **Tcl_WaitForEvent** knows for sure that there are no additional events to process (e.g. it returned because the time elapsed). Finally, a return value of -1 means that the event loop is no longer operational and the application should probably unwind and terminate. Under Windows this happens when a WM_QUIT message is received; under Unix it happens when **Tcl_WaitForEvent** would have waited forever because there were no active event sources and the timeout was infinite.

Tcl_AlertNotifier is used in multithreaded applications to allow any thread to "wake up" the notifier to alert it to new events on its queue. **Tcl_AlertNotifier** requires as an argument the notifier handle returned by **Tcl_InitNotifier**.

If the notifier will be used with an external event loop, then it must also support the **Tcl_SetTimer** interface. **Tcl_SetTimer** is invoked by **Tcl_SetMaxBlockTime** whenever the maximum blocking time has been reduced. **Tcl_SetTimer** should arrange for the external event loop to invoke **Tcl_ServiceAll** after the specified interval even if no events have occurred. This interface is needed because **Tcl_WaitForEvent** is not invoked when there is an external event loop. If the notifier will only be used from **Tcl_DoOneEvent**, then **Tcl_SetTimer** need not do anything.

Tcl_ServiceModeHook is called by the platform-independent portion of the notifier when client code makes a call to **Tcl_SetServiceMode**. This hook is provided to support operating systems that require special event handling when the application is in a modal loop (the Windows notifier, for instance, uses this hook to create a communication window).

On Unix systems, the file event source also needs support from the notifier. The file event source consists of the **Tcl_CreateFileHandler** and **Tcl_DeleteFileHandler** procedures, which are described in the **Tcl_CreateFileHandler** manual page.

The **Tcl_Sleep** and **Tcl_DoOneEvent** interfaces are described in their respective manual pages.

The easiest way to create a new notifier is to look at the code for an existing notifier, such as the files **unix/tclUnixNotfy.c** or **win/tclWinNotify.c** in the Tcl source distribution.

Replacing the notifier

A notifier that has been written according to the conventions above can also be installed in a running process in place of the standard notifier. This mechanism is used so that a single executable can be used (with the standard notifier) as a stand-alone program and reused (with a replacement notifier in a loadable extension) as an extension to another program, such as a Web browser plugin.

To do this, the extension makes a call to **Tcl_SetNotifier** passing a pointer to a **Tcl_NotifierProcs** data structure. The structure has the following layout:

```
typedef struct Tcl_NotifierProcs {
    Tcl_SetTimerProc *setTimerProc;
    Tcl_WaitForEventProc *waitForEventProc;
    Tcl_CreateFileHandlerProc *createFileHandlerProc;
    Tcl_DeleteFileHandlerProc *deleteFileHandlerProc;
    Tcl_InitNotifierProc *initNotifierProc;
    Tcl_FinalizeNotifierProc *finalizeNotifierProc;
    Tcl_AlertNotifierProc *alertNotifierProc;
    Tcl_ServiceModeHookProc *serviceModeHookProc;
} Tcl_NotifierProcs;
```

Following the call to **Tcl_SetNotifier**, the pointers given in the **Tcl_NotifierProcs** structure replace whatever notifier had been installed in the process.

It is extraordinarily unwise to replace a running notifier. Normally, **Tcl_SetNotifier** should be called at process initialization time before the first call to **Tcl_InitNotifier**.

External event loops

The notifier interfaces are designed so that Tcl can be embedded into applications that have their own private event loops. In this case, the application does not call **Tcl_DoOneEvent** except in the case of recursive event loops such as calls to the Tcl commands **update** or **vwait**. Most of the time is spent in the external event loop of the application. In this case the notifier must arrange for the external event loop to call back into Tcl when something happens on the various Tcl event sources. These callbacks should arrange for appropriate Tcl events to be placed on the Tcl event queue.

Because the external event loop is not calling **Tcl_DoOneEvent** on a regular basis, it is up to the notifier to arrange for **Tcl_ServiceEvent** to be called whenever events are pending on the Tcl event queue. The easiest way to do this is to invoke **Tcl_ServiceAll** at the end of each callback from the external event loop. This will ensure that all of the event sources are polled, any queued events are serviced, and any pending idle handlers are processed before returning control to the application. In addition, event sources that need to poll for events can call **Tcl_SetMaxBlockTime** to force the external event loop to call Tcl even if no events are available on the system event queue.

As a side effect of processing events detected in the main external event loop, Tcl may invoke **Tcl_DoOneEvent** to start a recursive event loop in commands like **vwait**. **Tcl_DoOneEvent** will invoke the external event loop, which will result in callbacks as described in the preceding paragraph, which will result in calls to **Tcl_ServiceAll**. However, in these cases it is undesirable to service events in **Tcl_ServiceAll**. Servicing events there is unnecessary because control will immediately return to the external event loop and hence to **Tcl_DoOneEvent**, which can service the events itself. Furthermore, **Tcl_DoOneEvent** is supposed to service only a single event, whereas **Tcl_ServiceAll** normally services all pending events. To handle this situation, **Tcl_DoOneEvent** sets a flag for **Tcl_ServiceAll** that causes it to return without servicing any events. This flag is called the *service mode*; **Tcl_DoOneEvent** restores it to its previous value before it returns.

In some cases, however, it may be necessary for **Tcl_ServiceAll** to service events even when it has been invoked from **Tcl_DoOneEvent**. This happens when there is yet another recursive event loop invoked via an event handler called by **Tcl_DoOneEvent** (such as one that is part of a native widget). In this case, **Tcl_DoOneEvent** may not have a chance to service events so **Tcl_ServiceAll** must service them all. Any recursive event loop that calls an external event loop rather than **Tcl_DoOneEvent** must reset the service mode so that all events get processed in **Tcl_ServiceAll**. This is done by invoking the **Tcl_SetServiceMode** procedure. If **Tcl_SetServiceMode** is passed **TCL_SERVICE_NONE**, then calls to **Tcl_ServiceAll** will return immediately without processing any events. If **Tcl_SetServiceMode** is passed **TCL_SERVICE_ALL**, then calls to **Tcl_ServiceAll** will behave normally. **Tcl_SetServiceMode** returns the previous value of the service mode, which should be restored when the recursive loop exits. **Tcl_GetServiceMode** returns the current value of the service mode.

Keywords: Notifier, event, event queue, event sources, file events, timer, idle, service mode, threads

Copyright (c) 1998-1999 Scriptics Corporation
Copyright (c) 1995-1997 Sun Microsystems, Inc.

OOInitStubs

Name

Tcl_OOInitStubs - initialize library access to TclOO functionality

Synopsis

#include <tclOO.h>

const char *
Tcl_OOInitStubs(*interp*)

Arguments

Tcl_Interp *_interp_ (in)

The Tcl interpreter that the TclOO API is integrated with and whose C interface is going to be used.

Description

When an extension library is going to use the C interface exposed by TclOO, it should use **Tcl_OOInitStubs** to initialize its access to that interface from within its *_**Init** (or *_**SafeInit**) function, passing in the *interp* that was passed into that routine as context. If the result of calling **Tcl_OOInitStubs** is NULL, the initialization failed and an error message will have been left in the interpreter's result. Otherwise, the initialization succeeded and the TclOO API may thereafter be used; the version of the TclOO API is returned.

When using this function, either the C #define symbol **USE_TCLOO_STUBS** should be defined and your library code linked against the Tcl stub library, or that #define symbol should *not* be defined and your library code linked against the Tcl main library directly.

Backward compatibility note

If you are linking against the Tcl 8.5 forward compatibility package for TclOO, *only* the stub-enabled configuration is supported and you should also link against the TclOO independent

stub library; that library is an integrated part of the main Tcl stub library in Tcl 8.6.

Keywords: OOInitStubs, stubs

Copyright (c) 2012 Donal K. Fellows

Object

Name

Tcl_NewObj, Tcl_DuplicateObj, Tcl_IncrRefCount, Tcl_DecrRefCount, Tcl_IsShared, Tcl_InvalidateStringRep - manipulate Tcl values

Synopsis

#include <tcl.h>

Tcl_Obj *
Tcl_NewObj()

Tcl_Obj *
Tcl_DuplicateObj(*objPtr*)

Tcl_IncrRefCount(*objPtr*)

Tcl_DecrRefCount(*objPtr*)

int
Tcl_IsShared(*objPtr*)

Tcl_InvalidateStringRep(*objPtr*)

Arguments

Tcl_Obj ****objPtr* **(in)
Points to a value; must have been the result of a previous call to **Tcl_NewObj**.

Introduction

This man page presents an overview of Tcl values (called **Tcl_Obj**s for historical reasons) and how they are used. It also describes generic procedures for managing Tcl values. These procedures are used to create and copy values, and increment and decrement the count of references (pointers) to values. The procedures are used in conjunction with ones that operate on specific types of values such as **Tcl_GetIntFromObj** and **Tcl_ListObjAppendElement**. The individual procedures are described along with the data structures they manipulate.

Tcl's *dual-ported* values provide a general-purpose mechanism for storing and exchanging Tcl values. They largely replace the use of strings in Tcl. For example, they are used to store variable values, command arguments, command results, and scripts. Tcl values behave like strings but also hold an internal representation that can be manipulated more efficiently. For example, a Tcl list is now represented as a value that holds the list's string representation as well as an array of pointers to the values for each list element. Dual-ported values avoid most runtime type conversions. They also improve the speed of many operations since an appropriate representation is immediately available. The compiler itself uses Tcl values to cache the instruction bytecodes resulting from compiling scripts.

The two representations are a cache of each other and are computed lazily. That is, each representation is only computed when necessary, it is computed from the other representation,

and, once computed, it is saved. In addition, a change in one representation invalidates the other one. As an example, a Tcl program doing integer calculations can operate directly on a variable's internal machine integer representation without having to constantly convert between integers and strings. Only when it needs a string representing the variable's value, say to print it, will the program regenerate the string representation from the integer. Although values contain an internal representation, their semantics are defined in terms of strings: an up-to-date string can always be obtained, and any change to the value will be reflected in that string when the value's string representation is fetched. Because of this representation invalidation and regeneration, it is dangerous for extension writers to access **Tcl_Obj** fields directly. It is better to access Tcl_Obj information using procedures like **Tcl_GetStringFromObj** and **Tcl_GetString**.

Values are allocated on the heap and are referenced using a pointer to their **Tcl_Obj** structure. Values are shared as much as possible. This significantly reduces storage requirements because some values such as long lists are very large. Also, most Tcl values are only read and never modified. This is especially true for procedure arguments, which can be shared between the caller and the called procedure. Assignment and argument binding is done by simply assigning a pointer to the value. Reference counting is used to determine when it is safe to reclaim a value's storage.

Tcl values are typed. A value's internal representation is controlled by its type. Several types are predefined in the Tcl core including integer, double, list, and bytecode. Extension writers can extend the set of types by defining their own **Tcl_ObjType** structs.

The tcl_obj structure

Each Tcl value is represented by a **Tcl_Obj** structure which is defined as follows.

```
typedef struct Tcl_Obj {
    int refCount;
    char *bytes;
    int length;
    const Tcl_ObjType *typePtr;
    union {
        long longValue;
        double doubleValue;
        void *otherValuePtr;
        Tcl_WideInt wideValue;
        struct {
            void *ptr1;
            void *ptr2;
        } twoPtrValue;
        struct {
            void *ptr;
            unsigned long value;
        } ptrAndLongRep;
    } internalRep;
} Tcl_Obj;
```

The *bytes* and the *length* members together hold a value's UTF-8 string representation, which is a *counted string* not containing null bytes (UTF-8 null characters should be encoded as a two

byte sequence: 192, 128.) *bytes* points to the first byte of the string representation. The *length* member gives the number of bytes. The byte array must always have a null byte after the last data byte, at offset *length*; this allows string representations to be treated as conventional null-terminated C strings. C programs use **Tcl_GetStringFromObj** and **Tcl_GetString** to get a value's string representation. If *bytes* is NULL, the string representation is invalid.

A value's type manages its internal representation. The member *typePtr* points to the Tcl_ObjType structure that describes the type. If *typePtr* is NULL, the internal representation is invalid.

The *internalRep* union member holds a value's internal representation. This is either a (long) integer, a double-precision floating-point number, a pointer to a value containing additional information needed by the value's type to represent the value, a Tcl_WideInt integer, two arbitrary pointers, or a pair made up of an unsigned long integer and a pointer.

The *refCount* member is used to tell when it is safe to free a value's storage. It holds the count of active references to the value. Maintaining the correct reference count is a key responsibility of extension writers. Reference counting is discussed below in the section **STORAGE MANAGEMENT OF VALUES**.

Although extension writers can directly access the members of a Tcl_Obj structure, it is much better to use the appropriate procedures and macros. For example, extension writers should never read or update *refCount* directly; they should use macros such as **Tcl_IncrRefCount** and **Tcl_IsShared** instead.

A key property of Tcl values is that they hold two representations. A value typically starts out containing only a string representation: it is untyped and has a NULL *typePtr*. A value containing an empty string or a copy of a specified string is created using **Tcl_NewObj** or **Tcl_NewStringObj** respectively. A value's string value is gotten with **Tcl_GetStringFromObj** or **Tcl_GetString** and changed with **Tcl_SetStringObj**. If the value is later passed to a procedure like **Tcl_GetIntFromObj** that requires a specific internal representation, the procedure will create one and set the value's *typePtr*. The internal representation is computed from the string representation. A value's two representations are duals of each other: changes made to one are reflected in the other. For example, **Tcl_ListObjReplace** will modify a value's internal representation and the next call to **Tcl_GetStringFromObj** or **Tcl_GetString** will reflect that change.

Representations are recomputed lazily for efficiency. A change to one representation made by a procedure such as **Tcl_ListObjReplace** is not reflected immediately in the other representation. Instead, the other representation is marked invalid so that it is only regenerated if it is needed later. Most C programmers never have to be concerned with how this is done and simply use procedures such as **Tcl_GetBooleanFromObj** or **Tcl_ListObjIndex**. Programmers that implement their own value types must check for invalid representations and mark representations invalid when necessary. The procedure **Tcl_InvalidateStringRep** is used to mark a value's string representation invalid and to free any storage associated with the old string representation.

Values usually remain one type over their life, but occasionally a value must be converted from one type to another. For example, a C program might build up a string in a value with repeated calls to **Tcl_AppendToObj**, and then call **Tcl_ListObjIndex** to extract a list element from the value. The same value holding the same string value can have several different internal

representations at different times. Extension writers can also force a value to be converted from one type to another using the **Tcl_ConvertToType** procedure. Only programmers that create new value types need to be concerned about how this is done. A procedure defined as part of the value type's implementation creates a new internal representation for a value and changes its *typePtr*. See the man page for **Tcl_RegisterObjType** to see how to create a new value type.

Example of the lifetime of a value

As an example of the lifetime of a value, consider the following sequence of commands:

```
set x 123
```

This assigns to *x* an untyped value whose *bytes* member points to **123** and *length* member contains 3. The value's *typePtr* member is NULL.

```
puts "x is $x"
```

x's string representation is valid (since *bytes* is non-NULL) and is fetched for the command.

```
incr x
```

The **incr** command first gets an integer from *x*'s value by calling **Tcl_GetIntFromObj**. This procedure checks whether the value is already an integer value. Since it is not, it converts the value by setting the value's internal representation to the integer **123** and setting the value's *typePtr* to point to the integer Tcl_ObjType structure. Both representations are now valid. **incr** increments the value's integer internal representation then invalidates its string representation (by calling **Tcl_InvalidateStringRep**) since the string representation no longer corresponds to the internal representation.

```
puts "x is now $x"
```

The string representation of *x*'s value is needed and is recomputed. The string representation is now **124** and both representations are again valid.

Storage management of values

Tcl values are allocated on the heap and are shared as much as possible to reduce storage requirements. Reference counting is used to determine when a value is no longer needed and can safely be freed. A value just created by **Tcl_NewObj** or **Tcl_NewStringObj** has *refCount* 0. The macro **Tcl_IncrRefCount** increments the reference count when a new reference to the value is created. The macro **Tcl_DecrRefCount** decrements the count when a reference is no longer needed and, if the value's reference count drops to zero, frees its storage. A value shared by different code or data structures has *refCount* greater than 1. Incrementing a value's reference count ensures that it will not be freed too early or have its value change accidentally.

As an example, the bytecode interpreter shares argument values between calling and called Tcl procedures to avoid having to copy values. It assigns the call's argument values to the procedure's formal parameter variables. In doing so, it calls **Tcl_IncrRefCount** to increment the reference count of each argument since there is now a new reference to it from the formal parameter. When the called procedure returns, the interpreter calls **Tcl_DecrRefCount** to decrement each argument's reference count. When a value's reference count drops less than or equal to zero, **Tcl_DecrRefCount** reclaims its storage. Most command procedures do not have to be concerned about reference counting since they use a value's value immediately and do not

retain a pointer to the value after they return. However, if they do retain a pointer to a value in a data structure, they must be careful to increment its reference count since the retained pointer is a new reference.

Command procedures that directly modify values such as those for **lappend** and **linsert** must be careful to copy a shared value before changing it. They must first check whether the value is shared by calling **Tcl_IsShared**. If the value is shared they must copy the value by using **Tcl_DuplicateObj**; this returns a new duplicate of the original value that has *refCount* 0. If the value is not shared, the command procedure "owns" the value and can safely modify it directly. For example, the following code appears in the command procedure that implements **linsert**. This procedure modifies the list value passed to it in *objv[1]* by inserting *objc-3* new elements before *index*.

```
listPtr = objv[1];
if (Tcl_IsShared(listPtr)) {
    listPtr = Tcl_DuplicateObj(listPtr);
}
result = Tcl_ListObjReplace(interp, listPtr, index, 0,
        (objc-3), &(objv[3]));
```

As another example, **incr**'s command procedure must check whether the variable's value is shared before incrementing the integer in its internal representation. If it is shared, it needs to duplicate the value in order to avoid accidentally changing values in other data structures.

Keywords: Object, internal representation, value, value creation, value type, reference counting, string representation, type conversion

Copyright (c) 1996-1997 Sun Microsystems, Inc.

ObjectType

Name

Tcl_RegisterObjType, Tcl_GetObjType, Tcl_AppendAllObjTypes, Tcl_ConvertToType - manipulate Tcl value types

Synopsis

#include <tcl.h>

Tcl_RegisterObjType(*typePtr*)

const Tcl_ObjType *
Tcl_GetObjType(*typeName*)

int
Tcl_AppendAllObjTypes(*interp*, *objPtr*)

int
Tcl_ConvertToType(*interp*, *objPtr*, *typePtr*)

Arguments

const Tcl_ObjType ****typePtr* **(in)

Points to the structure containing information about the Tcl value type. This storage must live forever, typically by being statically allocated.

const char *_typeName_ (in)
The name of a Tcl value type that **Tcl_GetObjType** should look up.

Tcl_Interp *_interp_ (in)
Interpreter to use for error reporting.

Tcl_Obj *_objPtr_ (in)
For **Tcl_AppendAllObjTypes**, this points to the value onto which it appends the name of each value type as a list element. For **Tcl_ConvertToType**, this points to a value that must have been the result of a previous call to **Tcl_NewObj**.

Description

The procedures in this man page manage Tcl value types (sometimes referred to as object types or **Tcl_ObjType**s for historical reasons). They are used to register new value types, look up types, and force conversions from one type to another.

Tcl_RegisterObjType registers a new Tcl value type in the table of all value types that **Tcl_GetObjType** can look up by name. There are other value types supported by Tcl as well, which Tcl chooses not to register. Extensions can likewise choose to register the value types they create or not. The argument *typePtr* points to a Tcl_ObjType structure that describes the new type by giving its name and by supplying pointers to four procedures that implement the type. If the type table already contains a type with the same name as in *typePtr*, it is replaced with the new type. The Tcl_ObjType structure is described in the section **THE TCL_OBJTYPE STRUCTURE** below.

Tcl_GetObjType returns a pointer to the registered Tcl_ObjType with name *typeName*. It returns NULL if no type with that name is registered.

Tcl_AppendAllObjTypes appends the name of each registered value type as a list element onto the Tcl value referenced by *objPtr*. The return value is **TCL_OK** unless there was an error converting *objPtr* to a list value; in that case **TCL_ERROR** is returned.

Tcl_ConvertToType converts a value from one type to another if possible. It creates a new internal representation for *objPtr* appropriate for the target type *typePtr* and sets its *typePtr* member as determined by calling the *typePtr->setFromAnyProc* routine. Any internal representation for *objPtr*'s old type is freed. If an error occurs during conversion, it returns **TCL_ERROR** and leaves an error message in the result value for *interp* unless *interp* is NULL. Otherwise, it returns **TCL_OK**. Passing a NULL *interp* allows this procedure to be used as a test whether the conversion can be done (and in fact was done).

In many cases, the *typePtr->setFromAnyProc* routine will set *objPtr->typePtr* to the argument value *typePtr*, but that is no longer guaranteed. The *setFromAnyProc* is free to set the internal representation for *objPtr* to make use of another related Tcl_ObjType, if it sees fit.

The tcl_objtype structure

Extension writers can define new value types by defining four procedures and initializing a Tcl_ObjType structure to describe the type. Extension writers may also pass a pointer to their Tcl_ObjType structure to **Tcl_RegisterObjType** if they wish to permit other extensions to look up their Tcl_ObjType by name with the **Tcl_GetObjType** routine. The **Tcl_ObjType** structure is defined as follows:

```
typedef struct Tcl_ObjType {
    const char *name;
    Tcl_FreeInternalRepProc *freeIntRepProc;
    Tcl_DupInternalRepProc *dupIntRepProc;
    Tcl_UpdateStringProc *updateStringProc;
    Tcl_SetFromAnyProc *setFromAnyProc;
} Tcl_ObjType;
```

The name field

The *name* member describes the name of the type, e.g. **int**. When a type is registered, this is the name used by callers of **Tcl_GetObjType** to lookup the type. For unregistered types, the *name* field is primarily of value for debugging. The remaining four members are pointers to procedures called by the generic Tcl value code:

The setfromanyproc field

The *setFromAnyProc* member contains the address of a function called to create a valid internal representation from a value's string representation.

```
typedef int Tcl_SetFromAnyProc(
        Tcl_Interp *interp,
        Tcl_Obj *objPtr);
```

If an internal representation cannot be created from the string, it returns **TCL_ERROR** and puts a message describing the error in the result value for *interp* unless *interp* is NULL. If *setFromAnyProc* is successful, it stores the new internal representation, sets *objPtr*'s *typePtr* member to point to the **Tcl_ObjType** struct corresponding to the new internal representation, and returns **TCL_OK**. Before setting the new internal representation, the *setFromAnyProc* must free any internal representation of *objPtr*'s old type; it does this by calling the old type's *freeIntRepProc* if it is not NULL.

As an example, the *setFromAnyProc* for the built-in Tcl list type gets an up-to-date string representation for *objPtr* by calling **Tcl_GetStringFromObj**. It parses the string to verify it is in a valid list format and to obtain each element value in the list, and, if this succeeds, stores the list elements in *objPtr*'s internal representation and sets *objPtr*'s *typePtr* member to point to the list type's Tcl_ObjType structure.

Do not release *objPtr*'s old internal representation unless you replace it with a new one or reset the *typePtr* member to NULL.

The *setFromAnyProc* member may be set to NULL, if the routines making use of the internal representation have no need to derive that internal representation from an arbitrary string value. However, in this case, passing a pointer to the type to **Tcl_ConvertToType** will lead to a panic, so to avoid this possibility, the type should *not* be registered.

The updatestringproc field

The *updateStringProc* member contains the address of a function called to create a valid string representation from a value's internal representation.

```
typedef void Tcl_UpdateStringProc(
        Tcl_Obj *objPtr);
```

objPtr's *bytes* member is always NULL when it is called. It must always set *bytes* non-NULL before returning. We require the string representation's byte array to have a null after the last byte, at offset *length*, and to have no null bytes before that; this allows string representations to be treated as conventional null character-terminated C strings. These restrictions are easily met by using Tcl's internal UTF encoding for the string representation, same as one would do for other Tcl routines accepting string values as arguments. Storage for the byte array must be allocated in the heap by **Tcl_Alloc** or **ckalloc**. Note that *updateStringProc*s must allocate enough storage for the string's bytes and the terminating null byte.

The *updateStringProc* for Tcl's built-in double type, for example, calls Tcl_PrintDouble to write to a buffer of size TCL_DOUBLE_SPACE, then allocates and copies the string representation to just enough space to hold it. A pointer to the allocated space is stored in the *bytes* member.

The *updateStringProc* member may be set to NULL, if the routines making use of the internal representation are written so that the string representation is never invalidated. Failure to meet this obligation will lead to panics or crashes when **Tcl_GetStringFromObj** or other similar routines ask for the string representation.

The dupintrepproc field

The *dupIntRepProc* member contains the address of a function called to copy an internal representation from one value to another.

```
typedef void Tcl_DupInternalRepProc(
        Tcl_Obj *srcPtr,
        Tcl_Obj *dupPtr);
```

dupPtr's internal representation is made a copy of *srcPtr*'s internal representation. Before the call, *srcPtr*'s internal representation is valid and *dupPtr*'s is not. *srcPtr*'s value type determines what copying its internal representation means.

For example, the *dupIntRepProc* for the Tcl integer type simply copies an integer. The built-in list type's *dupIntRepProc* uses a far more sophisticated scheme to continue sharing storage as much as it reasonably can.

The freeintrepproc field

The *freeIntRepProc* member contains the address of a function that is called when a value is freed.

```
typedef void Tcl_FreeInternalRepProc(
        Tcl_Obj *objPtr);
```

The *freeIntRepProc* function can deallocate the storage for the value's internal representation and do other type-specific processing necessary when a value is freed.

For example, the list type's *freeIntRepProc* respects the storage sharing scheme established by the *dupIntRepProc* so that it only frees storage when the last value sharing it is being freed.

The *freeIntRepProc* member can be set to NULL to indicate that the internal representation does not require freeing. The *freeIntRepProc* implementation must not access the *bytes* member of the value, since Tcl makes its own internal uses of that field during value deletion. The defined tasks for the *freeIntRepProc* have no need to consult the *bytes* member.

Keywords: ObjectType, internal representation, value, value type, string representation, type

conversion

Copyright (c) 1996-1997 Sun Microsystems, Inc.

OpenFileChnl

Name

Tcl_OpenFileChannel, Tcl_OpenCommandChannel, Tcl_MakeFileChannel, Tcl_GetChannel, Tcl_GetChannelNames, Tcl_GetChannelNamesEx, Tcl_RegisterChannel, Tcl_UnregisterChannel, Tcl_DetachChannel, Tcl_IsStandardChannel, Tcl_Close, Tcl_CloseEx, Tcl_ReadChars, Tcl_Read, Tcl_GetsObj, Tcl_Gets, Tcl_WriteObj, Tcl_WriteChars, Tcl_Write, Tcl_Flush, Tcl_Seek, Tcl_Tell, Tcl_TruncateChannel, Tcl_GetChannelOption, Tcl_SetChannelOption, Tcl_Eof, Tcl_InputBlocked, Tcl_InputBuffered, Tcl_OutputBuffered, Tcl_Ungets, Tcl_ReadRaw, Tcl_WriteRaw - buffered I/O facilities using channels

Synopsis

#include <tcl.h>

Tcl_Channel
Tcl_OpenFileChannel(*interp*, *fileName*, *mode*, *permissions*)

Tcl_Channel
Tcl_OpenCommandChannel(*interp*, *argc*, *argv*, *flags*)

Tcl_Channel
Tcl_MakeFileChannel(*handle*, *readOrWrite*)

Tcl_Channel
Tcl_GetChannel(*interp*, *channelName*, *modePtr*)

int
Tcl_GetChannelNames(*interp*)

int
Tcl_GetChannelNamesEx(*interp*, *pattern*)

void
Tcl_RegisterChannel(*interp*, *channel*)

int
Tcl_UnregisterChannel(*interp*, *channel*)

int
Tcl_DetachChannel(*interp*, *channel*)

int
Tcl_IsStandardChannel(*channel*)

int
Tcl_Close(*interp*, *channel*)

int
Tcl_CloseEx(*interp*, *channel*, *closeFlags*)

int
Tcl_ReadChars(*channel*, *readObjPtr*, *charsToRead*, *appendFlag*)

int

Tcl_Read(*channel*, *readBuf*, *bytesToRead*)

int
Tcl_GetsObj(*channel*, *lineObjPtr*)

int
Tcl_Gets(*channel*, *lineRead*)

int
Tcl_Ungets(*channel*, *input*, *inputLen*, *addAtEnd*)

int
Tcl_WriteObj(*channel*, *writeObjPtr*)

int
Tcl_WriteChars(*channel*, *charBuf*, *bytesToWrite*)

int
Tcl_Write(*channel*, *byteBuf*, *bytesToWrite*)

int
Tcl_ReadRaw(*channel*, *readBuf*, *bytesToRead*)

int
Tcl_WriteRaw(*channel*, *byteBuf*, *bytesToWrite*)

int
Tcl_Eof(*channel*)

int
Tcl_Flush(*channel*)

int
Tcl_InputBlocked(*channel*)

int
Tcl_InputBuffered(*channel*)

int
Tcl_OutputBuffered(*channel*)

Tcl_WideInt
Tcl_Seek(*channel*, *offset*, *seekMode*)

Tcl_WideInt
Tcl_Tell(*channel*)

int
Tcl_TruncateChannel(*channel*, *length*)

int
Tcl_GetChannelOption(*interp*, *channel*, *optionName*, *optionValue*)

int
Tcl_SetChannelOption(*interp*, *channel*, *optionName*, *newValue*)

Arguments

Tcl_Interp *_interp_ (in)
Used for error reporting and to look up a channel registered in it.

const char *_fileName_ (in)
The name of a local or network file.

const char *_mode_ (in)
Specifies how the file is to be accessed. May have any of the values allowed for the *mode* argument to the Tcl **open** command.

int _permissions_ (in)
POSIX-style permission flags such as 0644. If a new file is created, these permissions will be set on the created file.

int _argc_ (in)
The number of elements in *argv*.

const char **_argv_ (in)
Arguments for constructing a command pipeline. These values have the same meaning as the non-switch arguments to the Tcl **exec** command.

int _flags_ (in)
Specifies the disposition of the stdio handles in pipeline: OR-ed combination of **TCL_STDIN**, **TCL_STDOUT**, **TCL_STDERR**, and **TCL_ENFORCE_MODE**. If **TCL_STDIN** is set, stdin for the first child in the pipe is the pipe channel, otherwise it is the same as the standard input of the invoking process; likewise for **TCL_STDOUT** and **TCL_STDERR**. If **TCL_ENFORCE_MODE** is not set, then the pipe can redirect stdio handles to override the stdio handles for which **TCL_STDIN**, **TCL_STDOUT** and **TCL_STDERR** have been set. If it is set, then such redirections cause an error.

ClientData _handle_ (in)
Operating system specific handle for I/O to a file. For Unix this is a file descriptor, for Windows it is a HANDLE.

int _readOrWrite_ (in)
OR-ed combination of **TCL_READABLE** and **TCL_WRITABLE** to indicate what operations are valid on *handle*.

const char *_channelName_ (in)
The name of the channel.

int *_modePtr_ (out)
Points at an integer variable that will receive an OR-ed combination of **TCL_READABLE** and **TCL_WRITABLE** denoting whether the channel is open for reading and writing.

const char *_pattern_ (in)
The pattern to match on, passed to Tcl_StringMatch, or NULL.

Tcl_Channel _channel_ (in)
A Tcl channel for input or output. Must have been the return value from a procedure such as **Tcl_OpenFileChannel**.

Tcl_Obj *_readObjPtr_ (in/out)
A pointer to a Tcl value in which to store the characters read from the channel.

int _charsToRead_ (in)
The number of characters to read from the channel. If the channel's encoding is **binary**, this is equivalent to the number of bytes to read from the channel.

int _closeFlags_ (in)
If **0**, the channel is closed in both directions. If **TCL_CLOSE_READ**, the channel is only closed

for reading. If **TCL_CLOSE_WRITE**, the channel is only closed for writing. These flags must not be combined.

int *appendFlag* (in)
If non-zero, data read from the channel will be appended to the value. Otherwise, the data will replace the existing contents of the value.

char **readBuf* (out)
A buffer in which to store the bytes read from the channel.

int *bytesToRead* (in)
The number of bytes to read from the channel. The buffer *readBuf* must be large enough to hold this many bytes.

Tcl_Obj **lineObjPtr* (in/out)
A pointer to a Tcl value in which to store the line read from the channel. The line read will be appended to the current value of the value.

Tcl_DString **lineRead* (in/out)
A pointer to a Tcl dynamic string in which to store the line read from the channel. Must have been initialized by the caller. The line read will be appended to any data already in the dynamic string.

const char **input* (in)
The input to add to a channel buffer.

int *inputLen* (in)
Length of the input

int *addAtEnd* (in)
Flag indicating whether the input should be added to the end or beginning of the channel buffer.

Tcl_Obj **writeObjPtr* (in)
A pointer to a Tcl value whose contents will be output to the channel.

const char **charBuf* (in)
A buffer containing the characters to output to the channel.

const char **byteBuf* (in)
A buffer containing the bytes to output to the channel.

int *bytesToWrite* (in)
The number of bytes to consume from *charBuf* or *byteBuf* and output to the channel.

Tcl_WideInt *offset* (in)
How far to move the access point in the channel at which the next input or output operation will be applied, measured in bytes from the position given by *seekMode*. May be either positive or negative.

int *seekMode* (in)
Relative to which point to seek; used with *offset* to calculate the new access point for the channel. Legal values are **SEEK_SET**, **SEEK_CUR**, and **SEEK_END**.

Tcl_WideInt *length* (in)
The (non-negative) length to truncate the channel the channel to.

const char **optionName* (in)
The name of an option applicable to this channel, such as **-blocking**. May have any of the values accepted by the **fconfigure** command.

Tcl_DString *_optionValue_ (in)
Where to store the value of an option or a list of all options and their values. Must have been initialized by the caller.

const char *_newValue_ (in)
New value for the option given by *optionName*.

Description

The Tcl channel mechanism provides a device-independent and platform-independent mechanism for performing buffered input and output operations on a variety of file, socket, and device types. The channel mechanism is extensible to new channel types, by providing a low-level channel driver for the new type; the channel driver interface is described in the manual entry for **Tcl_CreateChannel**. The channel mechanism provides a buffering scheme modeled after Unix's standard I/O, and it also allows for nonblocking I/O on channels.

The procedures described in this manual entry comprise the C APIs of the generic layer of the channel architecture. For a description of the channel driver architecture and how to implement channel drivers for new types of channels, see the manual entry for **Tcl_CreateChannel**.

Tcl_openfilechannel

Tcl_OpenFileChannel opens a file specified by *fileName* and returns a channel handle that can be used to perform input and output on the file. This API is modeled after the **fopen** procedure of the Unix standard I/O library. The syntax and meaning of all arguments is similar to those given in the Tcl **open** command when opening a file. If an error occurs while opening the channel, **Tcl_OpenFileChannel** returns NULL and records a POSIX error code that can be retrieved with **Tcl_GetErrno**. In addition, if *interp* is non-NULL, **Tcl_OpenFileChannel** leaves an error message in *interp*'s result after any error. As of Tcl 8.4, the value-based API **Tcl_FSOpenFileChannel** should be used in preference to **Tcl_OpenFileChannel** wherever possible.

The newly created channel is not registered in the supplied interpreter; to register it, use **Tcl_RegisterChannel**, described below. If one of the standard channels, **stdin**, **stdout** or **stderr** was previously closed, the act of creating the new channel also assigns it as a replacement for the standard channel.

Tcl_opencommandchannel

Tcl_OpenCommandChannel provides a C-level interface to the functions of the **exec** and **open** commands. It creates a sequence of subprocesses specified by the *argv* and *argc* arguments and returns a channel that can be used to communicate with these subprocesses. The *flags* argument indicates what sort of communication will exist with the command pipeline.

If the **TCL_STDIN** flag is set then the standard input for the first subprocess will be tied to the channel: writing to the channel will provide input to the subprocess. If **TCL_STDIN** is not set, then standard input for the first subprocess will be the same as this application's standard input. If **TCL_STDOUT** is set then standard output from the last subprocess can be read from the channel; otherwise it goes to this application's standard output. If **TCL_STDERR** is set, standard error output for all subprocesses is returned to the channel and results in an error when the channel is closed; otherwise it goes to this application's standard error. If **TCL_ENFORCE_MODE** is not set, then *argc* and *argv* can redirect the stdio handles to override **TCL_STDIN**, **TCL_STDOUT**, and **TCL_STDERR**; if it is set, then it is an error for argc and argv to override

stdio channels for which **TCL_STDIN**, **TCL_STDOUT**, and **TCL_STDERR** have been set.

If an error occurs while opening the channel, **Tcl_OpenCommandChannel** returns NULL and records a POSIX error code that can be retrieved with **Tcl_GetErrno**. In addition, **Tcl_OpenCommandChannel** leaves an error message in the interpreter's result. *interp* cannot be NULL.

The newly created channel is not registered in the supplied interpreter; to register it, use **Tcl_RegisterChannel**, described below. If one of the standard channels, **stdin**, **stdout** or **stderr** was previously closed, the act of creating the new channel also assigns it as a replacement for the standard channel.

Tcl_makefilechannel

Tcl_MakeFileChannel makes a **Tcl_Channel** from an existing, platform-specific, file handle. The newly created channel is not registered in the supplied interpreter; to register it, use **Tcl_RegisterChannel**, described below. If one of the standard channels, **stdin**, **stdout** or **stderr** was previously closed, the act of creating the new channel also assigns it as a replacement for the standard channel.

Tcl_getchannel

Tcl_GetChannel returns a channel given the *channelName* used to create it with **Tcl_CreateChannel** and a pointer to a Tcl interpreter in *interp*. If a channel by that name is not registered in that interpreter, the procedure returns NULL. If the *modePtr* argument is not NULL, it points at an integer variable that will receive an OR-ed combination of **TCL_READABLE** and **TCL_WRITABLE** describing whether the channel is open for reading and writing.

Tcl_GetChannelNames and **Tcl_GetChannelNamesEx** write the names of the registered channels to the interpreter's result as a list value. **Tcl_GetChannelNamesEx** will filter these names according to the *pattern*. If *pattern* is NULL, then it will not do any filtering. The return value is **TCL_OK** if no errors occurred writing to the result, otherwise it is **TCL_ERROR**, and the error message is left in the interpreter's result.

Tcl_registerchannel

Tcl_RegisterChannel adds a channel to the set of channels accessible in *interp*. After this call, Tcl programs executing in that interpreter can refer to the channel in input or output operations using the name given in the call to **Tcl_CreateChannel**. After this call, the channel becomes the property of the interpreter, and the caller should not call **Tcl_Close** for the channel; the channel will be closed automatically when it is unregistered from the interpreter.

Code executing outside of any Tcl interpreter can call **Tcl_RegisterChannel** with *interp* as NULL, to indicate that it wishes to hold a reference to this channel. Subsequently, the channel can be registered in a Tcl interpreter and it will only be closed when the matching number of calls to **Tcl_UnregisterChannel** have been made. This allows code executing outside of any interpreter to safely hold a reference to a channel that is also registered in a Tcl interpreter.

This procedure interacts with the code managing the standard channels. If no standard channels were initialized before the first call to **Tcl_RegisterChannel**, they will get initialized by that call. See **Tcl_StandardChannels** for a general treatise about standard channels and the behavior of the Tcl library with regard to them.

Tcl_unregisterchannel

Tcl_UnregisterChannel removes a channel from the set of channels accessible in *interp*. After this call, Tcl programs will no longer be able to use the channel's name to refer to the channel in that interpreter. If this operation removed the last registration of the channel in any interpreter, the channel is also closed and destroyed.

Code not associated with a Tcl interpreter can call **Tcl_UnregisterChannel** with *interp* as NULL, to indicate to Tcl that it no longer holds a reference to that channel. If this is the last reference to the channel, it will now be closed. **Tcl_UnregisterChannel** is very similar to **Tcl_DetachChannel** except that it will also close the channel if no further references to it exist.

Tcl_detachchannel

Tcl_DetachChannel removes a channel from the set of channels accessible in *interp*. After this call, Tcl programs will no longer be able to use the channel's name to refer to the channel in that interpreter. Beyond that, this command has no further effect. It cannot be used on the standard channels (**stdout**, **stderr**, **stdin**), and will return **TCL_ERROR** if passed one of those channels.

Code not associated with a Tcl interpreter can call **Tcl_DetachChannel** with *interp* as NULL, to indicate to Tcl that it no longer holds a reference to that channel. If this is the last reference to the channel, unlike **Tcl_UnregisterChannel**, it will not be closed.

Tcl_isstandardchannel

Tcl_IsStandardChannel tests whether a channel is one of the three standard channels, **stdin**, **stdout** or **stderr**. If so, it returns 1, otherwise 0.

No attempt is made to check whether the given channel or the standard channels are initialized or otherwise valid.

Tcl_close and tcl_closeex

Tcl_Close destroys the channel *channel*, which must denote a currently open channel. The channel should not be registered in any interpreter when **Tcl_Close** is called. Buffered output is flushed to the channel's output device prior to destroying the channel, and any buffered input is discarded. If this is a blocking channel, the call does not return until all buffered data is successfully sent to the channel's output device. If this is a nonblocking channel and there is buffered output that cannot be written without blocking, the call returns immediately; output is flushed in the background and the channel will be closed once all of the buffered data has been output. In this case errors during flushing are not reported.

Tcl_CloseEx allows for both full closing and half-closing of channels depending on its **closeFlags** parameter. See the description of the parameter above. It is an error to attempt to close the channel for a direction for which it is not open. The channel is destroyed only when it has been closed for both reading and writing. Only socket and command pipe channels support half-closing.

If the channel was closed successfully, **Tcl_Close** and **Tcl_CloseEx** return **TCL_OK**. If an error occurs, they return **TCL_ERROR** and record a POSIX error code that can be retrieved with **Tcl_GetErrno**. If the channel is being closed synchronously and an error occurs during closing of the channel and *interp* is not NULL, an error message is left in the interpreter's result.

Note: it is not safe to call the channel closing functions on a channel that has been registered using **Tcl_RegisterChannel**; see the documentation for **Tcl_RegisterChannel**, above, for

details. If the channel has ever been given as the **chan** argument in a call to **Tcl_RegisterChannel**, you should instead use **Tcl_UnregisterChannel**, which will internally call **Tcl_Close** when all calls to **Tcl_RegisterChannel** have been matched by corresponding calls to **Tcl_UnregisterChannel**.

Tcl_readchars and tcl_read

Tcl_ReadChars consumes bytes from *channel*, converting the bytes to UTF-8 based on the channel's encoding and storing the produced data in *readObjPtr*'s string representation. The return value of **Tcl_ReadChars** is the number of characters, up to *charsToRead*, that were stored in *readObjPtr*. If an error occurs while reading, the return value is -1 and **Tcl_ReadChars** records a POSIX error code that can be retrieved with **Tcl_GetErrno**.

Setting *charsToRead* to **-1** will cause the command to read all characters currently available (non-blocking) or everything until eof (blocking mode).

The return value may be smaller than the value to read, indicating that less data than requested was available. This is called a *short read*. In blocking mode, this can only happen on an end-of-file. In nonblocking mode, a short read can also occur if there is not enough input currently available: **Tcl_ReadChars** returns a short count rather than waiting for more data.

If the channel is in blocking mode, a return value of zero indicates an end-of-file condition. If the channel is in nonblocking mode, a return value of zero indicates either that no input is currently available or an end-of-file condition. Use **Tcl_Eof** and **Tcl_InputBlocked** to tell which of these conditions actually occurred.

Tcl_ReadChars translates the various end-of-line representations into the canonical **\n** internal representation according to the current end-of-line recognition mode. End-of-line recognition and the various platform-specific modes are described in the manual entry for the Tcl **fconfigure** command.

As a performance optimization, when reading from a channel with the encoding **binary**, the bytes are not converted to UTF-8 as they are read. Instead, they are stored in *readObjPtr*'s internal representation as a byte-array value. The string representation of this value will only be constructed if it is needed (e.g., because of a call to **Tcl_GetStringFromObj**). In this way, byte-oriented data can be read from a channel, manipulated by calling **Tcl_GetByteArrayFromObj** and related functions, and then written to a channel without the expense of ever converting to or from UTF-8.

Tcl_Read is similar to **Tcl_ReadChars**, except that it does not do encoding conversions, regardless of the channel's encoding. It is deprecated and exists for backwards compatibility with non-internationalized Tcl extensions. It consumes bytes from *channel* and stores them in *readBuf*, performing end-of-line translations on the way. The return value of **Tcl_Read** is the number of bytes, up to *bytesToRead*, written in *readBuf*. The buffer produced by **Tcl_Read** is not null-terminated. Its contents are valid from the zeroth position up to and excluding the position indicated by the return value.

Tcl_ReadRaw is the same as **Tcl_Read** but does not compensate for stacking. While **Tcl_Read** (and the other functions in the API) always get their data from the topmost channel in the stack the supplied channel is part of, **Tcl_ReadRaw** does not. Thus this function is **only** usable for transformational channel drivers, i.e. drivers used in the middle of a stack of channels, to move data from the channel below into the transformation.

Tcl_getsobj and tcl_gets

Tcl_GetsObj consumes bytes from *channel*, converting the bytes to UTF-8 based on the channel's encoding, until a full line of input has been seen. If the channel's encoding is **binary**, each byte read from the channel is treated as an individual Unicode character. All of the characters of the line except for the terminating end-of-line character(s) are appended to *lineObjPtr*'s string representation. The end-of-line character(s) are read and discarded.

If a line was successfully read, the return value is greater than or equal to zero and indicates the number of bytes stored in *lineObjPtr*. If an error occurs, **Tcl_GetsObj** returns -1 and records a POSIX error code that can be retrieved with **Tcl_GetErrno**. **Tcl_GetsObj** also returns -1 if the end of the file is reached; the **Tcl_Eof** procedure can be used to distinguish an error from an end-of-file condition.

If the channel is in nonblocking mode, the return value can also be -1 if no data was available or the data that was available did not contain an end-of-line character. When -1 is returned, the **Tcl_InputBlocked** procedure may be invoked to determine if the channel is blocked because of input unavailability.

Tcl_Gets is the same as **Tcl_GetsObj** except the resulting characters are appended to the dynamic string given by *lineRead* rather than a Tcl value.

Tcl_ungets

Tcl_Ungets is used to add data to the input queue of a channel, at either the head or tail of the queue. The pointer *input* points to the data that is to be added. The length of the input to add is given by *inputLen*. A non-zero value of *addAtEnd* indicates that the data is to be added at the end of queue; otherwise it will be added at the head of the queue. If *channel* has a "sticky" EOF set, no data will be added to the input queue. **Tcl_Ungets** returns *inputLen* or -1 if an error occurs.

Tcl_writechars, tcl_writeobj, and tcl_write

Tcl_WriteChars accepts *bytesToWrite* bytes of character data at *charBuf*. The UTF-8 characters in the buffer are converted to the channel's encoding and queued for output to *channel*. If *bytesToWrite* is negative, **Tcl_WriteChars** expects *charBuf* to be null-terminated and it outputs everything up to the null.

Data queued for output may not appear on the output device immediately, due to internal buffering. If the data should appear immediately, call **Tcl_Flush** after the call to **Tcl_WriteChars** , or set the **-buffering** option on the channel to **none**. If you wish the data to appear as soon as a complete line is accepted for output, set the **-buffering** option on the channel to **line** mode.

The return value of **Tcl_WriteChars** is a count of how many bytes were accepted for output to the channel. This is either greater than zero to indicate success or -1 to indicate that an error occurred. If an error occurs, **Tcl_WriteChars** records a POSIX error code that may be retrieved with **Tcl_GetErrno**.

Newline characters in the output data are translated to platform-specific end-of-line sequences according to the **-translation** option for the channel. This is done even if the channel has no encoding.

Tcl_WriteObj is similar to **Tcl_WriteChars** except it accepts a Tcl value whose contents will be output to the channel. The UTF-8 characters in *writeObjPtr*'s string representation are converted

to the channel's encoding and queued for output to *channel*. As a performance optimization, when writing to a channel with the encoding **binary**, UTF-8 characters are not converted as they are written. Instead, the bytes in *writeObjPtr*'s internal representation as a byte-array value are written to the channel. The byte-array representation of the value will be constructed if it is needed. In this way, byte-oriented data can be read from a channel, manipulated by calling **Tcl_GetByteArrayFromObj** and related functions, and then written to a channel without the expense of ever converting to or from UTF-8.

Tcl_Write is similar to **Tcl_WriteChars** except that it does not do encoding conversions, regardless of the channel's encoding. It is deprecated and exists for backwards compatibility with non-internationalized Tcl extensions. It accepts *bytesToWrite* bytes of data at *byteBuf* and queues them for output to *channel*. If *bytesToWrite* is negative, **Tcl_Write** expects *byteBuf* to be null-terminated and it outputs everything up to the null.

Tcl_WriteRaw is the same as **Tcl_Write** but does not compensate for stacking. While **Tcl_Write** (and the other functions in the API) always feed their input to the topmost channel in the stack the supplied channel is part of, **Tcl_WriteRaw** does not. Thus this function is **only** usable for transformational channel drivers, i.e. drivers used in the middle of a stack of channels, to move data from the transformation into the channel below it.

Tcl_flush

Tcl_Flush causes all of the buffered output data for *channel* to be written to its underlying file or device as soon as possible. If the channel is in blocking mode, the call does not return until all the buffered data has been sent to the channel or some error occurred. The call returns immediately if the channel is nonblocking; it starts a background flush that will write the buffered data to the channel eventually, as fast as the channel is able to absorb it.

The return value is normally **TCL_OK**. If an error occurs, **Tcl_Flush** returns **TCL_ERROR** and records a POSIX error code that can be retrieved with **Tcl_GetErrno**.

Tcl_seek

Tcl_Seek moves the access point in *channel* where subsequent data will be read or written. Buffered output is flushed to the channel and buffered input is discarded, prior to the seek operation.

Tcl_Seek normally returns the new access point. If an error occurs, **Tcl_Seek** returns -1 and records a POSIX error code that can be retrieved with **Tcl_GetErrno**. After an error, the access point may or may not have been moved.

Tcl_tell

Tcl_Tell returns the current access point for a channel. The returned value is -1 if the channel does not support seeking.

Tcl_truncatechannel

Tcl_TruncateChannel truncates the file underlying *channel* to a given *length* of bytes. It returns **TCL_OK** if the operation succeeded, and **TCL_ERROR** otherwise.

Tcl_getchanneloption

Tcl_GetChannelOption retrieves, in *optionValue*, the value of one of the options currently in effect for a channel, or a list of all options and their values. The *channel* argument identifies the

channel for which to query an option or retrieve all options and their values. If *optionName* is not NULL, it is the name of the option to query; the option's value is copied to the Tcl dynamic string denoted by *optionValue*. If *optionName* is NULL, the function stores an alternating list of option names and their values in *optionValue*, using a series of calls to **Tcl_DStringAppendElement**. The various preexisting options and their possible values are described in the manual entry for the Tcl **fconfigure** command. Other options can be added by each channel type. These channel type specific options are described in the manual entry for the Tcl command that creates a channel of that type; for example, the additional options for TCP-based channels are described in the manual entry for the Tcl **socket** command. The procedure normally returns **TCL_OK**. If an error occurs, it returns **TCL_ERROR** and calls **Tcl_SetErrno** to store an appropriate POSIX error code.

Tcl_setchanneloption

Tcl_SetChannelOption sets a new value *newValue* for an option *optionName* on *channel*. The procedure normally returns **TCL_OK**. If an error occurs, it returns **TCL_ERROR**; in addition, if *interp* is non-NULL, **Tcl_SetChannelOption** leaves an error message in the interpreter's result.

Tcl_eof

Tcl_Eof returns a nonzero value if *channel* encountered an end of file during the last input operation.

Tcl_inputblocked

Tcl_InputBlocked returns a nonzero value if *channel* is in nonblocking mode and the last input operation returned less data than requested because there was insufficient data available. The call always returns zero if the channel is in blocking mode.

Tcl_inputbuffered

Tcl_InputBuffered returns the number of bytes of input currently buffered in the internal buffers for a channel. If the channel is not open for reading, this function always returns zero.

Tcl_outputbuffered

Tcl_OutputBuffered returns the number of bytes of output currently buffered in the internal buffers for a channel. If the channel is not open for writing, this function always returns zero.

Platform issues

The handles returned from **Tcl_GetChannelHandle** depend on the platform and the channel type. On Unix platforms, the handle is always a Unix file descriptor as returned from the **open** system call. On Windows platforms, the handle is a file **HANDLE** when the channel was created with **Tcl_OpenFileChannel**, **Tcl_OpenCommandChannel**, or **Tcl_MakeFileChannel**. Other channel types may return a different type of handle on Windows platforms.

Keywords: OpenFileChnl, access point, blocking, buffered I/O, channel, channel driver, end of file, flush, input, nonblocking, output, read, seek, write

Copyright (c) 1996-1997 Sun Microsystems, Inc.

OpenTcp

Name

Tcl_OpenTcpClient, Tcl_MakeTcpClientChannel, Tcl_OpenTcpServer - procedures to open channels using TCP sockets

Synopsis

#include <tcl.h>

Tcl_Channel
Tcl_OpenTcpClient(*interp*, *port*, *host*, *myaddr*, *myport*, *async*)

Tcl_Channel
Tcl_MakeTcpClientChannel(*sock*)

Tcl_Channel
Tcl_OpenTcpServer(*interp*, *port*, *myaddr*, *proc*, *clientData*)

Arguments

Tcl_Interp *_interp_ (in)
Tcl interpreter to use for error reporting. If non-NULL and an error occurs, an error message is left in the interpreter's result.

int _port_ (in)
A port number to connect to as a client or to listen on as a server.

const char *_host_ (in)
A string specifying a host name or address for the remote end of the connection.

int _myport_ (in)
A port number for the client's end of the socket. If 0, a port number is allocated at random.

const char *_myaddr_ (in)
A string specifying the host name or address for network interface to use for the local end of the connection. If NULL, a default interface is chosen.

int _async_ (in)
If nonzero, the client socket is connected asynchronously to the server.

ClientData _sock_ (in)
Platform-specific handle for client TCP socket.

Tcl_TcpAcceptProc *_proc_ (in)
Pointer to a procedure to invoke each time a new connection is accepted via the socket.

ClientData _clientData_ (in)
Arbitrary one-word value to pass to *proc*.

Description

These functions are convenience procedures for creating channels that communicate over TCP sockets. The operations on a channel are described in the manual entry for **Tcl_OpenFileChannel**.

Tcl_opentcpclient

Tcl_OpenTcpClient opens a client TCP socket connected to a *port* on a specific *host*, and returns a channel that can be used to communicate with the server. The host to connect to can be specified either as a domain name style name (e.g. **www.sunlabs.com**), or as a string containing the alphanumeric representation of its four-byte address (e.g. **127.0.0.1**). Use the string **localhost** to connect to a TCP socket on the host on which the function is invoked.

The *myaddr* and *myport* arguments allow a client to specify an address for the local end of the connection. If *myaddr* is NULL, then an interface is chosen automatically by the operating system. If *myport* is 0, then a port number is chosen at random by the operating system.

If *async* is zero, the call to **Tcl_OpenTcpClient** returns only after the client socket has either successfully connected to the server, or the attempted connection has failed. If *async* is nonzero the socket is connected asynchronously and the returned channel may not yet be connected to the server when the call to **Tcl_OpenTcpClient** returns. If the channel is in blocking mode and an input or output operation is done on the channel before the connection is completed or fails, that operation will wait until the connection either completes successfully or fails. If the channel is in nonblocking mode, the input or output operation will return immediately and a subsequent call to **Tcl_InputBlocked** on the channel will return nonzero.

The returned channel is opened for reading and writing. If an error occurs in opening the socket, **Tcl_OpenTcpClient** returns NULL and records a POSIX error code that can be retrieved with **Tcl_GetErrno**. In addition, if *interp* is non-NULL, an error message is left in the interpreter's result.

The newly created channel is not registered in the supplied interpreter; to register it, use **Tcl_RegisterChannel**. If one of the standard channels, **stdin**, **stdout** or **stderr** was previously closed, the act of creating the new channel also assigns it as a replacement for the standard channel.

Tcl_maketcpclientchannel

Tcl_MakeTcpClientChannel creates a **Tcl_Channel** around an existing, platform specific, handle for a client TCP socket.

The newly created channel is not registered in the supplied interpreter; to register it, use **Tcl_RegisterChannel**. If one of the standard channels, **stdin**, **stdout** or **stderr** was previously closed, the act of creating the new channel also assigns it as a replacement for the standard channel.

Tcl_opentcpserver

Tcl_OpenTcpServer opens a TCP socket on the local host on a specified *port* and uses the Tcl event mechanism to accept requests from clients to connect to it. The *myaddr* argument specifies the network interface. If *myaddr* is NULL the special address INADDR_ANY should be used to allow connections from any network interface. Each time a client connects to this socket, Tcl creates a channel for the new connection and invokes *proc* with information about the channel. *Proc* must match the following prototype:

```
typedef void Tcl_TcpAcceptProc(
        ClientData clientData,
        Tcl_Channel channel,
        char *hostName,
        int port);
```

The *clientData* argument will be the same as the *clientData* argument to **Tcl_OpenTcpServer**, *channel* will be the handle for the new channel, *hostName* points to a string containing the name of the client host making the connection, and *port* will contain the client's port number. The new channel is opened for both input and output. If *proc* raises an error, the connection is closed automatically. *Proc* has no return value, but if it wishes to reject the connection it can close

channel.

Tcl_OpenTcpServer normally returns a pointer to a channel representing the server socket. If an error occurs, **Tcl_OpenTcpServer** returns NULL and records a POSIX error code that can be retrieved with **Tcl_GetErrno**. In addition, if the interpreter is non-NULL, an error message is left in the interpreter's result.

The channel returned by **Tcl_OpenTcpServer** cannot be used for either input or output. It is simply a handle for the socket used to accept connections. The caller can close the channel to shut down the server and disallow further connections from new clients.

TCP server channels operate correctly only in applications that dispatch events through **Tcl_DoOneEvent** or through Tcl commands such as **vwait**; otherwise Tcl will never notice that a connection request from a remote client is pending.

The newly created channel is not registered in the supplied interpreter; to register it, use **Tcl_RegisterChannel**. If one of the standard channels, **stdin**, **stdout** or **stderr** was previously closed, the act of creating the new channel also assigns it as a replacement for the standard channel.

Platform issues

On Unix platforms, the socket handle is a Unix file descriptor as returned by the **socket** system call. On the Windows platform, the socket handle is a **SOCKET** as defined in the WinSock API.

Keywords: OpenTcp, channel, client, server, socket, TCP

Copyright (c) 1996-7 Sun Microsystems, Inc.

Panic

Name

Tcl_Panic, Tcl_PanicVA, Tcl_SetPanicProc - report fatal error and abort

Synopsis

#include <tcl.h>

void
Tcl_Panic(*format*, *arg*, *arg*, ...)

void
Tcl_PanicVA(*format*, *argList*)

void
Tcl_SetPanicProc(*panicProc*)

Arguments

const char* *format* (in)
A printf-style format string.

***arg* (in)**
Arguments matching the format string.

va_list *argList* (in)
An argument list of arguments matching the format string. Must have been initialized using **va_start**, and cleared using **va_end**.

Tcl_PanicProc *_panicProc_ (in)
Procedure to report fatal error message and abort.

Description

When the Tcl library detects that its internal data structures are in an inconsistent state, or that its C procedures have been called in a manner inconsistent with their documentation, it calls **Tcl_Panic** to display a message describing the error and abort the process. The *format* argument is a format string describing how to format the remaining arguments *arg* into an error message, according to the same formatting rules used by the **printf** family of functions. The same formatting rules are also used by the built-in Tcl command **format**.

In a freshly loaded Tcl library, **Tcl_Panic** prints the formatted error message to the standard error file of the process, and then calls **abort** to terminate the process. **Tcl_Panic** does not return. On Windows, when a debugger is running, the formatted error message is sent to the debugger instead. If the windows executable does not have a stderr channel (e.g. **wish.exe**), then a system dialog box is used to display the panic message.

Tcl_SetPanicProc may be used to modify the behavior of **Tcl_Panic**. The *panicProc* argument should match the type **Tcl_PanicProc**:

```
typedef void Tcl_PanicProc(
        const char *format,
        arg, arg,...);
```

After **Tcl_SetPanicProc** returns, any future calls to **Tcl_Panic** will call *panicProc*, passing along the *format* and *arg* arguments. *panicProc* should avoid making calls into the Tcl library, or into other libraries that may call the Tcl library, since the original call to **Tcl_Panic** indicates the Tcl library is not in a state of reliable operation.

The typical use of **Tcl_SetPanicProc** arranges for the error message to be displayed or reported in a manner more suitable for the application or the platform.

Although the primary callers of **Tcl_Panic** are the procedures of the Tcl library, **Tcl_Panic** is a public function and may be called by any extension or application that wishes to abort the process and have a panic message displayed the same way that panic messages from Tcl will be displayed.

Tcl_PanicVA is the same as **Tcl_Panic** except that instead of taking a variable number of arguments it takes an argument list.

Keywords: Panic, abort, fatal, error

ParseArgs

Name

Tcl_ParseArgsObjv - parse arguments according to a tabular description

Synopsis

#include <tcl.h>

int
Tcl_ParseArgsObjv(*interp*, *argTable*, *objcPtr*, *objv*, *remObjv*)

Arguments

Tcl_Interp *_interp_ (out)
Where to store error messages.

const Tcl_ArgvInfo *_argTable_ (in)
Pointer to array of option descriptors.

int *_objcPtr_ (in/out)
A pointer to variable holding number of arguments in *objv*. Will be modified to hold number of arguments left in the unprocessed argument list stored in *remObjv*.

Tcl_Obj *const *_objv_ (in)
The array of arguments to be parsed.

Tcl_Obj ***_remObjv_ (out)
Pointer to a variable that will hold the array of unprocessed arguments. Should be NULL if no return of unprocessed arguments is required. If *objcPtr* is updated to a non-zero value, the array returned through this must be deallocated using **ckfree**.

Description

The **Tcl_ParseArgsObjv** function provides a system for parsing argument lists of the form "**-someName** *someValue* ...". Such argument lists are commonly found both in the arguments to a program and in the arguments to an individual Tcl command. This parser assumes that the order of the arguments does not matter, other than in so far as later copies of a duplicated option overriding earlier ones.

The argument array is described by the *objcPtr* and *objv* parameters, and an array of unprocessed arguments is returned through the *objcPtr* and *remObjv* parameters; if no return of unprocessed arguments is desired, the *remObjv* parameter should be NULL. If any problems happen, including if the "generate help" option is selected, an error message is left in the interpreter result and TCL_ERROR is returned. Otherwise, the interpreter result is left unchanged and TCL_OK is returned.

The collection of arguments to be parsed is described by the *argTable* parameter. This points to a table of descriptor structures that is terminated by an entry with the *type* field set to TCL_ARGV_END. As convenience, the following prototypical entries are provided:

TCL_ARGV_AUTO_HELP
Enables the argument processor to provide help when passed the argument "**-help**".

TCL_ARGV_AUTO_REST
Instructs the argument processor that arguments after "**--**" are to be unprocessed.

TCL_ARGV_TABLE_END
Marks the end of the table of argument descriptors.

Argument descriptor entries

Each entry of the argument descriptor table must be a structure of type **Tcl_ArgvInfo**. The structure is defined as this:

```
typedef struct {
    int type;
    const char *keyStr;
    void *srcPtr;
```

```
    void *dstPtr;
    const char *helpStr;
    ClientData clientData;
} Tcl_ArgvInfo;
```

The *keyStr* field contains the name of the option; by convention, this will normally begin with a "**-**" character. The *type*, *srcPtr*, *dstPtr* and *clientData* fields describe the interpretation of the value of the argument, as described below. The *helpStr* field gives some text that is used to provide help to users when they request it.

As noted above, the *type* field is used to describe the interpretation of the argument's value. The following values are acceptable values for *type*:

TCL_ARGV_CONSTANT
: The argument does not take any following value argument. If this argument is present, the (integer) value of the *srcPtr* field is copied to the variable pointed to by the *dstPtr* field. The *clientData* field is ignored.

TCL_ARGV_END
: This value marks the end of all option descriptors in the table. All other fields are ignored.

TCL_ARGV_FLOAT
: This argument takes a following floating point value argument. The value (once parsed by **Tcl_GetDoubleFromObj**) will be stored as a double-precision value in the variable pointed to by the *dstPtr* field. The *srcPtr* and *clientData* fields are ignored.

TCL_ARGV_FUNC
: This argument optionally takes a following value argument; it is up to the handler callback function passed in *srcPtr* to decide. That function will have the following signature:

```
typedef int (Tcl_ArgvFuncProc)(
        ClientData clientData,
        Tcl_Obj *objPtr,
        void *dstPtr);
```

The result is a boolean value indicating whether to consume the following argument. The *clientData* is the value from the table entry, the *objPtr* is the value that represents the following argument or NULL if there are no following arguments at all, and the *dstPtr* argument to the **Tcl_ArgvFuncProc** is the location to write the parsed value to.

TCL_ARGV_GENFUNC
: This argument takes zero or more following arguments; the handler callback function passed in *srcPtr* returns how many (or a negative number to signal an error, in which case it should also set the interpreter result). The function will have the following signature:

```
typedef int (Tcl_ArgvGenFuncProc)(
        ClientData clientData,
        Tcl_Interp *interp,
        int objc,
        Tcl_Obj *const *objv,
        void *dstPtr);
```

The *clientData* is the value from the table entry, the *interp* is where to store any error messages,

objc and *objv* describe an array of all the remaining arguments, and *dstPtr* argument to the **Tcl_ArgvGenFuncProc** is the location to write the parsed value (or values) to.

TCL_ARGV_HELP
: This special argument does not take any following value argument, but instead causes **Tcl_ParseArgsObjv** to generate an error message describing the arguments supported. All other fields except the *helpStr* field are ignored.

TCL_ARGV_INT
: This argument takes a following integer value argument. The value (once parsed by **Tcl_GetIntFromObj**) will be stored as an int in the variable pointed to by the *dstPtr* field. The *srcPtr* field is ignored.

TCL_ARGV_REST
: This special argument does not take any following value argument, but instead marks all following arguments to be left unprocessed. The *srcPtr*, *dstPtr* and *clientData* fields are ignored.

TCL_ARGV_STRING
: This argument takes a following string value argument. A pointer to the string will be stored at *dstPtr*; the string inside will have a lifetime linked to the lifetime of the string representation of the argument value that it came from, and so should be copied if it needs to be retained. The *srcPtr* and *clientData* fields are ignored.

Keywords: ParseArgs, argument, parse

Copyright (c) 2008 Donal K. Fellows

ParseCmd

Name

Tcl_ParseCommand, Tcl_ParseExpr, Tcl_ParseBraces, Tcl_ParseQuotedString, Tcl_ParseVarName, Tcl_ParseVar, Tcl_FreeParse, Tcl_EvalTokens, Tcl_EvalTokensStandard - parse Tcl scripts and expressions

Synopsis

#include <tcl.h>

int
Tcl_ParseCommand(*interp*, *start*, *numBytes*, *nested*, *parsePtr*)

int
Tcl_ParseExpr(*interp*, *start*, *numBytes*, *parsePtr*)

int
Tcl_ParseBraces(*interp*, *start*, *numBytes*, *parsePtr*, *append*, *termPtr*)

int
Tcl_ParseQuotedString(*interp*, *start*, *numBytes*, *parsePtr*, *append*, *termPtr*)

int
Tcl_ParseVarName(*interp*, *start*, *numBytes*, *parsePtr*, *append*)

const char *
Tcl_ParseVar(*interp*, *start*, *termPtr*)

Tcl_FreeParse(*usedParsePtr*)

Tcl_Obj *
Tcl_EvalTokens(*interp*, *tokenPtr*, *numTokens*)

int
Tcl_EvalTokensStandard(*interp*, *tokenPtr*, *numTokens*)

Arguments

Tcl_Interp *_interp_ (out)
For procedures other than **Tcl_FreeParse**, **Tcl_EvalTokens** and **Tcl_EvalTokensStandard**, used only for error reporting; if NULL, then no error messages are left after errors. For **Tcl_EvalTokens** and **Tcl_EvalTokensStandard**, determines the context for evaluating the script and also is used for error reporting; must not be NULL.

const char *_start_ (in)
Pointer to first character in string to parse.

int _numBytes_ (in)
Number of bytes in string to parse, not including any terminating null character. If less than 0 then the script consists of all characters following *start* up to the first null character.

int _nested_ (in)
Non-zero means that the script is part of a command substitution so an unquoted close bracket should be treated as a command terminator. If zero, close brackets have no special meaning.

int _append_ (in)
Non-zero means that *_parsePtr_ already contains valid tokens; the new tokens should be appended to those already present. Zero means that *_parsePtr_ is uninitialized; any information in it is ignored. This argument is normally 0.

Tcl_Parse *_parsePtr_ (out)
Points to structure to fill in with information about the parsed command, expression, variable name, etc. Any previous information in this structure is ignored, unless *append* is non-zero in a call to **Tcl_ParseBraces**, **Tcl_ParseQuotedString**, or **Tcl_ParseVarName**.

const char **_termPtr_ (out)
If not NULL, points to a location where **Tcl_ParseBraces**, **Tcl_ParseQuotedString**, and **Tcl_ParseVar** will store a pointer to the character just after the terminating character (the close-brace, the last character of the variable name, or the close-quote (respectively)) if the parse was successful.

Tcl_Parse *_usedParsePtr_ (in)
Points to structure that was filled in by a previous call to **Tcl_ParseCommand**, **Tcl_ParseExpr**, **Tcl_ParseVarName**, etc.

Description

These procedures parse Tcl commands or portions of Tcl commands such as expressions or references to variables. Each procedure takes a pointer to a script (or portion thereof) and fills in the structure pointed to by *parsePtr* with a collection of tokens describing the information that was parsed. The procedures normally return **TCL_OK**. However, if an error occurs then they return **TCL_ERROR**, leave an error message in *interp*'s result (if *interp* is not NULL), and leave nothing in *parsePtr*.

Tcl_ParseCommand is a procedure that parses Tcl scripts. Given a pointer to a script, it parses the first command from the script. If the command was parsed successfully, **Tcl_ParseCommand** returns **TCL_OK** and fills in the structure pointed to by *parsePtr* with information about the structure of the command (see below for details). If an error occurred in parsing the command then **TCL_ERROR** is returned, an error message is left in *interp*'s result, and no information is left at **parsePtr*.

Tcl_ParseExpr parses Tcl expressions. Given a pointer to a script containing an expression, **Tcl_ParseExpr** parses the expression. If the expression was parsed successfully, **Tcl_ParseExpr** returns **TCL_OK** and fills in the structure pointed to by *parsePtr* with information about the structure of the expression (see below for details). If an error occurred in parsing the command then **TCL_ERROR** is returned, an error message is left in *interp*'s result, and no information is left at **parsePtr*.

Tcl_ParseBraces parses a string or command argument enclosed in braces such as **{hello}** or **{string \t with \t tabs}** from the beginning of its argument *start*. The first character of *start* must be **{**. If the braced string was parsed successfully, **Tcl_ParseBraces** returns **TCL_OK**, fills in the structure pointed to by *parsePtr* with information about the structure of the string (see below for details), and stores a pointer to the character just after the terminating **}** in the location given by **termPtr*. If an error occurs while parsing the string then **TCL_ERROR** is returned, an error message is left in *interp*'s result, and no information is left at **parsePtr* or **termPtr*.

Tcl_ParseQuotedString parses a double-quoted string such as **"sum is [expr {$a+$b}]"** from the beginning of the argument *start*. The first character of *start* must be ". If the double-quoted string was parsed successfully, **Tcl_ParseQuotedString** returns **TCL_OK**, fills in the structure pointed to by *parsePtr* with information about the structure of the string (see below for details), and stores a pointer to the character just after the terminating " in the location given by **termPtr*. If an error occurs while parsing the string then **TCL_ERROR** is returned, an error message is left in *interp*'s result, and no information is left at **parsePtr* or **termPtr*.

Tcl_ParseVarName parses a Tcl variable reference such as **$abc** or **$x([expr {$index + 1}])** from the beginning of its *start* argument. The first character of *start* must be **$**. If a variable name was parsed successfully, **Tcl_ParseVarName** returns **TCL_OK** and fills in the structure pointed to by *parsePtr* with information about the structure of the variable name (see below for details). If an error occurs while parsing the command then **TCL_ERROR** is returned, an error message is left in *interp*'s result (if *interp* is not NULL), and no information is left at **parsePtr*.

Tcl_ParseVar parses a Tcl variable reference such as **$abc** or **$x([expr {$index + 1}])** from the beginning of its *start* argument. The first character of *start* must be **$**. If the variable name is parsed successfully, **Tcl_ParseVar** returns a pointer to the string value of the variable. If an error occurs while parsing, then NULL is returned and an error message is left in *interp*'s result.

The information left at **parsePtr* by **Tcl_ParseCommand**, **Tcl_ParseExpr**, **Tcl_ParseBraces**, **Tcl_ParseQuotedString**, and **Tcl_ParseVarName** may include dynamically allocated memory. If these five parsing procedures return **TCL_OK** then the caller must invoke **Tcl_FreeParse** to release the storage at **parsePtr*. These procedures ignore any existing information in **parsePtr* (unless *append* is non-zero), so if repeated calls are being made to any of them then **Tcl_FreeParse** must be invoked once after each call.

Tcl_EvalTokensStandard evaluates a sequence of parse tokens from a Tcl_Parse structure. The

tokens typically consist of all the tokens in a word or all the tokens that make up the index for a reference to an array variable. **Tcl_EvalTokensStandard** performs the substitutions requested by the tokens and concatenates the resulting values. The return value from **Tcl_EvalTokensStandard** is a Tcl completion code with one of the values **TCL_OK**, **TCL_ERROR**, **TCL_RETURN**, **TCL_BREAK**, or **TCL_CONTINUE**, or possibly some other integer value originating in an extension. In addition, a result value or error message is left in *interp*'s result; it can be retrieved using **Tcl_GetObjResult**.

Tcl_EvalTokens differs from **Tcl_EvalTokensStandard** only in the return convention used: it returns the result in a new Tcl_Obj. The reference count of the value returned as result has been incremented, so the caller must invoke **Tcl_DecrRefCount** when it is finished with the value. If an error or other exception occurs while evaluating the tokens (such as a reference to a non-existent variable) then the return value is NULL and an error message is left in *interp*'s result. The use of **Tcl_EvalTokens** is deprecated.

Tcl_parse structure

Tcl_ParseCommand, **Tcl_ParseExpr**, **Tcl_ParseBraces**, **Tcl_ParseQuotedString**, and **Tcl_ParseVarName** return parse information in two data structures, Tcl_Parse and Tcl_Token:

```
typedef struct Tcl_Parse {
    const char *commentStart;
    int commentSize;
    const char *commandStart;
    int commandSize;
    int numWords;
    Tcl_Token *tokenPtr;
    int numTokens;
    ...
} Tcl_Parse;

typedef struct Tcl_Token {
    int type;
    const char *start;
    int size;
    int numComponents;
} Tcl_Token;
```

The first five fields of a Tcl_Parse structure are filled in only by **Tcl_ParseCommand**. These fields are not used by the other parsing procedures.

Tcl_ParseCommand fills in a Tcl_Parse structure with information that describes one Tcl command and any comments that precede the command. If there are comments, the *commentStart* field points to the **#** character that begins the first comment and *commentSize* indicates the number of bytes in all of the comments preceding the command, including the newline character that terminates the last comment. If the command is not preceded by any comments, *commentSize* is 0. **Tcl_ParseCommand** also sets the *commandStart* field to point to the first character of the first word in the command (skipping any comments and leading space) and *commandSize* gives the total number of bytes in the command, including the character pointed to by *commandStart* up to and including the newline, close bracket, or semicolon

character that terminates the command. The *numWords* field gives the total number of words in the command.

All parsing procedures set the remaining fields, *tokenPtr* and *numTokens*. The *tokenPtr* field points to the first in an array of Tcl_Token structures that describe the components of the entity being parsed. The *numTokens* field gives the total number of tokens present in the array. Each token contains four fields. The *type* field selects one of several token types that are described below. The *start* field points to the first character in the token and the *size* field gives the total number of characters in the token. Some token types, such as **TCL_TOKEN_WORD** and **TCL_TOKEN_VARIABLE**, consist of several component tokens, which immediately follow the parent token; the *numComponents* field describes how many of these there are. The *type* field has one of the following values:

TCL_TOKEN_WORD
: This token ordinarily describes one word of a command but it may also describe a quoted or braced string in an expression. The token describes a component of the script that is the result of concatenating together a sequence of subcomponents, each described by a separate subtoken. The token starts with the first non-blank character of the component (which may be a double-quote or open brace) and includes all characters in the component up to but not including the space, semicolon, close bracket, close quote, or close brace that terminates the component. The *numComponents* field counts the total number of sub-tokens that make up the word, including sub-tokens of **TCL_TOKEN_VARIABLE** and **TCL_TOKEN_BS** tokens.

TCL_TOKEN_SIMPLE_WORD
: This token has the same meaning as **TCL_TOKEN_WORD**, except that the word is guaranteed to consist of a single **TCL_TOKEN_TEXT** sub-token. The *numComponents* field is always 1.

TCL_TOKEN_EXPAND_WORD
: This token has the same meaning as **TCL_TOKEN_WORD**, except that the command parser notes this word began with the expansion prefix **{*}**, indicating that after substitution, the list value of this word should be expanded to form multiple arguments in command evaluation. This token type can only be created by Tcl_ParseCommand.

TCL_TOKEN_TEXT
: The token describes a range of literal text that is part of a word. The *numComponents* field is always 0.

TCL_TOKEN_BS
: The token describes a backslash sequence such as **\n** or **\0xA3**. The *numComponents* field is always 0.

TCL_TOKEN_COMMAND
: The token describes a command whose result must be substituted into the word. The token includes the square brackets that surround the command. The *numComponents* field is always 0 (the nested command is not parsed; call **Tcl_ParseCommand** recursively if you want to see its tokens).

TCL_TOKEN_VARIABLE
: The token describes a variable substitution, including the **$**, variable name, and array index (if there is one) up through the close parenthesis that terminates the index. This token is followed by one or more additional tokens that describe the variable name and array index. If *numComponents* is 1 then the variable is a scalar and the next token is a **TCL_TOKEN_TEXT**

token that gives the variable name. If *numComponents* is greater than 1 then the variable is an array: the first sub-token is a **TCL_TOKEN_TEXT** token giving the array name and the remaining sub-tokens are **TCL_TOKEN_TEXT**, **TCL_TOKEN_BS**, **TCL_TOKEN_COMMAND**, and **TCL_TOKEN_VARIABLE** tokens that must be concatenated to produce the array index. The *numComponents* field includes nested sub-tokens that are part of **TCL_TOKEN_VARIABLE** tokens in the array index.

TCL_TOKEN_SUB_EXPR

The token describes one subexpression of an expression (or an entire expression). A subexpression may consist of a value such as an integer literal, variable substitution, or parenthesized subexpression; it may also consist of an operator and its operands. The token starts with the first non-blank character of the subexpression up to but not including the space, brace, close-paren, or bracket that terminates the subexpression. This token is followed by one or more additional tokens that describe the subexpression. If the first sub-token after the **TCL_TOKEN_SUB_EXPR** token is a **TCL_TOKEN_OPERATOR** token, the subexpression consists of an operator and its token operands. If the operator has no operands, the subexpression consists of just the **TCL_TOKEN_OPERATOR** token. Each operand is described by a **TCL_TOKEN_SUB_EXPR** token. Otherwise, the subexpression is a value described by one of the token types **TCL_TOKEN_WORD**, **TCL_TOKEN_TEXT**, **TCL_TOKEN_BS**, **TCL_TOKEN_COMMAND**, **TCL_TOKEN_VARIABLE**, and **TCL_TOKEN_SUB_EXPR**. The *numComponents* field counts the total number of sub-tokens that make up the subexpression; this includes the sub-tokens for any nested **TCL_TOKEN_SUB_EXPR** tokens.

TCL_TOKEN_OPERATOR

The token describes one operator of an expression such as **&&** or **hypot**. A **TCL_TOKEN_OPERATOR** token is always preceded by a **TCL_TOKEN_SUB_EXPR** token that describes the operator and its operands; the **TCL_TOKEN_SUB_EXPR** token's *numComponents* field can be used to determine the number of operands. A binary operator such as * is followed by two **TCL_TOKEN_SUB_EXPR** tokens that describe its operands. A unary operator like **-** is followed by a single **TCL_TOKEN_SUB_EXPR** token for its operand. If the operator is a math function such as **log10**, the **TCL_TOKEN_OPERATOR** token will give its name and the following **TCL_TOKEN_SUB_EXPR** tokens will describe its operands; if there are no operands (as with **rand**), no **TCL_TOKEN_SUB_EXPR** tokens follow. There is one trinary operator,**?**, that appears in if-then-else subexpressions such as *x***?***y***:***z*; in this case, the**?** **TCL_TOKEN_OPERATOR** token is followed by three **TCL_TOKEN_SUB_EXPR** tokens for the operands *x*, *y*, and *z*. The *numComponents* field for a **TCL_TOKEN_OPERATOR** token is always 0. After **Tcl_ParseCommand** returns, the first token pointed to by the *tokenPtr* field of the Tcl_Parse structure always has type **TCL_TOKEN_WORD** or **TCL_TOKEN_SIMPLE_WORD** or **TCL_TOKEN_EXPAND_WORD**. It is followed by the sub-tokens that must be concatenated to produce the value of that word. The next token is the **TCL_TOKEN_WORD** or **TCL_TOKEN_SIMPLE_WORD** of **TCL_TOKEN_EXPAND_WORD** token for the second word, followed by sub-tokens for that word, and so on until all *numWords* have been accounted for. After **Tcl_ParseExpr** returns, the first token pointed to by the *tokenPtr* field of the Tcl_Parse structure always has type **TCL_TOKEN_SUB_EXPR**. It is followed by the sub-tokens that must be evaluated to produce the value of the expression. Only the token information in the Tcl_Parse structure is modified: the *commentStart*, *commentSize*, *commandStart*, and *commandSize* fields are not modified by **Tcl_ParseExpr**. After **Tcl_ParseBraces** returns, the

array of tokens pointed to by the *tokenPtr* field of the Tcl_Parse structure will contain a single **TCL_TOKEN_TEXT** token if the braced string does not contain any backslash-newlines. If the string does contain backslash-newlines, the array of tokens will contain one or more **TCL_TOKEN_TEXT** or **TCL_TOKEN_BS** sub-tokens that must be concatenated to produce the value of the string. If the braced string was just **{}** (that is, the string was empty), the single **TCL_TOKEN_TEXT** token will have a *size* field containing zero; this ensures that at least one token appears to describe the braced string. Only the token information in the Tcl_Parse structure is modified: the *commentStart*, *commentSize*, *commandStart*, and *commandSize* fields are not modified by **Tcl_ParseBraces**. After **Tcl_ParseQuotedString** returns, the array of tokens pointed to by the *tokenPtr* field of the Tcl_Parse structure depends on the contents of the quoted string. It will consist of one or more **TCL_TOKEN_TEXT**, **TCL_TOKEN_BS**, **TCL_TOKEN_COMMAND**, and **TCL_TOKEN_VARIABLE** sub-tokens. The array always contains at least one token; for example, if the argument *start* is empty, the array returned consists of a single **TCL_TOKEN_TEXT** token with a zero *size* field. Only the token information in the Tcl_Parse structure is modified: the *commentStart*, *commentSize*, *commandStart*, and *commandSize* fields are not modified. After **Tcl_ParseVarName** returns, the first token pointed to by the *tokenPtr* field of the Tcl_Parse structure always has type **TCL_TOKEN_VARIABLE**. It is followed by the sub-tokens that make up the variable name as described above. The total length of the variable name is contained in the *size* field of the first token. As in **Tcl_ParseExpr**, only the token information in the Tcl_Parse structure is modified by **Tcl_ParseVarName**: the *commentStart*, *commentSize*, *commandStart*, and *commandSize* fields are not modified. All of the character pointers in the Tcl_Parse and Tcl_Token structures refer to characters in the *start* argument passed to **Tcl_ParseCommand**, **Tcl_ParseExpr**, **Tcl_ParseBraces**, **Tcl_ParseQuotedString**, and **Tcl_ParseVarName**. There are additional fields in the Tcl_Parse structure after the *numTokens* field, but these are for the private use of **Tcl_ParseCommand**, **Tcl_ParseExpr**, **Tcl_ParseBraces**, **Tcl_ParseQuotedString**, and **Tcl_ParseVarName**; they should not be referenced by code outside of these procedures.

Keywords: ParseCmd, backslash substitution, braces, command, expression, parse, token, variable substitution

Copyright (c) 1997 Sun Microsystems, Inc.

PkgRequire

Name

Tcl_PkgRequire, Tcl_PkgRequireEx, Tcl_PkgRequireProc, Tcl_PkgPresent, Tcl_PkgPresentEx, Tcl_PkgProvide, Tcl_PkgProvideEx - package version control

Synopsis

#include <tcl.h>

const char *
Tcl_PkgRequire(*interp*, *name*, *version*, *exact*)

const char *
Tcl_PkgRequireEx(*interp*, *name*, *version*, *exact*, *clientDataPtr*)

int

Tcl_PkgRequireProc(*interp*, *name*, *objc*, *objv*, *clientDataPtr*)

const char *
Tcl_PkgPresent(*interp*, *name*, *version*, *exact*)

const char *
Tcl_PkgPresentEx(*interp*, *name*, *version*, *exact*, *clientDataPtr*)

int
Tcl_PkgProvide(*interp*, *name*, *version*)

int
Tcl_PkgProvideEx(*interp*, *name*, *version*, *clientData*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter where package is needed or available.

const char *_name_ (in)
Name of package.

const char *_version_ (in)
A version specification string as described for **package require**.

int _exact_ (in)
Non-zero means that only the particular version specified by *version* is acceptable. Zero means that newer versions than *version* are also acceptable as long as they have the same major version number as *version*.

const void *_clientData_ (in)
Arbitrary value to be associated with the package.

void *_clientDataPtr_ (out)
Pointer to place to store the value associated with the matching package. It is only changed if the pointer is not NULL and the function completed successfully. The storage can be any pointer type with the same size as a void pointer.

int _objc_ (in)
Number of requirements.

Tcl_Obj* _objv[]_ (in)
Array of requirements.

Description

These procedures provide C-level interfaces to Tcl's package and version management facilities.

Tcl_PkgRequire is equivalent to the **package require** command, **Tcl_PkgPresent** is equivalent to the **package present** command, and **Tcl_PkgProvide** is equivalent to the **package provide** command.

See the documentation for the Tcl commands for details on what these procedures do.

If **Tcl_PkgPresent** or **Tcl_PkgRequire** complete successfully they return a pointer to the version string for the version of the package that is provided in the interpreter (which may be different than *version*); if an error occurs they return NULL and leave an error message in the interpreter's result.

Tcl_PkgProvide returns **TCL_OK** if it completes successfully; if an error occurs it returns

TCL_ERROR and leaves an error message in the interpreter's result.

Tcl_PkgProvideEx, **Tcl_PkgPresentEx** and **Tcl_PkgRequireEx** allow the setting and retrieving of the client data associated with the package. In all other respects they are equivalent to the matching functions.

Tcl_PkgRequireProc is the form of **package require** handling multiple requirements. The other forms are present for backward compatibility and translate their invocations to this form.

Keywords: PkgRequire, package, present, provide, require, version

Copyright (c) 1996 Sun Microsystems, Inc.

Preserve

Name

Tcl_Preserve, Tcl_Release, Tcl_EventuallyFree - avoid freeing storage while it is being used

Synopsis

#include <tcl.h>

Tcl_Preserve(*clientData*)

Tcl_Release(*clientData*)

Tcl_EventuallyFree(*clientData*, *freeProc*)

Arguments

ClientData *clientData* (in)
Token describing structure to be freed or reallocated. Usually a pointer to memory for structure.

Tcl_FreeProc ****freeProc* (in)
Procedure to invoke to free *clientData*.

Description

These three procedures help implement a simple reference count mechanism for managing storage. They are designed to solve a problem having to do with widget deletion, but are also useful in many other situations. When a widget is deleted, its widget record (the structure holding information specific to the widget) must be returned to the storage allocator. However, it is possible that the widget record is in active use by one of the procedures on the stack at the time of the deletion. This can happen, for example, if the command associated with a button widget causes the button to be destroyed: an X event causes an event-handling C procedure in the button to be invoked, which in turn causes the button's associated Tcl command to be executed, which in turn causes the button to be deleted, which in turn causes the button's widget record to be de-allocated. Unfortunately, when the Tcl command returns, the button's event-handling procedure will need to reference the button's widget record. Because of this, the widget record must not be freed as part of the deletion, but must be retained until the event-handling procedure has finished with it. In other situations where the widget is deleted, it may be possible to free the widget record immediately.

Tcl_Preserve and **Tcl_Release** implement short-term reference counts for their *clientData* argument. The *clientData* argument identifies an object and usually consists of the address of a structure. The reference counts guarantee that an object will not be freed until each call to

Tcl_Preserve for the object has been matched by calls to **Tcl_Release**. There may be any number of unmatched **Tcl_Preserve** calls in effect at once.

Tcl_EventuallyFree is invoked to free up its *clientData* argument. It checks to see if there are unmatched **Tcl_Preserve** calls for the object. If not, then **Tcl_EventuallyFree** calls *freeProc* immediately. Otherwise **Tcl_EventuallyFree** records the fact that *clientData* needs eventually to be freed. When all calls to **Tcl_Preserve** have been matched with calls to **Tcl_Release** then *freeProc* will be called by **Tcl_Release** to do the cleanup.

All the work of freeing the object is carried out by *freeProc*. *FreeProc* must have arguments and result that match the type **Tcl_FreeProc**:

```
typedef void Tcl_FreeProc(
        char *blockPtr);
```

The *blockPtr* argument to *freeProc* will be the same as the *clientData* argument to **Tcl_EventuallyFree**. The type of *blockPtr* (**char ***) is different than the type of the *clientData* argument to **Tcl_EventuallyFree** for historical reasons, but the value is the same.

When the *clientData* argument to **Tcl_EventuallyFree** refers to storage allocated and returned by a prior call to **Tcl_Alloc**, **ckalloc**, or another function of the Tcl library, then the *freeProc* argument should be given the special value of **TCL_DYNAMIC**.

This mechanism can be used to solve the problem described above by placing **Tcl_Preserve** and **Tcl_Release** calls around actions that may cause undesired storage re-allocation. The mechanism is intended only for short-term use (i.e. while procedures are pending on the stack); it will not work efficiently as a mechanism for long-term reference counts. The implementation does not depend in any way on the internal structure of the objects being freed; it keeps the reference counts in a separate structure.

Keywords: Preserve, free, reference count, storage

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

PrintDbl

Name

Tcl_PrintDouble - Convert floating value to string

Synopsis

#include <tcl.h>

Tcl_PrintDouble(*interp*, *value*, *dst*)

Arguments

Tcl_Interp *_interp_ (in)
Before Tcl 8.0, the **tcl_precision** variable in this interpreter controlled the conversion. As of Tcl 8.0, this argument is ignored and the conversion is controlled by the **tcl_precision** variable that is now shared by all interpreters.

double _value_ (in)
Floating-point value to be converted.

char *_dst_ (out)

Where to store the string representing *value*. Must have at least **TCL_DOUBLE_SPACE** characters of storage.

Description

Tcl_PrintDouble generates a string that represents the value of *value* and stores it in memory at the location given by *dst*. It uses **%g** format to generate the string, with one special twist: the string is guaranteed to contain either a "." or an "e" so that it does not look like an integer. Where **%g** would generate an integer with no decimal point, **Tcl_PrintDouble** adds ".0".

If the **tcl_precision** value is non-zero, the result will have precisely that many digits of significance. If the value is zero (the default), the result will have the fewest digits needed to represent the number in such a way that **Tcl_NewDoubleObj** will generate the same number when presented with the given string. IEEE semantics of rounding to even apply to the conversion.

Keywords: PrintDbl, conversion, double-precision, floating-point, string

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

RecEvalObj

Name

Tcl_RecordAndEvalObj - save command on history list before evaluating

Synopsis

#include <tcl.h>

int
Tcl_RecordAndEvalObj(*interp*, *cmdPtr*, *flags*)

Arguments

Tcl_Interp *_interp_ (in)
Tcl interpreter in which to evaluate command.

Tcl_Obj *_cmdPtr_ (in)
Points to a Tcl value containing a command (or sequence of commands) to execute.

int _flags_ (in)
An OR'ed combination of flag bits. **TCL_NO_EVAL** means record the command but do not evaluate it. **TCL_EVAL_GLOBAL** means evaluate the command at global level instead of the current stack level.

Description

Tcl_RecordAndEvalObj is invoked to record a command as an event on the history list and then execute it using **Tcl_EvalObjEx** (or **Tcl_GlobalEvalObj** if the **TCL_EVAL_GLOBAL** bit is set in *flags*). It returns a completion code such as **TCL_OK** just like **Tcl_EvalObjEx**, as well as a result value containing additional information (a result value or error message) that can be retrieved using **Tcl_GetObjResult**. If you do not want the command recorded on the history list then you should invoke **Tcl_EvalObjEx** instead of **Tcl_RecordAndEvalObj**. Normally **Tcl_RecordAndEvalObj** is only called with top-level commands typed by the user, since the

purpose of history is to allow the user to re-issue recently invoked commands. If the *flags* argument contains the **TCL_NO_EVAL** bit then the command is recorded without being evaluated.

Keywords: RecEvalObj, command, event, execute, history, interpreter, value, record

Copyright (c) 1997 Sun Microsystems, Inc.

RecordEval

Name

Tcl_RecordAndEval - save command on history list before evaluating

Synopsis

#include <tcl.h>

int
Tcl_RecordAndEval(*interp*, *cmd*, *flags*)

Arguments

Tcl_Interp *_interp_ (in)
Tcl interpreter in which to evaluate command.

const char *_cmd_ (in)
Command (or sequence of commands) to execute.

int _flags_ (in)
An OR'ed combination of flag bits. **TCL_NO_EVAL** means record the command but do not evaluate it. **TCL_EVAL_GLOBAL** means evaluate the command at global level instead of the current stack level.

Description

Tcl_RecordAndEval is invoked to record a command as an event on the history list and then execute it using **Tcl_Eval** (or **Tcl_GlobalEval** if the **TCL_EVAL_GLOBAL** bit is set in *flags*). It returns a completion code such as **TCL_OK** just like **Tcl_Eval** and it leaves information in the interpreter's result. If you do not want the command recorded on the history list then you should invoke **Tcl_Eval** instead of **Tcl_RecordAndEval**. Normally **Tcl_RecordAndEval** is only called with top-level commands typed by the user, since the purpose of history is to allow the user to re-issue recently-invoked commands. If the *flags* argument contains the **TCL_NO_EVAL** bit then the command is recorded without being evaluated.

Note that **Tcl_RecordAndEval** has been largely replaced by the value-based procedure **Tcl_RecordAndEvalObj**. That value-based procedure records and optionally executes a command held in a Tcl value instead of a string.

Keywords: RecordEval, command, event, execute, history, interpreter, record

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

RegConfig

Name

Tcl_RegisterConfig - procedures to register embedded configuration information

Synopsis

#include <tcl.h>

void
Tcl_RegisterConfig(*interp*, *pkgName*, *configuration*, *valEncoding*)

Arguments

Tcl_Interp *_interp_ (in)
Refers to the interpreter the embedded configuration information is registered for. Must not be NULL.

const char *_pkgName_ (in)
Contains the name of the package registering the embedded configuration as ASCII string. This means that this information is in UTF-8 too. Must not be NULL.

const Tcl_Config *_configuration_ (in)
Refers to an array of Tcl_Config entries containing the information embedded in the binary library. Must not be NULL. The end of the array is signaled by either a key identical to NULL, or a key referring to the empty string.

const char *_valEncoding_ (in)
Contains the name of the encoding used to store the configuration values as ASCII string. This means that this information is in UTF-8 too. Must not be NULL.

Description

The function described here has its base in TIP 59 and provides extensions with support for the embedding of configuration information into their binary library and the generation of a Tcl-level interface for querying this information.

To embed configuration information into their binary library an extension has to define a non-volatile array of Tcl_Config entries in one if its source files and then call **Tcl_RegisterConfig** to register that information.

Tcl_RegisterConfig takes four arguments; first, a reference to the interpreter we are registering the information with, second, the name of the package registering its configuration information, third, a pointer to an array of structures, and fourth a string declaring the encoding used by the configuration values.

The string *valEncoding* contains the name of an encoding known to Tcl. All these names are use only characters in the ASCII subset of UTF-8 and are thus implicitly in the UTF-8 encoding. It is expected that keys are legible English text and therefore using the ASCII subset of UTF-8. In other words, they are expected to be in UTF-8 too. The values associated with the keys can be any string however. For these the contents of *valEncoding* define which encoding was used to represent the characters of the strings.

Each element of the *configuration* array refers to two strings containing the key and the value associated with that key. The end of the array is signaled by either an empty key or a key identical to NULL. The function makes **no** copy of the *configuration* array. This means that the caller has to make sure that the memory holding this array is never released. This is the meaning behind the word **non-volatile** used earlier. The easiest way to accomplish this is to define a global static array of Tcl_Config entries. See the file "generic/tclPkgConfig.c" in the

sources of the Tcl core for an example.

When called **Tcl_RegisterConfig** will

(1)
create a namespace having the provided *pkgName*, if not yet existing.

(2)
create the command **pkgconfig** in that namespace and link it to the provided information so that the keys from *configuration* and their associated values can be retrieved through calls to **pkgconfig**. The command **pkgconfig** will provide two subcommands, **list** and **get**:

::*pkgName*::pkgconfig list
Returns a list containing the names of all defined keys.

::*pkgName*::pkgconfig get *key*
Returns the configuration value associated with the specified *key*.

Tcl_config

The **Tcl_Config** structure contains the following fields:

```
typedef struct Tcl_Config {
    const char *key;
    const char *value;
} Tcl_Config;
```

Keywords: RegConfig, embedding, configuration, binary library

Copyright (c) 2002 Andreas Kupries andreas_kupries@users.sourceforge.net

RegExp

Name

Tcl_RegExpMatch, Tcl_RegExpCompile, Tcl_RegExpExec, Tcl_RegExpRange, Tcl_GetRegExpFromObj, Tcl_RegExpMatchObj, Tcl_RegExpExecObj, Tcl_RegExpGetInfo - Pattern matching with regular expressions

Synopsis

#include <tcl.h>

int
Tcl_RegExpMatchObj(*interp*, *textObj*, *patObj*)

int
Tcl_RegExpMatch(*interp*, *text*, *pattern*)

Tcl_RegExp
Tcl_RegExpCompile(*interp*, *pattern*)

int
Tcl_RegExpExec(*interp*, *regexp*, *text*, *start*)

void
Tcl_RegExpRange(*regexp*, *index*, *startPtr*, *endPtr*)

Tcl_RegExp
Tcl_GetRegExpFromObj(*interp*, *patObj*, *cflags*)

int
Tcl_RegExpExecObj(*interp*, *regexp*, *textObj*, *offset*, *nmatches*, *eflags*)

void
Tcl_RegExpGetInfo(*regexp*, *infoPtr*)

Arguments

Tcl_Interp *_interp_ (in)
Tcl interpreter to use for error reporting. The interpreter may be NULL if no error reporting is desired.

Tcl_Obj *_textObj_ (in/out)
Refers to the value from which to get the text to search. The internal representation of the value may be converted to a form that can be efficiently searched.

Tcl_Obj *_patObj_ (in/out)
Refers to the value from which to get a regular expression. The compiled regular expression is cached in the value.

const char *_text_ (in)
Text to search for a match with a regular expression.

const char *_pattern_ (in)
String in the form of a regular expression pattern.

Tcl_RegExp _regexp_ (in)
Compiled regular expression. Must have been returned previously by **Tcl_GetRegExpFromObj** or **Tcl_RegExpCompile**.

const char *_start_ (in)
If *text* is just a portion of some other string, this argument identifies the beginning of the larger string. If it is not the same as *text*, then no "^" matches will be allowed.

int _index_ (in)
Specifies which range is desired: 0 means the range of the entire match, 1 or greater means the range that matched a parenthesized sub-expression.

const char **_startPtr_ (out)
The address of the first character in the range is stored here, or NULL if there is no such range.

const char **_endPtr_ (out)
The address of the character just after the last one in the range is stored here, or NULL if there is no such range.

int _cflags_ (in)
OR-ed combination of the compilation flags **TCL_REG_ADVANCED**, **TCL_REG_EXTENDED**, **TCL_REG_BASIC**, **TCL_REG_EXPANDED**, **TCL_REG_QUOTE**, **TCL_REG_NOCASE**, **TCL_REG_NEWLINE**, **TCL_REG_NLSTOP**, **TCL_REG_NLANCH**, **TCL_REG_NOSUB**, and **TCL_REG_CANMATCH**. See below for more information.

int _offset_ (in)
The character offset into the text where matching should begin. The value of the offset has no impact on ^ matches. This behavior is controlled by *eflags*.

int _nmatches_ (in)
The number of matching subexpressions that should be remembered for later use. If this

value is 0, then no subexpression match information will be computed. If the value is -1, then all of the matching subexpressions will be remembered. Any other value will be taken as the maximum number of subexpressions to remember.

int *eflags* (in)
OR-ed combination of the execution flags **TCL_REG_NOTBOL** and **TCL_REG_NOTEOL**. See below for more information.

Tcl_RegExpInfo **infoPtr* (out)
The address of the location where information about a previous match should be stored by **Tcl_RegExpGetInfo**.

Description

Tcl_RegExpMatch determines whether its *pattern* argument matches *regexp*, where *regexp* is interpreted as a regular expression using the rules in the **re_syntax** reference page. If there is a match then **Tcl_RegExpMatch** returns 1. If there is no match then **Tcl_RegExpMatch** returns 0. If an error occurs in the matching process (e.g. *pattern* is not a valid regular expression) then **Tcl_RegExpMatch** returns -1 and leaves an error message in the interpreter result. **Tcl_RegExpMatchObj** is similar to **Tcl_RegExpMatch** except it operates on the Tcl values *textObj* and *patObj* instead of UTF strings. **Tcl_RegExpMatchObj** is generally more efficient than **Tcl_RegExpMatch**, so it is the preferred interface.

Tcl_RegExpCompile, **Tcl_RegExpExec**, and **Tcl_RegExpRange** provide lower-level access to the regular expression pattern matcher. **Tcl_RegExpCompile** compiles a regular expression string into the internal form used for efficient pattern matching. The return value is a token for this compiled form, which can be used in subsequent calls to **Tcl_RegExpExec** or **Tcl_RegExpRange**. If an error occurs while compiling the regular expression then **Tcl_RegExpCompile** returns NULL and leaves an error message in the interpreter result. Note: the return value from **Tcl_RegExpCompile** is only valid up to the next call to **Tcl_RegExpCompile**; it is not safe to retain these values for long periods of time.

Tcl_RegExpExec executes the regular expression pattern matcher. It returns 1 if *text* contains a range of characters that match *regexp*, 0 if no match is found, and -1 if an error occurs. In the case of an error, **Tcl_RegExpExec** leaves an error message in the interpreter result. When searching a string for multiple matches of a pattern, it is important to distinguish between the start of the original string and the start of the current search. For example, when searching for the second occurrence of a match, the *text* argument might point to the character just after the first match; however, it is important for the pattern matcher to know that this is not the start of the entire string, so that it does not allow "^" atoms in the pattern to match. The *start* argument provides this information by pointing to the start of the overall string containing *text*. *Start* will be less than or equal to *text*; if it is less than *text* then no ^ matches will be allowed.

Tcl_RegExpRange may be invoked after **Tcl_RegExpExec** returns; it provides detailed information about what ranges of the string matched what parts of the pattern. **Tcl_RegExpRange** returns a pair of pointers in **startPtr* and **endPtr* that identify a range of characters in the source string for the most recent call to **Tcl_RegExpExec**. *Index* indicates which of several ranges is desired: if *index* is 0, information is returned about the overall range of characters that matched the entire pattern; otherwise, information is returned about the range of characters that matched the *index*'th parenthesized subexpression within the pattern. If there is no range corresponding to *index* then NULL is stored in **startPtr* and **endPtr*.

Tcl_GetRegExpFromObj, **Tcl_RegExpExecObj**, and **Tcl_RegExpGetInfo** are value interfaces that provide the most direct control of Henry Spencer's regular expression library. For users that need to modify compilation and execution options directly, it is recommended that you use these interfaces instead of calling the internal regexp functions. These interfaces handle the details of UTF to Unicode translations as well as providing improved performance through caching in the pattern and string values.

Tcl_GetRegExpFromObj attempts to return a compiled regular expression from the *patObj*. If the value does not already contain a compiled regular expression it will attempt to create one from the string in the value and assign it to the internal representation of the *patObj*. The return value of this function is of type **Tcl_RegExp**. The return value is a token for this compiled form, which can be used in subsequent calls to **Tcl_RegExpExecObj** or **Tcl_RegExpGetInfo**. If an error occurs while compiling the regular expression then **Tcl_GetRegExpFromObj** returns NULL and leaves an error message in the interpreter result. The regular expression token can be used as long as the internal representation of *patObj* refers to the compiled form. The *cflags* argument is a bit-wise OR of zero or more of the following flags that control the compilation of *patObj*:

TCL_REG_ADVANCED
: Compile advanced regular expressions ("ARE"s). This mode corresponds to the normal regular expression syntax accepted by the Tcl **regexp** and **regsub** commands.

TCL_REG_EXTENDED
: Compile extended regular expressions ("ERE"s). This mode corresponds to the regular expression syntax recognized by Tcl 8.0 and earlier versions.

TCL_REG_BASIC
: Compile basic regular expressions ("BRE"s). This mode corresponds to the regular expression syntax recognized by common Unix utilities like **sed** and **grep**. This is the default if no flags are specified.

TCL_REG_EXPANDED
: Compile the regular expression (basic, extended, or advanced) using an expanded syntax that allows comments and whitespace. This mode causes non-backslashed non-bracket-expression white space and #-to-end-of-line comments to be ignored.

TCL_REG_QUOTE
: Compile a literal string, with all characters treated as ordinary characters.

TCL_REG_NOCASE
: Compile for matching that ignores upper/lower case distinctions.

TCL_REG_NEWLINE
: Compile for newline-sensitive matching. By default, newline is a completely ordinary character with no special meaning in either regular expressions or strings. With this flag, "[^" bracket expressions and "." never match newline, "^" matches an empty string after any newline in addition to its normal function, and "$" matches an empty string before any newline in addition to its normal function. **REG_NEWLINE** is the bit-wise OR of **REG_NLSTOP** and **REG_NLANCH**.

TCL_REG_NLSTOP
: Compile for partial newline-sensitive matching, with the behavior of "[^" bracket expressions and "." affected, but not the behavior of "^" and "$". In this mode, "[^" bracket expressions and "." never match newline.

TCL_REG_NLANCH

Compile for inverse partial newline-sensitive matching, with the behavior of "^" and "$" (the "anchors") affected, but not the behavior of "[^" bracket expressions and ".". In this mode "^" matches an empty string after any newline in addition to its normal function, and "$" matches an empty string before any newline in addition to its normal function.

TCL_REG_NOSUB
Compile for matching that reports only success or failure, not what was matched. This reduces compile overhead and may improve performance. Subsequent calls to **Tcl_RegExpGetInfo** or **Tcl_RegExpRange** will not report any match information.

TCL_REG_CANMATCH
Compile for matching that reports the potential to complete a partial match given more text (see below).

Only one of **TCL_REG_EXTENDED**, **TCL_REG_ADVANCED**, **TCL_REG_BASIC**, and **TCL_REG_QUOTE** may be specified.

Tcl_RegExpExecObj executes the regular expression pattern matcher. It returns 1 if *objPtr* contains a range of characters that match *regexp*, 0 if no match is found, and -1 if an error occurs. In the case of an error, **Tcl_RegExpExecObj** leaves an error message in the interpreter result. The *nmatches* value indicates to the matcher how many subexpressions are of interest. If *nmatches* is 0, then no subexpression match information is recorded, which may allow the matcher to make various optimizations. If the value is -1, then all of the subexpressions in the pattern are remembered. If the value is a positive integer, then only that number of subexpressions will be remembered. Matching begins at the specified Unicode character index given by *offset*. Unlike **Tcl_RegExpExec**, the behavior of anchors is not affected by the offset value. Instead the behavior of the anchors is explicitly controlled by the *eflags* argument, which is a bit-wise OR of zero or more of the following flags:

TCL_REG_NOTBOL
The starting character will not be treated as the beginning of a line or the beginning of the string, so "^" will not match there. Note that this flag has no effect on how "**\A**" matches.

TCL_REG_NOTEOL
The last character in the string will not be treated as the end of a line or the end of the string, so "$" will not match there. Note that this flag has no effect on how "**\Z**" matches.

Tcl_RegExpGetInfo retrieves information about the last match performed with a given regular expression *regexp*. The *infoPtr* argument contains a pointer to a structure that is defined as follows:

```
typedef struct Tcl_RegExpInfo {
    int nsubs;
    Tcl_RegExpIndices *matches;
    long extendStart;
} Tcl_RegExpInfo;
```

The *nsubs* field contains a count of the number of parenthesized subexpressions within the regular expression. If the **TCL_REG_NOSUB** was used, then this value will be zero. The *matches* field points to an array of *nsubs*+1 values that indicate the bounds of each subexpression matched. The first element in the array refers to the range matched by the entire regular expression, and subsequent elements refer to the parenthesized subexpressions in the order

that they appear in the pattern. Each element is a structure that is defined as follows:

```
typedef struct Tcl_RegExpIndices {
    long start;
    long end;
} Tcl_RegExpIndices;
```

The *start* and *end* values are Unicode character indices relative to the offset location within *objPtr* where matching began. The *start* index identifies the first character of the matched subexpression. The *end* index identifies the first character after the matched subexpression. If the subexpression matched the empty string, then *start* and *end* will be equal. If the subexpression did not participate in the match, then *start* and *end* will be set to -1.

The *extendStart* field in **Tcl_RegExpInfo** is only set if the **TCL_REG_CANMATCH** flag was used. It indicates the first character in the string where a match could occur. If a match was found, this will be the same as the beginning of the current match. If no match was found, then it indicates the earliest point at which a match might occur if additional text is appended to the string. If it is no match is possible even with further text, this field will be set to -1.

Keywords: RegExp, match, pattern, regular expression, string, subexpression, Tcl_RegExpIndices, Tcl_RegExpInfo

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 1998-1999 Scriptics Corporation

SaveResult

Name

Tcl_SaveInterpState, Tcl_RestoreInterpState, Tcl_DiscardInterpState, Tcl_SaveResult, Tcl_RestoreResult, Tcl_DiscardResult - save and restore an interpreter's state

Synopsis

#include <tcl.h>

Tcl_InterpState
Tcl_SaveInterpState(*interp*, *status*)

int
Tcl_RestoreInterpState(*interp*, *state*)

Tcl_DiscardInterpState(*state*)

Tcl_SaveResult(*interp*, *savedPtr*)

Tcl_RestoreResult(*interp*, *savedPtr*)

Tcl_DiscardResult(*savedPtr*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter for which state should be saved.

int _status_ (in)
Return code value to save as part of interpreter state.

Tcl_InterpState *state* (in)
Saved state token to be restored or discarded.

Tcl_SavedResult *savedPtr (in)
Pointer to location where interpreter result should be saved or restored.

Description

These routines allows a C procedure to take a snapshot of the current state of an interpreter so that it can be restored after a call to **Tcl_Eval** or some other routine that modifies the interpreter state. There are two triplets of routines meant to work together.

The first triplet stores the snapshot of interpreter state in an opaque token returned by **Tcl_SaveInterpState**. That token value may then be passed back to one of **Tcl_RestoreInterpState** or **Tcl_DiscardInterpState**, depending on whether the interp state is to be restored. So long as one of the latter two routines is called, Tcl will take care of memory management.

The second triplet stores the snapshot of only the interpreter result (not its complete state) in memory allocated by the caller. These routines are passed a pointer to **Tcl_SavedResult** that is used to store enough information to restore the interpreter result. **Tcl_SavedResult** can be allocated on the stack of the calling procedure. These routines do not save the state of any error information in the interpreter (e.g. the **-errorcode** or **-errorinfo** return options, when an error is in progress).

Because the routines **Tcl_SaveInterpState**, **Tcl_RestoreInterpState**, and **Tcl_DiscardInterpState** perform a superset of the functions provided by the other routines, any new code should only make use of the more powerful routines. The older, weaker routines **Tcl_SaveResult**, **Tcl_RestoreResult**, and **Tcl_DiscardResult** continue to exist only for the sake of existing programs that may already be using them.

Tcl_SaveInterpState takes a snapshot of those portions of interpreter state that make up the full result of script evaluation. This include the interpreter result, the return code (passed in as the *status* argument, and any return options, including **-errorinfo** and **-errorcode** when an error is in progress. This snapshot is returned as an opaque token of type **Tcl_InterpState**. The call to **Tcl_SaveInterpState** does not itself change the state of the interpreter. Unlike **Tcl_SaveResult**, it does not reset the interpreter.

Tcl_RestoreInterpState accepts a **Tcl_InterpState** token previously returned by **Tcl_SaveInterpState** and restores the state of the interp to the state held in that snapshot. The return value of **Tcl_RestoreInterpState** is the status value originally passed to **Tcl_SaveInterpState** when the snapshot token was created.

Tcl_DiscardInterpState is called to release a **Tcl_InterpState** token previously returned by **Tcl_SaveInterpState** when that snapshot is not to be restored to an interp.

The **Tcl_InterpState** token returned by **Tcl_SaveInterpState** must eventually be passed to either **Tcl_RestoreInterpState** or **Tcl_DiscardInterpState** to avoid a memory leak. Once the **Tcl_InterpState** token is passed to one of them, the token is no longer valid and should not be used anymore.

Tcl_SaveResult moves the string and value results of *interp* into the location specified by *statePtr*. **Tcl_SaveResult** clears the result for *interp* and leaves the result in its normal empty initialized state.

Tcl_RestoreResult moves the string and value results from *statePtr* back into *interp*. Any result or error that was already in the interpreter will be cleared. The *statePtr* is left in an uninitialized state and cannot be used until another call to **Tcl_SaveResult**.

Tcl_DiscardResult releases the saved interpreter state stored at **statePtr**. The state structure is left in an uninitialized state and cannot be used until another call to **Tcl_SaveResult**.

Once **Tcl_SaveResult** is called to save the interpreter result, either **Tcl_RestoreResult** or **Tcl_DiscardResult** must be called to properly clean up the memory associated with the saved state.

Keywords: SaveResult, result, state, interp

Copyright (c) 1997 Sun Microsystems, Inc.
Contributions from Don Porter, NIST, 2004. (not subject to US copyright)

SetChanErr

Name

Tcl_SetChannelError, Tcl_SetChannelErrorInterp, Tcl_GetChannelError, Tcl_GetChannelErrorInterp - functions to create/intercept Tcl errors by channel drivers.

Synopsis

#include <tcl.h>

void
Tcl_SetChannelError(*chan*, *msg*)

void
Tcl_SetChannelErrorInterp(*interp*, *msg*)

void
Tcl_GetChannelError(*chan*, *msgPtr*)

void
Tcl_GetChannelErrorInterp(*interp*, *msgPtr*)

Arguments

Tcl_Channel *chan* (in)
Refers to the Tcl channel whose bypass area is accessed.

Tcl_Interp* *interp* (in)
Refers to the Tcl interpreter whose bypass area is accessed.

Tcl_Obj* *msg* (in)
Error message put into a bypass area. A list of return options and values, followed by a string message. Both message and the option/value information are optional.

Tcl_Obj *msgPtr* (out)**
Reference to a place where the message stored in the accessed bypass area can be stored in.

Description

The current definition of a Tcl channel driver does not permit the direct return of arbitrary error messages, except for the setting and retrieval of channel options. All other functions are restricted to POSIX error codes.

The functions described here overcome this limitation. Channel drivers are allowed to use **Tcl_SetChannelError** and **Tcl_SetChannelErrorInterp** to place arbitrary error messages in **bypass areas** defined for channels and interpreters. And the generic I/O layer uses **Tcl_GetChannelError** and **Tcl_GetChannelErrorInterp** to look for messages in the bypass areas and arrange for their return as errors. The POSIX error codes set by a driver are used now if and only if no messages are present.

Tcl_SetChannelError stores error information in the bypass area of the specified channel. The number of references to the **msg** value goes up by one. Previously stored information will be discarded, by releasing the reference held by the channel. The channel reference must not be NULL.

Tcl_SetChannelErrorInterp stores error information in the bypass area of the specified interpreter. The number of references to the **msg** value goes up by one. Previously stored information will be discarded, by releasing the reference held by the interpreter. The interpreter reference must not be NULL.

Tcl_GetChannelError places either the error message held in the bypass area of the specified channel into *msgPtr*, or NULL; and resets the bypass, that is, after an invocation all following invocations will return NULL, until an intervening invocation of **Tcl_SetChannelError** with a non-NULL message. The *msgPtr* must not be NULL. The reference count of the message is not touched. The reference previously held by the channel is now held by the caller of the function and it is its responsibility to release that reference when it is done with the value.

Tcl_GetChannelErrorInterp places either the error message held in the bypass area of the specified interpreter into *msgPtr*, or NULL; and resets the bypass, that is, after an invocation all following invocations will return NULL, until an intervening invocation of **Tcl_SetChannelErrorInterp** with a non-NULL message. The *msgPtr* must not be NULL. The reference count of the message is not touched. The reference previously held by the interpreter is now held by the caller of the function and it is its responsibility to release that reference when it is done with the value.

Which functions of a channel driver are allowed to use which bypass function is listed below, as is which functions of the public channel API may leave a messages in the bypass areas.

Tcl_DriverCloseProc
: May use **Tcl_SetChannelErrorInterp**, and only this function.

Tcl_DriverInputProc
: May use **Tcl_SetChannelError**, and only this function.

Tcl_DriverOutputProc
: May use **Tcl_SetChannelError**, and only this function.

Tcl_DriverSeekProc
: May use **Tcl_SetChannelError**, and only this function.

Tcl_DriverWideSeekProc
: May use **Tcl_SetChannelError**, and only this function.

Tcl_DriverSetOptionProc
: Has already the ability to pass arbitrary error messages. Must *not* use any of the new functions.

Tcl_DriverGetOptionProc
: Has already the ability to pass arbitrary error messages. Must *not* use any of the new

functions.

Tcl_DriverWatchProc
Must *not* use any of the new functions. Is internally called and has no ability to return any type of error whatsoever.

Tcl_DriverBlockModeProc
May use **Tcl_SetChannelError**, and only this function.

Tcl_DriverGetHandleProc
Must *not* use any of the new functions. It is only a low-level function, and not used by Tcl commands.

Tcl_DriverHandlerProc
Must *not* use any of the new functions. Is internally called and has no ability to return any type of error whatsoever. Given the information above the following public functions of the Tcl C API are affected by these changes; when these functions are called, the channel may now contain a stored arbitrary error message requiring processing by the caller.

```
Tcl_Flush    Tcl_GetsObj    Tcl_Gets
Tcl_ReadChars Tcl_ReadRaw   Tcl_Read
Tcl_Seek     Tcl_StackChannel Tcl_Tell
Tcl_WriteChars Tcl_WriteObj Tcl_WriteRaw
Tcl_Write
```

All other API functions are unchanged. In particular, the functions below leave all their error information in the interpreter result.

```
Tcl_Close    Tcl_UnstackChannel Tcl_UnregisterChannel
```

Keywords: SetChanErr, channel driver, error messages, channel type

Copyright (c) 2005 Andreas Kupries andreas_kupries@users.sourceforge.net

SetErrno

Name

Tcl_SetErrno, Tcl_GetErrno, Tcl_ErrnoId, Tcl_ErrnoMsg - manipulate errno to store and retrieve error codes

Synopsis

#include <tcl.h>

void
Tcl_SetErrno(*errorCode*)

int
Tcl_GetErrno()

const char *
Tcl_ErrnoId()

const char *
Tcl_ErrnoMsg(*errorCode*)

Arguments

int *errorCode* (in)
A POSIX error code such as **ENOENT**.

Description

Tcl_SetErrno and **Tcl_GetErrno** provide portable access to the **errno** variable, which is used to record a POSIX error code after system calls and other operations such as **Tcl_Gets**. These procedures are necessary because global variable accesses cannot be made across module boundaries on some platforms.

Tcl_SetErrno sets the **errno** variable to the value of the *errorCode* argument C procedures that wish to return error information to their callers via **errno** should call **Tcl_SetErrno** rather than setting **errno** directly.

Tcl_GetErrno returns the current value of **errno**. Procedures wishing to access **errno** should call this procedure instead of accessing **errno** directly.

Tcl_ErrnoId and **Tcl_ErrnoMsg** return string representations of **errno** values. **Tcl_ErrnoId** returns a machine-readable textual identifier such as "EACCES" that corresponds to the current value of **errno**. **Tcl_ErrnoMsg** returns a human-readable string such as "permission denied" that corresponds to the value of its *errorCode* argument. The *errorCode* argument is typically the value returned by **Tcl_GetErrno**. The strings returned by these functions are statically allocated and the caller must not free or modify them.

Keywords: SetErrno, errno, error code, global variables

Copyright (c) 1996 Sun Microsystems, Inc.

SetRecLmt

Name

Tcl_SetRecursionLimit - set maximum allowable nesting depth in interpreter

Synopsis

#include <tcl.h>

int
Tcl_SetRecursionLimit(*interp*, *depth*)

Arguments

Tcl_Interp **interp* (in)
Interpreter whose recursion limit is to be set. Must be greater than zero.

int *depth* (in)
New limit for nested calls to **Tcl_Eval** for *interp*.

Description

At any given time Tcl enforces a limit on the number of recursive calls that may be active for **Tcl_Eval** and related procedures such as **Tcl_GlobalEval**. Any call to **Tcl_Eval** that exceeds this depth is aborted with an error. By default the recursion limit is 1000.

Tcl_SetRecursionLimit may be used to change the maximum allowable nesting depth for an interpreter. The *depth* argument specifies a new limit for *interp*, and **Tcl_SetRecursionLimit**

returns the old limit. To read out the old limit without modifying it, invoke **Tcl_SetRecursionLimit** with *depth* equal to 0.

The **Tcl_SetRecursionLimit** only sets the size of the Tcl call stack: it cannot by itself prevent stack overflows on the C stack being used by the application. If your machine has a limit on the size of the C stack, you may get stack overflows before reaching the limit set by **Tcl_SetRecursionLimit**. If this happens, see if there is a mechanism in your system for increasing the maximum size of the C stack.

Keywords: SetRecLmt, nesting depth, recursion

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

SetResult

Name

Tcl_SetObjResult, Tcl_GetObjResult, Tcl_SetResult, Tcl_GetStringResult, Tcl_AppendResult, Tcl_AppendResultVA, Tcl_AppendElement, Tcl_ResetResult, Tcl_TransferResult, Tcl_FreeResult - manipulate Tcl result

Synopsis

#include <tcl.h>

Tcl_SetObjResult(*interp*, *objPtr*)

Tcl_Obj *
Tcl_GetObjResult(*interp*)

Tcl_SetResult(*interp*, *result*, *freeProc*)

const char *
Tcl_GetStringResult(*interp*)

Tcl_AppendResult(*interp*, *result*, *result*, ..., (**char** *)**NULL**)

Tcl_AppendResultVA(*interp*, *argList*)

Tcl_ResetResult(*interp*)

Tcl_TransferResult(*sourceInterp*, *code*, *targetInterp*)

Tcl_AppendElement(*interp*, *element*)

Tcl_FreeResult(*interp*)

Arguments

Tcl_Interp *_interp_ (out)
Interpreter whose result is to be modified or read.

Tcl_Obj *_objPtr_ (in)
Tcl value to become result for *interp*.

char *_result_ (in)
String value to become result for *interp* or to be appended to the existing result.

const char *_element_ (in)
String value to append as a list element to the existing result of *interp*.

Tcl_FreeProc *_freeProc_ (in)

Address of procedure to call to release storage at *result*, or **TCL_STATIC**, **TCL_DYNAMIC**, or **TCL_VOLATILE**.

va_list *argList* (in)
An argument list which must have been initialized using **va_start**, and cleared using **va_end**.

Tcl_Interp *sourceInterp (in)
Interpreter that the result and return options should be transferred from.

Tcl_Interp *targetInterp (in)
Interpreter that the result and return options should be transferred to.

int *code* (in)
Return code value that controls transfer of return options.

Description

The procedures described here are utilities for manipulating the result value in a Tcl interpreter. The interpreter result may be either a Tcl value or a string. For example, **Tcl_SetObjResult** and **Tcl_SetResult** set the interpreter result to, respectively, a value and a string. Similarly, **Tcl_GetObjResult** and **Tcl_GetStringResult** return the interpreter result as a value and as a string. The procedures always keep the string and value forms of the interpreter result consistent. For example, if **Tcl_SetObjResult** is called to set the result to a value, then **Tcl_GetStringResult** is called, it will return the value's string representation.

Tcl_SetObjResult arranges for *objPtr* to be the result for *interp*, replacing any existing result. The result is left pointing to the value referenced by *objPtr*. *objPtr*'s reference count is incremented since there is now a new reference to it from *interp*. The reference count for any old result value is decremented and the old result value is freed if no references to it remain.

Tcl_GetObjResult returns the result for *interp* as a value. The value's reference count is not incremented; if the caller needs to retain a long-term pointer to the value they should use **Tcl_IncrRefCount** to increment its reference count in order to keep it from being freed too early or accidentally changed.

Tcl_SetResult arranges for *result* to be the result for the current Tcl command in *interp*, replacing any existing result. The *freeProc* argument specifies how to manage the storage for the *result* argument; it is discussed in the section **THE TCL_FREEPROC ARGUMENT TO TCL_SETRESULT** below. If *result* is **NULL**, then *freeProc* is ignored and **Tcl_SetResult** re-initializes *interp*'s result to point to an empty string.

Tcl_GetStringResult returns the result for *interp* as a string. If the result was set to a value by a **Tcl_SetObjResult** call, the value form will be converted to a string and returned. If the value's string representation contains null bytes, this conversion will lose information. For this reason, programmers are encouraged to write their code to use the new value API procedures and to call **Tcl_GetObjResult** instead.

Tcl_ResetResult clears the result for *interp* and leaves the result in its normal empty initialized state. If the result is a value, its reference count is decremented and the result is left pointing to an unshared value representing an empty string. If the result is a dynamically allocated string, its memory is free*d and the result is left as a empty string. **Tcl_ResetResult** also clears the error state managed by **Tcl_AddErrorInfo**, **Tcl_AddObjErrorInfo**, and **Tcl_SetErrorCode**.

Tcl_AppendResult makes it easy to build up Tcl results in pieces. It takes each of its *result* arguments and appends them in order to the current result associated with *interp*. If the result is

in its initialized empty state (e.g. a command procedure was just invoked or **Tcl_ResetResult** was just called), then **Tcl_AppendResult** sets the result to the concatenation of its *result* arguments. **Tcl_AppendResult** may be called repeatedly as additional pieces of the result are produced. **Tcl_AppendResult** takes care of all the storage management issues associated with managing *interp*'s result, such as allocating a larger result area if necessary. It also manages conversion to and from the *result* field of the *interp* so as to handle backward-compatibility with old-style extensions. Any number of *result* arguments may be passed in a single call; the last argument in the list must be (char *)NULL.

Tcl_AppendResultVA is the same as **Tcl_AppendResult** except that instead of taking a variable number of arguments it takes an argument list.

Tcl_TransferResult transfers interpreter state from *sourceInterp* to *targetInterp*. The two interpreters must have been created in the same thread. If *sourceInterp* and *targetInterp* are the same, nothing is done. Otherwise, **Tcl_TransferResult** moves the result from *sourceInterp* to *targetInterp*, and resets the result in *sourceInterp*. It also moves the return options dictionary as controlled by the return code value *code* in the same manner as **Tcl_GetReturnOptions**.

Deprecated interfaces

Old string procedures

Use of the following procedures is deprecated since they manipulate the Tcl result as a string. Procedures such as **Tcl_SetObjResult** that manipulate the result as a value can be significantly more efficient.

Tcl_AppendElement is similar to **Tcl_AppendResult** in that it allows results to be built up in pieces. However, **Tcl_AppendElement** takes only a single *element* argument and it appends that argument to the current result as a proper Tcl list element. **Tcl_AppendElement** adds backslashes or braces if necessary to ensure that *interp*'s result can be parsed as a list and that *element* will be extracted as a single element. Under normal conditions, **Tcl_AppendElement** will add a space character to *interp*'s result just before adding the new list element, so that the list elements in the result are properly separated. However if the new list element is the first in a list or sub-list (i.e. *interp*'s current result is empty, or consists of the single character "{", or ends in the characters " {") then no space is added.

Tcl_FreeResult performs part of the work of **Tcl_ResetResult**. It frees up the memory associated with *interp*'s result. It also sets *interp->freeProc* to zero, but does not change *interp->result* or clear error state. **Tcl_FreeResult** is most commonly used when a procedure is about to replace one result value with another.

Direct access to interp->result

It used to be legal for programs to directly read and write *interp->result* to manipulate the interpreter result. The Tcl headers no longer permit this access by default, and C code still doing this must be updated to use supported routines **Tcl_GetObjResult**, **Tcl_GetStringResult**, **Tcl_SetObjResult**, and **Tcl_SetResult**. As a migration aid, access can be restored with the compiler directive

```
#define USE_INTERP_RESULT
```

but this is meant only to offer life support to otherwise dead code.

The tcl_freeproc argument to tcl_setresult

Tcl_SetResult's *freeProc* argument specifies how the Tcl system is to manage the storage for the *result* argument. If **Tcl_SetResult** or **Tcl_SetObjResult** are called at a time when *interp* holds a string result, they do whatever is necessary to dispose of the old string result (see the **Tcl_Interp** manual entry for details on this).

If *freeProc* is **TCL_STATIC** it means that *result* refers to an area of static storage that is guaranteed not to be modified until at least the next call to **Tcl_Eval**. If *freeProc* is **TCL_DYNAMIC** it means that *result* was allocated with a call to **Tcl_Alloc** and is now the property of the Tcl system. **Tcl_SetResult** will arrange for the string's storage to be released by calling **Tcl_Free** when it is no longer needed. If *freeProc* is **TCL_VOLATILE** it means that *result* points to an area of memory that is likely to be overwritten when **Tcl_SetResult** returns (e.g. it points to something in a stack frame). In this case **Tcl_SetResult** will make a copy of the string in dynamically allocated storage and arrange for the copy to be the result for the current Tcl command.

If *freeProc* is not one of the values **TCL_STATIC**, **TCL_DYNAMIC**, and **TCL_VOLATILE**, then it is the address of a procedure that Tcl should call to free the string. This allows applications to use non-standard storage allocators. When Tcl no longer needs the storage for the string, it will call *freeProc*. *FreeProc* should have arguments and result that match the type **Tcl_FreeProc**:

```
typedef void Tcl_FreeProc(
        char *blockPtr);
```

When *freeProc* is called, its *blockPtr* will be set to the value of *result* passed to **Tcl_SetResult**.

Keywords: SetResult, append, command, element, list, value, result, return value, interpreter

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

SetVar

Name

Tcl_SetVar2Ex, Tcl_SetVar, Tcl_SetVar2, Tcl_ObjSetVar2, Tcl_GetVar2Ex, Tcl_GetVar, Tcl_GetVar2, Tcl_ObjGetVar2, Tcl_UnsetVar, Tcl_UnsetVar2 - manipulate Tcl variables

Synopsis

#include <tcl.h>

Tcl_Obj *
Tcl_SetVar2Ex(*interp*, *name1*, *name2*, *newValuePtr*, *flags*)

const char *
Tcl_SetVar(*interp*, *varName*, *newValue*, *flags*)

const char *
Tcl_SetVar2(*interp*, *name1*, *name2*, *newValue*, *flags*)

Tcl_Obj *
Tcl_ObjSetVar2(*interp*, *part1Ptr*, *part2Ptr*, *newValuePtr*, *flags*)

Tcl_Obj *
Tcl_GetVar2Ex(*interp*, *name1*, *name2*, *flags*)

const char *
Tcl_GetVar(*interp*, *varName*, *flags*)

const char *
Tcl_GetVar2(*interp*, *name1*, *name2*, *flags*)

Tcl_Obj *
Tcl_ObjGetVar2(*interp*, *part1Ptr*, *part2Ptr*, *flags*)

int
Tcl_UnsetVar(*interp*, *varName*, *flags*)

int
Tcl_UnsetVar2(*interp*, *name1*, *name2*, *flags*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter containing variable.

const char *_name1_ (in)
Contains the name of an array variable (if *name2* is non-NULL) or (if *name2* is NULL) either the name of a scalar variable or a complete name including both variable name and index. May include **::** namespace qualifiers to specify a variable in a particular namespace.

const char *_name2_ (in)
If non-NULL, gives name of element within array; in this case *name1* must refer to an array variable.

Tcl_Obj *_newValuePtr_ (in)
Points to a Tcl value containing the new value for the variable.

int _flags_ (in)
OR-ed combination of bits providing additional information. See below for valid values.

const char *_varName_ (in)
Name of variable. May include **::** namespace qualifiers to specify a variable in a particular namespace. May refer to a scalar variable or an element of an array.

const char *_newValue_ (in)
New value for variable, specified as a null-terminated string. A copy of this value is stored in the variable.

Tcl_Obj *_part1Ptr_ (in)
Points to a Tcl value containing the variable's name. The name may include a series of **::** namespace qualifiers to specify a variable in a particular namespace. May refer to a scalar variable or an element of an array variable.

Tcl_Obj *_part2Ptr_ (in)
If non-NULL, points to a value containing the name of an element within an array and *part1Ptr* must refer to an array variable.

Description

These procedures are used to create, modify, read, and delete Tcl variables from C code.

Tcl_SetVar2Ex, **Tcl_SetVar**, **Tcl_SetVar2**, and **Tcl_ObjSetVar2** will create a new variable or modify an existing one. These procedures set the given variable to the value given by *newValuePtr* or *newValue* and return a pointer to the variable's new value, which is stored in Tcl's

variable structure. **Tcl_SetVar2Ex** and **Tcl_ObjSetVar2** take the new value as a Tcl_Obj and return a pointer to a Tcl_Obj. **Tcl_SetVar** and **Tcl_SetVar2** take the new value as a string and return a string; they are usually less efficient than **Tcl_ObjSetVar2**. Note that the return value may be different than the *newValuePtr* or *newValue* argument, due to modifications made by write traces. If an error occurs in setting the variable (e.g. an array variable is referenced without giving an index into the array) NULL is returned and an error message is left in *interp*'s result if the **TCL_LEAVE_ERR_MSG** *flag* bit is set.

Tcl_GetVar2Ex, **Tcl_GetVar**, **Tcl_GetVar2**, and **Tcl_ObjGetVar2** return the current value of a variable. The arguments to these procedures are treated in the same way as the arguments to the procedures described above. Under normal circumstances, the return value is a pointer to the variable's value. For **Tcl_GetVar2Ex** and **Tcl_ObjGetVar2** the value is returned as a pointer to a Tcl_Obj. For **Tcl_GetVar** and **Tcl_GetVar2** the value is returned as a string; this is usually less efficient, so **Tcl_GetVar2Ex** or **Tcl_ObjGetVar2** are preferred. If an error occurs while reading the variable (e.g. the variable does not exist or an array element is specified for a scalar variable), then NULL is returned and an error message is left in *interp*'s result if the **TCL_LEAVE_ERR_MSG** *flag* bit is set.

Tcl_UnsetVar and **Tcl_UnsetVar2** may be used to remove a variable, so that future attempts to read the variable will return an error. The arguments to these procedures are treated in the same way as the arguments to the procedures above. If the variable is successfully removed then **TCL_OK** is returned. If the variable cannot be removed because it does not exist then **TCL_ERROR** is returned and an error message is left in *interp*'s result if the **TCL_LEAVE_ERR_MSG** *flag* bit is set. If an array element is specified, the given element is removed but the array remains. If an array name is specified without an index, then the entire array is removed.

The name of a variable may be specified to these procedures in four ways:

[1]
If **Tcl_SetVar**, **Tcl_GetVar**, or **Tcl_UnsetVar** is invoked, the variable name is given as a single string, *varName*. If *varName* contains an open parenthesis and ends with a close parenthesis, then the value between the parentheses is treated as an index (which can have any string value) and the characters before the first open parenthesis are treated as the name of an array variable. If *varName* does not have parentheses as described above, then the entire string is treated as the name of a scalar variable.

[2]
If the *name1* and *name2* arguments are provided and *name2* is non-NULL, then an array element is specified and the array name and index have already been separated by the caller: *name1* contains the name and *name2* contains the index. An error is generated if *name1* contains an open parenthesis and ends with a close parenthesis (array element) and *name2* is non-NULL.

[3]
If *name2* is NULL, *name1* is treated just like *varName* in case [1] above (it can be either a scalar or an array element variable name). The *flags* argument may be used to specify any of several options to the procedures. It consists of an OR-ed combination of the following bits.

TCL_GLOBAL_ONLY
Under normal circumstances the procedures look up variables as follows. If a procedure call is

active in *interp*, the variable is looked up at the current level of procedure call. Otherwise, the variable is looked up first in the current namespace, then in the global namespace. However, if this bit is set in *flags* then the variable is looked up only in the global namespace even if there is a procedure call active. If both **TCL_GLOBAL_ONLY** and **TCL_NAMESPACE_ONLY** are given, **TCL_GLOBAL_ONLY** is ignored.

TCL_NAMESPACE_ONLY
If this bit is set in *flags* then the variable is looked up only in the current namespace; if a procedure is active its variables are ignored, and the global namespace is also ignored unless it is the current namespace.

TCL_LEAVE_ERR_MSG
If an error is returned and this bit is set in *flags*, then an error message will be left in the interpreter's result, where it can be retrieved with **Tcl_GetObjResult** or **Tcl_GetStringResult**. If this flag bit is not set then no error message is left and the interpreter's result will not be modified.

TCL_APPEND_VALUE
If this bit is set then *newValuePtr* or *newValue* is appended to the current value instead of replacing it. If the variable is currently undefined, then the bit is ignored. This bit is only used by the **Tcl_Set*** procedures.

TCL_LIST_ELEMENT
If this bit is set, then *newValue* is converted to a valid Tcl list element before setting (or appending to) the variable. A separator space is appended before the new list element unless the list element is going to be the first element in a list or sublist (i.e. the variable's current value is empty, or contains the single character "{", or ends in " }"). When appending, the original value of the variable must also be a valid list, so that the operation is the appending of a new list element onto a list. **Tcl_GetVar** and **Tcl_GetVar2** return the current value of a variable. The arguments to these procedures are treated in the same way as the arguments to **Tcl_SetVar** and **Tcl_SetVar2**. Under normal circumstances, the return value is a pointer to the variable's value (which is stored in Tcl's variable structure and will not change before the next call to **Tcl_SetVar** or **Tcl_SetVar2**). **Tcl_GetVar** and **Tcl_GetVar2** use the flag bits **TCL_GLOBAL_ONLY** and **TCL_LEAVE_ERR_MSG**, both of which have the same meaning as for **Tcl_SetVar**. If an error occurs in reading the variable (e.g. the variable does not exist or an array element is specified for a scalar variable), then NULL is returned. **Tcl_UnsetVar** and **Tcl_UnsetVar2** may be used to remove a variable, so that future calls to **Tcl_GetVar** or **Tcl_GetVar2** for the variable will return an error. The arguments to these procedures are treated in the same way as the arguments to **Tcl_GetVar** and **Tcl_GetVar2**. If the variable is successfully removed then **TCL_OK** is returned. If the variable cannot be removed because it does not exist then **TCL_ERROR** is returned. If an array element is specified, the given element is removed but the array remains. If an array name is specified without an index, then the entire array is removed.

Keywords: SetVar, array, get variable, interpreter, scalar, set, unset, value, variable

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

Signal

Name
Tcl_SignalId, Tcl_SignalMsg - Convert signal codes

Synopsis
#include <tcl.h>

const char *
Tcl_SignalId(*sig*)

const char *
Tcl_SignalMsg(*sig*)

Arguments
int *sig* (in)
A POSIX signal number such as **SIGPIPE**.

Description
Tcl_SignalId and **Tcl_SignalMsg** return a string representation of the provided signal number (*sig*). **Tcl_SignalId** returns a machine-readable textual identifier such as "SIGPIPE". **Tcl_SignalMsg** returns a human-readable string such as "bus error". The strings returned by these functions are statically allocated and the caller must not free or modify them.

Keywords: Signal, signals, signal numbers

Copyright (c) 2001 ActiveState Tool Corp.

Sleep

Name
Tcl_Sleep - delay execution for a given number of milliseconds

Synopsis
#include <tcl.h>

Tcl_Sleep(*ms*)

Arguments
int *ms* (in)
Number of milliseconds to sleep.

Description
This procedure delays the calling process by the number of milliseconds given by the *ms* parameter and returns after that time has elapsed. It is typically used for things like flashing a button, where the delay is short and the application need not do anything while it waits. For longer delays where the application needs to respond to other events during the delay, the procedure **Tcl_CreateTimerHandler** should be used instead of **Tcl_Sleep**.

Keywords: Sleep, time, wait

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

SourceRCFile

Name

Tcl_SourceRCFile - source the Tcl rc file

Synopsis

#include <tcl.h>

void
Tcl_SourceRCFile(*interp*)

Arguments

Tcl_Interp *_interp_ (in)
Tcl interpreter to source rc file into.

Description

Tcl_SourceRCFile is used to source the Tcl rc file at startup. It is typically invoked by Tcl_Main or Tk_Main. The name of the file sourced is obtained from the global variable **tcl_rcFileName** in the interpreter given by *interp*. If this variable is not defined, or if the file it indicates cannot be found, no action is taken.

Keywords: SourceRCFile, application-specific initialization, main program, rc file

Copyright (c) 1998-2000 Scriptics Corporation.
All rights reserved.

SplitList

Name

Tcl_SplitList, Tcl_Merge, Tcl_ScanElement, Tcl_ConvertElement, Tcl_ScanCountedElement, Tcl_ConvertCountedElement - manipulate Tcl lists

Synopsis

#include <tcl.h>

int
Tcl_SplitList(*interp*, *list*, *argcPtr*, *argvPtr*)

char *
Tcl_Merge(*argc*, *argv*)

int
Tcl_ScanElement(*src*, *flagsPtr*)

int
Tcl_ScanCountedElement(*src*, *length*, *flagsPtr*)

int
Tcl_ConvertElement(*src*, *dst*, *flags*)

int
Tcl_ConvertCountedElement(*src*, *length*, *dst*, *flags*)

Arguments

Tcl_Interp *_interp_ (out)
Interpreter to use for error reporting. If NULL, then no error message is left.

const char *_list_ (in)
Pointer to a string with proper list structure.

int *_argcPtr_ (out)
Filled in with number of elements in *list*.

const char ***_argvPtr_ (out)
*argvPtr will be filled in with the address of an array of pointers to the strings that are the extracted elements of *list*. There will be *argcPtr valid entries in the array, followed by a NULL entry.

int _argc_ (in)
Number of elements in *argv*.

const char *const *_argv_ (in)
Array of strings to merge together into a single list. Each string will become a separate element of the list.

const char *_src_ (in)
String that is to become an element of a list.

int *_flagsPtr_ (in)
Pointer to word to fill in with information about *src*. The value of *flagsPtr must be passed to **Tcl_ConvertElement**.

int _length_ (in)
Number of bytes in string *src*.

char *_dst_ (in)
Place to copy converted list element. Must contain enough characters to hold converted string.

int _flags_ (in)
Information about *src*. Must be value returned by previous call to **Tcl_ScanElement**, possibly OR-ed with **TCL_DONT_USE_BRACES**.

Description

These procedures may be used to disassemble and reassemble Tcl lists. **Tcl_SplitList** breaks a list up into its constituent elements, returning an array of pointers to the elements using *argcPtr* and *argvPtr*. While extracting the arguments, **Tcl_SplitList** obeys the usual rules for backslash substitutions and braces. The area of memory pointed to by *argvPtr is dynamically allocated; in addition to the array of pointers, it also holds copies of all the list elements. It is the caller's responsibility to free up all of this storage. For example, suppose that you have called **Tcl_SplitList** with the following code:

```
int argc, code;
char *string;
char **argv;
...
code = Tcl_SplitList(interp, string, &argc, &argv);
```

Then you should eventually free the storage with a call like the following:

```
Tcl_Free((char *) argv);
```

Tcl_SplitList normally returns **TCL_OK**, which means the list was successfully parsed. If there was a syntax error in *list*, then **TCL_ERROR** is returned and the interpreter's result will point to an error message describing the problem (if *interp* was not NULL). If **TCL_ERROR** is returned then no memory is allocated and **argvPtr* is not modified.

Tcl_Merge is the inverse of **Tcl_SplitList**: it takes a collection of strings given by *argc* and *argv* and generates a result string that has proper list structure. This means that commands like **index** may be used to extract the original elements again. In addition, if the result of **Tcl_Merge** is passed to **Tcl_Eval**, it will be parsed into *argc* words whose values will be the same as the *argv* strings passed to **Tcl_Merge**. **Tcl_Merge** will modify the list elements with braces and/or backslashes in order to produce proper Tcl list structure. The result string is dynamically allocated using **Tcl_Alloc**; the caller must eventually release the space using **Tcl_Free**.

If the result of **Tcl_Merge** is passed to **Tcl_SplitList**, the elements returned by **Tcl_SplitList** will be identical to those passed into **Tcl_Merge**. However, the converse is not true: if **Tcl_SplitList** is passed a given string, and the resulting *argc* and *argv* are passed to **Tcl_Merge**, the resulting string may not be the same as the original string passed to **Tcl_SplitList**. This is because **Tcl_Merge** may use backslashes and braces differently than the original string.

Tcl_ScanElement and **Tcl_ConvertElement** are the procedures that do all of the real work of **Tcl_Merge**. **Tcl_ScanElement** scans its *src* argument and determines how to use backslashes and braces when converting it to a list element. It returns an overestimate of the number of characters required to represent *src* as a list element, and it stores information in **flagsPtr* that is needed by **Tcl_ConvertElement**.

Tcl_ConvertElement is a companion procedure to **Tcl_ScanElement**. It does the actual work of converting a string to a list element. Its *flags* argument must be the same as the value returned by **Tcl_ScanElement**. **Tcl_ConvertElement** writes a proper list element to memory starting at **dst* and returns a count of the total number of characters written, which will be no more than the result returned by **Tcl_ScanElement**. **Tcl_ConvertElement** writes out only the actual list element without any leading or trailing spaces: it is up to the caller to include spaces between adjacent list elements.

Tcl_ConvertElement uses one of two different approaches to handle the special characters in *src*. Wherever possible, it handles special characters by surrounding the string with braces. This produces clean-looking output, but cannot be used in some situations, such as when *src* contains unmatched braces. In these situations, **Tcl_ConvertElement** handles special characters by generating backslash sequences for them. The caller may insist on the second approach by OR-ing the flag value returned by **Tcl_ScanElement** with **TCL_DONT_USE_BRACES**. Although this will produce an uglier result, it is useful in some special situations, such as when **Tcl_ConvertElement** is being used to generate a portion of an argument for a Tcl command. In this case, surrounding *src* with curly braces would cause the command not to be parsed correctly.

By default, **Tcl_ConvertElement** will use quoting in its output to be sure the first character of an element is not the hash character ("#".) This is to be sure the first element of any list passed to **eval** is not mis-parsed as the beginning of a comment. When a list element is not the first element of a list, this quoting is not necessary. When the caller can be sure that the element is not the first element of a list, it can disable quoting of the leading hash character by OR-ing the

flag value returned by **Tcl_ScanElement** with **TCL_DONT_QUOTE_HASH**.

Tcl_ScanCountedElement and **Tcl_ConvertCountedElement** are the same as **Tcl_ScanElement** and **Tcl_ConvertElement**, except the length of string *src* is specified by the *length* argument, and the string may contain embedded nulls.

Keywords: SplitList, backslash, convert, element, list, merge, split, strings

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

SplitPath

Name

Tcl_SplitPath, Tcl_JoinPath, Tcl_GetPathType - manipulate platform-dependent file paths

Synopsis

#include <tcl.h>

Tcl_SplitPath(*path*, *argcPtr*, *argvPtr*)

char *
Tcl_JoinPath(*argc*, *argv*, *resultPtr*)

Tcl_PathType
Tcl_GetPathType(*path*)

Arguments

const char *_path_ (in)
File path in a form appropriate for the current platform (see the **filename** manual entry for acceptable forms for path names).

int *_argcPtr_ (out)
Filled in with number of path elements in *path*.

const char ***_argvPtr_ (out)
**argvPtr* will be filled in with the address of an array of pointers to the strings that are the extracted elements of *path*. There will be **argcPtr* valid entries in the array, followed by a NULL entry.

int _argc_ (in)
Number of elements in *argv*.

const char *const *_argv_ (in)
Array of path elements to merge together into a single path.

Tcl_DString *_resultPtr_ (in/out)
A pointer to an initialized **Tcl_DString** to which the result of **Tcl_JoinPath** will be appended.

Description

These procedures have been superseded by the Tcl-value-aware procedures in the **FileSystem** man page, which are more efficient.

These procedures may be used to disassemble and reassemble file paths in a platform independent manner: they provide C-level access to the same functionality as the **file split**, **file join**, and **file pathtype** commands.

Tcl_SplitPath breaks a path into its constituent elements, returning an array of pointers to the elements using *argcPtr* and *argvPtr*. The area of memory pointed to by **argvPtr* is dynamically allocated; in addition to the array of pointers, it also holds copies of all the path elements. It is the caller's responsibility to free all of this storage. For example, suppose that you have called **Tcl_SplitPath** with the following code:

```
int argc;
char *path;
char **argv;
...
Tcl_SplitPath(string, &argc, &argv);
```

Then you should eventually free the storage with a call like the following:

```
Tcl_Free((char *) argv);
```

Tcl_JoinPath is the inverse of **Tcl_SplitPath**: it takes a collection of path elements given by *argc* and *argv* and generates a result string that is a properly constructed path. The result string is appended to *resultPtr*. *ResultPtr* must refer to an initialized **Tcl_DString**.

If the result of **Tcl_SplitPath** is passed to **Tcl_JoinPath**, the result will refer to the same location, but may not be in the same form. This is because **Tcl_SplitPath** and **Tcl_JoinPath** eliminate duplicate path separators and return a normalized form for each platform.

Tcl_GetPathType returns the type of the specified *path*, where **Tcl_PathType** is one of **TCL_PATH_ABSOLUTE**, **TCL_PATH_RELATIVE**, or **TCL_PATH_VOLUME_RELATIVE**. See the **filename** manual entry for a description of the path types for each platform.

Keywords: SplitPath, file, filename, join, path, split, type

Copyright (c) 1996 Sun Microsystems, Inc.

StaticPkg

Name

Tcl_StaticPackage - make a statically linked package available via the 'load' command

Synopsis

#include <tcl.h>

Tcl_StaticPackage(*interp*, *prefix*, *initProc*, *safeInitProc*)

Arguments

Tcl_Interp ****interp* **(in)
: If not NULL, points to an interpreter into which the package has already been loaded (i.e., the caller has already invoked the appropriate initialization procedure). NULL means the package has not yet been incorporated into any interpreter.

const char ****prefix* **(in)
: Prefix for library initialization function; should be properly capitalized (first letter upper-case, all others lower-case).

Tcl_PackageInitProc ****initProc* **(in)
: Procedure to invoke to incorporate this package into a trusted interpreter.

Tcl_PackageInitProc ****safeInitProc* **(in)
Procedure to call to incorporate this package into a safe interpreter (one that will execute untrusted scripts). NULL means the package cannot be used in safe interpreters.

Description

This procedure may be invoked to announce that a package has been linked statically with a Tcl application and, optionally, that it has already been loaded into an interpreter. Once **Tcl_StaticPackage** has been invoked for a package, it may be loaded into interpreters using the **load** command. **Tcl_StaticPackage** is normally invoked only by the **Tcl_AppInit** procedure for the application, not by packages for themselves (**Tcl_StaticPackage** should only be invoked for statically loaded packages, and code in the package itself should not need to know whether the package is dynamically or statically loaded).

When the **load** command is used later to load the package into an interpreter, one of *initProc* and *safeInitProc* will be invoked, depending on whether the target interpreter is safe or not. *initProc* and *safeInitProc* must both match the following prototype:

```
typedef int Tcl_PackageInitProc(
        Tcl_Interp *interp);
```

The *interp* argument identifies the interpreter in which the package is to be loaded. The initialization procedure must return **TCL_OK** or **TCL_ERROR** to indicate whether or not it completed successfully; in the event of an error it should set the interpreter's result to point to an error message. The result or error from the initialization procedure will be returned as the result of the **load** command that caused the initialization procedure to be invoked.

Keywords: StaticPkg, initialization procedure, package, static linking

Copyright (c) 1995-1996 Sun Microsystems, Inc.

StdChannels

Name

Tcl_StandardChannels - How the Tcl library deals with the standard channels

Description

This page explains the initialization and use of standard channels in the Tcl library.

The term *standard channels* comes out of the Unix world and refers to the three channels automatically opened by the OS for each new application. They are **stdin**, **stdout** and **stderr**. The first is the standard input an application can read from, the other two refer to writable channels, one for regular output and the other for error messages.

Tcl generalizes this concept in a cross-platform way and exposes standard channels to the script level.

Application programming interfaces

The public API procedures dealing directly with standard channels are **Tcl_GetStdChannel** and **Tcl_SetStdChannel**. Additional public APIs to consider are **Tcl_RegisterChannel**, **Tcl_CreateChannel** and **Tcl_GetChannel**.

Initialization of tcl standard channels

Standard channels are initialized by the Tcl library in three cases: when explicitly requested, when implicitly required before returning channel information, or when implicitly required during registration of a new channel.

These cases differ in how they handle unavailable platform- specific standard channels. (A channel is not "available" if it could not be successfully opened; for example, in a Tcl application run as a Windows NT service.)

1)
A single standard channel is initialized when it is explicitly specified in a call to **Tcl_SetStdChannel**. The states of the other standard channels are unaffected.

Missing platform-specific standard channels do not matter here. This approach is not available at the script level.

2)
All uninitialized standard channels are initialized to platform-specific default values:

(a)
when open channels are listed with **Tcl_GetChannelNames** (or the **file channels** script command), or

(b)
when information about any standard channel is requested with a call to **Tcl_GetStdChannel**, or with a call to **Tcl_GetChannel** which specifies one of the standard names (**stdin**, **stdout** and **stderr**). In case of missing platform-specific standard channels, the Tcl standard channels are considered as initialized and then immediately closed. This means that the first three Tcl channels then opened by the application are designated as the Tcl standard channels.

3)
All uninitialized standard channels are initialized to platform-specific default values when a user-requested channel is registered with **Tcl_RegisterChannel**. In case of unavailable platform-specific standard channels the channel whose creation caused the initialization of the Tcl standard channels is made a normal channel. The next three Tcl channels opened by the application are designated as the Tcl standard channels. In other words, of the first four Tcl channels opened by the application the second to fourth are designated as the Tcl standard channels.

Re-initialization of tcl standard channels

Once a Tcl standard channel is initialized through one of the methods above, closing this Tcl standard channel will cause the next call to **Tcl_CreateChannel** to make the new channel the new standard channel, too. If more than one Tcl standard channel was closed **Tcl_CreateChannel** will fill the empty slots in the order **stdin**, **stdout** and **stderr**.

Tcl_CreateChannel will not try to reinitialize an empty slot if that slot was not initialized before. It is this behavior which enables an application to employ method 1 of initialization, i.e. to create and designate their own Tcl standard channels.

Shell-specific details

Tclsh

The Tcl shell (or rather the function **Tcl_Main**, which forms the core of the shell's implementation) uses method 2 to initialize the standard channels.

Wish

The windowing shell (or rather the function **Tk_MainEx**, which forms the core of the shell's implementation) uses method 1 to initialize the standard channels (See **Tk_InitConsoleChannels**) on non-Unix platforms. On Unix platforms, **Tk_MainEx** implicitly uses method 2 to initialize the standard channels.

Keywords: StdChannels, standard channels

Copyright (c) 2001 ActiveState Corporation

StrMatch

Name

Tcl_StringMatch, Tcl_StringCaseMatch - test whether a string matches a pattern

Synopsis

#include <tcl.h>

int
Tcl_StringMatch(*str*, *pattern*)

int
Tcl_StringCaseMatch(*str*, *pattern*, *flags*)

Arguments

const char *_str_ (in)
String to test.

const char *_pattern_ (in)
Pattern to match against string. May contain special characters from the set *?\[].

int _flags_ (in)
OR-ed combination of match flags, currently only **TCL_MATCH_NOCASE**. 0 specifies a case-sensitive search.

Description

This utility procedure determines whether a string matches a given pattern. If it does, then **Tcl_StringMatch** returns 1. Otherwise **Tcl_StringMatch** returns 0. The algorithm used for matching is the same algorithm used in the **string match** Tcl command and is similar to the algorithm used by the C-shell for file name matching; see the Tcl manual entry for details.

In **Tcl_StringCaseMatch**, the algorithm is the same, but you have the option to make the matching case-insensitive. If you choose this (by passing **TCL_MATCH_NOCASE**), then the string and pattern are essentially matched in the lower case.

Keywords: StrMatch, match, pattern, string

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

StringObj

Name

Tcl_NewStringObj, Tcl_NewUnicodeObj, Tcl_SetStringObj, Tcl_SetUnicodeObj, Tcl_GetStringFromObj, Tcl_GetString, Tcl_GetUnicodeFromObj, Tcl_GetUnicode, Tcl_GetUniChar, Tcl_GetCharLength, Tcl_GetRange, Tcl_AppendToObj, Tcl_AppendUnicodeToObj, Tcl_AppendObjToObj, Tcl_AppendStringsToObj, Tcl_AppendStringsToObjVA, Tcl_AppendLimitedToObj, Tcl_Format, Tcl_AppendFormatToObj, Tcl_ObjPrintf, Tcl_AppendPrintfToObj, Tcl_SetObjLength, Tcl_AttemptSetObjLength, Tcl_ConcatObj - manipulate Tcl values as strings

Synopsis

#include <tcl.h>

Tcl_Obj *
Tcl_NewStringObj(*bytes*, *length*)

Tcl_Obj *
Tcl_NewUnicodeObj(*unicode*, *numChars*)

void
Tcl_SetStringObj(*objPtr*, *bytes*, *length*)

void
Tcl_SetUnicodeObj(*objPtr*, *unicode*, *numChars*)

char *
Tcl_GetStringFromObj(*objPtr*, *lengthPtr*)

char *
Tcl_GetString(*objPtr*)

Tcl_UniChar *
Tcl_GetUnicodeFromObj(*objPtr*, *lengthPtr*)

Tcl_UniChar *
Tcl_GetUnicode(*objPtr*)

Tcl_UniChar
Tcl_GetUniChar(*objPtr*, *index*)

int
Tcl_GetCharLength(*objPtr*)

Tcl_Obj *
Tcl_GetRange(*objPtr*, *first*, *last*)

void
Tcl_AppendToObj(*objPtr*, *bytes*, *length*)

void
Tcl_AppendUnicodeToObj(*objPtr*, *unicode*, *numChars*)

void
Tcl_AppendObjToObj(*objPtr*, *appendObjPtr*)

void
Tcl_AppendStringsToObj(*objPtr*, *string*, *string*, ... (**char** *)**NULL**)

void

Tcl_Obj *
Tcl_AppendStringsToObjVA(*objPtr*, *argList*)

void
Tcl_AppendLimitedToObj(*objPtr*, *bytes*, *length*, *limit*, *ellipsis*)

Tcl_Obj *
Tcl_Format(*interp*, *format*, *objc*, *objv*)

int
Tcl_AppendFormatToObj(*interp*, *objPtr*, *format*, *objc*, *objv*)

Tcl_Obj *
Tcl_ObjPrintf(*format*, ...)

void
Tcl_AppendPrintfToObj(*objPtr*, *format*, ...)

void
Tcl_SetObjLength(*objPtr*, *newLength*)

int
Tcl_AttemptSetObjLength(*objPtr*, *newLength*)

Tcl_Obj *
Tcl_ConcatObj(*objc*, *objv*)

Arguments

const char *_bytes_ (in)
Points to the first byte of an array of UTF-8-encoded bytes used to set or append to a string value. This byte array may contain embedded null characters unless *numChars* is negative. (Applications needing null bytes should represent them as the two-byte sequence *\300\200*, use **Tcl_ExternalToUtf** to convert, or **Tcl_NewByteArrayObj** if the string is a collection of uninterpreted bytes.)

int _length_ (in)
The number of bytes to copy from *bytes* when initializing, setting, or appending to a string value. If negative, all bytes up to the first null are used.

const Tcl_UniChar *_unicode_ (in)
Points to the first byte of an array of Unicode characters used to set or append to a string value. This byte array may contain embedded null characters unless *numChars* is negative.

int _numChars_ (in)
The number of Unicode characters to copy from *unicode* when initializing, setting, or appending to a string value. If negative, all characters up to the first null character are used.

int _index_ (in)
The index of the Unicode character to return.

int _first_ (in)
The index of the first Unicode character in the Unicode range to be returned as a new value.

int _last_ (in)
The index of the last Unicode character in the Unicode range to be returned as a new value.

Tcl_Obj *_objPtr_ (in/out)
Points to a value to manipulate.

Tcl_Obj *_appendObjPtr_ (in)

The value to append to *objPtr* in **Tcl_AppendObjToObj**.

int *lengthPtr (out)
If non-NULL, the location where **Tcl_GetStringFromObj** will store the length of a value's string representation.

const char *string (in)
Null-terminated string value to append to *objPtr*.

va_list argList (in)
An argument list which must have been initialized using **va_start**, and cleared using **va_end**.

int limit (in)
Maximum number of bytes to be appended.

const char *ellipsis (in)
Suffix to append when the limit leads to string truncation. If NULL is passed then the suffix "..." is used.

const char *format (in)
Format control string including % conversion specifiers.

int objc (in)
The number of elements to format or concatenate.

Tcl_Obj *objv[] (in)
The array of values to format or concatenate.

int newLength (in)
New length for the string value of *objPtr*, not including the final null character.

Description

The procedures described in this manual entry allow Tcl values to be manipulated as string values. They use the internal representation of the value to store additional information to make the string manipulations more efficient. In particular, they make a series of append operations efficient by allocating extra storage space for the string so that it does not have to be copied for each append. Also, indexing and length computations are optimized because the Unicode string representation is calculated and cached as needed. When using the **Tcl_Append*** family of functions where the interpreter's result is the value being appended to, it is important to call Tcl_ResetResult first to ensure you are not unintentionally appending to existing data in the result value.

Tcl_NewStringObj and **Tcl_SetStringObj** create a new value or modify an existing value to hold a copy of the string given by *bytes* and *length*. **Tcl_NewUnicodeObj** and **Tcl_SetUnicodeObj** create a new value or modify an existing value to hold a copy of the Unicode string given by *unicode* and *numChars*. **Tcl_NewStringObj** and **Tcl_NewUnicodeObj** return a pointer to a newly created value with reference count zero. All four procedures set the value to hold a copy of the specified string. **Tcl_SetStringObj** and **Tcl_SetUnicodeObj** free any old string representation as well as any old internal representation of the value.

Tcl_GetStringFromObj and **Tcl_GetString** return a value's string representation. This is given by the returned byte pointer and (for **Tcl_GetStringFromObj**) length, which is stored in *lengthPtr* if it is non-NULL. If the value's UTF string representation is invalid (its byte pointer is NULL), the string representation is regenerated from the value's internal representation. The storage referenced by the returned byte pointer is owned by the value manager. It is passed

back as a writable pointer so that extension author creating their own **Tcl_ObjType** will be able to modify the string representation within the **Tcl_UpdateStringProc** of their **Tcl_ObjType**. Except for that limited purpose, the pointer returned by **Tcl_GetStringFromObj** or **Tcl_GetString** should be treated as read-only. It is recommended that this pointer be assigned to a (const char *) variable. Even in the limited situations where writing to this pointer is acceptable, one should take care to respect the copy-on-write semantics required by **Tcl_Obj**'s, with appropriate calls to **Tcl_IsShared** and **Tcl_DuplicateObj** prior to any in-place modification of the string representation. The procedure **Tcl_GetString** is used in the common case where the caller does not need the length of the string representation.

Tcl_GetUnicodeFromObj and **Tcl_GetUnicode** return a value's value as a Unicode string. This is given by the returned pointer and (for **Tcl_GetUnicodeFromObj**) length, which is stored in *lengthPtr* if it is non-NULL. The storage referenced by the returned byte pointer is owned by the value manager and should not be modified by the caller. The procedure **Tcl_GetUnicode** is used in the common case where the caller does not need the length of the unicode string representation.

Tcl_GetUniChar returns the *index*'th character in the value's Unicode representation. The index is assumed to be in the appropriate range.

Tcl_GetRange returns a newly created value comprised of the characters between *first* and *last* (inclusive) in the value's Unicode representation. If the value's Unicode representation is invalid, the Unicode representation is regenerated from the value's string representation. If *first* < 0, then the returned string starts at the beginning of the value. If *last* < 0, then the returned string ends at the end of the value.

Tcl_GetCharLength returns the number of characters (as opposed to bytes) in the string value.

Tcl_AppendToObj appends the data given by *bytes* and *length* to the string representation of the value specified by *objPtr*. If the value has an invalid string representation, then an attempt is made to convert *bytes* to the Unicode format. If the conversion is successful, then the converted form of *bytes* is appended to the value's Unicode representation. Otherwise, the value's Unicode representation is invalidated and converted to the UTF format, and *bytes* is appended to the value's new string representation. Eventually buffer growth is done by large allocations to optimize multiple calls.

Tcl_AppendUnicodeToObj appends the Unicode string given by *unicode* and *numChars* to the value specified by *objPtr*. If the value has an invalid Unicode representation, then *unicode* is converted to the UTF format and appended to the value's string representation. Appends are optimized to handle repeated appends relatively efficiently (it over-allocates the string or Unicode space to avoid repeated reallocations and copies of value's string value).

Tcl_AppendObjToObj is similar to **Tcl_AppendToObj**, but it appends the string or Unicode value (whichever exists and is best suited to be appended to *objPtr*) of *appendObjPtr* to *objPtr*.

Tcl_AppendStringsToObj is similar to **Tcl_AppendToObj** except that it can be passed more than one value to append and each value must be a null-terminated string (i.e. none of the values may contain internal null characters). Any number of *string* arguments may be provided, but the last argument must be (char *)NULL to indicate the end of the list.

Tcl_AppendStringsToObjVA is the same as **Tcl_AppendStringsToObj** except that instead of taking a variable number of arguments it takes an argument list.

Tcl_AppendLimitedToObj is similar to **Tcl_AppendToObj** except that it imposes a limit on how

many bytes are appended. This can be handy when the string to be appended might be very large, but the value being constructed should not be allowed to grow without bound. A common usage is when constructing an error message, where the end result should be kept short enough to be read. Bytes from *bytes* are appended to *objPtr*, but no more than *limit* bytes total are to be appended. If the limit prevents all *length* bytes that are available from being appended, then the appending is done so that the last bytes appended are from the string *ellipsis*. This allows for an indication of the truncation to be left in the string. When *length* is **-1**, all bytes up to the first zero byte are appended, subject to the limit. When *ellipsis* is NULL, the default string**...** is used. When *ellipsis* is non-NULL, it must point to a zero-byte-terminated string in Tcl's internal UTF encoding. The number of bytes appended can be less than the lesser of *length* and *limit* when appending fewer bytes is necessary to append only whole multi-byte characters.

Tcl_Format is the C-level interface to the engine of the **format** command. The actual command procedure for **format** is little more than

```
Tcl_Format(interp, Tcl_GetString(objv[1]), objc-2, objv+2);
```

The *objc* Tcl_Obj values in *objv* are formatted into a string according to the conversion specification in *format* argument, following the documentation for the **format** command. The resulting formatted string is converted to a new Tcl_Obj with refcount of zero and returned. If some error happens during production of the formatted string, NULL is returned, and an error message is recorded in *interp*, if *interp* is non-NULL.

Tcl_AppendFormatToObj is an appending alternative form of **Tcl_Format** with functionality equivalent to:

```
Tcl_Obj *newPtr = Tcl_Format(interp, format, objc, objv);
if (newPtr == NULL) return TCL_ERROR;
Tcl_AppendObjToObj(objPtr, newPtr);
Tcl_DecrRefCount(newPtr);
return TCL_OK;
```

but with greater convenience and efficiency when the appending functionality is needed.

Tcl_ObjPrintf serves as a replacement for the common sequence

```
char buf[SOME_SUITABLE_LENGTH];
sprintf(buf, format, ...);
Tcl_NewStringObj(buf, -1);
```

but with greater convenience and no need to determine **SOME_SUITABLE_LENGTH**. The formatting is done with the same core formatting engine used by **Tcl_Format**. This means the set of supported conversion specifiers is that of the **format** command but the behavior is as similar as possible to **sprintf**. Format specifiers which were added by C99 (like "hh", "ll", "j", "z", "t", "L") are not supported. When a conversion specifier passed to **Tcl_ObjPrintf** includes a precision, the value is taken as a number of bytes, as **sprintf** does, and not as a number of characters, as **format** does. This is done on the assumption that C code is more likely to know how many bytes it is passing around than the number of encoded characters those bytes happen to represent. The variable number of arguments passed in should be of the types that would be suitable for passing to **sprintf**. Note in this example usage, *x* is of type **int**.

```
int x = 5;
Tcl_Obj *objPtr = Tcl_ObjPrintf("Value is %d", x);
```

If the value of *format* contains internal inconsistencies or invalid specifier formats, the formatted string result produced by **Tcl_ObjPrintf** will be an error message describing the error. It is impossible however to provide runtime protection against mismatches between the format and any subsequent arguments. Compile-time protection may be provided by some compilers.

Tcl_AppendPrintfToObj is an appending alternative form of **Tcl_ObjPrintf** with functionality equivalent to

```
Tcl_Obj *newPtr = Tcl_ObjPrintf(format, ...);
Tcl_AppendObjToObj(objPtr, newPtr);
Tcl_DecrRefCount(newPtr);
```

but with greater convenience and efficiency when the appending functionality is needed.

The **Tcl_SetObjLength** procedure changes the length of the string value of its *objPtr* argument. If the *newLength* argument is greater than the space allocated for the value's string, then the string space is reallocated and the old value is copied to the new space; the bytes between the old length of the string and the new length may have arbitrary values. If the *newLength* argument is less than the current length of the value's string, with *objPtr->length* is reduced without reallocating the string space; the original allocated size for the string is recorded in the value, so that the string length can be enlarged in a subsequent call to **Tcl_SetObjLength** without reallocating storage. In all cases **Tcl_SetObjLength** leaves a null character at *objPtr->bytes[newLength]*.

Tcl_AttemptSetObjLength is identical in function to **Tcl_SetObjLength** except that if sufficient memory to satisfy the request cannot be allocated, it does not cause the Tcl interpreter to **panic**. Thus, if *newLength* is greater than the space allocated for the value's string, and there is not enough memory available to satisfy the request, **Tcl_AttemptSetObjLength** will take no action and return 0 to indicate failure. If there is enough memory to satisfy the request, **Tcl_AttemptSetObjLength** behaves just like **Tcl_SetObjLength** and returns 1 to indicate success.

The **Tcl_ConcatObj** function returns a new string value whose value is the space-separated concatenation of the string representations of all of the values in the *objv* array. **Tcl_ConcatObj** eliminates leading and trailing white space as it copies the string representations of the *objv* array to the result. If an element of the *objv* array consists of nothing but white space, then that value is ignored entirely. This white-space removal was added to make the output of the **concat** command cleaner-looking. **Tcl_ConcatObj** returns a pointer to a newly-created value whose ref count is zero.

Keywords: StringObj, append, internal representation, value, value type, string value, string type , string representation, concat, concatenate, unicode

Copyright (c) 1994-1997 Sun Microsystems, Inc.

SubstObj

Name

Tcl_SubstObj - perform substitutions on Tcl values

Synopsis

#include <tcl.h>

Tcl_Obj *
Tcl_SubstObj(*interp*, *objPtr*, *flags*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter in which to execute Tcl scripts and lookup variables. If an error occurs, the interpreter's result is modified to hold an error message.

Tcl_Obj *_objPtr_ (in)
A Tcl value containing the string to perform substitutions on.

int _flags_ (in)
OR'ed combination of flag bits that specify which substitutions to perform. The flags **TCL_SUBST_COMMANDS**, **TCL_SUBST_VARIABLES** and **TCL_SUBST_BACKSLASHES** are currently supported, and **TCL_SUBST_ALL** is provided as a convenience for the common case where all substitutions are desired.

Description

The **Tcl_SubstObj** function is used to perform substitutions on strings in the fashion of the **subst** command. It gets the value of the string contained in *objPtr* and scans it, copying characters and performing the chosen substitutions as it goes to an output value which is returned as the result of the function. In the event of an error occurring during the execution of a command or variable substitution, the function returns NULL and an error message is left in *interp*'s result.

Three kinds of substitutions are supported. When the **TCL_SUBST_BACKSLASHES** bit is set in *flags*, sequences that look like backslash substitutions for Tcl commands are replaced by their corresponding character.

When the **TCL_SUBST_VARIABLES** bit is set in *flags*, sequences that look like variable substitutions for Tcl commands are replaced by the contents of the named variable.

When the **TCL_SUBST_COMMANDS** bit is set in *flags*, sequences that look like command substitutions for Tcl commands are replaced by the result of evaluating that script. Where an uncaught "continue exception" occurs during the evaluation of a command substitution, an empty string is substituted for the command. Where an uncaught "break exception" occurs during the evaluation of a command substitution, the result of the whole substitution on *objPtr* will be truncated at the point immediately before the start of the command substitution, and no characters will be added to the result or substitutions performed after that point.

Keywords: SubstObj, backslash substitution, command substitution, variable substitution

Copyright (c) 2001 Donal K. Fellows

TCL_MEM_DEBUG

Name

TCL_MEM_DEBUG - Compile-time flag to enable Tcl memory debugging

Description

When Tcl is compiled with **TCL_MEM_DEBUG** defined, a powerful set of memory debugging aids is included in the compiled binary. This includes C and Tcl functions which can aid with debugging memory leaks, memory allocation overruns, and other memory related errors.

Enabling memory debugging

To enable memory debugging, Tcl should be recompiled from scratch with **TCL_MEM_DEBUG** defined (e.g. by passing the *--enable-symbols=mem* flag to the *configure* script when building). This will also compile in a non-stub version of **Tcl_InitMemory** to add the **memory** command to Tcl.

TCL_MEM_DEBUG must be either left defined for all modules or undefined for all modules that are going to be linked together. If they are not, link errors will occur, with either **Tcl_DbCkfree** and **Tcl_DbCkalloc** or **Tcl_Alloc** and **Tcl_Free** being undefined.

Once memory debugging support has been compiled into Tcl, the C functions **Tcl_ValidateAllMemory**, and **Tcl_DumpActiveMemory**, and the Tcl **memory** command can be used to validate and examine memory usage.

Guard zones

When memory debugging is enabled, whenever a call to **ckalloc** is made, slightly more memory than requested is allocated so the memory debugging code can keep track of the allocated memory, and eight-byte "guard zones" are placed in front of and behind the space that will be returned to the caller. (The sizes of the guard zones are defined by the C #define **LOW_GUARD_SIZE** and #define **HIGH_GUARD_SIZE** in the file *generic/tclCkalloc.c* — it can be extended if you suspect large overwrite problems, at some cost in performance.) A known pattern is written into the guard zones and, on a call to **ckfree**, the guard zones of the space being freed are checked to see if either zone has been modified in any way. If one has been, the guard bytes and their new contents are identified, and a "low guard failed" or "high guard failed" message is issued. The "guard failed" message includes the address of the memory packet and the file name and line number of the code that called **ckfree**. This allows you to detect the common sorts of one-off problems, where not enough space was allocated to contain the data written, for example.

Debugging difficult memory corruption problems

Normally, Tcl compiled with memory debugging enabled will make it easy to isolate a corruption problem. Turning on memory validation with the memory command can help isolate difficult problems. If you suspect (or know) that corruption is occurring before the Tcl interpreter comes up far enough for you to issue commands, you can set **MEM_VALIDATE** define, recompile tclCkalloc.c and rebuild Tcl. This will enable memory validation from the first call to **ckalloc**, again, at a large performance impact.

If you are desperate and validating memory on every call to **ckalloc** and **ckfree** is not enough, you can explicitly call **Tcl_ValidateAllMemory** directly at any point. It takes a *char* * and an *int* which are normally the filename and line number of the caller, but they can actually be anything you want. Remember to remove the calls after you find the problem.

Keywords: TCL_MEM_DEBUG, memory, debug

Copyright (c) 1992-1999 Karl Lehenbauer & Mark Diekhans.

Copyright (c) 2000 Scriptics Corporation.
All rights reserved.

TclZlib

Name

Tcl_ZlibAdler32, Tcl_ZlibCRC32, Tcl_ZlibDeflate, Tcl_ZlibInflate, Tcl_ZlibStreamChecksum, Tcl_ZlibStreamClose, Tcl_ZlibStreamEof, Tcl_ZlibStreamGet, Tcl_ZlibStreamGetCommandName, Tcl_ZlibStreamInit, Tcl_ZlibStreamPut - compression and decompression functions

Synopsis

#include <tcl.h>

int
Tcl_ZlibDeflate(*interp*, *format*, *dataObj*, *level*, *dictObj*)

int
Tcl_ZlibInflate(*interp*, *format*, *dataObj*, *bufferSize*, *dictObj*)

unsigned int
Tcl_ZlibCRC32(*initValue*, *bytes*, *length*)

unsigned int
Tcl_ZlibAdler32(*initValue*, *bytes*, *length*)

int
Tcl_ZlibStreamInit(*interp*, *mode*, *format*, *level*, *dictObj*, *zshandlePtr*)

Tcl_Obj *
Tcl_ZlibStreamGetCommandName(*zshandle*)

int
Tcl_ZlibStreamEof(*zshandle*)

int
Tcl_ZlibStreamClose(*zshandle*)

int
Tcl_ZlibStreamReset(*zshandle*)

int
Tcl_ZlibStreamChecksum(*zshandle*)

int
Tcl_ZlibStreamPut(*zshandle*, *dataObj*, *flush*)

int
Tcl_ZlibStreamGet(*zshandle*, *dataObj*, *count*)

Tcl_ZlibStreamSetCompressionDictionary(*zshandle*, *compDict*)

Arguments

Tcl_Interp ****interp* **(in)
The interpreter to store resulting compressed or uncompressed data in. Also where any error messages are written. For **Tcl_ZlibStreamInit**, this can be NULL to create a stream that is not bound to a command.

int *format* (in)
What format of compressed data to work with. Must be one of **TCL_ZLIB_FORMAT_ZLIB** for zlib-format data, **TCL_ZLIB_FORMAT_GZIP** for gzip-format data, or **TCL_ZLIB_FORMAT_RAW** for raw compressed data. In addition, for decompression only, **TCL_ZLIB_FORMAT_AUTO** may also be chosen which can automatically detect whether the compressed data was in zlib or gzip format.

Tcl_Obj **dataObj* (in/out)
A byte-array value containing the data to be compressed or decompressed, or to which the data extracted from the stream is appended when passed to **Tcl_ZlibStreamGet**.

int *level* (in)
What level of compression to use. Should be a number from 0 to 9 or one of the following: **TCL_ZLIB_COMPRESS_NONE** for no compression, **TCL_ZLIB_COMPRESS_FAST** for fast but inefficient compression, **TCL_ZLIB_COMPRESS_BEST** for slow but maximal compression, or **TCL_ZLIB_COMPRESS_DEFAULT** for the level recommended by the zlib library.

Tcl_Obj **dictObj* (in/out)
A dictionary that contains, or which will be updated to contain, a description of the gzip header associated with the compressed data. Only useful when the *format* is **TCL_ZLIB_FORMAT_GZIP** or **TCL_ZLIB_FORMAT_AUTO**. If a NULL is passed, a default header will be used on compression and the header will be ignored (apart from integrity checks) on decompression. See the section **GZIP OPTIONS DICTIONARY** for details about the contents of this dictionary.

unsigned int *initValue* (in)
The initial value for the checksum algorithm.

int *bufferSize* (in)
A hint as to what size of buffer is to be used to receive the data. Use 0 to use a geric guess based on the input data.

unsigned char **bytes* (in)
An array of bytes to run the checksum algorithm over, or NULL to get the recommended initial value for the checksum algorithm.

int *length* (in)
The number of bytes in the array.

int *mode* (in)
What mode to operate the stream in. Should be either **TCL_ZLIB_STREAM_DEFLATE** for a compressing stream or **TCL_ZLIB_STREAM_INFLATE** for a decompressing stream.

Tcl_ZlibStream **zshandlePtr* (out)
A pointer to a variable in which to write the abstract token for the stream upon successful creation.

Tcl_ZlibStream *zshandle* (in)
The abstract token for the stream to operate on.

int *flush* (in)
Whether and how to flush the stream after writing the data to it. Must be one of: **TCL_ZLIB_NO_FLUSH** if no flushing is to be done, **TCL_ZLIB_FLUSH** if the currently compressed data must be made available for access using **Tcl_ZlibStreamGet**, **TCL_ZLIB_FULLFLUSH** if the stream must be put into a state where the decompressor can recover from on corruption, or **TCL_ZLIB_FINALIZE** to ensure that the stream is finished and

that any trailer demanded by the format is written.

int *count* (in)
The maximum number of bytes to get from the stream, or -1 to get all remaining bytes from the stream's buffers.

Tcl_Obj *compDict* (in)
A byte array value that is the compression dictionary to use with the stream. Note that this is *not a Tcl dictionary*, and it is recommended that this only ever be used with streams that were created with their *format* set to **TCL_ZLIB_FORMAT_ZLIB** because the other formats have no mechanism to indicate whether a compression dictionary was present other than to fail on decompression.

Description

These functions form the interface from the Tcl library to the Zlib library by Jean-loup Gailly and Mark Adler.

Tcl_ZlibDeflate and **Tcl_ZlibInflate** respectively compress and decompress the data contained in the *dataObj* argument, according to the *format* and, for compression, *level* arguments. The dictionary in the *dictObj* parameter is used to convey additional header information about the compressed data when the compression format supports it; currently, the dictionary is only used when the *format* parameter is **TCL_ZLIB_FORMAT_GZIP** or **TCL_ZLIB_FORMAT_AUTO**. For details of the contents of the dictionary, see the **GZIP OPTIONS DICTIONARY** section below. Upon success, both functions leave the resulting compressed or decompressed data in a byte-array value that is the Tcl interpreter's result; the returned value is a standard Tcl result code.

Tcl_ZlibAdler32 and **Tcl_ZlibCRC32** compute checksums on arrays of bytes, returning the computed checksum. Checksums are computed incrementally, allowing data to be processed one block at a time, but this requires the caller to maintain the current checksum and pass it in as the *initValue* parameter; the initial value to use for this can be obtained by using NULL for the *bytes* parameter instead of a pointer to the array of bytes to compute the checksum over. Thus, typical usage in the single data block case is like this:

```
checksum = Tcl_ZlibCRC32(Tcl_ZlibCRC32(0,NULL,0), data, length);
```

Note that the Adler-32 algorithm is not a real checksum, but instead is a related type of hash that works best on longer data.

Zlib streams

Tcl_ZlibStreamInit creates a compressing or decompressing stream that is linked to a Tcl command, according to its arguments, and provides an abstract token for the stream and returns a normal Tcl result code; **Tcl_ZlibStreamGetCommandName** returns the name of that command given the stream token, or NULL if the stream has no command. Streams are not designed to be thread-safe; each stream should only ever be used from the thread that created it. When working with gzip streams, a dictionary (fields as given in the **GZIP OPTIONS DICTIONARY** section below) can be given via the *dictObj* parameter that on compression allows control over the generated headers, and on decompression allows discovery of the existing headers. Note that the dictionary will be written to on decompression once sufficient data has been read to have a complete header. This means that the dictionary must be an unshared value in that case; a blank value created with **Tcl_NewObj** is suggested.

Once a stream has been constructed, **Tcl_ZlibStreamPut** is used to add data to the stream and **Tcl_ZlibStreamGet** is used to retrieve data from the stream after processing. Both return normal Tcl result codes and leave an error message in the result of the interpreter that the stream is registered with in the error case (if such a registration has been performed). With **Tcl_ZlibStreamPut**, the data buffer value passed to it should not be modified afterwards. With **Tcl_ZlibStreamGet**, the data buffer value passed to it will have the data bytes appended to it. Internally to the stream, data is kept compressed so as to minimize the cost of buffer space.

Tcl_ZlibStreamChecksum returns the checksum computed over the uncompressed data according to the format, and **Tcl_ZlibStreamEof** returns a boolean value indicating whether the end of the uncompressed data has been reached.

Tcl_ZlibStreamSetCompressionDictionary is used to control the compression dictionary used with the stream, a compression dictionary being an array of bytes (such as might be created with **Tcl_NewByteArrayObj**) that is used to initialize the compression engine rather than leaving it to create it on the fly from the data being compressed. Setting a compression dictionary allows for more efficient compression in the case where the start of the data is highly regular, but it does require both the compressor and the decompressor to agreee on the value to use. Compression dictionaries are only fully supported for zlib-format data; on compression, they must be set before any data is sent in with **Tcl_ZlibStreamPut**, and on decompression they should be set when **Tcl_ZlibStreamGet** produces an **error** with its **-errorcode** set to "**ZLIB NEED_DICT** *code*"; the *code* will be the Adler-32 checksum (see **Tcl_ZlibAdler32**) of the compression dictionary sought. (Note that this is only true for zlib-format streams; gzip streams ignore compression dictionaries as the format specification doesn't permit them, and raw streams just produce a data error if the compression dictionary is missing or incorrect.)

If you wish to clear a stream and reuse it for a new compression or decompression action, **Tcl_ZlibStreamReset** will do this and return a normal Tcl result code to indicate whether it was successful; if the stream is registered with an interpreter, an error message will be left in the interpreter result when this function returns TCL_ERROR. Finally, **Tcl_ZlibStreamClose** will clean up the stream and delete the associated command: using **Tcl_DeleteCommand** on the stream's command is equivalent (when such a command exists).

Gzip options dictionary

The *dictObj* parameter to **Tcl_ZlibDeflate**, **Tcl_ZlibInflate** and **Tcl_ZlibStreamInit** is used to pass a dictionary of options about that is used to describe the gzip header in the compressed data. When creating compressed data, the dictionary is read and when unpacking compressed data the dictionary is written (in which case the *dictObj* parameter must refer to an unshared dictionary value).

The following fields in the dictionary value are understood. All other fields are ignored. No field is required when creating a gzip-format stream.

comment
: This holds the comment field of the header, if present. If absent, no comment was supplied (on decompression) or will be created (on compression).

crc
: A boolean value describing whether a CRC of the header is computed. Note that the **gzip** program does *not* use or allow a CRC on the header.

filename

The name of the file that held the uncompressed data. This should not contain any directory separators, and should be sanitized before use on decompression with **file tail**.

os
The operating system type code field from the header (if not the "unknown" value). See RFC 1952 for the meaning of these codes. On compression, if this is absent then the field will be set to the "unknown" value.

size
The size of the uncompressed data. This is ignored on compression; the size of the data compressed depends on how much data is supplied to the compression engine.

time
The time field from the header if non-zero, expected to be the time that the file named by the **filename** field was modified. Suitable for use with **clock format**. On creation, the right value to use is that from **clock seconds** or **file mtime**.

type
The type of the uncompressed data (either **binary** or **text**) if known.

Portability notes

These functions will fail gracefully if Tcl is not linked with the zlib library.

Keywords: TclZlib, compress, decompress, deflate, gzip, inflate

Copyright (c) 2008 Donal K. Fellows

Tcl_Main

Name

Tcl_Main, Tcl_MainEx, Tcl_MainExW, Tcl_SetStartupScript, Tcl_GetStartupScript, Tcl_SetMainLoop - main program, startup script, and event loop definition for Tcl-based applications

Synopsis

#include <tcl.h>

Tcl_Main(*argc*, *argv*, *appInitProc*)

Tcl_MainEx(*argc*, *charargv*, *appInitProc*, *interp*)

Tcl_MainExW(*argc*, *wideargv*, *appInitProc*, *interp*)

Tcl_SetStartupScript(*path*, *encoding*)

Tcl_Obj *
Tcl_GetStartupScript(*encodingPtr*)

Tcl_SetMainLoop(*mainLoopProc*)

Arguments

int *argc* (in)
Number of elements in *argv*.

char **argv[]* (in)
Array of strings containing command-line arguments. On Windows, when using -DUNICODE, the parameter type changes to wchar_t *.

char **charargv[]* (in)

As argv, but does not change type to wchar_t.

char ***wideargv[]* **(in)
As argv, but type is always wchar_t.

Tcl_AppInitProc ***appInitProc* **(in)
Address of an application-specific initialization procedure. The value for this argument is usually **Tcl_AppInit**.

Tcl_Obj ***path* **(in)
Name of file to use as startup script, or NULL.

const char ***encoding* **(in)
Encoding of file to use as startup script, or NULL.

const char ****encodingPtr* **(out)
If non-NULL, location to write a copy of the (const char *) pointing to the encoding name.

Tcl_MainLoopProc ***mainLoopProc* **(in)
Address of an application-specific event loop procedure.

Tcl_Interp ***interp* **(in)
Already created Tcl Interpreter.

Description

Tcl_Main can serve as the main program for Tcl-based shell applications. A "shell application" is a program like tclsh or wish that supports both interactive interpretation of Tcl and evaluation of a script contained in a file given as a command line argument. **Tcl_Main** is offered as a convenience to developers of shell applications, so they do not have to reproduce all of the code for proper initialization of the Tcl library and interactive shell operation. Other styles of embedding Tcl in an application are not supported by **Tcl_Main**. Those must be achieved by calling lower level functions in the Tcl library directly.

The **Tcl_Main** function has been offered by the Tcl library since release Tcl 7.4. In older releases of Tcl, the Tcl library itself defined a function **main**, but that lacks flexibility of embedding style and having a function **main** in a library (particularly a shared library) causes problems on many systems. Having **main** in the Tcl library would also make it hard to use Tcl in C++ programs, since C++ programs must have special C++ **main** functions.

Normally each shell application contains a small **main** function that does nothing but invoke **Tcl_Main**. **Tcl_Main** then does all the work of creating and running a **tclsh**-like application.

Tcl_Main is not provided by the public interface of Tcl's stub library. Programs that call **Tcl_Main** must be linked against the standard Tcl library. Extensions (stub-enabled or not) are not intended to call **Tcl_Main**.

Tcl_Main is not thread-safe. It should only be called by a single main thread of a multi-threaded application. This restriction is not a problem with normal use described above.

Tcl_Main and therefore all applications based upon it, like **tclsh**, use **Tcl_GetStdChannel** to initialize the standard channels to their default values. See **Tcl_StandardChannels** for more information.

Tcl_Main supports two modes of operation, depending on whether the filename and encoding of a startup script has been established. The routines **Tcl_SetStartupScript** and **Tcl_GetStartupScript** are the tools for controlling this configuration of **Tcl_Main**.

Tcl_SetStartupScript registers the value *path* as the name of the file for **Tcl_Main** to evaluate as its startup script. The value *encoding* is Tcl's name for the encoding used to store the text in that file. A value of **NULL** for *encoding* is a signal to use the system encoding. A value of **NULL** for *path* erases any existing registration so that **Tcl_Main** will not evaluate any startup script.

Tcl_GetStartupScript queries the registered file name and encoding set by the most recent **Tcl_SetStartupScript** call in the same thread. The stored file name is returned, and the stored encoding name is written to space pointed to by *encodingPtr*, when that is not NULL.

The file name and encoding values managed by the routines **Tcl_SetStartupScript** and **Tcl_GetStartupScript** are stored per-thread. Although the storage and retrieval functions of these routines work in any thread, only those calls in the same main thread as **Tcl_Main** can have any influence on it.

The caller of **Tcl_Main** may call **Tcl_SetStartupScript** first to establish its desired startup script. If **Tcl_Main** finds that no such startup script has been established, it consults the first few arguments in *argv*. If they match ?**-encoding** *name*? *fileName*, where *fileName* does not begin with the character -, then *fileName* is taken to be the name of a file containing a *startup script*, and *name* is taken to be the name of the encoding of the contents of that file. **Tcl_Main** then calls **Tcl_SetStartupScript** with these values.

Tcl_Main then defines in its main interpreter the Tcl variables *argc*, *argv*, *argv0*, and *tcl_interactive*, as described in the documentation for **tclsh**.

When it has finished its own initialization, but before it processes commands, **Tcl_Main** calls the procedure given by the *appInitProc* argument. This procedure provides a "hook" for the application to perform its own initialization of the interpreter created by **Tcl_Main**, such as defining application-specific commands. The application initialization routine might also call **Tcl_SetStartupScript** to (re-)set the file and encoding to be used as a startup script. The procedure must have an interface that matches the type **Tcl_AppInitProc**:

```
typedef int Tcl_AppInitProc(
        Tcl_Interp *interp);
```

AppInitProc is almost always a pointer to **Tcl_AppInit**; for more details on this procedure, see the documentation for **Tcl_AppInit**.

When the *appInitProc* is finished, **Tcl_Main** calls **Tcl_GetStartupScript** to determine what startup script has been requested, if any. If a startup script has been provided, **Tcl_Main** attempts to evaluate it. Otherwise, interactive mode begins with examination of the variable *tcl_rcFileName* in the main interpreter. If that variable exists and holds the name of a readable file, the contents of that file are evaluated in the main interpreter. Then interactive operations begin, with prompts and command evaluation results written to the standard output channel, and commands read from the standard input channel and then evaluated. The prompts written to the standard output channel may be customized by defining the Tcl variables *tcl_prompt1* and *tcl_prompt2* as described in the documentation for **tclsh**. The prompts and command evaluation results are written to the standard output channel only if the Tcl variable *tcl_interactive* in the main interpreter holds a non-zero integer value.

Tcl_SetMainLoop allows setting an event loop procedure to be run. This allows, for example, Tk to be dynamically loaded and set its event loop. The event loop will run following the startup script. If you are in interactive mode, setting the main loop procedure will cause the prompt to

become fileevent based and then the loop procedure is called. When the loop procedure returns in interactive mode, interactive operation will continue. The main loop procedure must have an interface that matches the type **Tcl_MainLoopProc**:

```
typedef void Tcl_MainLoopProc(void);
```

Tcl_Main does not return. Normally a program based on **Tcl_Main** will terminate when the **exit** command is evaluated. In interactive mode, if an EOF or channel error is encountered on the standard input channel, then **Tcl_Main** itself will evaluate the **exit** command after the main loop procedure (if any) returns. In non-interactive mode, after **Tcl_Main** evaluates the startup script, and the main loop procedure (if any) returns, **Tcl_Main** will also evaluate the **exit** command.

Keywords: Tcl_Main, application-specific initialization, command-line arguments, main program

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2000 Ajuba Solutions.

Thread

Name

Tcl_ConditionNotify, Tcl_ConditionWait, Tcl_ConditionFinalize, Tcl_GetThreadData, Tcl_MutexLock, Tcl_MutexUnlock, Tcl_MutexFinalize, Tcl_CreateThread, Tcl_JoinThread - Tcl thread support

Synopsis

#include <tcl.h>

void
Tcl_ConditionNotify(*condPtr*)

void
Tcl_ConditionWait(*condPtr*, *mutexPtr*, *timePtr*)

void
Tcl_ConditionFinalize(*condPtr*)

Void *
Tcl_GetThreadData(*keyPtr*, *size*)

void
Tcl_MutexLock(*mutexPtr*)

void
Tcl_MutexUnlock(*mutexPtr*)

void
Tcl_MutexFinalize(*mutexPtr*)

int
Tcl_CreateThread(*idPtr*, *proc*, *clientData*, *stackSize*, *flags*)

int
Tcl_JoinThread(*id*, *result*)

Arguments

Tcl_Condition ****condPtr* **(in)

A condition variable, which must be associated with a mutex lock.

Tcl_Mutex *_mutexPtr_ (in)
A mutex lock.

const Tcl_Time *_timePtr_ (in)
A time limit on the condition wait. NULL to wait forever. Note that a polling value of 0 seconds does not make much sense.

Tcl_ThreadDataKey *_keyPtr_ (in)
This identifies a block of thread local storage. The key should be static and process-wide, yet each thread will end up associating a different block of storage with this key.

int *_size_ (in)
The size of the thread local storage block. This amount of data is allocated and initialized to zero the first time each thread calls **Tcl_GetThreadData**.

Tcl_ThreadId *_idPtr_ (out)
The referred storage will contain the id of the newly created thread as returned by the operating system.

Tcl_ThreadId _id_ (in)
Id of the thread waited upon.

Tcl_ThreadCreateProc *_proc_ (in)
This procedure will act as the **main()** of the newly created thread. The specified *clientData* will be its sole argument.

ClientData _clientData_ (in)
Arbitrary information. Passed as sole argument to the *proc*.

int _stackSize_ (in)
The size of the stack given to the new thread.

int _flags_ (in)
Bitmask containing flags allowing the caller to modify behavior of the new thread.

int *_result_ (out)
The referred storage is used to place the exit code of the thread waited upon into it.

Introduction

Beginning with the 8.1 release, the Tcl core is thread safe, which allows you to incorporate Tcl into multithreaded applications without customizing the Tcl core. Starting with the 8.6 release, Tcl multithreading support is on by default. To disable Tcl multithreading support, you must include the **-\|-disable-threads** option to **configure** when you configure and compile your Tcl core.

An important constraint of the Tcl threads implementation is that *only the thread that created a Tcl interpreter can use that interpreter*. In other words, multiple threads can not access the same Tcl interpreter. (However, a single thread can safely create and use multiple interpreters.)

Description

Tcl provides **Tcl_CreateThread** for creating threads. The caller can determine the size of the stack given to the new thread and modify the behavior through the supplied *flags*. The value **TCL_THREAD_STACK_DEFAULT** for the *stackSize* indicates that the default size as specified by the operating system is to be used for the new thread. As for the flags, currently only the values **TCL_THREAD_NOFLAGS** and **TCL_THREAD_JOINABLE** are defined. The first of them invokes

the default behavior with no special settings. Using the second value marks the new thread as *joinable*. This means that another thread can wait for the such marked thread to exit and join it.

Restrictions: On some UNIX systems the pthread-library does not contain the functionality to specify the stack size of a thread. The specified value for the stack size is ignored on these systems. Windows currently does not support joinable threads. This flag value is therefore ignored on this platform.

Tcl provides the **Tcl_ExitThread** and **Tcl_FinalizeThread** functions for terminating threads and invoking optional per-thread exit handlers. See the **Tcl_Exit** page for more information on these procedures.

The **Tcl_JoinThread** function is provided to allow threads to wait upon the exit of another thread, which must have been marked as joinable through usage of the **TCL_THREAD_JOINABLE**-flag during its creation via **Tcl_CreateThread**.

Trying to wait for the exit of a non-joinable thread or a thread which is already waited upon will result in an error. Waiting for a joinable thread which already exited is possible, the system will retain the necessary information until after the call to **Tcl_JoinThread**. This means that not calling **Tcl_JoinThread** for a joinable thread will cause a memory leak.

The **Tcl_GetThreadData** call returns a pointer to a block of thread-private data. Its argument is a key that is shared by all threads and a size for the block of storage. The storage is automatically allocated and initialized to all zeros the first time each thread asks for it. The storage is automatically deallocated by **Tcl_FinalizeThread**.

Synchronization and communication

Tcl provides **Tcl_ThreadQueueEvent** and **Tcl_ThreadAlert** for handling event queuing in multithreaded applications. See the **Notifier** manual page for more information on these procedures.

A mutex is a lock that is used to serialize all threads through a piece of code by calling **Tcl_MutexLock** and **Tcl_MutexUnlock**. If one thread holds a mutex, any other thread calling **Tcl_MutexLock** will block until **Tcl_MutexUnlock** is called. A mutex can be destroyed after its use by calling **Tcl_MutexFinalize**. The result of locking a mutex twice from the same thread is undefined. On some platforms it will result in a deadlock. The **Tcl_MutexLock**, **Tcl_MutexUnlock** and **Tcl_MutexFinalize** procedures are defined as empty macros if not compiling with threads enabled. For declaration of mutexes the **TCL_DECLARE_MUTEX** macro should be used. This macro assures correct mutex handling even when the core is compiled without threads enabled.

A condition variable is used as a signaling mechanism: a thread can lock a mutex and then wait on a condition variable with **Tcl_ConditionWait**. This atomically releases the mutex lock and blocks the waiting thread until another thread calls **Tcl_ConditionNotify**. The caller of **Tcl_ConditionNotify** should have the associated mutex held by previously calling **Tcl_MutexLock**, but this is not enforced. Notifying the condition variable unblocks all threads waiting on the condition variable, but they do not proceed until the mutex is released with **Tcl_MutexUnlock**. The implementation of **Tcl_ConditionWait** automatically locks the mutex before returning.

The caller of **Tcl_ConditionWait** should be prepared for spurious notifications by calling **Tcl_ConditionWait** within a while loop that tests some invariant.

A condition variable can be destroyed after its use by calling **Tcl_ConditionFinalize**.

The **Tcl_ConditionNotify**, **Tcl_ConditionWait** and **Tcl_ConditionFinalize** procedures are defined as empty macros if not compiling with threads enabled.

Initialization

All of these synchronization objects are self-initializing. They are implemented as opaque pointers that should be NULL upon first use. The mutexes and condition variables are either cleaned up by process exit handlers (if living that long) or explicitly by calls to **Tcl_MutexFinalize** or **Tcl_ConditionFinalize**. Thread local storage is reclaimed during **Tcl_FinalizeThread**.

Script-level access to threads

Tcl provides no built-in commands for scripts to use to create, manage, or join threads, nor any script-level access to mutex or condition variables. It provides such facilities only via C interfaces, and leaves it up to packages to expose these matters to the script level. One such package is the **Thread** package.

Example

To create a thread with portable code, its implementation function should be declared as follows:

```
static Tcl_ThreadCreateProc MyThreadImplFunc;
```

It should then be defined like this example, which just counts up to a given value and then finishes.

```
static Tcl_ThreadCreateType
MyThreadImplFunc(
    ClientData clientData)
{
    int i, limit = (int) clientData;
    for (i=0 ; i<limit ; i++) {
        /* doing nothing at all here */
    }
    TCL_THREAD_CREATE_RETURN;
}
```

To create the above thread, make it execute, and wait for it to finish, we would do this:

```
int limit = 1000000000;
ClientData limitData = (void*)((intptr_t) limit);
Tcl_ThreadId id;    /* holds identity of thread created */
int result;

if (Tcl_CreateThread(&id, MyThreadImplFunc, limitData,
        TCL_THREAD_STACK_DEFAULT,
        TCL_THREAD_JOINABLE) != TCL_OK) {
    /* Thread did not create correctly */
    return;
}
```

```
/* Do something else for a while here */
if (Tcl_JoinThread(id, &result) != TCL_OK) {
    /* Thread did not finish properly */
    return;
}
/* All cleaned up nicely */
```

Keywords: Thread, mutex, condition variable, thread local storage

Copyright (c) 1999 Scriptics Corporation
Copyright (c) 1998 Sun Microsystems, Inc.

ToUpper

Name

Tcl_UniCharToUpper, Tcl_UniCharToLower, Tcl_UniCharToTitle, Tcl_UtfToUpper, Tcl_UtfToLower, Tcl_UtfToTitle - routines for manipulating the case of Unicode characters and UTF-8 strings

Synopsis

#include <tcl.h>

Tcl_UniChar
Tcl_UniCharToUpper(*ch*)

Tcl_UniChar
Tcl_UniCharToLower(*ch*)

Tcl_UniChar
Tcl_UniCharToTitle(*ch*)

int
Tcl_UtfToUpper(*str*)

int
Tcl_UtfToLower(*str*)

int
Tcl_UtfToTitle(*str*)

Arguments

int *ch* (in)
The Unicode character to be converted.

char **str* (in/out)
Pointer to UTF-8 string to be converted in place.

Description

The first three routines convert the case of individual Unicode characters:

If *ch* represents a lower-case character, **Tcl_UniCharToUpper** returns the corresponding upper-case character. If no upper-case character is defined, it returns the character unchanged.

If *ch* represents an upper-case character, **Tcl_UniCharToLower** returns the corresponding lower-case character. If no lower-case character is defined, it returns the character unchanged.

If *ch* represents a lower-case character, **Tcl_UniCharToTitle** returns the corresponding title-case character. If no title-case character is defined, it returns the corresponding upper-case character. If no upper-case character is defined, it returns the character unchanged. Title-case is defined for a small number of characters that have a different appearance when they are at the beginning of a capitalized word.

The next three routines convert the case of UTF-8 strings in place in memory:

Tcl_UtfToUpper changes every UTF-8 character in *str* to upper-case. Because changing the case of a character may change its size, the byte offset of each character in the resulting string may differ from its original location. **Tcl_UtfToUpper** writes a null byte at the end of the converted string. **Tcl_UtfToUpper** returns the new length of the string in bytes. This new length is guaranteed to be no longer than the original string length.

Tcl_UtfToLower is the same as **Tcl_UtfToUpper** except it turns each character in the string into its lower-case equivalent.

Tcl_UtfToTitle is the same as **Tcl_UtfToUpper** except it turns the first character in the string into its title-case equivalent and all following characters into their lower-case equivalents.

Bugs

At this time, the case conversions are only defined for the Unicode plane 0 characters. The result for Unicode characters above 0xFFFF is undefined, but - actually - only the lower 16 bits of the character value is handled.

Keywords: ToUpper, utf, unicode, tolower, totitle, case

Copyright (c) 1997 Sun Microsystems, Inc.

TraceCmd

Name

Tcl_CommandTraceInfo, Tcl_TraceCommand, Tcl_UntraceCommand - monitor renames and deletes of a command

Synopsis

#include <tcl.h>

ClientData
Tcl_CommandTraceInfo(*interp*, *cmdName*, *flags*, *proc*, *prevClientData*)

int
Tcl_TraceCommand(*interp*, *cmdName*, *flags*, *proc*, *clientData*)

void
Tcl_UntraceCommand(*interp*, *cmdName*, *flags*, *proc*, *clientData*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter containing the command.

const char *_cmdName_ (in)
Name of command.

int _flags_ (in)

OR'ed collection of the values **TCL_TRACE_RENAME** and **TCL_TRACE_DELETE**.

Tcl_CommandTraceProc ****proc* **(in)**
Procedure to call when specified operations occur to *cmdName*.

ClientData *clientData* (in)
Arbitrary argument to pass to *proc*.

ClientData *prevClientData* (in)
If non-NULL, gives last value returned by **Tcl_CommandTraceInfo**, so this call will return information about next trace. If NULL, this call will return information about first trace.

Description

Tcl_TraceCommand allows a C procedure to monitor operations performed on a Tcl command, so that the C procedure is invoked whenever the command is renamed or deleted. If the trace is created successfully then **Tcl_TraceCommand** returns **TCL_OK**. If an error occurred (e.g. *cmdName* specifies a non-existent command) then **TCL_ERROR** is returned and an error message is left in the interpreter's result.

The *flags* argument to **Tcl_TraceCommand** indicates when the trace procedure is to be invoked. It consists of an OR'ed combination of any of the following values:

TCL_TRACE_RENAME
Invoke *proc* whenever the command is renamed.

TCL_TRACE_DELETE
Invoke *proc* when the command is deleted. Whenever one of the specified operations occurs to the command, *proc* will be invoked. It should have arguments and result that match the type **Tcl_CommandTraceProc**:

```
typedef void Tcl_CommandTraceProc(
        ClientData clientData,
        Tcl_Interp *interp,
        const char *oldName,
        const char *newName,
        int flags);
```

The *clientData* and *interp* parameters will have the same values as those passed to **Tcl_TraceCommand** when the trace was created. *ClientData* typically points to an application-specific data structure that describes what to do when *proc* is invoked. *OldName* gives the name of the command being renamed, and *newName* gives the name that the command is being renamed to (or NULL when the command is being deleted.) *Flags* is an OR'ed combination of bits potentially providing several pieces of information. One of the bits **TCL_TRACE_RENAME** and **TCL_TRACE_DELETE** will be set in *flags* to indicate which operation is being performed on the command. The bit **TCL_TRACE_DESTROYED** will be set in *flags* if the trace is about to be destroyed; this information may be useful to *proc* so that it can clean up its own internal data structures (see the section **TCL_TRACE_DESTROYED** below for more details). Because the deletion of commands can take place as part of the deletion of the interp that contains them, *proc* must be careful about checking what the passed in *interp* value can be called upon to do. The routine **Tcl_InterpDeleted** is an important tool for this. When **Tcl_InterpDeleted** returns 1, *proc* will not be able to invoke any scripts in *interp*. The function of *proc* in that circumstance is limited to the cleanup of its own data structures.

Tcl_UntraceCommand may be used to remove a trace. If the command specified by *interp*, *cmdName*, and *flags* has a trace set with *flags*, *proc*, and *clientData*, then the corresponding trace is removed. If no such trace exists, then the call to **Tcl_UntraceCommand** has no effect. The same bits are valid for *flags* as for calls to **Tcl_TraceCommand**.

Tcl_CommandTraceInfo may be used to retrieve information about traces set on a given command. The return value from **Tcl_CommandTraceInfo** is the *clientData* associated with a particular trace. The trace must be on the command specified by the *interp*, *cmdName*, and *flags* arguments (note that currently the flags are ignored; *flags* should be set to 0 for future compatibility) and its trace procedure must the same as the *proc* argument. If the *prevClientData* argument is NULL then the return value corresponds to the first (most recently created) matching trace, or NULL if there are no matching traces. If the *prevClientData* argument is not NULL, then it should be the return value from a previous call to **Tcl_CommandTraceInfo**. In this case, the new return value will correspond to the next matching trace after the one whose *clientData* matches *prevClientData*, or NULL if no trace matches *prevClientData* or if there are no more matching traces after it. This mechanism makes it possible to step through all of the traces for a given command that have the same *proc*.

Calling commands during traces

During rename traces, the command being renamed is visible with both names simultaneously, and the command still exists during delete traces, unless the interp that contains it is being deleted. However, there is no mechanism for signaling that an error occurred in a trace procedure, so great care should be taken that errors do not get silently lost.

Multiple traces

It is possible for multiple traces to exist on the same command. When this happens, all of the trace procedures will be invoked on each access, in order from most-recently-created to least-recently-created. Attempts to delete the command during a delete trace will fail silently, since the command is already scheduled for deletion anyway. If the command being renamed is renamed by one of its rename traces, that renaming takes precedence over the one that triggered the trace and the collection of traces will not be reexecuted; if several traces rename the command, the last renaming takes precedence.

Tcl_trace_destroyed flag

In a delete callback to *proc*, the **TCL_TRACE_DESTROYED** bit is set in *flags*.

Keywords: TraceCmd, clientData, trace, command

Copyright (c) 2002 Donal K. Fellows

TraceVar

Name

Tcl_TraceVar, Tcl_TraceVar2, Tcl_UntraceVar, Tcl_UntraceVar2, Tcl_VarTraceInfo, Tcl_VarTraceInfo2 - monitor accesses to a variable

Synopsis

#include <tcl.h>

int

Tcl_TraceVar(*interp*, *varName*, *flags*, *proc*, *clientData*)

int
Tcl_TraceVar2(*interp*, *name1*, *name2*, *flags*, *proc*, *clientData*)

Tcl_UntraceVar(*interp*, *varName*, *flags*, *proc*, *clientData*)

Tcl_UntraceVar2(*interp*, *name1*, *name2*, *flags*, *proc*, *clientData*)

ClientData
Tcl_VarTraceInfo(*interp*, *varName*, *flags*, *proc*, *prevClientData*)

ClientData
Tcl_VarTraceInfo2(*interp*, *name1*, *name2*, *flags*, *proc*, *prevClientData*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter containing variable.

const char *_varName_ (in)
Name of variable. May refer to a scalar variable, to an array variable with no index, or to an array variable with a parenthesized index.

int _flags_ (in)
OR-ed combination of the values **TCL_TRACE_READS**, **TCL_TRACE_WRITES**, **TCL_TRACE_UNSETS**, **TCL_TRACE_ARRAY**, **TCL_GLOBAL_ONLY**, **TCL_NAMESPACE_ONLY**, **TCL_TRACE_RESULT_DYNAMIC** and **TCL_TRACE_RESULT_OBJECT**. Not all flags are used by all procedures. See below for more information.

Tcl_VarTraceProc *_proc_ (in)
Procedure to invoke whenever one of the traced operations occurs.

ClientData _clientData_ (in)
Arbitrary one-word value to pass to *proc*.

const char *_name1_ (in)
Name of scalar or array variable (without array index).

const char *_name2_ (in)
For a trace on an element of an array, gives the index of the element. For traces on scalar variables or on whole arrays, is NULL.

ClientData _prevClientData_ (in)
If non-NULL, gives last value returned by **Tcl_VarTraceInfo** or **Tcl_VarTraceInfo2**, so this call will return information about next trace. If NULL, this call will return information about first trace.

Description

Tcl_TraceVar allows a C procedure to monitor and control access to a Tcl variable, so that the C procedure is invoked whenever the variable is read or written or unset. If the trace is created successfully then **Tcl_TraceVar** returns **TCL_OK**. If an error occurred (e.g. *varName* specifies an element of an array, but the actual variable is not an array) then **TCL_ERROR** is returned and an error message is left in the interpreter's result.

The *flags* argument to **Tcl_TraceVar** indicates when the trace procedure is to be invoked and provides information for setting up the trace. It consists of an OR-ed combination of any of the following values:

TCL_GLOBAL_ONLY
Normally, the variable will be looked up at the current level of procedure call; if this bit is set then the variable will be looked up at global level, ignoring any active procedures.

TCL_NAMESPACE_ONLY
Normally, the variable will be looked up at the current level of procedure call; if this bit is set then the variable will be looked up in the current namespace, ignoring any active procedures.

TCL_TRACE_READS
Invoke *proc* whenever an attempt is made to read the variable.

TCL_TRACE_WRITES
Invoke *proc* whenever an attempt is made to modify the variable.

TCL_TRACE_UNSETS
Invoke *proc* whenever the variable is unset. A variable may be unset either explicitly by an **unset** command, or implicitly when a procedure returns (its local variables are automatically unset) or when the interpreter is deleted (all variables are automatically unset).

TCL_TRACE_ARRAY
Invoke *proc* whenever the array command is invoked. This gives the trace procedure a chance to update the array before array names or array get is called. Note that this is called before an array set, but that will trigger write traces.

TCL_TRACE_RESULT_DYNAMIC
The result of invoking the *proc* is a dynamically allocated string that will be released by the Tcl library via a call to **ckfree**. Must not be specified at the same time as **TCL_TRACE_RESULT_OBJECT**.

TCL_TRACE_RESULT_OBJECT
The result of invoking the *proc* is a Tcl_Obj* (cast to a char*) with a reference count of at least one. The ownership of that reference will be transferred to the Tcl core for release (when the core has finished with it) via a call to **Tcl_DecrRefCount**. Must not be specified at the same time as **TCL_TRACE_RESULT_DYNAMIC**. Whenever one of the specified operations occurs on the variable, *proc* will be invoked. It should have arguments and result that match the type **Tcl_VarTraceProc**:

```
typedef char *Tcl_VarTraceProc(
        ClientData clientData,
        Tcl_Interp *interp,
        const char *name1,
        const char *name2,
        int flags);
```

The *clientData* and *interp* parameters will have the same values as those passed to **Tcl_TraceVar** when the trace was created. *ClientData* typically points to an application-specific data structure that describes what to do when *proc* is invoked. *Name1* and *name2* give the name of the traced variable in the normal two-part form (see the description of **Tcl_TraceVar2** below for details). *Flags* is an OR-ed combination of bits providing several pieces of information. One of the bits **TCL_TRACE_READS**, **TCL_TRACE_WRITES**, **TCL_TRACE_ARRAY**, or **TCL_TRACE_UNSETS** will be set in *flags* to indicate which operation is being performed on the variable. The bit **TCL_GLOBAL_ONLY** will be set whenever the variable being accessed is a global one not accessible from the current level of procedure call: the trace procedure will need

to pass this flag back to variable-related procedures like **Tcl_GetVar** if it attempts to access the variable. The bit **TCL_NAMESPACE_ONLY** will be set whenever the variable being accessed is a namespace one not accessible from the current level of procedure call: the trace procedure will need to pass this flag back to variable-related procedures like **Tcl_GetVar** if it attempts to access the variable. The bit **TCL_TRACE_DESTROYED** will be set in *flags* if the trace is about to be destroyed; this information may be useful to *proc* so that it can clean up its own internal data structures (see the section **TCL_TRACE_DESTROYED** below for more details). Lastly, the bit **TCL_INTERP_DESTROYED** will be set if the entire interpreter is being destroyed. When this bit is set, *proc* must be especially careful in the things it does (see the section **TCL_INTERP_DESTROYED** below). The trace procedure's return value should normally be NULL; see **ERROR RETURNS** below for information on other possibilities.

Tcl_UntraceVar may be used to remove a trace. If the variable specified by *interp*, *varName*, and *flags* has a trace set with *flags*, *proc*, and *clientData*, then the corresponding trace is removed. If no such trace exists, then the call to **Tcl_UntraceVar** has no effect. The same bits are valid for *flags* as for calls to **Tcl_TraceVar**.

Tcl_VarTraceInfo may be used to retrieve information about traces set on a given variable. The return value from **Tcl_VarTraceInfo** is the *clientData* associated with a particular trace. The trace must be on the variable specified by the *interp*, *varName*, and *flags* arguments (only the **TCL_GLOBAL_ONLY** and **TCL_NAMESPACE_ONLY** bits from *flags* is used; other bits are ignored) and its trace procedure must the same as the *proc* argument. If the *prevClientData* argument is NULL then the return value corresponds to the first (most recently created) matching trace, or NULL if there are no matching traces. If the *prevClientData* argument is not NULL, then it should be the return value from a previous call to **Tcl_VarTraceInfo**. In this case, the new return value will correspond to the next matching trace after the one whose *clientData* matches *prevClientData*, or NULL if no trace matches *prevClientData* or if there are no more matching traces after it. This mechanism makes it possible to step through all of the traces for a given variable that have the same *proc*.

Two-part names

The procedures **Tcl_TraceVar2**, **Tcl_UntraceVar2**, and **Tcl_VarTraceInfo2** are identical to **Tcl_TraceVar**, **Tcl_UntraceVar**, and **Tcl_VarTraceInfo**, respectively, except that the name of the variable consists of two parts. *Name1* gives the name of a scalar variable or array, and *name2* gives the name of an element within an array. When *name2* is NULL, *name1* may contain both an array and an element name: if the name contains an open parenthesis and ends with a close parenthesis, then the value between the parentheses is treated as an element name (which can have any string value) and the characters before the first open parenthesis are treated as the name of an array variable. If *name2* is NULL and *name1* does not refer to an array element it means that either the variable is a scalar or the trace is to be set on the entire array rather than an individual element (see WHOLE-ARRAY TRACES below for more information).

Accessing variables during traces

During read, write, and array traces, the trace procedure can read, write, or unset the traced variable using **Tcl_GetVar2**, **Tcl_SetVar2**, and other procedures. While *proc* is executing, traces are temporarily disabled for the variable, so that calls to **Tcl_GetVar2** and **Tcl_SetVar2** will not cause *proc* or other trace procedures to be invoked again. Disabling only occurs for the variable whose trace procedure is active; accesses to other variables will still be traced. However, if a

variable is unset during a read or write trace then unset traces will be invoked.

During unset traces the variable has already been completely expunged. It is possible for the trace procedure to read or write the variable, but this will be a new version of the variable. Traces are not disabled during unset traces as they are for read and write traces, but existing traces have been removed from the variable before any trace procedures are invoked. If new traces are set by unset trace procedures, these traces will be invoked on accesses to the variable by the trace procedures.

Callback timing

When read tracing has been specified for a variable, the trace procedure will be invoked whenever the variable's value is read. This includes **set** Tcl commands, **$**-notation in Tcl commands, and invocations of the **Tcl_GetVar** and **Tcl_GetVar2** procedures. *Proc* is invoked just before the variable's value is returned. It may modify the value of the variable to affect what is returned by the traced access. If it unsets the variable then the access will return an error just as if the variable never existed.

When write tracing has been specified for a variable, the trace procedure will be invoked whenever the variable's value is modified. This includes **set** commands, commands that modify variables as side effects (such as **catch** and **scan**), and calls to the **Tcl_SetVar** and **Tcl_SetVar2** procedures). *Proc* will be invoked after the variable's value has been modified, but before the new value of the variable has been returned. It may modify the value of the variable to override the change and to determine the value actually returned by the traced access. If it deletes the variable then the traced access will return an empty string.

When array tracing has been specified, the trace procedure will be invoked at the beginning of the array command implementation, before any of the operations like get, set, or names have been invoked. The trace procedure can modify the array elements with **Tcl_SetVar** and **Tcl_SetVar2**.

When unset tracing has been specified, the trace procedure will be invoked whenever the variable is destroyed. The traces will be called after the variable has been completely unset.

Whole-array traces

If a call to **Tcl_TraceVar** or **Tcl_TraceVar2** specifies the name of an array variable without an index into the array, then the trace will be set on the array as a whole. This means that *proc* will be invoked whenever any element of the array is accessed in the ways specified by *flags*. When an array is unset, a whole-array trace will be invoked just once, with *name1* equal to the name of the array and *name2* NULL; it will not be invoked once for each element.

Multiple traces

It is possible for multiple traces to exist on the same variable. When this happens, all of the trace procedures will be invoked on each access, in order from most-recently-created to least-recently-created. When there exist whole-array traces for an array as well as traces on individual elements, the whole-array traces are invoked before the individual-element traces. If a read or write trace unsets the variable then all of the unset traces will be invoked but the remainder of the read and write traces will be skipped.

Error returns

Under normal conditions trace procedures should return NULL, indicating successful completion.

If *proc* returns a non-NULL value it signifies that an error occurred. The return value must be a pointer to a static character string containing an error message, unless (*exactly* one of) the **TCL_TRACE_RESULT_DYNAMIC** and **TCL_TRACE_RESULT_OBJECT** flags is set, which specify that the result is either a dynamic string (to be released with **ckfree**) or a Tcl_Obj* (cast to char* and to be released with **Tcl_DecrRefCount**) containing the error message. If a trace procedure returns an error, no further traces are invoked for the access and the traced access aborts with the given message. Trace procedures can use this facility to make variables read-only, for example (but note that the value of the variable will already have been modified before the trace procedure is called, so the trace procedure will have to restore the correct value).

The return value from *proc* is only used during read and write tracing. During unset traces, the return value is ignored and all relevant trace procedures will always be invoked.

Restrictions

A trace procedure can be called at any time, even when there are partially formed results stored in the interpreter. If the trace procedure does anything that could damage this result (such as calling **Tcl_Eval**) then it must use the **Tcl_SaveInterpState** and related routines to save and restore the original state of the interpreter before it returns.

Undefined variables

It is legal to set a trace on an undefined variable. The variable will still appear to be undefined until the first time its value is set. If an undefined variable is traced and then unset, the unset will fail with an error ("no such variable"), but the trace procedure will still be invoked.

Tcl_trace_destroyed flag

In an unset callback to *proc*, the **TCL_TRACE_DESTROYED** bit is set in *flags* if the trace is being removed as part of the deletion. Traces on a variable are always removed whenever the variable is deleted; the only time **TCL_TRACE_DESTROYED** is not set is for a whole-array trace invoked when only a single element of an array is unset.

Tcl_interp_destroyed

When an interpreter is destroyed, unset traces are called for all of its variables. The **TCL_INTERP_DESTROYED** bit will be set in the *flags* argument passed to the trace procedures. Trace procedures must be extremely careful in what they do if the **TCL_INTERP_DESTROYED** bit is set. It is not safe for the procedures to invoke any Tcl procedures on the interpreter, since its state is partially deleted. All that trace procedures should do under these circumstances is to clean up and free their own internal data structures.

Bugs

Tcl does not do any error checking to prevent trace procedures from misusing the interpreter during traces with **TCL_INTERP_DESTROYED** set.

Array traces are not yet integrated with the Tcl **info exists** command, nor is there Tcl-level access to array traces.

Keywords: TraceVar, clientData, trace, variable

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

Translate

Name

Tcl_TranslateFileName - convert file name to native form and replace tilde with home directory

Synopsis

#include <tcl.h>

char *
Tcl_TranslateFileName(*interp*, *name*, *bufferPtr*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter in which to report an error, if any.

const char *_name_ (in)
File name, which may start with a "~".

Tcl_DString *_bufferPtr_ (in/out)
If needed, this dynamic string is used to store the new file name. At the time of the call it should be uninitialized or free. The caller must eventually call **Tcl_DStringFree** to free up anything stored here.

Description

This utility procedure translates a file name to a platform-specific form which, after being converted to the appropriate encoding, is suitable for passing to the local operating system. In particular, it converts network names into native form and does tilde substitution.

However, with the advent of the newer **Tcl_FSGetNormalizedPath** and **Tcl_FSGetNativePath**, there is no longer any need to use this procedure. In particular, **Tcl_FSGetNativePath** performs all the necessary translation and encoding conversion, is virtual-filesystem aware, and caches the native result for faster repeated calls. Finally **Tcl_FSGetNativePath** does not require you to free anything afterwards.

If **Tcl_TranslateFileName** has to do tilde substitution or translate the name then it uses the dynamic string at *_bufferPtr_ to hold the new string it generates. After **Tcl_TranslateFileName** returns a non-NULL result, the caller must eventually invoke **Tcl_DStringFree** to free any information placed in *_bufferPtr_. The caller need not know whether or not **Tcl_TranslateFileName** actually used the string; **Tcl_TranslateFileName** initializes *_bufferPtr_ even if it does not use it, so the call to **Tcl_DStringFree** will be safe in either case.

If an error occurs (e.g. because there was no user by the given name) then NULL is returned and an error message will be left in the interpreter's result. When an error occurs, **Tcl_TranslateFileName** frees the dynamic string itself so that the caller need not call **Tcl_DStringFree**.

The caller is responsible for making sure that the interpreter's result has its default empty value when **Tcl_TranslateFileName** is invoked.

Keywords: Translate, file name, home directory, tilde, user

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1998 Sun Microsystems, Inc.

UniCharIsAlpha

Name

Tcl_UniCharIsAlnum, Tcl_UniCharIsAlpha, Tcl_UniCharIsControl, Tcl_UniCharIsDigit, Tcl_UniCharIsGraph, Tcl_UniCharIsLower, Tcl_UniCharIsPrint, Tcl_UniCharIsPunct, Tcl_UniCharIsSpace, Tcl_UniCharIsUpper, Tcl_UniCharIsWordChar - routines for classification of Tcl_UniChar characters

Synopsis

#include <tcl.h>

int
Tcl_UniCharIsAlnum(*ch*)

int
Tcl_UniCharIsAlpha(*ch*)

int
Tcl_UniCharIsControl(*ch*)

int
Tcl_UniCharIsDigit(*ch*)

int
Tcl_UniCharIsGraph(*ch*)

int
Tcl_UniCharIsLower(*ch*)

int
Tcl_UniCharIsPrint(*ch*)

int
Tcl_UniCharIsPunct(*ch*)

int
Tcl_UniCharIsSpace(*ch*)

int
Tcl_UniCharIsUpper(*ch*)

int
Tcl_UniCharIsWordChar(*ch*)

Arguments

int *ch* (in)
The Tcl_UniChar to be examined.

Description

All of the routines described examine Unicode characters and return a boolean value. A non-zero return value means that the character does belong to the character class associated with the called routine. The rest of this document just describes the character classes associated with the various routines.

Character classes

Tcl_UniCharIsAlnum tests if the character is an alphanumeric Unicode character.

Tcl_UniCharIsAlpha tests if the character is an alphabetic Unicode character.

Tcl_UniCharIsControl tests if the character is a Unicode control character.

Tcl_UniCharIsDigit tests if the character is a numeric Unicode character.

Tcl_UniCharIsGraph tests if the character is any Unicode print character except space.

Tcl_UniCharIsLower tests if the character is a lowercase Unicode character.

Tcl_UniCharIsPrint tests if the character is a Unicode print character.

Tcl_UniCharIsPunct tests if the character is a Unicode punctuation character.

Tcl_UniCharIsSpace tests if the character is a whitespace Unicode character.

Tcl_UniCharIsUpper tests if the character is an uppercase Unicode character.

Tcl_UniCharIsWordChar tests if the character is alphanumeric or a connector punctuation mark.

Keywords: UniCharIsAlpha, unicode, classification

Copyright (c) 1997 Sun Microsystems, Inc.

UpVar

Name

Tcl_UpVar, Tcl_UpVar2 - link one variable to another

Synopsis

#include <tcl.h>

int
Tcl_UpVar(*interp*, *frameName*, *sourceName*, *destName*, *flags*)

int
Tcl_UpVar2(*interp*, *frameName*, *name1*, *name2*, *destName*, *flags*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter containing variables; also used for error reporting.

const char *_frameName_ (in)
Identifies the stack frame containing source variable. May have any of the forms accepted by the **upvar** command, such as **#0** or **1**.

const char *_sourceName_ (in)
Name of source variable, in the frame given by *frameName*. May refer to a scalar variable or to an array variable with a parenthesized index.

const char *_destName_ (in)
Name of destination variable, which is to be linked to source variable so that references to *destName* refer to the other variable. Must not currently exist except as an upvar-ed variable.

int _flags_ (in)
One of **TCL_GLOBAL_ONLY**, **TCL_NAMESPACE_ONLY** or 0; if non-zero, then *destName* is a

global or namespace variable; otherwise it is local to the current procedure (or current namespace if no procedure is active).

const char *_name1_ (in)
First part of source variable's name (scalar name, or name of array without array index).

const char *_name2_ (in)
If source variable is an element of an array, gives the index of the element. For scalar source variables, is NULL.

Description

Tcl_UpVar and **Tcl_UpVar2** provide the same functionality as the **upvar** command: they make a link from a source variable to a destination variable, so that references to the destination are passed transparently through to the source. The name of the source variable may be specified either as a single string such as **xyx** or **a(24)** (by calling **Tcl_UpVar**) or in two parts where the array name has been separated from the element name (by calling **Tcl_UpVar2**). The destination variable name is specified in a single string; it may not be an array element.

Both procedures return either **TCL_OK** or **TCL_ERROR**, and they leave an error message in the interpreter's result if an error occurs.

As with the **upvar** command, the source variable need not exist; if it does exist, unsetting it later does not destroy the link. The destination variable may exist at the time of the call, but if so it must exist as a linked variable.

Keywords: UpVar, linked variable, variable

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

Utf

Name

Tcl_UniChar, Tcl_UniCharToUtf, Tcl_UtfToUniChar, Tcl_UniCharToUtfDString, Tcl_UtfToUniCharDString, Tcl_UniCharLen, Tcl_UniCharNcmp, Tcl_UniCharNcasecmp, Tcl_UniCharCaseMatch, Tcl_UtfNcmp, Tcl_UtfNcasecmp, Tcl_UtfCharComplete, Tcl_NumUtfChars, Tcl_UtfFindFirst, Tcl_UtfFindLast, Tcl_UtfNext, Tcl_UtfPrev, Tcl_UniCharAtIndex, Tcl_UtfAtIndex, Tcl_UtfBackslash - routines for manipulating UTF-8 strings

Synopsis

#include <tcl.h>

typedef ... **Tcl_UniChar**;

int
Tcl_UniCharToUtf(*ch*, *buf*)

int
Tcl_UtfToUniChar(*src*, *chPtr*)

char *
Tcl_UniCharToUtfDString(*uniStr*, *uniLength*, *dsPtr*)

Tcl_UniChar *
Tcl_UtfToUniCharDString(*src*, *length*, *dsPtr*)

int
Tcl_UniCharLen(*uniStr*)

int
Tcl_UniCharNcmp(*ucs*, *uct*, *numChars*)

int
Tcl_UniCharNcasecmp(*ucs*, *uct*, *numChars*)

int
Tcl_UniCharCaseMatch(*uniStr*, *uniPattern*, *nocase*)

int
Tcl_UtfNcmp(*cs*, *ct*, *numChars*)

int
Tcl_UtfNcasecmp(*cs*, *ct*, *numChars*)

int
Tcl_UtfCharComplete(*src*, *length*)

int
Tcl_NumUtfChars(*src*, *length*)

const char *
Tcl_UtfFindFirst(*src*, *ch*)

const char *
Tcl_UtfFindLast(*src*, *ch*)

const char *
Tcl_UtfNext(*src*)

const char *
Tcl_UtfPrev(*src*, *start*)

Tcl_UniChar
Tcl_UniCharAtIndex(*src*, *index*)

const char *
Tcl_UtfAtIndex(*src*, *index*)

int
Tcl_UtfBackslash(*src*, *readPtr*, *dst*)

Arguments

char ***_buf_ **(out)**
Buffer in which the UTF-8 representation of the Tcl_UniChar is stored. At most **TCL_UTF_MAX** bytes are stored in the buffer.

int _ch_ (in)
The Unicode character to be converted or examined.

Tcl_UniChar ***_chPtr_ **(out)**
Filled with the Tcl_UniChar represented by the head of the UTF-8 string.

const char ***_src_ **(in)**
Pointer to a UTF-8 string.

const char ***_cs_ **(in)**

Pointer to a UTF-8 string.

const char *_ct_ (in)
Pointer to a UTF-8 string.

const Tcl_UniChar *_uniStr_ (in)
A null-terminated Unicode string.

const Tcl_UniChar *_ucs_ (in)
A null-terminated Unicode string.

const Tcl_UniChar *_uct_ (in)
A null-terminated Unicode string.

const Tcl_UniChar *_uniPattern_ (in)
A null-terminated Unicode string.

int _length_ (in)
The length of the UTF-8 string in bytes (not UTF-8 characters). If negative, all bytes up to the first null byte are used.

int _uniLength_ (in)
The length of the Unicode string in characters. Must be greater than or equal to 0.

Tcl_DString *_dsPtr_ (in/out)
A pointer to a previously initialized **Tcl_DString**.

unsigned long _numChars_ (in)
The number of characters to compare.

const char *_start_ (in)
Pointer to the beginning of a UTF-8 string.

int _index_ (in)
The index of a character (not byte) in the UTF-8 string.

int *_readPtr_ (out)
If non-NULL, filled with the number of bytes in the backslash sequence, including the backslash character.

char *_dst_ (out)
Buffer in which the bytes represented by the backslash sequence are stored. At most **TCL_UTF_MAX** bytes are stored in the buffer.

int _nocase_ (in)
Specifies whether the match should be done case-sensitive (0) or case-insensitive (1).

Description

These routines convert between UTF-8 strings and Tcl_UniChars. A Tcl_UniChar is a Unicode character represented as an unsigned, fixed-size quantity. A UTF-8 character is a Unicode character represented as a varying-length sequence of up to **TCL_UTF_MAX** bytes. A multibyte UTF-8 sequence consists of a lead byte followed by some number of trail bytes.

TCL_UTF_MAX is the maximum number of bytes that it takes to represent one Unicode character in the UTF-8 representation.

Tcl_UniCharToUtf stores the Tcl_UniChar *ch* as a UTF-8 string in starting at *buf*. The return value is the number of bytes stored in *buf*.

Tcl_UtfToUniChar reads one UTF-8 character starting at *src* and stores it as a Tcl_UniChar in **chPtr*. The return value is the number of bytes read from *src*. The caller must ensure that the

source buffer is long enough such that this routine does not run off the end and dereference non-existent or random memory; if the source buffer is known to be null-terminated, this will not happen. If the input is not in proper UTF-8 format, **Tcl_UtfToUniChar** will store the first byte of *src* in **chPtr* as a Tcl_UniChar between 0x80 and 0xFF and return 1.

Tcl_UniCharToUtfDString converts the given Unicode string to UTF-8, storing the result in a previously initialized **Tcl_DString**. You must specify *uniLength*, the length of the given Unicode string. The return value is a pointer to the UTF-8 representation of the Unicode string. Storage for the return value is appended to the end of the **Tcl_DString**.

Tcl_UtfToUniCharDString converts the given UTF-8 string to Unicode, storing the result in the previously initialized **Tcl_DString**. In the argument *length*, you may either specify the length of the given UTF-8 string in bytes or "-1", in which case **Tcl_UtfToUniCharDString** uses **strlen** to calculate the length. The return value is a pointer to the Unicode representation of the UTF-8 string. Storage for the return value is appended to the end of the **Tcl_DString**. The Unicode string is terminated with a Unicode null character.

Tcl_UniCharLen corresponds to **strlen** for Unicode characters. It accepts a null-terminated Unicode string and returns the number of Unicode characters (not bytes) in that string.

Tcl_UniCharNcmp and **Tcl_UniCharNcasecmp** correspond to **strncmp** and **strncasecmp**, respectively, for Unicode characters. They accept two null-terminated Unicode strings and the number of characters to compare. Both strings are assumed to be at least *numChars* characters long. **Tcl_UniCharNcmp** compares the two strings character-by-character according to the Unicode character ordering. It returns an integer greater than, equal to, or less than 0 if the first string is greater than, equal to, or less than the second string respectively.
Tcl_UniCharNcasecmp is the Unicode case insensitive version.

Tcl_UniCharCaseMatch is the Unicode equivalent to **Tcl_StringCaseMatch**. It accepts a null-terminated Unicode string, a Unicode pattern, and a boolean value specifying whether the match should be case sensitive and returns whether the string matches the pattern.

Tcl_UtfNcmp corresponds to **strncmp** for UTF-8 strings. It accepts two null-terminated UTF-8 strings and the number of characters to compare. (Both strings are assumed to be at least *numChars* characters long.) **Tcl_UtfNcmp** compares the two strings character-by-character according to the Unicode character ordering. It returns an integer greater than, equal to, or less than 0 if the first string is greater than, equal to, or less than the second string respectively.

Tcl_UtfNcasecmp corresponds to **strncasecmp** for UTF-8 strings. It is similar to **Tcl_UtfNcmp** except comparisons ignore differences in case when comparing upper, lower or title case characters.

Tcl_UtfCharComplete returns 1 if the source UTF-8 string *src* of *length* bytes is long enough to be decoded by **Tcl_UtfToUniChar**/**Tcl_UtfNext**, or 0 otherwise. This function does not guarantee that the UTF-8 string is properly formed. This routine is used by procedures that are operating on a byte at a time and need to know if a full Tcl_UniChar has been seen.

Tcl_NumUtfChars corresponds to **strlen** for UTF-8 strings. It returns the number of Tcl_UniChars that are represented by the UTF-8 string *src*. The length of the source string is *length* bytes. If the length is negative, all bytes up to the first null byte are used.

Tcl_UtfFindFirst corresponds to **strchr** for UTF-8 strings. It returns a pointer to the first occurrence of the Tcl_UniChar *ch* in the null-terminated UTF-8 string *src*. The null terminator is considered part of the UTF-8 string.

Tcl_UtfFindLast corresponds to **strrchr** for UTF-8 strings. It returns a pointer to the last occurrence of the Tcl_UniChar *ch* in the null-terminated UTF-8 string *src*. The null terminator is considered part of the UTF-8 string.

Given *src*, a pointer to some location in a UTF-8 string, **Tcl_UtfNext** returns a pointer to the next UTF-8 character in the string. The caller must not ask for the next character after the last character in the string if the string is not terminated by a null character. **Tcl_UtfCharComplete** can be used in that case to make sure enough bytes are available before calling **Tcl_UtfNext**.

Tcl_UtfPrev is used to step backward through but not beyond the UTF-8 string that begins at *start*. If the UTF-8 string is made up entirely of complete and well-formed characters, and *src* points to the lead byte of one of those characters (or to the location one byte past the end of the string), then repeated calls of **Tcl_UtfPrev** will return pointers to the lead bytes of each character in the string, one character at a time, terminating when it returns *start*.

When the conditions of completeness and well-formedness may not be satisfied, a more precise description of the function of **Tcl_UtfPrev** is necessary. It always returns a pointer greater than or equal to *start*; that is, always a pointer to a location in the string. It always returns a pointer to a byte that begins a character when scanning for characters beginning from *start*. When *src* is greater than *start*, it always returns a pointer less than *src* and greater than or equal to (*src* - **TCL_UTF_MAX**). The character that begins at the returned pointer is the first one that either includes the byte *src[-1]*, or might include it if the right trail bytes are present at *src* and greater. **Tcl_UtfPrev** never reads the byte *src[0]* nor the byte *start[-1]* nor the byte *src[*-**TCL_UTF_MAX** *-1]*.

Tcl_UniCharAtIndex corresponds to a C string array dereference or the Pascal Ord() function. It returns the Tcl_UniChar represented at the specified character (not byte) *index* in the UTF-8 string *src*. The source string must contain at least *index* characters. Behavior is undefined if a negative *index* is given.

Tcl_UtfAtIndex returns a pointer to the specified character (not byte) *index* in the UTF-8 string *src*. The source string must contain at least *index* characters. This is equivalent to calling **Tcl_UtfToUniChar** *index* times. If a negative *index* is given, the return pointer points to the first character in the source string.

Tcl_UtfBackslash is a utility procedure used by several of the Tcl commands. It parses a backslash sequence and stores the properly formed UTF-8 character represented by the backslash sequence in the output buffer *dst*. At most **TCL_UTF_MAX** bytes are stored in the buffer. **Tcl_UtfBackslash** modifies **readPtr* to contain the number of bytes in the backslash sequence, including the backslash character. The return value is the number of bytes stored in the output buffer.

See the **Tcl** manual entry for information on the valid backslash sequences. All of the sequences described in the Tcl manual entry are supported by **Tcl_UtfBackslash**.

Keywords: Utf, unicode, backslash

Copyright (c) 1997 Sun Microsystems, Inc.

WrongNumArgs

Name

Tcl_WrongNumArgs - generate standard error message for wrong number of arguments

Synopsis

#include <tcl.h>

Tcl_WrongNumArgs(*interp*, *objc*, *objv*, *message*)

Arguments

Tcl_Interp *interp* (in)
Interpreter in which error will be reported: error message gets stored in its result value.

int *objc* (in)
Number of leading arguments from *objv* to include in error message.

Tcl_Obj *const *objv[]* (in)
Arguments to command that had the wrong number of arguments.

const char **message* (in)
Additional error information to print after leading arguments from *objv*. This typically gives the acceptable syntax of the command. This argument may be NULL.

Description

Tcl_WrongNumArgs is a utility procedure that is invoked by command procedures when they discover that they have received the wrong number of arguments. **Tcl_WrongNumArgs** generates a standard error message and stores it in the result value of *interp*. The message includes the *objc* initial elements of *objv* plus *message*. For example, if *objv* consists of the values **foo** and **bar**, *objc* is 1, and *message* is "**fileName count**" then *interp*'s result value will be set to the following string:

```
wrong # args: should be "foo fileName count"
```

If *objc* is 2, the result will be set to the following string:

```
wrong # args: should be "foo bar fileName count"
```

Objc is usually 1, but may be 2 or more for commands like **string** and the Tk widget commands, which use the first argument as a subcommand.

Some of the values in the *objv* array may be abbreviations for a subcommand. The command **Tcl_GetIndexFromObj** will convert the abbreviated string value into an *indexObject*. If an error occurs in the parsing of the subcommand we would like to use the full subcommand name rather than the abbreviation. If the **Tcl_WrongNumArgs** command finds any *indexObjects* in the *objv* array it will use the full subcommand name in the error message instead of the abbreviated name that was originally passed in. Using the above example, let us assume that *bar* is actually an abbreviation for *barfly* and the value is now an *indexObject* because it was passed to **Tcl_GetIndexFromObj**. In this case the error message would be:

```
wrong # args: should be "foo barfly fileName count"
```

Keywords: WrongNumArgs, command, error message, wrong number of arguments

Copyright (c) 1994-1997 Sun Microsystems, Inc.

Tk C API

3DBorder

Name

Tk_Alloc3DBorderFromObj, Tk_Get3DBorder, Tk_Get3DBorderFromObj, Tk_Draw3DRectangle, Tk_Fill3DRectangle, Tk_Draw3DPolygon, Tk_Fill3DPolygon, Tk_3DVerticalBevel, Tk_3DHorizontalBevel, Tk_SetBackgroundFromBorder, Tk_NameOf3DBorder, Tk_3DBorderColor, Tk_3DBorderGC, Tk_Free3DBorderFromObj, Tk_Free3DBorder - draw borders with three-dimensional appearance

Synopsis

#include <tk.h>

Tk_3DBorder
Tk_Alloc3DBorderFromObj(*interp*, *tkwin*, *objPtr*)

Tk_3DBorder
Tk_Get3DBorder(*interp*, *tkwin*, *colorName*)

Tk_3DBorder
Tk_Get3DBorderFromObj(*tkwin*, *objPtr*)

Tk_Draw3DRectangle(*tkwin*, *drawable*, *border*, *x*, *y*, *width*, *height*, *borderWidth*, *relief*)

Tk_Fill3DRectangle(*tkwin*, *drawable*, *border*, *x*, *y*, *width*, *height*, *borderWidth*, *relief*)

Tk_Draw3DPolygon(*tkwin*, *drawable*, *border*, *pointPtr*, *numPoints*, *polyBorderWidth*, *leftRelief*)

Tk_Fill3DPolygon(*tkwin*, *drawable*, *border*, *pointPtr*, *numPoints*, *polyBorderWidth*, *leftRelief*)

Tk_3DVerticalBevel(*tkwin*, *drawable*, *border*, *x*, *y*, *width*, *height*, *leftBevel*, *relief*)

Tk_3DHorizontalBevel(*tkwin*, *drawable*, *border*, *x*, *y*, *width*, *height*, *leftIn*, *rightIn*, *topBevel*, *relief*)

Tk_SetBackgroundFromBorder(*tkwin*, *border*)

const char *
Tk_NameOf3DBorder(*border*)

XColor *
Tk_3DBorderColor(*border*)

GC *
Tk_3DBorderGC(*tkwin*, *border*, *which*)

Tk_Free3DBorderFromObj(*tkwin*, *objPtr*)

Tk_Free3DBorder(*border*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter to use for error reporting.

Tk_Window _tkwin_ (in)
Token for window (for all procedures except **Tk_Get3DBorder**, must be the window for which the border was allocated).

Tcl_Obj *_objPtr_ (in)
Pointer to value whose value describes color corresponding to background (flat areas).

Illuminated edges will be brighter than this and shadowed edges will be darker than this.

const char *_colorName_ (in)
Same as *objPtr* except value is supplied as a string rather than a value.

Drawable _drawable_ (in)
X token for window or pixmap; indicates where graphics are to be drawn. Must either be the X window for *tkwin* or a pixmap with the same screen and depth as *tkwin*.

Tk_3DBorder _border_ (in)
Token for border previously allocated in call to **Tk_Get3DBorder**.

int _x_ (in)
X-coordinate of upper-left corner of rectangle describing border or bevel, in pixels.

int _y_ (in)
Y-coordinate of upper-left corner of rectangle describing border or bevel, in pixels.

int _width_ (in)
Width of rectangle describing border or bevel, in pixels.

int _height_ (in)
Height of rectangle describing border or bevel, in pixels.

int _borderWidth_ (in)
Width of border in pixels. Positive means border is inside rectangle given by *x*, *y*, *width*, *height*, negative means border is outside rectangle.

int _relief_ (in)
Indicates 3-D position of interior of value relative to exterior; should be **TK_RELIEF_RAISED**, **TK_RELIEF_SUNKEN**, **TK_RELIEF_GROOVE**, **TK_RELIEF_SOLID**, or **TK_RELIEF_RIDGE** (may also be **TK_RELIEF_FLAT** for **Tk_Fill3DRectangle**).

XPoint *_pointPtr_ (in)
Pointer to array of points describing the set of vertices in a polygon. The polygon need not be closed (it will be closed automatically if it is not).

int _numPoints_ (in)
Number of points at **pointPtr*.

int _polyBorderWidth_ (in)
Width of border in pixels. If positive, border is drawn to left of trajectory given by *pointPtr*; if negative, border is drawn to right of trajectory. If *leftRelief* is **TK_RELIEF_GROOVE** or **TK_RELIEF_RIDGE** then the border is centered on the trajectory.

int _leftRelief_ (in)
Height of left side of polygon's path relative to right. **TK_RELIEF_RAISED** means left side should appear higher and **TK_RELIEF_SUNKEN** means right side should appear higher; **TK_RELIEF_GROOVE** and **TK_RELIEF_RIDGE** mean the obvious things. For **Tk_Fill3DPolygon**, **TK_RELIEF_FLAT** may also be specified to indicate no difference in height.

int _leftBevel_ (in)
Non-zero means this bevel forms the left side of the value; zero means it forms the right side.

int _leftIn_ (in)
Non-zero means that the left edge of the horizontal bevel angles in, so that the bottom of the edge is farther to the right than the top. Zero means the edge angles out, so that the bottom is farther to the left than the top.

int *rightIn* (in)
Non-zero means that the right edge of the horizontal bevel angles in, so that the bottom of the edge is farther to the left than the top. Zero means the edge angles out, so that the bottom is farther to the right than the top.

int *topBevel* (in)
Non-zero means this bevel forms the top side of the value; zero means it forms the bottom side.

int *which* (in)
Specifies which of the border's graphics contexts is desired. Must be **TK_3D_FLAT_GC**, **TK_3D_LIGHT_GC**, or **TK_3D_DARK_GC**.

Description

These procedures provide facilities for drawing window borders in a way that produces a three-dimensional appearance. **Tk_Alloc3DBorderFromObj** allocates colors and Pixmaps needed to draw a border in the window given by the *tkwin* argument. The value of *objPtr* is a standard Tk color name that determines the border colors. The color indicated by *objPtr* will not actually be used in the border; it indicates the background color for the window (i.e. a color for flat surfaces). The illuminated portions of the border will appear brighter than indicated by *objPtr* , and the shadowed portions of the border will appear darker than *objPtr*.

Tk_Alloc3DBorderFromObj returns a token that may be used in later calls to **Tk_Draw3DRectangle**. If an error occurs in allocating information for the border (e.g. a bogus color name was given) then NULL is returned and an error message is left as the result of interpreter *interp*. If it returns successfully, **Tk_Alloc3DBorderFromObj** caches information about the return value in *objPtr*, which speeds up future calls to **Tk_Alloc3DBorderFromObj** with the same *objPtr* and *tkwin*.

Tk_Get3DBorder is identical to **Tk_Alloc3DBorderFromObj** except that the color is specified with a string instead of a value. This prevents **Tk_Get3DBorder** from caching the return value, so **Tk_Get3DBorder** is less efficient than **Tk_Alloc3DBorderFromObj**.

Tk_Get3DBorderFromObj returns the token for an existing border, given the window and color name used to create the border. **Tk_Get3DBorderFromObj** does not actually create the border; it must already have been created with a previous call to **Tk_Alloc3DBorderFromObj** or **Tk_Get3DBorder**. The return value is cached in *objPtr*, which speeds up future calls to **Tk_Get3DBorderFromObj** with the same *objPtr* and *tkwin*.

Once a border structure has been created, **Tk_Draw3DRectangle** may be invoked to draw the border. The *tkwin* argument specifies the window for which the border was allocated, and *drawable* specifies a window or pixmap in which the border is to be drawn. *Drawable* need not refer to the same window as *tkwin*, but it must refer to a compatible pixmap or window: one associated with the same screen and with the same depth as *tkwin*. The *x*, *y*, *width*, and *height* arguments define the bounding box of the border region within *drawable* (usually *x* and *y* are zero and *width* and *height* are the dimensions of the window), and *borderWidth* specifies the number of pixels actually occupied by the border. The *relief* argument indicates which of several three-dimensional effects is desired: **TK_RELIEF_RAISED** means that the interior of the rectangle should appear raised relative to the exterior of the rectangle, and **TK_RELIEF_SUNKEN** means that the interior should appear depressed. **TK_RELIEF_GROOVE** and **TK_RELIEF_RIDGE** mean that there should appear to be a groove or ridge around the

exterior of the rectangle.

Tk_Fill3DRectangle is somewhat like **Tk_Draw3DRectangle** except that it first fills the rectangular area with the background color (one corresponding to the color used to create *border*). Then it calls **Tk_Draw3DRectangle** to draw a border just inside the outer edge of the rectangular area. The argument *relief* indicates the desired effect (**TK_RELIEF_FLAT** means no border should be drawn; all that happens is to fill the rectangle with the background color).

The procedure **Tk_Draw3DPolygon** may be used to draw more complex shapes with a three-dimensional appearance. The *pointPtr* and *numPoints* arguments define a trajectory, *polyBorderWidth* indicates how wide the border should be (and on which side of the trajectory to draw it), and *leftRelief* indicates which side of the trajectory should appear raised. **Tk_Draw3DPolygon** draws a border around the given trajectory using the colors from *border* to produce a three-dimensional appearance. If the trajectory is non-self-intersecting, the appearance will be a raised or sunken polygon shape. The trajectory may be self-intersecting, although it's not clear how useful this is.

Tk_Fill3DPolygon is to **Tk_Draw3DPolygon** what **Tk_Fill3DRectangle** is to **Tk_Draw3DRectangle**: it fills the polygonal area with the background color from *border*, then calls **Tk_Draw3DPolygon** to draw a border around the area (unless *leftRelief* is **TK_RELIEF_FLAT**; in this case no border is drawn).

The procedures **Tk_3DVerticalBevel** and **Tk_3DHorizontalBevel** provide lower-level drawing primitives that are used by procedures such as **Tk_Draw3DRectangle**. These procedures are also useful in their own right for drawing rectilinear border shapes. **Tk_3DVerticalBevel** draws a vertical beveled edge, such as the left or right side of a rectangle, and **Tk_3DHorizontalBevel** draws a horizontal beveled edge, such as the top or bottom of a rectangle. Each procedure takes *x*, *y*, *width*, and *height* arguments that describe the rectangular area of the beveled edge (e.g., *width* is the border width for **Tk_3DVerticalBevel**). The *leftBorder* and *topBorder* arguments indicate the position of the border relative to the "inside" of the value, and *relief* indicates the relief of the inside of the value relative to the outside. **Tk_3DVerticalBevel** just draws a rectangular region. **Tk_3DHorizontalBevel** draws a trapezoidal region to generate mitered corners; it should be called after **Tk_3DVerticalBevel** (otherwise **Tk_3DVerticalBevel** will overwrite the mitering in the corner). The *leftIn* and *rightIn* arguments to **Tk_3DHorizontalBevel** describe the mitering at the corners; a value of 1 means that the bottom edge of the trapezoid will be shorter than the top, 0 means it will be longer. For example, to draw a rectangular border the top bevel should be drawn with 1 for both *leftIn* and *rightIn*, and the bottom bevel should be drawn with 0 for both arguments.

The procedure **Tk_SetBackgroundFromBorder** will modify the background pixel and/or pixmap of *tkwin* to produce a result compatible with *border*. For color displays, the resulting background will just be the color specified when *border* was created; for monochrome displays, the resulting background will be a light stipple pattern, in order to distinguish the background from the illuminated portion of the border.

Given a token for a border, the procedure **Tk_NameOf3DBorder** will return the color name that was used to create the border.

The procedure **Tk_3DBorderColor** returns the XColor structure that will be used for flat surfaces drawn for its *border* argument by procedures like **Tk_Fill3DRectangle**. The return value corresponds to the color name that was used to create the border. The XColor, and its associated

pixel value, will remain allocated as long as *border* exists.

The procedure **Tk_3DBorderGC** returns one of the X graphics contexts that are used to draw the border. The argument *which* selects which one of the three possible GC's: **TK_3D_FLAT_GC** returns the context used for flat surfaces, **TK_3D_LIGHT_GC** returns the context for light shadows, and **TK_3D_DARK_GC** returns the context for dark shadows.

When a border is no longer needed, **Tk_Free3DBorderFromObj** or **Tk_Free3DBorder** should be called to release the resources associated with it. For **Tk_Free3DBorderFromObj** the border to release is specified with the window and color name used to create the border; for **Tk_Free3DBorder** the border to release is specified with the Tk_3DBorder token for the border. There should be exactly one call to **Tk_Free3DBorderFromObj** or **Tk_Free3DBorder** for each call to **Tk_Alloc3DBorderFromObj** or **Tk_Get3DBorder**.

Keywords: 3DBorder, 3D, background, border, color, depressed, illumination, value, polygon, raised, shadow, three-dimensional effect

Copyright (c) 1990-1993 The Regents of the University of California.
Copyright (c) 1994-1998 Sun Microsystems, Inc.

AddOption

Name

Tk_AddOption - Add an option to the option database

Synopsis

#include <tk.h>

void
Tk_AddOption(*tkwin*, *name*, *value*, *priority*)

Arguments

Tk_Window *tkwin* (in)
Token for window.

const char **name* (in)
Multi-element name of option.

const char **value* (in)
Value of option.

int *priority* (in)
Overall priority level to use for option.

Description

This procedure is invoked to add an option to the database associated with *tkwin*'s main window. *Name* contains the option being specified and consists of names and/or classes separated by asterisks or dots, in the usual X format. *Value* contains the text string to associate with *name*; this value will be returned in calls to **Tk_GetOption**. *Priority* specifies the priority of the value; when options are queried using **Tk_GetOption**, the value with the highest priority is returned. *Priority* must be between 0 and **TK_MAX_PRIO** (100). Some common priority values are:

TK_WIDGET_DEFAULT_PRIO (20)
Used for default values hard-coded into widgets.

TK_STARTUP_FILE_PRIO (40)
Used for options specified in application-specific startup files.

TK_USER_DEFAULT_PRIO (60)
Used for options specified in user-specific defaults files, such as **.Xdefaults**, resource databases loaded into the X server, or user-specific startup files.

TK_INTERACTIVE_PRIO (80)
Used for options specified interactively after the application starts running.

Keywords: AddOption, class, name, option, add

Copyright (c) 1998-2000 by Scriptics Corporation.
All rights reserved.

BindTable

Name

Tk_CreateBindingTable, Tk_DeleteBindingTable, Tk_CreateBinding, Tk_DeleteBinding, Tk_GetBinding, Tk_GetAllBindings, Tk_DeleteAllBindings, Tk_BindEvent - invoke scripts in response to X events

Synopsis

#include <tk.h>

Tk_BindingTable
Tk_CreateBindingTable(*interp*)

Tk_DeleteBindingTable(*bindingTable*)

unsigned long
Tk_CreateBinding(*interp*, *bindingTable*, *object*, *eventString*, *script*, *append*)

int
Tk_DeleteBinding(*interp*, *bindingTable*, *object*, *eventString*)

const char *
Tk_GetBinding(*interp*, *bindingTable*, *object*, *eventString*)

Tk_GetAllBindings(*interp*, *bindingTable*, *object*)

Tk_DeleteAllBindings(*bindingTable*, *object*)

Tk_BindEvent(*bindingTable*, *eventPtr*, *tkwin*, *numObjects*, *objectPtr*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter to use when invoking bindings in binding table. Also used for returning results and errors from binding procedures.

Tk_BindingTable _bindingTable_ (in)
Token for binding table; must have been returned by some previous call to **Tk_CreateBindingTable**.

ClientData _object_ (in)
Identifies object with which binding is associated.

const char *_eventString_ (in)

String describing event sequence.

const char *_script_ (in)
Tcl script to invoke when binding triggers.

int _append_ (in)
Non-zero means append *script* to existing script for binding, if any; zero means replace existing script with new one.

XEvent *_eventPtr_ (in)
X event to match against bindings in *bindingTable*.

Tk_Window _tkwin_ (in)
Identifier for any window on the display where the event occurred. Used to find display-related information such as key maps.

int _numObjects_ (in)
Number of object identifiers pointed to by *objectPtr*.

ClientData *_objectPtr_ (in)
Points to an array of object identifiers: bindings will be considered for each of these objects in order from first to last.

Description

These procedures provide a general-purpose mechanism for creating and invoking bindings. Bindings are organized in terms of *binding tables*. A binding table consists of a collection of bindings plus a history of recent events. Within a binding table, bindings are associated with *objects*. The meaning of an object is defined by clients of the binding package. For example, Tk keeps uses one binding table to hold all of the bindings created by the **bind** command. For this table, objects are pointers to strings such as window names, class names, or other binding tags such as **all**. Tk also keeps a separate binding table for each canvas widget, which manages bindings created by the canvas's **bind** widget command; within this table, an object is either a pointer to the internal structure for a canvas item or a Tk_Uid identifying a tag.

The procedure **Tk_CreateBindingTable** creates a new binding table and associates *interp* with it (when bindings in the table are invoked, the scripts will be evaluated in *interp*). **Tk_CreateBindingTable** returns a token for the table, which must be used in calls to other procedures such as **Tk_CreateBinding** or **Tk_BindEvent**.

Tk_DeleteBindingTable frees all of the state associated with a binding table. Once it returns the caller should not use the *bindingTable* token again.

Tk_CreateBinding adds a new binding to an existing table. The *object* argument identifies the object with which the binding is to be associated, and it may be any one-word value. Typically it is a pointer to a string or data structure. The *eventString* argument identifies the event or sequence of events for the binding; see the documentation for the **bind** command for a description of its format. *script* is the Tcl script to be evaluated when the binding triggers. *append* indicates what to do if there already exists a binding for *object* and *eventString*: if *append* is zero then *script* replaces the old script; if *append* is non-zero then the new script is appended to the old one. **Tk_CreateBinding** returns an X event mask for all the events associated with the bindings. This information may be useful to invoke **XSelectInput** to select relevant events, or to disallow the use of certain events in bindings. If an error occurred while creating the binding (e.g., *eventString* refers to a non-existent event), then 0 is returned and an error message is left as the result of interpreter *interp*.

Tk_DeleteBinding removes from *bindingTable* the binding given by *object* and *eventString*, if such a binding exists. **Tk_DeleteBinding** always returns **TCL_OK**. In some cases it may reset the interpreter result to the default empty value.

Tk_GetBinding returns a pointer to the script associated with *eventString* and *object* in *bindingTable*. If no such binding exists then NULL is returned and an error message is left as the result of interpreter *interp*.

Tk_GetAllBindings returns in *interp*'s result a list of all the event strings for which there are bindings in *bindingTable* associated with *object*. If there are no bindings for *object*, the result will be an empty string.

Tk_DeleteAllBindings deletes all of the bindings in *bindingTable* that are associated with *object*.

Tk_BindEvent is called to process an event. It makes a copy of the event in an internal history list associated with the binding table, then it checks for bindings that match the event. **Tk_BindEvent** processes each of the objects pointed to by *objectPtr* in turn. For each object, it finds all the bindings that match the current event history, selects the most specific binding using the priority mechanism described in the documentation for **bind**, and invokes the script for that binding. If there are no matching bindings for a particular object, then the object is skipped. **Tk_BindEvent** continues through all of the objects, handling exceptions such as errors, **break**, and **continue** as described in the documentation for **bind**.

Keywords: BindTable, binding, event, object, script

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

CanvPsY

Name

Tk_CanvasPsY, Tk_CanvasPsBitmap, Tk_CanvasPsColor, Tk_CanvasPsFont, Tk_CanvasPsPath, Tk_CanvasPsStipple - utility procedures for generating Postscript for canvases

Synopsis

#include <tk.h>

double
Tk_CanvasPsY(*canvas*, *canvasY*)

int
Tk_CanvasPsBitmap(*interp*, *canvas*, *bitmap*, *x*, *y*, *width*, *height*)

int
Tk_CanvasPsColor(*interp*, *canvas*, *colorPtr*)

int
Tk_CanvasPsFont(*interp*, *canvas*, *tkFont*)

Tk_CanvasPsPath(*interp*, *canvas*, *coordPtr*, *numPoints*)

int
Tk_CanvasPsStipple(*interp*, *canvas*, *bitmap*)

Arguments

Tk_Canvas *canvas* (in)
A token that identifies a canvas widget for which Postscript is being generated.

double *canvasY* (in)
Y-coordinate in the space of the canvas.

Tcl_Interp *interp* (in/out)
A Tcl interpreter; Postscript is appended to its result, or the result may be replaced with an error message.

Pixmap *bitmap* (in)
Bitmap to use for generating Postscript.

int *x* (in)
X-coordinate within *bitmap* of left edge of region to output.

int *y* (in)
Y-coordinate within *bitmap* of top edge of region to output.

int *width* (in)
Width of region of bitmap to output, in pixels.

int *height* (in)
Height of region of bitmap to output, in pixels.

XColor *colorPtr* (in)
Information about color value to set in Postscript.

Tk_Font *tkFont* (in)
Font for which Postscript is to be generated.

double *coordPtr* (in)
Pointer to an array of coordinates for one or more points specified in canvas coordinates. The order of values in *coordPtr* is x1, y1, x2, y2, x3, y3, and so on.

int *numPoints* (in)
Number of points at *coordPtr*.

Description

These procedures are called by canvas type managers to carry out common functions related to generating Postscript. Most of the procedures take a *canvas* argument, which refers to a canvas widget for which Postscript is being generated.

Tk_CanvasPsY takes as argument a y-coordinate in the space of a canvas and returns the value that should be used for that point in the Postscript currently being generated for *canvas*. Y coordinates require transformation because Postscript uses an origin at the lower-left corner whereas X uses an origin at the upper-left corner. Canvas x coordinates can be used directly in Postscript without transformation.

Tk_CanvasPsBitmap generates Postscript to describe a region of a bitmap. The Postscript is generated in proper image data format for Postscript, i.e., as data between angle brackets, one bit per pixel. The Postscript is appended to the result of interpreter *interp* and **TCL_OK** is returned unless an error occurs, in which case **TCL_ERROR** is returned and the interpreter result is overwritten with an error message.

Tk_CanvasPsColor generates Postscript to set the current color to correspond to its *colorPtr* argument, taking into account any color map specified in the **postscript** command. It appends

the Postscript to the interpreter *interp*'s result and returns **TCL_OK** unless an error occurs, in which case **TCL_ERROR** is returned and the interpreter's result is overwritten with an error message.

Tk_CanvasPsFont generates Postscript that sets the current font to match *tkFont* as closely as possible. **Tk_CanvasPsFont** takes into account any font map specified in the **postscript** command, and it does the best it can at mapping X fonts to Postscript fonts. It appends the Postscript to interpreter *interp*'s result and returns **TCL_OK** unless an error occurs, in which case **TCL_ERROR** is returned and the interpreter's result is overwritten with an error message.

Tk_CanvasPsPath generates Postscript to set the current path to the set of points given by *coordPtr* and *numPoints*. It appends the resulting Postscript to the result of interpreter *interp*.

Tk_CanvasPsStipple generates Postscript that will fill the current path in stippled fashion. It uses *bitmap* as the stipple pattern and the current Postscript color; ones in the stipple bitmap are drawn in the current color, and zeroes are not drawn at all. The Postscript is appended to interpreter *interp*'s result and **TCL_OK** is returned, unless an error occurs, in which case **TCL_ERROR** is returned and the interpreter's result is overwritten with an error message.

Keywords: CanvPsY, bitmap, canvas, color, font, path, Postscript, stipple

Copyright (c) 1994-1996 Sun Microsystems, Inc.

CanvTkwin

Name

Tk_CanvasTkwin, Tk_CanvasGetCoord, Tk_CanvasDrawableCoords, Tk_CanvasSetStippleOrigin, Tk_CanvasWindowCoords, Tk_CanvasEventuallyRedraw, Tk_CanvasTagsOption - utility procedures for canvas type managers

Synopsis

#include <tk.h>

Tk_Window
Tk_CanvasTkwin(*canvas*)

int
Tk_CanvasGetCoord(*interp*, *canvas*, *string*, *doublePtr*)

Tk_CanvasDrawableCoords(*canvas*, *x*, *y*, *drawableXPtr*, *drawableYPtr*)

Tk_CanvasSetStippleOrigin(*canvas*, *gc*)

Tk_CanvasWindowCoords(*canvas*, *x*, *y*, *screenXPtr*, *screenYPtr*)

Tk_CanvasEventuallyRedraw(*canvas*, *x1*, *y1*, *x2*, *y2*)

Tk_OptionParseProc ***Tk_CanvasTagsParseProc**;

Tk_OptionPrintProc ***Tk_CanvasTagsPrintProc**;

Arguments

Tk_Canvas *canvas* (in)
A token that identifies a canvas widget.

Tcl_Interp **interp* (in/out)
Interpreter to use for error reporting.

const char *_string_ (in)
Textual description of a canvas coordinate.

double *_doublePtr_ (out)
Points to place to store a converted coordinate.

double _x_ (in)
An x coordinate in the space of the canvas.

double _y_ (in)
A y coordinate in the space of the canvas.

short *_drawableXPtr_ (out)
Pointer to a location in which to store an x coordinate in the space of the drawable currently being used to redisplay the canvas.

short *_drawableYPtr_ (out)
Pointer to a location in which to store a y coordinate in the space of the drawable currently being used to redisplay the canvas.

GC _gc_ (out)
Graphics context to modify.

short *_screenXPtr_ (out)
Points to a location in which to store the screen coordinate in the canvas window that corresponds to *x*.

short *_screenYPtr_ (out)
Points to a location in which to store the screen coordinate in the canvas window that corresponds to *y*.

int _x1_ (in)
Left edge of the region that needs redisplay. Only pixels at or to the right of this coordinate need to be redisplayed.

int _y1_ (in)
Top edge of the region that needs redisplay. Only pixels at or below this coordinate need to be redisplayed.

int _x2_ (in)
Right edge of the region that needs redisplay. Only pixels to the left of this coordinate need to be redisplayed.

int _y2_ (in)
Bottom edge of the region that needs redisplay. Only pixels above this coordinate need to be redisplayed.

Description

These procedures are called by canvas type managers to perform various utility functions.

Tk_CanvasTkwin returns the Tk_Window associated with a particular canvas.

Tk_CanvasGetCoord translates a string specification of a coordinate (such as **2p** or **1.6c**) into a double-precision canvas coordinate. If *string* is a valid coordinate description then **Tk_CanvasGetCoord** stores the corresponding canvas coordinate at **doublePtr* and returns **TCL_OK**. Otherwise it stores an error message in the interpreter result and returns **TCL_ERROR**.

Tk_CanvasDrawableCoords is called by type managers during redisplay to compute where to draw things. Given *x* and *y* coordinates in the space of the canvas,

Tk_CanvasDrawableCoords computes the corresponding pixel in the drawable that is currently being used for redisplay; it returns those coordinates in **drawableXPtr* and **drawableYPtr*. This procedure should not be invoked except during redisplay.

Tk_CanvasSetStippleOrigin is also used during redisplay. It sets the stipple origin in *gc* so that stipples drawn with *gc* in the current offscreen pixmap will line up with stipples drawn with origin (0,0) in the canvas's actual window. **Tk_CanvasSetStippleOrigin** is needed in order to guarantee that stipple patterns line up properly when the canvas is redisplayed in small pieces. Redisplays are carried out in double-buffered fashion where a piece of the canvas is redrawn in an offscreen pixmap and then copied back onto the screen. In this approach the stipple origins in graphics contexts need to be adjusted during each redisplay to compensate for the position of the off-screen pixmap relative to the window. If an item is being drawn with stipples, its type manager typically calls **Tk_CanvasSetStippleOrigin** just before using *gc* to draw something; after it is finished drawing, the type manager calls **XSetTSOrigin** to restore the origin in *gc* back to (0,0) (the restore is needed because graphics contexts are shared, so they cannot be modified permanently).

Tk_CanvasWindowCoords is similar to **Tk_CanvasDrawableCoords** except that it returns coordinates in the canvas's window on the screen, instead of coordinates in an off-screen pixmap.

Tk_CanvasEventuallyRedraw may be invoked by a type manager to inform Tk that a portion of a canvas needs to be redrawn. The *x1*, *y1*, *x2*, and *y2* arguments specify the region that needs to be redrawn, in canvas coordinates. Type managers rarely need to invoke **Tk_CanvasEventuallyRedraw**, since Tk can normally figure out when an item has changed and make the redisplay request on its behalf (this happens, for example whenever Tk calls a *configureProc* or *scaleProc*). The only time that a type manager needs to call **Tk_CanvasEventuallyRedraw** is if an item has changed on its own without being invoked through one of the procedures in its Tk_ItemType; this could happen, for example, in an image item if the image is modified using image commands.

Tk_CanvasTagsParseProc and **Tk_CanvasTagsPrintProc** are procedures that handle the **-tags** option for canvas items. The code of a canvas type manager will not call these procedures directly, but will use their addresses to create a **Tk_CustomOption** structure for the **-tags** option. The code typically looks like this:

```
static const Tk_CustomOption tagsOption = {Tk_CanvasTagsParseProc,
    Tk_CanvasTagsPrintProc, NULL
};

static const Tk_ConfigSpec configSpecs[] = {
    ...
    {TK_CONFIG_CUSTOM, "-tags", NULL, NULL,
        NULL, 0, TK_CONFIG_NULL_OK, &tagsOption},
    ...
};
```

Keywords: CanvTkwin, canvas, focus, item type, redisplay, selection, type manager

Copyright (c) 1994-1996 Sun Microsystems, Inc.

CanvTxtInfo

Name

Tk_CanvasTextInfo - additional information for managing text items in canvases

Synopsis

#include <tk.h>

Tk_CanvasTextInfo *
Tk_CanvasGetTextInfo(*canvas*)

Arguments

Tk_Canvas *canvas* (in)
A token that identifies a particular canvas widget.

Description

Textual canvas items are somewhat more complicated to manage than other items, due to things like the selection and the input focus. **Tk_CanvasGetTextInfo** may be invoked by a type manager to obtain additional information needed for items that display text. The return value from **Tk_CanvasGetTextInfo** is a pointer to a structure that is shared between Tk and all the items that display text. The structure has the following form:

```
typedef struct Tk_CanvasTextInfo {
    Tk_3DBorder selBorder;
    int selBorderWidth;
    XColor *selFgColorPtr;
    Tk_Item *selItemPtr;
    int selectFirst;
    int selectLast;
    Tk_Item *anchorItemPtr;
    int selectAnchor;
    Tk_3DBorder insertBorder;
    int insertWidth;
    int insertBorderWidth;
    Tk_Item *focusItemPtr;
    int gotFocus;
    int cursorOn;
} Tk_CanvasTextInfo;
```

The **selBorder** field identifies a Tk_3DBorder that should be used for drawing the background under selected text. *selBorderWidth* gives the width of the raised border around selected text, in pixels. *selFgColorPtr* points to an XColor that describes the foreground color to be used when drawing selected text. *selItemPtr* points to the item that is currently selected, or NULL if there is no item selected or if the canvas does not have the selection. *selectFirst* and *selectLast* give the indices of the first and last selected characters in *selItemPtr*, as returned by the *indexProc* for that item. *anchorItemPtr* points to the item that currently has the selection anchor; this is not necessarily the same as *selItemPtr*. *selectAnchor* is an index that identifies the anchor position within *anchorItemPtr*. *insertBorder* contains a Tk_3DBorder to use when drawing the insertion

cursor; *insertWidth* gives the total width of the insertion cursor in pixels, and *insertBorderWidth* gives the width of the raised border around the insertion cursor. *focusItemPtr* identifies the item that currently has the input focus, or NULL if there is no such item. *gotFocus* is 1 if the canvas widget has the input focus and 0 otherwise. *cursorOn* is 1 if the insertion cursor should be drawn in *focusItemPtr* and 0 if it should not be drawn; this field is toggled on and off by Tk to make the cursor blink.

The structure returned by **Tk_CanvasGetTextInfo** is shared between Tk and the type managers; typically the type manager calls **Tk_CanvasGetTextInfo** once when an item is created and then saves the pointer in the item's record. Tk will update information in the Tk_CanvasTextInfo; for example, a **configure** widget command might change the *selBorder* field, or a **select** widget command might change the *selectFirst* field, or Tk might change *cursorOn* in order to make the insertion cursor flash on and off during successive redisplays.

Type managers should treat all of the fields of the Tk_CanvasTextInfo structure as read-only, except for *selItemPtr*, *selectFirst*, *selectLast*, and *selectAnchor*. Type managers may change *selectFirst*, *selectLast*, and *selectAnchor* to adjust for insertions and deletions in the item (but only if the item is the current owner of the selection or anchor, as determined by *selItemPtr* or *anchorItemPtr*). If all of the selected text in the item is deleted, the item should set *selItemPtr* to NULL to indicate that there is no longer a selection.

Keywords: CanvTxtInfo, canvas, focus, insertion cursor, selection, selection anchor, text

Copyright (c) 1994-1996 Sun Microsystems, Inc.

Clipboard

Name

Tk_ClipboardClear, Tk_ClipboardAppend - Manage the clipboard

Synopsis

#include <tk.h>

int
Tk_ClipboardClear(*interp*, *tkwin*)

int
Tk_ClipboardAppend(*interp*, *tkwin*, *target*, *format*, *buffer*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter to use for reporting errors.

Tk_Window _tkwin_ (in)
Window that determines which display's clipboard to manipulate.

Atom _target_ (in)
Conversion type for this clipboard item; has same meaning as *target* argument to **Tk_CreateSelHandler**.

Atom _format_ (in)
Representation to use when data is retrieved; has same meaning as *format* argument to **Tk_CreateSelHandler**.

const char *_buffer_ (in)

Null terminated string containing the data to be appended to the clipboard.

Description

These two procedures manage the clipboard for Tk. The clipboard is typically managed by calling **Tk_ClipboardClear** once, then calling **Tk_ClipboardAppend** to add data for any number of targets.

Tk_ClipboardClear claims the CLIPBOARD selection and frees any data items previously stored on the clipboard in this application. It normally returns **TCL_OK**, but if an error occurs it returns **TCL_ERROR** and leaves an error message in interpreter *interp*'s result. **Tk_ClipboardClear** must be called before a sequence of **Tk_ClipboardAppend** calls can be issued.

Tk_ClipboardAppend appends a buffer of data to the clipboard. The first buffer for a given *target* determines the *format* for that *target*. Any successive appends for that *target* must have the same format or an error will be returned. **Tk_ClipboardAppend** returns **TCL_OK** if the buffer is successfully copied onto the clipboard. If the clipboard is not currently owned by the application, either because **Tk_ClipboardClear** has not been called or because ownership of the clipboard has changed since the last call to **Tk_ClipboardClear**, **Tk_ClipboardAppend** returns **TCL_ERROR** and leaves an error message in the result of interpreter *interp*.

In order to guarantee atomicity, no event handling should occur between **Tk_ClipboardClear** and the following **Tk_ClipboardAppend** calls (otherwise someone could retrieve a partially completed clipboard or claim ownership away from this application).

Tk_ClipboardClear may invoke callbacks, including arbitrary Tcl scripts, as a result of losing the CLIPBOARD selection, so any calling function should take care to be re-entrant at the point **Tk_ClipboardClear** is invoked.

Keywords: Clipboard, append, clear, format, type

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

ClrSelect

Name

Tk_ClearSelection - Deselect a selection

Synopsis

#include <tk.h>

Tk_ClearSelection(*tkwin*, *selection*)

Arguments

Tk_Window *tkwin* (in)
The selection will be cleared from the display containing this window.

Atom *selection* (in)
The name of selection to be cleared.

Description

Tk_ClearSelection cancels the selection specified by the atom *selection* for the display containing *tkwin*. The selection need not be in *tkwin* itself or even in *tkwin*'s application. If there

is a window anywhere on *tkwin*'s display that owns *selection*, the window will be notified and the selection will be cleared. If there is no owner for *selection* on the display, then the procedure has no effect.

Keywords: ClrSelect, clear, selection

Copyright (c) 1992-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

ConfigWidg

Name

Tk_ConfigureWidget, Tk_ConfigureInfo, Tk_ConfigureValue, Tk_FreeOptions - process configuration options for widgets

Synopsis

#include <tk.h>

int
Tk_ConfigureWidget(*interp*, *tkwin*, *specs*, *argc*, *argv*, *widgRec*, *flags*)

int
Tk_ConfigureInfo(*interp*, *tkwin*, *specs*, *widgRec*, *argvName*, *flags*)

int
Tk_ConfigureValue(*interp*, *tkwin*, *specs*, *widgRec*, *argvName*, *flags*)

Tk_FreeOptions(*specs*, *widgRec*, *display*, *flags*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter to use for returning error messages.

Tk_Window _tkwin_ (in)
Window used to represent widget (needed to set up X resources).

const Tk_ConfigSpec *_specs_ (in)
Pointer to table specifying legal configuration options for this widget.

int _argc_ (in)
Number of arguments in *argv*.

const char **_argv_ (in)
Command-line options for configuring widget.

char *_widgRec_ (in/out)
Points to widget record structure. Fields in this structure get modified by **Tk_ConfigureWidget** to hold configuration information.

int _flags_ (in)
If non-zero, then it specifies an OR-ed combination of flags that control the processing of configuration information. **TK_CONFIG_ARGV_ONLY** causes the option database and defaults to be ignored, and flag bits **TK_CONFIG_USER_BIT** and higher are used to selectively disable entries in *specs*.

type name _type_ (in)
The name of the type of a widget record.

field name *field* (in)
The name of a field in records of type *type*.

const char *argvName* (in)
The name used on Tcl command lines to refer to a particular option (e.g. when creating a widget or invoking the **configure** widget command). If non-NULL, then information is returned only for this option. If NULL, then information is returned for all available options.

Display *display* (in)
Display containing widget whose record is being freed; needed in order to free up resources.

Description

Note that **Tk_ConfigureWidget** should be replaced with the new **Tcl_Obj** based API, **Tk_SetOptions**. The old interface is retained for backward compatibility.

Tk_ConfigureWidget is called to configure various aspects of a widget, such as colors, fonts, border width, etc. It is intended as a convenience procedure to reduce the amount of code that must be written in individual widget managers to handle configuration information. It is typically invoked when widgets are created, and again when the **configure** command is invoked for a widget. Although intended primarily for widgets, **Tk_ConfigureWidget** can be used in other situations where *argc*-*argv* information is to be used to fill in a record structure, such as configuring graphical elements for a canvas widget or entries of a menu.

Tk_ConfigureWidget processes a table specifying the configuration options that are supported (*specs*) and a collection of command-line arguments (*argc* and *argv*) to fill in fields of a record (*widgRec*). It uses the option database and defaults specified in *specs* to fill in fields of *widgRec* that are not specified in *argv*. **Tk_ConfigureWidget** normally returns the value **TCL_OK**; in this case it does not modify *interp*. If an error occurs then **TCL_ERROR** is returned and **Tk_ConfigureWidget** will leave an error message in interpreter *interp*'s result in the standard Tcl fashion. In the event of an error return, some of the fields of *widgRec* could already have been set, if configuration information for them was successfully processed before the error occurred. The other fields will be set to reasonable initial values so that **Tk_FreeOptions** can be called for cleanup.

The *specs* array specifies the kinds of configuration options expected by the widget. Each of its entries specifies one configuration option and has the following structure:

```
typedef struct {
    int type;
    const char *argvName;
    const char *dbName;
    const char *dbClass;
    const char *defValue;
    int offset;
    int specFlags;
    const Tk_CustomOption *customPtr;
} Tk_ConfigSpec;
```

The *type* field indicates what type of configuration option this is (e.g. **TK_CONFIG_COLOR** for a color value, or **TK_CONFIG_INT** for an integer value). The *type* field indicates how to use the value of the option (more on this below). The *argvName* field is a string such as "-font" or "-bg",

which is compared with the values in *argv* (if *argvName* is NULL it means this is a grouped entry; see **GROUPED ENTRIES** below). The *dbName* and *dbClass* fields are used to look up a value for this option in the option database. The *defValue* field specifies a default value for this configuration option if no value is specified in either *argv* or the option database. *Offset* indicates where in *widgRec* to store information about this option, and *specFlags* contains additional information to control the processing of this configuration option (see FLAGS below). The last field, *customPtr*, is only used if *type* is **TK_CONFIG_CUSTOM**; see CUSTOM OPTION TYPES below.

Tk_ConfigureWidget first processes *argv* to see which (if any) configuration options are specified there. *Argv* must contain an even number of fields; the first of each pair of fields must match the *argvName* of some entry in *specs* (unique abbreviations are acceptable), and the second field of the pair contains the value for that configuration option. If there are entries in *spec* for which there were no matching entries in *argv*, **Tk_ConfigureWidget** uses the *dbName* and *dbClass* fields of the *specs* entry to probe the option database; if a value is found, then it is used as the value for the option. Finally, if no entry is found in the option database, the *defValue* field of the *specs* entry is used as the value for the configuration option. If the *defValue* is NULL, or if the **TK_CONFIG_DONT_SET_DEFAULT** bit is set in *flags*, then there is no default value and this *specs* entry will be ignored if no value is specified in *argv* or the option database.

Once a string value has been determined for a configuration option, **Tk_ConfigureWidget** translates the string value into a more useful form, such as a color if *type* is **TK_CONFIG_COLOR** or an integer if *type* is **TK_CONFIG_INT**. This value is then stored in the record pointed to by *widgRec*. This record is assumed to contain information relevant to the manager of the widget; its exact type is unknown to **Tk_ConfigureWidget**. The *offset* field of each *specs* entry indicates where in *widgRec* to store the information about this configuration option. You should use the **Tk_Offset** macro to generate *offset* values (see below for a description of **Tk_Offset**). The location indicated by *widgRec* and *offset* will be referred to as the "target" in the descriptions below.

The *type* field of each entry in *specs* determines what to do with the string value of that configuration option. The legal values for *type*, and the corresponding actions, are:

TK_CONFIG_ACTIVE_CURSOR
: The value must be an ASCII string identifying a cursor in a form suitable for passing to **Tk_GetCursor**. The value is converted to a **Tk_Cursor** by calling **Tk_GetCursor** and the result is stored in the target. In addition, the resulting cursor is made the active cursor for *tkwin* by calling **XDefineCursor**. If **TK_CONFIG_NULL_OK** is specified in *specFlags* then the value may be an empty string, in which case the target and *tkwin*'s active cursor will be set to **None**. If the previous value of the target was not **None**, then it is freed by passing it to **Tk_FreeCursor**.

TK_CONFIG_ANCHOR
: The value must be an ASCII string identifying an anchor point in one of the ways accepted by **Tk_GetAnchor**. The string is converted to a **Tk_Anchor** by calling **Tk_GetAnchor** and the result is stored in the target.

TK_CONFIG_BITMAP
: The value must be an ASCII string identifying a bitmap in a form suitable for passing to **Tk_GetBitmap**. The value is converted to a **Pixmap** by calling **Tk_GetBitmap** and the result is stored in the target. If **TK_CONFIG_NULL_OK** is specified in *specFlags* then the value may

be an empty string, in which case the target is set to **None**. If the previous value of the target was not **None**, then it is freed by passing it to **Tk_FreeBitmap**.

TK_CONFIG_BOOLEAN
The value must be an ASCII string specifying a boolean value. Any of the values "true", "yes", "on", or "1", or an abbreviation of one of these values, means true; any of the values "false", "no", "off", or "0", or an abbreviation of one of these values, means false. The target is expected to be an integer; for true values it will be set to 1 and for false values it will be set to 0.

TK_CONFIG_BORDER
The value must be an ASCII string identifying a border color in a form suitable for passing to **Tk_Get3DBorder**. The value is converted to a (**Tk_3DBorder** *) by calling **Tk_Get3DBorder** and the result is stored in the target. If **TK_CONFIG_NULL_OK** is specified in *specFlags* then the value may be an empty string, in which case the target will be set to NULL. If the previous value of the target was not NULL, then it is freed by passing it to **Tk_Free3DBorder**.

TK_CONFIG_CAP_STYLE
The value must be an ASCII string identifying a cap style in one of the ways accepted by **Tk_GetCapStyle**. The string is converted to an integer value corresponding to the cap style by calling **Tk_GetCapStyle** and the result is stored in the target.

TK_CONFIG_COLOR
The value must be an ASCII string identifying a color in a form suitable for passing to **Tk_GetColor**. The value is converted to an (**XColor** *) by calling **Tk_GetColor** and the result is stored in the target. If **TK_CONFIG_NULL_OK** is specified in *specFlags* then the value may be an empty string, in which case the target will be set to **None**. If the previous value of the target was not NULL, then it is freed by passing it to **Tk_FreeColor**.

TK_CONFIG_CURSOR
This option is identical to **TK_CONFIG_ACTIVE_CURSOR** except that the new cursor is not made the active one for *tkwin*.

TK_CONFIG_CUSTOM
This option allows applications to define new option types. The *customPtr* field of the entry points to a structure defining the new option type. See the section **CUSTOM OPTION TYPES** below for details.

TK_CONFIG_DOUBLE
The value must be an ASCII floating-point number in the format accepted by **strtol**. The string is converted to a **double** value, and the value is stored in the target.

TK_CONFIG_END
Marks the end of the table. The last entry in *specs* must have this type; all of its other fields are ignored and it will never match any arguments.

TK_CONFIG_FONT
The value must be an ASCII string identifying a font in a form suitable for passing to **Tk_GetFont**. The value is converted to a **Tk_Font** by calling **Tk_GetFont** and the result is stored in the target. If **TK_CONFIG_NULL_OK** is specified in *specFlags* then the value may be an empty string, in which case the target will be set to NULL. If the previous value of the target was not NULL, then it is freed by passing it to **Tk_FreeFont**.

TK_CONFIG_INT
The value must be an ASCII integer string in the format accepted by **strtol** (e.g. "0" and "0x"

prefixes may be used to specify octal or hexadecimal numbers, respectively). The string is converted to an integer value and the integer is stored in the target.

TK_CONFIG_JOIN_STYLE
The value must be an ASCII string identifying a join style in one of the ways accepted by **Tk_GetJoinStyle**. The string is converted to an integer value corresponding to the join style by calling **Tk_GetJoinStyle** and the result is stored in the target.

TK_CONFIG_JUSTIFY
The value must be an ASCII string identifying a justification method in one of the ways accepted by **Tk_GetJustify**. The string is converted to a **Tk_Justify** by calling **Tk_GetJustify** and the result is stored in the target.

TK_CONFIG_MM
The value must specify a screen distance in one of the forms acceptable to **Tk_GetScreenMM**. The string is converted to double-precision floating-point distance in millimeters and the value is stored in the target.

TK_CONFIG_PIXELS
The value must specify screen units in one of the forms acceptable to **Tk_GetPixels**. The string is converted to an integer distance in pixels and the value is stored in the target.

TK_CONFIG_RELIEF
The value must be an ASCII string identifying a relief in a form suitable for passing to **Tk_GetRelief**. The value is converted to an integer relief value by calling **Tk_GetRelief** and the result is stored in the target.

TK_CONFIG_STRING
A copy of the value is made by allocating memory space with **Tcl_Alloc** and copying the value into the dynamically-allocated space. A pointer to the new string is stored in the target. If **TK_CONFIG_NULL_OK** is specified in *specFlags* then the value may be an empty string, in which case the target will be set to NULL. If the previous value of the target was not NULL, then it is freed by passing it to **Tcl_Free**.

TK_CONFIG_SYNONYM
This *type* value identifies special entries in *specs* that are synonyms for other entries. If an *argv* value matches the *argvName* of a **TK_CONFIG_SYNONYM** entry, the entry is not used directly. Instead, **Tk_ConfigureWidget** searches *specs* for another entry whose *argvName* is the same as the *dbName* field in the **TK_CONFIG_SYNONYM** entry; this new entry is used just as if its *argvName* had matched the *argv* value. The synonym mechanism allows multiple *argv* values to be used for a single configuration option, such as "-background" and "-bg".

TK_CONFIG_UID
The value is translated to a **Tk_Uid** (by passing it to **Tk_GetUid**). The resulting value is stored in the target. If **TK_CONFIG_NULL_OK** is specified in *specFlags* and the value is an empty string then the target will be set to NULL.

TK_CONFIG_WINDOW
The value must be a window path name. It is translated to a **Tk_Window** token and the token is stored in the target.

Grouped entries

In some cases it is useful to generate multiple resources from a single configuration value. For example, a color name might be used both to generate the background color for a widget (using

TK_CONFIG_COLOR) and to generate a 3-D border to draw around the widget (using **TK_CONFIG_BORDER**). In cases like this it is possible to specify that several consecutive entries in *specs* are to be treated as a group. The first entry is used to determine a value (using its *argvName*, *dbName*, *dbClass*, and *defValue* fields). The value will be processed several times (one for each entry in the group), generating multiple different resources and modifying multiple targets within *widgRec*. Each of the entries after the first must have a NULL value in its *argvName* field; this indicates that the entry is to be grouped with the entry that precedes it. Only the *type* and *offset* fields are used from these follow-on entries.

Flags

The *flags* argument passed to **Tk_ConfigureWidget** is used in conjunction with the *specFlags* fields in the entries of *specs* to provide additional control over the processing of configuration options. These values are used in three different ways as described below.

First, if the *flags* argument to **Tk_ConfigureWidget** has the **TK_CONFIG_ARGV_ONLY** bit set (i.e., *flags* | **TK_CONFIG_ARGV_ONLY**!= 0), then the option database and *defValue* fields are not used. In this case, if an entry in *specs* does not match a field in *argv* then nothing happens: the corresponding target is not modified. This feature is useful when the goal is to modify certain configuration options while leaving others in their current state, such as when a **configure** widget command is being processed.

Second, the *specFlags* field of an entry in *specs* may be used to control the processing of that entry. Each *specFlags* field may consists of an OR-ed combination of the following values:

TK_CONFIG_COLOR_ONLY
: If this bit is set then the entry will only be considered if the display for *tkwin* has more than one bit plane. If the display is monochromatic then this *specs* entry will be ignored.

TK_CONFIG_MONO_ONLY
: If this bit is set then the entry will only be considered if the display for *tkwin* has exactly one bit plane. If the display is not monochromatic then this *specs* entry will be ignored.

TK_CONFIG_NULL_OK
: This bit is only relevant for some types of entries (see the descriptions of the various entry types above). If this bit is set, it indicates that an empty string value for the field is acceptable and if it occurs then the target should be set to NULL or **None**, depending on the type of the target. This flag is typically used to allow a feature to be turned off entirely, e.g. set a cursor value to **None** so that a window simply inherits its parent's cursor. If this bit is not set then empty strings are processed as strings, which generally results in an error.

TK_CONFIG_DONT_SET_DEFAULT
: If this bit is one, it means that the *defValue* field of the entry should only be used for returning the default value in **Tk_ConfigureInfo**. In calls to **Tk_ConfigureWidget** no default will be supplied for entries with this flag set; it is assumed that the caller has already supplied a default value in the target location. This flag provides a performance optimization where it is expensive to process the default string: the client can compute the default once, save the value, and provide it before calling **Tk_ConfigureWidget**.

TK_CONFIG_OPTION_SPECIFIED
: This bit is deprecated. It used to be set and cleared by **Tk_ConfigureWidget** so that callers could detect what entries were specified in *argv*, but it was removed because it was inherently thread-unsafe. Code that wishes to detect what options were specified should use

Tk_SetOptions instead. The **TK_CONFIG_MONO_ONLY** and **TK_CONFIG_COLOR_ONLY** flags are typically used to specify different default values for monochrome and color displays. This is done by creating two entries in *specs* that are identical except for their *defValue* and *specFlags* fields. One entry should have the value **TK_CONFIG_MONO_ONLY** in its *specFlags* and the default value for monochrome displays in its *defValue*; the other entry should have the value **TK_CONFIG_COLOR_ONLY** in its *specFlags* and the appropriate *defValue* for color displays. Third, it is possible to use *flags* and *specFlags* together to selectively disable some entries. This feature is not needed very often. It is useful in cases where several similar kinds of widgets are implemented in one place. It allows a single *specs* table to be created with all the configuration options for all the widget types. When processing a particular widget type, only entries relevant to that type will be used. This effect is achieved by setting the high-order bits (those in positions equal to or greater than **TK_CONFIG_USER_BIT**) in *specFlags* values or in *flags*. In order for a particular entry in *specs* to be used, its high-order bits must match exactly the high-order bits of the *flags* value passed to **Tk_ConfigureWidget**. If a *specs* table is being used for N different widget types, then N of the high-order bits will be used. Each *specs* entry will have one of more of those bits set in its *specFlags* field to indicate the widget types for which this entry is valid. When calling **Tk_ConfigureWidget**, *flags* will have a single one of these bits set to select the entries for the desired widget type. For a working example of this feature, see the code in tkButton.c.

Tk_offset

The **Tk_Offset** macro is provided as a safe way of generating the *offset* values for entries in Tk_ConfigSpec structures. It takes two arguments: the name of a type of record, and the name of a field in that record. It returns the byte offset of the named field in records of the given type.

Tk_configureinfo

The **Tk_ConfigureInfo** procedure may be used to obtain information about one or all of the options for a given widget. Given a token for a window (*tkwin*), a table describing the configuration options for a class of widgets (*specs*), a pointer to a widget record containing the current information for a widget (*widgRec*), and a NULL *argvName* argument, **Tk_ConfigureInfo** generates a string describing all of the configuration options for the window. The string is placed in interpreter *interp*'s result. Under normal circumstances it returns **TCL_OK**; if an error occurs then it returns **TCL_ERROR** and the interpreter's result will contain an error message.

If *argvName* is NULL, then the value left in the interpreter's result by **Tk_ConfigureInfo** consists of a list of one or more entries, each of which describes one configuration option (i.e. one entry in *specs*). Each entry in the list will contain either two or five values. If the corresponding entry in *specs* has type **TK_CONFIG_SYNONYM**, then the list will contain two values: the *argvName* for the entry and the *dbName* (synonym name). Otherwise the list will contain five values: *argvName*, *dbName*, *dbClass*, *defValue*, and current value. The current value is computed from the appropriate field of *widgRec* by calling procedures like **Tk_NameOfColor**.

If the *argvName* argument to **Tk_ConfigureInfo** is non-NULL, then it indicates a single option, and information is returned only for that option. The string placed in the interpreter's result will be a list containing two or five values as described above; this will be identical to the corresponding sublist that would have been returned if *argvName* had been NULL.

The *flags* argument to **Tk_ConfigureInfo** is used to restrict the *specs* entries to consider, just as for **Tk_ConfigureWidget**.

Tk_configurevalue

Tk_ConfigureValue takes arguments similar to **Tk_ConfigureInfo**; instead of returning a list of values, it just returns the current value of the option given by *argvName* (*argvName* must not be NULL). The value is returned in interpreter *interp*'s result and **TCL_OK** is normally returned as the procedure's result. If an error occurs in **Tk_ConfigureValue** (e.g., *argvName* is not a valid option name), **TCL_ERROR** is returned and an error message is left in the interpreter's result. This procedure is typically called to implement **cget** widget commands.

Tk_freeoptions

The **Tk_FreeOptions** procedure may be invoked during widget cleanup to release all of the resources associated with configuration options. It scans through *specs* and for each entry corresponding to a resource that must be explicitly freed (e.g. those with type **TK_CONFIG_COLOR**), it frees the resource in the widget record. If the field in the widget record does not refer to a resource (e.g. it contains a null pointer) then no resource is freed for that entry. After freeing a resource, **Tk_FreeOptions** sets the corresponding field of the widget record to null.

Custom option types

Applications can extend the built-in configuration types with additional configuration types by writing procedures to parse and print options of the a type and creating a structure pointing to those procedures:

```
typedef struct Tk_CustomOption {
    Tk_OptionParseProc *parseProc;
    Tk_OptionPrintProc *printProc;
    ClientData clientData;
} Tk_CustomOption;

typedef int Tk_OptionParseProc(
        ClientData clientData,
        Tcl_Interp *interp,
        Tk_Window tkwin,
        const char *value,
        char *widgRec,
        int offset);

typedef const char *Tk_OptionPrintProc(
        ClientData clientData,
        Tk_Window tkwin,
        char *widgRec,
        int offset,
        Tcl_FreeProc **freeProcPtr);
```

The Tk_CustomOption structure contains three fields, which are pointers to the two procedures and a *clientData* value to be passed to those procedures when they are invoked. The *clientData* value typically points to a structure containing information that is needed by the procedures when they are parsing and printing options.

The *parseProc* procedure is invoked by **Tk_ConfigureWidget** to parse a string and store the resulting value in the widget record. The *clientData* argument is a copy of the *clientData* field in the Tk_CustomOption structure. The *interp* argument points to a Tcl interpreter used for error reporting. *Tkwin* is a copy of the *tkwin* argument to **Tk_ConfigureWidget**. The *value* argument is a string describing the value for the option; it could have been specified explicitly in the call to **Tk_ConfigureWidget** or it could come from the option database or a default. *Value* will never be a null pointer but it may point to an empty string. *RecordPtr* is the same as the *widgRec* argument to **Tk_ConfigureWidget**; it points to the start of the widget record to modify. The last argument, *offset*, gives the offset in bytes from the start of the widget record to the location where the option value is to be placed. The procedure should translate the string to whatever form is appropriate for the option and store the value in the widget record. It should normally return **TCL_OK**, but if an error occurs in translating the string to a value then it should return **TCL_ERROR** and store an error message in interpreter *interp*'s result.

The *printProc* procedure is called by **Tk_ConfigureInfo** to produce a string value describing an existing option. Its *clientData*, *tkwin*, *widgRec*, and *offset* arguments all have the same meaning as for Tk_OptionParseProc procedures. The *printProc* procedure should examine the option whose value is stored at *offset* in *widgRec*, produce a string describing that option, and return a pointer to the string. If the string is stored in dynamically-allocated memory, then the procedure must set **freeProcPtr* to the address of a procedure to call to free the string's memory; **Tk_ConfigureInfo** will call this procedure when it is finished with the string. If the result string is stored in static memory then *printProc* need not do anything with the *freeProcPtr* argument.

Once *parseProc* and *printProc* have been defined and a Tk_CustomOption structure has been created for them, options of this new type may be manipulated with Tk_ConfigSpec entries whose *type* fields are **TK_CONFIG_CUSTOM** and whose *customPtr* fields point to the Tk_CustomOption structure.

Examples

Although the explanation of **Tk_ConfigureWidget** is fairly complicated, its actual use is pretty straightforward. The easiest way to get started is to copy the code from an existing widget. The library implementation of frames (tkFrame.c) has a simple configuration table, and the library implementation of buttons (tkButton.c) has a much more complex table that uses many of the fancy *specFlags* mechanisms.

Keywords: ConfigWidg, anchor, bitmap, boolean, border, cap style, color, configuration options, cursor, custom, double, font, integer, join style, justify, millimeters, pixels, relief, synonym, uid

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

ConfigWind

Name

Tk_ConfigureWindow, Tk_MoveWindow, Tk_ResizeWindow, Tk_MoveResizeWindow, Tk_SetWindowBorderWidth, Tk_ChangeWindowAttributes, Tk_SetWindowBackground, Tk_SetWindowBackgroundPixmap, Tk_SetWindowBorder, Tk_SetWindowBorderPixmap, Tk_SetWindowColormap, Tk_DefineCursor, Tk_UndefineCursor - change window configuration or attributes

Synopsis

#include <tk.h>

Tk_ConfigureWindow(*tkwin*, *valueMask*, *valuePtr*)

Tk_MoveWindow(*tkwin*, *x*, *y*)

Tk_ResizeWindow(*tkwin*, *width*, *height*)

Tk_MoveResizeWindow(*tkwin*, *x*, *y*, *width*, *height*)

Tk_SetWindowBorderWidth(*tkwin*, *borderWidth*)

Tk_ChangeWindowAttributes(*tkwin*, *valueMask*, *attsPtr*)

Tk_SetWindowBackground(*tkwin*, *pixel*)

Tk_SetWindowBackgroundPixmap(*tkwin*, *pixmap*)

Tk_SetWindowBorder(*tkwin*, *pixel*)

Tk_SetWindowBorderPixmap(*tkwin*, *pixmap*)

Tk_SetWindowColormap(*tkwin*, *colormap*)

Tk_DefineCursor(*tkwin*, *cursor*)

Tk_UndefineCursor(*tkwin*)

Arguments

Tk_Window *tkwin* (in)
Token for window.

unsigned int *valueMask* (in)
OR-ed mask of values like **CWX** or **CWBorderPixel**, indicating which fields of **valuePtr* or **attsPtr* to use.

XWindowChanges **valuePtr* (in)
Points to a structure containing new values for the configuration parameters selected by *valueMask*. Fields not selected by *valueMask* are ignored.

int *x* (in)
New x-coordinate for *tkwin*'s top left pixel (including border, if any) within tkwin's parent.

int *y* (in)
New y-coordinate for *tkwin*'s top left pixel (including border, if any) within tkwin's parent.

int *width* (in)
New width for *tkwin* (interior, not including border).

int *height* (in)
New height for *tkwin* (interior, not including border).

int *borderWidth* (in)
New width for *tkwin*'s border.

XSetWindowAttributes **attsPtr* (in)
Points to a structure containing new values for the attributes given by the *valueMask* argument. Attributes not selected by *valueMask* are ignored.

unsigned long *pixel* (in)
New background or border color for window.

Pixmap *pixmap* (in)

New pixmap to use for background or border of *tkwin*. WARNING: cannot necessarily be deleted immediately, as for Xlib calls. See note below.

Colormap *colormap* (in)
New colormap to use for *tkwin*.

Tk_Cursor *cursor* (in)
New cursor to use for *tkwin*. If **None** is specified, then *tkwin* will not have its own cursor; it will use the cursor of its parent.

Description

These procedures are analogous to the X library procedures with similar names, such as **XConfigureWindow**. Each one of the above procedures calls the corresponding X procedure and also saves the configuration information in Tk's local structure for the window. This allows the information to be retrieved quickly by the application (using macros such as **Tk_X** and **Tk_Height**) without having to contact the X server. In addition, if no X window has actually been created for *tkwin* yet, these procedures do not issue X operations or cause event handlers to be invoked; they save the information in Tk's local structure for the window; when the window is created later, the saved information will be used to configure the window.

See the X library documentation for details on what these procedures do and how they use their arguments.

In the procedures **Tk_ConfigureWindow**, **Tk_MoveWindow**, **Tk_ResizeWindow**, **Tk_MoveResizeWindow**, and **Tk_SetWindowBorderWidth**, if *tkwin* is an internal window then event handlers interested in configure events are invoked immediately, before the procedure returns. If *tkwin* is a top-level window then the event handlers will be invoked later, after X has seen the request and returned an event for it.

Applications using Tk should never call procedures like **XConfigureWindow** directly; they should always use the corresponding Tk procedures.

The size and location of a window should only be modified by the appropriate geometry manager for that window and never by a window itself (but see **Tk_MoveToplevelWindow** for moving a top-level window).

You may not use **Tk_ConfigureWindow** to change the stacking order of a window (*valueMask* may not contain the **CWSibling** or **CWStackMode** bits). To change the stacking order, use the procedure **Tk_RestackWindow**.

The procedure **Tk_SetWindowColormap** will automatically add *tkwin* to the **TK_COLORMAP_WINDOWS** property of its nearest top-level ancestor if the new colormap is different from that of *tkwin*'s parent and *tkwin* is not already in the **TK_COLORMAP_WINDOWS** property.

Bugs

Tk_SetWindowBackgroundPixmap and **Tk_SetWindowBorderPixmap** differ slightly from their Xlib counterparts in that the *pixmap* argument may not necessarily be deleted immediately after calling one of these procedures. This is because *tkwin*'s window may not exist yet at the time of the call, in which case *pixmap* is merely saved and used later when *tkwin*'s window is actually created. If you wish to delete *pixmap*, then call **Tk_MakeWindowExist** first to be sure that *tkwin*'s window exists and *pixmap* has been passed to the X server.

A similar problem occurs for the *cursor* argument passed to **Tk_DefineCursor**. The solution is the same as for pixmaps above: call **Tk_MakeWindowExist** before freeing the cursor.

Keywords: ConfigWind, attributes, border, color, configure, height, pixel, pixmap, width, window, x, y

Copyright (c) 1990-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

CoordToWin

Name

Tk_CoordsToWindow - Find window containing a point

Synopsis

#include <tk.h>

Tk_Window
Tk_CoordsToWindow(*rootX*, *rootY*, *tkwin*)

Arguments

int *rootX* (in)
X-coordinate (in root window coordinates).

int *rootY* (in)
Y-coordinate (in root window coordinates).

Tk_Window *tkwin* (in)
Token for window that identifies application.

Description

Tk_CoordsToWindow locates the window that contains a given point. The point is specified in root coordinates with *rootX* and *rootY* (if a virtual-root window manager is in use then *rootX* and *rootY* are in the coordinate system of the virtual root window). The return value from the procedure is a token for the window that contains the given point. If the point is not in any window, or if the containing window is not in the same application as *tkwin*, then NULL is returned.

The containing window is decided using the same rules that determine which window contains the mouse cursor: if a parent and a child both contain the point then the child gets preference, and if two siblings both contain the point then the highest one in the stacking order (i.e. the one that's visible on the screen) gets preference.

Keywords: CoordToWin, containing, coordinates, root window

Copyright (c) 1990-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

CrtCmHdlr

Name

Tk_CreateClientMessageHandler, Tk_DeleteClientMessageHandler - associate procedure callback with ClientMessage type X events

Synopsis

#include <tk.h>

Tk_CreateClientMessageHandler(*proc*)

Tk_DeleteClientMessageHandler(*proc*)

Arguments

Tk_ClientMessageProc *_proc_ (in)

Procedure to invoke whenever a ClientMessage X event occurs on any display.

Description

Tk_CreateClientMessageHandler arranges for *proc* to be invoked in the future whenever a ClientMessage X event occurs that is not handled by **WM_PROTOCOL**.
Tk_CreateClientMessageHandler is intended for use by applications which need to watch X ClientMessage events, such as drag and drop applications.

The callback to *proc* will be made by **Tk_HandleEvent**; this mechanism only works in programs that dispatch events through **Tk_HandleEvent** (or through other Tk procedures that call **Tk_HandleEvent**, such as **Tcl_DoOneEvent** or **Tk_MainLoop**).

Proc should have arguments and result that match the type **Tk_ClientMessageProc**:

```
typedef int Tk_ClientMessageProc(
        Tk_Window tkwin,
        XEvent *eventPtr);
```

The *tkwin* parameter to *proc* is the Tk window which is associated with this event. *EventPtr* is a pointer to the X event.

Whenever an X ClientMessage event is processed by **Tk_HandleEvent**, the *proc* is called if it was not handled as a **WM_PROTOCOL**. The return value from *proc* is normally 0. A non-zero return value indicates that the event is not to be handled further; that is, *proc* has done all processing that is to be allowed for the event.

If there are multiple ClientMessage event handlers, each one is called for each event, in the order in which they were established.

Tk_DeleteClientMessageHandler may be called to delete a previously-created ClientMessage event handler: it deletes each handler it finds that matches the *proc* argument. If no such handler exists, then **Tk_DeleteClientMessageHandler** returns without doing anything. Although Tk supports it, it's probably a bad idea to have more than one callback with the same *proc* argument.

Keywords: CrtCmHdlr, bind, callback, event, handler

Copyright (c) 2000 Ajuba Solutions.

CrtConsoleChan

Name

Tk_InitConsoleChannels - Install the console channels as standard channels

Synopsis

#include <tk.h>

Tk_InitConsoleChannels(*interp*)

Arguments

Tcl_Interp ****interp* **(in)**
Interpreter in which the console channels are created.

Description

Tk_InitConsoleChannels is invoked to create a set of console channels and install them as the standard channels. All I/O on these channels will be discarded until **Tk_CreateConsoleWindow** is called to attach the console to a text widget.

This function is for use by shell applications based on Tk, like **wish**, on platforms which have no standard channels in graphical mode, like Win32.

The *interp* argument is the interpreter in which to create and install the console channels.

NOTE: If this function is used it has to be called before the first call to **Tcl_RegisterChannel**, directly, or indirectly through other channel functions. Because otherwise the standard channels will be already initialized to the system defaults, which will be nonsensical for the case **Tk_InitConsoleChannels** is for.

Keywords: CrtConsoleChan, standard channels, console

Copyright (c) 2007 ActiveState Software Inc.

CrtErrHdlr

Name

Tk_CreateErrorHandler, Tk_DeleteErrorHandler - handle X protocol errors

Synopsis

#include <tk.h>

Tk_ErrorHandler
Tk_CreateErrorHandler(*display*, *error*, *request*, *minor*, *proc*, *clientData*)

Tk_DeleteErrorHandler(*handler*)

Arguments

Display ****display* **(in)**
Display whose errors are to be handled.

int *error* (in)
Match only error events with this value in the *error_code* field. If -1, then match any *error_code* value.

int *request* (in)
Match only error events with this value in the *request_code* field. If -1, then match any *request_code* value.

int *minor* (in)
Match only error events with this value in the *minor_code* field. If -1, then match any *minor_code* value.

Tk_ErrorProc ****proc* **(in)
Procedure to invoke whenever an error event is received for *display* and matches *error*, *request*, and *minor*. NULL means ignore any matching errors.

ClientData *clientData* (in)
Arbitrary one-word value to pass to *proc*.

Tk_ErrorHandler *handler* (in)
Token for error handler to delete (return value from a previous call to **Tk_CreateErrorHandler**).

Description

Tk_CreateErrorHandler arranges for a particular procedure (*proc*) to be called whenever certain protocol errors occur on a particular display (*display*). Protocol errors occur when the X protocol is used incorrectly, such as attempting to map a window that does not exist. See the Xlib documentation for **XSetErrorHandler** for more information on the kinds of errors that can occur. For *proc* to be invoked to handle a particular error, five things must occur:

[1]
The error must pertain to *display*.

[2]
Either the *error* argument to **Tk_CreateErrorHandler** must have been -1, or the *error* argument must match the *error_code* field from the error event.

[3]
Either the *request* argument to **Tk_CreateErrorHandler** must have been -1, or the *request* argument must match the *request_code* field from the error event.

[4]
Either the *minor* argument to **Tk_CreateErrorHandler** must have been -1, or the *minor* argument must match the *minor_code* field from the error event.

[5]
The protocol request to which the error pertains must have been made when the handler was active (see below for more information). *Proc* should have arguments and result that match the following type:

```
typedef int Tk_ErrorProc(
        ClientData clientData,
        XErrorEvent *errEventPtr);
```

The *clientData* parameter to *proc* is a copy of the *clientData* argument given to **Tcl_CreateErrorHandler** when the callback was created. Typically, *clientData* points to a data structure containing application-specific information that is needed to deal with the error. *ErrEventPtr* is a pointer to the X error event. The procedure *proc* should return an integer value. If it returns 0 it means that *proc* handled the error completely and there is no need to take any other action for the error. If it returns non-zero it means *proc* was unable to handle the error.

If a value of NULL is specified for *proc*, all matching errors will be ignored: this will produce the same result as if a procedure had been specified that always returns 0.

If more than more than one handler matches a particular error, then they are invoked in turn. The handlers will be invoked in reverse order of creation: most recently declared handler first. If

any handler returns 0, then subsequent (older) handlers will not be invoked. If no handler returns 0, then Tk invokes X's default error handler, which prints an error message and aborts the program. If you wish to have a default handler that deals with errors that no other handler can deal with, then declare it first.

The X documentation states that "the error handler should not call any functions (directly or indirectly) on the display that will generate protocol requests or that will look for input events." This restriction applies to handlers declared by **Tk_CreateErrorHandler**; disobey it at your own risk.

Tk_DeleteErrorHandler may be called to delete a previously-created error handler. The *handler* argument identifies the error handler, and should be a value returned by a previous call to **Tk_CreateEventHandler**.

A particular error handler applies to errors resulting from protocol requests generated between the call to **Tk_CreateErrorHandler** and the call to **Tk_DeleteErrorHandler**. However, the actual callback to *proc* may not occur until after the **Tk_DeleteErrorHandler** call, due to buffering in the client and server. If an error event pertains to a protocol request made just before calling **Tk_DeleteErrorHandler**, then the error event may not have been processed before the **Tk_DeleteErrorHandler** call. When this situation arises, Tk will save information about the handler and invoke the handler's *proc* later when the error event finally arrives. If an application wishes to delete an error handler and know for certain that all relevant errors have been processed, it should first call **Tk_DeleteErrorHandler** and then call **XSync**; this will flush out any buffered requests and errors, but will result in a performance penalty because it requires communication to and from the X server. After the **XSync** call Tk is guaranteed not to call any error handlers deleted before the **XSync** call.

For the Tk error handling mechanism to work properly, it is essential that application code never calls **XSetErrorHandler** directly; applications should use only **Tk_CreateErrorHandler**.

Keywords: CrtErrHdlr, callback, error, event, handler

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

CrtGenHdlr

Name

Tk_CreateGenericHandler, Tk_DeleteGenericHandler - associate procedure callback with all X events

Synopsis

#include <tk.h>

Tk_CreateGenericHandler(*proc*, *clientData*)

Tk_DeleteGenericHandler(*proc*, *clientData*)

Arguments

Tk_GenericProc ****proc* **(in)
Procedure to invoke whenever any X event occurs on any display.

ClientData ***clientData*** **(in)**
Arbitrary one-word value to pass to *proc*.

Description

Tk_CreateGenericHandler arranges for *proc* to be invoked in the future whenever any X event occurs. This mechanism is *not* intended for dispatching X events on windows managed by Tk (you should use **Tk_CreateEventHandler** for this purpose). **Tk_CreateGenericHandler** is intended for other purposes, such as tracing X events, monitoring events on windows not owned by Tk, accessing X-related libraries that were not originally designed for use with Tk, and so on.

The callback to *proc* will be made by **Tk_HandleEvent**; this mechanism only works in programs that dispatch events through **Tk_HandleEvent** (or through other Tk procedures that call **Tk_HandleEvent**, such as **Tcl_DoOneEvent** or **Tk_MainLoop**).

Proc should have arguments and result that match the type **Tk_GenericProc**:

```
typedef int Tk_GenericProc(
        ClientData clientData,
        XEvent *eventPtr);
```

The *clientData* parameter to *proc* is a copy of the *clientData* argument given to **Tk_CreateGenericHandler** when the callback was created. Typically, *clientData* points to a data structure containing application-specific information about how to handle events. *EventPtr* is a pointer to the X event.

Whenever an X event is processed by **Tk_HandleEvent**, *proc* is called. The return value from *proc* is normally 0. A non-zero return value indicates that the event is not to be handled further; that is, *proc* has done all processing that is to be allowed for the event.

If there are multiple generic event handlers, each one is called for each event, in the order in which they were established.

Tk_DeleteGenericHandler may be called to delete a previously-created generic event handler: it deletes each handler it finds that matches the *proc* and *clientData* arguments. If no such handler exists, then **Tk_DeleteGenericHandler** returns without doing anything. Although Tk supports it, it's probably a bad idea to have more than one callback with the same *proc* and *clientData* arguments.

Establishing a generic event handler does nothing to ensure that the process will actually receive the X events that the handler wants to process. For example, it is the caller's responsibility to invoke **XSelectInput** to select the desired events, if that is necessary.

Keywords: CrtGenHdlr, bind, callback, event, handler

Copyright (c) 1992-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

CrtImgType

Name

Tk_CreateImageType, Tk_GetImageMasterData, Tk_GetImageModelData, Tk_InitImageArgs - define new kind of image

Synopsis

#include <tk.h>

Tk_CreateImageType(*typePtr*)

ClientData
Tk_GetImageMasterData(*interp*, *name*, *typePtrPtr*)

ClientData
Tk_GetImageModelData(*interp*, *name*, *typePtrPtr*)

Tk_InitImageArgs(*interp*, *argc*, *argvPtr*)

Arguments

const Tk_ImageType *_typePtr_ (in)
Structure that defines the new type of image. For Tk 8.4 and earlier this must be static: a pointer to this structure is retained by the image code. In Tk 8.5, this limitation was relaxed.

Tcl_Interp *_interp_ (in)
Interpreter in which image was created.

const char *_name_ (in)
Name of existing image.

Tk_ImageType **_typePtrPtr_ (out)
Points to word in which to store a pointer to type information for the given image, if it exists.

int _argc_ (in)
Number of arguments

char ***_argvPtr_ (in/out)
Pointer to argument list

Description

Tk_CreateImageType is invoked to define a new kind of image. An image type corresponds to a particular value of the *type* argument for the **image create** command. There may exist any number of different image types, and new types may be defined dynamically by calling **Tk_CreateImageType**. For example, there might be one type for 2-color bitmaps, another for multi-color images, another for dithered images, another for video, and so on.

The code that implements a new image type is called an *image manager*. It consists of a collection of procedures plus three different kinds of data structures. The first data structure is a Tk_ImageType structure, which contains the name of the image type and pointers to five procedures provided by the image manager to deal with images of this type:

```
typedef struct Tk_ImageType {
    const char *name;
    Tk_ImageCreateProc *createProc;
    Tk_ImageGetProc *getProc;
    Tk_ImageDisplayProc *displayProc;
    Tk_ImageFreeProc *freeProc;
    Tk_ImageDeleteProc *deleteProc;
} Tk_ImageType;
```

The fields of this structure will be described in later subsections of this entry.

The second major data structure manipulated by an image manager is called an *image model*; it contains overall information about a particular image, such as the values of the configuration options specified in an **image create** command. There will usually be one of these structures for each invocation of the **image create** command.

The third data structure related to images is an *image instance*. There will usually be one of these structures for each usage of an image in a particular widget. It is possible for a single image to appear simultaneously in multiple widgets, or even multiple times in the same widget. Furthermore, different instances may be on different screens or displays. The image instance data structure describes things that may vary from instance to instance, such as colors and graphics contexts for redisplay. There is usually one instance structure for each **-image** option specified for a widget or canvas item.

The following subsections describe the fields of a Tk_ImageType in more detail.

Name

typePtr->name provides a name for the image type. Once **Tk_CreateImageType** returns, this name may be used in **image create** commands to create images of the new type. If there already existed an image type by this name then the new image type replaces the old one.

Createproc

typePtr->createProc provides the address of a procedure for Tk to call whenever **image create** is invoked to create an image of the new type. *typePtr->createProc* must match the following prototype:

```
typedef int Tk_ImageCreateProc(
        Tcl_Interp *interp,
        const char *name,
        int objc,
        Tcl_Obj *const objv[],
        const Tk_ImageType *typePtr,
        Tk_ImageMaster model,
        ClientData *modelDataPtr);
```

The *interp* argument is the interpreter in which the **image** command was invoked, and *name* is the name for the new image, which was either specified explicitly in the **image** command or generated automatically by the **image** command. The *objc* and *objv* arguments describe all the configuration options for the new image (everything after the name argument to **image**). The *model* argument is a token that refers to Tk's information about this image; the image manager must return this token to Tk when invoking the **Tk_ImageChanged** procedure. Typically *createProc* will parse *objc* and *objv* and create an image model data structure for the new image. *createProc* may store an arbitrary one-word value at **modelDataPtr*, which will be passed back to the image manager when other callbacks are invoked. Typically the value is a pointer to the model data structure for the image.

If *createProc* encounters an error, it should leave an error message in the interpreter result and return **TCL_ERROR**; otherwise it should return **TCL_OK**.

createProc should call **Tk_ImageChanged** in order to set the size of the image and request an initial redisplay.

Getproc

typePtr->getProc is invoked by Tk whenever a widget calls **Tk_GetImage** to use a particular image. This procedure must match the following prototype:

```
typedef ClientData Tk_ImageGetProc(
        Tk_Window tkwin,
        ClientData modelData);
```

The *tkwin* argument identifies the window in which the image will be used and *modelData* is the value returned by *createProc* when the image model was created. *getProc* will usually create a data structure for the new instance, including such things as the resources needed to display the image in the given window. *getProc* returns a one-word token for the instance, which is typically the address of the instance data structure. Tk will pass this value back to the image manager when invoking its *displayProc* and *freeProc* procedures.

Displayproc

typePtr->displayProc is invoked by Tk whenever an image needs to be displayed (i.e., whenever a widget calls **Tk_RedrawImage**). *displayProc* must match the following prototype:

```
typedef void Tk_ImageDisplayProc(
        ClientData instanceData,
        Display *display,
        Drawable drawable,
        int imageX,
        int imageY,
        int width,
        int height,
        int drawableX,
        int drawableY);
```

The *instanceData* will be the same as the value returned by *getProc* when the instance was created. *display* and *drawable* indicate where to display the image; *drawable* may be a pixmap rather than the window specified to *getProc* (this is usually the case, since most widgets double-buffer their redisplay to get smoother visual effects). *imageX*, *imageY*, *width*, and *height* identify the region of the image that must be redisplayed. This region will always be within the size of the image as specified in the most recent call to **Tk_ImageChanged**. *drawableX* and *drawableY* indicate where in *drawable* the image should be displayed; *displayProc* should display the given region of the image so that point (*imageX*, *imageY*) in the image appears at (*drawableX*, *drawableY*) in *drawable*.

Freeproc

typePtr->freeProc contains the address of a procedure that Tk will invoke when an image instance is released (i.e., when **Tk_FreeImage** is invoked). This can happen, for example, when a widget is deleted or a image item in a canvas is deleted, or when the image displayed in a widget or canvas item is changed. *freeProc* must match the following prototype:

```
typedef void Tk_ImageFreeProc(
        ClientData instanceData,
        Display *display);
```

The *instanceData* will be the same as the value returned by *getProc* when the instance was created, and *display* is the display containing the window for the instance. *freeProc* should release any resources associated with the image instance, since the instance will never be used

again.

Deleteproc

typePtr->deleteProc is a procedure that Tk invokes when an image is being deleted (i.e. when the **image delete** command is invoked). Before invoking *deleteProc* Tk will invoke *freeProc* for each of the image's instances. *deleteProc* must match the following prototype:

```
typedef void Tk_ImageDeleteProc(
        ClientData modelData);
```

The *modelData* argument will be the same as the value stored in **modelDataPtr* by *createProc* when the image was created. *deleteProc* should release any resources associated with the image.

Tk_getimagemodeldata

The procedure **Tk_GetImageMasterData** may be invoked to retrieve information about an image. For example, an image manager can use this procedure to locate its image model data for an image. If there exists an image named *name* in the interpreter given by *interp*, then **typePtrPtr* is filled in with type information for the image (the *typePtr* value passed to **Tk_CreateImageType** when the image type was registered) and the return value is the ClientData value returned by the *createProc* when the image was created (this is typically a pointer to the image model data structure). If no such image exists then NULL is returned and NULL is stored at **typePtrPtr*.

Tk_GetImageModelData is synonym for **Tk_GetImageMasterData**

Legacy interface support

In Tk 8.2 and earlier, the definition of **Tk_ImageCreateProc** was incompatibly different, with the following prototype:

```
typedef int Tk_ImageCreateProc(
        Tcl_Interp *interp,
        char *name,
        int argc,
        char **argv,
        Tk_ImageType *typePtr,
        Tk_ImageMaster model,
        ClientData *modelDataPtr);
```

Legacy programs and libraries dating from those days may still contain code that defines extended Tk image types using the old interface. The Tk header file will still support this legacy interface if the code is compiled with the macro **USE_OLD_IMAGE** defined.

Tk_ImageModel is synonym for *Tk_ImageMaster*

When the **USE_OLD_IMAGE** legacy support is enabled, you may see the routine **Tk_InitImageArgs** in use. This was a migration tool used to create stub-enabled extensions that could be loaded into interps containing all versions of Tk 8.1 and later. Tk 8.5 no longer provides this routine, but uses a macro to convert any attempted calls of this routine into an empty comment. Any stub-enabled extension providing an extended image type via the legacy interface that is compiled against Tk 8.5 headers and linked against the Tk 8.5 stub library will

produce a file that can be loaded only into interps with Tk 8.5 or later; that is, the normal stub-compatibility rules. If a developer needs to generate from such code a file that is loadable into interps with Tk 8.4 or earlier, they must use Tk 8.4 headers and stub libraries to do so.

Any new code written today should not make use of the legacy interfaces. Expect their support to go away in Tk 9.

Keywords: CrtImgType, image manager, image type, instance, model

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

CrtItemType

Name

Tk_CreateItemType, Tk_GetItemTypes - define new kind of canvas item

Synopsis

#include <tk.h>

Tk_CreateItemType(*typePtr*)

Tk_ItemType *
Tk_GetItemTypes()

Arguments

Tk_ItemType ****typePtr* **(in)
Structure that defines the new type of canvas item.

Introduction

Tk_CreateItemType is invoked to define a new kind of canvas item described by the *typePtr* argument. An item type corresponds to a particular value of the *type* argument to the **create** widget command for canvases, and the code that implements a canvas item type is called a *type manager*. Tk defines several built-in item types, such as **rectangle** and **text** and **image**, but **Tk_CreateItemType** allows additional item types to be defined. Once **Tk_CreateItemType** returns, the new item type may be used in new or existing canvas widgets just like the built-in item types.

Tk_GetItemTypes returns a pointer to the first in the list of all item types currently defined for canvases. The entries in the list are linked together through their *nextPtr* fields, with the end of the list marked by a NULL *nextPtr*.

You may find it easier to understand the rest of this manual entry by looking at the code for an existing canvas item type such as bitmap (in the file tkCanvBmap.c) or text (tkCanvText.c). The easiest way to create a new type manager is to copy the code for an existing type and modify it for the new type.

Tk provides a number of utility procedures for the use of canvas type managers, such as **Tk_CanvasCoords** and **Tk_CanvasPsColor**; these are described in separate manual entries.

Data structures

A type manager consists of a collection of procedures that provide a standard set of operations on items of that type. The type manager deals with three kinds of data structures. The first data

structure is a Tk_ItemType; it contains information such as the name of the type and pointers to the standard procedures implemented by the type manager:

```
typedef struct Tk_ItemType {
    const char *name;
    int itemSize;
    Tk_ItemCreateProc *createProc;
    const Tk_ConfigSpec *configSpecs;
    Tk_ItemConfigureProc *configProc;
    Tk_ItemCoordProc *coordProc;
    Tk_ItemDeleteProc *deleteProc;
    Tk_ItemDisplayProc *displayProc;
    int alwaysRedraw;
    Tk_ItemPointProc *pointProc;
    Tk_ItemAreaProc *areaProc;
    Tk_ItemPostscriptProc *postscriptProc;
    Tk_ItemScaleProc *scaleProc;
    Tk_ItemTranslateProc *translateProc;
    Tk_ItemIndexProc *indexProc;
    Tk_ItemCursorProc *icursorProc;
    Tk_ItemSelectionProc *selectionProc;
    Tk_ItemInsertProc *insertProc;
    Tk_ItemDCharsProc *dCharsProc;
    Tk_ItemType *nextPtr;
} Tk_ItemType;
```

The fields of a Tk_ItemType structure are described in more detail later in this manual entry. When **Tk_CreateItemType** is called, its *typePtr* argument must point to a structure with all of the fields initialized except *nextPtr*, which Tk sets to link all the types together into a list. The structure must be in permanent memory (either statically allocated or dynamically allocated but never freed); Tk retains a pointer to this structure.

The second data structure manipulated by a type manager is an *item record*. For each item in a canvas there exists one item record. All of the items of a given type generally have item records with the same structure, but different types usually have different formats for their item records. The first part of each item record is a header with a standard structure defined by Tk via the type Tk_Item; the rest of the item record is defined by the type manager. A type manager must define its item records with a Tk_Item as the first field. For example, the item record for bitmap items is defined as follows:

```
typedef struct BitmapItem {
    Tk_Item header;
    double x, y;
    Tk_Anchor anchor;
    Pixmap bitmap;
    XColor *fgColor;
    XColor *bgColor;
    GC gc;
} BitmapItem;
```

The *header* substructure contains information used by Tk to manage the item, such as its identifier, its tags, its type, and its bounding box. The fields starting with *x* belong to the type manager: Tk will never read or write them. The type manager should not need to read or write any of the fields in the header except for four fields whose names are *x1*, *y1*, *x2*, and *y2*. These fields give a bounding box for the items using integer canvas coordinates: the item should not cover any pixels with x-coordinate lower than *x1* or y-coordinate lower than *y1*, nor should it cover any pixels with x-coordinate greater than or equal to *x2* or y-coordinate greater than or equal to *y2*. It is up to the type manager to keep the bounding box up to date as the item is moved and reconfigured.

Whenever Tk calls a procedure in a type manager it passes in a pointer to an item record. The argument is always passed as a pointer to a Tk_Item; the type manager will typically cast this into a pointer to its own specific type, such as BitmapItem.

The third data structure used by type managers has type Tk_Canvas; it serves as an opaque handle for the canvas widget as a whole. Type managers need not know anything about the contents of this structure. A Tk_Canvas handle is typically passed in to the procedures of a type manager, and the type manager can pass the handle back to library procedures such as Tk_CanvasTkwin to fetch information about the canvas.

Tk_itemtype fields

Name

This section and the ones that follow describe each of the fields in a Tk_ItemType structure in detail. The *name* field provides a string name for the item type. Once **Tk_CreateImageType** returns, this name may be used in **create** widget commands to create items of the new type. If there already existed an item type by this name then the new item type replaces the old one.

Flags (in alwaysredraw)

The *typePtr->alwaysRedraw* field (so named for historic reasons) contains a collection of flag bits that modify how the canvas core interacts with the item. The following bits are defined:

1
: Indicates that the item should always be redrawn when any part of the canvas is redrawn, rather than only when the bounding box of the item overlaps the area being redrawn. This is used by window items, for example, which need to unmap subwindows that are not on the screen.

TK_CONFIG_OBJS
: Indicates that operations which would otherwise take a string (or array of strings) actually

take a Tcl_Obj reference (or an array of such references). The operations to which this applies are the *configProc*, the *coordProc*, the *createProc*, the *indexProc* and the *insertProc*.

TK_MOVABLE_POINTS
Indicates that the item supports the *dCharsProc*, *indexProc* and *insertProc* with the same semantics as Tk's built-in line and polygon types, and that hence individual coordinate points can be moved. Must not be set if any of the above methods is NULL.

Itemsize

typePtr->itemSize gives the size in bytes of item records of this type, including the Tk_Item header. Tk uses this size to allocate memory space for items of the type. All of the item records for a given type must have the same size. If variable length fields are needed for an item (such as a list of points for a polygon), the type manager can allocate a separate object of variable length and keep a pointer to it in the item record.

Createproc

typePtr->createProc points to a procedure for Tk to call whenever a new item of this type is created. *typePtr->createProc* must match the following prototype:

```
typedef int Tk_ItemCreateProc(
        Tcl_Interp *interp,
        Tk_Canvas canvas,
        Tk_Item *itemPtr,
        int objc,
        Tcl_Obj *const objv[]);
```

The *interp* argument is the interpreter in which the canvas's **create** widget command was invoked, and *canvas* is a handle for the canvas widget. *itemPtr* is a pointer to a newly-allocated item of size *typePtr->itemSize*. Tk has already initialized the item's header (the first **sizeof(Tk_ItemType)** bytes). The *objc* and *objv* arguments describe all of the arguments to the **create** command after the *type* argument. Note that if **TK_CONFIG_OBJS** is not set in the *typePtr->alwaysRedraw* field, the *objv* parameter will actually contain a pointer to an array of constant strings. For example, in the widget command:

```
.c create rectangle 10 20 50 50 -fill black
```

objc will be **6** and *objv*[0] will contain the integer object **10**.

createProc should use *objc* and *objv* to initialize the type-specific parts of the item record and set an initial value for the bounding box in the item's header. It should return a standard Tcl completion code and leave an error message in the interpreter result if an error occurs. If an error occurs Tk will free the item record, so *createProc* must be sure to leave the item record in a clean state if it returns an error (e.g., it must free any additional memory that it allocated for the item).

Configspecs

Each type manager must provide a standard table describing its configuration options, in a form suitable for use with **Tk_ConfigureWidget**. This table will normally be used by *typePtr->createProc* and *typePtr->configProc*, but Tk also uses it directly to retrieve option information in the **itemcget** and **itemconfigure** widget commands. *typePtr->configSpecs* must

point to the configuration table for this type. Note: Tk provides a custom option type **tk_CanvasTagsOption** for implementing the **-tags** option; see an existing type manager for an example of how to use it in *configSpecs*.

Configproc

typePtr->configProc is called by Tk whenever the **itemconfigure** widget command is invoked to change the configuration options for a canvas item. This procedure must match the following prototype:

```
typedef int Tk_ItemConfigureProc(
        Tcl_Interp *interp,
        Tk_Canvas canvas,
        Tk_Item *itemPtr,
        int objc,
        Tcl_Obj *const objv[],
        int flags);
```

The *interp* argument identifies the interpreter in which the widget command was invoked, *canvas* is a handle for the canvas widget, and *itemPtr* is a pointer to the item being configured. *objc* and *objv* contain the configuration options. Note that if **TK_CONFIG_OBJS** is not set in the *typePtr->alwaysRedraw* field, the *objv* parameter will actually contain a pointer to an array of constant strings. For example, if the following command is invoked:

```
.c itemconfigure 2 -fill red -outline black
```

objc is **4** and *objv* contains the string objects **-fill** through **black**. *objc* will always be an even value. The *flags* argument contains flags to pass to **Tk_ConfigureWidget**; currently this value is always **TK_CONFIG_ARGV_ONLY** when Tk invokes *typePtr->configProc*, but the type manager's *createProc* procedure will usually invoke *configProc* with different flag values.

typePtr->configProc returns a standard Tcl completion code and leaves an error message in the interpreter result if an error occurs. It must update the item's bounding box to reflect the new configuration options.

Coordproc

typePtr->coordProc is invoked by Tk to implement the **coords** widget command for an item. It must match the following prototype:

```
typedef int Tk_ItemCoordProc(
        Tcl_Interp *interp,
        Tk_Canvas canvas,
        Tk_Item *itemPtr,
        int objc,
        Tcl_Obj *const objv[]);
```

The arguments *interp*, *canvas*, and *itemPtr* all have the standard meanings, and *objc* and *objv* describe the coordinate arguments. Note that if **TK_CONFIG_OBJS** is not set in the *typePtr->alwaysRedraw* field, the *objv* parameter will actually contain a pointer to an array of constant strings. For example, if the following widget command is invoked:

```
.c coords 2 30 90
```

objc will be **2** and **objv** will contain the integer objects **30** and **90**.

The *coordProc* procedure should process the new coordinates, update the item appropriately (e.g., it must reset the bounding box in the item's header), and return a standard Tcl completion code. If an error occurs, *coordProc* must leave an error message in the interpreter result.

Deleteproc

typePtr->deleteProc is invoked by Tk to delete an item and free any resources allocated to it. It must match the following prototype:

```
typedef void Tk_ItemDeleteProc(
        Tk_Canvas canvas,
        Tk_Item *itemPtr,
        Display *display);
```

The *canvas* and *itemPtr* arguments have the usual interpretations, and *display* identifies the X display containing the canvas. *deleteProc* must free up any resources allocated for the item, so that Tk can free the item record. *deleteProc* should not actually free the item record; this will be done by Tk when *deleteProc* returns.

Displayproc

typePtr->displayProc is invoked by Tk to redraw an item on the screen. It must match the following prototype:

```
typedef void Tk_ItemDisplayProc(
        Tk_Canvas canvas,
        Tk_Item *itemPtr,
        Display *display,
        Drawable dst,
        int x,
        int y,
        int width,
        int height);
```

The *canvas* and *itemPtr* arguments have the usual meaning. *display* identifies the display containing the canvas, and *dst* specifies a drawable in which the item should be rendered; typically this is an off-screen pixmap, which Tk will copy into the canvas's window once all relevant items have been drawn. *x*, *y*, *width*, and *height* specify a rectangular region in canvas coordinates, which is the area to be redrawn; only information that overlaps this area needs to be redrawn. Tk will not call *displayProc* unless the item's bounding box overlaps the redraw area, but the type manager may wish to use the redraw area to optimize the redisplay of the item.

Because of scrolling and the use of off-screen pixmaps for double-buffered redisplay, the item's coordinates in *dst* will not necessarily be the same as those in the canvas. *displayProc* should call **Tk_CanvasDrawableCoords** to transform coordinates from those of the canvas to those of *dst*.

Normally an item's *displayProc* is only invoked if the item overlaps the area being displayed. However, if bit zero of *typePtr->alwaysRedraw* is 1, (i.e.\| "*typePtr->alwaysRedraw & 1 == 1*")

then *displayProc* is invoked during every redisplay operation, even if the item does not overlap the area of redisplay; this is useful for cases such as window items, where the subwindow needs to be unmapped when it is off the screen.

Pointproc

typePtr->pointProc is invoked by Tk to find out how close a given point is to a canvas item. Tk uses this procedure for purposes such as locating the item under the mouse or finding the closest item to a given point. The procedure must match the following prototype:

```
typedef double Tk_ItemPointProc(
        Tk_Canvas canvas,
        Tk_Item *itemPtr,
        double *pointPtr);
```

canvas and *itemPtr* have the usual meaning. *pointPtr* points to an array of two numbers giving the x and y coordinates of a point. *pointProc* must return a real value giving the distance from the point to the item, or 0 if the point lies inside the item.

Areaproc

typePtr->areaProc is invoked by Tk to find out the relationship between an item and a rectangular area. It must match the following prototype:

```
typedef int Tk_ItemAreaProc(
        Tk_Canvas canvas,
        Tk_Item *itemPtr,
        double *rectPtr);
```

canvas and *itemPtr* have the usual meaning. *rectPtr* points to an array of four real numbers; the first two give the x and y coordinates of the upper left corner of a rectangle, and the second two give the x and y coordinates of the lower right corner. *areaProc* must return -1 if the item lies entirely outside the given area, 0 if it lies partially inside and partially outside the area, and 1 if it lies entirely inside the area.

Postscriptproc

typePtr->postscriptProc is invoked by Tk to generate Postscript for an item during the **postscript** widget command. If the type manager is not capable of generating Postscript then *typePtr->postscriptProc* should be NULL. The procedure must match the following prototype:

```
typedef int Tk_ItemPostscriptProc(
        Tcl_Interp *interp,
        Tk_Canvas canvas,
        Tk_Item *itemPtr,
        int prepass);
```

The *interp*, *canvas*, and *itemPtr* arguments all have standard meanings; *prepass* will be described below. If *postscriptProc* completes successfully, it should append Postscript for the item to the information in the interpreter result (e.g. by calling **Tcl_AppendResult**, not **Tcl_SetResult**) and return **TCL_OK**. If an error occurs, *postscriptProc* should clear the result and replace its contents with an error message; then it should return **TCL_ERROR**.

Tk provides a collection of utility procedures to simplify *postscriptProc*. For example,

Tk_CanvasPsColor will generate Postscript to set the current color to a given Tk color and **Tk_CanvasPsFont** will set up font information. When generating Postscript, the type manager is free to change the graphics state of the Postscript interpreter, since Tk places **gsave** and **grestore** commands around the Postscript for the item. The type manager can use canvas x coordinates directly in its Postscript, but it must call **Tk_CanvasPsY** to convert y coordinates from the space of the canvas (where the origin is at the upper left) to the space of Postscript (where the origin is at the lower left).

In order to generate Postscript that complies with the Adobe Document Structuring Conventions, Tk actually generates Postscript in two passes. It calls each item's *postscriptProc* in each pass. The only purpose of the first pass is to collect font information (which is done by **Tk_CanvasPsFont**); the actual Postscript is discarded. Tk sets the *prepass* argument to *postscriptProc* to 1 during the first pass; the type manager can use *prepass* to skip all Postscript generation except for calls to **Tk_CanvasPsFont**. During the second pass *prepass* will be 0, so the type manager must generate complete Postscript.

Scaleproc

typePtr->scaleProc is invoked by Tk to rescale a canvas item during the **scale** widget command. The procedure must match the following prototype:

```
typedef void Tk_ItemScaleProc(
        Tk_Canvas canvas,
        Tk_Item *itemPtr,
        double originX,
        double originY,
        double scaleX,
        double scaleY);
```

The *canvas* and *itemPtr* arguments have the usual meaning. *originX* and *originY* specify an origin relative to which the item is to be scaled, and *scaleX* and *scaleY* give the x and y scale factors. The item should adjust its coordinates so that a point in the item that used to have coordinates *x* and *y* will have new coordinates *x[fm]* and *y[fm]*, where

```
x[fm] = originX + scaleX µ (x - originX)
y[fm] = originY + scaleY µ (y - originY)
```

scaleProc must also update the bounding box in the item's header.

Translateproc

typePtr->translateProc is invoked by Tk to translate a canvas item during the **move** widget command. The procedure must match the following prototype:

```
typedef void Tk_ItemTranslateProc(
        Tk_Canvas canvas,
        Tk_Item *itemPtr,
        double deltaX,
        double deltaY);
```

The *canvas* and *itemPtr* arguments have the usual meaning, and *deltaX* and *deltaY* give the amounts that should be added to each x and y coordinate within the item. The type manager

should adjust the item's coordinates and update the bounding box in the item's header.

Indexproc

typePtr->indexProc is invoked by Tk to translate a string index specification into a numerical index, for example during the **index** widget command. It is only relevant for item types that support indexable text or coordinates; *typePtr->indexProc* may be specified as NULL for non-textual item types if they do not support detailed coordinate addressing. The procedure must match the following prototype:

```
typedef int Tk_ItemIndexProc(
        Tcl_Interp *interp,
        Tk_Canvas canvas,
        Tk_Item *itemPtr,
        Tcl_Obj *indexObj,
        int *indexPtr);
```

The *interp*, *canvas*, and *itemPtr* arguments all have the usual meaning. *indexObj* contains a textual description of an index, and *indexPtr* points to an integer value that should be filled in with a numerical index. Note that if **TK_CONFIG_OBJS** is not set in the *typePtr->alwaysRedraw* field, the *indexObj* parameter will actually contain a pointer to a constant string. It is up to the type manager to decide what forms of index are supported (e.g., numbers, **insert**, **sel.first**, **end**, etc.). *indexProc* should return a Tcl completion code and set the interpreter result in the event of an error.

Icursorproc

typePtr->icursorProc is invoked by Tk during the **icursor** widget command to set the position of the insertion cursor in a textual item. It is only relevant for item types that support an insertion cursor; *typePtr->icursorProc* may be specified as NULL for item types that do not support an insertion cursor. The procedure must match the following prototype:

```
typedef void Tk_ItemCursorProc(
        Tk_Canvas canvas,
        Tk_Item *itemPtr,
        int index);
```

canvas and *itemPtr* have the usual meanings, and *index* is an index into the item's text, as returned by a previous call to *typePtr->insertProc*. The type manager should position the insertion cursor in the item just before the character given by *index*. Whether or not to actually display the insertion cursor is determined by other information provided by **Tk_CanvasGetTextInfo**.

Selectionproc

typePtr->selectionProc is invoked by Tk during selection retrievals; it must return part or all of the selected text in the item (if any). It is only relevant for item types that support text; *typePtr->selectionProc* may be specified as NULL for non-textual item types. The procedure must match the following prototype:

```
typedef int Tk_ItemSelectionProc(
        Tk_Canvas canvas,
        Tk_Item *itemPtr,
        int offset,
        char *buffer,
        int maxBytes);
```

canvas and *itemPtr* have the usual meanings. *offset* is an offset in bytes into the selection where 0 refers to the first byte of the selection; it identifies the first character that is to be returned in this call. *buffer* points to an area of memory in which to store the requested bytes, and *maxBytes* specifies the maximum number of bytes to return. *selectionProc* should extract up to *maxBytes* characters from the selection and copy them to *maxBytes*; it should return a count of the number of bytes actually copied, which may be less than *maxBytes* if there are not *offset+maxBytes* bytes in the selection.

Insertproc

typePtr->insertProc is invoked by Tk during the **insert** widget command to insert new text or coordinates into a canvas item. It is only relevant for item types that support the **insert** method; *typePtr->insertProc* may be specified as NULL for other item types. The procedure must match the following prototype:

```
typedef void Tk_ItemInsertProc(
        Tk_Canvas canvas,
        Tk_Item *itemPtr,
        int index,
        Tcl_Obj *obj);
```

canvas and *itemPtr* have the usual meanings. *index* is an index into the item's text, as returned by a previous call to *typePtr->insertProc*, and *obj* contains new text to insert just before the character given by *index*. Note that if **TK_CONFIG_OBJS** is not set in the *typePtr->alwaysRedraw* field, the *obj* parameter will actually contain a pointer to a constant string to be inserted. If the item supports modification of the coordinates list by this

The type manager should insert the text and recompute the bounding box in the item's header.

Dcharsproc

typePtr->dCharsProc is invoked by Tk during the **dchars** widget command to delete a range of text from a canvas item or a range of coordinates from a pathed item. It is only relevant for item types that support text; *typePtr->dCharsProc* may be specified as NULL for non-textual item types that do not want to support coordinate deletion. The procedure must match the following prototype:

```
typedef void Tk_ItemDCharsProc(
        Tk_Canvas canvas,
        Tk_Item *itemPtr,
        int first,
        int last);
```

canvas and *itemPtr* have the usual meanings. *first* and *last* give the indices of the first and last bytes to be deleted, as returned by previous calls to *typePtr->indexProc*. The type manager

should delete the specified characters and update the bounding box in the item's header.

Keywords: CrtItemType, canvas, focus, item type, selection, type manager

Copyright (c) 1994-1995 Sun Microsystems, Inc.

CrtPhImgFmt

Name

Tk_CreatePhotoImageFormat - define new file format for photo images

Synopsis

#include <tk.h>

Tk_CreatePhotoImageFormat(*formatPtr*)

Arguments

const Tk_PhotoImageFormat *_formatPtr_ (in)

Structure that defines the new file format.

Description

Tk_CreatePhotoImageFormat is invoked to define a new file format for image data for use with photo images. The code that implements an image file format is called an image file format handler, or handler for short. The photo image code maintains a list of handlers that can be used to read and write data to or from a file. Some handlers may also support reading image data from a string or converting image data to a string format. The user can specify which handler to use with the **-format** image configuration option or the **-format** option to the **read** and **write** photo image subcommands.

An image file format handler consists of a collection of procedures plus a Tk_PhotoImageFormat structure, which contains the name of the image file format and pointers to six procedures provided by the handler to deal with files and strings in this format. The Tk_PhotoImageFormat structure contains the following fields:

```
typedef struct Tk_PhotoImageFormat {
    const char *name;
    Tk_ImageFileMatchProc *fileMatchProc;
    Tk_ImageStringMatchProc *stringMatchProc;
    Tk_ImageFileReadProc *fileReadProc;
    Tk_ImageStringReadProc *stringReadProc;
    Tk_ImageFileWriteProc *fileWriteProc;
    Tk_ImageStringWriteProc *stringWriteProc;
} Tk_PhotoImageFormat;
```

The handler need not provide implementations of all six procedures. For example, the procedures that handle string data would not be provided for a format in which the image data are stored in binary, and could therefore contain null characters. If any procedure is not implemented, the corresponding pointer in the Tk_PhotoImageFormat structure should be set to NULL. The handler must provide the *fileMatchProc* procedure if it provides the *fileReadProc* procedure, and the *stringMatchProc* procedure if it provides the *stringReadProc* procedure.

Name

formatPtr->name provides a name for the image type. Once **Tk_CreatePhotoImageFormat** returns, this name may be used in the **-format** photo image configuration and subcommand option. The manual page for the photo image (photo(n)) describes how image file formats are chosen based on their names and the value given to the **-format** option. The first character of *formatPtr->name* must not be an uppercase character from the ASCII character set (that is, one of the characters **A**-**Z**). Such names are used only for legacy interface support (see below).

Filematchproc

formatPtr->fileMatchProc provides the address of a procedure for Tk to call when it is searching for an image file format handler suitable for reading data in a given file. *formatPtr->fileMatchProc* must match the following prototype:

```
typedef int Tk_ImageFileMatchProc(
        Tcl_Channel chan,
        const char *fileName,
        Tcl_Obj *format,
        int *widthPtr,
        int *heightPtr,
        Tcl_Interp *interp);
```

The *fileName* argument is the name of the file containing the image data, which is open for reading as *chan*. The *format* argument contains the value given for the **-format** option, or NULL if the option was not specified. If the data in the file appears to be in the format supported by this handler, the *formatPtr->fileMatchProc* procedure should store the width and height of the image in **widthPtr* and **heightPtr* respectively, and return 1. Otherwise it should return 0.

Stringmatchproc

formatPtr->stringMatchProc provides the address of a procedure for Tk to call when it is searching for an image file format handler for suitable for reading data from a given string. *formatPtr->stringMatchProc* must match the following prototype:

```
typedef int Tk_ImageStringMatchProc(
        Tcl_Obj *data,
        Tcl_Obj *format,
        int *widthPtr,
        int *heightPtr,
        Tcl_Interp *interp);
```

The *data* argument points to the object containing the image data. The *format* argument contains the value given for the **-format** option, or NULL if the option was not specified. If the data in the string appears to be in the format supported by this handler, the *formatPtr->stringMatchProc* procedure should store the width and height of the image in * *widthPtr* and **heightPtr* respectively, and return 1. Otherwise it should return 0.

Filereadproc

formatPtr->fileReadProc provides the address of a procedure for Tk to call to read data from an image file into a photo image. *formatPtr->fileReadProc* must match the following prototype:

```
typedef int Tk_ImageFileReadProc(
        Tcl_Interp *interp,
        Tcl_Channel chan,
        const char *fileName,
        Tcl_Obj *format,
        PhotoHandle imageHandle,
        int destX, int destY,
        int width, int height,
        int srcX, int srcY);
```

The *interp* argument is the interpreter in which the command was invoked to read the image; it should be used for reporting errors. The image data is in the file named *fileName*, which is open for reading as *chan*. The *format* argument contains the value given for the **-format** option, or NULL if the option was not specified. The image data in the file, or a subimage of it, is to be read into the photo image identified by the handle *imageHandle*. The subimage of the data in the file is of dimensions *width* x *height* and has its top-left corner at coordinates (*srcX*,*srcY*). It is to be stored in the photo image with its top-left corner at coordinates (*destX*,*destY*) using the **Tk_PhotoPutBlock** procedure. The return value is a standard Tcl return value.

Stringreadproc

formatPtr->stringReadProc provides the address of a procedure for Tk to call to read data from a string into a photo image. *formatPtr->stringReadProc* must match the following prototype:

```
typedef int Tk_ImageStringReadProc(
        Tcl_Interp *interp,
        Tcl_Obj *data,
        Tcl_Obj *format,
        PhotoHandle imageHandle,
        int destX, int destY,
        int width, int height,
        int srcX, int srcY);
```

The *interp* argument is the interpreter in which the command was invoked to read the image; it should be used for reporting errors. The *data* argument points to the image data in object form. The *format* argument contains the value given for the **-format** option, or NULL if the option was not specified. The image data in the string, or a subimage of it, is to be read into the photo image identified by the handle *imageHandle*. The subimage of the data in the string is of dimensions *width* x *height* and has its top-left corner at coordinates (*srcX*,*srcY*). It is to be stored in the photo image with its top-left corner at coordinates (*destX*,*destY*) using the **Tk_PhotoPutBlock** procedure. The return value is a standard Tcl return value.

Filewriteproc

formatPtr->fileWriteProc provides the address of a procedure for Tk to call to write data from a photo image to a file. *formatPtr->fileWriteProc* must match the following prototype:

```
typedef int Tk_ImageFileWriteProc(
        Tcl_Interp *interp,
        const char *fileName,
        Tcl_Obj *format,
        Tk_PhotoImageBlock *blockPtr);
```

The *interp* argument is the interpreter in which the command was invoked to write the image; it should be used for reporting errors. The image data to be written are in memory and are described by the Tk_PhotoImageBlock structure pointed to by *blockPtr*; see the manual page FindPhoto(3) for details. The *fileName* argument points to the string giving the name of the file in which to write the image data. The *format* argument contains the value given for the **-format** option, or NULL if the option was not specified. The format string can contain extra characters after the name of the format. If appropriate, the *formatPtr->fileWriteProc* procedure may interpret these characters to specify further details about the image file. The return value is a standard Tcl return value.

Stringwriteproc

formatPtr->stringWriteProc provides the address of a procedure for Tk to call to translate image data from a photo image into a string. *formatPtr->stringWriteProc* must match the following prototype:

```
typedef int Tk_ImageStringWriteProc(
        Tcl_Interp *interp,
        Tcl_Obj *format,
        Tk_PhotoImageBlock *blockPtr);
```

The *interp* argument is the interpreter in which the command was invoked to convert the image; it should be used for reporting errors. The image data to be converted are in memory and are described by the Tk_PhotoImageBlock structure pointed to by *blockPtr*; see the manual page FindPhoto(3) for details. The data for the string should be put in the interpreter *interp* result. The *format* argument contains the value given for the **-format** option, or NULL if the option was not specified. The format string can contain extra characters after the name of the format. If appropriate, the *formatPtr->stringWriteProc* procedure may interpret these characters to specify further details about the image file. The return value is a standard Tcl return value.

Legacy interface support

In Tk 8.2 and earlier, the definition of all the function pointer types stored in fields of a **Tk_PhotoImageFormat** struct were incompatibly different. Legacy programs and libraries dating from those days may still contain code that defines extended Tk photo image formats using the old interface. The Tk header file will still support this legacy interface if the code is compiled with the macro **USE_OLD_IMAGE** defined. Alternatively, the legacy interfaces are used if the first character of *formatPtr->name* is an uppercase ASCII character (**A**-**Z**), and explicit casts are used to forgive the type mismatch. For example,

```
static Tk_PhotoImageFormat myFormat = {
    "MyFormat",
    (Tk_ImageFileMatchProc *) FileMatch,
    NULL,
```

```
    (Tk_ImageFileReadProc *) FileRead,
    NULL,
    NULL,
    NULL
};
```

would define a minimal **Tk_PhotoImageFormat** that operates provide only file reading capability, where **FileMatch** and **FileRead** are written according to the legacy interfaces of Tk 8.2 or earlier.

Any stub-enabled extension providing an extended photo image format via the legacy interface enabled by the **USE_OLD_IMAGE** macro that is compiled against Tk 8.5 headers and linked against the Tk 8.5 stub library will produce a file that can be loaded only into interps with Tk 8.5 or later; that is, the normal stub-compatibility rules. If a developer needs to generate from such code a file that is loadable into interps with Tk 8.4 or earlier, they must use Tk 8.4 headers and stub libraries to do so.

Any new code written today should not make use of the legacy interfaces. Expect their support to go away in Tk 9.

Keywords: CrtPhImgFmt, photo image, image file

Copyright (c) 1994 The Australian National University
Copyright (c) 1994-1997 Sun Microsystems, Inc.

CrtSelHdlr

Name

Tk_CreateSelHandler, Tk_DeleteSelHandler - arrange to handle requests for a selection

Synopsis

#include <tk.h>

Tk_CreateSelHandler(*tkwin*, *selection*, *target*, *proc*, *clientData*, *format*)

Tk_DeleteSelHandler(*tkwin*, *selection*, *target*)

Arguments

Tk_Window *tkwin* (in)
Window for which *proc* will provide selection information.

Atom *selection* (in)
The name of the selection for which *proc* will provide selection information.

Atom *target* (in)
Form in which *proc* can provide the selection (e.g. STRING or FILE_NAME). Corresponds to *type* arguments in **selection** commands.

Tk_SelectionProc **proc* (in)
Procedure to invoke whenever the selection is owned by *tkwin* and the selection contents are requested in the format given by *target*.

ClientData *clientData* (in)
Arbitrary one-word value to pass to *proc*.

Atom *format* (in)

If the selection requestor is not in this process, *format* determines the representation used to transmit the selection to its requestor.

Description

Tk_CreateSelHandler arranges for a particular procedure (*proc*) to be called whenever *selection* is owned by *tkwin* and the selection contents are requested in the form given by *target* . *Target* should be one of the entries defined in the left column of Table 2 of the X Inter-Client Communication Conventions Manual (ICCCM) or any other form in which an application is willing to present the selection. The most common form is STRING.

Proc should have arguments and result that match the type **Tk_SelectionProc**:

```
typedef int Tk_SelectionProc(
        ClientData clientData,
        int offset,
        char *buffer,
        int maxBytes);
```

The *clientData* parameter to *proc* is a copy of the *clientData* argument given to **Tk_CreateSelHandler**. Typically, *clientData* points to a data structure containing application-specific information that is needed to retrieve the selection. *Offset* specifies an offset position into the selection, *buffer* specifies a location at which to copy information about the selection, and *maxBytes* specifies the amount of space available at *buffer*. *Proc* should place a NULL-terminated string at *buffer* containing *maxBytes* or fewer characters (not including the terminating NULL), and it should return a count of the number of non-NULL characters stored at *buffer*. If the selection no longer exists (e.g. it once existed but the user deleted the range of characters containing it), then *proc* should return -1.

When transferring large selections, Tk will break them up into smaller pieces (typically a few thousand bytes each) for more efficient transmission. It will do this by calling *proc* one or more times, using successively higher values of *offset* to retrieve successive portions of the selection. If *proc* returns a count less than *maxBytes* it means that the entire remainder of the selection has been returned. If *proc*'s return value is *maxBytes* it means there may be additional information in the selection, so Tk must make another call to *proc* to retrieve the next portion.

Proc always returns selection information in the form of a character string. However, the ICCCM allows for information to be transmitted from the selection owner to the selection requestor in any of several formats, such as a string, an array of atoms, an array of integers, etc. The *format* argument to **Tk_CreateSelHandler** indicates what format should be used to transmit the selection to its requestor (see the middle column of Table 2 of the ICCCM for examples). If *format* is not STRING, then Tk will take the value returned by *proc* and divided it into fields separated by white space. If *format* is ATOM, then Tk will return the selection as an array of atoms, with each field in *proc*'s result treated as the name of one atom. For any other value of *format*, Tk will return the selection as an array of 32-bit values where each field of *proc*'s result is treated as a number and translated to a 32-bit value. In any event, the *format* atom is returned to the selection requestor along with the contents of the selection.

If **Tk_CreateSelHandler** is called when there already exists a handler for *selection* and *target* on *tkwin*, then the existing handler is replaced with a new one.

Tk_DeleteSelHandler removes the handler given by *tkwin*, *selection*, and *target*, if such a

handler exists. If there is no such handler then it has no effect.

Keywords: CrtSelHdlr, format, handler, selection, target

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

CrtWindow

Name

Tk_CreateWindow, Tk_CreateWindowFromPath, Tk_DestroyWindow, Tk_MakeWindowExist - create or delete window

Synopsis

#include <tk.h>

Tk_Window
Tk_CreateWindow(*interp*, *parent*, *name*, *topLevScreen*)

Tk_Window
Tk_CreateAnonymousWindow(*interp*, *parent*, *topLevScreen*)

Tk_Window
Tk_CreateWindowFromPath(*interp*, *tkwin*, *pathName*, *topLevScreen*)

Tk_DestroyWindow(*tkwin*)

Tk_MakeWindowExist(*tkwin*)

Arguments

Tcl_Interp *_interp_ (out)
Tcl interpreter to use for error reporting. If no error occurs, then **interp* is not modified.

Tk_Window _parent_ (in)
Token for the window that is to serve as the logical parent of the new window.

const char *_name_ (in)
Name to use for this window. Must be unique among all children of the same *parent*.

const char *_topLevScreen_ (in)
Has same format as *screenName*. If NULL, then new window is created as an internal window. If non-NULL, new window is created as a top-level window on screen *topLevScreen*. If *topLevScreen* is an empty string ("") then new window is created as top-level window of *parent*'s screen.

Tk_Window _tkwin_ (in)
Token for window.

const char *_pathName_ (in)
Name of new window, specified as path name within application (e.g. **.a.b.c**).

Description

The procedures **Tk_CreateWindow**, **Tk_CreateAnonymousWindow**, and **Tk_CreateWindowFromPath** are used to create new windows for use in Tk-based applications. Each of the procedures returns a token that can be used to manipulate the window in other calls to the Tk library. If the window could not be created successfully, then NULL is returned and the

result of interpreter *interp* is modified to hold an error message.

Tk supports two different kinds of windows: internal windows and top-level windows. An internal window is an interior window of a Tk application, such as a scrollbar or menu bar or button. A top-level window is one that is created as a child of a screen's root window, rather than as an interior window, but which is logically part of some existing main window. Examples of top-level windows are pop-up menus and dialog boxes.

New windows may be created by calling **Tk_CreateWindow**. If the *topLevScreen* argument is NULL, then the new window will be an internal window. If *topLevScreen* is non-NULL, then the new window will be a top-level window: *topLevScreen* indicates the name of a screen and the new window will be created as a child of the root window of *topLevScreen*. In either case Tk will consider the new window to be the logical child of *parent*: the new window's path name will reflect this fact, options may be specified for the new window under this assumption, and so on. The only difference is that new X window for a top-level window will not be a child of *parent*'s X window. For example, a pull-down menu's *parent* would be the button-like window used to invoke it, which would in turn be a child of the menu bar window. A dialog box might have the application's main window as its parent.

Tk_CreateAnonymousWindow differs from **Tk_CreateWindow** in that it creates an unnamed window. This window will be manipulatable only using C interfaces, and will not be visible to Tcl scripts. Both interior windows and top-level windows may be created with **Tk_CreateAnonymousWindow**.

Tk_CreateWindowFromPath offers an alternate way of specifying new windows. In **Tk_CreateWindowFromPath** the new window is specified with a token for any window in the target application (*tkwin*), plus a path name for the new window. It produces the same effect as **Tk_CreateWindow** and allows both top-level and internal windows to be created, depending on the value of *topLevScreen*. In calls to **Tk_CreateWindowFromPath**, as in calls to **Tk_CreateWindow**, the parent of the new window must exist at the time of the call, but the new window must not already exist.

The window creation procedures do not actually issue the command to X to create a window. Instead, they create a local data structure associated with the window and defer the creation of the X window. The window will actually be created by the first call to **Tk_MapWindow**. Deferred window creation allows various aspects of the window (such as its size, background color, etc.) to be modified after its creation without incurring any overhead in the X server. When the window is finally mapped all of the window attributes can be set while creating the window.

The value returned by a window-creation procedure is not the X token for the window (it cannot be, since X has not been asked to create the window yet). Instead, it is a token for Tk's local data structure for the window. Most of the Tk library procedures take Tk_Window tokens, rather than X identifiers. The actual X window identifier can be retrieved from the local data structure using the **Tk_WindowId** macro; see the manual entry for **Tk_WindowId** for details.

Tk_DestroyWindow deletes a window and all the data structures associated with it, including any event handlers created with **Tk_CreateEventHandler**. In addition, **Tk_DestroyWindow** will delete any children of *tkwin* recursively (where children are defined in the Tk sense, consisting of all windows that were created with the given window as *parent*). If *tkwin* is an internal window, then event handlers interested in destroy events are invoked immediately. If *tkwin* is a top-level or main window, then the event handlers will be invoked later, after X has

seen the request and returned an event for it.

If a window has been created but has not been mapped, so no X window exists, it is possible to force the creation of the X window by calling **Tk_MakeWindowExist**. This procedure issues the X commands to instantiate the window given by *tkwin*.

Keywords: CrtWindow, create, deferred creation, destroy, display, internal window, screen, top-level window, window

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

DeleteImg

Name

Tk_DeleteImage - Destroy an image.

Synopsis

#include <tk.h>

Tk_DeleteImage(*interp*, *name*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter for which the image was created.

const char *_name_ (in)
Name of the image.

Description

Tk_DeleteImage deletes the image given by *interp* and *name*, if there is one. All instances of that image will redisplay as empty regions. If the given image does not exist then the procedure has no effect.

Keywords: DeleteImg, delete image, image manager

Copyright (c) 1995-1996 Sun Microsystems, Inc.

DrawFocHlt

Name

Tk_DrawFocusHighlight - draw the traversal highlight ring for a widget

Synopsis

#include <tk.h>

Tk_DrawFocusHighlight(*tkwin*, *gc*, *width*, *drawable*)

Arguments

Tk_Window _tkwin_ (in)
Window for which the highlight is being drawn. Used to retrieve the window's dimensions, among other things.

GC _gc_ (in)

Graphics context to use for drawing the highlight.

int *width* (in)
Width of the highlight ring, in pixels.

Drawable *drawable* (in)
Drawable in which to draw the highlight; usually an offscreen pixmap for double buffering.

Description

Tk_DrawFocusHighlight is a utility procedure that draws the traversal highlight ring for a widget. It is typically invoked by widgets during redisplay.

Keywords: DrawFocHlt, focus, traversal highlight

Copyright (c) 1995-1996 Sun Microsystems, Inc.

EventHndlr

Name

Tk_CreateEventHandler, Tk_DeleteEventHandler - associate procedure callback with an X event

Synopsis

#include <tk.h>

Tk_CreateEventHandler(*tkwin*, *mask*, *proc*, *clientData*)

Tk_DeleteEventHandler(*tkwin*, *mask*, *proc*, *clientData*)

Arguments

Tk_Window *tkwin* (in)
Token for window in which events may occur.

unsigned long *mask* (in)
Bit-mask of events (such as **ButtonPressMask**) for which *proc* should be called.

Tk_EventProc **proc* (in)
Procedure to invoke whenever an event in *mask* occurs in the window given by *tkwin*.

ClientData *clientData* (in)
Arbitrary one-word value to pass to *proc*.

Description

Tk_CreateEventHandler arranges for *proc* to be invoked in the future whenever one of the event types specified by *mask* occurs in the window specified by *tkwin*. The callback to *proc* will be made by **Tk_HandleEvent**; this mechanism only works in programs that dispatch events through **Tk_HandleEvent** (or through other Tk procedures that call **Tk_HandleEvent**, such as **Tcl_DoOneEvent** or **Tk_MainLoop**).

Proc should have arguments and result that match the type **Tk_EventProc**:

```
typedef void Tk_EventProc(
        ClientData clientData,
        XEvent *eventPtr);
```

The *clientData* parameter to *proc* is a copy of the *clientData* argument given to **Tk_CreateEventHandler** when the callback was created. Typically, *clientData* points to a data

structure containing application-specific information about the window in which the event occurred. *EventPtr* is a pointer to the X event, which will be one of the ones specified in the *mask* argument to **Tk_CreateEventHandler**.

Tk_DeleteEventHandler may be called to delete a previously-created event handler: it deletes the first handler it finds that is associated with *tkwin* and matches the *mask*, *proc*, and *clientData* arguments. If no such handler exists, then **Tk_HandleEvent** returns without doing anything. Although Tk supports it, it's probably a bad idea to have more than one callback with the same *mask*, *proc*, and *clientData* arguments. When a window is deleted all of its handlers will be deleted automatically; in this case there is no need to call **Tk_DeleteEventHandler**.

If multiple handlers are declared for the same type of X event on the same window, then the handlers will be invoked in the order they were created.

Keywords: EventHndlr, bind, callback, event, handler

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

FindPhoto

Name

Tk_FindPhoto, Tk_PhotoPutBlock, Tk_PhotoPutZoomedBlock, Tk_PhotoGetImage, Tk_PhotoBlank, Tk_PhotoExpand, Tk_PhotoGetSize, Tk_PhotoSetSize - manipulate the image data stored in a photo image.

Synopsis

#include <tk.h>

Tk_PhotoHandle
Tk_FindPhoto(*interp*, *imageName*)

int
Tk_PhotoPutBlock(*interp*, *handle*, *blockPtr*, *x*, *y*, *width*, *height*, *compRule*)

int
Tk_PhotoPutZoomedBlock(*interp*, *handle*, *blockPtr*, *x*, *y*, *width*, *height*, *zoomX*, *zoomY*, *subsampleX*, *subsampleY*, *compRule*)

int
Tk_PhotoGetImage(*handle*, *blockPtr*)

void
Tk_PhotoBlank(*handle*)

int
Tk_PhotoExpand(*interp*, *handle*, *width*, *height*)

void
Tk_PhotoGetSize(*handle*, *widthPtr*, *heightPtr*)

int
Tk_PhotoSetSize(*interp*. *handle*, *width*, *height*)

Arguments

Tcl_Interp *_interp_ (in)

Interpreter in which image was created and in which error reporting is to be done.

const char *_imageName_ (in)
Name of the photo image.

Tk_PhotoHandle _handle_ (in)
Opaque handle identifying the photo image to be affected.

Tk_PhotoImageBlock *_blockPtr_ (in)
Specifies the address and storage layout of image data.

int _x_ (in)
Specifies the X coordinate where the top-left corner of the block is to be placed within the image.

int _y_ (in)
Specifies the Y coordinate where the top-left corner of the block is to be placed within the image.

int _width_ (in)
Specifies the width of the image area to be affected (for **Tk_PhotoPutBlock**) or the desired image width (for **Tk_PhotoExpand** and **Tk_PhotoSetSize**).

int _compRule_ (in)
Specifies the compositing rule used when combining transparent pixels in a block of data with a photo image. Must be one of **TK_PHOTO_COMPOSITE_OVERLAY** (which puts the block of data over the top of the existing photo image, with the previous contents showing through in the transparent bits) or **TK_PHOTO_COMPOSITE_SET** (which discards the existing photo image contents in the rectangle covered by the data block.)

int _height_ (in)
Specifies the height of the image area to be affected (for **Tk_PhotoPutBlock**) or the desired image height (for **Tk_PhotoExpand** and **Tk_PhotoSetSize**).

int *_widthPtr_ (out)
Pointer to location in which to store the image width.

int *_heightPtr_ (out)
Pointer to location in which to store the image height.

int _subsampleX_ (in)
Specifies the subsampling factor in the X direction for input image data.

int _subsampleY_ (in)
Specifies the subsampling factor in the Y direction for input image data.

int _zoomX_ (in)
Specifies the zoom factor to be applied in the X direction to pixels being written to the photo image.

int _zoomY_ (in)
Specifies the zoom factor to be applied in the Y direction to pixels being written to the photo image.

Description

Tk_FindPhoto returns an opaque handle that is used to identify a particular photo image to the other procedures. The parameter is the name of the image, that is, the name specified to the **image create photo** command, or assigned by that command if no name was specified. If

imageName does not exist or is not a photo image, **Tk_FindPhoto** returns NULL.

Tk_PhotoPutBlock is used to supply blocks of image data to be displayed. The call affects an area of the image of size *width* x *height* pixels, with its top-left corner at coordinates (*x*,*y*). All of *width*, *height*, *x*, and *y* must be non-negative. If part of this area lies outside the current bounds of the image, the image will be expanded to include the area, unless the user has specified an explicit image size with the **-width** and/or **-height** widget configuration options (see photo(n)); in that case the area is silently clipped to the image boundaries.

The *block* parameter is a pointer to a **Tk_PhotoImageBlock** structure, defined as follows:

```
typedef struct {
    unsigned char *pixelPtr;
    int width;
    int height;
    int pitch;
    int pixelSize;
    int offset[4];
} Tk_PhotoImageBlock;
```

The *pixelPtr* field points to the first pixel, that is, the top-left pixel in the block. The *width* and *height* fields specify the dimensions of the block of pixels. The *pixelSize* field specifies the address difference between two horizontally adjacent pixels. It should be 4 for RGB and 2 for grayscale image data. Other values are possible, if the offsets in the *offset* array are adjusted accordingly (e.g. for red, green and blue data stored in different planes). Using such a layout is strongly discouraged, though. Due to a bug, it might not work correctly if an alpha channel is provided. (see the **BUGS** section below). The *pitch* field specifies the address difference between two vertically adjacent pixels. The *offset* array contains the offsets from the address of a pixel to the addresses of the bytes containing the red, green, blue and alpha (transparency) components. If the offsets for red, green and blue are equal, the image is interpreted as grayscale. If they differ, RGB data is assumed. Normally the offsets will be 0, 1, 2, 3 for RGB data and 0, 0, 0, 1 for grayscale. It is possible to provide image data without an alpha channel by setting the offset for alpha to a negative value and adjusting the *pixelSize* field accordingly. This use is discouraged, though (see the **BUGS** section below).

The *compRule* parameter to **Tk_PhotoPutBlock** specifies a compositing rule that says what to do with transparent pixels. The value **TK_PHOTO_COMPOSITE_OVERLAY** says that the previous contents of the photo image should show through, and the value **TK_PHOTO_COMPOSITE_SET** says that the previous contents of the photo image should be completely ignored, and the values from the block be copied directly across. The behavior in Tk8.3 and earlier was equivalent to having **TK_PHOTO_COMPOSITE_OVERLAY** as a compositing rule.

The value given for the *width* and *height* parameters to **Tk_PhotoPutBlock** do not have to correspond to the values specified in *block*. If they are smaller, **Tk_PhotoPutBlock** extracts a sub-block from the image data supplied. If they are larger, the data given are replicated (in a tiled fashion) to fill the specified area. These rules operate independently in the horizontal and vertical directions.

Tk_PhotoPutBlock normally returns **TCL_OK**, though if it cannot allocate sufficient memory to hold the resulting image, **TCL_ERROR** is returned instead and, if the *interp* argument is

non-NULL, an error message is placed in the interpreter's result.

Tk_PhotoPutZoomedBlock works like **Tk_PhotoPutBlock** except that the image can be reduced or enlarged for display. The *subsampleX* and *subsampleY* parameters allow the size of the image to be reduced by subsampling. **Tk_PhotoPutZoomedBlock** will use only pixels from the input image whose X coordinates are multiples of *subsampleX*, and whose Y coordinates are multiples of *subsampleY*. For example, an image of 512x512 pixels can be reduced to 256x256 by setting *subsampleX* and *subsampleY* to 2.

The *zoomX* and *zoomY* parameters allow the image to be enlarged by pixel replication. Each pixel of the (possibly subsampled) input image will be written to a block *zoomX* pixels wide and *zoomY* pixels high of the displayed image. Subsampling and zooming can be used together for special effects.

Tk_PhotoGetImage can be used to retrieve image data from a photo image. **Tk_PhotoGetImage** fills in the structure pointed to by the *blockPtr* parameter with values that describe the address and layout of the image data that the photo image has stored internally. The values are valid until the image is destroyed or its size is changed.

It is possible to modify an image by writing directly to the data the *pixelPtr* field points to. The size of the image cannot be changed this way, though. Also, changes made by writing directly to *pixelPtr* will not be immediately visible, but only after a call to **Tk_ImageChanged** or after an event that causes the interested widgets to redraw themselves. For these reasons usually it is preferable to make changes to a copy of the image data and write it back with **Tk_PhotoPutBlock** or **Tk_PhotoPutZoomedBlock**.

Tk_PhotoGetImage returns 1 for compatibility with the corresponding procedure in the old photo widget.

Tk_PhotoBlank blanks the entire area of the photo image. Blank areas of a photo image are transparent.

Tk_PhotoExpand requests that the widget's image be expanded to be at least *width* x *height* pixels in size. The width and/or height are unchanged if the user has specified an explicit image width or height with the **-width** and/or **-height** configuration options, respectively. If the image data are being supplied in many small blocks, it is more efficient to use **Tk_PhotoExpand** or **Tk_PhotoSetSize** at the beginning rather than allowing the image to expand in many small increments as image blocks are supplied.

Tk_PhotoExpand normally returns **TCL_OK**, though if it cannot allocate sufficient memory to hold the resulting image, **TCL_ERROR** is returned instead and, if the *interp* argument is non-NULL, an error message is placed in the interpreter's result.

Tk_PhotoSetSize specifies the size of the image, as if the user had specified the given *width* and *height* values to the **-width** and **-height** configuration options. A value of zero for *width* or *height* does not change the image's width or height, but allows the width or height to be changed by subsequent calls to **Tk_PhotoPutBlock**, **Tk_PhotoPutZoomedBlock** or **Tk_PhotoExpand**.

Tk_PhotoSetSize normally returns **TCL_OK**, though if it cannot allocate sufficient memory to hold the resulting image, **TCL_ERROR** is returned instead and, if the *interp* argument is non-NULL, an error message is placed in the interpreter's result.

Tk_PhotoGetSize returns the dimensions of the image in **widthPtr* and **heightPtr*.

Portability

In Tk 8.3 and earlier, **Tk_PhotoPutBlock** and **Tk_PhotoPutZoomedBlock** had different signatures. If you want to compile code that uses the old interface against 8.4 without updating your code, compile it with the flag -DUSE_COMPOSITELESS_PHOTO_PUT_BLOCK. Code linked using Stubs against older versions of Tk will continue to work.

In Tk 8.4, **Tk_PhotoPutBlock**, **Tk_PhotoPutZoomedBlock**, **Tk_PhotoExpand** and **Tk_PhotoSetSize** did not take an *interp* argument or return any result code. If insufficient memory was available for an image, Tk would panic. This behaviour is still supported if you compile your extension with the additional flag -DUSE_PANIC_ON_PHOTO_ALLOC_FAILURE. Code linked using Stubs against older versions of Tk will continue to work.

Bugs

The **Tk_PhotoImageBlock** structure used to provide image data to **Tk_PhotoPutBlock** promises great flexibility in the layout of the data (e.g. separate planes for the red, green, blue and alpha channels). Unfortunately, the implementation fails to hold this promise. The problem is that the *pixelSize* field is (incorrectly) used to determine whether the image has an alpha channel. Currently, if the offset for the alpha channel is greater than or equal to *pixelSize*, **tk_PhotoPutblock** assumes no alpha data is present and makes the image fully opaque. This means that for layouts where the channels are separate (or any other exotic layout where *pixelSize* has to be smaller than the alpha offset), the alpha channel will not be read correctly. In order to be on the safe side if this issue will be corrected in a future release, it is strongly recommended you always provide alpha data - even if the image has no transparency - and only use the "standard" layout with a *pixelSize* of 2 for grayscale and 4 for RGB data with *offset*s of 0, 0, 0, 1 or 0, 1, 2, 3 respectively.

Credits

The code for the photo image type was developed by Paul Mackerras, based on his earlier photo widget code.

Keywords: FindPhoto, photo, image

Copyright (c) 1994 The Australian National University
Copyright (c) 1994-1996 Sun Microsystems, Inc.

FontId

Name

Tk_FontId, Tk_GetFontMetrics, Tk_PostscriptFontName - accessor functions for fonts

Synopsis

#include <tk.h>

Font
Tk_FontId(*tkfont*)

Tk_GetFontMetrics(*tkfont*, *fmPtr*)

int
Tk_PostscriptFontName(*tkfont*, *dsPtr*)

Arguments

Tk_Font *tkfont* (in)
Opaque font token being queried. Must have been returned by a previous call to **Tk_GetFont**.

Tk_FontMetrics **fmPtr* (out)
Pointer to structure in which the font metrics for *tkfont* will be stored. See **DATA STRUCTURES** below for details.

Tcl_DString **dsPtr* (out)
Pointer to an initialized **Tcl_DString** to which the name of the Postscript font that corresponds to *tkfont* will be appended.

Description

Given a *tkfont*, **Tk_FontId** returns the token that should be selected into an XGCValues structure in order to construct a graphics context that can be used to draw text in the specified font.

Tk_GetFontMetrics computes the ascent, descent, and linespace of the *tkfont* in pixels and stores those values in the structure pointer to by *fmPtr*. These values can be used in computations such as to space multiple lines of text, to align the baselines of text in different fonts, and to vertically align text in a given region. See the documentation for the **font** command for definitions of the terms ascent, descent, and linespace, used in font metrics.

Tk_PostscriptFontName maps a *tkfont* to the corresponding Postscript font name that should be used when printing. The return value is the size in points of the *tkfont* and the Postscript font name is appended to *dsPtr*. *DsPtr* must refer to an initialized **Tcl_DString**. Given a "reasonable" Postscript printer, the following screen font families should print correctly:

Avant Garde, **Arial**, **Bookman**, **Courier**, **Courier New**, **Geneva**, **Helvetica**, **Monaco**, **New Century Schoolbook**, **New York**, **Palatino**, **Symbol**, **Times**, **Times New Roman**, **Zapf Chancery**, and **Zapf Dingbats**. Any other font families may not print correctly because the computed Postscript font name may be incorrect or not exist on the printer.

Data structures

The **Tk_FontMetrics** data structure is used by **Tk_GetFontMetrics** to return information about a font and is defined as follows:

```
typedef struct Tk_FontMetrics {
    int ascent;
    int descent;
    int linespace;
} Tk_FontMetrics;
```

The *ascent* field is the amount in pixels that the tallest letter sticks up above the baseline, plus any extra blank space added by the designer of the font.

The *descent* is the largest amount in pixels that any letter sticks below the baseline, plus any extra blank space added by the designer of the font.

The *linespace* is the sum of the ascent and descent. How far apart two lines of text in the same font should be placed so that none of the characters in one line overlap any of the characters in the other line.

Keywords: FontId, font, measurement, Postscript

Copyright (c) 1996 Sun Microsystems, Inc.

FreeXId

Name

Tk_FreeXId - make X resource identifier available for reuse

Synopsis

#include <tk.h>

Tk_FreeXId(*display*, *id*)

Arguments

Display ****display* **(in)**

Display for which *id* was allocated.

XID *id* (in)

Identifier of X resource (window, font, pixmap, cursor, graphics context, or colormap) that is no longer in use.

Description

The default allocator for resource identifiers provided by Xlib is very simple-minded and does not allow resource identifiers to be re-used. If a long-running application reaches the end of the resource id space, it will generate an X protocol error and crash. Tk replaces the default id allocator with its own allocator, which allows identifiers to be reused. In order for this to work, **Tk_FreeXId** must be called to tell the allocator about resources that have been freed. Tk automatically calls **Tk_FreeXId** whenever it frees a resource, so if you use procedures like **Tk_GetFont**, **Tk_GetGC**, and **Tk_GetPixmap** then you need not call **Tk_FreeXId**. However, if you allocate resources directly from Xlib, for example by calling **XCreatePixmap**, then you should call **Tk_FreeXId** when you call the corresponding Xlib free procedure, such as **XFreePixmap**. If you do not call **Tk_FreeXId** then the resource identifier will be lost, which could cause problems if the application runs long enough to lose all of the available identifiers.

Keywords: FreeXId, resource identifier

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

GeomReq

Name

Tk_GeometryRequest, Tk_SetMinimumRequestSize, Tk_SetInternalBorder, Tk_SetInternalBorderEx - specify desired geometry or internal border for a window

Synopsis

#include <tk.h>

Tk_GeometryRequest(*tkwin*, *reqWidth*, *reqHeight*)

Tk_SetMinimumRequestSize(*tkwin*, *minWidth*, *minHeight*)

Tk_SetInternalBorder(*tkwin*, *width*)

Tk_SetInternalBorderEx(*tkwin*, *left*, *right*, *top*, *bottom*)

Arguments

Tk_Window *tkwin* (in)
Window for which geometry is being requested.

int *reqWidth* (in)
Desired width for *tkwin*, in pixel units.

int *reqHeight* (in)
Desired height for *tkwin*, in pixel units.

int *minWidth* (in)
Desired minimum requested width for *tkwin*, in pixel units.

int *minHeight* (in)
Desired minimum requested height for *tkwin*, in pixel units.

int *width* (in)
Space to leave for internal border for *tkwin*, in pixel units.

int *left* (in)
Space to leave for left side of internal border for *tkwin*, in pixel units.

int *right* (in)
Space to leave for right side of internal border for *tkwin*, in pixel units.

int *top* (in)
Space to leave for top side of internal border for *tkwin*, in pixel units.

int *bottom* (in)
Space to leave for bottom side of internal border for *tkwin*, in pixel units.

Description

Tk_GeometryRequest is called by widget code to indicate its preference for the dimensions of a particular window. The arguments to **Tk_GeometryRequest** are made available to the geometry manager for the window, which then decides on the actual geometry for the window. Although geometry managers generally try to satisfy requests made to **Tk_GeometryRequest**, there is no guarantee that this will always be possible. Widget code should not assume that a geometry request will be satisfied until it receives a **ConfigureNotify** event indicating that the geometry change has occurred. Widget code should never call procedures like **Tk_ResizeWindow** directly. Instead, it should invoke **Tk_GeometryRequest** and leave the final geometry decisions to the geometry manager.

If *tkwin* is a top-level window, then the geometry information will be passed to the window manager using the standard ICCCM protocol.

Tk_SetInternalBorder is called by widget code to indicate that the widget has an internal border. This means that the widget draws a decorative border inside the window instead of using the standard X borders, which are external to the window's area. For example, internal borders are used to draw 3-D effects. *Width* specifies the width of the border in pixels. Geometry managers will use this information to avoid placing any children of *tkwin* overlapping the outermost *width* pixels of *tkwin*'s area.

Tk_SetInternalBorderEx works like **Tk_SetInternalBorder** but lets you specify different widths for different sides of the window.

Tk_SetMinimumRequestSize is called by widget code to indicate that a geometry manager should request at least this size for the widget. This allows a widget to have some control over its size when a propagating geometry manager is used inside it.

The information specified in calls to **Tk_GeometryRequest**, **Tk_SetMinimumRequestSize**, **Tk_SetInternalBorder** and **Tk_SetInternalBorderEx** can be retrieved using the macros **Tk_ReqWidth**, **Tk_ReqHeight**, **Tk_MinReqWidth**, **Tk_MinReqHeight**, **Tk_MinReqWidth**, **Tk_InternalBorderLeft**, **Tk_InternalBorderRight**, **Tk_InternalBorderTop** and **Tk_InternalBorderBottom**. See the **Tk_WindowId** manual entry for details.

Keywords: GeomReq, geometry, request

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

GetAnchor

Name

Tk_GetAnchorFromObj, Tk_GetAnchor, Tk_NameOfAnchor - translate between strings and anchor positions

Synopsis

#include <tk.h>

int
Tk_GetAnchorFromObj(*interp*, *objPtr*, *anchorPtr*)

int
Tk_GetAnchor(*interp*, *string*, *anchorPtr*)

const char *
Tk_NameOfAnchor(*anchor*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter to use for error reporting, or NULL.

Tcl_Obj *_objPtr_ (in/out)
String value contains name of anchor point: "**n**", "**ne**", "**e**", "**se**", "**s**", "**sw**", "**w**", "**nw**", or "**center**"; internal rep will be modified to cache corresponding Tk_Anchor. In the case of "**center**" on input, a non-empty abbreviation of it may also be used on input.

const char *_string_ (in)
Same as *objPtr* except description of anchor point is passed as a string.

int *_anchorPtr_ (out)
Pointer to location in which to store anchor position corresponding to *objPtr* or *string*.

Tk_Anchor _anchor_ (in)
Anchor position, e.g. **TCL_ANCHOR_CENTER**.

Description

Tk_GetAnchorFromObj places in **anchorPtr* an anchor position (enumerated type **Tk_Anchor**) corresponding to *objPtr*'s value. The result will be one of **TK_ANCHOR_N**, **TK_ANCHOR_NE**, **TK_ANCHOR_E**, **TK_ANCHOR_SE**, **TK_ANCHOR_S**, **TK_ANCHOR_SW**, **TK_ANCHOR_W**,

TK_ANCHOR_NW, or **TK_ANCHOR_CENTER**. Anchor positions are typically used for indicating a point on an object that will be used to position the object, e.g. **TK_ANCHOR_N** means position the top center point of the object at a particular place.

Under normal circumstances the return value is **TCL_OK** and *interp* is unused. If *string* does not contain a valid anchor position or an abbreviation of one of these names, **TCL_ERROR** is returned, **anchorPtr* is unmodified, and an error message is stored in *interp*'s result if *interp* is not NULL. **Tk_GetAnchorFromObj** caches information about the return value in *objPtr*, which speeds up future calls to **Tk_GetAnchorFromObj** with the same *objPtr*.

Tk_GetAnchor is identical to **Tk_GetAnchorFromObj** except that the description of the anchor is specified with a string instead of an object. This prevents **Tk_GetAnchor** from caching the return value, so **Tk_GetAnchor** is less efficient than **Tk_GetAnchorFromObj**.

Tk_NameOfAnchor is the logical inverse of **Tk_GetAnchor**. Given an anchor position such as **TK_ANCHOR_N** it returns a statically-allocated string corresponding to *anchor*. If *anchor* is not a legal anchor value, then "unknown anchor position" is returned.

Keywords: GetAnchor, anchor position

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1998 Sun Microsystems, Inc.

GetBitmap

Name

Tk_AllocBitmapFromObj, Tk_GetBitmap, Tk_GetBitmapFromObj, Tk_DefineBitmap, Tk_NameOfBitmap, Tk_SizeOfBitmap, Tk_FreeBitmapFromObj, Tk_FreeBitmap - maintain database of single-plane pixmaps

Synopsis

#include <tk.h>

Pixmap
Tk_AllocBitmapFromObj(*interp*, *tkwin*, *objPtr*)

Pixmap
Tk_GetBitmap(*interp*, *tkwin*, *info*)

Pixmap
Tk_GetBitmapFromObj(*tkwin*, *objPtr*)

int
Tk_DefineBitmap(*interp*, *name*, *source*, *width*, *height*)

const char *
Tk_NameOfBitmap(*display*, *bitmap*)

Tk_SizeOfBitmap(*display*, *bitmap*, *widthPtr*, *heightPtr*)

Tk_FreeBitmapFromObj(*tkwin*, *objPtr*)

Tk_FreeBitmap(*display*, *bitmap*)

Arguments

Tcl_Interp ****interp* **(in)

Interpreter to use for error reporting; if NULL then no error message is left after errors.

Tk_Window *tkwin* (in)
Token for window in which the bitmap will be used.

Tcl_Obj **objPtr* (in/out)
String value describes desired bitmap; internal rep will be modified to cache pointer to corresponding Pixmap.

const char **info* (in)
Same as *objPtr* except description of bitmap is passed as a string and resulting Pixmap is not cached.

const char **name* (in)
Name for new bitmap to be defined.

const void **source* (in)
Data for bitmap, in standard bitmap format. Must be stored in static memory whose value will never change.

int *width* (in)
Width of bitmap.

int *height* (in)
Height of bitmap.

int **widthPtr* (out)
Pointer to word to fill in with *bitmap*'s width.

int **heightPtr* (out)
Pointer to word to fill in with *bitmap*'s height.

Display **display* (in)
Display for which *bitmap* was allocated.

Pixmap *bitmap* (in)
Identifier for a bitmap allocated by **Tk_AllocBitmapFromObj** or **Tk_GetBitmap**.

Description

These procedures manage a collection of bitmaps (one-plane pixmaps) being used by an application. The procedures allow bitmaps to be re-used efficiently, thereby avoiding server overhead, and also allow bitmaps to be named with character strings.

Tk_AllocBitmapFromObj returns a Pixmap identifier for a bitmap that matches the description in *objPtr* and is suitable for use in *tkwin*. It re-uses an existing bitmap, if possible, and creates a new one otherwise. *ObjPtr*'s value must have one of the following forms:

@fileName
FileName must be the name of a file containing a bitmap description in the standard X11 format.

name
Name must be the name of a bitmap defined previously with a call to **Tk_DefineBitmap**. The following names are pre-defined by Tk:

error
The international "don't" symbol: a circle with a diagonal line across it.

gray75
75% gray: a checkerboard pattern where three out of four bits are on.

gray50
50% gray: a checkerboard pattern where every other bit is on.

gray25
25% gray: a checkerboard pattern where one out of every four bits is on.

gray12
12.5% gray: a pattern where one-eighth of the bits are on, consisting of every fourth pixel in every other row.

hourglass
An hourglass symbol.

info
A large letter "i".

questhead
The silhouette of a human head, with a question mark in it.

question
A large question-mark.

warning
A large exclamation point. In addition, the following pre-defined names are available only on the **Macintosh** platform:

document
A generic document.

stationery
Document stationery.

edition
The *edition* symbol.

application
Generic application icon.

accessory
A desk accessory.

folder
Generic folder icon.

pfolder
A locked folder.

trash
A trash can.

floppy
A floppy disk.

ramdisk
A floppy disk with chip.

cdrom
A cd disk icon.

preferences
A folder with prefs symbol.

querydoc
A database document icon.

stop
A stop sign.

note
A face with balloon words.

caution
A triangle with an exclamation point.

Under normal conditions, **Tk_AllocBitmapFromObj** returns an identifier for the requested bitmap. If an error occurs in creating the bitmap, such as when *objPtr* refers to a non-existent file, then **None** is returned and an error message is left in *interp*'s result if *interp* is not NULL. **Tk_AllocBitmapFromObj** caches information about the return value in *objPtr*, which speeds up future calls to procedures such as **Tk_AllocBitmapFromObj** and **Tk_GetBitmapFromObj**.

Tk_GetBitmap is identical to **Tk_AllocBitmapFromObj** except that the description of the bitmap is specified with a string instead of an object. This prevents **Tk_GetBitmap** from caching the return value, so **Tk_GetBitmap** is less efficient than **Tk_AllocBitmapFromObj**.

Tk_GetBitmapFromObj returns the token for an existing bitmap, given the window and description used to create the bitmap. **Tk_GetBitmapFromObj** does not actually create the bitmap; the bitmap must already have been created with a previous call to **Tk_AllocBitmapFromObj** or **Tk_GetBitmap**. The return value is cached in *objPtr*, which speeds up future calls to **Tk_GetBitmapFromObj** with the same *objPtr* and *tkwin*.

Tk_DefineBitmap associates a name with in-memory bitmap data so that the name can be used in later calls to **Tk_AllocBitmapFromObj** or **Tk_GetBitmap**. The *nameId* argument gives a name for the bitmap; it must not previously have been used in a call to **Tk_DefineBitmap**. The arguments *source*, *width*, and *height* describe the bitmap. **Tk_DefineBitmap** normally returns **TCL_OK**; if an error occurs (e.g. a bitmap named *nameId* has already been defined) then **TCL_ERROR** is returned and an error message is left in interpreter *interp*'s result. Note: **Tk_DefineBitmap** expects the memory pointed to by *source* to be static: **Tk_DefineBitmap** does not make a private copy of this memory, but uses the bytes pointed to by *source* later in calls to **Tk_AllocBitmapFromObj** or **Tk_GetBitmap**.

Typically **Tk_DefineBitmap** is used by **#include**-ing a bitmap file directly into a C program and then referencing the variables defined by the file. For example, suppose there exists a file **stip.bitmap**, which was created by the **bitmap** program and contains a stipple pattern. The following code uses **Tk_DefineBitmap** to define a new bitmap named **foo**:

```
Pixmap bitmap;
#include "stip.bitmap"
Tk_DefineBitmap(interp, "foo", stip_bits,
    stip_width, stip_height);
...
bitmap = Tk_GetBitmap(interp, tkwin, "foo");
```

This code causes the bitmap file to be read at compile-time and incorporates the bitmap information into the program's executable image. The same bitmap file could be read at run-time using **Tk_GetBitmap**:

```
Pixmap bitmap;
bitmap = Tk_GetBitmap(interp, tkwin, "@stip.bitmap");
```

The second form is a bit more flexible (the file could be modified after the program has been compiled, or a different string could be provided to read a different file), but it is a little slower and requires the bitmap file to exist separately from the program.

Tk maintains a database of all the bitmaps that are currently in use. Whenever possible, it will return an existing bitmap rather than creating a new one. When a bitmap is no longer used, Tk will release it automatically. This approach can substantially reduce server overhead, so **Tk_AllocBitmapFromObj** and **Tk_GetBitmap** should generally be used in preference to Xlib procedures like **XReadBitmapFile**.

The bitmaps returned by **Tk_AllocBitmapFromObj** and **Tk_GetBitmap** are shared, so callers should never modify them. If a bitmap must be modified dynamically, then it should be created by calling Xlib procedures such as **XReadBitmapFile** or **XCreatePixmap** directly.

The procedure **Tk_NameOfBitmap** is roughly the inverse of **Tk_GetBitmap**. Given an X Pixmap argument, it returns the textual description that was passed to **Tk_GetBitmap** when the bitmap was created. *Bitmap* must have been the return value from a previous call to **Tk_AllocBitmapFromObj** or **Tk_GetBitmap**.

Tk_SizeOfBitmap returns the dimensions of its *bitmap* argument in the words pointed to by the *widthPtr* and *heightPtr* arguments. As with **Tk_NameOfBitmap**, *bitmap* must have been created by **Tk_AllocBitmapFromObj** or **Tk_GetBitmap**.

When a bitmap is no longer needed, **Tk_FreeBitmapFromObj** or **Tk_FreeBitmap** should be called to release it. For **Tk_FreeBitmapFromObj** the bitmap to release is specified with the same information used to create it; for **Tk_FreeBitmap** the bitmap to release is specified with its Pixmap token. There should be exactly one call to **Tk_FreeBitmapFromObj** or **Tk_FreeBitmap** for each call to **Tk_AllocBitmapFromObj** or **Tk_GetBitmap**.

Bugs

In determining whether an existing bitmap can be used to satisfy a new request, **Tk_AllocBitmapFromObj** and **Tk_GetBitmap** consider only the immediate value of the string description. For example, when a file name is passed to **Tk_GetBitmap**, **Tk_GetBitmap** will assume it is safe to re-use an existing bitmap created from the same file name: it will not check to see whether the file itself has changed, or whether the current directory has changed, thereby causing the name to refer to a different file.

Keywords: GetBitmap, bitmap, pixmap

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1998 Sun Microsystems, Inc.

GetCapStyl

Name

Tk_GetCapStyle, Tk_NameOfCapStyle - translate between strings and cap styles

Synopsis

#include <tk.h>

int
Tk_GetCapStyle(*interp*, *string*, *capPtr*)

const char *
Tk_NameOfCapStyle(*cap*)

Arguments

Tcl_Interp ***interp* **(in)
Interpreter to use for error reporting.

const char ***string* **(in)
String containing name of cap style - one of "**butt**", "**projecting**", or "**round**" - or a unique abbreviation of one.

int ***capPtr* **(out)
Pointer to location in which to store X cap style corresponding to *string*.

int *cap* (in)
Cap style: one of **CapButt**, **CapProjecting**, or **CapRound**.

Description

Tk_GetCapStyle places in **capPtr* the X cap style corresponding to *string*. This will be one of the values **CapButt**, **CapProjecting**, or **CapRound**. Cap styles are typically used in X graphics contexts to indicate how the end-points of lines should be capped. See the X documentation for information on what each style implies.

Under normal circumstances the return value is **TCL_OK** and *interp* is unused. If *string* does not contain a valid cap style or an abbreviation of one of these names, then an error message is stored in interpreter *interp*'s result, **TCL_ERROR** is returned, and **capPtr* is unmodified.

Tk_NameOfCapStyle is the logical inverse of **Tk_GetCapStyle**. Given a cap style such as **CapButt** it returns a statically-allocated string corresponding to *cap*. If *cap* is not a legal cap style, then "unknown cap style" is returned.

Keywords: GetCapStyl, butt, cap style, projecting, round

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

GetClrmap

Name

Tk_GetColormap, Tk_PreserveColormap, Tk_FreeColormap - allocate and free colormaps

Synopsis

#include <tk.h>

Colormap
Tk_GetColormap(*interp*, *tkwin*, *string*)

Tk_PreserveColormap(*display*, *colormap*)

Tk_FreeColormap(*display*, *colormap*)

Arguments

Tcl_Interp ***interp* **(in)
Interpreter to use for error reporting.

Tk_Window *tkwin* (in)

Token for window in which colormap will be used.

const char *_string_ (in)
Selects a colormap: either **new** or the name of a window with the same screen and visual as *tkwin*.

Display *_display_ (in)
Display for which *colormap* was allocated.

Colormap _colormap_ (in)
Colormap to free or preserve; must have been returned by a previous call to **Tk_GetColormap** or **Tk_GetVisual**.

Description

These procedures are used to manage colormaps. **Tk_GetColormap** returns a colormap suitable for use in *tkwin*. If its *string* argument is **new** then a new colormap is created; otherwise *string* must be the name of another window with the same screen and visual as *tkwin*, and the colormap from that window is returned. If *string* does not make sense, or if it refers to a window on a different screen from *tkwin* or with a different visual than *tkwin*, then **Tk_GetColormap** returns **None** and leaves an error message in *interp*'s result.

Tk_PreserveColormap increases the internal reference count for a colormap previously returned by **Tk_GetColormap**, which allows the colormap to be stored in several locations without knowing which order they will be released.

Tk_FreeColormap should be called when a colormap returned by **Tk_GetColormap** is no longer needed. Tk maintains a reference count for each colormap returned by **Tk_GetColormap** , so there should eventually be one call to **Tk_FreeColormap** for each call to **Tk_GetColormap** and each call to **Tk_PreserveColormap**. When a colormap's reference count becomes zero, Tk releases the X colormap.

Tk_GetVisual and **Tk_GetColormap** work together, in that a new colormap created by **Tk_GetVisual** may later be returned by **Tk_GetColormap**. The reference counting mechanism for colormaps includes both procedures, so callers of **Tk_GetVisual** must also call **Tk_FreeColormap** to release the colormap. If **Tk_GetColormap** is called with a *string* value of **new** then the resulting colormap will never be returned by **Tk_GetVisual**; however, it can be used in other windows by calling **Tk_GetColormap** with the original window's name as *string*.

Keywords: GetClrmap, colormap, visual

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

GetColor

Name

Tk_AllocColorFromObj, Tk_GetColor, Tk_GetColorFromObj, Tk_GetColorByValue, Tk_NameOfColor, Tk_FreeColorFromObj, Tk_FreeColor - maintain database of colors

Synopsis

#include <tk.h>

XColor *
Tk_AllocColorFromObj(*interp*, *tkwin*, *objPtr*)

XColor *
Tk_GetColor(*interp*, *tkwin*, *name*)

XColor *
Tk_GetColorFromObj(*tkwin*, *objPtr*)

XColor *
Tk_GetColorByValue(*tkwin*, *prefPtr*)

const char *
Tk_NameOfColor(*colorPtr*)

GC
Tk_GCForColor(*colorPtr*, *drawable*)

Tk_FreeColorFromObj(*tkwin*, *objPtr*)

Tk_FreeColor(*colorPtr*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter to use for error reporting.

Tk_Window _tkwin_ (in)
Token for window in which color will be used.

Tcl_Obj *_objPtr_ (in/out)
String value describes desired color; internal rep will be modified to cache pointer to corresponding (XColor *).

const char *_name_ (in)
Same as *objPtr* except description of color is passed as a string and resulting (XColor *) is not cached.

XColor *_prefPtr_ (in)
Indicates red, green, and blue intensities of desired color.

XColor *_colorPtr_ (in)
Pointer to X color information. Must have been allocated by previous call to **Tk_AllocColorFromObj**, **Tk_GetColor** or **Tk_GetColorByValue**, except when passed to **Tk_NameOfColor**.

Drawable _drawable_ (in)
Drawable in which the result graphics context will be used. Must have same screen and depth as the window for which the color was allocated.

Description

These procedures manage the colors being used by a Tk application. They allow colors to be shared whenever possible, so that colormap space is preserved, and they pick closest available colors when colormap space is exhausted.

Given a textual description of a color, **Tk_AllocColorFromObj** locates a pixel value that may be used to render the color in a particular window. The desired color is specified with a value whose string value must have one of the following forms:

colorname
Any of the valid textual names for a color defined in the server's color database file, such as

red or **PeachPuff**.

#RGB

...see next...

#RRGGBB

...see next...

#RRRGGGBBB

...see next...

#RRRRGGGGBBBB

A numeric specification of the red, green, and blue intensities to use to display the color. Each *R*, *G*, or *B* represents a single hexadecimal digit. The four forms permit colors to be specified with 4-bit, 8-bit, 12-bit or 16-bit values. When fewer than 16 bits are provided for each color, they represent the most significant bits of the color, while the lower unfilled bits will be repeatedly replicated from the available higher bits. For example, #3a7 is the same as #3333aaaa7777. **Tk_AllocColorFromObj** returns a pointer to an XColor structure; the structure indicates the exact intensities of the allocated color (which may differ slightly from those requested, depending on the limitations of the screen) and a pixel value that may be used to draw with the color in *tkwin*. If an error occurs in **Tk_AllocColorFromObj** (such as an unknown color name) then NULL is returned and an error message is stored in *interp*'s result if *interp* is not NULL. If the colormap for *tkwin* is full, **Tk_AllocColorFromObj** will use the closest existing color in the colormap. **Tk_AllocColorFromObj** caches information about the return value in *objPtr*, which speeds up future calls to procedures such as **Tk_AllocColorFromObj** and **Tk_GetColorFromObj**. **Tk_GetColor** is identical to **Tk_AllocColorFromObj** except that the description of the color is specified with a string instead of a value. This prevents **Tk_GetColor** from caching the return value, so **Tk_GetColor** is less efficient than **Tk_AllocColorFromObj**. **Tk_GetColorFromObj** returns the token for an existing color, given the window and description used to create the color. **Tk_GetColorFromObj** does not actually create the color; the color must already have been created with a previous call to **Tk_AllocColorFromObj** or **Tk_GetColor**. The return value is cached in *objPtr*, which speeds up future calls to **Tk_GetColorFromObj** with the same *objPtr* and *tkwin*. **Tk_GetColorByValue** is similar to **Tk_GetColor** except that the desired color is indicated with the *red*, *green*, and *blue* fields of the structure pointed to by *colorPtr*. This package maintains a database of all the colors currently in use. If the same color is requested multiple times from **Tk_GetColor** or **Tk_AllocColorFromObj** (e.g. by different windows), or if the same intensities are requested multiple times from **Tk_GetColorByValue**, then existing pixel values will be re-used. Re-using an existing pixel avoids any interaction with the window server, which makes the allocation much more efficient. These procedures also provide a portable interface that works across all platforms. For this reason, you should generally use **Tk_AllocColorFromObj**, **Tk_GetColor**, or **Tk_GetColorByValue** instead of lower level procedures like **XAllocColor**. Since different calls to this package may return the same shared pixel value, callers should never change the color of a pixel returned by the procedures. If you need to change a color value dynamically, you should use **XAllocColorCells** to allocate the pixel value for the color. The procedure **Tk_NameOfColor** is roughly the inverse of **Tk_GetColor**. If its *colorPtr* argument was created by **Tk_AllocColorFromObj** or **Tk_GetColor** then the return value is the string that was used to create the color. If *colorPtr* was created by a call to **Tk_GetColorByValue**, or by any other mechanism, then the return value is a string that could be passed to **Tk_GetColor** to return

the same color. Note: the string returned by **Tk_NameOfColor** is only guaranteed to persist until the next call to **Tk_NameOfColor**. **Tk_GCForColor** returns a graphics context whose **foreground** field is the pixel allocated for *colorPtr* and whose other fields all have default values. This provides an easy way to do basic drawing with a color. The graphics context is cached with the color and will exist only as long as *colorPtr* exists; it is freed when the last reference to *colorPtr* is freed by calling **Tk_FreeColor**. When a color is no longer needed **Tk_FreeColorFromObj** or **Tk_FreeColor** should be called to release it. For **Tk_FreeColorFromObj** the color to release is specified with the same information used to create it; for **Tk_FreeColor** the color to release is specified with a pointer to its XColor structure. There should be exactly one call to **Tk_FreeColorFromObj** or **Tk_FreeColor** for each call to **Tk_AllocColorFromObj**, **Tk_GetColor**, or **Tk_GetColorByValue**.

Keywords: GetColor, color, intensity, value, pixel value

Copyright (c) 1990-1991 The Regents of the University of California.
Copyright (c) 1994-1998 Sun Microsystems, Inc.

GetCursor

Name

Tk_AllocCursorFromObj, Tk_GetCursor, Tk_GetCursorFromObj, Tk_GetCursorFromData, Tk_NameOfCursor, Tk_FreeCursorFromObj, Tk_FreeCursor - maintain database of cursors

Synopsis

#include <tk.h>

Tk_Cursor
Tk_AllocCursorFromObj(*interp*, *tkwin*, *objPtr*)

Tk_Cursor
Tk_GetCursor(*interp*, *tkwin*, *name*)

Tk_Cursor
Tk_GetCursorFromObj(*tkwin*, *objPtr*)

Tk_Cursor
Tk_GetCursorFromData(*interp*, *tkwin*, *source*, *mask*, *width*, *height*, *xHot*, *yHot*, *fg*, *bg*)

const char *
Tk_NameOfCursor(*display*, *cursor*)

Tk_FreeCursorFromObj(*tkwin*, *objPtr*)

Tk_FreeCursor(*display*, *cursor*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter to use for error reporting.

Tk_Window _tkwin_ (in)
Token for window in which the cursor will be used.

Tcl_Obj *_objPtr_ (in/out)
Description of cursor; see below for possible values. Internal rep will be modified to cache pointer to corresponding Tk_Cursor.

const char *_name_ (in)
Same as *objPtr* except description of cursor is passed as a string and resulting Tk_Cursor is not cached.

const char *_source_ (in)
Data for cursor cursor, in standard cursor format.

const char *_mask_ (in)
Data for mask cursor, in standard cursor format.

int _width_ (in)
Width of *source* and *mask*.

int _height_ (in)
Height of *source* and *mask*.

int _xHot_ (in)
X-location of cursor hot-spot.

int _yHot_ (in)
Y-location of cursor hot-spot.

Tk_Uid _fg_ (in)
Textual description of foreground color for cursor.

Tk_Uid _bg_ (in)
Textual description of background color for cursor.

Display *_display_ (in)
Display for which *cursor* was allocated.

Tk_Cursor _cursor_ (in)
Opaque Tk identifier for cursor. If passed to **Tk_FreeCursor**, must have been returned by some previous call to **Tk_GetCursor** or **Tk_GetCursorFromData**.

Description

These procedures manage a collection of cursors being used by an application. The procedures allow cursors to be re-used efficiently, thereby avoiding server overhead, and also allow cursors to be named with character strings.

Tk_AllocCursorFromObj takes as argument an object describing a cursor, and returns an opaque Tk identifier for a cursor corresponding to the description. It re-uses an existing cursor if possible and creates a new one otherwise. **Tk_AllocCursorFromObj** caches information about the return value in *objPtr*, which speeds up future calls to procedures such as **Tk_AllocCursorFromObj** and **Tk_GetCursorFromObj**. If an error occurs in creating the cursor, such as when *objPtr* refers to a non-existent file, then **None** is returned and an error message will be stored in *interp*'s result if *interp* is not NULL. *ObjPtr* must contain a standard Tcl list with one of the following forms:

***name*\0[*fgColor*\0[*bgColor*]]**
Name is the name of a cursor in the standard X cursor cursor, i.e., any of the names defined in **cursorcursor.h**, without the **XC_**. Some example values are **X_cursor**, **hand2**, or **left_ptr**. Appendix B of "The X Window System" by Scheifler & Gettys has illustrations showing what each of these cursors looks like. If *fgColor* and *bgColor* are both specified, they give the foreground and background colors to use for the cursor (any of the forms acceptable to **Tk_GetColor** may be used). If only *fgColor* is specified, then there will be no background

color: the background will be transparent. If no colors are specified, then the cursor will use black for its foreground color and white for its background color.

The Macintosh version of Tk supports all of the X cursors and will also accept any of the standard Mac cursors including **ibeam**, **crosshair**, **watch**, **plus**, and **arrow**. In addition, Tk will load Macintosh cursor resources of the types **crsr** (color) and **CURS** (black and white) by the name of the resource. The application and all its open dynamic library's resource files will be searched for the named cursor. If there are conflicts color cursors will always be loaded in preference to black and white cursors.

@sourceName\0maskName\0fgColor\0bgColor
In this form, *sourceName* and *maskName* are the names of files describing cursors for the cursor's source bits and mask. Each file must be in standard X11 cursor format. *FgColor* and *bgColor* indicate the colors to use for the cursor, in any of the forms acceptable to **Tk_GetColor**. This form of the command will not work on Macintosh or Windows computers.

@sourceName\0fgColor
This form is similar to the one above, except that the source is used as mask also. This means that the cursor's background is transparent. This form of the command will not work on Macintosh or Windows computers.

@sourceName
This form only works on Windows, and will load a Windows system cursor (**.ani** or **.cur**) from the file specified in *sourceName*. **Tk_GetCursor** is identical to **Tk_AllocCursorFromObj** except that the description of the cursor is specified with a string instead of an object. This prevents **Tk_GetCursor** from caching the return value, so **Tk_GetCursor** is less efficient than **Tk_AllocCursorFromObj**. **Tk_GetCursorFromObj** returns the token for an existing cursor, given the window and description used to create the cursor. **Tk_GetCursorFromObj** does not actually create the cursor; the cursor must already have been created with a previous call to **Tk_AllocCursorFromObj** or **Tk_GetCursor**. The return value is cached in *objPtr*, which speeds up future calls to **Tk_GetCursorFromObj** with the same *objPtr* and *tkwin*. **Tk_GetCursorFromData** allows cursors to be created from in-memory descriptions of their source and mask cursors. *Source* points to standard cursor data for the cursor's source bits, and *mask* points to standard cursor data describing which pixels of *source* are to be drawn and which are to be considered transparent. *Width* and *height* give the dimensions of the cursor, *xHot* and *yHot* indicate the location of the cursor's hot-spot (the point that is reported when an event occurs), and *fg* and *bg* describe the cursor's foreground and background colors textually (any of the forms suitable for **Tk_GetColor** may be used). Typically, the arguments to **Tk_GetCursorFromData** are created by including a cursor file directly into the source code for a program, as in the following example:

```
Tk_Cursor cursor;
#include "source.cursor"
#include "mask.cursor"
cursor = Tk_GetCursorFromData(interp, tkwin, source_bits,
    mask_bits, source_width, source_height, source_x_hot,
    source_y_hot, Tk_GetUid("red"), Tk_GetUid("blue"));
```

Under normal conditions **Tk_GetCursorFromData** will return an identifier for the requested cursor. If an error occurs in creating the cursor then **None** is returned and an error message will

be stored in *interp*'s result.

Tk_AllocCursorFromObj, **Tk_GetCursor**, and **Tk_GetCursorFromData** maintain a database of all the cursors they have created. Whenever possible, a call to **Tk_AllocCursorFromObj**, **Tk_GetCursor**, or **Tk_GetCursorFromData** will return an existing cursor rather than creating a new one. This approach can substantially reduce server overhead, so the Tk procedures should generally be used in preference to Xlib procedures like **XCreateFontCursor** or **XCreatePixmapCursor**, which create a new cursor on each call. The Tk procedures are also more portable than the lower-level X procedures.

The procedure **Tk_NameOfCursor** is roughly the inverse of **Tk_GetCursor**. If its *cursor* argument was created by **Tk_GetCursor**, then the return value is the *name* argument that was passed to **Tk_GetCursor** to create the cursor. If *cursor* was created by a call to **Tk_GetCursorFromData**, or by any other mechanism, then the return value is a hexadecimal string giving the X identifier for the cursor. Note that the string returned by **Tk_NameOfCursor** is only guaranteed to persist until the next call to **Tk_NameOfCursor**. Also, this call is not portable except for cursors returned by **Tk_GetCursor**.

When a cursor returned by **Tk_AllocCursorFromObj**, **Tk_GetCursor**, or **Tk_GetCursorFromData** is no longer needed, **Tk_FreeCursorFromObj** or **Tk_FreeCursor** should be called to release it. For **Tk_FreeCursorFromObj** the cursor to release is specified with the same information used to create it; for **Tk_FreeCursor** the cursor to release is specified with its Tk_Cursor token. There should be exactly one call to **Tk_FreeCursor** for each call to **Tk_AllocCursorFromObj**, **Tk_GetCursor**, or **Tk_GetCursorFromData**.

Bugs

In determining whether an existing cursor can be used to satisfy a new request, **Tk_AllocCursorFromObj**, **Tk_GetCursor**, and **Tk_GetCursorFromData** consider only the immediate values of their arguments. For example, when a file name is passed to **Tk_GetCursor**, **Tk_GetCursor** will assume it is safe to re-use an existing cursor created from the same file name: it will not check to see whether the file itself has changed, or whether the current directory has changed, thereby causing the name to refer to a different file. Similarly, **Tk_GetCursorFromData** assumes that if the same *source* pointer is used in two different calls, then the pointers refer to the same data; it does not check to see if the actual data values have changed.

Keywords: GetCursor, cursor

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1998 Sun Microsystems, Inc.

GetDash

Name

Tk_GetDash - convert from string to valid dash structure.

Synopsis

#include <tk.h>

int
Tk_GetDash(*interp*, *string*, *dashPtr*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter to use for error reporting.

const char *_string_ (in)
Textual value to be converted.

Tk_Dash *_dashPtr_ (out)
Points to place to store the dash pattern value converted from *string*. Must not be NULL.

Description

These procedure parses the string and fills in the result in the Tk_Dash structure. The string can be a list of integers or a character string containing only "**.,-_**" and spaces. If all goes well, **TCL_OK** is returned and a dash descriptor is stored in the variable pointed to by *dashPtr*. If *string* does not have the proper syntax then **TCL_ERROR** is returned, an error message is left in the interpreter's result, and nothing is stored at **dashPtr*.

The first possible syntax is a list of integers. Each element represents the number of pixels of a line segment. Only the odd segments are drawn using the "outline" color. The other segments are drawn transparent.

The second possible syntax is a character list containing only 5 possible characters "**.,-_** ". The space can be used to enlarge the space between other line elements, and can not occur in the first position of the string. Some examples:

```
    -dash .     = -dash {2 4}
    -dash -     = -dash {6 4}
    -dash -.    = -dash {6 4 2 4}
    -dash -..   = -dash {6 4 2 4 2 4}
    -dash {. }  = -dash {2 8}
    -dash ,     = -dash {4 4}
```

The main difference between this syntax and the numeric is that it is shape-conserving. This means that all values in the dash list will be multiplied by the line width before display. This ensures that "." will always be displayed as a dot and "-" always as a dash regardless of the line width.

On systems where only a limited set of dash patterns, the dash pattern will be displayed as the most close dash pattern that is available. For example, on Windows only the first 4 of the above examples are available; the last 2 examples will be displayed identically to the first one.

Keywords: GetDash, dash, conversion

Copyright (c) 1989-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

GetFont

Name

Tk_AllocFontFromObj, Tk_GetFont, Tk_GetFontFromObj, Tk_NameOfFont, Tk_FreeFontFromObj, Tk_FreeFont - maintain database of fonts

Synopsis

#include <tk.h>

Tk_Font
Tk_AllocFontFromObj(*interp*, *tkwin*, *objPtr*)

Tk_Font
Tk_GetFont(*interp*, *tkwin*, *string*)

Tk_Font
Tk_GetFontFromObj(*tkwin*, *objPtr*)

const char *
Tk_NameOfFont(*tkfont*)

Tk_Font
Tk_FreeFontFromObj(*tkwin*, *objPtr*)

void
Tk_FreeFont(*tkfont*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter to use for error reporting. If **NULL**, then no error messages are left after errors.

Tk_Window _tkwin_ (in)
Token for window in which font will be used.

Tcl_Obj *_objPtr_ (in/out)
Gives name or description of font. See documentation for the **font** command for details on acceptable formats. Internal rep will be modified to cache corresponding Tk_Font.

const char *_string_ (in)
Same as *objPtr* except description of font is passed as a string and resulting Tk_Font is not cached.

Tk_Font _tkfont_ (in)
Opaque font token.

Description

Tk_AllocFontFromObj finds the font indicated by *objPtr* and returns a token that represents the font. The return value can be used in subsequent calls to procedures such as **Tk_GetFontMetrics**, **Tk_MeasureChars**, and **Tk_FreeFont**. The Tk_Font token will remain valid until **Tk_FreeFontFromObj** or **Tk_FreeFont** is called to release it. *ObjPtr* can contain either a symbolic name or a font description; see the documentation for the **font** command for a description of the valid formats. If **Tk_AllocFontFromObj** is unsuccessful (because, for example, *objPtr* did not contain a valid font specification) then it returns **NULL** and leaves an error message in *interp*'s result if *interp* is not **NULL**. **Tk_AllocFontFromObj** caches information about the return value in *objPtr*, which speeds up future calls to procedures such as **Tk_AllocFontFromObj** and **Tk_GetFontFromObj**.

Tk_GetFont is identical to **Tk_AllocFontFromObj** except that the description of the font is specified with a string instead of an object. This prevents **Tk_GetFont** from caching the matching Tk_Font, so **Tk_GetFont** is less efficient than **Tk_AllocFontFromObj**.

Tk_GetFontFromObj returns the token for an existing font, given the window and description used to create the font. **Tk_GetFontFromObj** does not actually create the font; the font must

already have been created with a previous call to **Tk_AllocFontFromObj** or **Tk_GetFont**. The return value is cached in *objPtr*, which speeds up future calls to **Tk_GetFontFromObj** with the same *objPtr* and *tkwin*.

Tk_AllocFontFromObj and **Tk_GetFont** maintain a database of all fonts they have allocated. If the same font is requested multiple times (e.g. by different windows or for different purposes), then a single Tk_Font will be shared for all uses. The underlying resources will be freed automatically when no-one is using the font anymore.

The procedure **Tk_NameOfFont** is roughly the inverse of **Tk_GetFont**. Given a *tkfont* that was created by **Tk_GetFont** (or **Tk_AllocFontFromObj**), the return value is the *string* argument that was passed to **Tk_GetFont** to create the font. The string returned by **Tk_NameOfFont** is only guaranteed to persist until the *tkfont* is deleted. The caller must not modify this string.

When a font is no longer needed, **Tk_FreeFontFromObj** or **Tk_FreeFont** should be called to release it. For **Tk_FreeFontFromObj** the font to release is specified with the same information used to create it; for **Tk_FreeFont** the font to release is specified with its Tk_Font token. There should be exactly one call to **Tk_FreeFontFromObj** or **Tk_FreeFont** for each call to **Tk_AllocFontFromObj** or **Tk_GetFont**.

Keywords: GetFont, font

Copyright (c) 1990-1992 The Regents of the University of California.
Copyright (c) 1994-1998 Sun Microsystems, Inc.

GetGC

Name

Tk_GetGC, Tk_FreeGC - maintain database of read-only graphics contexts

Synopsis

#include <tk.h>

GC
Tk_GetGC(*tkwin*, *valueMask*, *valuePtr*)

Tk_FreeGC(*display*, *gc*)

Arguments

Tk_Window *tkwin* (in)
Token for window in which the graphics context will be used.

unsigned long *valueMask* (in)
Mask of bits (such as **GCForeground** or **GCStipple**) indicating which fields of **valuePtr* are valid.

XGCValues **valuePtr* (in)
Pointer to structure describing the desired values for the graphics context.

Display **display* (in)
Display for which *gc* was allocated.

GC *gc* (in)
X identifier for graphics context that is no longer needed. Must have been allocated by **Tk_GetGC**.

Description

Tk_GetGC and **Tk_FreeGC** manage a collection of graphics contexts being used by an application. The procedures allow graphics contexts to be shared, thereby avoiding the server overhead that would be incurred if a separate GC were created for each use. **Tk_GetGC** takes arguments describing the desired graphics context and returns an X identifier for a GC that fits the description. The graphics context that is returned will have default values in all of the fields not specified explicitly by *valueMask* and *valuePtr*.

Tk_GetGC maintains a database of all the graphics contexts it has created. Whenever possible, a call to **Tk_GetGC** will return an existing graphics context rather than creating a new one. This approach can substantially reduce server overhead, so **Tk_GetGC** should generally be used in preference to the Xlib procedure **XCreateGC**, which creates a new graphics context on each call.

Since the return values of **Tk_GetGC** are shared, callers should never modify the graphics contexts returned by **Tk_GetGC**. If a graphics context must be modified dynamically, then it should be created by calling **XCreateGC** instead of **Tk_GetGC**.

When a graphics context is no longer needed, **Tk_FreeGC** should be called to release it. There should be exactly one call to **Tk_FreeGC** for each call to **Tk_GetGC**. When a graphics context is no longer in use anywhere (i.e. it has been freed as many times as it has been gotten) **Tk_FreeGC** will release it to the X server and delete it from the database.

Keywords: GetGC, graphics context

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

GetHINSTANCE

Name

Tk_GetHINSTANCE - retrieve the global application instance handle

Synopsis

#include <tkPlatDecls.h>

HINSTANCE
Tk_GetHINSTANCE()

Description

Tk_GetHINSTANCE returns the Windows application instance handle for the Tk application. This function is only available on Windows platforms.

Keywords: GetHINSTANCE, identifier, instance

Copyright (c) 1998-2000 by Scriptics Corporation.
All rights reserved.

GetHWND

Name

Tk_GetHWND, Tk_AttachHWND - manage interactions between the Windows handle and an X window

Synopsis

#include <tkPlatDecls.h>

HWND
Tk_GetHWND(*window*)

Window
Tk_AttachHWND(*tkwin*, *hwnd*)

Arguments

Window *window* (in)
X token for window.

Tk_Window *tkwin* (in)
Tk window for window.

HWND *hwnd* (in)
Windows HWND for window.

Description

Tk_GetHWND returns the Windows HWND identifier for X Windows window given by *window*.

Tk_AttachHWND binds the Windows HWND identifier to the specified Tk_Window given by *tkwin*. It returns an X Windows window that encapsulates the HWND.

Keywords: GetHWND, identifier, window

Copyright (c) 1998-2000 by Scriptics Corporation.
All rights reserved.

GetImage

Name

Tk_GetImage, Tk_RedrawImage, Tk_SizeOfImage, Tk_FreeImage - use an image in a widget

Synopsis

#include <tk.h>

Tk_Image
Tk_GetImage(*interp*, *tkwin*, *name*, *changeProc*, *clientData*)

Tk_RedrawImage(*image*, *imageX*, *imageY*, *width*, *height*, *drawable*, *drawableX*, *drawableY*)

Tk_SizeOfImage(*image*, *widthPtr*, *heightPtr*)

Tk_FreeImage(*image*)

Arguments

Tcl_Interp **interp* (in)
Place to leave error message.

Tk_Window *tkwin* (in)
Window in which image will be used.

const char **name* (in)
Name of image.

Tk_ImageChangedProc **changeProc* (in)

Procedure for Tk to invoke whenever image content or size changes.

ClientData *clientData* (in)
One-word value for Tk to pass to *changeProc*.

Tk_Image *image* (in)
Token for image instance; must have been returned by a previous call to **Tk_GetImage**.

int *imageX* (in)
X-coordinate of upper-left corner of region of image to redisplay (measured in pixels from the image's upper-left corner).

int *imageY* (in)
Y-coordinate of upper-left corner of region of image to redisplay (measured in pixels from the image's upper-left corner).

int *width*
Width of region of image to redisplay.

int *height*
Height of region of image to redisplay.

Drawable *drawable* (in)
Where to display image. Must either be window specified to **Tk_GetImage** or a pixmap compatible with that window.

int *drawableX* (in)
Where to display image in *drawable*: this is the x-coordinate in *drawable* where x-coordinate *imageX* of the image should be displayed.

int *drawableY* (in)
Where to display image in *drawable*: this is the y-coordinate in *drawable* where y-coordinate *imageY* of the image should be displayed.

int *widthPtr* (out)
Store width of *image* (in pixels) here.

int *heightPtr* (out)
Store height of *image* (in pixels) here.

Description

These procedures are invoked by widgets that wish to display images. **Tk_GetImage** is invoked by a widget when it first decides to display an image. *name* gives the name of the desired image and *tkwin* identifies the window where the image will be displayed. **Tk_GetImage** looks up the image in the table of existing images and returns a token for a new instance of the image. If the image does not exist then **Tk_GetImage** returns NULL and leaves an error message in interpreter *interp*'s result.

When a widget wishes to actually display an image it must call **Tk_RedrawImage**, identifying the image (*image*), a region within the image to redisplay (*imageX*, *imageY*, *width*, and *height*), and a place to display the image (*drawable*, *drawableX*, and *drawableY*). Tk will then invoke the appropriate image manager, which will display the requested portion of the image before returning.

A widget can find out the dimensions of an image by calling **Tk_SizeOfImage**: the width and height will be stored in the locations given by *widthPtr* and *heightPtr*, respectively.

When a widget is finished with an image (e.g., the widget is being deleted or it is going to use a

different image instead of the current one), it must call **Tk_FreeImage** to release the image instance. The widget should never again use the image token after passing it to **Tk_FreeImage**. There must be exactly one call to **Tk_FreeImage** for each call to **Tk_GetImage**.

If the contents or size of an image changes, then any widgets using the image will need to find out about the changes so that they can redisplay themselves. The *changeProc* and *clientData* arguments to **Tk_GetImage** are used for this purpose. *changeProc* will be called by Tk whenever a change occurs in the image; it must match the following prototype:

```
typedef void Tk_ImageChangedProc(
        ClientData clientData,
        int x,
        int y,
        int width,
        int height,
        int imageWidth,
        int imageHeight);
```

The *clientData* argument to *changeProc* is the same as the *clientData* argument to **Tk_GetImage**. It is usually a pointer to the widget record for the widget or some other data structure managed by the widget. The arguments *x*, *y*, *width*, and *height* identify a region within the image that must be redisplayed; they are specified in pixels measured from the upper-left corner of the image. The arguments *imageWidth* and *imageHeight* give the image's (new) size.

Keywords: GetImage, images, redisplay

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

GetJoinStl

Name

Tk_GetJoinStyle, Tk_NameOfJoinStyle - translate between strings and join styles

Synopsis

#include <tk.h>

int
Tk_GetJoinStyle(*interp*, *string*, *joinPtr*)

const char *
Tk_NameOfJoinStyle(*join*)

Arguments

Tcl_Interp ****interp* **(in)**
Interpreter to use for error reporting.

const char ****string* **(in)**
String containing name of join style - one of "**bevel**", "**miter**", or "**round**" - or a unique abbreviation of one.

int ****joinPtr* **(out)**
Pointer to location in which to store X join style corresponding to *string*.

int *join* (in)
Join style: one of **JoinBevel**, **JoinMiter**, **JoinRound**.

Description

Tk_GetJoinStyle places in **joinPtr* the X join style corresponding to *string*, which will be one of **JoinBevel**, **JoinMiter**, or **JoinRound**. Join styles are typically used in X graphics contexts to indicate how adjacent line segments should be joined together. See the X documentation for information on what each style implies.

Under normal circumstances the return value is **TCL_OK** and *interp* is unused. If *string* does not contain a valid join style or an abbreviation of one of these names, then an error message is stored in interpreter *interp*'s result, **TCL_ERROR** is returned, and **joinPtr* is unmodified.

Tk_NameOfJoinStyle is the logical inverse of **Tk_GetJoinStyle**. Given a join style such as **JoinBevel** it returns a statically-allocated string corresponding to *join*. If *join* is not a legal join style, then "unknown join style" is returned.

Keywords: GetJoinStl, bevel, join style, miter, round

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

GetJustify

Name

Tk_GetJustifyFromObj, Tk_GetJustify, Tk_NameOfJustify - translate between strings and justification styles

Synopsis

#include <tk.h>

int
Tk_GetJustifyFromObj(*interp*, *objPtr*, *justifyPtr*)

int
Tk_GetJustify(*interp*, *string*, *justifyPtr*)

const char *
Tk_NameOfJustify(*justify*)

Arguments

Tcl_Interp **interp* (in)
Interpreter to use for error reporting, or NULL.

Tcl_Obj **objPtr* (in/out)
String value contains name of justification style - one of "**left**", "**right**", or "**center**" - or a unique abbreviation of one. The internal rep will be modified to cache corresponding justify value.

const char **string* (in)
Same as *objPtr* except description of justification style is passed as a string.

int **justifyPtr* (out)
Pointer to location in which to store justify value corresponding to *objPtr* or *string*.

Tk_Justify *justify* (in)
Justification style (one of the values listed below).

Description

Tk_GetJustifyFromObj places in **justifyPtr* the justify value corresponding to *objPtr*'s value. This value will be one of the following:

TK_JUSTIFY_LEFT
Means that the text on each line should start at the left edge of the line; as a result, the right edges of lines may be ragged.

TK_JUSTIFY_RIGHT
Means that the text on each line should end at the right edge of the line; as a result, the left edges of lines may be ragged.

TK_JUSTIFY_CENTER
Means that the text on each line should be centered; as a result, both the left and right edges of lines may be ragged. Under normal circumstances the return value is **TCL_OK** and *interp* is unused. If *objPtr* does not contain a valid justification style or an abbreviation of one of these names, **TCL_ERROR** is returned, **justifyPtr* is unmodified, and an error message is stored in *interp*'s result if *interp* is not NULL. **Tk_GetJustifyFromObj** caches information about the return value in *objPtr*, which speeds up future calls to **Tk_GetJustifyFromObj** with the same *objPtr*. **Tk_GetJustify** is identical to **Tk_GetJustifyFromObj** except that the description of the justification is specified with a string instead of an object. This prevents **Tk_GetJustify** from caching the return value, so **Tk_GetJustify** is less efficient than **Tk_GetJustifyFromObj** . **Tk_NameOfJustify** is the logical inverse of **Tk_GetJustify**. Given a justify value it returns a statically-allocated string corresponding to *justify*. If *justify* is not a legal justify value, then "unknown justification style" is returned.

Keywords: GetJustify, center, fill, justification, string

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1998 Sun Microsystems, Inc.

GetOption

Name

Tk_GetOption - retrieve an option from the option database

Synopsis

#include <tk.h>

Tk_Uid
Tk_GetOption(*tkwin*, *name*, *class*)

Arguments

Tk_Window *tkwin* (in)
Token for window.

const char **name* (in)
Name of desired option.

const char **class* (in)

Class of desired option. Null means there is no class for this option; do lookup based on name only.

Description

This procedure is invoked to retrieve an option from the database associated with *tkwin*'s main window. If there is an option for *tkwin* that matches the given *name* or *class*, then it is returned in the form of a Tk_Uid. If multiple options match *name* and *class*, then the highest-priority one is returned. If no option matches, then NULL is returned.

Tk_GetOption caches options related to *tkwin* so that successive calls for the same *tkwin* will execute much more quickly than successive calls for different windows.

Keywords: GetOption, class, name, option, retrieve

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

GetPixels

Name

Tk_GetPixelsFromObj, Tk_GetPixels, Tk_GetMMFromObj, Tk_GetScreenMM - translate between strings and screen units

Synopsis

#include <tk.h>

int
Tk_GetPixelsFromObj(*interp*, *tkwin*, *objPtr*, *intPtr*)

int
Tk_GetPixels(*interp*, *tkwin*, *string*, *intPtr*)

int
Tk_GetMMFromObj(*interp*, *tkwin*, *objPtr*, *doublePtr*)

int
Tk_GetScreenMM(*interp*, *tkwin*, *string*, *doublePtr*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter to use for error reporting.

Tk_Window _tkwin_ (in)
Window whose screen geometry determines the conversion between absolute units and pixels.

Tcl_Obj *_objPtr_ (in/out)
String value specifies a distance on the screen; internal rep will be modified to cache converted distance.

const char *_string_ (in)
Same as *objPtr* except specification of distance is passed as a string.

int *_intPtr_ (out)
Pointer to location in which to store converted distance in pixels.

double *_doublePtr_ (out)
Pointer to location in which to store converted distance in millimeters.

Description

These procedures take as argument a specification of distance on the screen (*objPtr* or *string*) and compute the corresponding distance either in integer pixels or floating-point millimeters. In either case, *objPtr* or *string* specifies a screen distance as a floating-point number followed by one of the following characters that indicates units:

<none>
The number specifies a distance in pixels.

c
The number specifies a distance in centimeters on the screen.

i
The number specifies a distance in inches on the screen.

m
The number specifies a distance in millimeters on the screen.

p
The number specifies a distance in printer's points (1/72 inch) on the screen.
Tk_GetPixelsFromObj converts the value of *objPtr* to the nearest even number of pixels and stores that value at *_intPtr_. It returns **TCL_OK** under normal circumstances. If an error occurs (e.g. *objPtr* contains a number followed by a character that is not one of the ones above) then **TCL_ERROR** is returned and an error message is left in *interp*'s result if *interp* is not NULL. **Tk_GetPixelsFromObj** caches information about the return value in *objPtr*, which speeds up future calls to **Tk_GetPixelsFromObj** with the same *objPtr*. **Tk_GetPixels** is identical to **Tk_GetPixelsFromObj** except that the screen distance is specified with a string instead of an object. This prevents **Tk_GetPixels** from caching the return value, so **Tk_GetPixels** is less efficient than **Tk_GetPixelsFromObj**. **Tk_GetMMFromObj** and **Tk_GetScreenMM** are similar to **Tk_GetPixelsFromObj** and **Tk_GetPixels** (respectively) except that they convert the screen distance to millimeters and store a double-precision floating-point result at *_doublePtr_.

Keywords: GetPixels, centimeters, convert, inches, millimeters, pixels, points, screen units

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1998 Sun Microsystems, Inc.

GetPixmap

Name

Tk_GetPixmap, Tk_FreePixmap - allocate and free pixmaps

Synopsis

#include <tk.h>

Pixmap
Tk_GetPixmap(*display*, *d*, *width*, *height*, *depth*)

Tk_FreePixmap(*display*, *pixmap*)

Arguments

Display *_display_ (in)
X display for the pixmap.

Drawable _d_ (in)
Pixmap or window where the new pixmap will be used for drawing.

int _width_ (in)
Width of pixmap.

int _height_ (in)
Height of pixmap.

int _depth_ (in)
Number of bits per pixel in pixmap.

Pixmap _pixmap_ (in)
Pixmap to destroy.

Description

These procedures are identical to the Xlib procedures **XCreatePixmap** and **XFreePixmap**, except that they have extra code to manage X resource identifiers so that identifiers for deleted pixmaps can be reused in the future. It is important for Tk applications to use these procedures rather than **XCreatePixmap** and **XFreePixmap**; otherwise long-running applications may run out of resource identifiers.

Tk_GetPixmap creates a pixmap suitable for drawing in *d*, with dimensions given by *width*, *height*, and *depth*, and returns its identifier. **Tk_FreePixmap** destroys the pixmap given by *pixmap* and makes its resource identifier available for reuse.

Keywords: GetPixmap, pixmap, resource identifier

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

GetRelief

Name

Tk_GetReliefFromObj, Tk_GetRelief, Tk_NameOfRelief - translate between strings and relief values

Synopsis

#include <tk.h>

int
Tk_GetReliefFromObj(*interp*, *objPtr*, *reliefPtr*)

int
Tk_GetRelief(*interp*, *name*, *reliefPtr*)

const char *
Tk_NameOfRelief(*relief*)

Arguments

Tcl_Interp *_interp_ (in)

Interpreter to use for error reporting.

Tcl_Obj *_objPtr_ (in/out)
String value contains name of relief, one of "**flat**", "**groove**", "**raised**", "**ridge**", "**solid**", or "**sunken**" (or any unique abbreviation thereof on input); the internal rep will be modified to cache corresponding relief value.

const char *_name_ (in)
Same as *objPtr* except description of relief is passed as a string.

int *_reliefPtr_ (out)
Pointer to location in which to store relief value corresponding to *objPtr* or *name*.

int _relief_ (in)
Relief value (one of **TK_RELIEF_FLAT**, **TK_RELIEF_RAISED**, **TK_RELIEF_SUNKEN**, **TK_RELIEF_GROOVE**, **TK_RELIEF_SOLID**, or **TK_RELIEF_RIDGE**).

Description

Tk_GetReliefFromObj places in **reliefPtr* the relief value corresponding to the value of *objPtr*. This value will be one of **TK_RELIEF_FLAT**, **TK_RELIEF_RAISED**, **TK_RELIEF_SUNKEN**, **TK_RELIEF_GROOVE**, **TK_RELIEF_SOLID**, or **TK_RELIEF_RIDGE**. Under normal circumstances the return value is **TCL_OK** and *interp* is unused. If *objPtr* does not contain one of the valid relief names or an abbreviation of one of them, then **TCL_ERROR** is returned, **reliefPtr* is unmodified, and an error message is stored in *interp*'s result if *interp* is not NULL. **Tk_GetReliefFromObj** caches information about the return value in *objPtr*, which speeds up future calls to **Tk_GetReliefFromObj** with the same *objPtr*.

Tk_GetRelief is identical to **Tk_GetReliefFromObj** except that the description of the relief is specified with a string instead of an object. This prevents **Tk_GetRelief** from caching the return value, so **Tk_GetRelief** is less efficient than **Tk_GetReliefFromObj**.

Tk_NameOfRelief is the logical inverse of **Tk_GetRelief**. Given a relief value it returns the corresponding string (**flat**, **raised**, **sunken**, **groove**, **solid**, or **ridge**). If *relief* is not a legal relief value, then "unknown relief" is returned.

Keywords: GetRelief, name, relief, string

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1998 Sun Microsystems, Inc.

GetRootCrd

Name

Tk_GetRootCoords - Compute root-window coordinates of window

Synopsis

#include <tk.h>

Tk_GetRootCoords(*tkwin*, *xPtr*, *yPtr*)

Arguments

Tk_Window _tkwin_ (in)
Token for window.

int *_xPtr_ (out)

Pointer to location in which to store root-window x-coordinate corresponding to left edge of *tkwin*'s border.

int *yPtr (out)
Pointer to location in which to store root-window y-coordinate corresponding to top edge of *tkwin*'s border.

Description

This procedure scans through the structural information maintained by Tk to compute the root-window coordinates corresponding to the upper-left corner of *tkwin*'s border. If *tkwin* has no border, then **Tk_GetRootCoords** returns the root-window coordinates corresponding to location (0,0) in *tkwin*. **Tk_GetRootCoords** is relatively efficient, since it does not have to communicate with the X server.

Keywords: GetRootCrd, coordinates, root window

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

GetScroll

Name

Tk_GetScrollInfoObj, Tk_GetScrollInfo - parse arguments for scrolling commands

Synopsis

#include <tk.h>

int
Tk_GetScrollInfoObj(*interp*, *objc*, *objv*, *fractionPtr*, *stepsPtr*)

int
Tk_GetScrollInfo(*interp*, *argc*, *argv*, *fractionPtr*, *stepsPtr*)

Arguments

Tcl_Interp *interp (in)
Interpreter to use for error reporting.

int objc (in)
Number of Tcl_Obj's in *objv* array.

Tcl_Obj *const * objv (in)
Argument objects. These represent the entire widget command, of which the first word is typically the widget name and the second word is typically **xview** or **yview**.

int argc (in)
Number of strings in *argv* array.

const char ** argv (in)
Argument strings. These represent the entire widget command, of which the first word is typically the widget name and the second word is typically **xview** or **yview**.

double *fractionPtr (out)
Filled in with fraction from **moveto** option, if any.

int *stepsPtr (out)
Filled in with line or page count from **scroll** option, if any. The value may be negative.

Description

Tk_GetScrollInfoObj parses the arguments expected by widget scrolling commands such as **xview** and **yview**. It receives the entire list of words that make up a widget command and parses the words starting with *objv*[2]. The words starting with *objv*[2] must have one of the following forms:

```
moveto fraction
scroll number units
scroll number pages
```

Any of the **moveto**, **scroll**, **units**, and **pages** keywords may be abbreviated. If *objv* has the **moveto** form, **TK_SCROLL_MOVETO** is returned as result and **fractionPtr* is filled in with the *fraction* argument to the command, which must be a proper real value. If *objv* has the **scroll** form, **TK_SCROLL_UNITS** or **TK_SCROLL_PAGES** is returned and **stepsPtr* is filled in with the *number* value, which must be a proper integer. If an error occurs in parsing the arguments, **TK_SCROLL_ERROR** is returned and an error message is left in interpreter *interp*'s result.

Tk_GetScrollInfo is identical in function to **Tk_GetScrollInfoObj**. However, **Tk_GetScrollInfo** accepts string arguments, making it more appropriate for use with legacy widgets.

Keywords: GetScroll, parse, scrollbar, scrolling command, xview, yview

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

GetSelect

Name

Tk_GetSelection - retrieve the contents of a selection

Synopsis

#include <tk.h>

int
Tk_GetSelection(*interp*, *tkwin*, *selection*, *target*, *proc*, *clientData*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter to use for reporting errors.

Tk_Window _tkwin_ (in)
Window on whose behalf to retrieve the selection (determines display from which to retrieve).

Atom _selection_ (in)
The name of the selection to be retrieved.

Atom _target_ (in)
Form in which to retrieve selection.

Tk_GetSelProc *_proc_ (in)
Procedure to invoke to process pieces of the selection as they are retrieved.

ClientData _clientData_ (in)
Arbitrary one-word value to pass to *proc*.

Description

Tk_GetSelection retrieves the selection specified by the atom *selection* in the format specified by *target*. The selection may actually be retrieved in several pieces; as each piece is retrieved, *proc* is called to process the piece. *Proc* should have arguments and result that match the type **Tk_GetSelProc**:

```
typedef int Tk_GetSelProc(
        ClientData clientData,
        Tcl_Interp *interp,
        char *portion);
```

The *clientData* and *interp* parameters to *proc* will be copies of the corresponding arguments to **Tk_GetSelection**. *Portion* will be a pointer to a string containing part or all of the selection. For large selections, *proc* will be called several times with successive portions of the selection. The X Inter-Client Communication Conventions Manual allows a selection to be returned in formats other than strings, e.g. as an array of atoms or integers. If this happens, Tk converts the selection back into a string before calling *proc*. If a selection is returned as an array of atoms, Tk converts it to a string containing the atom names separated by white space. For any other format besides string, Tk converts a selection to a string containing hexadecimal values separated by white space.

Tk_GetSelection returns to its caller when the selection has been completely retrieved and processed by *proc*, or when a fatal error has occurred (e.g. the selection owner did not respond promptly). **Tk_GetSelection** normally returns **TCL_OK**; if an error occurs, it returns **TCL_ERROR** and leaves an error message in interpreter *interp*'s result. *Proc* should also return either **TCL_OK** or **TCL_ERROR**. If *proc* encounters an error in dealing with the selection, it should leave an error message in the interpreter result and return **TCL_ERROR**; this will abort the selection retrieval.

Keywords: GetSelect, format, get, selection retrieval

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

GetUid

Name

Tk_GetUid, Tk_Uid - convert from string to unique identifier

Synopsis

#include <tk.h>

Tk_Uid
Tk_GetUid(*string*)

Arguments

char ****string* **(in)**
String for which the corresponding unique identifier is desired.

Description

Tk_GetUid returns the unique identifier corresponding to *string*. Unique identifiers are similar to atoms in Lisp, and are used in Tk to speed up comparisons and searches. A unique identifier (type Tk_Uid) is a string pointer and may be used anywhere that a variable of type "char *" could be used. However, there is guaranteed to be exactly one unique identifier for any given string value. If **Tk_GetUid** is called twice, once with string *a* and once with string *b*, and if *a* and *b* have the same string value (strcmp(a, b) == 0), then **Tk_GetUid** will return exactly the same Tk_Uid value for each call (Tk_GetUid(a) == Tk_GetUid(b)). This means that variables of type Tk_Uid may be compared directly (x == y) without having to call **strcmp**. In addition, the return value from **Tk_GetUid** will have the same string value as its argument (strcmp(Tk_GetUid(a), a) == 0).

Keywords: GetUid, atom, unique identifier

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

GetVRoot

Name

Tk_GetVRootGeometry - Get location and size of virtual root for window

Synopsis

#include <tk.h>

Tk_GetVRootGeometry(*tkwin*, *xPtr*, *yPtr*, *widthPtr*, *heightPtr*)

Arguments

Tk_Window *tkwin* (in)
Token for window whose virtual root is to be queried.

int *xPtr* (out)
Points to word in which to store x-offset of virtual root.

int *yPtr* (out)
Points to word in which to store y-offset of virtual root.

int *widthPtr* (out)
Points to word in which to store width of virtual root.

int *heightPtr* (out)
Points to word in which to store height of virtual root.

Description

Tk_GetVRootGeometry returns geometry information about the virtual root window associated with *tkwin*. The "associated" virtual root is the one in which *tkwin*'s nearest top-level ancestor (or *tkwin* itself if it is a top-level window) has been reparented by the window manager. This window is identified by a **__SWM_ROOT** or **__WM_ROOT** property placed on the top-level window by the window manager. If *tkwin* is not associated with a virtual root (e.g. because the window manager does not use virtual roots) then **xPtr* and **yPtr* will be set to 0 and **widthPtr* and **heightPtr* will be set to the dimensions of the screen containing *tkwin*.

Keywords: GetVRoot, geometry, height, location, virtual root, width, window manager

Copyright (c) 1990 The Regents of the University of California.

Copyright (c) 1994-1996 Sun Microsystems, Inc.

GetVisual

Name

Tk_GetVisual - translate from string to visual

Synopsis

#include <tk.h>

Visual *
Tk_GetVisual(*interp*, *tkwin*, *string*, *depthPtr*, *colormapPtr*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter to use for error reporting.

Tk_Window _tkwin_ (in)
Token for window in which the visual will be used.

const char *_string_ (in)
String that identifies the desired visual. See below for valid formats.

int *_depthPtr_ (out)
Depth of returned visual gets stored here.

Colormap *_colormapPtr_ (out)
If non-NULL then a suitable colormap for visual is found and its identifier is stored here.

Description

Tk_GetVisual takes a string description of a visual and finds a suitable X Visual for use in *tkwin*, if there is one. It returns a pointer to the X Visual structure for the visual and stores the number of bits per pixel for it at **depthPtr*. If *string* is unrecognizable or if no suitable visual could be found, then NULL is returned and **Tk_GetVisual** leaves an error message in interpreter *interp*'s result. If *colormap* is non-NULL then **Tk_GetVisual** also locates an appropriate colormap for use with the result visual and stores its X identifier at **colormapPtr*.

The *string* argument specifies the desired visual in one of the following ways:

class depth
The string consists of a class name followed by an integer depth, with any amount of white space (including none) in between. *class* selects what sort of visual is desired and must be one of **directcolor**, **grayscale**, **greyscale**, **pseudocolor**, **staticcolor**, **staticgray**, **staticgrey**, or **truecolor**, or a unique abbreviation. *depth* specifies how many bits per pixel are needed for the visual. If possible, **Tk_GetVisual** will return a visual with this depth; if there is no visual of the desired depth then **Tk_GetVisual** looks first for a visual with greater depth, then one with less depth.

default
Use the default visual for *tkwin*'s screen.

pathName
Use the visual for the window given by *pathName*. *pathName* must be the name of a window on the same screen as *tkwin*.

number
Use the visual whose X identifier is *number*.

best ?*depth*?
Choose the "best possible" visual, using the following rules, in decreasing order of priority:

(a)
a visual that has exactly the desired depth is best, followed by a visual with greater depth than requested (but as little extra as possible), followed by a visual with less depth than requested (but as great a depth as possible);

(b)
if no *depth* is specified, then the deepest available visual is chosen;

(c)
pseudocolor is better than **truecolor** or **directcolor**, which are better than **staticcolor**, which is better than **staticgray** or **grayscale**;

(d)
the default visual for the screen is better than any other visual.

Credits
The idea for **Tk_GetVisual**, and the first implementation, came from Paul Mackerras.

Keywords: GetVisual, colormap, screen, visual

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

Grab

Name
Tk_Grab, Tk_Ungrab - manipulate grab state in an application

Synopsis
#include <tk.h>

int
Tk_Grab(*interp*, *tkwin*, *grabGlobal*)

void
Tk_Ungrab(*tkwin*)

Arguments
Tcl_Interp *_interp_ (in)
Interpreter to use for error reporting

Tk_Window _tkwin_ (in)
Window on whose behalf the pointer is to be grabbed or released

int _grabGlobal_ (in)
Boolean indicating whether the grab is global or application local

Description
These functions are used to set or release a global or application local grab. When a grab is set on a particular window in a Tk application, mouse and keyboard events can only be received by

that window and its descendants. Mouse and keyboard events for windows outside the tree rooted at *tkwin* will be redirected to *tkwin*. If the grab is global, then all mouse and keyboard events for windows outside the tree rooted at *tkwin* (even those intended for windows in other applications) will be redirected to *tkwin*. If the grab is application local, only mouse and keyboard events intended for a windows within the same application (but outside the tree rooted at *tkwin*) will be redirected.

Tk_Grab sets a grab on a particular window. *Tkwin* specifies the window on whose behalf the pointer is to be grabbed. *GrabGlobal* indicates whether the grab should be global or application local; if it is non-zero, it means the grab should be global. Normally, **Tk_Grab** returns **TCL_OK**; if an error occurs and the grab cannot be set, **TCL_ERROR** is returned and an error message is left if *interp*'s result. Once this call completes successfully, no window outside the tree rooted at *tkwin* will receive pointer- or keyboard-related events until the next call to Tk_Ungrab. If a previous grab was in effect within the application, then it is replaced with a new one.

Tk_Ungrab releases a grab on the mouse pointer and keyboard, if there is one set on the window given by *tkwin*. Once a grab is released, pointer and keyboard events will start being delivered to other windows again.

Keywords: Grab, window

Copyright (c) 1998-2000 by Scriptics Corporation.
All rights reserved.

HWNDToWindow

Name

Tk_HWNDToWindow - Find Tk's window information for a Windows window

Synopsis

#include <tkPlatDecls.h>

Tk_Window
Tk_HWNDToWindow(*hwnd*)

Arguments

HWND *hwnd* (in)
Windows handle for the window.

Description

Given a Windows HWND window identifier, this procedure returns the corresponding Tk_Window handle. If there is no Tk_Window corresponding to *hwnd* then NULL is returned.

Keywords: HWNDToWindow, Windows window id

Copyright (c) 1998-2000 by Scriptics Corporation.
All rights reserved.

HandleEvent

Name

Tk_HandleEvent - invoke event handlers for window system events

Synopsis

#include <tk.h>

Tk_HandleEvent(*eventPtr*)

Arguments

XEvent ****eventPtr* **(in)

Pointer to X event to dispatch to relevant handler(s). It is important that all unused fields of the structure be set to zero.

Description

Tk_HandleEvent is a lower-level procedure that deals with window events. It is called by **Tcl_ServiceEvent** (and indirectly by **Tcl_DoOneEvent**), and in a few other cases within Tk. It makes callbacks to any window event handlers (created by calls to **Tk_CreateEventHandler**) that match *eventPtr* and then returns. In some cases it may be useful for an application to bypass the Tk event queue and call **Tk_HandleEvent** directly instead of calling **Tcl_QueueEvent** followed by **Tcl_ServiceEvent**.

This procedure may be invoked recursively. For example, it is possible to invoke **Tk_HandleEvent** recursively from a handler called by **Tk_HandleEvent**. This sort of operation is useful in some modal situations, such as when a notifier has been popped up and an application wishes to wait for the user to click a button in the notifier before doing anything else.

Keywords: HandleEvent, callback, event, handler, window

Copyright (c) 1990-1992 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

IdToWindow

Name

Tk_IdToWindow - Find Tk's window information for an X window

Synopsis

#include <tk.h>

Tk_Window
Tk_IdToWindow(*display*, *window*)

Arguments

Display ****display* **(in)

X display containing the window.

Window *window* (in)

X id for window.

Description

Given an X window identifier and the X display it corresponds to, this procedure returns the corresponding Tk_Window handle. If there is no Tk_Window corresponding to *window* then NULL is returned.

Keywords: IdToWindow, X window id

Copyright (c) 1995-1996 Sun Microsystems, Inc.

ImgChanged

Name

Tk_ImageChanged - notify widgets that image needs to be redrawn

Synopsis

#include <tk.h>

Tk_ImageChanged(*model*, *x*, *y*, *width*, *height*, *imageWidth*, *imageHeight*)

Arguments

Tk_ImageMaster *model* (in)
Token for image, which was passed to image's *createProc* when the image was created.

int *x* (in)
X-coordinate of upper-left corner of region that needs redisplay (measured from upper-left corner of image).

int *y* (in)
Y-coordinate of upper-left corner of region that needs redisplay (measured from upper-left corner of image).

int *width* (in)
Width of region that needs to be redrawn, in pixels.

int *height* (in)
Height of region that needs to be redrawn, in pixels.

int *imageWidth* (in)
Current width of image, in pixels.

int *imageHeight* (in)
Current height of image, in pixels.

Description

An image manager calls **Tk_ImageChanged** for an image whenever anything happens that requires the image to be redrawn. As a result of calling **Tk_ImageChanged**, any widgets using the image are notified so that they can redisplay themselves appropriately. The *model* argument identifies the image, and *x*, *y*, *width*, and *height* specify a rectangular region within the image that needs to be redrawn. *imageWidth* and *imageHeight* specify the image's (new) size.

Tk_ImageModel is synonym for *Tk_ImageMaster*

An image manager should call **Tk_ImageChanged** during its *createProc* to specify the image's initial size and to force redisplay if there are existing instances for the image. If any of the pixel values in the image should change later on, **Tk_ImageChanged** should be called again with *x*, *y* , *width*, and *height* values that cover all the pixels that changed. If the size of the image should change, then **Tk_ImageChanged** must be called to indicate the new size, even if no pixels need to be redisplayed.

Keywords: ImgChanged, images, redisplay, image size changes

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

Inactive

Name

Tk_GetUserInactiveTime, Tk_ResetUserInactiveTime - discover user inactivity time

Synopsis

#include <tk.h>

long
Tk_GetUserInactiveTime(*display*)

Tk_ResetUserInactiveTime(*display*)

Arguments

Display ****display* **(in)
The display on which the user inactivity timer is to be queried or reset.

Description

Tk_GetUserInactiveTime returns the number of milliseconds that have passed since the last user interaction (usually via keyboard or mouse) with the respective display. On systems and displays that do not support querying the user inactivity time, **-1** is returned.
Tk_ResetUserInactiveTime resets the user inactivity timer of the given display to zero. On windowing systems that do not support multiple displays *display* can be passed as **NULL**.

Keywords: Inactive, idle

Copyright (c) 1998-2000 by Scriptics Corporation.
All rights reserved.

InternAtom

Name

Tk_InternAtom, Tk_GetAtomName - manage cache of X atoms

Synopsis

#include <tk.h>

Atom
Tk_InternAtom(*tkwin*, *name*)

const char *
Tk_GetAtomName(*tkwin*, *atom*)

Arguments

Tk_Window *tkwin* (in)
Token for window. Used to map atom or name relative to a particular display.

const char ****name* **(in)
String name for which atom is desired.

Atom *atom* (in)
Atom for which corresponding string name is desired.

Description

These procedures are similar to the Xlib procedures **XInternAtom** and **XGetAtomName**. **Tk_InternAtom** returns the atom identifier associated with string given by *name*; the atom identifier is only valid for the display associated with *tkwin*. **Tk_GetAtomName** returns the string associated with *atom* on *tkwin*'s display. The string returned by **Tk_GetAtomName** is in Tk's storage: the caller need not free this space when finished with the string, and the caller should not modify the contents of the returned string. If there is no atom *atom* on *tkwin*'s display, then **Tk_GetAtomName** returns the string "?bad atom?".

Tk caches the information returned by **Tk_InternAtom** and **Tk_GetAtomName** so that future calls for the same information can be serviced from the cache without contacting the server. Thus **Tk_InternAtom** and **Tk_GetAtomName** are generally much faster than their Xlib counterparts, and they should be used in place of the Xlib procedures.

Keywords: InternAtom, atom, cache, display

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

MainLoop

Name

Tk_MainLoop - loop for events until all windows are deleted

Synopsis

#include <tk.h>

Tk_MainLoop()

Description

Tk_MainLoop is a procedure that loops repeatedly calling **Tcl_DoOneEvent**. It returns only when there are no applications left in this process (i.e. no main windows exist anymore). Most windowing applications will call **Tk_MainLoop** after initialization; the main execution of the application will consist entirely of callbacks invoked via **Tcl_DoOneEvent**.

Keywords: MainLoop, application, event, main loop

Copyright (c) 1990-1992 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

MainWin

Name

Tk_MainWindow, Tk_GetNumMainWindows - functions for querying main window information

Synopsis

#include <tk.h>

Tk_Window
Tk_MainWindow(*interp*)

int
Tk_GetNumMainWindows()

Arguments

Tcl_Interp *_interp_ (in/out)
Interpreter associated with the application.

Description

A main window is a special kind of toplevel window used as the outermost window in an application.

If *interp* is associated with a Tk application then **Tk_MainWindow** returns the application's main window. If there is no Tk application associated with *interp* then **Tk_MainWindow** returns NULL and leaves an error message in interpreter *interp*'s result.

Tk_GetNumMainWindows returns a count of the number of main windows currently open in the current thread.

Keywords: MainWin, application, main window

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

MaintGeom

Name

Tk_MaintainGeometry, Tk_UnmaintainGeometry - maintain geometry of one window relative to another

Synopsis

#include <tk.h>

Tk_MaintainGeometry(*window*, *container*, *x*, *y*, *width*, *height*)

Tk_UnmaintainGeometry(*window*, *container*)

Arguments

Tk_Window *_window_ (in)
Window whose geometry is to be controlled.

Tk_Window *_container_ (in)
Window relative to which *window*'s geometry will be controlled.

int *x* (in)
Desired x-coordinate of *window* in *container*, measured in pixels from the inside of *container*'s left border to the outside of *window*'s left border.

int *y* (in)
Desired y-coordinate of *window* in *container*, measured in pixels from the inside of *container*'s top border to the outside of *window*'s top border.

int *width* (in)
Desired width for *window*, in pixels.

int *height* (in)
Desired height for *window*, in pixels.

Description

Tk_MaintainGeometry and **Tk_UnmaintainGeometry** make it easier for geometry managers to deal with windows whose containers are not their parents. Three problems arise if the container for a window is not its parent:

[1]
The x- and y-position of the window must be translated from the coordinate system of the container to that of the parent before positioning the window.

[2]
If the container window, or any of its ancestors up to the window's parent, is moved, then the window must be repositioned within its parent in order to maintain the correct position relative to the container.

[3]
If the container or one of its ancestors is mapped or unmapped, then the window must be mapped or unmapped to correspond. None of these problems is an issue if the parent and container are the same. For example, if the container or one of its ancestors is unmapped, the window is automatically removed by the screen by X. **Tk_MaintainGeometry** deals with these problems for windows whose containers are not their parents, as well as handling the simpler case of windows whose container are their parents. **Tk_MaintainGeometry** is typically called by a window manager once it has decided where a window should be positioned relative to its container. **Tk_MaintainGeometry** translates the coordinates to the coordinate system of *window*'s parent and then moves and resizes the window appropriately. Furthermore, it remembers the desired position and creates event handlers to monitor the container and all of its ancestors up to (but not including) the window's parent. If any of these windows is moved, mapped, or unmapped, the window will be adjusted so that it is mapped only when the container is mapped and its geometry relative to the container remains as specified by *x*, *y*, *width*, and *height*. When a window manager relinquishes control over a window, or if it decides that it does not want the window to appear on the screen under any conditions, it calls **Tk_UnmaintainGeometry**. **Tk_UnmaintainGeometry** unmaps the window and cancels any previous calls to **Tk_MaintainGeometry** for the *container-window* pair, so that the window's geometry and mapped state are no longer maintained automatically. **Tk_UnmaintainGeometry** need not be called by a geometry manager if the window, the container, or any of the container's ancestors is destroyed: Tk will call it automatically. If **Tk_MaintainGeometry** is called repeatedly for the same *container-window* pair, the information from the most recent call supersedes any older information. If **Tk_UnmaintainGeometry** is called for a *container-window* pair that is is not currently managed, the call has no effect.

Keywords: MaintGeom, geometry manager, map, container, parent, position, window, unmap

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

ManageGeom

Name

Tk_ManageGeometry - arrange to handle geometry requests for a window

Synopsis

#include <tk.h>

Tk_ManageGeometry(*tkwin*, *mgrPtr*, *clientData*)

Arguments

Tk_Window *tkwin* (in)
Token for window to be managed.

const Tk_GeomMgr *mgrPtr (in)
Pointer to data structure containing information about the geometry manager, or NULL to indicate that *tkwin*'s geometry should not be managed anymore. The data structure pointed to by *mgrPtr* must be static: Tk keeps a reference to it as long as the window is managed.

ClientData *clientData* (in)
Arbitrary one-word value to pass to geometry manager callbacks.

Description

Tk_ManageGeometry arranges for a particular geometry manager, described by the *mgrPtr* argument, to control the geometry of a particular content window, given by *tkwin*. If *tkwin* was previously managed by some other geometry manager, the previous manager loses control in favor of the new one. If *mgrPtr* is NULL, geometry management is cancelled for *tkwin*.

The structure pointed to by *mgrPtr* contains information about the geometry manager:

```
typedef struct {
    const char *name;
    Tk_GeomRequestProc *requestProc;
    Tk_GeomLostSlaveProc *lostSlaveProc;
} Tk_GeomMgr;
```

The *name* field is the textual name for the geometry manager, such as **pack** or **place**; this value will be returned by the command **winfo manager**.

requestProc is a procedure in the geometry manager that will be invoked whenever **Tk_GeometryRequest** is called by the content window to change its desired geometry. *requestProc* should have arguments and results that match the type **Tk_GeomRequestProc**:

```
typedef void Tk_GeomRequestProc(
        ClientData clientData,
        Tk_Window tkwin);
```

The parameters to *requestProc* will be identical to the corresponding parameters passed to **Tk_ManageGeometry**. *clientData* usually points to a data structure containing application-specific information about how to manage *tkwin*'s geometry.

The *lostSlaveProc* field of *mgrPtr* points to another procedure in the geometry manager. Tk will invoke *lostSlaveProc* if some other manager calls **Tk_ManageGeometry** to claim *tkwin* away from the current geometry manager. *lostSlaveProc* is not invoked if **Tk_ManageGeometry** is called with a NULL value for *mgrPtr* (presumably the current geometry manager has made this call, so it already knows that the window is no longer managed), nor is it called if *mgrPtr* is the same as the window's current geometry manager. *lostSlaveProc* should have arguments and results that match the following prototype:

```
typedef void Tk_GeomLostSlaveProc(
        ClientData clientData,
        Tk_Window tkwin);
```

The parameters to *lostSlaveProc* will be identical to the corresponding parameters passed to **Tk_ManageGeometry**.

Keywords: ManageGeom, callback, geometry, managed, request, unmanaged

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

MapWindow

Name

Tk_MapWindow, Tk_UnmapWindow - map or unmap a window

Synopsis

#include <tk.h>

Tk_Window
Tk_MapWindow(*tkwin*)

Tk_UnmapWindow(*tkwin*)

Arguments

Tk_Window *tkwin* (in)
Token for window.

Description

These procedures may be used to map and unmap windows managed by Tk. **Tk_MapWindow** maps the window given by *tkwin*, and also creates an X window corresponding to *tkwin* if it does not already exist. See the **Tk_CreateWindow** manual entry for information on deferred window creation. **Tk_UnmapWindow** unmaps *tkwin*'s window from the screen.

If *tkwin* is a child window (i.e. **Tk_CreateWindow** was used to create a child window), then event handlers interested in map and unmap events are invoked immediately. If *tkwin* is not an internal window, then the event handlers will be invoked later, after X has seen the request and returned an event for it.

These procedures should be used in place of the X procedures **XMapWindow** and **XUnmapWindow**, since they update Tk's local data structure for *tkwin*. Applications using Tk should not invoke **XMapWindow** and **XUnmapWindow** directly.

Keywords: MapWindow, map, unmap, window

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

MeasureChar

Name

Tk_MeasureChars, Tk_TextWidth, Tk_DrawChars, Tk_UnderlineChars - routines to measure and

display simple single-line strings.

Synopsis

#include <tk.h>

int
Tk_MeasureChars(*tkfont*, *string*, *numBytes*, *maxPixels*, *flags*, *lengthPtr*)

int
Tk_TextWidth(*tkfont*, *string*, *numBytes*)

Tk_DrawChars(*display*, *drawable*, *gc*, *tkfont*, *string*, *numBytes*, *x*, *y*)

Tk_UnderlineChars(*display*, *drawable*, *gc*, *tkfont*, *string*, *x*, *y*, *firstByte*, *lastByte*)

Arguments

Tk_Font *tkfont* (in)
Token for font in which text is to be drawn or measured. Must have been returned by a previous call to **Tk_GetFont**.

const char **string* (in)
Text to be measured or displayed. Need not be null terminated. Any non-printing meta-characters in the string (such as tabs, newlines, and other control characters) will be measured or displayed in a platform-dependent manner.

int *numBytes* (in)
The maximum number of bytes to consider when measuring or drawing *string*. Must be greater than or equal to 0.

int *maxPixels* (in)
If *maxPixels* is >= 0, it specifies the longest permissible line length in pixels. Characters from *string* are processed only until this many pixels have been covered. If *maxPixels* is < 0, then the line length is unbounded and the *flags* argument is ignored.

int *flags* (in)
Various flag bits OR-ed together: **TK_PARTIAL_OK** means include a character as long as any part of it fits in the length given by *maxPixels*; otherwise, a character must fit completely to be considered. **TK_WHOLE_WORDS** means stop on a word boundary, if possible. If **TK_AT_LEAST_ONE** is set, it means return at least one character even if no characters could fit in the length given by *maxPixels*. If **TK_AT_LEAST_ONE** is set and **TK_WHOLE_WORDS** is also set, it means that if not even one word fits on the line, return the first few letters of the word that did fit; if not even one letter of the word fit, then the first letter will still be returned.

int **lengthPtr* (out)
Filled with the number of pixels occupied by the number of characters returned as the result of **Tk_MeasureChars**.

Display **display* (in)
Display on which to draw.

Drawable *drawable* (in)
Window or pixmap in which to draw.

GC *gc* (in)
Graphics context for drawing characters. The font selected into this GC must be the same as the *tkfont*.

int *x, y* (in)
Coordinates at which to place the left edge of the baseline when displaying *string*.

int *firstByte* (in)
The index of the first byte of the first character to underline in the *string*. Underlining begins at the left edge of this character.

int *lastByte* (in)
The index of the first byte of the last character up to which the underline will be drawn. The character specified by *lastByte* will not itself be underlined.

Description

These routines are for measuring and displaying simple single-font, single-line strings. To measure and display single-font, multi-line, justified text, refer to the documentation for **Tk_ComputeTextLayout**. There is no programming interface in the core of Tk that supports multi-font, multi-line text; support for that behavior must be built on top of simpler layers. Note that the interfaces described here are byte-oriented not character-oriented, so index values coming from Tcl scripts need to be converted to byte offsets using the **Tcl_UtfAtIndex** and related routines.

A glyph is the displayable picture of a letter, number, or some other symbol. Not all character codes in a given font have a glyph. Characters such as tabs, newlines/returns, and control characters that have no glyph are measured and displayed by these procedures in a platform-dependent manner; under X, they are replaced with backslashed escape sequences, while under Windows and Macintosh hollow or solid boxes may be substituted. Refer to the documentation for **Tk_ComputeTextLayout** for a programming interface that supports the platform-independent expansion of tab characters into columns and newlines/returns into multi-line text.

Tk_MeasureChars is used both to compute the length of a given string and to compute how many characters from a string fit in a given amount of space. The return value is the number of bytes from *string* that fit in the space specified by *maxPixels* subject to the conditions described by *flags*. If all characters fit, the return value will be *numBytes*. **lengthPtr* is filled with the computed width, in pixels, of the portion of the string that was measured. For example, if the return value is 5, then **lengthPtr* is filled with the distance between the left edge of *string*[0] and the right edge of *string*[4].

Tk_TextWidth is a wrapper function that provides a simpler interface to the **Tk_MeasureChars** function. The return value is how much space in pixels the given *string* needs.

Tk_DrawChars draws the *string* at the given location in the given *drawable*.

Tk_UnderlineChars underlines the given range of characters in the given *string*. It does not draw the characters (which are assumed to have been displayed previously by **Tk_DrawChars**); it just draws the underline. This procedure is used to underline a few characters without having to construct an underlined font. To produce natively underlined text, the appropriate underlined font should be constructed and used.

Keywords: MeasureChar, font, measurement

Copyright (c) 1996 Sun Microsystems, Inc.

MoveToplev

Name

Tk_MoveToplevelWindow - Adjust the position of a top-level window

Synopsis

#include <tk.h>

Tk_MoveToplevelWindow(*tkwin*, *x*, *y*)

Arguments

Tk_Window *tkwin* (in)
Token for top-level window to move.

int *x* (in)
New x-coordinate for the top-left pixel of *tkwin*'s border, or the top-left pixel of the decorative border supplied for *tkwin* by the window manager, if there is one.

int *y* (in)
New y-coordinate for the top-left pixel of *tkwin*'s border, or the top-left pixel of the decorative border supplied for *tkwin* by the window manager, if there is one.

Description

In general, a window should never set its own position; this should be done only by the geometry manger that is responsible for the window. For top-level windows the window manager is effectively the geometry manager; Tk provides interface code between the application and the window manager to convey the application's desires to the geometry manager. The desired size for a top-level window is conveyed using the usual **Tk_GeometryRequest** mechanism. The procedure **Tk_MoveToplevelWindow** may be used by an application to request a particular position for a top-level window; this procedure is similar in function to the **wm geometry** Tcl command except that negative offsets cannot be specified. It is invoked by widgets such as menus that want to appear at a particular place on the screen.

When **Tk_MoveToplevelWindow** is called it does not immediately pass on the new desired location to the window manager; it defers this action until all other outstanding work has been completed, using the **Tcl_DoWhenIdle** mechanism.

Keywords: MoveToplev, position, top-level window, window manager

Copyright (c) 1990-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

Name

Name

Tk_Name, Tk_PathName, Tk_NameToWindow - convert between names and window tokens

Synopsis

#include <tk.h>

Tk_Uid
Tk_Name(*tkwin*)

char *
Tk_PathName(*tkwin*)

Tk_Window
Tk_NameToWindow(*interp*, *pathName*, *tkwin*)

Arguments

Tk_Window *tkwin* (in)
Token for window.

Tcl_Interp *interp* (out)
Interpreter to use for error reporting.

const char *pathName (in)
Character string containing path name of window.

Description

Each window managed by Tk has two names, a short name that identifies a window among children of the same parent, and a path name that identifies the window uniquely among all the windows belonging to the same main window. The path name is used more often in Tk than the short name; many commands, like **bind**, expect path names as arguments.

The **Tk_Name** macro returns a window's short name, which is the same as the *name* argument passed to **Tk_CreateWindow** when the window was created. The value is returned as a Tk_Uid, which may be used just like a string pointer but also has the properties of a unique identifier (see the manual entry for **Tk_GetUid** for details).

The **Tk_PathName** macro returns a hierarchical name for *tkwin*. Path names have a structure similar to file names in Unix but with dots between elements instead of slashes: the main window for an application has the path name "."; its children have names like ".a" and ".b"; their children have names like ".a.aa" and ".b.bb"; and so on. A window is considered to be a child of another window for naming purposes if the second window was named as the first window's *parent* when the first window was created. This is not always the same as the X window hierarchy. For example, a pop-up is created as a child of the root window, but its logical parent will usually be a window within the application.

The procedure **Tk_NameToWindow** returns the token for a window given its path name (the *pathName* argument) and another window belonging to the same main window (*tkwin*). It normally returns a token for the named window, but if no such window exists **Tk_NameToWindow** leaves an error message in interpreter *interp*'s result and returns NULL. The *tkwin* argument to **Tk_NameToWindow** is needed because path names are only unique within a single application hierarchy. If, for example, a single process has opened two main windows, each will have a separate naming hierarchy and the same path name might appear in each of the hierarchies. Normally *tkwin* is the main window of the desired hierarchy, but this need not be the case: any window in the desired hierarchy may be used.

Keywords: Name, path name, token, window

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

NameOfImg

Name

Tk_NameOfImage - Return name of image.

Synopsis

#include <tk.h>

const char *
Tk_NameOfImage(*imageMaster*)

Arguments

Tk_ImageMaster *imageMaster* (in)
Token for image, which was passed to image manager's *createProc* when the image was created.

Description

This procedure is invoked by image managers to find out the name of an image. Given the token for the image, it returns the string name for the image.

Tk_ImageModel is synonym for *Tk_ImageMaster*

Keywords: NameOfImg, image manager, image name

Copyright (c) 1995-1996 Sun Microsystems, Inc.

OwnSelect

Name

Tk_OwnSelection - make a window the owner of the primary selection

Synopsis

#include <tk.h>

Tk_OwnSelection(*tkwin*, *selection*, *proc*, *clientData*)

Arguments

Tk_Window *tkwin* (in)
Window that is to become new selection owner.

Atom *selection* (in)
The name of the selection to be owned, such as XA_PRIMARY.

Tk_LostSelProc **proc* (in)
Procedure to invoke when *tkwin* loses selection ownership later.

ClientData *clientData* (in)
Arbitrary one-word value to pass to *proc*.

Description

Tk_OwnSelection arranges for *tkwin* to become the new owner of the selection specified by the atom *selection*. After this call completes, future requests for the selection will be directed to handlers created for *tkwin* using **Tk_CreateSelHandler**. When *tkwin* eventually loses the selection ownership, *proc* will be invoked so that the window can clean itself up (e.g. by unhighlighting the selection). *Proc* should have arguments and result that match the type **Tk_LostSelProc**:

```
typedef void Tk_LostSelProc(
        ClientData clientData);
```

The *clientData* parameter to *proc* is a copy of the *clientData* argument given to **Tk_OwnSelection**, and is usually a pointer to a data structure containing application-specific information about *tkwin*.

Keywords: OwnSelect, own, selection owner

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

ParseArgv

Name

Tk_ParseArgv - process command-line options

Synopsis

#include <tk.h>

int
Tk_ParseArgv(*interp*, *tkwin*, *argcPtr*, *argv*, *argTable*, *flags*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter to use for returning error messages.

Tk_Window _tkwin_ (in)
Window to use when arguments specify Tk options. If NULL, then no Tk options will be processed.

int _argcPtr_ (in/out)
Pointer to number of arguments in argv; gets modified to hold number of unprocessed arguments that remain after the call.

const char **_argv_ (in/out)
Command line arguments passed to main program. Modified to hold unprocessed arguments that remain after the call.

Tk_ArgvInfo *_argTable_ (in)
Array of argument descriptors, terminated by element with type **TK_ARGV_END**.

int _flags_ (in)
If non-zero, then it specifies one or more flags that control the parsing of arguments. Different flags may be OR'ed together. The flags currently defined are **TK_ARGV_DONT_SKIP_FIRST_ARG**, **TK_ARGV_NO_ABBREV**, **TK_ARGV_NO_LEFTOVERS**, and **TK_ARGV_NO_DEFAULTS**.

Description

Tk_ParseArgv processes an array of command-line arguments according to a table describing the kinds of arguments that are expected. Each of the arguments in *argv* is processed in turn: if it matches one of the entries in *argTable*, the argument is processed according to that entry and discarded. The arguments that do not match anything in *argTable* are copied down to the beginning of *argv* (retaining their original order) and returned to the caller. At the end of the call

Tk_ParseArgv sets **argcPtr* to hold the number of arguments that are left in *argv*, and *argv[*argcPtr]* will hold the value NULL. Normally, **Tk_ParseArgv** assumes that *argv[0]* is a command name, so it is treated like an argument that does not match *argTable* and returned to the caller; however, if the **TK_ARGV_DONT_SKIP_FIRST_ARG** bit is set in *flags* then *argv[0]* will be processed just like the other elements of *argv*.

Tk_ParseArgv normally returns the value **TCL_OK**. If an error occurs while parsing the arguments, then **TCL_ERROR** is returned and **Tk_ParseArgv** will leave an error message in the result of interpreter *interp* in the standard Tcl fashion. In the event of an error return, **argvPtr* will not have been modified, but *argv* could have been partially modified. The possible causes of errors are explained below.

The *argTable* array specifies the kinds of arguments that are expected; each of its entries has the following structure:

```
typedef struct {
    const char *key;
    int type;
    char *src;
    char *dst;
    const char *help;
} Tk_ArgvInfo;
```

The *key* field is a string such as "-display" or "-bg" that is compared with the values in *argv*. *Type* indicates how to process an argument that matches *key* (more on this below). *Src* and *dst* are additional values used in processing the argument. Their exact usage depends on *type*, but typically *src* indicates a value and *dst* indicates where to store the value. The **char *** declarations for *src* and *dst* are placeholders: the actual types may be different. Lastly, *help* is a string giving a brief description of this option; this string is printed when users ask for help about command-line options.

When processing an argument in *argv*, **Tk_ParseArgv** compares the argument to each of the *key*'s in *argTable*. **Tk_ParseArgv** selects the first specifier whose *key* matches the argument exactly, if such a specifier exists. Otherwise **Tk_ParseArgv** selects a specifier for which the argument is a unique abbreviation. If the argument is a unique abbreviation for more than one specifier, then an error is returned. If there is no matching entry in *argTable*, then the argument is skipped and returned to the caller.

Once a matching argument specifier is found, **Tk_ParseArgv** processes the argument according to the *type* field of the specifier. The argument that matched *key* is called "the matching argument" in the descriptions below. As part of the processing, **Tk_ParseArgv** may also use the next argument in *argv* after the matching argument, which is called "the following argument". The legal values for *type*, and the processing that they cause, are as follows:

TK_ARGV_END
: Marks the end of the table. The last entry in *argTable* must have this type; all of its other fields are ignored and it will never match any arguments.

TK_ARGV_CONSTANT
: *Src* is treated as an integer and *dst* is treated as a pointer to an integer. *Src* is stored at **dst*. The matching argument is discarded.

TK_ARGV_INT

The following argument must contain an integer string in the format accepted by **strtol** (e.g. "0" and "0x" prefixes may be used to specify octal or hexadecimal numbers, respectively). *Dst* is treated as a pointer to an integer; the following argument is converted to an integer value and stored at **dst*. *Src* is ignored. The matching and following arguments are discarded from *argv*.

TK_ARGV_FLOAT
The following argument must contain a floating-point number in the format accepted by **strtol**. *Dst* is treated as the address of a double-precision floating point value; the following argument is converted to a double-precision value and stored at **dst*. The matching and following arguments are discarded from *argv*.

TK_ARGV_STRING
In this form, *dst* is treated as a pointer to a (char *); **Tk_ParseArgv** stores at **dst* a pointer to the following argument, and discards the matching and following arguments from *argv*. *Src* is ignored.

TK_ARGV_UID
This form is similar to **TK_ARGV_STRING**, except that the argument is turned into a Tk_Uid by calling **Tk_GetUid**. *Dst* is treated as a pointer to a Tk_Uid; **Tk_ParseArgv** stores at **dst* the Tk_Uid corresponding to the following argument, and discards the matching and following arguments from *argv*. *Src* is ignored.

TK_ARGV_CONST_OPTION
This form causes a Tk option to be set (as if the **option** command had been invoked). The *src* field is treated as a pointer to a string giving the value of an option, and *dst* is treated as a pointer to the name of the option. The matching argument is discarded. If *tkwin* is NULL, then argument specifiers of this type are ignored (as if they did not exist).

TK_ARGV_OPTION_VALUE
This form is similar to **TK_ARGV_CONST_OPTION**, except that the value of the option is taken from the following argument instead of from *src*. *Dst* is used as the name of the option. *Src* is ignored. The matching and following arguments are discarded. If *tkwin* is NULL, then argument specifiers of this type are ignored (as if they did not exist).

TK_ARGV_OPTION_NAME_VALUE
In this case the following argument is taken as the name of a Tk option and the argument after that is taken as the value for that option. Both *src* and *dst* are ignored. All three arguments are discarded from *argv*. If *tkwin* is NULL, then argument specifiers of this type are ignored (as if they did not exist).

TK_ARGV_HELP
When this kind of option is encountered, **Tk_ParseArgv** uses the *help* fields of *argTable* to format a message describing all the valid arguments. The message is placed in interpreter *interp*'s result and **Tk_ParseArgv** returns **TCL_ERROR**. When this happens, the caller normally prints the help message and aborts. If the *key* field of a **TK_ARGV_HELP** specifier is NULL, then the specifier will never match any arguments; in this case the specifier simply provides extra documentation, which will be included when some other **TK_ARGV_HELP** entry causes help information to be returned.

TK_ARGV_REST
This option is used by programs or commands that allow the last several of their options to be the name and/or options for some other program. If a **TK_ARGV_REST** argument is found,

then **Tk_ParseArgv** does not process any of the remaining arguments; it returns them all at the beginning of *argv* (along with any other unprocessed arguments). In addition, **Tk_ParseArgv** treats *dst* as the address of an integer value, and stores at **dst* the index of the first of the **TK_ARGV_REST** options in the returned *argv*. This allows the program to distinguish the **TK_ARGV_REST** options from other unprocessed options that preceded the **TK_ARGV_REST**.

TK_ARGV_FUNC

For this kind of argument, *src* is treated as the address of a procedure, which is invoked to process the following argument. The procedure should have the following structure:

```
int
func(dst, key, nextArg)
    char *dst;
    char *key;
    char *nextArg;
{
}
```

The *dst* and *key* parameters will contain the corresponding fields from the *argTable* entry, and *nextArg* will point to the following argument from *argv* (or NULL if there are not any more arguments left in *argv*). If *func* uses *nextArg* (so that **Tk_ParseArgv** should discard it), then it should return 1. Otherwise it should return 0 and **TkParseArgv** will process the following argument in the normal fashion. In either event the matching argument is discarded.

TK_ARGV_GENFUNC

This form provides a more general procedural escape. It treats *src* as the address of a procedure, and passes that procedure all of the remaining arguments. The procedure should have the following form:

```
int
genfunc(dst, interp, key, argc, argv)
    char *dst;
    Tcl_Interp *interp;
    char *key;
    int argc;
    char **argv;
{
}
```

The *dst* and *key* parameters will contain the corresponding fields from the *argTable* entry. *Interp* will be the same as the *interp* argument to **Tcl_ParseArgv**. *Argc* and *argv* refer to all of the options after the matching one. *Genfunc* should behave in a fashion similar to **Tk_ParseArgv**: parse as many of the remaining arguments as it can, then return any that are left by compacting them to the beginning of *argv* (starting at *argv*[0]). *Genfunc* should return a count of how many arguments are left in *argv*; **Tk_ParseArgv** will process them. If *genfunc* encounters an error then it should leave an error message in interpreter *interp*'s result, in the usual Tcl fashion, and return -1; when this happens **Tk_ParseArgv** will abort its processing and return **TCL_ERROR**.

Flags

TK_ARGV_DONT_SKIP_FIRST_ARG
: **Tk_ParseArgv** normally treats *argv[0]* as a program or command name, and returns it to the caller just as if it had not matched *argTable*. If this flag is given, then *argv[0]* is not given special treatment.

TK_ARGV_NO_ABBREV
: Normally, **Tk_ParseArgv** accepts unique abbreviations for *key* values in *argTable*. If this flag is given then only exact matches will be acceptable.

TK_ARGV_NO_LEFTOVERS
: Normally, **Tk_ParseArgv** returns unrecognized arguments to the caller. If this bit is set in *flags* then **Tk_ParseArgv** will return an error if it encounters any argument that does not match *argTable*. The only exception to this rule is *argv[0]*, which will be returned to the caller with no errors as long as **TK_ARGV_DONT_SKIP_FIRST_ARG** is not specified.

TK_ARGV_NO_DEFAULTS
: Normally, **Tk_ParseArgv** searches an internal table of standard argument specifiers in addition to *argTable*. If this bit is set in *flags*, then **Tk_ParseArgv** will use only *argTable* and not its default table.

Example

Here is an example definition of an *argTable* and some sample command lines that use the options. Note the effect on *argc* and *argv*; arguments processed by **Tk_ParseArgv** are eliminated from *argv*, and *argc* is updated to reflect reduced number of arguments.

```
/*
 * Define and set default values for globals.
 */
int debugFlag = 0;
int numReps = 100;
char defaultFileName[] = "out";
char *fileName = defaultFileName;
Boolean exec = FALSE;

/*
 * Define option descriptions.
 */
Tk_ArgvInfo argTable[] = {
    {"-X", TK_ARGV_CONSTANT, (char *) 1, (char *) &debugFlag,
        "Turn on debugging printfs"},
    {"-N", TK_ARGV_INT, NULL, (char *) &numReps,
        "Number of repetitions"},
    {"-of", TK_ARGV_STRING, NULL, (char *) &fileName,
        "Name of file for output"},
    {"x", TK_ARGV_REST, NULL, (char *) &exec,
        "File to exec, followed by any arguments (must be last
    argument)."},
    {NULL, TK_ARGV_END, NULL, NULL,
```

```
        NULL}
};

main(argc, argv)
    int argc;
    char *argv[];
{
    ...

    if (Tk_ParseArgv(interp, tkwin, &argc, argv, argTable, 0) !=
    TCL_OK) {
        fprintf(stderr, "%s\n",
    Tcl_GetString(Tcl_GetObjResult(interp)));
        exit(1);
    }

    /*
     * Remainder of the program.
     */
}
```

Note that default values can be assigned to variables named in *argTable*: the variables will only be overwritten if the particular arguments are present in *argv*. Here are some example command lines and their effects.

```
prog -N 200 infile        # just sets the numReps variable to 200
prog -of out200 infile    # sets fileName to reference "out200"
prog -XN 10 infile        # sets the debug flag, also sets numReps
```

In all of the above examples, *argc* will be set by **Tk_ParseArgv** to 2, *argv*[0] will be "prog", *argv*[1] will be "infile", and *argv*[2] will be NULL.

Keywords: ParseArgv, arguments, command line, options

Copyright (c) 1990-1992 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

QWinEvent

Name

Tk_CollapseMotionEvents, Tk_QueueWindowEvent - Add a window event to the Tcl event queue

Synopsis

#include <tk.h>

int
Tk_CollapseMotionEvents(*display*, *collapse*)

Tk_QueueWindowEvent(*eventPtr*, *position*)

Arguments

Display *display* (in)
Display for which to control motion event collapsing.

int *collapse* (in)
Indicates whether motion events should be collapsed or not.

XEvent *eventPtr* (in)
An event to add to the event queue. It is important that all unused fields of the structure be set to zero.

Tcl_QueuePosition *position* (in)
Where to add the new event in the queue: **TCL_QUEUE_TAIL**, **TCL_QUEUE_HEAD**, or **TCL_QUEUE_MARK**.

Description

Tk_QueueWindowEvent places a window event on Tcl's internal event queue for eventual servicing. It creates a Tcl_Event structure, copies the event into that structure, and calls **Tcl_QueueEvent** to add the event to the queue. When the event is eventually removed from the queue it is processed just like all window events.

When multiple motion events are received for the same window in rapid succession, they are collapsed by default. This behavior can be controlled with **Tk_CollapseMotionEvents**. **Tk_CollapseMotionEvents** always returns the previous value for collapse behavior on the *display*.

The *position* argument to **Tk_QueueWindowEvent** has the same significance as for **Tcl_QueueEvent**; see the documentation for **Tcl_QueueEvent** for details.

Keywords: QWinEvent, callback, clock, handler, modal timeout, events

Copyright (c) 1995-1996 Sun Microsystems, Inc.

Restack

Name

Tk_RestackWindow - Change a window's position in the stacking order

Synopsis

#include <tk.h>

int
Tk_RestackWindow(*tkwin*, *aboveBelow*, *other*)

Arguments

Tk_Window *tkwin* (in)
Token for window to restack.

int *aboveBelow* (in)
Indicates new position of *tkwin* relative to *other*; must be **Above** or **Below**.

Tk_Window *other* (in)
Tkwin will be repositioned just above or below this window. Must be a sibling of *tkwin* or a descendant of a sibling. If NULL then *tkwin* is restacked above or below all siblings.

Description

Tk_RestackWindow changes the stacking order of *window* relative to its siblings. If *other* is specified as NULL then *window* is repositioned at the top or bottom of its stacking order, depending on whether *aboveBelow* is **Above** or **Below**. If *other* has a non-NULL value then *window* is repositioned just above or below *other*.

The *aboveBelow* argument must have one of the symbolic values **Above** or **Below**. Both of these values are defined by the include file <X11/Xlib.h>.

Keywords: Restack, above, below, obscure, stacking order

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

RestrictEv

Name

Tk_RestrictEvents - filter and selectively delay X events

Synopsis

#include <tk.h>

Tk_RestrictProc *
Tk_RestrictEvents(*proc*, *arg*, *prevArgPtr*)

Arguments

Tk_RestrictProc ****proc* **(in)**
Predicate procedure to call to filter incoming X events. NULL means do not restrict events at all.

ClientData *arg* (in)
Arbitrary argument to pass to *proc*.

ClientData ****prevArgPtr* **(out)**
Pointer to place to save argument to previous restrict procedure.

Description

This procedure is useful in certain situations where applications are only prepared to receive certain X events. After **Tk_RestrictEvents** is called, **Tcl_DoOneEvent** (and hence **Tk_MainLoop**) will filter X input events through *proc*. *Proc* indicates whether a given event is to be processed immediately, deferred until some later time (e.g. when the event restriction is lifted), or discarded. *Proc* is a procedure with arguments and result that match the type **Tk_RestrictProc**:

```
typedef Tk_RestrictAction Tk_RestrictProc(
        ClientData arg,
        XEvent *eventPtr);
```

The *arg* argument is a copy of the *arg* passed to **Tk_RestrictEvents**; it may be used to provide *proc* with information it needs to filter events. The *eventPtr* points to an event under consideration. *Proc* returns a restrict action (enumerated type **Tk_RestrictAction**) that indicates what **Tcl_DoOneEvent** should do with the event. If the return value is **TK_PROCESS_EVENT**, then the event will be handled immediately. If the return value is

TK_DEFER_EVENT, then the event will be left on the event queue for later processing. If the return value is **TK_DISCARD_EVENT**, then the event will be removed from the event queue and discarded without being processed.

Tk_RestrictEvents uses its return value and *prevArgPtr* to return information about the current event restriction procedure (a NULL return value means there are currently no restrictions). These values may be used to restore the previous restriction state when there is no longer any need for the current restriction.

There are very few places where **Tk_RestrictEvents** is needed. In most cases, the best way to restrict events is by changing the bindings with the **bind** Tcl command or by calling **Tk_CreateEventHandler** and **Tk_DeleteEventHandler** from C. The main place where **Tk_RestrictEvents** must be used is when performing synchronous actions (for example, if you need to wait for a particular event to occur on a particular window but you do not want to invoke any handlers for any other events). The "obvious" solution in these situations is to call **XNextEvent** or **XWindowEvent**, but these procedures cannot be used because Tk keeps its own event queue that is separate from the X event queue. Instead, call **Tk_RestrictEvents** to set up a filter, then call **Tcl_DoOneEvent** to retrieve the desired event(s).

Keywords: RestrictEv, delay, event, filter, restriction

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

SetAppName

Name

Tk_SetAppName - Set the name of an application for 'send' commands

Synopsis

#include <tk.h>

const char *
Tk_SetAppName(*tkwin*, *name*)

Arguments

Tk_Window *tkwin* (in)
Token for window in application. Used only to select a particular application.

const char **name* (in)
Name under which to register the application.

Description

Tk_SetAppName associates a name with a given application and records that association on the display containing with the application's main window. After this procedure has been invoked, other applications on the display will be able to use the **send** command to invoke operations in the application. If *name* is already in use by some other application on the display, then a new name will be generated by appending " **#2**" to *name*; if this name is also in use, the number will be incremented until an unused name is found. The return value from the procedure is a pointer to the name actually used.

If the application already has a name when **Tk_SetAppName** is called, then the new name

replaces the old name.

Tk_SetAppName also adds a **send** command to the application's interpreter, which can be used to send commands from this application to others on any of the displays where the application has windows.

The application's name registration persists until the interpreter is deleted or the **send** command is deleted from *interp*, at which point the name is automatically unregistered and the application becomes inaccessible via **send**. The application can be made accessible again by calling **Tk_SetAppName**.

Tk_SetAppName is called automatically by **Tk_Init**, so applications do not normally need to call it explicitly.

The command **tk appname** provides Tcl-level access to the functionality of **Tk_SetAppName**.

Keywords: SetAppName, application, name, register, send command

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

SetCaret

Name

Tk_SetCaretPos - set the display caret location

Synopsis

#include <tk.h>

int
Tk_SetCaretPos(*tkwin*, *x*, *y*, *height*)

Arguments

Tk_Window *tkwin* (in)
Token for window.

int *x* (in)
Window-relative x coordinate.

int *y* (in)
Window-relative y coordinate.

int *h* (in)
Height of the caret in the window.

Description

Tk_SetCaretPos sets the caret location for the display of the specified Tk_Window *tkwin*. The caret is the per-display cursor location used for indicating global focus (e.g. to comply with Microsoft Accessibility guidelines), as well as for location of the over-the-spot XIM (X Input Methods) or Windows IME windows.

Keywords: SetCaret, caret, cursor

Copyright (c) 2002 ActiveState Corporation.

SetClass

Name

Tk_SetClass, Tk_Class - set or retrieve a window's class

Synopsis

#include <tk.h>

Tk_SetClass(*tkwin*, *class*)

Tk_Uid
Tk_Class(*tkwin*)

Arguments

Tk_Window *tkwin* (in)
Token for window.

char **class* (in)
New class name for window.

Description

Tk_SetClass is called to associate a class with a particular window. The *class* string identifies the type of the window; all windows with the same general class of behavior (button, menu, etc.) should have the same class. By convention all class names start with a capital letter, and there exists a Tcl command with the same name as each class (except all in lower-case) which can be used to create and manipulate windows of that class. A window's class string is initialized to NULL when the window is created.

For main windows, Tk automatically propagates the name and class to the WM_CLASS property used by window managers. This happens either when a main window is actually created (e.g. in **Tk_MakeWindowExist**), or when **Tk_SetClass** is called, whichever occurs later. If a main window has not been assigned a class then Tk will not set the WM_CLASS property for the window.

Tk_Class is a macro that returns the current value of *tkwin*'s class. The value is returned as a Tk_Uid, which may be used just like a string pointer but also has the properties of a unique identifier (see the manual entry for **Tk_GetUid** for details). If *tkwin* has not yet been given a class, then **Tk_Class** will return NULL.

Keywords: SetClass, class, unique identifier, window, window manager

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

SetClassProcs

Name

Tk_SetClassProcs - register widget specific procedures

Synopsis

#include <tk.h>

Tk_SetClassProcs(*tkwin*, *procs*, *instanceData*)

Arguments

Tk_Window *tkwin* (in)
Token for window to modify.

const Tk_ClassProcs **procs* (in)
Pointer to data structure containing widget specific procedures. The data structure pointed to by *procs* must be static: Tk keeps a reference to it as long as the window exists.

ClientData *instanceData* (in)
Arbitrary one-word value to pass to widget callbacks.

Description

Tk_SetClassProcs is called to register a set of procedures that are used as callbacks in different places.

The structure pointed to by *procs* contains the following:

```
typedef struct Tk_ClassProcs {
    unsigned int size;
    Tk_ClassWorldChangedProc *worldChangedProc;
    Tk_ClassCreateProc *createProc;
    Tk_ClassModalProc *modalProc;
} Tk_ClassProcs;
```

The *size* field is used to simplify future expansion of the structure. It should always be set to (literally) **sizeof(Tk_ClassProcs)**.

worldChangedProc is invoked when the system has altered in some way that requires some reaction from the widget. For example, when a font alias (see the **font** manual entry) is reconfigured, widgets configured to use that font alias must update their display accordingly. *worldChangedProc* should have arguments and results that match the type **Tk_ClassWorldChangedProc**:

```
typedef void Tk_ClassWorldChangedProc(
        ClientData instanceData);
```

The *instanceData* parameter passed to the *worldChangedProc* will be identical to the *instanceData* parameter passed to **Tk_SetClassProcs**.

createProc is used to create platform-dependent windows. It is invoked by **Tk_MakeWindowExist**. *createProc* should have arguments and results that match the type **Tk_ClassCreateProc**:

```
typedef Window Tk_ClassCreateProc(
        Tk_Window tkwin,
        Window parent,
        ClientData instanceData);
```

The *tkwin* and *instanceData* parameters will be identical to the *tkwin* and *instanceData* parameters passed to **Tk_SetClassProcs**. The *parent* parameter will be the parent of the window to be created. The *createProc* should return the created window.

modalProc is invoked after all bindings on a widget have been triggered in order to handle a modal loop. *modalProc* should have arguments and results that match the type **Tk_ClassModalProc**:

```
typedef void Tk_ClassModalProc(
        Tk_Window tkwin,
        XEvent *eventPtr);
```

The *tkwin* parameter to *modalProc* will be identical to the *tkwin* parameter passed to **Tk_SetClassProcs**. The *eventPtr* parameter will be a pointer to an XEvent structure describing the event being processed.

Keywords: SetClassProcs, callback, class

Copyright (c) 2000 Ajuba Solutions.

SetGrid

Name

Tk_SetGrid, Tk_UnsetGrid - control the grid for interactive resizing

Synopsis

#include <tk.h>

Tk_SetGrid(*tkwin*, *reqWidth*, *reqHeight*, *widthInc*, *heightInc*)

Tk_UnsetGrid(*tkwin*)

Arguments

Tk_Window *tkwin* (in)
Token for window.

int *reqWidth* (in)
Width in grid units that corresponds to the pixel dimension *tkwin* has requested via **Tk_GeometryRequest**.

int *reqHeight* (in)
Height in grid units that corresponds to the pixel dimension *tkwin* has requested via **Tk_GeometryRequest**.

int *widthInc* (in)
Width of one grid unit, in pixels.

int *heightInc* (in)
Height of one grid unit, in pixels.

Description

Tk_SetGrid turns on gridded geometry management for *tkwin*'s toplevel window and specifies the geometry of the grid. **Tk_SetGrid** is typically invoked by a widget when its **setGrid** option is true. It restricts interactive resizing of *tkwin*'s toplevel window so that the space allocated to the toplevel is equal to its requested size plus or minus even multiples of *widthInc* and *heightInc*. Furthermore, the *reqWidth* and *reqHeight* values are passed to the window manager so that it can report the window's size in grid units during interactive resizes. If *tkwin*'s configuration changes (e.g., the size of a grid unit changes) then the widget should invoke **Tk_SetGrid** again with the new information.

Tk_UnsetGrid cancels gridded geometry management for *tkwin*'s toplevel window.

For each toplevel window there can be at most one internal window with gridding enabled. If

Tk_SetGrid or **Tk_UnsetGrid** is invoked when some other window is already controlling gridding for *tkwin*'s toplevel, the calls for the new window have no effect.

See the **wm** manual entry for additional information on gridded geometry management.

Keywords: SetGrid, grid, window, window manager

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

SetOptions

Name

Tk_CreateOptionTable, Tk_DeleteOptionTable, Tk_InitOptions, Tk_SetOptions, Tk_FreeSavedOptions, Tk_RestoreSavedOptions, Tk_GetOptionValue, Tk_GetOptionInfo, Tk_FreeConfigOptions, Tk_Offset - process configuration options

Synopsis

#include <tk.h>

Tk_OptionTable
Tk_CreateOptionTable(*interp*, *templatePtr*)

Tk_DeleteOptionTable(*optionTable*)

int
Tk_InitOptions(*interp*, *recordPtr*, *optionTable*, *tkwin*)

int
Tk_SetOptions(*interp*, *recordPtr*, *optionTable*, *objc*, *objv*, *tkwin*, *savePtr*, *maskPtr*)

Tk_FreeSavedOptions(*savedPtr*)

Tk_RestoreSavedOptions(*savedPtr*)

Tcl_Obj *
Tk_GetOptionValue(*interp*, *recordPtr*, *optionTable*, *namePtr*, *tkwin*)

Tcl_Obj *
Tk_GetOptionInfo(*interp*, *recordPtr*, *optionTable*, *namePtr*, *tkwin*)

Tk_FreeConfigOptions(*recordPtr*, *optionTable*, *tkwin*)

int
Tk_Offset(*type*, *field*)

Arguments

Tcl_Interp *_interp_ (in)
A Tcl interpreter. Most procedures use this only for returning error messages; if it is NULL then no error messages are returned. For **Tk_CreateOptionTable** the value cannot be NULL; it gives the interpreter in which the option table will be used.

const Tk_OptionSpec *_templatePtr_ (in)
Points to an array of static information that describes the configuration options that are supported. Used to build a Tk_OptionTable. The information pointed to by this argument must exist for the lifetime of the Tk_OptionTable.

Tk_OptionTable _optionTable_ (in)

Token for an option table. Must have been returned by a previous call to **Tk_CreateOptionTable**.

char *_recordPtr_ (in/out)
Points to structure in which values of configuration options are stored; fields of this record are modified by procedures such as **Tk_SetOptions** and read by procedures such as **Tk_GetOptionValue**.

Tk_Window _tkwin_ (in)
For options such as **TK_OPTION_COLOR**, this argument indicates the window in which the option will be used. If *optionTable* uses no window-dependent options, then a NULL value may be supplied for this argument.

int _objc_ (in)
Number of values in *objv*.

Tcl_Obj *_const_ *_objv_ (in)
Command-line arguments for setting configuring options.

Tk_SavedOptions *_savePtr_ (out)
If not NULL, the structure pointed to by this argument is filled in with the old values of any options that were modified and old values are restored automatically if an error occurs in **Tk_SetOptions**.

int *_maskPtr_ (out)
If not NULL, the word pointed to by *maskPtr* is filled in with the bit-wise OR of the *typeMask* fields for the options that were modified.

Tk_SavedOptions *_savedPtr_ (in/out)
Points to a structure previously filled in by **Tk_SetOptions** with old values of modified options.

Tcl_Obj *_namePtr_ (in)
The value of this object is the name of a particular option. If NULL is passed to **Tk_GetOptionInfo** then information is returned for all options. Must not be NULL when **Tk_GetOptionValue** is called.

type name _type_ (in)
The name of the type of a record.

field name _field_ (in)
The name of a field in records of type *type*.

Description

These procedures handle most of the details of parsing configuration options such as those for Tk widgets. Given a description of what options are supported, these procedures handle all the details of parsing options and storing their values into a C structure associated with the widget or object. The procedures were designed primarily for widgets in Tk, but they can also be used for other kinds of objects that have configuration options. In the rest of this manual page "widget" will be used to refer to the object whose options are being managed; in practice the object may not actually be a widget. The term "widget record" is used to refer to the C-level structure in which information about a particular widget or object is stored.

Note that the easiest way to learn how to use these procedures is to look at a working example. In Tk, the simplest example is the code that implements the button family of widgets, which is in **tkButton.c**. Other examples are in **tkSquare.c** and **tkMenu.c**.

In order to use these procedures, the code that implements the widget must contain a static array of Tk_OptionSpec structures. This is a template that describes the various options supported by that class of widget; there is a separate template for each kind of widget. The template contains information such as the name of each option, its type, its default value, and where the value of the option is stored in the widget record. See TEMPLATES below for more detail.

In order to process configuration options efficiently, the static template must be augmented with additional information that is available only at runtime. The procedure **Tk_CreateOptionTable** creates this dynamic information from the template and returns a Tk_OptionTable token that describes both the static and dynamic information. All of the other procedures, such as **Tk_SetOptions**, take a Tk_OptionTable token as argument. Typically, **Tk_CreateOptionTable** is called the first time that a widget of a particular class is created and the resulting Tk_OptionTable is used in the future for all widgets of that class. A Tk_OptionTable may be used only in a single interpreter, given by the *interp* argument to **Tk_CreateOptionTable**. When an option table is no longer needed **Tk_DeleteOptionTable** should be called to free all of its resources. All of the option tables for a Tcl interpreter are freed automatically if the interpreter is deleted.

Tk_InitOptions is invoked when a new widget is created to set the default values. **Tk_InitOptions** is passed a token for an option table (*optionTable*) and a pointer to a widget record (*recordPtr*), which is the C structure that holds information about this widget. **Tk_InitOptions** uses the information in the option table to choose an appropriate default for each option, then it stores the default value directly into the widget record, overwriting any information that was already present in the widget record. **Tk_InitOptions** normally returns **TCL_OK**. If an error occurred while setting the default values (e.g., because a default value was erroneous) then **TCL_ERROR** is returned and an error message is left in *interp*'s result if *interp* is not NULL. For any widget's configuration option that has **TK_OPTION_DONT_SET_DEFAULT** set in its *flags* field, the above initialization is fully skipped, see below.

Tk_SetOptions is invoked to modify configuration options based on information specified in a Tcl command. The command might be one that creates a new widget, or a command that modifies options on an existing widget. The *objc* and *objv* arguments describe the values of the arguments from the Tcl command. *Objv* must contain an even number of objects: the first object of each pair gives the name of an option and the second object gives the new value for that option. **Tk_SetOptions** looks up each name in *optionTable*, checks that the new value of the option conforms to the type in *optionTable*, and stores the value of the option into the widget record given by *recordPtr*. **Tk_SetOptions** normally returns **TCL_OK**. If an error occurred (such as an unknown option name or an illegal option value) then **TCL_ERROR** is returned and an error message is left in *interp*'s result if *interp* is not NULL.

Tk_SetOptions has two additional features. First, if the *maskPtr* argument is not NULL then it points to an integer value that is filled in with information about the options that were modified. For each option in the template passed to **Tk_CreateOptionTable** there is a *typeMask* field. The bits of this field are defined by the code that implements the widget; for example, each bit might correspond to a particular configuration option. Alternatively, bits might be used functionally. For example, one bit might be used for redisplay: all options that affect the widget's display, such that changing the option requires the widget to be redisplayed, might have that bit set. Another bit might indicate that the geometry of the widget must be recomputed, and so on.

Tk_SetOptions OR's together the *typeMask* fields from all the options that were modified and returns this value at **maskPtr*; the caller can then use this information to optimize itself so that, for example, it does not redisplay the widget if the modified options do not affect the widget's appearance.

The second additional feature of **Tk_SetOptions** has to do with error recovery. If an error occurs while processing configuration options, this feature makes it possible to restore all the configuration options to their previous values. Errors can occur either while processing options in **Tk_SetOptions** or later in the caller. In many cases the caller does additional processing after **Tk_SetOptions** returns; for example, it might use an option value to set a trace on a variable and may detect an error if the variable is an array instead of a scalar. Error recovery is enabled by passing in a non-NULL value for the *savePtr* argument to **Tk_SetOptions**; this should be a pointer to an uninitialized Tk_SavedOptions structure on the caller's stack. **Tk_SetOptions** overwrites the structure pointed to by *savePtr* with information about the old values of any options modified by the procedure. If **Tk_SetOptions** returns successfully, the caller uses the structure in one of two ways. If the caller completes its processing of the new options without any errors, then it must pass the structure to **Tk_FreeSavedOptions** so that the old values can be freed. If the caller detects an error in its processing of the new options, then it should pass the structure to **Tk_RestoreSavedOptions**, which will copy the old values back into the widget record and free the new values. If **Tk_SetOptions** detects an error then it automatically restores any options that had already been modified and leaves **savePtr* in an empty state: the caller need not call either **Tk_FreeSavedOptions** or **Tk_RestoreSavedOptions**. If the *savePtr* argument to **Tk_SetOptions** is NULL then **Tk_SetOptions** frees each old option value immediately when it sets a new value for the option. In this case, if an error occurs in the third option, the old values for the first two options cannot be restored.

Tk_GetOptionValue returns the current value of a configuration option for a particular widget. The *namePtr* argument contains the name of an option; **Tk_GetOptionValue** uses *optionTable* to lookup the option and extract its value from the widget record pointed to by *recordPtr*, then it returns an object containing that value. If an error occurs (e.g., because *namePtr* contains an unknown option name) then NULL is returned and an error message is left in *interp*'s result unless *interp* is NULL.

Tk_GetOptionInfo returns information about configuration options in a form suitable for **configure** widget commands. If the *namePtr* argument is not NULL, it points to an object that gives the name of a configuration option; **Tk_GetOptionInfo** returns an object containing a list with five elements, which are the name of the option, the name and class used for the option in the option database, the default value for the option, and the current value for the option. If the *namePtr* argument is NULL, then **Tk_GetOptionInfo** returns information about all options in the form of a list of lists; each sublist describes one option. Synonym options are handled differently depending on whether *namePtr* is NULL: if *namePtr* is NULL then the sublist for each synonym option has only two elements, which are the name of the option and the name of the other option that it refers to; if *namePtr* is non-NULL and names a synonym option then the object returned is the five-element list for the other option that the synonym refers to. If an error occurs (e.g., because *namePtr* contains an unknown option name) then NULL is returned and an error message is left in *interp*'s result unless *interp* is NULL.

Tk_FreeConfigOptions must be invoked when a widget is deleted. It frees all of the resources associated with any of the configuration options defined in *recordPtr* by *optionTable*.

The **Tk_Offset** macro is provided as a safe way of generating the *objOffset* and *internalOffset* values for entries in Tk_OptionSpec structures. It takes two arguments: the name of a type of record, and the name of a field in that record. It returns the byte offset of the named field in records of the given type.

Templates

The array of Tk_OptionSpec structures passed to **Tk_CreateOptionTable** via its *templatePtr* argument describes the configuration options supported by a particular class of widgets. Each structure specifies one configuration option and has the following fields:

```
typedef struct {
    Tk_OptionType type;
    const char *optionName;
    const char *dbName;
    const char *dbClass;
    const char *defValue;
    int objOffset;
    int internalOffset;
    int flags;
    const void *clientData;
    int typeMask;
} Tk_OptionSpec;
```

The *type* field indicates what kind of configuration option this is (e.g. **TK_OPTION_COLOR** for a color value, or **TK_OPTION_INT** for an integer value). *Type* determines how the value of the option is parsed (more on this below). The *optionName* field is a string such as **-font** or **-bg**; it is the name used for the option in Tcl commands and passed to procedures via the *objc* or *namePtr* arguments. The *dbName* and *dbClass* fields are used by **Tk_InitOptions** to look up a default value for this option in the option database; if *dbName* is NULL then the option database is not used by **Tk_InitOptions** for this option. The *defValue* field specifies a default value for this configuration option if no value is specified in the option database. The *objOffset* and *internalOffset* fields indicate where to store the value of this option in widget records (more on this below); values for the *objOffset* and *internalOffset* fields should always be generated with the **Tk_Offset** macro. The *flags* field contains additional information to control the processing of this configuration option (see below for details). *ClientData* provides additional type-specific data needed by certain types. For instance, for **TK_OPTION_COLOR** types, *clientData* is a string giving the default value to use on monochrome displays. See the descriptions of the different types below for details. The last field, *typeMask*, is used by **Tk_SetOptions** to return information about which options were modified; see the description of **Tk_SetOptions** above for details.

When **Tk_InitOptions** and **Tk_SetOptions** store the value of an option into the widget record, they can do it in either of two ways. If the *objOffset* field of the Tk_OptionSpec is greater than or equal to zero, then the value of the option is stored as a (Tcl_Obj *) at the location in the widget record given by *objOffset*. If the *internalOffset* field of the Tk_OptionSpec is greater than or equal to zero, then the value of the option is stored in a type-specific internal form at the location in the widget record given by *internalOffset*. For example, if the option's type is **TK_OPTION_INT** then the internal form is an integer. If the *objOffset* or *internalOffset* field is negative then the

value is not stored in that form. At least one of the offsets must be greater than or equal to zero.

The *flags* field consists of one or more bits ORed together. The following flags are supported:

TK_OPTION_NULL_OK
: If this bit is set for an option then an empty string will be accepted as the value for the option and the resulting internal form will be a NULL pointer, a zero value, or **None**, depending on the type of the option. If the flag is not set then empty strings will result in errors. **TK_OPTION_NULL_OK** is typically used to allow a feature to be turned off entirely, e.g. set a cursor value to **None** so that a window simply inherits its parent's cursor. Not all option types support the **TK_OPTION_NULL_OK** flag; for those that do, there is an explicit indication of that fact in the descriptions below.

TK_OPTION_DONT_SET_DEFAULT
: If this bit is set for an option then no default value will be set in **Tk_InitOptions** for this option. Neither the option database, nor any system default value, nor *optionTable* are used to give a default value to this option. Instead it is assumed that the caller has already supplied a default value in the widget code.

The *type* field of each Tk_OptionSpec structure determines how to parse the value of that configuration option. The legal value for *type*, and the corresponding actions, are described below. If the type requires a *tkwin* value to be passed into procedures like **Tk_SetOptions**, or if it uses the *clientData* field of the Tk_OptionSpec, then it is indicated explicitly; if not mentioned, the type requires neither *tkwin* nor *clientData*.

TK_OPTION_ANCHOR
: The value must be a standard anchor position such as **ne** or **center**. The internal form is a Tk_Anchor value like the ones returned by **Tk_GetAnchorFromObj**.

TK_OPTION_BITMAP
: The value must be a standard Tk bitmap name. The internal form is a Pixmap token like the ones returned by **Tk_AllocBitmapFromObj**. This option type requires *tkwin* to be supplied to procedures such as **Tk_SetOptions**, and it supports the **TK_OPTION_NULL_OK** flag.

TK_OPTION_BOOLEAN
: The value must be a standard boolean value such as **true** or **no**. The internal form is an integer with value 0 or 1. Note that if the *objOffset* field is not used, information about the original value of this option will be lost.

TK_OPTION_BORDER
: The value must be a standard color name such as **red** or **#ff8080**. The internal form is a Tk_3DBorder token like the ones returned by **Tk_Alloc3DBorderFromObj**. This option type requires *tkwin* to be supplied to procedures such as **Tk_SetOptions**, and it supports the **TK_OPTION_NULL_OK** flag.

TK_OPTION_COLOR
: The value must be a standard color name such as **red** or **#ff8080**. The internal form is an (XColor *) token like the ones returned by **Tk_AllocColorFromObj**. This option type requires *tkwin* to be supplied to procedures such as **Tk_SetOptions**, and it supports the **TK_OPTION_NULL_OK** flag.

TK_OPTION_CURSOR
: The value must be a standard cursor name such as **cross** or **@foo**. The internal form is a Tk_Cursor token like the ones returned by **Tk_AllocCursorFromObj**. This option type requires

tkwin to be supplied to procedures such as **Tk_SetOptions**, and when the option is set the cursor for the window is changed by calling **XDefineCursor**. This option type also supports the **TK_OPTION_NULL_OK** flag.

TK_OPTION_CUSTOM
This option allows applications to define new option types. The clientData field of the entry points to a structure defining the new option type. See the section **CUSTOM OPTION TYPES** below for details.

TK_OPTION_DOUBLE
The string value must be a floating-point number in the format accepted by **strtol**. The internal form is a C **double** value. This option type supports the **TK_OPTION_NULL_OK** flag; if a NULL value is set, the internal representation is set to zero.

TK_OPTION_END
Marks the end of the template. There must be a Tk_OptionSpec structure with *type* **TK_OPTION_END** at the end of each template. If the *clientData* field of this structure is not NULL, then it points to an additional array of Tk_OptionSpec's, which is itself terminated by another **TK_OPTION_END** entry. Templates may be chained arbitrarily deeply. This feature allows common options to be shared by several widget classes.

TK_OPTION_FONT
The value must be a standard font name such as **Times 16**. The internal form is a Tk_Font handle like the ones returned by **Tk_AllocFontFromObj**. This option type requires *tkwin* to be supplied to procedures such as **Tk_SetOptions**, and it supports the **TK_OPTION_NULL_OK** flag.

TK_OPTION_INT
The string value must be an integer in the format accepted by **strtol** (e.g. **0** and **0x** prefixes may be used to specify octal or hexadecimal numbers, respectively). The internal form is a C **int** value.

TK_OPTION_JUSTIFY
The value must be a standard justification value such as **left**. The internal form is a Tk_Justify like the values returned by **Tk_GetJustifyFromObj**.

TK_OPTION_PIXELS
The value must specify a screen distance such as **2i** or **6.4**. The internal form is an integer value giving a distance in pixels, like the values returned by **Tk_GetPixelsFromObj**. Note that if the *objOffset* field is not used, information about the original value of this option will be lost. See **OBJOFFSET VS. INTERNALOFFSET** below for details. This option type supports the **TK_OPTION_NULL_OK** flag; if a NULL value is set, the internal representation is set to INT_MIN.

TK_OPTION_RELIEF
The value must be standard relief such as **raised**. The internal form is an integer relief value such as **TK_RELIEF_RAISED**. This option type supports the **TK_OPTION_NULL_OK** flag; if a NULL value is set, the internal representation is set to **TK_RELIEF_NULL**.

TK_OPTION_STRING
The value may be any string. The internal form is a (char *) pointer that points to a dynamically allocated copy of the value. This option type supports the **TK_OPTION_NULL_OK** flag.

TK_OPTION_STRING_TABLE

For this type, *clientData* is a pointer to an array of strings suitable for passing to **Tcl_GetIndexFromObj**. The value must be one of the strings in the table, or a unique abbreviation of one of the strings. The internal form is an integer giving the index into the table of the matching string, like the return value from **Tcl_GetStringFromObj**. This option type supports the **TK_OPTION_NULL_OK** flag; if a NULL value is set, the internal representation is set to -1.

TK_OPTION_SYNONYM
This type is used to provide alternative names for an option (for example, **-bg** is often used as a synonym for **-background**). The **clientData** field is a string that gives the name of another option in the same table. Whenever the synonym option is used, the information from the other option will be used instead.

TK_OPTION_WINDOW
The value must be a window path name. The internal form is a **Tk_Window** token for the window. This option type requires *tkwin* to be supplied to procedures such as **Tk_SetOptions** (in order to identify the application), and it supports the **TK_OPTION_NULL_OK** flag.

Storage management issues

If a field of a widget record has its offset stored in the *objOffset* or *internalOffset* field of a Tk_OptionSpec structure then the procedures described here will handle all of the storage allocation and resource management issues associated with the field. When the value of an option is changed, **Tk_SetOptions** (or **Tk_FreeSavedOptions**) will automatically free any resources associated with the old value, such as Tk_Fonts for **TK_OPTION_FONT** options or dynamically allocated memory for **TK_OPTION_STRING** options. For an option stored as an object using the *objOffset* field of a Tk_OptionSpec, the widget record shares the object pointed to by the *objv* value from the call to **Tk_SetOptions**. The reference count for this object is incremented when a pointer to it is stored in the widget record and decremented when the option is modified. When the widget is deleted **Tk_FreeConfigOptions** should be invoked; it will free the resources associated with all options and decrement reference counts for any objects.

However, the widget code is responsible for storing NULL or **None** in all pointer and token fields before invoking **Tk_InitOptions**. This is needed to allow proper cleanup in the rare case where an error occurs in **Tk_InitOptions**.

Objoffset vs. internaloffset

In most cases it is simplest to use the *internalOffset* field of a Tk_OptionSpec structure and not the *objOffset* field. This makes the internal form of the value immediately available to the widget code so the value does not have to be extracted from an object each time it is used. However, there are two cases where the *objOffset* field is useful. The first case is for **TK_OPTION_PIXELS** options. In this case, the internal form is an integer pixel value that is valid only for a particular screen. If the value of the option is retrieved, it will be returned as a simple number. For example, after the command **.b configure -borderwidth 2m**, the command **.b configure -borderwidth** might return 7, which is the integer pixel value corresponding to **2m**. Unfortunately, this loses the original screen-independent value. Thus for **TK_OPTION_PIXELS** options it is better to use the *objOffset* field. In this case the original value of the option is retained in the object and can be returned when the option is retrieved. In most cases it is convenient to use the *internalOffset* field as well, so that the integer value is immediately available for use in the widget code (alternatively, **Tk_GetPixelsFromObj** can be used to

extract the integer value from the object whenever it is needed). Note that the problem of losing information on retrievals exists only for **TK_OPTION_PIXELS** options.

The second reason to use the *objOffset* field is in order to implement new types of options not supported by these procedures. To implement a new type of option, you can use **TK_OPTION_STRING** as the type in the Tk_OptionSpec structure and set the *objOffset* field but not the *internalOffset* field. Then, after calling **Tk_SetOptions**, convert the object to internal form yourself.

Ttk widgets do not support the *internalOffset* machinery. Option values of Ttk widgets are always stored as (Tcl_Obj *), meaning that the *objOffset* field must be used.

Custom option types

Applications can extend the built-in configuration types with additional configuration types by writing procedures to parse, print, free, and restore saved copies of the type and creating a structure pointing to those procedures:

```
typedef struct Tk_ObjCustomOption {
    const char *name;
    Tk_CustomOptionSetProc *setProc;
    Tk_CustomOptionGetProc *getProc;
    Tk_CustomOptionRestoreProc *restoreProc;
    Tk_CustomOptionFreeProc *freeProc;
    ClientData clientData;
} Tk_ObjCustomOption;

typedef int Tk_CustomOptionSetProc(
    ClientData clientData,
    Tcl_Interp *interp,
    Tk_Window tkwin,
    Tcl_Obj **valuePtr,
    char *recordPtr,
    int internalOffset,
    char *saveInternalPtr,
    int flags);

typedef Tcl_Obj *Tk_CustomOptionGetProc(
    ClientData clientData,
    Tk_Window tkwin,
    char *recordPtr,
    int internalOffset);

typedef void Tk_CustomOptionRestoreProc(
    ClientData clientData,
    Tk_Window tkwin,
    char *internalPtr,
    char *saveInternalPtr);
```

```
typedef void Tk_CustomOptionFreeProc(
    ClientData clientData,
    Tk_Window tkwin,
    char *internalPtr);
```

The Tk_ObjCustomOption structure contains six fields: a name for the custom option type; pointers to the four procedures; and a *clientData* value to be passed to those procedures when they are invoked. The *clientData* value typically points to a structure containing information that is needed by the procedures when they are parsing and printing options. *RestoreProc* and *freeProc* may be NULL, indicating that no function should be called for those operations.

The *setProc* procedure is invoked by **Tk_SetOptions** to convert a Tcl_Obj into an internal representation and store the resulting value in the widget record. The arguments are:

clientData
A copy of the *clientData* field in the Tk_ObjCustomOption structure.

interp
A pointer to a Tcl interpreter, used for error reporting.

Tkwin
A copy of the *tkwin* argument to **Tk_SetOptions**

valuePtr
A pointer to a reference to a Tcl_Obj describing the new value for the option; it could have been specified explicitly in the call to **Tk_SetOptions** or it could come from the option database or a default. If the objOffset for the option is non-negative (the option value is stored as a (Tcl_Obj *) in the widget record), the Tcl_Obj pointer referenced by *valuePtr* is the pointer that will be stored at the objOffset for the option. *SetProc* may modify the value if necessary; for example, *setProc* may change the value to NULL to support the **TK_OPTION_NULL_OK** flag.

recordPtr
A pointer to the start of the widget record to modify.

internalOffset
Offset in bytes from the start of the widget record to the location where the internal representation of the option value is to be placed.

saveInternalPtr
A pointer to storage allocated in a Tk_SavedOptions structure for the internal representation of the original option value. Before setting the option to its new value, *setProc* should set the value referenced by *saveInternalPtr* to the original value of the option in order to support **Tk_RestoreSavedOptions**.

flags
A copy of the *flags* field in the Tk_OptionSpec structure for the option

SetProc returns a standard Tcl result: **TCL_OK** to indicate successful processing, or **TCL_ERROR** to indicate a failure of any kind. An error message may be left in the Tcl interpreter given by *interp* in the case of an error.

The *getProc* procedure is invoked by **Tk_GetOptionValue** and **Tk_GetOptionInfo** to retrieve a Tcl_Obj representation of the internal representation of an option. The *clientData* argument is a copy of the *clientData* field in the Tk_ObjCustomOption structure. *Tkwin* is a copy of the *tkwin*

argument to **Tk_GetOptionValue** or **Tk_GetOptionInfo**. *RecordPtr* is a pointer to the beginning of the widget record to query. *InternalOffset* is the offset in bytes from the beginning of the widget record to the location where the internal representation of the option value is stored. *GetProc* must return a pointer to a Tcl_Obj representing the value of the option.

The *restoreProc* procedure is invoked by **Tk_RestoreSavedOptions** to restore a previously saved internal representation of a custom option value. The *clientData* argument is a copy of the *clientData* field in the Tk_ObjCustomOption structure. *Tkwin* is a copy of the *tkwin* argument to **Tk_GetOptionValue** or **Tk_GetOptionInfo**. *InternalPtr* is a pointer to the location where internal representation of the option value is stored. *SaveInternalPtr* is a pointer to the saved value. *RestoreProc* must copy the value from *saveInternalPtr* to *internalPtr* to restore the value. *RestoreProc* need not free any memory associated with either *internalPtr* or *saveInternalPtr*; *freeProc* will be invoked to free that memory if necessary. *RestoreProc* has no return value.

The *freeProc* procedure is invoked by **Tk_SetOptions** and **Tk_FreeSavedOptions** to free any storage allocated for the internal representation of a custom option. The *clientData* argument is a copy of the *clientData* field in the Tk_ObjCustomOption structure. *Tkwin* is a copy of the *tkwin* argument to **Tk_GetOptionValue** or **Tk_GetOptionInfo**. *InternalPtr* is a pointer to the location where the internal representation of the option value is stored. The *freeProc* must free any storage associated with the option. *FreeProc* has no return value.

Keywords: SetOptions, anchor, bitmap, boolean, border, color, configuration option, cursor, double, font, integer, justify, pixels, relief, screen distance, synonym

Copyright (c) 1998 Sun Microsystems, Inc.

SetVisual

Name

Tk_SetWindowVisual - change visual characteristics of window

Synopsis

#include <tk.h>

int
Tk_SetWindowVisual(*tkwin*, *visual*, *depth*, *colormap*)

Arguments

Tk_Window *tkwin* (in)
Token for window.

Visual **visual* (in)
New visual type to use for *tkwin*.

int *depth* (in)
Number of bits per pixel desired for *tkwin*.

Colormap *colormap* (in)
New colormap for *tkwin*, which must be compatible with *visual* and *depth*.

Description

When Tk creates a new window it assigns it the default visual characteristics (visual, depth, and colormap) for its screen. **Tk_SetWindowVisual** may be called to change them.

Tk_SetWindowVisual must be called before the window has actually been created in X (e.g. before **Tk_MapWindow** or **Tk_MakeWindowExist** has been invoked for the window). The safest thing is to call **Tk_SetWindowVisual** immediately after calling **Tk_CreateWindow**. If *tkwin* has already been created before **Tk_SetWindowVisual** is called then it returns 0 and does not make any changes; otherwise it returns 1 to signify that the operation completed successfully.

Note: **Tk_SetWindowVisual** should not be called if you just want to change a window's colormap without changing its visual or depth; call **Tk_SetWindowColormap** instead.

Keywords: SetVisual, colormap, depth, visual

Copyright (c) 1992 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

StrictMotif

Name

Tk_StrictMotif - Return value of tk_strictMotif variable

Synopsis

#include <tk.h>

int
Tk_StrictMotif(*tkwin*)

Arguments

Tk_Window *tkwin* (in)
Token for window.

Description

This procedure returns the current value of the **tk_strictMotif** variable in the interpreter associated with *tkwin*'s application. The value is returned as an integer that is either 0 or 1. 1 means that strict Motif compliance has been requested, so anything that is not part of the Motif specification should be avoided. 0 means that "Motif-like" is good enough, and extra features are welcome.

This procedure uses a link to the Tcl variable to provide much faster access to the variable's value than could be had by calling **Tcl_GetVar**.

Keywords: StrictMotif, Motif compliance, tk_strictMotif variable

Copyright (c) 1995-1996 Sun Microsystems, Inc.

TextLayout

Name

Tk_ComputeTextLayout, Tk_FreeTextLayout, Tk_DrawTextLayout, Tk_UnderlineTextLayout, Tk_PointToChar, Tk_CharBbox, Tk_DistanceToTextLayout, Tk_IntersectTextLayout, Tk_TextLayoutToPostscript - routines to measure and display single-font, multi-line, justified text.

Synopsis

#include <tk.h>

Tk_TextLayout
Tk_ComputeTextLayout(*tkfont*, *string*, *numChars*, *wrapLength*, *justify*, *flags*, *widthPtr*, *heightPtr*)

void
Tk_FreeTextLayout(*layout*)

void
Tk_DrawTextLayout(*display*, *drawable*, *gc*, *layout*, *x*, *y*, *firstChar*, *lastChar*)

void
Tk_UnderlineTextLayout(*display*, *drawable*, *gc*, *layout*, *x*, *y*, *underline*)

int
Tk_PointToChar(*layout*, *x*, *y*)

int
Tk_CharBbox(*layout*, *index*, *xPtr*, *yPtr*, *widthPtr*, *heightPtr*)

int
Tk_DistanceToTextLayout(*layout*, *x*, *y*)

int
Tk_IntersectTextLayout(*layout*, *x*, *y*, *width*, *height*)

void
Tk_TextLayoutToPostscript(*interp*, *layout*)

Arguments

Tk_Font *tkfont* (in)
Font to use when constructing and displaying a text layout. The *tkfont* must remain valid for the lifetime of the text layout. Must have been returned by a previous call to **Tk_GetFont**.

const char **string* (in)
Potentially multi-line string whose dimensions are to be computed and stored in the text layout. The *string* must remain valid for the lifetime of the text layout.

int *numChars* (in)
The number of characters to consider from *string*. If *numChars* is less than 0, then assumes *string* is null terminated and uses **Tcl_NumUtfChars** to determine the length of *string*.

int *wrapLength* (in)
Longest permissible line length, in pixels. Lines in *string* will automatically be broken at word boundaries and wrapped when they reach this length. If *wrapLength* is too small for even a single character to fit on a line, it will be expanded to allow one character to fit on each line. If *wrapLength* is <= 0, there is no automatic wrapping; lines will get as long as they need to be and only wrap if a newline/return character is encountered.

Tk_Justify *justify* (in)
How to justify the lines in a multi-line text layout. Possible values are **TK_JUSTIFY_LEFT**, **TK_JUSTIFY_CENTER**, or **TK_JUSTIFY_RIGHT**. If the text layout only occupies a single line, then *justify* is irrelevant.

int *flags* (in)
Various flag bits OR-ed together. **TK_IGNORE_TABS** means that tab characters should not be

expanded to the next tab stop. **TK_IGNORE_NEWLINES** means that newline/return characters should not cause a line break. If either tabs or newlines/returns are ignored, then they will be treated as regular characters, being measured and displayed in a platform-dependent manner as described in **Tk_MeasureChars**, and will not have any special behaviors.

int *_widthPtr_ (out)
If non-NULL, filled with either the width, in pixels, of the widest line in the text layout, or the width, in pixels, of the bounding box for the character specified by *index*.

int *_heightPtr_ (out)
If non-NULL, filled with either the total height, in pixels, of all the lines in the text layout, or the height, in pixels, of the bounding box for the character specified by *index*.

Tk_TextLayout _layout_ (in)
A token that represents the cached layout information about the single-font, multi-line, justified piece of text. This token is returned by **Tk_ComputeTextLayout**.

Display *_display_ (in)
Display on which to draw.

Drawable _drawable_ (in)
Window or pixmap in which to draw.

GC _gc_ (in)
Graphics context to use for drawing text layout. The font selected in this GC must correspond to the *tkfont* used when constructing the text layout.

int _x, y_ (in)
Point, in pixels, at which to place the upper-left hand corner of the text layout when it is being drawn, or the coordinates of a point (with respect to the upper-left hand corner of the text layout) to check against the text layout.

int _firstChar_ (in)
The index of the first character to draw from the given text layout. The number 0 means to draw from the beginning.

int _lastChar_ (in)
The index of the last character up to which to draw. The character specified by *lastChar* itself will not be drawn. A number less than 0 means to draw all characters in the text layout.

int _underline_ (in)
Index of the single character to underline in the text layout, or a number less than 0 for no underline.

int _index_ (in)
The index of the character whose bounding box is desired. The bounding box is computed with respect to the upper-left hand corner of the text layout.

int *_xPtr, *yPtr_ (out)
Filled with the upper-left hand corner, in pixels, of the bounding box for the character specified by *index*. Either or both *xPtr* and *yPtr* may be NULL, in which case the corresponding value is not calculated.

int _width, height_ (in)
Specifies the width and height, in pixels, of the rectangular area to compare for intersection against the text layout.

Tcl_Interp ****interp* **(out)
Postscript code that will print the text layout is appended to the result of interpreter *interp*.

Description

These routines are for measuring and displaying single-font, multi-line, justified text. To measure and display simple single-font, single-line strings, refer to the documentation for **Tk_MeasureChars**. There is no programming interface in the core of Tk that supports multi-font, multi-line text; support for that behavior must be built on top of simpler layers. Note that unlike the lower level text display routines, the functions described here all operate on character-oriented lengths and indices rather than byte-oriented values. See the description of **Tcl_UtfAtIndex** for more details on converting between character and byte offsets.

The routines described here are built on top of the programming interface described in the **Tk_MeasureChars** documentation. Tab characters and newline/return characters may be treated specially by these procedures, but all other characters are passed through to the lower level.

Tk_ComputeTextLayout computes the layout information needed to display a single-font, multi-line, justified *string* of text and returns a Tk_TextLayout token that holds this information. This token is used in subsequent calls to procedures such as **Tk_DrawTextLayout**, **Tk_DistanceToTextLayout**, and **Tk_FreeTextLayout**. The *string* and *tkfont* used when computing the layout must remain valid for the lifetime of this token.

Tk_FreeTextLayout is called to release the storage associated with *layout* when it is no longer needed. A *layout* should not be used in any other text layout procedures once it has been released.

Tk_DrawTextLayout uses the information in *layout* to display a single-font, multi-line, justified string of text at the specified location.

Tk_UnderlineTextLayout uses the information in *layout* to display an underline below an individual character. This procedure does not draw the text, just the underline. To produce natively underlined text, an underlined font should be constructed and used. All characters, including tabs, newline/return characters, and spaces at the ends of lines, can be underlined using this method. However, the underline will never be drawn outside of the computed width of *layout*; the underline will stop at the edge for any character that would extend partially outside of *layout*, and the underline will not be visible at all for any character that would be located completely outside of the layout.

Tk_PointToChar uses the information in *layout* to determine the character closest to the given point. The point is specified with respect to the upper-left hand corner of the *layout*, which is considered to be located at (0, 0). Any point whose *y*-value is less that 0 will be considered closest to the first character in the text layout; any point whose *y*-value is greater than the height of the text layout will be considered closest to the last character in the text layout. Any point whose *x*-value is less than 0 will be considered closest to the first character on that line; any point whose *x*-value is greater than the width of the text layout will be considered closest to the last character on that line. The return value is the index of the character that was closest to the point, or one more than the index of any character (to indicate that the point was after the end of the string and that the corresponding caret would be at the end of the string). Given a *layout* with no characters, the value 0 will always be returned, referring to a hypothetical zero-width placeholder character.

Tk_CharBbox uses the information in *layout* to return the bounding box for the character specified by *index*. The width of the bounding box is the advance width of the character, and does not include any left or right bearing. Any character that extends partially outside of *layout* is considered to be truncated at the edge. Any character that would be located completely outside of *layout* is considered to be zero-width and pegged against the edge. The height of the bounding box is the line height for this font, extending from the top of the ascent to the bottom of the descent; information about the actual height of individual letters is not available. For measurement purposes, a *layout* that contains no characters is considered to contain a single zero-width placeholder character at index 0. If *index* was not a valid character index, the return value is 0 and **xPtr*, **yPtr*, **widthPtr*, and **heightPtr* are unmodified. Otherwise, if *index* did specify a valid, the return value is non-zero, and **xPtr*, **yPtr*, **widthPtr*, and **heightPtr* are filled with the bounding box information for the character. If any of *xPtr*, *yPtr*, *widthPtr*, or *heightPtr* are NULL, the corresponding value is not calculated or stored.

Tk_DistanceToTextLayout computes the shortest distance in pixels from the given point (*x, y*) to the characters in *layout*. Newline/return characters and non-displaying space characters that occur at the end of individual lines in the text layout are ignored for hit detection purposes, but tab characters are not. The return value is 0 if the point actually hits the *layout*. If the point did not hit the *layout* then the return value is the distance in pixels from the point to the *layout*.

Tk_IntersectTextLayout determines whether a *layout* lies entirely inside, entirely outside, or overlaps a given rectangle. Newline/return characters and non-displaying space characters that occur at the end of individual lines in the *layout* are ignored for intersection calculations. The return value is -1 if the *layout* is entirely outside of the rectangle, 0 if it overlaps, and 1 if it is entirely inside of the rectangle.

Tk_TextLayoutToPostscript outputs code consisting of a Postscript array of strings that represent the individual lines in *layout*. It is the responsibility of the caller to take the Postscript array of strings and add some Postscript function operate on the array to render each of the lines. The code that represents the Postscript array of strings is appended to interpreter *interp*'s result.

Display model

When measuring a text layout, space characters that occur at the end of a line are ignored. The space characters still exist and the insertion point can be positioned amongst them, but their additional width is ignored when justifying lines or returning the total width of a text layout. All end-of-line space characters are considered to be attached to the right edge of the line; this behavior is logical for left-justified text and reasonable for center-justified text, but not very useful when editing right-justified text. Spaces are considered variable width characters; the first space that extends past the edge of the text layout is clipped to the edge, and any subsequent spaces on the line are considered zero width and pegged against the edge. Space characters that occur in the middle of a line of text are not suppressed and occupy their normal space width.

Tab characters are not ignored for measurement calculations. If wrapping is turned on and there are enough tabs on a line, the next tab will wrap to the beginning of the next line. There are some possible strange interactions between tabs and justification; tab positions are calculated and the line length computed in a left-justified world, and then the whole resulting line is shifted so it is centered or right-justified, causing the tab columns not to align any more.

When wrapping is turned on, lines may wrap at word breaks (space or tab characters) or newline/returns. A dash or hyphen character in the middle of a word is not considered a word break. **Tk_ComputeTextLayout** always attempts to place at least one word on each line. If it cannot because the *wrapLength* is too small, the word will be broken and as much as fits placed on the line and the rest on subsequent line(s). If *wrapLength* is so small that not even one character can fit on a given line, the *wrapLength* is ignored for that line and one character will be placed on the line anyhow. When wrapping is turned off, only newline/return characters may cause a line break.

When a text layout has been created using an underlined *tkfont*, then any space characters that occur at the end of individual lines, newlines/returns, and tabs will not be displayed underlined when **Tk_DrawTextLayout** is called, because those characters are never actually drawn - they are merely placeholders maintained in the *layout*.

Keywords: TextLayout, font

Copyright (c) 1996 Sun Microsystems, Inc.

TkInitStubs

Name

Tk_InitStubs - initialize the Tk stubs mechanism

Synopsis

#include <tk.h>

const char *
Tk_InitStubs(*interp*, *version*, *exact*)

Arguments

Tcl_Interp *_interp_ (in)
Tcl interpreter handle.

char *_version_ (in)
A version string consisting of one or more decimal numbers separated by dots.

int _exact_ (in)
Non-zero means that only the particular Tk version specified by *version* is acceptable. Zero means that versions newer than *version* are also acceptable as long as they have the same major version number as *version*.

Introduction

The Tcl stubs mechanism defines a way to dynamically bind extensions to a particular Tcl implementation at run time. the stubs mechanism requires no changes to applications incorporating Tcl/Tk interpreters. Only developers creating C-based Tcl/Tk extensions need to take steps to use the stubs mechanism with their extensions. See the **Tcl_InitStubs** page for more information.

Enabling the stubs mechanism for a Tcl/Tk extension requires the following steps:

1)
Call **Tcl_InitStubs** in the extension before calling any other Tcl functions.

2)

Call **Tk_InitStubs** if the extension before calling any other Tk functions.

2)
Define the **USE_TCL_STUBS** and the **USE_TK_STUBS** symbols. Typically, you would include the **-DUSE_TCL_STUBS** and the **-DUSE_TK_STUBS** flags when compiling the extension.

3)
Link the extension with the Tcl and Tk stubs libraries instead of the standard Tcl and Tk libraries. On Unix platforms, the library names are *libtclstub8.4.a* and *libtkstub8.4.a*; on Windows platforms, the library names are *tclstub84.lib* and *tkstub84.lib*. Adjust the library names with appropriate version number but note that the extension may only be used with versions of Tcl/Tk that have that version number or higher.

Description

Tk_InitStubs attempts to initialize the Tk stub table pointers and ensure that the correct version of Tk is loaded. In addition to an interpreter handle, it accepts as arguments a version number and a Boolean flag indicating whether the extension requires an exact version match or not. If *exact* is 0, then the extension is indicating that newer versions of Tk are acceptable as long as they have the same major version number as *version*; non-zero means that only the specified *version* is acceptable. **Tcl_InitStubs** returns a string containing the actual version of Tk satisfying the request, or NULL if the Tk version is not acceptable, does not support the stubs mechanism, or any other error condition occurred.

Keywords: TkInitStubs, stubs

Copyright (c) 1999 Scriptics Corporation

Tk_Init

Name

Tk_Init, Tk_SafeInit - add Tk to an interpreter and make a new Tk application.

Synopsis

#include <tk.h>

int
Tk_Init(*interp*)

int
Tk_SafeInit(*interp*)

Arguments

Tcl_Interp ****interp* **(in)
Interpreter in which to load Tk. Tk should not already be loaded in this interpreter.

Description

Tk_Init is the package initialization procedure for Tk. It is normally invoked by the **Tcl_AppInit** procedure for an application or by the **load** command. **Tk_Init** adds all of Tk's commands to *interp* and creates a new Tk application, including its main window. If the initialization is successful **Tk_Init** returns **TCL_OK**; if there is an error it returns **TCL_ERROR**. **Tk_Init** also leaves a result or error message in interpreter *interp*'s result.

If there is a variable **argv** in *interp*, **Tk_Init** treats the contents of this variable as a list of options for the new Tk application. The options may have any of the forms documented for the **wish** application (in fact, **wish** uses Tk_Init to process its command-line arguments).

Tk_SafeInit is identical to **Tk_Init** except that it removes all Tk commands that are considered unsafe. Those commands and the reasons for their exclusion are:

bell
Continuous ringing of the bell is a nuisance.

clipboard
A malicious script could replace the contents of the clipboard with the string "**rm -r ***" and lead to surprises when the contents of the clipboard are pasted.

grab
Grab can be used to block the user from using any other applications.

menu
Menus can be used to cover the entire screen and to steal input from the user.

selection
See clipboard.

send
Send can be used to cause unsafe interpreters to execute commands.

tk
The tk command recreates the send command, which is unsafe.

tkwait
Tkwait can block the containing process forever

toplevel
Toplevels can be used to cover the entire screen and to steal input from the user.

wm
If toplevels are ever allowed, wm can be used to remove decorations, move windows around, etc.

Keywords: Tk_Init, safe, application, initialization, load, main window

Copyright (c) 1995-1996 Sun Microsystems, Inc.

Tk_Main

Name

Tk_Main - main program for Tk-based applications

Synopsis

#include <tk.h>

Tk_Main(*argc*, *argv*, *appInitProc*)

Arguments

int *argc* (in)
Number of elements in *argv*.

char **argv[]* (in)
Array of strings containing command-line arguments. On Windows, when using -DUNICODE,

the parameter type changes to wchar_t *.

Tcl_AppInitProc ****appInitProc* **(in)
Address of an application-specific initialization procedure. The value for this argument is usually **Tcl_AppInit**.

Description

Tk_Main acts as the main program for most Tk-based applications. Starting with Tk 4.0 it is not called **main** anymore because it is part of the Tk library and having a function **main** in a library (particularly a shared library) causes problems on many systems. Having **main** in the Tk library would also make it hard to use Tk in C++ programs, since C++ programs must have special C++ **main** functions.

Normally each application contains a small **main** function that does nothing but invoke **Tk_Main** . **Tk_Main** then does all the work of creating and running a **wish**-like application.

When it is has finished its own initialization, but before it processes commands, **Tk_Main** calls the procedure given by the *appInitProc* argument. This procedure provides a "hook" for the application to perform its own initialization, such as defining application-specific commands. The procedure must have an interface that matches the type **Tcl_AppInitProc**:

```
typedef int Tcl_AppInitProc(
        Tcl_Interp *interp);
```

AppInitProc is almost always a pointer to **Tcl_AppInit**; for more details on this procedure, see the documentation for **Tcl_AppInit**.

Tk_Main functions much the same as **Tcl_Main**. In particular, **Tk_Main** supports both an interactive mode and a startup script mode, with the file name and encoding of a startup script under the control of the **Tcl_SetStartupScript** and **Tcl_GetStartupScript** routines. However it calls **Tk_MainLoop** after processing any supplied script, and in interactive uses events registered with **Tcl_CreateFileHandler** to process user input.

Keywords: Tk_Main, application-specific initialization, command-line arguments, main program

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

WindowId

Name

Tk_WindowId, Tk_Parent, Tk_Display, Tk_DisplayName, Tk_ScreenNumber, Tk_Screen, Tk_X, Tk_Y, Tk_Width, Tk_Height, Tk_Changes, Tk_Attributes, Tk_IsContainer, Tk_IsEmbedded, Tk_IsMapped, Tk_IsTopLevel, Tk_ReqWidth, Tk_ReqHeight, Tk_MinReqWidth, Tk_MinReqHeight, Tk_InternalBorderLeft, Tk_InternalBorderRight, Tk_InternalBorderTop, Tk_InternalBorderBottom, Tk_Visual, Tk_Depth, Tk_Colormap, Tk_Interp - retrieve information from Tk's local data structure

Synopsis

#include <tk.h>

Window
Tk_WindowId(*tkwin*)

Tk_Window

Tk_Parent(*tkwin*)

Display *
Tk_Display(*tkwin*)

const char *
Tk_DisplayName(*tkwin*)

int
Tk_ScreenNumber(*tkwin*)

Screen *
Tk_Screen(*tkwin*)

int
Tk_X(*tkwin*)

int
Tk_Y(*tkwin*)

int
Tk_Width(*tkwin*)

int
Tk_Height(*tkwin*)

XWindowChanges *
Tk_Changes(*tkwin*)

XSetWindowAttributes *
Tk_Attributes(*tkwin*)

int
Tk_IsContainer(*tkwin*)

int
Tk_IsEmbedded(*tkwin*)

int
Tk_IsMapped(*tkwin*)

int
Tk_IsTopLevel(*tkwin*)

int
Tk_ReqWidth(*tkwin*)

int
Tk_ReqHeight(*tkwin*)

int
Tk_MinReqWidth(*tkwin*)

int
Tk_MinReqHeight(*tkwin*)

int
Tk_InternalBorderLeft(*tkwin*)

int
Tk_InternalBorderRight(*tkwin*)

int
Tk_InternalBorderTop(*tkwin*)

int
Tk_InternalBorderBottom(*tkwin*)

Visual *
Tk_Visual(*tkwin*)

int
Tk_Depth(*tkwin*)

Colormap
Tk_Colormap(*tkwin*)

Tcl_Interp *
Tk_Interp(*tkwin*)

Arguments

Tk_Window *tkwin* (in)
Token for window.

Description

Tk_WindowId and the other names listed above are all macros that return fields from Tk's local data structure for *tkwin*. None of these macros requires any interaction with the server; it is safe to assume that all are fast.

Tk_WindowId returns the X identifier for *tkwin*, or **NULL** if no X window has been created for *tkwin* yet.

Tk_Parent returns Tk's token for the logical parent of *tkwin*. The parent is the token that was specified when *tkwin* was created, or NULL for main windows.

Tk_Interp returns the Tcl interpreter associated with a *tkwin* or NULL if there is an error.

Tk_Display returns a pointer to the Xlib display structure corresponding to *tkwin*. **Tk_DisplayName** returns an ASCII string identifying *tkwin*'s display. **Tk_ScreenNumber** returns the index of *tkwin*'s screen among all the screens of *tkwin*'s display. **Tk_Screen** returns a pointer to the Xlib structure corresponding to *tkwin*'s screen.

Tk_X, **Tk_Y**, **Tk_Width**, and **Tk_Height** return information about *tkwin*'*s* location within its parent and its size. The location information refers to the upper-left pixel in the window, or its border if there is one. The width and height information refers to the interior size of the window, not including any border. **Tk_Changes** returns a pointer to a structure containing all of the above information plus a few other fields. **Tk_Attributes** returns a pointer to an XSetWindowAttributes structure describing all of the attributes of the *tkwin*'s window, such as background pixmap, event mask, and so on (Tk keeps track of all this information as it is changed by the application). Note: it is essential that applications use Tk procedures like **Tk_ResizeWindow** instead of X procedures like **XResizeWindow**, so that Tk can keep its data structures up-to-date.

Tk_IsContainer returns a non-zero value if *tkwin* is a container, and that some other application may be embedding itself inside *tkwin*.

Tk_IsEmbedded returns a non-zero value if *tkwin* is not a free-standing window, but rather is

embedded in some other application.

Tk_IsMapped returns a non-zero value if *tkwin* is mapped and zero if *tkwin* is not mapped.

Tk_IsTopLevel returns a non-zero value if *tkwin* is a top-level window (its X parent is the root window of the screen) and zero if *tkwin* is not a top-level window.

Tk_ReqWidth and **Tk_ReqHeight** return information about the window's requested size. These values correspond to the last call to **Tk_GeometryRequest** for *tkwin*.

Tk_MinReqWidth and **Tk_MinReqHeight** return information about the window's minimum requested size. These values correspond to the last call to **Tk_SetMinimumRequestSize** for *tkwin*.

Tk_InternalBorderLeft, **Tk_InternalBorderRight**, **Tk_InternalBorderTop** and **Tk_InternalBorderBottom** return the width of one side of the internal border that has been requested for *tkwin*, or 0 if no internal border was requested. The return value is simply the last value passed to **Tk_SetInternalBorder** or **Tk_SetInternalBorderEx** for *tkwin*.

Tk_Visual, **Tk_Depth**, and **Tk_Colormap** return information about the visual characteristics of a window. **Tk_Visual** returns the visual type for the window, **Tk_Depth** returns the number of bits per pixel, and **Tk_Colormap** returns the current colormap for the window. The visual characteristics are normally set from the defaults for the window's screen, but they may be overridden by calling **Tk_SetWindowVisual**.

Keywords: WindowId, attributes, colormap, depth, display, height, geometry manager, identifier, mapped, requested size, screen, top-level, visual, width, window, x, y

Copyright (c) 1990-1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

ttk_Geometry

Name

Ttk_MakeBox, Ttk_PadBox, Ttk_ExpandBox, Ttk_PackBox, Ttk_StickBox, Ttk_PlaceBox, Ttk_BoxContains, Ttk_MakePadding, Ttk_UniformPadding, Ttk_AddPadding, Ttk_RelievePadding, Ttk_GetPaddingFromObj, Ttk_GetBorderFromObj, Ttk_GetStickyFromObj - Tk themed geometry utilities

Synopsis

#include <tkTheme.h>

Ttk_Box
Ttk_MakeBox(int *x*, int *y*, int *width*, int *height*);

Ttk_Box
Ttk_PadBox(Ttk_Box *parcel*, Ttk_Padding *padding*);

Ttk_Box
Ttk_ExpandBox(Ttk_Box *parcel*, Ttk_Padding *padding*);

Ttk_Box
Ttk_PackBox(Ttk_Box **cavity*, int *width*, int *height*, Ttk_Side *side*);

Ttk_Box
Ttk_StickBox(Ttk_Box *parcel*, int *width*, int *height*, unsigned *sticky*);

Ttk_Box
Ttk_PlaceBox(Ttk_Box *_cavity_, int _width_, int _height_, Ttk_Side _side_, unsigned _sticky_);

Ttk_Box
Ttk_AnchorBox(Ttk_Box _parcel_, int _width_, int _height_, Tk_Anchor _anchor_);

Ttk_Padding
Ttk_MakePadding(short _left_, short _top_, short _right_, short _bottom_);

Ttk_Padding
Ttk_UniformPadding(short _border_);

Ttk_Padding
Ttk_AddPadding(Ttk_Padding _padding1_, Ttk_Padding _padding2_;

Ttk_Padding
Ttk_RelievePadding(Ttk_Padding _padding_, int _relief_);

int
Ttk_BoxContains(Ttk_Box _box_, int _x_, int _y_);

int
Ttk_GetPaddingFromObj(Tcl_Interp *_interp_, Tk_Window _tkwin_, Tcl_Obj *_objPtr_, Ttk_Padding * _padding_rtn_);

int
Ttk_GetBorderFromObj(Tcl_Interp *_interp_, Tcl_Obj *_objPtr_, Ttk_Padding *_padding_rtn_);

int
Ttk_GetStickyFromObj(Tcl_Interp *_interp_, Tcl_Obj *_objPtr_, int *_sticky_rtn_);

Arguments

Tk_Anchor _anchor_ (in)
One of the symbolic constants **TK_ANCHOR_N**, **TK_ANCHOR_NE**, etc. See _Tk_GetAnchorFromObj(3)_.

Ttk_Box * _cavity_ (in/out)
A rectangular region from which a parcel is allocated.

short _border_ (in)
Extra padding (in pixels) to add uniformly to each side of a region.

short _bottom_ (in)
Extra padding (in pixels) to add to the bottom of a region.

Ttk_Box _box_ (in)
Specifies a rectangular region.

Ttk_Box * _box_rtn_ (out)
A rectangular region.

int _height_ (in)
The height in pixels of a region.

Tcl_Interp * _interp_ (in)
Used to store error messages.

int _left_ (in)
Extra padding (in pixels) to add to the left side of a region.

Tcl_Obj * _objPtr_ (in)

String value contains a symbolic name to be converted to an enumerated value or bitmask. Internal rep may be be modified to cache corresponding value.

Ttk_Padding *padding* (in)
Extra padding to add on the inside of a region.

Ttk_Padding * *padding_rtn* (out)
Padding present in the inside of a region.

Ttk_Box *parcel* (in)
A rectangular region, allocated from a cavity.

int *relief* (in)
One of the standard Tk relief options (**TK_RELIEF_RAISED**, **TK_RELIEF_SUNKEN**, etc.). See **Tk_GetReliefFromObj**.

short *right* (in)
Extra padding (in pixels) to add to the right side of a region.

Ttk_Side *side* (in)
One of **TTK_SIDE_LEFT**, **TTK_SIDE_TOP**, **TTK_SIDE_RIGHT**, or **TTK_SIDE_BOTTOM**.

unsigned *sticky* (in)
A bitmask containing one or more of the bits **TTK_STICK_W** (west, or left), **TTK_STICK_E** (east, or right), **TTK_STICK_N** (north, or top), and **TTK_STICK_S** (south, or bottom). **TTK_FILL_X** is defined as a synonym for (**TTK_STICK_W**|**TTK_STICK_E**), **TTK_FILL_Y** is a synonym for (**TTK_STICK_N**|**TTK_STICK_S**), and **TTK_FILL_BOTH** is a synonym for (**TTK_FILL_X**|**TTK_FILL_Y**). See also: *grid(n)*.

Tk_Window *tkwin* (in)
Window whose screen geometry determines the conversion between absolute units and pixels.

short *top* (in)
Extra padding at the top of a region.

int *width* (in)
The width in pixels of a region.

int *x* (in)
X coordinate of upper-left corner of region.

int *y* (in)
Y coordinate of upper-left corner of region.

Boxes

The **Ttk_Box** structure represents a rectangular region of a window:

```
typedef struct {
    int x;
    int y;
    int width;
    int height;
} Ttk_Box;
```

All coordinates are relative to the window.

Ttk_MakeBox is a convenience routine that constructs a **Ttk_Box** structure representing a

region *width* pixels wide, *height* pixels tall, at the specified *x, y* coordinates.

Ttk_PadBox returns a new box located inside the specified *parcel*, shrunken according to the left, top, right, and bottom margins specified by *padding*.

Ttk_ExpandBox is the inverse of **Ttk_PadBox**: it returns a new box surrounding the specified *parcel*, expanded according to the left, top, right, and bottom margins specified by *padding*.

Ttk_PackBox allocates a parcel *width* by *height* pixels wide on the specified *side* of the *cavity*, and shrinks the *cavity* accordingly.

Ttk_StickBox places a box with the requested *width* and *height* inside the *parcel* according to the *sticky* bits.

Ttk_PlaceBox combines **Ttk_PackBox** and **Ttk_StickBox**: it allocates a parcel on the specified *side* of the *cavity*, places a box of the requested size inside the parcel according to *sticky*, and shrinks the *cavity*.

Ttk_AnchorBox places a box with the requested *width* and *height* inside the *parcel* according to the specified *anchor* option.

Ttk_BoxContains tests if the specified *x, y* coordinate lies within the rectangular region *box*.

Paddding

The **Ttk_Padding** structure is used to represent borders, internal padding, and external margins:

```
typedef struct {
    short left;
    short top;
    short right;
    short bottom;
} Ttk_Padding;
```

Ttk_MakePadding is a convenience routine that constructs a **Ttk_Padding** structure with the specified left, top, right, and bottom components.

Ttk_UniformPadding constructs a **Ttk_Padding** structure with all components equal to the specified *border*.

Ttk_AddPadding adds two **Ttk_Padding**s together and returns a combined padding containing the sum of the individual padding components.

Ttk_RelievePadding adds an extra 2 pixels of padding to *padding* according to the specified *relief*. If *relief* is **TK_RELIEF_SUNKEN**, adds two pixels at the top and left so the inner region is shifted down and to the left. If it is **TK_RELIEF_RAISED**, adds two pixels at the bottom and right so the inner region is shifted up and to the right. Otherwise, adds 1 pixel on all sides. This is typically used in element geometry procedures to simulate a "pressed-in" look for pushbuttons.

Conversion routines

Ttk_GetPaddingFromObj converts the string in *objPtr* to a **Ttk_Padding** structure. The string representation is a list of up to four length specifications "*left top right bottom*". If fewer than four elements are specified, *bottom* defaults to *top*, *right* defaults to *left*, and *top* defaults to *left*. See **Tk_GetPixelsFromObj(3)** for the syntax of length specifications.

Ttk_GetBorderFromObj is the same as **Ttk_GetPaddingFromObj** except that the lengths are specified as integers (i.e., resolution-dependent values like *3m* are not allowed).

Ttk_GetStickyFromObj converts the string in *objPtr* to a *sticky* bitmask. The string contains zero or more of the characters **n**, **s**, **e**, or **w**.

Keywords: ttk_Geometry, geometry, padding, margins, box, region, sticky, relief

Copyright (c) 2004 Joe English

ttk_Theme

Name

Ttk_CreateTheme, Ttk_GetTheme, Ttk_GetDefaultTheme, Ttk_GetCurrentTheme - create and use Tk themes.

Synopsis

```
Ttk_Theme Ttk_CreateTheme(interp, name, parentTheme);
Ttk_Theme Ttk_GetTheme(interp, name);
Ttk_Theme Ttk_GetDefaultTheme(interp);
Ttk_Theme Ttk_GetCurrentTheme(interp);
```

Arguments

Tcl_Interp * *interp* (in)
The Tcl interpreter in which to register/query available themes.

Ttk_Theme *parentTheme* (in)
Fallback or parent theme from which the new theme will inherit elements and layouts.

const char * *name* (in)
The name of the theme.

Description

Keywords: ttk_Theme

Copyright (c) 2003 Joe English

License

Tcl C API license terms

This software is copyrighted by the Regents of the University of California, Sun Microsystems, Inc., Scriptics Corporation, ActiveState Corporation and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT,

INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7014 (b) (3) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Tk C API license terms

This software is copyrighted by the Regents of the University of California, Sun Microsystems, Inc., Scriptics Corporation, ActiveState Corporation, Apple Inc. and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (b) (3) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Index

3

3D 284
3DBorder 284

A

abort 98, 200
above 398
Access 8
access 126
access point 196
add 285
AddErrInfo 12
AddOption 285
alias 36
Alloc 13
allocation 13
AllowExc 14
alpha 135
anchor 303, 414
anchor position 345
append 70, 155, 230, 248, 294
AppInit 15
application 15, 140, 381, 382, 400, 422
application-specific initialization 235, 258, 423
argument 15, 203
arguments 396
array 233
AssocData 17
association 17
Async 19
asynchronous event 19
atom 374, 381
attributes 306, 426

B

BackgdErr 20
background 20, 72, 284
Backslash 21
backslash 238, 278
backslash substitution 209, 249
below 398
beta 135
bevel 365
bgerror 20

binary 126
binary code 156
binary data 23
binary library 216
bind 54, 64, 307, 311, 336
binding 287
BindTable 287
bitmap 289, 303, 349, 414
blocking 50, 52, 196
boolean 22, 99, 101, 130, 152, 303, 414
BoolObj 22
border 284, 303, 306, 414
box 430
braces 209
break 14
buffered I/O 196
butt 350
byte array 23
ByteArrObj 23

C

cache 381
callback 24, 52, 56, 65, 77, 78, 98, 149, 307, 310, 311, 336, 378, 385, 397, 403
CallDel 24
Cancel 25
canvas 289, 291, 293, 326
CanvPsY 289
CanvTkwin 291
CanvTxtInfo 293
cap style 303, 350
caret 400
case 263
center 366
centimeters 368
channel 27, 52, 131, 196, 199
channel closing 52
channel driver 50, 196, 225
channel registration 50
channel type 50, 225
child 36, 72
ChnlStack 27
Class 31
class 285, 367, 401, 403
classification 273
cleanup 24, 98
clear 294, 295

client 199
clientData 265, 270
Clipboard 294
clock 65, 397
ClrSelect 295
CmdCmplt 31
color 284, 289, 303, 306, 354, 414
colormap 351, 376, 415, 426
command 15, 36, 54, 58, 64, 68, 92, 165, 167, 209, 214, 230, 265, 279
command line 396
command substitution 249
command-line arguments 258, 423
commands 149
complete command 31
compress 255
compression 27
Concat 32
concat 248
concatenate 32, 248
condition variable 262
configuration 216
configuration option 414
configuration options 303
configure 306
ConfigWidg 303
ConfigWind 306
console 308
constructor 31, 160
container 383
containing 306
continue 14
conversion 130, 213, 358
convert 89, 238, 368
coordinates 306, 371
CoordToWin 306
create 54, 58, 64, 68, 334
CrtAlias 36
CrtChannel 50
CrtChnlHdlr 52
CrtCloseHdlr 52
CrtCmHdlr 307
CrtCommand 54
CrtConsoleChan 308
CrtErrHdlr 310
CrtFileHdlr 56

CrtGenHdlr 311
CrtImgType 316
CrtInterp 58
CrtItemType 326
CrtMathFnc 60
CrtObjCmd 64
CrtPhImgFmt 330
CrtSelHdlr 332
CrtTimerHdlr 65
CrtTrace 68
CrtWindow 334
cursor 303, 357, 400, 414
custom 303

D

dash 358
data 17
date 134
debug 80, 250
decompress 255
defer 78
deferred creation 334
deflate 255
delay 399
delete 24, 54, 58, 64, 68
delete image 334
DeleteImg 334
deletion procedure 17
depressed 284
depth 415, 426
destroy 334
detach 72
DetachPids 72
dict 75
dict value 75
dictionary 75
dictionary value 75
DictObj 75
display 334, 381, 426
DoOneEvent 77
double 79, 99, 101, 130, 303, 414
double type 79
double value 79
double-precision 213
DoubleObj 79
DoWhenIdle 78
DrawFocHlt 335

DString 70
DumpActiveMemory 80
dynamic loading 98
dynamic string 70

E

element 230, 238
embedding 216
Encoding 89
end application 98
end of file 196
Ensemble 92
Environment 93
errno 226
error 12, 20, 200, 310
error code 226
error message 279
error messages 225
Eval 96
evaluate 99, 101
event 77, 177, 214, 287, 307, 310, 311, 336, 378, 381, 399
event queue 177
event sources 177
EventHndlr 336
events 52, 397
exception 14
executable file 126
execute 96, 165, 214
Exit 98
exposed commands 36
expression 60, 99, 101, 209
ExprLong 99
ExprLongObj 101

F

fatal 200
file 56, 96, 239
file events 177
file handle 131
file name 271
filename 239
FileSystem 126
fill 366
filter 399
FindExec 126
FindPhoto 340
floating-point 130, 213

flush 196
focus 291, 293, 326, 335
font 289, 303, 341, 360, 387, 414, 420
FontId 341
format 294, 332, 373
free 13, 70, 146, 212
FreeXId 342

G

geometry 344, 374, 385, 430
geometry manager 383, 426
GeomReq 344
get 373
get variable 233
GetAnchor 345
GetBitmap 349
GetCapStyl 350
GetClrmap 351
GetColor 354
GetCursor 357
GetCwd 127
GetDash 358
GetFont 360
GetGC 361
GetHINSTANCE 361
GetHostName 128
GetHWND 362
GetImage 364
GetIndex 129
GetInt 130
GetJoinStl 365
GetJustify 366
GetOpnFl 131
GetOption 367
GetPixels 368
GetPixmap 369
GetRelief 370
GetRootCrd 371
GetScroll 372
GetSelect 373
GetStdChan 132
GetTime 134
GetUid 374
GetVersion 135
GetVisual 376
GetVRoot 374
global 96, 165

global variables 226
Grab 377
graphics context 361
grid 404
gzip 255

H

HandleEvent 378
handler 19, 52, 56, 65, 77, 307, 310, 311, 332, 336, 378, 397
Hash 140
hash table 75, 140
height 306, 374, 426
hidden commands 36
history 214
home directory 271
hostname 128
HWNDToWindow 377

I

identifier 361, 362, 426
idle 77, 177, 380
idle callback 78
IdToWindow 378
illumination 284
image 340
image file 330
image manager 316, 334, 390
image name 390
image size changes 379
image type 316
images 364, 379
ImgChanged 379
Inactive 380
inches 368
index 129, 155
inflate 255
Init 140
initialization 15, 140, 422
initialization procedure 240
initialized 146
InitStubs 141
input 196
insert 155
insertion cursor 293
instance 316, 361
integer 99, 101, 130, 144, 152, 303, 414
integer type 144

integer value 144
intensity 354
internal representation 79, 144, 155, 182, 185, 248
internal window 334
internationalization 23
InternAtom 381
Interp 146
interp 20, 223
interpreter 14, 15, 17, 24, 36, 54, 58, 68, 140, 146, 149, 214, 230, 233
IntObj 144
invoke 36
item type 291, 326
iteration 75

J

join 239
join style 303, 365
justification 366
justify 303, 414

K

key 17, 140

L

length 155
Limit 149
link 152
linked variable 274
LinkVar 152
list 155, 230, 238
list type 155
list value 155
ListObj 155
Load 156
load 422
loading 156
location 374
lookup 140

M

main loop 381
main program 235, 258, 423
main window 382, 422
MainLoop 381
MaintGeom 383
MainWin 382
major 135
malloc 13, 146

managed 385
ManageGeom 385
map 383, 385
mapped 426
MapWindow 385
margins 430
match 221, 242
mathematical function 60
MeasureChar 387
measurement 341, 387
memory 13, 80, 250
merge 238
Method 160
millimeters 303, 368
minor 135
miter 365
modal timeout 397
model 316
Motif compliance 415
MoveToplev 388
mutex 262

N

Name 389
name 285, 367, 370, 400
NameOfImg 390
Namespace 167
namespace 54, 64
nesting depth 227
nonblocking 50, 52, 196
nonrecursive 165
Notifier 177
NRE 165

O

Object 182
object 31, 160, 287
ObjectType 185
obscure 398
OOInitStubs 178
OpenFileChnl 196
OpenTcp 199
option 129, 285, 367
options 396
output 196
own 391
OwnSelect 391

P

package 211, 240
padding 430
Panic 200
parent 36, 383
parse 21, 203, 209, 372
ParseArgs 203
ParseArgv 396
ParseCmd 209
partial command 31
patchlevel 135
path 239, 289
path name 389
pattern 221, 242
permissions 131
photo 340
photo image 330
pipeline 131
pixel 306
pixel value 354
pixels 303, 368, 414
pixmap 306, 349, 369
PkgRequire 211
points 368
polygon 284
position 383, 388
Postscript 289, 341
present 211
Preserve 212
PrintDbl 213
process 72
projecting 350
provide 211
pwd 127

Q

QWinEvent 397

R

raised 284
rc file 235
read 131, 196
read-only 152
real 152
realloc 13
RecEvalObj 214
record 214

RecordEval 214
recursion 227
redisplay 291, 364, 379
reference count 212
reference counting 182
RegConfig 216
RegExp 221
region 430
register 400
regular expression 221
release 135
relief 303, 370, 414, 430
replace 155
request 344, 385
requested size 426
require 211
resource 149
resource identifier 342, 369
Restack 398
RestrictEv 399
restriction 399
result 70, 96, 146, 165, 223, 230
retrieve 367
return value 230
root window 306, 371
round 350, 365

S

safe 422
SaveResult 223
scalar 233
screen 334, 376, 426
screen distance 414
screen units 368
script 96, 165, 287
scrollbar 372
scrolling command 372
search 140
seek 196
selection 291, 293, 295, 326, 332
selection anchor 293
selection owner 391
selection retrieval 373
send command 400
server 199
service mode 177
set 233

SetAppName 400
SetCaret 400
SetChanErr 225
SetClass 401
SetClassProcs 403
SetErrno 226
SetGrid 404
SetOptions 414
SetRecLmt 227
SetResult 230
SetVar 233
SetVisual 415
shadow 284
shared library 156
Signal 234
signal 19
signal numbers 234
signals 234
Sleep 234
socket 199
SourceRCFile 235
split 238, 239
SplitList 238
SplitPath 239
stack 12
stacking order 398
stackless 165
standard channel 132
standard channels 242, 308
standard error 132
standard input 132
standard output 132
stat 8, 126
state 223
static linking 240
StaticPkg 240
StdChannels 242
sticky 430
stipple 289
storage 212
StrictMotif 415
string 99, 101, 152, 213, 221, 242, 366, 370
string representation 79, 144, 155, 182, 185, 248
string type 248
string value 248
StringObj 248

strings 32, 238
StrMatch 242
stubs 141, 178, 421
subexpression 221
SubstObj 249
synonym 303, 414

T

table lookup 129
target 332
Tcl_Main 258
TCL_MEM_DEBUG 13, 250
Tcl_RegExpIndices 221
Tcl_RegExpInfo 221
Tcl_SaveInterpState 19
TclZlib 255
TCP 199
text 293
TextLayout 420
Thread 262
thread 19, 98
thread local storage 262
threads 177
three-dimensional effect 284
tilde 271
time 134, 149, 234
timer 65, 77, 177
Tk_Init 422
Tk_Main 423
tk_strictMotif variable 415
TkInitStubs 421
token 209, 389
tolower 263
top-level 426
top-level window 334, 388
totitle 263
ToUpper 263
trace 12, 68, 152, 265, 270
TraceCmd 265
TraceVar 270
Translate 271
traversal highlight 335
ttk_Geometry 430
ttk_Theme 430
type 239, 294
type conversion 182, 185
type manager 291, 326

U

uid 303
UniCharIsAlpha 273
unicode 23, 248, 263, 273, 278
unique identifier 374, 401
unloading 98
unmanaged 385
unmap 383, 385
unset 233
unwind 25
UpVar 274
user 271
Utf 278
utf 23, 89, 263

V

value 12, 22, 23, 64, 75, 79, 96, 99, 101, 129, 140, 144, 155, 165, 182, 185, 214, 230, 233, 248, 284, 354
value creation 182
value result 12
value type 79, 144, 155, 182, 185, 248
variable 12, 93, 152, 233, 270, 274
variable substitution 209, 249
version 135, 211
vfs 126
virtual filesystem 126
virtual root 374
visual 351, 376, 415, 426

W

wait 72, 234
width 306, 374, 426
window 306, 334, 362, 377, 378, 383, 385, 389, 401, 404, 426
window manager 374, 388, 401, 404
WindowId 426
Windows window id 377
write 131, 196
wrong number of arguments 279
WrongNumArgs 279

X

x 306, 426
X window id 378
xview 372

Y

y 306, 426
yview 372